

O'REILLY®

8-е издание

Изучаем vi и Vim

не просто редакторы



Арнольд Роббинс
Элберт Ханна

EIGHTH EDITION

Learning the vi and Vim Editors

Power and Agility Beyond Just Text Editing

Arnold Robbins and Elbert Hannah

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Изучаем vi и Vim

не просто редакторы

8-е издание

Арнольд Роббинс, Элберт Ханна



Санкт-Петербург • Москва • Минск

2024

Выпущено при поддержке:

КРОК

Арнольд Роббинс, Элберт Ханна
Изучаем vi и Vim. Не просто редакторы
8-е издание

Перевела с английского Надежда Яровая
Научный редактор Тимур Напреев

ББК 32.973.2-018
УДК 004.912

Роббинс Арнольд, Ханна Элберт

P58 Изучаем vi и Vim. Не просто редакторы. 8-е изд. — СПб.: Питер, 2023. — 528 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-2019-2

Среди текстовых редакторов, используемых программистами, самыми важными является vi и его вариации. В обновленном издании пользователи Unix и Linux изучат основы редактирования текста как в vi, так и в Vim (vi improved), прежде чем перейти к более продвинутым инструментам в каждой из программ. Авторы Арнольд Роббинс и Элберт Ханна описывают основные новейшие версии Vim.

Если вы программист или компьютерный аналитик либо работаете с веб- или консольными интерфейсами, Vim упростит решение сложных задач. Вы освоите многооконное редактирование, глобальный поиск/замену и прочие мощные инструменты для программистов, а также научитесь писать интерактивные макросы и сценарии, расширяющие возможности программы.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ISBN 978-1492078807 англ.

Authorized Russian translation of the English edition of Learning the vi and Vim Editors, 8th Edition ISBN 9781492078807 © 2022 Elbert Hannah and Arnold Robbins.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

ISBN 978-5-4461-2019-2

© Перевод на русский язык ООО «Прогресс книга», 2023

© Издание на русском языке, оформление ООО «Прогресс книга», 2023

© Серия «Библиотека программиста», 2023

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

Изготовлено в России. Изготовитель: ООО «Прогресс книга».

Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,

Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 08.2023. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12.000 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 22.06.23. Формат 70×100/16. Бумага офсетная. Усл. п. л. 42,570. Тираж 700. Заказ 0000.

Краткое содержание

Предисловие	18
-------------------	----

ЧАСТЬ I. ОСНОВНЫЕ ПРИЕМЫ РАБОТЫ В VI И VIM

Глава 1. Знакомство с vi и Vim	30
Глава 2. Основы редактирования	43
Глава 3. Эффективная навигация	70
Глава 4. За пределами основ	84
Глава 5. Знакомство с редактором ex	94
Глава 6. Глобальная замена	114
Глава 7. Продвинутое редактирование	143

ЧАСТЬ II. VIM

Глава 8. Vim (улучшенный vi): обзор и отличия от vi	190
Глава 9. Графический Vim (gvim)	217
Глава 10. Многооконный режим в Vim	246
Глава 11. Расширенные возможности Vim для программистов	271
Глава 12. Сценарии Vim	323
Глава 13. Прочие полезные возможности Vim	349
Глава 14. Несколько продвинутых приемов работы с Vim	374

ЧАСТЬ III. VIM В БОЛЕЕ ШИРОКОЙ СРЕДЕ

Глава 15. Vim как IDE: требуется сборка	390
Глава 16. vi повсюду	408
Глава 17. Эпилог	444

ПРИЛОЖЕНИЯ

Приложение А. Редакторы vi, ex и Vim	446
Приложение Б. Установка параметров	491
Приложение В. Более светлая сторона vi	504
Приложение Г. vi и Vim: исходный код и разработка	516
Об авторах	524
Иллюстрация на обложке	526

Оглавление

Предисловие	18
Структура книги	18
Способ подачи материала	20
Описание команд vi	20
Условные обозначения	21
Команды и клавиши	22
Список проблем	23
Что нужно знать перед началом работы	23
Использование примеров кода	23
О предыдущих изданиях	24
Восьмое издание	25
Что нового	25
Версии	25
Благодарности из шестого издания	26
Благодарности из седьмого издания	26
Благодарности в восьмом издании	27
От издательства	28

ЧАСТЬ I. ОСНОВНЫЕ ПРИЕМЫ РАБОТЫ В VI И VIM

Глава 1. Знакомство с vi и Vim	30
Текстовые редакторы и редактирование текста	30
Текстовые редакторы	30
Редактирование текста	33
Небольшая историческая справка	34
Открытие и закрытие файлов	36
Открытие файла из командной строки	36
Открытие файла из графического интерфейса	37
Проблемы при открытии файлов	38
Modus Operandi	39
Сохранение и закрытие файла	39
Закрытие файла без сохранения изменений	40
Проблемы при сохранении файлов	41
Упражнения	42

Глава 2. Основы редактирования	43
Команды vi	44
Перемещение курсора в командном режиме.....	45
Единичное перемещение	46
Числовые аргументы	48
Перемещение по строке	49
Перемещение по фрагментам текста	50
Базовые операции	51
Ввод текста.....	52
Добавление текста	53
Замена текста.....	53
Изменение регистра	56
Удаление текста	56
Перемещение текста.....	60
Копирование текста	61
Повторение или отмена последней команды.....	63
Другие способы вставки текста	64
Числовые аргументы для команд ввода	65
Объединение двух строк с помощью команды J	66
Проблемы с командами в vi	66
Индикаторы режимов	67
Перечень основных команд vi	67
Глава 3. Эффективная навигация	70
Перемещение по экранам.....	70
Прокрутка экрана.....	71
Изменение положения экрана с помощью команды z	72
Перерисовка экрана	72
Перемещение по видимой части экрана.....	73
Перемещение по строкам	74
Перемещение по текстовым фрагментам.....	74
Перемещение путем поиска.....	75
Повторный поиск	77
Поиск в текущей строке.....	78
Перемещение по номерам строк.....	80
Команда G (перейти к).....	80
Перечень команд перемещения в vi	81

Глава 4. За пределами основ.....	84
Дополнительные комбинации команд.....	84
Параметры загрузки vi и Vim.....	85
Перемещение в определенное место	86
Режим чтения	87
Восстановление содержимого из буфера	88
Использование регистров	89
Восстановление удаленного текста	89
Работа с именованными регистрами	90
Маркеры	92
Другие расширенные возможности редактирования	92
Перечень регистров и маркировки.....	93
Глава 5. Знакомство с редактором ex	94
Команды ex.....	95
Упражнение: редактор ex	97
Проблема при переходе в визуальный режим.....	98
Редактирование с ex	98
Адреса строк.....	98
Определение диапазона строк.....	99
Символы адресов строк	100
Шаблоны поиска	101
Переопределение местоположения текущей строки.....	102
Глобальный поиск	103
Сочетание команд ex	103
Сохранение файлов и завершение работы.....	104
Переименование буфера.....	105
Сохранение части файла.....	106
Добавление в сохраненный файл.....	106
Копирование файла в другой файл.....	106
Редактирование нескольких файлов одновременно.....	107
Вызов Vim для нескольких файлов.....	107
Использование списка аргументов	108
Вызов нового файла	109
Сочетание клавиш для имени файла.....	109
Переключение файлов в командном режиме.....	110
Правки между файлами.....	110
Краткое описание команд ex	111

Глава 6. Глобальная замена	114
Команда замены	114
Подтверждение замены	115
Глобальные действия в файле	117
Контекстно зависимая замена.....	117
Правила сопоставления с шаблоном.....	118
Метасимволы в шаблонах поиска.....	119
Выражения в квадратных скобках POSIX.....	122
Метасимволы в строках замены.....	124
Дополнительные приемы замены	126
Примеры сопоставления с шаблоном	127
Поиск общего класса слов.....	128
Перемещение фрагментов с помощью шаблонов.....	129
Еще примеры	130
Заключительные примеры сопоставления с шаблоном.....	136
Удаление неизвестного фрагмента текста	137
Переключение элементов в текстовой базе данных	138
Использование :g для повтора команды.....	140
Сбор строк	141
Глава 7. Продвинутое редактирование	143
Настройка vi и Vim	144
Команда :set.....	144
Файл .exrc.....	146
Переменные окружения	147
Некоторые полезные параметры	148
Выполнение команд Unix.....	149
Фильтрация текста с помощью команды.....	151
Сохранение команд.....	153
Сокращение слова	154
Команда map.....	155
Переназначение с помощью выноски	156
Защита клавиш от интерпретации ex	157
Пример сложного переназначения	158
Дополнительные примеры переназначения клавиш	160
Переназначение клавиш для режима ввода текста	162
Переназначение функциональных клавиш программируемой клавиатуры	163

Переназначение других специальных клавиш.....	165
Переназначение нескольких клавиш ввода.....	167
@-функции.....	168
Запуск регистров из ex.....	169
Использование сценариев ex.....	169
Зацикливание сценария оболочки	171
Встроенные документы	173
Сортировка фрагментов текста: пример сценария ex	174
Комментарии в сценариях ex.....	176
За пределами ex	176
Редактирование исходного кода программы.....	177
Управление отступом	177
Специальная команда поиска	180
Использование тегов	181
Расширенные теги.....	182

ЧАСТЬ II. VIM

Глава 8. Vim (улучшенный vi): обзор и отличия от vi.....	190
Немного о Vim	191
Обзор	192
Автор и история	192
Почему Vim?	193
Отличие от vi	194
Категории функций.....	194
Философия.....	198
Вспомогательные средства и простые режимы для новых пользователей	198
Встроенная справка.....	199
Параметры запуска и инициализации.....	201
Параметры командной строки.....	201
Поведения, связанные с именем команды.....	203
Системные и пользовательские конфигурационные файлы	204
Переменные окружения	206
Новые команды перемещения.....	208
Перемещение в визуальном режиме	209
Расширенные регулярные выражения	210
Расширенный откат изменений.....	213
Инкрементный поиск	215
Прокрутка слева направо.....	215
Резюме.....	215

Глава 9. Графический Vim (gvim)	217
Знакомство с gvim	218
Запуск gvim.....	218
Использование мыши.....	220
Полезные пункты меню	222
Настройка полос прокрутки, меню и панелей инструментов.....	224
Полосы прокрутки	224
Меню.....	225
Панели инструментов.....	232
Всплывающие подсказки	234
gvim в Microsoft Windows.....	235
gvim в X Window System	236
Запуск gvim в Microsoft Windows WSL.....	236
Установка gvim в WSL2	237
Установка сервера X для Windows.....	238
Настройка сервера X для Windows	239
Параметры GUI и краткое описание команд	244
Глава 10. Многооконный режим в Vim	246
Инициация многооконного редактирования	248
Инициация многооконности из командной строки.....	248
Многооконное редактирование внутри Vim.....	250
Открытие окон	251
Новые окна	251
Параметры во время разделения	251
Команды условного разделения	253
Краткое описание команд для окон	253
Перемещение по окнам (перемещение вашего курсора от окна к окну)	254
Перемещение окон	256
Перемещение окон (поворот или перестановка)	256
Перемещение окон и их перекомпоновка	256
Команды перемещения окон: краткое описание	257
Изменение размеров окон	258
Команды изменения размера окна	258
Параметры изменения размера окна	260
Краткое описание команд изменения размера окон	260
Буферы и их взаимодействие с окнами	262
Специальные буферы Vim	263
Скрытые буферы.....	263

Команды буфера	264
Краткое описание команд буфера	265
Управление тегами с помощью окон	266
Редактирование с вкладками	267
Заккрытие и выход из окон	269
Резюме	270
Глава 11. Расширенные возможности Vim для программистов	271
Свертывание и создание структуры (режим структуры)	272
Команды свертывания	274
Ручное свертывание	275
Создание структуры	282
Несколько слов о других методах свертывания	283
Автоматические и умные отступы	285
Расширения autoindent Vim для autoindent vi	286
Параметр smartindent	287
Параметр indentexpr	293
Заключительное слово об отступах	293
Завершение по ключевым словам и словарю	294
Команды завершения вставки	295
Заключительные комментарии по поводу автозавершения Vim	303
Стеки тегов	303
Подсветка синтаксиса	306
Первоначальный запуск	307
Настройка	308
Создание своего собственного файла синтаксиса	313
Компиляция и проверка ошибок в Vim	316
Дополнительные варианты использования окна Quickfix List	320
Пара заключительных слов о роли Vim для написания программ	322
Глава 12. Сценарии Vim	323
Какой ваш любимый цвет (цветовая схема)?	323
Условное выполнение	324
Переменные	326
Команда execute	327
Определение функций	329
Хитрый трюк	330
Настройка сценариев Vim с помощью глобальных переменных	331
Массивы	333

Динамическая конфигурация типа файла с помощью сценария	334
Автокоманды	334
Параметры проверки	336
Переменные буфера	337
Функция exists()	338
Автокоманды и группы	340
Удаление автокоманд	341
Некоторые соображения по поводу сценариев Vim	343
Полезный пример сценария Vim.....	343
Подробнее о переменных	344
Выражения.....	345
Расширения.....	345
Еще несколько слов о autocmd.....	345
Внутренние функции	346
Ресурсы	347
Глава 13. Прочие полезные возможности Vim	349
Напишите это по буквам (э-т-о)	349
Тезаурус.....	352
Редактирование двоичных файлов.....	353
Диграфы: символы, отличные от ASCII.....	355
Редактирование файлов в других местах	357
Навигация по каталогам и их изменение	359
Резервные копии в Vim	361
Преобразование текста в HTML	362
В чем же разница?	363
viminfo: итак, где же я остановился?.....	365
Параметр viminfo	365
Команда mksession	366
Какова длина моей строки?	368
Сокращенные версии команд и параметров Vim.....	370
Несколько простых хитростей (не обязательно специфичных для Vim).....	372
Другие ресурсы.....	373
Глава 14. Несколько продвинутых приемов работы с Vim	374
Несколько удобных переназначений.....	374
Упрощенный выход из Vim	374
Изменение размера вашего окна	375
Удвойте удовольствие	375

Переходим к более интересному	378
Поиск сложно запоминаемой команды	378
Анализ известной речи	380
Еще несколько случаев использования.....	384
Увеличиваем скорость	386
Улучшаем строку состояния	387
Резюме	388

ЧАСТЬ III. VIM В БОЛЕЕ ШИРОКОЙ СРЕДЕ

Глава 15. Vim как IDE: требуется сборка	390
Менеджеры плагинов	390
Поиск подходящего плагина	392
Зачем нам нужна IDE?	393
Самостоятельная работа.....	394
EditorConfig: последовательная настройка редактирования текста.....	394
NERDTree: обход дерева файлов внутри Vim.....	395
nerdtree-git-plugin: NERDTree с индикаторами состояния Git.....	395
Fugitive: запуск Git из Vim	396
Завершение	398
Termdebug: прямое использование GDB внутри Vim	402
Универсальные IDE	403
Кодировать — это здорово, но если я писатель?	406
Заключение	407
Глава 16. vi повсюду	408
Введение	408
Различные способы улучшения и оптимизации работы с командной строкой.....	408
Совместное использование нескольких оболочек	409
Библиотека readline.....	410
Оболочка Bash	410
Другие программы	413
Файл .inputrc	413
Другие оболочки Unix	415
Оболочка Z (zsh)	415
Сохраняйте как можно больше истории	416
Редактирование командной строки: некоторые заключительные мысли	417
Windows PowerShell.....	417

Инструменты разработчика	418
Драйвер Clewn GDB	418
CGDB: обязательные операции GDB	419
Vim внутри Visual Studio	420
Vim для Visual Studio Code	421
Утилиты Unix	425
Больше или меньше?	425
screen	427
И наконец, браузеры!	432
Wasavi	432
Vim + Chromium = Vimium	434
vi для MS Word и Outlook	439
Достойны упоминания: инструменты с некоторыми функциями vi	442
Google Mail.....	442
Microsoft PowerToys	442
Резюме	443
Глава 17. Эпилог.....	444

ПРИЛОЖЕНИЯ

Приложение А. Редакторы vi, ex и Vim	446
Синтаксис командной строки	446
Параметры командной строки	447
Обзор операций vi	450
Командный режим	450
Режим ввода.....	450
Синтаксис команд vi	450
Команды строки состояния	452
Команды vi	452
Команды перемещения	452
Команды вставки	456
Команды редактирования.....	458
Сохранение и выход	460
Доступ к нескольким файлам.....	461
Команды окон (Vim)	461
Взаимодействие с системой.....	463
Макросы	463
Прочие команды	464

Конфигурация vi	465
Команда :set	465
Пример файла .exrc	466
Основы ex	466
Синтаксис команд ex	466
Адреса	467
Символы адресов	467
Параметры	467
Алфавитный указатель команд ex	468
Приложение Б. Установка параметров	491
Параметры Heirloom и Solaris vi.....	491
Параметры Vim 8.2	495
Приложение В. Более светлая сторона vi	504
Получение доступа к файлам.....	504
Примеры файлов	505
Источник clewn	505
Онлайн-руководство vi.....	505
vi Powered!	506
vi для любителей Java	507
Педаль Vim.....	507
Поразите своих друзей!.....	508
Домашняя страница любителей Vi	510
Другой клон vi.....	511
Наслаждение чистым вкусом.....	514
Цитаты о vi	515
Приложение Г. vi и Vim: исходный код и разработка	516
Ничего схожего с оригиналом.....	516
Где взять Vim	517
Установка Vim для Unix и GNU/Linux.....	519
Установка Vim для окружений Windows.....	520
Установка Vim для окружения Macintosh	522
Другие операционные системы.....	523
Об авторах	524
Иллюстрация на обложке	526

Моей жене Мириам. Спасибо за твою любовь, терпение и поддержку.

Арнольд Роббинс, шестое, седьмое и восьмое издания

Моей жене Анне. Благодарю за любовь, моральную поддержку и вдохновение.

Спасибо, что ты всегда рядом.

Элберт Ханна, седьмое и восьмое издания

Предисловие

Редактирование текста — одна из наиболее распространенных задач в любой компьютерной системе, а `vi` входит в число самых полезных стандартных текстовых редакторов. С помощью `vi` вы сможете создавать и редактировать текстовые файлы.

Редактор `vi`, как и многие классические утилиты, разработанные на заре Unix®, может показаться трудным в освоении. Vim (`vi Improved`) — улучшенная версия программы, разработанная Брэмом Моленаром, разрушает этот стереотип. Vim включает в себя множество удобных функций, наглядных руководств и справочников.

Сегодня Vim — самая популярная версия редактора `vi`, поэтому в восьмом издании мы сфокусируемся именно на ней.

- Часть I помогает освоить базовые навыки `vi`, применимые ко всем версиям редактора `vi` в контексте Vim.
- Часть II посвящена расширенным функциям Vim.
- Часть III позволяет взглянуть на Vim в более широкой среде.
- Часть IV содержит приложения.

Структура книги

Книга состоит из 17 глав и 4 приложений. Часть I предназначена для быстрого освоения первичных приемов работы с `vi` и Vim, а также для развития навыков, которые позволят вам более эффективно использовать эти редакторы.

В первых двух главах представлены некоторые простые команды редактирования, доступные в `vi` и Vim. Очень важно довести выполнение данных команд до автоматизма, поэтому практикуйте их как можно чаще. В главе 2 продемонстрированы элементарные операции редактирования. В целом для быстрого ознакомления с `vi` и Vim достаточно этих двух глав.

Однако описываемые программы предназначены для гораздо большего, чем элементарная обработка текста. Разнообразие команд и возможностей позволяет вам сократить количество рутинной работы по редактированию текста. Главы 3 и 4 сосредоточены на более простых способах выполнения тех же задач. Во время первого прочтения вы получите представление о том, что могут `vi` и `Vim` и какие команды лучше использовать под ваши конкретные нужды. Позже вы сможете вернуться к этим главам для более глубокого изучения.

В главах 5–7 представлены инструменты, способные переложить большую часть редактирования на сами программы. Вы познакомитесь со строковым редактором `ex`, лежащим в основе `vi` и `Vim`, и узнаете, как выполнять `ex` команды из `vi` и `Vim`.

В части II описывается редактор `Vim` — самый популярный клон `vi` в XXI веке. В ней подробно рассказывается о множестве функций `Vim` и его отличиях от оригинального `vi`.

Глава 8 содержит общее введение в `Vim`. Здесь приведен обзор основных улучшений `Vim` по сравнению с `vi`, таких как встроенная справка, расширенные возможности по настройке инициализации, дополнительные команды перемещения, расширенные регулярные выражения и многое другое.

В главе 9 рассматривается `Vim` в современных средах с графическим интерфейсом, ставших сегодня стандартами для коммерческих систем GNU/Linux и других Unix-подобных ОС, а также Windows.

В главе 10 вы познакомитесь с многооконным режимом, который является, пожалуй, наиболее важным дополнением, которого нет в стандартной программе `vi`. В этой главе подробно рассказывается о создании и использовании нескольких окон.

Глава 11 посвящена использованию `Vim` в качестве редактора программ, помимо его функционала по обработке текста. Особую ценность представляют средства сворачивания и выделения, смарт-отступы, подсветка синтаксиса и ускорение цикла редактирования-компиляции-отладки.

В главе 12 рассматривается командный язык `Vim`, на котором создаются сценарии (скрипты) для настройки и адаптации `Vim` в соответствии с вашими потребностями. В целом простота работы с `Vim` обусловлена большим количеством сценариев, написанных другими пользователями, внесшими свой вклад в дистрибутив `Vim`.

Глава 13 охватывает ряд интересных моментов, которые не вписываются в предыдущие главы.

В главе 14 разобраны некоторые полезные «мощные приемы». Основанная на идее персональных настроек назначения клавиш, данная глава поможет узнать больше о том, как быть продуктивным.

Часть III касается ролей `vi` и Vim в более широком мире разработки программного обеспечения (ПО) и работы с компьютерами.

Глава 15 затрагивает основы работы с плагинами в Vim, фокусируясь на том, как превратить Vim из «простого» редактора в полноценную интегрированную среду разработки (IDE).

В главе 16 рассматриваются другие важные программные среды, в которых редактирование в стиле `vi` повлияет на рост производительности в целом.

В главе 17 подводятся краткие итоги всей книги.

Часть IV содержит полезные справочные материалы.

В приложении А перечислены все стандартные команды `vi` и `ex`, отсортированные по предназначению. Представлен алфавитный список команд `ex`. Также представлены избранные команды `vi` и `ex` из Vim.

В приложении Б перечислены параметры команды `set` в программах `vi` и Vim.

Приложение В содержит юмористические материалы, связанные с `vi`.

Приложение Г описывает, как установить программы `vi` и Vim в системах Unix, GNU/Linux, Windows и macOS.

Способ подачи материала

Наша цель — помочь новым пользователям `vi` и Vim без особого труда освоить эти программы. Изучение нового редактора, особенно такого мощного, как Vim, с виду кажется непосильной задачей. Мы представили основные концепции и команды в понятной и логичной форме.

После рассмотрения основ `vi` и Vim мы погружаемся в более подробное изучение Vim. Далее описаны условные обозначения, используемые в книге.

Описание команд `vi`

Каждую клавишу или сочетание клавиш, а также группу связанных команд мы сопровождаем кратким описанием, прежде чем ориентироваться на конкретные задачи. Затем мы приводим соответствующие команды для каждого конкретного случая, а также их описание и корректный синтаксис.

Условные обозначения

Синтаксис и примеры выделены таким шрифтом, а все имена файлов, команд и параметры программы — таким моноширинным шрифтом. Переменные (которые заменяем конкретным для той или иной ситуации значением) оформлены *курсивным моноширинным шрифтом*. В скобках они указаны как необязательные. Например, в строке ниже параметр *имя_файла* нужно заменить фактическим именем файла:

```
vi [имя_файла]
```

Квадратные скобки говорят о том, что команду `vi` можно вызвать без указания имени файла. Скобки вводить не нужно. В некоторых примерах показан результат выполнения команд, введенных в командной строке. В таких листингах то, что вам нужно ввести, отформатировано **полужирным шрифтом**, чтобы вы не спутали его с ответом системы.

Например:

```
$ ls  
ch01.xml ch02.xml ch03.xml ch04.xml
```

В примерах кода *обыкновенным курсивом* обозначен комментарий, который вводить не нужно, а в тексте — специальные термины и все, на что следует обратить внимание.

Следуя традиционному соглашению о документации Unix, ссылки вида `printf(3)` относятся к онлайн-руководству (доступ осуществляется с помощью команды `man`). Этот пример относится к записи о функции `printf()` в разделе 3 руководства. Введите команду `man -s 3 printf` (в большинстве систем эта команда сработает), чтобы открыть руководство.



Здесь приводится информация, на которую нужно обратить пристальное внимание.



Указывает на неочевидные вещи, которые могут представлять интерес.



Здесь описываются полезные сочетания клавиш или действия, экономящие время.

Команды и клавиши

Особые клавиши или их сочетания обозначаются шрифтом на сером фоне: «Нажмите клавишу **Esc**, чтобы перевести редактор в командный режим».

В книге вам будут встречаться таблицы с командами **vi/Vim** и результатами их действия.

Команда	Результат
ZZ	"practice" [New] 6L, 104C written Команда записи и сохранения, ZZ . Ваш файл сохранится как обычный файл на диске

В предыдущем примере команда **ZZ** приведена в левом столбце. В правом находится строка (или несколько строк) экрана, которая показывает результат выполнения команды. Положение курсора в таких примерах обозначено прямоугольником. В этом случае, поскольку команда **ZZ** сохраняет и записывает файл, вы видите строку состояния, отображаемую при записи файла; положение курсора не отображается. Под командой (результатом) приводится объяснение ее и того, что она делает.

В некоторых таблицах отражены команды оболочки и результат их выполнения. В таких случаях командам предшествует стандартное приглашение **\$**, а сама команда выделена жирным шрифтом.

Команда	Результат
\$ ls	ch01.asciidoc ch02.asciidoc ch03.asciidoc

Иногда команды **vi** можно вызвать, одновременно нажав **Ctrl** и дополнительную клавишу (например, **Ctrl+G**). В примерах кода перед именем ключа ставится символ **^**. То есть **^G** означает, что нужно удерживать нажатой клавишу **Ctrl** при нажатии клавиши **G**. Принято обозначать управляющие символы прописными буквами (**^G**, а не **^g**), даже если при их наборе вы *не* удерживаете нажатой клавишу **Shift**¹.

Кроме того, заглавные буквы отображаются с использованием обозначения сочетаний клавиш, например **Shift+X** для некоего символа **X**. Таким образом, а представляется как клавиша **A**, а буква **A** представляется как **Shift+A**.

¹ Возможно, это связано с тем, что на самих клавишах нанесены прописные буквы, а не строчные.

Список проблем

Список проблем приведен в тех разделах, где это наиболее актуально. Вы можете быстро просмотреть эти списки и вернуться к ним, если возникнут вопросы.

Что нужно знать перед началом работы

Материал книги подходит тем, кто обладает базовыми знаниями на уровне пользователя Unix. В частности, вы уже должны уметь:

- открывать окно терминала, чтобы перейти к командной строке;
- входить в систему и выходить из нее, обычно через `ssh`, если используется удаленная система;
- вводить команды в командную строку;
- изменять каталоги;
- выводить содержимое папок;
- создавать, копировать и удалять файлы.

Полезно также знать команду `grep` (программа глобального поиска) и уметь работать с регулярными выражениями.

Хотя современные системы позволяют запускать Vim из меню с графическим интерфейсом, вы теряете доступ к гибкости, которую обеспечивают опции командной строки Vim. На протяжении всей книги в наших примерах запуск `vi` и Vim происходит именно из командной строки.

Использование примеров кода

Дополнительные материалы (примеры кода, упражнения и т. д.) можно загрузить по адресу www.github.com/learning-vi/vi-files.

В общем случае все примеры кода из книги вы можете использовать в своих программах и в документации. Вам не нужно обращаться в издательство за разрешением, если вы не собираетесь воспроизводить существенные части программного кода. Если вы разрабатываете программу и используете в ней несколько фрагментов кода из книги, вам не нужно обращаться за разрешением. Но для продажи или распространения примеров из книги вам потребуется разрешение от издательства O'Reilly. Вы можете отвечать на вопросы, цитируя данную книгу или примеры из

нее, но для включения существенных объемов программного кода из книги в документацию вашего продукта потребуется разрешение.

Мы рекомендуем, но не требуем добавлять ссылку на первоисточник при цитировании. Под ссылкой на первоисточник мы подразумеваем указание авторов, издательства и ISBN.

За получением разрешения на использование значительных объемов программного кода из книги обращайтесь по адресу permissions@oreilly.com.

0 предыдущих изданиях

В пятом издании впервые были подробно рассмотрены команды редактора `ex`. В главах 5, 6 и 7 сложные функции `ex` и `vi` были разъяснены путем добавления дополнительных примеров, охватывающих такие темы, как синтаксис регулярных выражений, глобальная замена, файлы `.exrc`, аббревиатуры, сочетания клавиш и сценарии (скрипты). Несколько примеров были взяты из статей журнала *UnixWorld*. Уолтер Зинц написал руководство¹ по `vi`, состоящее из двух частей, которое учит новым функциям, а также содержит множество интересных примеров, иллюстрирующих ранее рассмотренные возможности. Также Рэй Шварц в одной из своих статей давал немало полезных советов, за которые мы ему благодарны².

В шестом издании рассмотрено четыре свободно доступных «клона» редактора `vi`. Многие из них функциональнее `vi`. Таким образом, существует семейство редакторов `vi`, и цель книги состояла в том, чтобы научить читателя основам использования каждого. В том издании на одинаковом уровне рассматривались `nvi`, `Vim`, `elvis` и `vile`. В приложении описывалась роль `vi` в культуре Unix и Интернета.

Седьмое издание взяло все лучшее из шестого. Время показало, что `Vim` остается самым популярным аналогом `vi`, поэтому седьмое издание значительно расширило охват функций этого редактора (также мы добавили слово `Vim` в название книги). Однако, чтобы сохранить актуальность материала для как можно большего числа пользователей, были сохранены и обновлены сведения о программах `nvi`, `elvis` и `vile`.

¹ Статьи `vi Tips for Power Users` и `Using vi to Automate Complex Edits` написаны Уолтером Зинцем для журнала *UnixWorld* в апреле и мае 1990 года соответственно.

² *Swartz R. Answers to Unix // UnixWorld, 1990. — August.*

Восьмое издание

В восьмом издании мы сохранили всю полезную информацию из седьмого. В наши дни Vim правит бал, поэтому мы расширили информацию о нем и удалили разделы, посвященные программам `nvi`, `elvis` и `vile`. В части I Vim теперь играет роль программы для выполнения инструкций и примеров. Кроме того, была удалена информация о проблемах со старыми версиями оригинальной программы `vi`, которые уже неактуальны. Мы попытались упорядочить материал и сделать книгу как можно более актуальной и полезной.

Что нового

Перечислим, что мы изменили в этом издании.

- Исправили ошибки в основном тексте.
- Тщательно пересмотрели и обновили материал в частях I и II.
- В части I перенесли акцент с оригинальной версии `vi` для Unix на `vi` в контексте Vim.
- Добавили оригинальную главу в часть II, а также дополнили часть III совершенно новыми главами.
- Переработали приложение В.
- Перенесли материал о загрузке и установке Vim из основного текста в приложение Г.
- Другие приложения также были обновлены.

Версии

Для тестирования различных функций `vi` использовались следующие программы:

- `vi` Heirloom со страницы <https://github.com/n-t-roff/heirloom-ex-vi>. Эта версия служит эталонной для оригинальной Unix-программы `vi`;
- Solaris 11 `/usr/xpg7/bin/vi` (в Solaris 11 по адресу `/usr/bin/vi` на самом деле расположена программа Vim! Версии `vi` в `/usr/xpg4/bin`, `/usr/xpg6/bin` и `/usr/xpg7/bin`, по-видимому, являются производными от оригинальной Unix-программы `vi`);
- версии Vim 8.0, 8.1 и 8.2 Брэма Моленара.

Благодарности из шестого издания

Прежде всего, спасибо моей жене Мириам за заботу о детях, пока я работал над книгой, особенно в «ведьмин час» перед едой. Спасибо ей за тишину в доме и мороженое.

Пол Манно из Технологического колледжа компьютерных наук Джорджии оказал неоценимую помощь в настройке печатного ПО. Сотрудники издательства O'Reilly & Associates Лен Мюллнер и Эрик Рэй помогли с программным обеспечением SGML, а макросы vi Джерри Пика для данного ПО оказались просто бесценны.

Хотя при подготовке нового и переработанного материала использовались различные программы, большая часть работы была выполнена с помощью Vim версий 4.5 и 5.0 под управлением GNU/Linux (Red Hat 4.2).

Спасибо Киту Бостику, Стиву Киркендаллу, Брэму Моленару, Полу Фоксу, Тому Дики и Кевину Бюттнеру, которые рецензировали книгу. Стив Киркендалл, Брэм Моленар, Пол Фокс, Том Дики и Кевин Бюттнер также предоставили важный материал для глав 8–12 (эти номера глав относятся к шестому изданию).

Без электричества, поставляемого энергетической компанией, невозможно пользоваться компьютером. Но, когда электричество есть, вы воспринимаете это как должное. То же самое касается написания книги — без редактора сделать это невозможно, но, когда редактор хорошо справляется со своей работой, о его вкладе в процесс очень легко забыть. Джиджи Эстабрук из O'Reilly — настоящая жемчужина. Мне было приятно работать с ней, и я ценю все, что она сделала и продолжает делать для меня. Наконец, большое спасибо технической группе O'Reilly & Associates.

*Арнольд Роббинс,
Раанана, Израиль.
Июнь 1998 года*

Благодарности из седьмого издания

И вновь Арнольд благодарит свою жену Мириам за ее любовь и поддержку. Его долг за спокойные часы работы и мороженое растет с каждым днем. Кроме того, большая признательность Джей Ди «Иллиада» Фрейзеру за отличные комиксы User friendly¹.

¹ Посетите страницу www.userfriendly.org, если никогда не слышали о комиксах User friendly.

Элберт хотел бы поблагодарить Анну, Калли, Бобби и своих родителей за то, что они не переставали восхищаться его работой в трудные времена, и за их заразительный энтузиазм.

Спасибо Киту Бостику и Стиву Киркендаллу за редактуру глав. Том Дики внес значительный вклад в переработку главы о `vile` и таблицы параметров команды `set` в приложении Б. Брэм Моленаар (разработчик Vim) также пересмотрел нашу книгу. Роберт П. Джей Дэй, Мэтт Фрай, Джудит Майерсон и Стивен Фиггинс оставили важные комментарии по всему тексту.

Арнольд и Элберт хотели бы поблагодарить Энди Орама и Изабель Кункл за редактуру текста, а также всех сотрудников издательства O'Reilly Media.

*Арнольд Роббинс,
Ноф-Аялон, Израиль.
Апрель 2008 года*

*Элберт Ханна,
Килдир, Иллинойс, США.
Апрель 2008 года*

Благодарности в восьмом издании

Мы хотели бы поблагодарить Кришнана Равикумара, чье электронное письмо Арнольду с просьбой о новом издании положило начало обновлению книги.

Мы также хотели бы поблагодарить технических рецензентов — Йехезкеля Берната, Роберта Пи Джей Дэй, Уилла Гальего, Хесуса Малса, Офру Мойал-Коэн, Пола Померло и Мириам Роббинс.

Арнольд вновь благодарит свою жену Мириам за то, что она справлялась без него, пока продолжалась работа над книгой.

Он также благодарит своих детей, Хану, Ривку, Нахума и Малку, а также собаку Софи.

Элберт хотел бы поблагодарить:

- свою жену Анну, которая *вновь* терпела его жуткий график и выкрутасы, пока писалась эта книга. Он также благодарит Бобби и Калли за их поддержку и ободрение по мере продвижения работы. Их перманентное жизнерадостное отношение каждый раз поднимало настроение. И он выражает особую благодарность внуку Дину. Одним из первых слов Дина было «книга», и Элберт надеется, что Дин имеет в виду именно эту;
- своего вест-хайленд-уайт-терьера Пончо, который крутился под ногами, пока Элберт писал седьмое издание. Пончо все еще жив, здоров и с нетерпением ждет

выхода восьмого издания. Он не умеет читать, но заодно твякает при виде Vim. (Так держать, Пончо! Лапы только на клавиатуре, никакой мыши!);

- коллег из СМЕ group, с которыми на протяжении прекрасных 13 лет он оттачивал свои навыки и обучал других работе в Vim; особая благодарность:
 - Скотту Финку за его заслуги как коллеги, начальника, сотрудника и друга, который всегда пытался разузнать как можно больше не только о редакторе, а обо всей вселенной Vim. В работе со Скоттом Элберт использовал Vim как «дзен» для совместного создания отличных приложений;
 - Полу Помероло за техническую редактуру книги и за честность при сравнении Vim и Emacs. И хотя Пол фанатеет от Emacs — он один из лучших сотрудников и друзей Элберта за эти 13 лет;
 - Майклу Шакко за демонстрацию программы Microsoft Visual Studio Code. Благодаря Майклу старый пес Элберт научился множеству новых трюков. (Майкл, ты сам как IDE!);
- наконец, Тони Ферраро, под напутствием которого он работал последние дни профессиональной деятельности. Тони всегда поощрял Элберта творить (техническую документацию), а Элберт старался. (Эта книга для тебя, Тони!)

Мы оба хотели бы поблагодарить Гэри О'Брайена и Ширу Эванс за терпеливую редактуру. Говорят, что руководить программистами сложнее, чем согнать кошек в стадо; без сомнения, то же самое относится и к работе с писателями. И снова мы благодарим производственный отдел издательства O'Reilly Media.

*Арнольд Роббинс,
Ноф-Аялон, Израиль.
Сентябрь 2021 года*

*Элберт Ханна,
Килдир, Иллинойс, США.
Сентябрь 2021 года*

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Основные приемы работы в vi и Vim

В части I рассматриваются основы работы с редакторами vi и Vim. Вы узнаете о продвинутых приемах, позволяющих использовать данные программы максимально эффективно. Эти главы охватывают функциональные возможности оригинального приложения vi, а также команды, которые можно использовать в любой версии редактора. В последующих главах обсуждаются расширенные функции Vim. В данной части мы затронем следующие темы.

- Глава 1 «Знакомство с vi и Vim».
- Глава 2 «Основы редактирования».
- Глава 3 «Эффективная навигация».
- Глава 4 «За пределами основ».
- Глава 5 «Знакомство с редактором ex».
- Глава 6 «Глобальная замена».
- Глава 7 «Продвинутое редактирование».

ГЛАВА 1

Знакомство с vi и Vim

Одна из наиболее важных повседневных задач на компьютере — работа с текстом: составление нового текста, редактирование и компоновка имеющегося текста, удаление или изменение неправильного и устаревшего текста. Все это выполняется в специальных текстовых процессорах, например в Microsoft Word. Будучи программистом, вы тоже работаете с текстом: с файлами исходного кода программ и вспомогательными файлами, необходимыми для разработки. Текстовые редакторы обрабатывают содержимое любых подобных файлов, независимо от того, содержат ли они данные, исходный код или естественный текст.

Книга посвящена обработке текста с помощью двух связанных редакторов: vi и Vim. Первый традиционно используется в качестве стандартного текстового редактора Unix¹. Второй же основан на консольном режиме vi и командном языке, благодаря чему обладает бóльшим количеством полезных возможностей, чем исходная программа.

Текстовые редакторы и редактирование текста

Текстовые редакторы

С течением времени текстовые редакторы Unix улучшались и эволюционировали. Первоначально существовали *линейные редакторы*, такие как ed и ex, предназначенные для использования на последовательных терминалах, которые печатали на бумаге с непрерывной подачей (да, люди действительно программировали такие

¹ Сегодня термин Unix включает в себя как коммерческие системы, производные от оригинальной кодовой базы Unix (например, Solaris, AIX и HP-UX), так и системы Unix с открытым исходным кодом (например, GNU/Linux и производные от BSD системы). Под это понятие подпадает терминальная среда macOS, подсистема Windows для Linux (WSL) в MS-Windows, а также Cygwin и другие аналогичные среды для Windows. Если не указано иное, то описанное в книге применимо ко всем этим системам.

вещи, включая одного из авторов данной книги!). Линейные редакторы были так названы, потому что работали над программой по одной или нескольким строкам за раз.

С появлением электронно-лучевых терминалов (ЭЛТ) с адресацией курсора линейные редакторы превратились в *экранные редакторы*, такие как vi и Emacs. Экранные редакторы позволяют работать с несколькими файлами одновременно и легко перемещаться по строкам текста на экране.

С появлением сред графического пользовательского интерфейса (GUI) экранные редакторы превратились в графические текстовые редакторы, в которых используется мышь для прокрутки видимой части файла, перехода к определенной точке и выделения текста. Примерами таких текстовых редакторов, основанных на X Window System, являются gedit в системах на базе Gnome и Notepad++ в Windows. Но существуют и другие.

Особый интерес для нас представляет то, что популярные экранные редакторы эволюционировали в графические: GNU Emacs предоставляет несколько окон X, как и Vim через свою версию gvim. Графические редакторы работают идентично своим оригинальным экранным версиям, что сглаживает переход к программам с GUI.

Из всех *стандартных* редакторов в системе Unix vi наиболее полезен¹. По сравнению с Emacs он доступен в практически идентичной форме на всех современных Unix-системах, обеспечивая, таким образом, своего рода общее средство редактирования текста². То же самое можно сказать и о редакторах ed и ex, но GUI-редакторы намного удобнее в использовании (настолько, что линейные редакторы больше не используются).



vi — сокращение от visual editor и произносится как «ви». Это наглядно показано на рис. 1.1.

Для большинства новичков интерфейс vi выглядит громоздким и непонятным: вместо того чтобы позволить вам просто печатать текст, здесь почти все клавиши клавиатуры используются для выполнения каких-либо команд. Чтобы набирать текст на экране, вы должны перейти из *командного режима*, в котором каждая клавиша отвечает за определенное действие, в специальный *режим ввода*. Кроме того, на первый взгляд кажется, что команд в этой программе огромное множество.

¹ Если у вас не установлен ни vi, ни Vim, см. приложение Г.

² GNU Emacs стала универсальной версией Emacs. Единственная проблема заключается в том, что она не входит в стандартную комплектацию большинства систем, ее нужно установить самостоятельно, в том числе в некоторых системах GNU/Linux.



Рис. 1.1. Как правильно произносить vi

Однако, как только вы начнете осваивать редактор, вы поймете, что он неплохо продуман. Вам понадобится лишь несколько нажатий клавиш, чтобы программа начала решать сложные задачи. По мере изучения vi вы узнаете сочетания клавиш, которые позволят компьютеру брать на себя большую часть работы.

vi и Vim (как и любые другие текстовые редакторы) не являются текстовыми процессорами класса «что видите, то и получаете». Если вы хотите создавать форматированные документы, вы должны ввести специальные инструкции (иногда называемые *кодами форматирования*), которые считаются отдельной программой форматирования для управления внешним видом выводимого документа. Например, если вы захотите сделать отступ в тексте абзаца, то потребуется ввести код с указанием, где начинается и заканчивается отступ. Коды форматирования позволяют экспериментировать с образом выводимого файла, и во многих отношениях они дают вам гораздо больше контроля над оформлением ваших документов, чем обычный текстовый процессор.

Коды форматирования — это специфические методы в так называемых *языках разметки*¹. В последние годы популярность языков разметки резко возросла, а особую известность приобрели Markdown и AsciiDoc². Возможно, наиболее популярным языком разметки сегодня является HTML, используемый при создании веб-страниц Интернета.

¹ От выделений красным маркером до «пометки» изменений в наборных гранках или пробках.

² Дополнительную информацию об этих языках можно найти на сайтах <https://en.wikipedia.org/wiki/Markdown> и <https://asciidoc.org> соответственно. Эта книга написана с помощью языка AsciiDoc.

Помимо вышеупомянутых языков разметки, Unix поддерживает пакет форматирования `troff`¹. Популярны и широко распространены программы форматирования TeX (www.ctan.org) и LaTeX (www.latex-project.org). Чтобы использовать любой из этих языков, достаточно открыть текстовый редактор.



Редактор vi поддерживает некоторые простые механизмы форматирования. Например, вы можете дать ему команду автоматически переносить слова при переходе к концу строки или автоматически делать отступы в новых строках. Кроме того, Vim способен проверять орфографию.

Как и любой навык, редактирование в vi требует практики, и чем больше, тем лучше. Привыкнув ко всем возможностям программы, вы, возможно, не захотите возвращаться к какому-либо «более простому» редактору.

Редактирование текста

Что такое редактирование? Во-первых, *ввод* (если, например, забыли какое-то слово или пропустили целое предложение) и *удаление* текста (случайный символ или целый абзац). Также *изменение* букв и слов (исправление орфографических ошибок, опечаток). Возможно, вам понадобится *переместить* текст из одной части документа в другую. И иногда может потребоваться *скопировать* текст, чтобы продублировать его.

В отличие от многих текстовых процессоров в vi командный режим является режимом по умолчанию. Сложные интерактивные изменения могут быть выполнены всего несколькими нажатиями клавиш. Чтобы ввести текст, сначала необходимо задать любую команду ввода.

Для основных команд используются один или два символа. Например:

- `i` — ввод;
- `cw` — изменить слово.

Используя буквы в качестве команд, вы можете очень быстро редактировать файл. Не нужно запоминать сотни горячих клавиш или тянуться пальцами до клавиш

¹ `troff` используется в ПО для лазерных принтеров и наборных устройств. Его брат-близнец `nroff` предназначен для линейных принтеров и терминалов. Оба понимают один и тот же язык ввода. Следуя общепринятому соглашению Unix, под названием `troff` мы ссылаемся на оба пакета. В настоящее время все, кто использует `troff`, работают в GNU-версии под названием `groff` (<https://www.gnu.org/software/groff/>).

в неудобных комбинациях. Вам не придется убирать руки с клавиатуры или возиться с многоуровневыми меню! Большинство команд можно запомнить по первым буквам, отражающим их названия на английском языке. Почти все команды следуют этому принципу и связаны друг с другом.

В целом команды `vi` и `Vim`:

- чувствительны к регистру (клавиши в верхнем и нижнем регистре означают разные действия; `I` отличается от `i`);
- не отображаются (не выводятся) на экране при вводе;
- не требуют нажатия `Enter` после ввода команды.

Существует также группа команд, которые отображаются в нижней строке экрана. Им предшествуют разные символы. Слеш (/) и знак вопроса (?) отвечают за команды поиска и обсуждаются в главе 3. Двоеточие (:) предваряет все команды `ex`. Команды `ex` — это те, которые используются линейным редактором `ex`. Редактор `ex` доступен вместе с любой версией `vi`, потому что `ex` — это базовый редактор, а `vi` — просто его «визуальный» режим. Большая часть команд и концепций `ex` обсуждается в главе 5, а в этой мы познакомимся с командами `ex` для закрытия файла без сохранения изменений.

Небольшая историческая справка

Прежде чем погрузиться во все тонкости `vi` и `Vim`, важно понять роль `vi` в вашей рабочей среде. В частности, мы разберемся во множестве обескураживающих сообщений об ошибках `vi`, а также выясним, как `Vim` эволюционировал по сравнению с оригинальным `vi`.

Редактор `vi` появился, когда в ходу были ЭЛТ-терминалы, подключенные через последовательные интерфейсы к центральным миникомпьютерам. Сотни различных моделей терминалов разрабатывались и использовались по всему миру. Каждый из них выполнял одни и те же действия (очищал экран, перемещал курсор и т. д.), но команды, необходимые для реализации этих действий, были разными. Кроме того, система `Unix` позволяла выбирать символы, которые будут использоваться для возврата, генерации сигнала прерывания и других задач, таких как приостановка и возобновление вывода, выполняемых в последовательных терминалах. Эти действия управлялись (и до сих пор управляются) с помощью команды `stty`.

В оригинальной версии `vi` для Berkeley `Unix` информация об управлении терминалом была отделена от кода (который сложно изменять) и помещена в текстовую

базу данных свойств терминала (легко поддающуюся изменениям), управляемую библиотекой `termcap`. В начале 1980-х годов System V представила двоичную базу данных информации терминала и библиотеку `terminfo`. Эти две библиотеки в целом были эквивалентны по своему функционалу. Чтобы определить, какой у вас терминал, требовалось установить переменную окружения `TERM`. Обычно это выполнялось в файле запуска оболочки `.profile` или `.login`.

Библиотека `termcap` больше не используется. Системы GNU/Linux и BSD работают с библиотекой `ncurses`, которая предоставляет совместимое надмножество базы данных и возможностей библиотеки System V `terminfo`.

Сегодня популярны эмуляторы терминалов в графической среде (например, Gnome Terminal). Система почти всегда самостоятельно настраивает значение переменной `TERM`.



Конечно, вы также можете использовать в Windows консольную программу Vim без GUI. Это пригодится, к примеру, при восстановлении системы в однопользовательском режиме. Однако сейчас мало кто пользуется данным приложением на постоянной основе.

Для повседневного использования лучше всего подойдет версия `vi` с графическим интерфейсом, например `gvim`. В системе Microsoft Windows и macOS она, вероятно, будет приложением по умолчанию. Однако при запуске `vi` (или какого-либо другого экранного редактора того же типа) внутри эмулятора терминала редактор по-прежнему будет обращаться к значению переменной `TERM`, библиотеке `terminfo` и к параметрам команды `stty`. Использование редактора `vi` внутри эмулятора терминала — простой способ изучить `vi` и Vim.

Следует также знать, что программа `vi` была разработана во времена, когда системы Unix были значительно менее стабильными, чем сегодня. Пользователь `vi` тех лет должен был быть готов к сбою системы в любой момент, и поэтому в `vi` реализована функция восстановления файлов, которые редактировались в момент выхода системы из строя¹. Итак, при изучении программ `vi` и Vim и при чтении описаний различных вероятных проблем учитывайте эти исторические события.

¹ К счастью, сейчас такого рода проблемы встречаются гораздо реже, хотя системы все еще могут выходить из строя из-за внешних обстоятельств, таких как отключение электроэнергии. Чтобы решить и эту проблему, обзаведитесь источником бесперебойного питания для настольной системы или емким аккумулятором на ноутбуке.

Открытие и закрытие файлов

Редактор **vi** можно использовать для редактирования любых текстовых файлов. Программа копирует файл в *буфер* (область памяти, выделяемая временно), отображает содержимое буфера (за раз вы видите только то, что помещается на вашем экране) и позволяет добавлять, удалять и изменять текст. Когда вы сохраняете внесенные изменения, программа копирует отредактированное содержимое буфера обратно в существующий файл, заменяя его под тем же именем. Помните, что вы всегда работаете с *копией* вашего файла в буфере и что вносимые изменения не влияют на исходный файл, пока вы не примените их, перезаписав содержимым из буфера. Сохранение изменений также называется записью буфера или чаще всего записью файла.

Открытие файла из командной строки

vim — это команда Unix, которая вызывает редактор Vim для обработки имеющегося или совершенно нового файла. Синтаксис команды **vim** выглядит так:

```
$ vim [имя_файла]
```

или

```
$ vi [имя_файла]
```

Чаще всего в современных системах команда **vi** — это просто ссылка на Vim. Квадратные скобки в приведенных выше командах указывают, что имя файла является необязательным. Скобки вводить не нужно. Знак **\$** — это приглашение командной строки.

Если имя файла опущено, редактор открывает безымянный буфер. Вы можете назначить имя при записи содержимого буфера в файл. На данный момент давайте придерживаться именования файла в командной строке.

Имя файла внутри каталога должно быть уникальным (в некоторых операционных системах каталоги называют *папками*; по сути, это одно и то же).

В системах Unix имя файла может состоять из любых восьмибитных символов, за исключением слеша (/), который считается разделителем между файлами и каталогами в пути, и ASCII NUL — символа со всеми нулевыми битами. Чтобы использовать пробелы в имени файла, потребуется ввести обратный слеш (\) перед самим пробелом (в системах Windows запрещено использовать в именах файлов обратный слеш (\) и двоеточие (:)). Однако на практике имена файлов обычно состоят из любой комбинации прописных и строчных букв, цифр и символов точки (.) и подчеркивания (_). Помните, что Unix чувствителен к регистру: строчные

буквы отличаются от прописных. Также имейте в виду, что вы должны нажать клавишу **Enter**, чтобы сообщить командной оболочке о завершении выполнения команды.

Если вы хотите создать и тут же открыть новый файл, придумайте ему имя и введите его с помощью команды **vi**. Например, если требуется создать и открыть новый файл с именем **practice** в текущем каталоге, наберите:

```
$ vi practice
```

Поскольку это новый файл, буфер пуст, и экран выглядит следующим образом:

```
~
~
~
"practice" [New file]
```

Тильды (~) в левом столбце экрана указывают, что в файле нет текста, даже пустых строк. Строка запроса (также называемая строкой состояния) в нижней части экрана отображает имя и статус файла.

Вы также можете отредактировать любой существующий текстовый файл в каталоге, указав его имя. Предположим, что существует Unix-файл по адресу **/home/john/letter**. Если вы уже находитесь в каталоге **/home/john**, используйте относительный путь. Например, команда ниже выводит на экран копию файла **letter**:

```
$ vi letter
```

Если вы находитесь в другом каталоге, укажите полный путь, чтобы начать редактировать файл:

```
$ vi /home/john/letter
```

Открытие файла из графического интерфейса

Хотя мы (настоятельно) рекомендуем научиться работать в командной строке, вы можете запустить Vim и открыть файл непосредственно из графического интерфейса. Как правило, необходимо щелкнуть правой кнопкой мыши на файле, а затем выбрать команду типа **Открыть с помощью** в контекстном меню. Если редактор Vim установлен правильно, это будет одним из доступных вариантов открытия файла.

Чтобы запустить Vim непосредственно из меню, необходимо с помощью **ex**-команды **:е имя_файла** указать редактору, какой файл редактировать.

Конкретную команду сложно привести, так как в настоящее время существует и используется множество различных графических сред.

Проблемы при открытии файлов

- *Появляется одно из следующих сообщений:*

```
Visual needs addressable cursor or upline capability
терминал: Unknown terminal type
Block device required
Not a typewriter
```

Это значит, что ваш тип терминала не определен или что-то не так с записями `terminfo`. Введите `:q` для выхода. Чтобы исправить проблему и начать работу в упрощенном режиме, чаще всего достаточно присвоить параметру `$TERM` значение `vt100`. Дополнительная информация есть в Интернете или на популярном форуме по техническим вопросам, таком как StackOverflow (<https://stackoverflow.com>).

- *Вы предполагаете, что файл существует, но появляется сообщение [new file].*

Убедитесь, что вы использовали правильный регистр букв в имени файла (системы Unix чувствительны к регистру в именах файлов). Если да, то вы, вероятно, находитесь не в том каталоге. Введите команду `:q` для выхода. Затем убедитесь, что вы открыли корректный каталог для этого файла (введите `pwd` в командной строке). Если путь верен, проверьте список файлов в каталоге (с помощью команды `ls`). Вероятно, файл записан под немного другим именем.

- *Вы запускаете vi, но получаете приглашение с двоеточием (указывающее, что вы находитесь в режиме строкового редактирования ex).*

Вероятно, вы ввели команду прерывания (обычно это сочетание клавиш `Ctrl+C`), прежде чем программа `vi` отрисовала экран. Запустите `vi`, введя в приглашении `ex (:)` команду `vi`.

- *Появляется одно из следующих сообщений:*

```
[Read only]
File is read only
Permission denied
```

`Read only` (только для чтения) означает, что вы не можете сохранять внесенные изменения, только просматривать файл. Возможно, вы запустили `vi` в режиме просмотра (с помощью команды `view` или `vi -R`) или у вас нет разрешения на перезапись файла (см. подраздел «Открытие файла из командной строки» ранее в этой главе).

- *Появляется одно из следующих сообщений:*

```
Bad file number
Block special file
Character special file
Directory
Executable
Non-ascii file
имя_файла non-ASCII
```

Файл, который вы хотите отредактировать, не является допустимым текстовым файлом. Введите `:q!`, чтобы завершить работу, а затем проверьте этот файл с помощью команды `file`.

- При попытке ввести команду `:q` из-за одной из ранее упомянутых проблем появляется сообщение:

```
E37: No write since last change (add ! to override)
```

Вы случайно изменили файл. Введите команду `:q!`, чтобы закрыть редактор. Ваши изменения, внесенные в ходе этого сеанса, не будут сохранены.

Modus Operandi

Как упоминалось ранее, концепция текущего режима имеет основополагающее значение для способа работы vi. Существует два режима: *командный режим* и *режим ввода текста* (режим команды `ex` можно считать третьим по счету, но пока мы не будем его рассматривать). При запуске активизируется командный режим, где каждое нажатие клавиши представляет собой команду¹. В режиме ввода все, что вы набираете, преобразовывается в текст.

Иногда можно случайно перейти в режим ввода или, наоборот, выйти из него. Так или иначе любые нажатия на клавиши, скорее всего, повлияют на содержимое ваших файлов непредвиденным образом.

Нажмите клавишу `Esc`, чтобы перевести редактор в командный режим. Если вы уже в нем, программа подаст звуковой сигнал (поэтому командный режим иногда называют сигнальным режимом).

Перейдя в командный режим, вы сможете приступить к исправлению любых случайных изменений, а затем продолжить редактирование текста (см. подразделы «Проблемы при удалении» и «Откат изменений» в главе 2).

Сохранение и закрытие файла

Чтобы в любой момент прекратить работу с файлом, сохранить внесенные изменения и вернуться в командную строку (если вы работаете в окне терминала), используйте команду `ZZ`. Обратите внимание, что `ZZ` пишется прописными буквами.

Предположим, что вы создали файл с именем `practice`, чтобы попрактиковаться с командами vi, и ввели шесть строк текста. Чтобы сохранить файл, сначала

¹ Обратите внимание, что программы vi и Vim имеют команды не для всех возможных клавиш. В командном режиме редактор ожидает нажатий кнопок, представляющих команды, а не текст в качестве содержимого файла. Мы воспользуемся преимуществами неиспользуемых клавиш позже, в разделе «Команда `map`» главы 7.

убедитесь, что вы находитесь в командном режиме, нажав клавишу **Esc**, а затем наберите **ZZ**.

Команда	Результат
ZZ	"practice" [New] 6L, 104C written При вводе команды записи ZZ ваш файл сохранится на диске как обычный файл
\$!s	ch01.asciidoc ch02.asciidoc ch03.asciidoc В списке файлов в каталоге будет файл с именем practice , который вы только что создали

Внесенные изменения можно сохранить и с помощью команд **ex**. Введите **:w**, чтобы сохранить (записать) ваш файл, не закрывая его. Если вы не вносили никаких изменений, просто введите **:q**, чтобы выйти. А чтобы сохранить ваши изменения и закрыть текущий файл, введите **:wq** (команда **:wq** эквивалентна **ZZ**). Мы подробно объясним, как использовать команды **ex**, в главе 5. Сейчас просто запомните эти несколько команд для записи и сохранения файлов.

Заккрытие файла без сохранения изменений

Если вы только начали изучать Vim, особенно будучи бесстрашным экспериментатором, вам пригодятся две другие команды **ex**, которые позволят избавиться от созданной вами путаницы.

Если вы хотите стереть все изменения, внесенные вами во время сеанса работы, а затем снова открыть исходный файл, то команда:

:e! Enter

откроет последнюю сохраненную версию файла, тем самым позволив вам начать заново.

Если вы не хотите сохранять свои изменения, а затем планируете выйти из редактора, то используйте следующую команду, которая немедленно завершит работу с редактируемым файлом и вернет вас в командную строку:

:q! Enter

Данные команды откатывают назад все изменения, внесенные в буфер с момента последнего сохранения файла. Редактор обычно не позволяет удалять изменения. Восклицательный знак, добавленный к команде **:e** или **:q**, снимает этот запрет, и программа выполняет операцию, даже если содержимое буфера было изменено.

Далее мы не будем напоминать о необходимости нажимать клавишу **Enter** для команд в режиме **ex**, однако это требуется, чтобы редактор их выполнял.

Проблемы при сохранении файлов

- Вы пытаетесь записать файл, но получаете одно из следующих сообщений:

```
File exists
File имя_файла exists - use w!
[Existing file]
File is read only
```

Введите `:w! имя_файла`, чтобы перезаписать существующий файл, или `:w имя_нового_файла`, чтобы сохранить отредактированную версию в новом файле.

- Вы хотите записать файл, но у вас нет для этого разрешения. Выводится сообщение *Permission denied* (Отказано в доступе).

Используйте команду `:w имя_нового_файла` для записи содержимого буфера в новый файл. Если у вас есть разрешение на запись в каталог, вы можете использовать команду `mv`, чтобы заменить исходную версию своей копией. Если у вас нет разрешения на запись в каталог, введите команду `:w путь/имя_файла`, чтобы записать содержимое буфера в файл в каталоге, для которого у вас есть разрешение (например, ваш домашний каталог или `/tmp`). Будьте осторожны, чтобы не перезаписать какие-либо существующие файлы в данной директории.

- Вы пытаетесь записать свой файл, но получаете сообщение, что хранилище заполнено.

Сегодня, когда 500-гигабайтный накопитель считается небольшим по емкости, подобные ошибки, как правило, встречаются редко. Тем не менее, если что-то подобное произойдет, у вас есть несколько вариантов решения такой проблемы. Во-первых, попробуйте записать свой файл на другой диск или в другой каталог (например, в `/tmp`), чтобы ваши данные были сохранены. Затем попробуйте сохранить содержимое буфера с помощью команды `ex :pre` (сокращенно от `:preserve`). Если и это не сработает, освободите пространство одним из следующих способов.

- Откройте файловый менеджер с графическим интерфейсом (например, Nautilus в GNU/Linux) и поищите старые ненужные файлы, которые можно удалить.
- Нажмите **Ctrl+Z**, чтобы приостановить работу vi и вернуться к командной строке. Затем стоит использовать команды Unix, чтобы найти большие файлы, которые являются кандидатами на удаление:
 - команда `df` показывает, сколько свободного места доступно на диске в данной файловой системе или в системе в целом;
 - команда `du` указывает, какой объем диска используется для данных файлов и каталогов. Команда `du -s * | sort -nr` выводит список файлов и каталогов, отсортированных по занимаемому ими пространству в порядке убывания.

Когда закончите удалять файлы, используйте команду `fg`, чтобы вернуться к программе vi; затем вы можете сохранить свою работу в обычном режиме.

Помимо нажатия **Ctrl+Z** и управления задачами, допускается ввести `:sh`, чтобы запустить новую оболочку. Нажмите **Ctrl+D** или введите команду `exit`, чтобы завершить работу в оболочке и вернуться к программе `vi` (это работает и с `gvim!`).

Вы также можете ввести что-то вроде `:!du -s *`, чтобы запустить консольную команду из `vi`, а затем вернуться к редактированию, когда команда будет выполнена.

Упражнения

Единственный способ изучить программы `vi` и `Vim` — практиковаться. Теперь вы знаете достаточно, чтобы создать новый файл и вернуться в командную строку. Создайте файл с именем `practice`, введите текст, а затем сохраните и закройте файл.

1. Откройте файл с именем `practice` в текущем каталоге:

\$ vi practice

2. Перейдите в режим ввода:

i

3. Введите текст:

любой текст

4. Вернитесь в командный режим:

Esc

5. Выйдите из `vi`, сохранив изменения:

ZZ

Основы редактирования

Прочитав эту главу, вы научитесь редактировать текст с помощью `vi` и `Vim`. Вы узнаете, как перемещать курсор и вносить простые правки. Если вы никогда не работали с редакторами `vi` и `Vim`, прочтите главу целиком.

Последующие главы помогут вам работать быстрее и эффективнее. Одно из самых главных преимуществ для опытного пользователя — огромный выбор инструментов, а явный *недостаток* для новичка в `vi` и `Vim` — множество различных команд редактора.

Поначалу нет необходимости запоминать каждую описанную команду `vi`. Начните с изучения основных команд, представленных в текущей главе, и обратите внимание на их общий синтаксис (не переживайте, мы укажем на эти места).

По мере обучения следите за дополнительными задачами, которые может выполнить редактор, а затем найдите команду, которая их выполняет. В последующих главах вы узнаете о более продвинутых функциях `vi` и `Vim`, но прежде, чем двигаться дальше, важно освоить основные.

В этой главе мы рассмотрим:

- перемещение курсора;
- базовые операции: добавление, изменение, удаление, перемещение и копирование текста;
- несколько вариантов перехода в режим ввода;
- объединение строк;
- индикаторы режимов.

Команды vi

Как упоминалось ранее, редакторы vi и Vim имеют два основных режима: *командный* и *режим ввода*. Командную строку (приглашение с двоеточием), в которой вы вводите команды ex, можно считать третьим режимом. Это более продвинутый режим, и его мы рассмотрим позже.

Впервые открывая файл, вы попадаете в командный режим, где редактор ожидает вашей команды. Команды позволяют перемещаться по содержимому файла, вносить правки или переключаться в режим ввода для добавления нового текста. Команды также позволяют закрыть файл (с сохранением изменений или без), чтобы вернуться к командной строке.

Эти режимы можно воспринимать как две разные клавиатуры: в режиме ввода клавиатура работает как обычно, в командном — каждая клавиша имеет новое значение или инициирует какую-либо инструкцию.

Существует несколько способов сообщить Vim, что вы хотите перейти в режим ввода. Самый простой — нажать клавишу i. При этом буква i не отобразится на экране, однако все, что вы напечатаете после, *возникнет* на дисплее и будет передаваться в буфер. Курсор отмечает текущую точку ввода¹. Чтобы программа Vim вышла из режима ввода, нажмите клавишу Esc. Это действие переместит курсор назад на один пробел (он встанет на последний введенный вами символ) и вернет вас в командный режим.

Предположим, что вы открыли новый файл и хотите вставить слово *introduction*. Если вы введете `iintroduction`, на экране появится:

```
introduction
```

Когда вы открываете новый файл, Vim запускается в командном режиме и интерпретирует первое нажатие клавиши I как команду ввода. С этого момента вы можете печатать текст, а чтобы вернуться в командный режим, необходимо нажать клавишу Esc. Если вам нужно исправить ошибку в режиме ввода, расположите курсор на месте ошибки и введите текст. В зависимости от вашего терминала и его настроек в процессе исправления программа может стереть ранее введенное или создать резервную копию поверх текста. В любом случае все резервные копии удалятся. Обратите внимание, что вы не можете вернуться назад ранее позиции, в которой вы вошли в режим ввода. Если вы отключили совместимость с vi, Vim позволяет вам обойти данное ограничение. В большинстве дистрибутивов GNU/Linux Vim настроен с отключенной совместимостью с vi, так что прием может сработать.

¹ В некоторых версиях редактор показывает, что вы находитесь в режиме ввода в строке состояния. Мы обсудим это в разделе «Индикаторы режимов».

В Vim есть параметр, позволяющий определять правое поле и автоматически возвращать курсор в начало. Попробуйте во время вставки текста нажать клавишу **Enter**, чтобы прервать строку.

Не всегда очевидно, какой режим активирован в текущий момент. Всякий раз, когда Vim действует не так, как ожидается, нажмите клавишу **Esc** один или два раза. Если воспроизводится звуковой сигнал, значит, вы находитесь в командном режиме¹.

Перемещение курсора в командном режиме

Обычно на добавление нового текста в режиме ввода тратится немного времени. Большую его часть вы будете вносить изменения в существующий текст, перемещаясь по файлу и выполняя различные операции.

В командном режиме вы можете поместить курсор в любое место файла. Поскольку все основные правки (изменение, удаление и копирование текста) начинаются с позиционирования курсора в тексте, важно научиться перемещать его по тексту как можно быстрее.

Существует несколько команд **vi** для перемещения курсора:

- вверх, вниз, влево или вправо — по одному *символу* за раз;
- вперед или назад по фрагментам *текста*, таким как слова, предложения или абзацы;
- вперед или назад по файлу на один *экран*.

На рис. 2.1 выделенная буква **s** в третьей строке обозначает текущее положение курсора. Круги показывают перемещение курсора из его текущего положения в положение после выполнения различных команд.

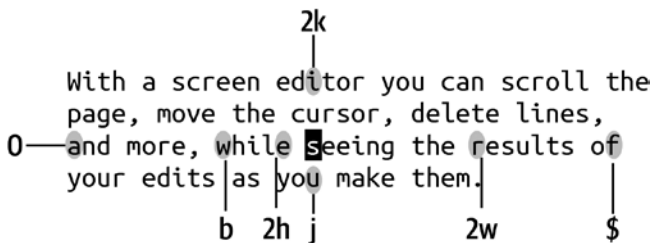


Рис. 2.1. Пример перемещения курсора из центрального **s** с помощью команд

¹ Если в вашей системе отключен звук, вам придется визуально определить, в каком режиме вы находитесь, наблюдая за тем, как редактор реагирует на ввод.

Единичное перемещение

Клавиши `h`, `j`, `k` и `l`, находящиеся прямо под рукой, позволяют перемещать курсор следующим образом:

- `h` — влево на один символ;
- `j` — вниз на одну строку;
- `k` — вверх на одну строку;
- `l` — вправо на один символ.

Вы также можете использовать клавиши со стрелками (`←`, `↓`, `↑`, `→`), `+` и `-` или сочетания клавиш `Ctrl+P` и `Ctrl+N` для перемещения вверх и вниз, а также клавиши `Enter` и `Backspace`, но они находятся далеко от центра клавиатуры.

Поначалу может показаться неудобным использовать буквенные клавиши вместо стрелок для перемещения курсора. Однако через некоторое время вы обнаружите, что возможность перемещаться по тексту, не отрывая пальцев от центра клавиатуры, — это невероятно удобно.

Перед тем как изменить положение курсора, нажмите клавишу `Esc`, чтобы активировать командный режим. Используйте клавиши `h`, `j`, `k` и `l` для перемещения вперед или назад по файлу от текущей позиции курсора. Передвинувшись до конца в одном направлении, вы услышите звуковой сигнал, а курсор остановится. Например, когда вы находитесь в начале или в конце строки, то для перехода к предыдущей или следующей строке вы не можете использовать клавиши `h` или `l` соответственно — для этого необходимы клавиши `j` или `k`¹. Аналогично у вас не получится переместить курсор мимо тильды (~), представляющей строку без текста, а также не выйдет расположить курсор в позиции над первой строкой текста.

ПОЧЕМУ ИМЕННО КЛАВИШИ H, J, K И L

Мэри Энн Хортон, работавшая в компании Berkeley Unix почти с самого ее основания, рассказывает такую историю.

Программа `vi` обладала очень мощными возможностями, хотя ее интерфейс во многом был схож с интерфейсом приложения Блокнот (Notepad). Студенты и преподаватели активно использовали все доступные инструменты, в том числе и «глобальную» команду, которая вносила одинаковые изменения во все строки, соответствующие определенному шаблону, или команды типа «удалить 13 абзацев» или «скопировать

¹ Vim с параметром `nocompatible` позволяет переходить от конца строки к следующей при нажатии Пробела. Чаще всего данный параметр установлен по умолчанию.

текст в соответствующих круглых скобках». Но освоить программу `vi` было не так-то просто, и начинающие пользователи для перемещения по содержимому файлов по привычке использовали клавиши со стрелками, как делали это в Блокноте (Notepad).

Однако клавиши со стрелками в `vi` не всегда корректно срабатывали, на что была своя причина. У пользователей было множество терминалов различных производителей, которые обрабатывали разные коды при нажатии данных клавиш.

Биллу не нужны были такие клавиши. Он нашел способ работать из дома, установив в своей квартире терминал Lear-Siegler ADM-3A. ADM-3A преподносился как «неинтеллектуальный терминал», потому что у него не было некоторых особенностей, например отдельных клавиш со стрелками, что позволяло продавать его по низкой на тот момент цене 995 долларов. Стрелки были нарисованы на клавишах `h`, `j`, `k` и `l`¹. Билл настроил команды `vi` следующим образом: нажатие клавиши `h` перемещало курсор влево, `j` — вниз, `k` — вверх и `l` — вправо. Всем пользователям нужно было нажимать буквы `h`, `j`, `k` и `l`, чтобы перемещаться по файлу.

Но что делать, если вы хотите ввести слово с буквой `h`? Редактор `vi`, как и `ed`, — это «режимный» редактор. То есть вы находитесь либо в командном режиме, когда нажимаемые клавиши обрабатываются как команды, либо в режиме ввода, где нажатия клавиш становятся содержимым, добавляемым в файл. Команда, подобная `i` (`insert`), переводит приложение в режим ввода, а клавиша `Escape` (`Esc`) возвращает в командный режим.

Как сделать так, чтобы клавиши со стрелками в `vi` работали? Эти специальные клавиши посылают две или три последовательности символов, обычно начинающиеся с `Escape`. Мы назвали их `escape-последовательностями`. Однако команда `Escape` уже имела свое назначение в `vi`. Она выводила редактор из режима ввода и, если вы уже были вне его, воспроизводила звуковой сигнал. Одна из первых важных функций, с которой пользователь сталкивается в `vi`, заключалась в том, что, если вы забыли, в каком режиме находитесь, вы нажимали клавишу `Esc` до тех пор, пока она не подаст звуковой сигнал, и тогда вы понимали, что находитесь в командном режиме.

Редактор `vi` использовал файл базы данных функций терминала под названием `termcap`, в котором указывалось, какие коды для конкретной модели терминала отправляются для перемещения курсора, очистки экрана и т. п. Осталось добавить последовательности клавиш `←`, `→`, `↑`, `↓` в `termcap`.

Компьютер получил команду `Escape` — значит ли это, что пользователь нажал клавишу `Esc` или клавишу со стрелкой? Должен ли редактор выйти из режима ввода или ему следует подождать, пока добавится текст, чтобы интерпретировать клавишу со стрелкой? Как только редактор пытался считать больше данных, программа зависала до тех пор, пока не поступала новая информация.

К счастью, новая возможность Unix позволила редактору ожидать введения еще одного символа. Если этот символ является частью допустимой `escape-последовательности`, `vi` продолжает считывание, ожидая, какую следующую клавишу нажмет пользователь. Если за этот короткий промежуток времени больше не было введено никаких символов, программа считала, что пользователь, должно быть, нажал клавишу `Esc`. Проблема решена!

¹ Фотографии клавиатуры этого терминала можно легко найти в Интернете.

Весной 1979 года я добавила команды `code` и `termcap` для клавиш `←`, `→`, `↑`, `↓`, `Home`, `Page Up` и других, которыми были оборудованы некоторые терминалы. Я настроила `termcap` так, как если бы у ADM-3а были клавиши со стрелками, которые посылали команды `h`, `j`, `k` и `l`; а затем я удалила привязанные команды для `h`, `j`, `k` и `l`. Мне казалось, все работало прекрасно.

Через день перед дверью моего кабинета собралась толпа разгневанных аспирантов с Питером во главе. Он хотел знать, зачем я изменила настройки его терминала. Я объяснила ему, что теперь клавиши со стрелками работают нормально и больше нет необходимости использовать буквенные символы.

Питер закатил глаза: «Ты не понимаешь, — сказал он, — нам нравится использовать клавиши `h`, `j`, `k` и `l`! Мы набираем команды вслепую, и наши пальцы находятся прямо над этими клавишами. Мы не хотим тянуться к краю клавиатуры и нажимать клавиши со стрелками. Верни все как было!» Все студенты подержали Питера.

Они были правы. Я вернула настройки по умолчанию и оставила функционал клавиш со стрелками. И поняла, насколько важны позиции клавиш для ключевых команд `vi`. Почти любая часто применяемая команда — это строчная буква. Я очень быстро освоила `vi` и по сей день предпочитаю `vi` для редактирования текстовых файлов. Я обучила несколько групп ИТ-специалистов максимально эффективному использованию мощных инструментов `vi` и Unix.

Числовые аргументы

Вам может потребоваться повторить команду несколько раз. Вместо того чтобы вводить ее снова и снова, допускается добавить к ней число — *количество повторений*, или *коэффициент повторения*.

На рис. 2.2 показано, как команда `4l` перемещает курсор на четыре позиции вправо, как если бы вы нажали `l` четыре раза подряд (`llll`).

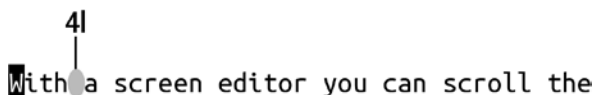


Рис. 2.2. Повторение команды с помощью чисел

Счетчик повторений указывается *перед* командой, потому что, если указать его после, программа `vi` не поймет, когда необходимо остановиться.

Способность редактора повторять операции открывает больше возможностей для каждой изучаемой вами команды. Имейте это в виду, когда мы будем изучать дополнительные команды.

Перемещение по строке

После сохранения файла `practice` программа Vim отобразит сообщение о том, сколько строк в этом файле. *Текстовые строки* не обязательно имеют ту же длину, что и строка, видимая на экране. Строка — это текст, введенный между новыми строками (чтобы добавить в файл символ *новой строки*, необходимо нажать клавишу `Enter` в режиме ввода). Программа Vim интерпретирует все введенные символы как одну строку до тех пор, пока вы не нажмете `Enter` (даже если они явно занимают несколько строк на экране).

В главе 1 мы упоминали, что в `vi` и Vim есть параметр, который позволяет устанавливать расстояние от правого края (конца строки), на котором автоматически вставляется символ новой строки. Этот параметр называется `wrapmargin` (коротко — `wm`). Вы можете установить `wrapmargin` в размере десяти символов:

```
:set wm=10
```

Данная команда не влияет на уже добавленные строки. Как только вы зададите параметр, попробуйте ввести несколько новых строк и увидите, что программа Vim автоматически переносит их. Подробнее о настройках мы поговорим в главе 7, потерпите немного!



Если поместить эту команду в файл с именем `.exrc` в домашнем каталоге, редактор будет автоматически выполнять ее при каждом запуске. Далее в книге мы рассмотрим загрузочные файлы `vi` и Vim.

Если вы не используете автоматический параметр `wrapmargin`, следует прерывать строки с помощью клавиши `Enter`, чтобы они были приемлемой длины.

Две полезные команды, которые связаны с перемещениями внутри строки, — это:

- `0` (цифра ноль) — переход в начало строки;
- `$` — переход в конец строки.

Нумерация строк продемонстрирована в следующем примере (номера строк можно отобразить с помощью параметра `number`, который включается вводом команды `:set nu` в командном режиме. См. главу 5). Обратите внимание, что номера строк не относятся к содержимому файла, программа отображает их для вашего удобства:

```
1 With a screen editor you can scroll the page,  
2 move the cursor, delete lines, insert characters,  
   and more, while seeing the results of your edits  
   as you make them.  
3 Screen editors are very popular.
```

Количество логических строк (три) не соответствует количеству видимых (пять) на экране. Если бы курсор был установлен на букву *d* в слове *delete* и вы ввели символ *\$*, курсор переместился бы в позицию, следующую за словом *them*. Если вы введете *0*, курсор переместится в обратном направлении, на букву *m* в слове *move* — в начало второй строки.

Перемещение по фрагментам текста

Вы можете перемещать курсор по фрагментам текста, таким как слова, предложения, абзацы и т. д.:

- **w** — переход вперед на одно слово (слова — это буквенно-цифровые символы);
- **W** — переход вперед на одно слово (слова разделяются пробелом);
- **b** — переход назад на одно слово (слова — это буквенно-цифровые символы);
- **B** — переход назад на одно слово (слова разделяются пробелом);
- **G** — переход к определенной строке.

Команда **w** перемещает курсор вперед на одно слово, считая символы и знаки препинания за слова. В примере ниже показано перемещение курсора с помощью **w**:

```
cursor, delete lines, insert characters,
```

Двигаться вперед по слову без учета символов и знаков препинания можно, используя команду **W**. Перемещение курсора с помощью прописной **W** выглядит следующим образом:

```
cursor, delete lines, insert characters,
```

Чтобы перейти назад на слово, используйте команду **b**. Заглавная буква **B** позволяет перемещаться назад по слову, не считая знаков препинания (на одно слово). Как упоминалось ранее, команды перемещения принимают числовые аргументы. Таким образом, вы можете повторять команды **w** или **b** с помощью чисел. Команда **2w** перемещает курсор вперед на два слова, **5B** перемещает курсор назад на пять слов, не считая знаков препинания.

Чтобы перейти к определенной строке, вы можете использовать команду **G**.

Команда **G** позволяет перейти в конец файла, **1G** — в его начало, а **42G** — на строку 42. Подробнее об этом читайте в подразделе «Команда **G** (перейти к)» следующей главы. Мы обсудим перемещение по предложениям и абзацам в главе 3. А пока попрактикуйтесь с уже известными вам командами с курсором, комбинируя их с числовыми множителями.

Базовые операции

Ввести текст в файл идеально с первого раза получается не всегда. Возникают опечатки, бывает, что хочется перефразировать предложение, или же программа аварийно завершает работу. При вводе текста важно уметь производить над ним базовые операции: изменение, удаление, перемещение и копирование. На рис. 2.3 показаны виды изменений, которые можно вносить в файл. Правки обозначены корректорскими знаками.

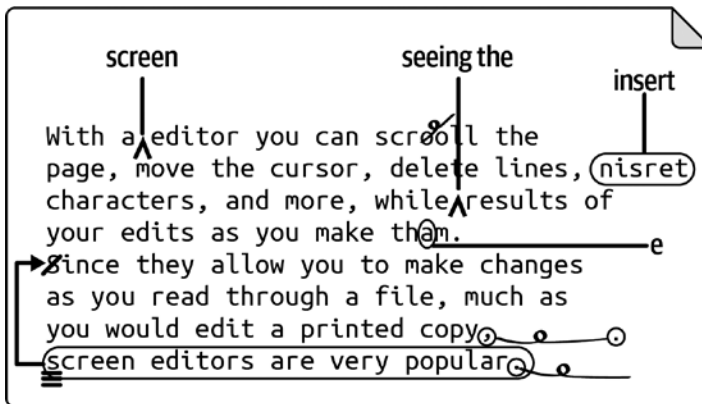


Рис. 2.3. Исправление текста

В программе `vi` вы можете выполнить любое из показанных изменений с помощью следующих клавиш: `i` для ввода текста (упоминалась ранее), `a` для добавления (присоединения), `s` для изменения и `d` для удаления. Чтобы переместить или скопировать текст, используется пара команд. Переместить текст можно с помощью клавиш `d` (удалить) и `p` (вставить), скопировать — `y` (поместить в буфер) и `p` (вставить). Вы также можете использовать клавиши `x` и `r` для удаления и замены одного символа соответственно. Некоторые команды при двойном вводе, такие как `dd`, означают «применить команду ко всей строке». Другие, написанные с заглавной буквы, например `P`, означают «применить операцию к тексту перед текущей строкой, а не после». Каждый тип редактирования описан в данном разделе. На рис. 2.4 показаны команды `vi`, которые используются для внесения изменений, отмеченных на рис. 2.3.

Текст этого файла можно использовать для практики; он находится в репозитории GitHub (<https://www.github.com/learning-vi/vi-files>). Дополнительные сведения см. в подразделе «Доступ к нескольким файлам» в приложении А.

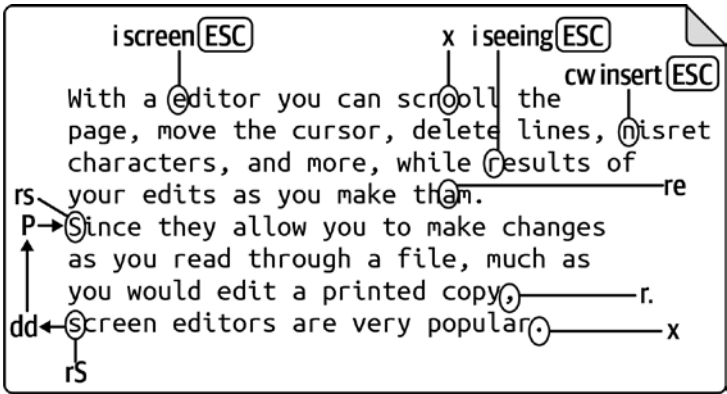


Рис. 2.4. Редактирование с помощью команд

Ввод текста

Мы уже сталкивались с командой `i` (insert), используемой для ввода текста в новый файл. Ее также можно применять при редактировании существующего текста, чтобы добавить недостающие символы, слова и предложения. Предположим, что в файле `practice` у вас есть такое предложение (обратите внимание на положение курсора):

```
you can scroll
the page, move the cursor, delete
lines, and insert characters.
```

Чтобы вставить фразу *With a screen editor* в начало предложения, введите следующее.

Команда	Результат
<code>2k</code>	<pre>You can scroll the page, move the cursor, delete lines, and insert characters.</pre> Переместите курсор на две строки вверх, к строке, куда нужно вставить текст, с помощью команды <code>k</code>
<code>iWith a</code>	<pre>With a You can scroll the page, move the cursor, delete lines, and insert characters.</pre> Нажмите <code>i</code>, чтобы перейти в режим ввода и вставить текст. Символ <code> </code> — это пробел
<code>screen editor</code> <code>Esc</code>	<pre>With a screen editor You can scroll the page, move the cursor, delete lines, and insert characters.</pre> Нажмите клавишу <code>Esc</code>, чтобы завершить добавление текста и вернуться в командный режим

Добавление текста

Вы можете добавить текст в любую позицию файла с помощью команды добавления (присоединения) `a` (`append`). Она работает почти так же, как `i`, за исключением того, что текст вставляется *после* курсора, а не *перед* ним. Возможно, вы заметили, что для перехода в режим ввода при нажатии клавиши `I` курсор не перемещается до тех пор, пока вы не начнете печатать. Напротив, когда вы нажимаете клавишу `a`, чтобы перейти в режим ввода, курсор перемещается на одну позицию вправо, а текст появляется за исходным положением курсора.

Замена текста

Вы можете заменить любой текст в вашем файле с помощью команды `c` (`change`). Чтобы совершить данную операцию, объедините команду `c` с командой перемещения, которая при этом служит *текстовым объектом* для работы команды `c`. Например, команду `c` допускается использовать для замены текста от позиции курсора:

- `cw` — до конца слова;
- `c2b` — вернуться на два слова назад;
- `c$` — в конец строки;
- `c0` — в начало строки.

После выдачи команды `c` вы можете заменить выделенный текст новым текстом любого размера, пустой строкой, одним словом или сотнями строк. Команда `c`, как и `i` и `a`, не выводит программу из режима ввода, пока вы не нажмете клавишу `Esc`.

Когда изменение затрагивает только текущую строку, `vi` помечает конец текста, подлежащего преобразованию, символом `$`, чтобы вы могли видеть, какая часть строки была затронута. Если Vim не находится в режиме совместимости, он ведет себя по-другому: просто удаляет текст, который нужно изменить, и переводит вас в режим ввода.

Слова

Чтобы изменить слово, объедините команды `c` и `w` (`word`). С помощью команды `cw` можно заменить слово более длинным или коротким (или любым фрагментом текста). Команда `cw` рассматривается как «удаление отмеченного слова и вставка нового текста до тех пор, пока не будет нажата клавиша `Esc`».

Предположим, у вас есть следующая строка в вашем файле `practice`, где нужно изменить `an` на `a screen`:

With an editor you can scroll the page,

Вам нужно изменить только одно слово.

Команда	Результат
w	With an editor you can scroll the page, Переместите курсор с помощью w в позицию, где вы хотите начать редактирование
cw	With an editor you can scroll the page, Дайте команду изменить слово. Vim удалит элемент <i>an</i> и перейдет в режим ввода
a screen Esc	With a screen editor you can scroll the page, Введите новый текст, а затем нажмите Esc , чтобы вернуться в командный режим

Команда cw также работает с частью слова. Например, чтобы изменить слово *spelling* на *spelled*, необходимо поместить курсор на *i*, ввести cw, затем напечатать *ed* и завершить действие с помощью клавиши Esc.

СИНТАКСИС КОМАНД VI

В командах изменения, которые мы упоминали ранее, вы, возможно, заметили следующую закономерность синтаксиса:

(команда)(текстовый объект)

Команда — это команда изменения *c*, а *текстовый объект* — это команда перемещения (круглые скобки вводить не нужно). Однако *c* — не единственная команда, для которой требуется текстовый объект. Команды *d* (delete) и *y* (yank) действуют по тому же принципу.

Помните, что команды перемещения принимают числовые аргументы, поэтому к текстовым объектам команд *c*, *d* и *y* можно добавлять числа. Например, команды *d2w* и *2dw* используются для удаления двух слов. Из этого следует, что большинство команд *vi* строятся согласно одному общему синтаксису:

(команда)(число)(текстовый объект)

или

(число)(команда)(текстовый объект)

Вот как это работает: *число* и *команда* не обязательны. Вы можете использовать лишь одну команду перемещения. Если вы добавляете *число*, действие команды повторяется.

С другой стороны, можно объединить команду (*c*, *d* или *y*) с *текстовым объектом*, чтобы получить команду редактирования. Поняв, какое множество комбинаций возможно, вы осознаете, насколько мощный редактор Vim!

Строки

Чтобы заменить текущую строку, используйте специальную команду `сс`. Не имеет значения, где находится курсор в строке — `сс` заменяет ее всю любым количеством текста, введенного до нажатия клавиши **Esc**.

В `vi` команда, подобная `св`, работает иначе, чем `сс`. При использовании `св` старый текст остается до тех пор, пока вы не введете поверх него другой, а любой оставшийся старый текст (вплоть до символа `$`) исчезнет, когда вы нажмете клавишу **Esc**. Однако при использовании команды `сс` сначала стирается старый текст, оставляя пустую строку для ввода нового.

Подход «ввода текста поверх» применяется к любой команде изменения, которая затрагивает менее целой строки, в то время как подход «пустая строка» применяется к любой команде изменения, которая затрагивает одну или несколько строк.

В редакторе Vim (если он не находится в режиме совместимости) обе команды просто удаляют указанный текст, а затем переходят в режим ввода.

Команда `С` заменяет символы от текущего положения курсора до конца строки. Эта операция похожа на действие объединения со специальным индикатором конца строки `$` (`с$`).

На самом деле команды `сс` и `С` являются сочетаниями для других команд, поэтому они не соответствуют общей форме команд `vi`, поскольку не позволяют указать текстовый объект в качестве конечной точки команды. Мы обсудим и другие сочетания, когда будем изучать команды `delete` и `yank`.

Символы

Еще один вариант замены задается командой `r` (`replace`). Она заменяет один символ другим. При этом *не* нужно нажимать клавишу **Esc**, чтобы вернуться в командный режим после внесения изменений. В следующем примере в строке допущена орфографическая ошибка:

Pith a screen editor you can scroll the page,

Необходимо исправить только одну букву. Здесь нет нужды использовать команду `св`, иначе придется перепечатывать все слово. Вместо этого используйте `r` для замены символа, на котором расположен курсор:

Команда	Результат
<code>rW</code>	W ith a screen editor you can scroll the page, Команда <code>rW</code> выдает команду <code>r</code> и заменяет символ <code>P</code> на <code>W</code>

Замена текста

Предположим, вы хотите изменить слово не целиком, а только несколько символов в нем. Команда (**s**) сама по себе заменяет один символ. Числом вы можете указать, какое количество символов необходимо заменить. Как и в случае с командой **c**, **vi** помечает последний символ текста знаком **\$**, чтобы вы увидели, какой фрагмент текста будет изменен. Vim просто удаляет текст и переходит в режим ввода (может показаться, что команда **s** похожа на **r**, но она переходит в режим ввода вместо прямой замены указанных символов).

Команда **S**, как это обычно бывает с командами верхнего регистра, позволяет изменять целые строки. В отличие от команды **C**, которая преобразует остальную часть строки от текущей позиции курсора, **S** удаляет всю строку, независимо от того, где находится курсор. Редактор перемещает вас в начало строки. Число перед командой обозначает замену соответствующего количества строк (**S** и **c** фактически равнозначны).

И **s**, и **S** переводят вас в режим ввода; когда вы закончите вводить новый текст, нажмите клавишу **Esc**.

Команды **R** и **r** заменяют текст. Они отличаются тем, что **R** просто переходит в режим «наложения» текста. Вводимые символы заменяют то, что отображается на экране, символ за символом, пока не будет нажата клавиша **Esc**. Если вы находитесь в середине абзаца и активируете **R**, вы можете «перекрыть» максимум одну строчку. А при нажатии клавиши **Enter** редактор создает новую строку, фактически переводя вас в режим ввода.

Изменение регистра

Изменение регистра буквы — это особая форма замены. Нажатие клавиши «тильда» (**~**) преобразует строчную букву в прописную и наоборот. Установите курсор на букву, которую вы хотите перевести в другой регистр, и нажмите клавишу **~**. Регистр буквы изменится, и курсор переместится к следующему символу. Укажите число перед командой, чтобы применить ее к нескольким символам. Если вы хотите изменить регистр нескольких строк одновременно, следует отфильтровать текст с помощью команды Unix, такой как **tr**. Подробнее об этом — в главе 7.

Удаление текста

Удалить любой текст в вашем файле вы можете с помощью команды **d** (delete). Команде **d**, как и **c**, требуется текстовый объект (с которым нужно работать). Вы можете удалять с помощью команды **dw** (по слову), **dd** и **D** (по строке) или с помощью других команд перемещения, о которых мы поговорим позже. Для любого вида

удаления необходимо переместиться на нужную позицию, а затем задать команду удаления (**d**) и текстовый объект, например команду **w** для удаления всего слова.

Слова

Предположим, у вас есть следующий текст в файле (курсор расположен так, как показано в коде):

```
Screen editors are are very popular,
since they allow you to make
changes as you read through a file.
```

Вам нужно удалить одно лишнее слово *are* в первой строке.

Команда	Результат
2W	Screen editors are are very popular, since they allow you to make changes as you read through a file. Переместите курсор к месту, с которого вы хотите начать редактирование (слово <i>are</i>)
dw	Screen editors are very popular, since they allow you to make changes as you read through a file. Выполните команду dw , чтобы целиком удалить слово <i>are</i>

Команда **dw** удаляет слово, начинающееся с позиции, где установлен курсор. Обратите внимание, что пробел после слова также удаляется.

Команду **dw** также можно использовать для удаления части слова. Например, вы хотите удалить окончание *ed* у слова *allowed*:

```
since they allowed you to make
```

Команда	Результат
dw	since they allow you to make Команда dw удаляет оставшуюся часть слова, начиная с позиции курсора

Команда **dw** всегда удаляет пробел перед следующим словом в строке, но нам это сейчас не нужно. Чтобы сохранить пробел между словами, используйте команду **de**, которая стирает символы только до конца слова. Команда **dE** удаляет все символы до конца слова, включая знаки препинания¹.

¹ Роберт П. Дж. Дэй подчеркивает, что команды **sw** и **se**, в отличие от **dw** и **de**, делают одно и то же.

Вы также можете удалять слова в обратном направлении (**db**), до конца строки (**d\$**) или до ее начала (**d0**) от местоположения курсора.

Давайте проясним различие между командами **dw** и **dW**. Предположим, у вас есть следующий текст в файле:

```
This doesn't compute.
```

Курсор в начале строки отображает разницу между **dw** и **dW** следующим образом.

Команда	Результат
w	This █ oesn't compute. Перемещает курсор к букве <i>d</i>
dw	This █ t compute. Удаляет слово под курсором, вплоть до знаков препинания, но не включая их
u	This █ oesn't compute. Восстанавливает изначальный вид строки
dW	This █ ompute. Удаляет слово под курсором, вплоть до следующего пробела

Строки

Команда **dd** целиком удаляет строку, на которой находится курсор (но не ее часть). Как и команда **cc**, **dd** — это специальная команда. Взяв за основу предыдущий пример и расположив курсор на слове *are*, можно удалить первые две строки:

```
Screen editors █are very popular,  
since they allow you to make  
changes as you read through a file.
```

Команда	Результат
2dd	█ changes as you read through a file. Удаляет две строки (2dd). Обратите внимание, что команда стирает строку целиком, даже если курсор не был установлен в ее начале

Команда **D** удаляет символы от местоположения курсора до конца строки (**D** — это сокращение для **d\$**). Например, вы можете удалить часть строки под курсором и справа от него:

```
Screen editors are very popular,  
since they allow you to make  
changes █s you read through a file.
```

Команда	Результат
D	Screen editors are very popular, since they allow you to make changes█ Удаляет части строки под курсором и справа от него (D)

Символы

Бывает, что нужно удалить только один-два символа. **r** — это специальная команда для замены одного символа, а **x** — для удаления. **x** стирает только тот символ, на котором находится курсор. В строке ниже вы можете удалить букву **z**, введя команду **x¹**, в то время как **X** удаляет символ перед курсором:

█ You can move text by deleting text and then

Поставьте перед любой из этих команд число, чтобы удалить соответствующее количество символов. Например, команда **5x** удаляет пять символов, сдвигая при этом курсор вправо на пять позиций. После использования команд **x** и **X** вы остаетесь в командном режиме.

Проблемы при удалении

- *Вы удалили текст по ошибке и хотите вернуть его обратно.*

Существует несколько способов восстановления удаленного текста. Если вы только что что-то удалили и хотите сделать откат изменений, введите команду **u** (**undo**), чтобы отменить последнюю операцию (например, **dd**). Это сработает только в том случае, если вы не выполняли никаких дополнительных действий, поскольку **u** отменяет только самую последнюю команду. В то же время клавиша **U** позволяет восстановить строку в ее первоначальное состояние, в каком она была до того, как к ней были применены *какие-либо* изменения.

Однако вы все равно можете восстановить недавно удаленный текст с помощью команды **p**, поскольку **vi** хранит последние девять удалений в специальных пронумерованных регистрах. Если вы знаете, например, что вам нужно восстановить третье удаление, введите команду ниже, чтобы «поместить» содержимое соответствующего регистра в строку под курсором:

"Зр

¹ Запомнить действие команды **x** можно, представив ее как «исправление» ошибок на печатной машинке. Только кто сейчас пользуется печатными машинками?

Эта команда работает только для удаленной *строки*. Слова или часть строки не сохраняются в регистре. Если вы хотите восстановить удаленное слово или какой-то фрагмент, используйте команду `p`. Она восстановит все, что вы удалили за последний сеанс.

Обратите внимание, что программа Vim поддерживает «бесконечный» откат изменений, что значительно облегчает жизнь. Дополнительные сведения см. в разделе «Расширенный откат изменений» главы 8.



Команда `u` отменяет любую последнюю операцию. Удаление двух слов путем двойного ввода `dw` — это две операции: `u` восстанавливает только последнее удаленное слово. Однако удаление двух слов с помощью команды `2dw` — это одна операция; `u` в этом случае восстанавливает оба удаленных слова.

Перемещение текста

Каждый раз, когда вы удаляете текстовый фрагмент, он сохраняется в специальном безымянном *регистре удаления*¹. Содержимое регистра перезаписывается при каждом следующем удалении.

В программе `vi`, чтобы переместить текст, вы удаляете его, а затем помещаете в другое место в файле, что соответствует операции «вырезать и вставить». После удаления текста установите курсор на нужную позицию в вашем файле и используйте команду `p` (`put`), чтобы вставить этот текст туда. Вы можете перемещать любой фрагмент текста, хотя данная функция больше подходит для строк, чем для слов.

Команда `p` помещает текст, находящийся в регистре удаления, *после* курсора, а `P` — *перед* ним. Если вы удаляете одну или несколько строк, команда `p` помещает удаленный текст на новую строку (или строки) под курсором, тогда как команда `P` помещает текст на новую строку (или строки) над курсором. Если вы удаляете часть строки, `p` помещает удаленный текст в текущую строку после курсора.

Предположим, в вашем файле `practice` есть текст, представленный ниже, и вы хотите переместить вторую строку, то есть вырезать и вставить текст ниже третьей строки.

```
You can move text by deleting it and then,  
like a "cut and paste,"  
placing the deleted text elsewhere in the file.  
each time you delete a text block.
```

¹ В более старой документации `vi` это называется буфером удаления. Мы используем термин Vim «регистр», чтобы избежать путаницы с буферами, в которых хранится содержимое файла.

Используя команду `delete`, вы можете переместить текст следующим образом.

Команда	Результат
<code>dd</code>	You can move text by deleting it and then, placing the deleted text elsewhere in the file. each time you delete a text block. Удаляет строку, на которой находится курсор (в нашем случае это вторая строка). Текст помещается в регистр удаления
<code>p</code>	You can move text by deleting it and then, placing that deleted text elsewhere in the file. Like a "cut and paste" each time you delete a text block. Выполните команду <code>p</code>, чтобы восстановить удаленную строку на строке под курсором. Чтобы итоговый текст выглядел связанным и законченным, вам также потребуется исправить орфографические ошибки и расставить знаки препинания (с помощью команды <code>r</code>)



Удалив текст, необходимо восстановить его перед следующей командой изменения или удаления. Если вы внесете еще одно изменение, сохраняющее текст в регистре удаления, ваш ранее удаленный текст будет утерян. Вы можете повторять ввод снова и снова, до тех пор пока не закончите с правками. В разделе «Использование регистров» главы 4 вы узнаете, как сохранить удаленный текст в именovanном регистре, чтобы вернуться к нему позднее.

Перестановка двух букв

Вы можете использовать команду `xr` (удалить символ и поместить после курсора), чтобы переместить две буквы. Например, в слове *твое* буквы *т*, *о* перепутаны местами. Чтобы исправить это, наведите курсор на букву *т*, нажмите `x`, а затем `p`.

Для переноса слов команды не существует. В подразделе «Команда `tap`» в главе 7 рассматривается короткая последовательность действий, которая меняет местами два слова.

Копирование текста

Для экономии времени на редактировании (и нажатиях клавиш) можно скопировать фрагмент текста, чтобы продублировать его в других местах с помощью двух команд: `y` (`yank`) и `p` (`put`). Команда `y` помещает выделенный текст в регистр удаления, где он хранится до тех пор, пока не будет выполнено другое сохранение (или удаление). Затем вы можете поместить эту копию в другое место файла с помощью `p`.

Как и в случае с командами изменения и удаления, команда **у** может быть объединена с любой командой перемещения (**уw**, **у\$**, **у0**, **4уу**). Она чаще всего применяется к строке (или нескольким) текста, потому что удаление и вставка слова обычно занимают больше времени, чем его ввод.

Команда **уу** работает со всей строкой точно так же, как это делают команды **dd** и **сс**. Но **У** отличается от **D** и **C**. Вместо того чтобы удалять из текущей позиции в конец строки, **У** удаляет всю строку. То есть **У** делает то же самое, что и **уу** (используйте **у\$**, чтобы перейти от текущей позиции к концу строки).

Предположим, в вашем файле `practice` есть следующий текст:

```
With a screen editor you can
scroll the page.
move the cursor.
delete lines.
```

Вам нужно составить три полных предложения, начиная каждое с фразы *With a screen editor you can*. Вместо того чтобы перемещаться по файлу и вносить это изменение снова и снова, достаточно использовать команды **у** и **р**.

Команда	Результат
уу	With a █screen editor you can scroll the page. move the cursor. delete lines. Выделите строку текста, которую вы хотите скопировать в регистр. Курсор может находиться в любом месте строки, которую нужно скопировать (или на первой строке группы строк)
2j	With a screen editor you can scroll the page. move th█ cursor. delete lines. Переместите курсор туда, куда вы хотите поместить текст
Р	With a screen editor you can scroll the page. With a screen editor you can move the cursor. delete lines. Поместите скопированный текст над строкой курсора командой Р
jp	With a screen editor you can scroll the page. With a screen editor you can move the cursor. With a screen editor you can delete lines. Переместите курсор на строчку вниз. Затем поместите скопированный текст под строкой курсора с помощью команды р

Команда `u` использует тот же регистр, что и `d`. Каждое новое удаление или копирование заменяет предыдущее содержимое регистра. Как мы увидим в разделе «Использование регистров» главы 4, с помощью команды `p` можно вставить до девяти удаленных записей. Вы также можете копировать/удалять содержимое 26 именованных регистров, что позволяет вам манипулировать несколькими текстовыми фрагментами одновременно.

Повторение или отмена последней команды

Результат каждой команды редактирования сохраняется во временном регистре до тех пор, пока вы не выполните следующую команду. Например, если вы вставляете *the* после слова в свой файл, команда, используемая для ввода текста, вместе с введенным вами текстом временно сохраняется.

Повтор

Каждый раз, выполняя одну и ту же команду редактирования снова и снова, вы можете сэкономить время, продублировав ее с помощью команды повтора, вызываемой клавишей «точка» (`.`). Установите курсор туда, где вы хотите повторить команду, и нажмите клавишу «точка».

Предположим, в вашем файле есть только следующие строки:

```
With a screen editor you can
scroll the page.
With a screen editor you can
move the cursor.
```

Вы можете удалить одну строку, а затем, чтобы удалить другую, просто введите точку.

Команда	Результат
<code>dd</code>	With a screen editor you can scroll the page. Move the cursor. Удаляет строку с помощью команды <code>dd</code>
<code>.</code>	With a screen editor you can scroll the page. Повторяет предыдущую команду

Откат изменений

Как упоминалось ранее, вы можете отменить последнюю команду, если допустите ошибку. Просто введите команду `u`. Курсор не обязательно должен находиться в строке, где была произведена первоначальная правка.

Продолжаем пример выше с удалением строк в файле.

Команда	Результат
u	With a screen editor you can scroll the page. Move the cursor.
Команда u отменяет последнюю команду и восстанавливает удаленную строку	

В редакторе vi команда U отменяет все изменения в строке, в которой расположен курсор. Как только вы переместите его, вы больше не сможете использовать команду U. У редактора Vim этого ограничения нет.

Обратите внимание, что вы можете отменить свою последнюю отмену с помощью команды u, переключаясь между двумя версиями текста. Команда u также отменяет результат операции U, а команда U отменяет любые изменения в строке, в том числе внесенные с помощью клавиши u.

Если вы используете Vim, вполне вероятно, что команда u работает по-другому, постепенно возвращая тексту первоначальный вид. Vim позволяет использовать сочетание клавиш **Ctrl+R** для повтора отмененной операции. В сочетании с функцией бесконечного отката изменений вы можете перемещаться назад и вперед по истории изменений в вашем файле. Дополнительные сведения см. в разделе «Расширенный откат изменений» главы 8.



Тот факт, что команда u может отменять результат выполнения самой себя, дает возможность более эффективно перемещаться по файлу. Если вы когда-нибудь захотите вернуться к последней правке, просто отмените ее. Вы вернетесь к соответствующей строке. В свою очередь, отменив отмену, вы останетесь на этой строке.

Другие способы вставки текста

Допустим, вы вставили текст перед курсором в такой последовательности:

вставляемый текст Esc

Вы также вставили текст после курсора с помощью команды a. Ниже представлены другие команды ввода текста относительно положения курсора (некоторые из них рассматривались ранее):

- **A** — добавить текст в конец текущей строки;
- **I** — вставить текст в начало текущей строки;
- **o** (строчная буква «о») — открыть пустую строку под курсором для текста;

- **O** (заглавная буква «о») — открыть пустую строку над курсором для текста;
- **s** — удалить символ у курсора и заменить текст;
- **S** — удалить текущую строку и заменить текст;
- **R** — заменить существующие символы новыми символами, начиная от курсора.

Все эти команды переводят вас в режим ввода. После ввода текста не забудьте нажать клавишу **Esc**, чтобы вернуться в командный режим.

Команды **A** (**append**) и **I** (**insert**) избавляют вас от необходимости перемещать курсор в конец или начало строки перед вызовом режима ввода. (Команда **A** экономит одно нажатие клавиши в сравнении с **\$a**. Хотя одно нажатие и кажется не такой уж большой экономией, но чем более опытным (и нетерпеливым) редактором вы становитесь, тем больше нажатий клавиш вам захочется пропустить.)

Команды **o** и **O** (**open**) избавляют вас от необходимости перемещать курсор. Вы можете вводить эти команды из любого места в строке.

Команды **s** и **S** (**substitute**) позволяют удалить символ или целую строку и заменить удаление любым количеством нового текста. **s** — эквивалент двухтактной команды **cПробел**, а **S** аналогична **cc**. Одно из лучших применений **s** — это замена одного символа на несколько символов.

R («большая» замена) полезна, когда вы хотите начать перерабатывать текст, но не знаете точно, насколько существенно. Например, вместо того чтобы гадать, задавать ли команду **3cw** или **4cw**, введите **R**, а затем текст замены.

Числовые аргументы для команд ввода

За исключением **o** и **O**, недавно перечисленные команды ввода (плюс **i** и **a**) принимают числовые префиксы. С ними вы можете использовать команды **i**, **I**, **a** и **A** для добавления подчеркиваний и иных чередующихся символов. Например, при вводе **50i* Esc** вставляется 50 звездочек, а при вводе **25a*- Esc** — 50 символов (25 пар со звездочками и дефисами). Лучше повторять только небольшую строку символов.

С числовым префиксом команда **r** заменяет указанное количество символов повторяющимся экземпляром одного символа. Например, в языках **C** или **C++**, чтобы изменить **| |** на **&&**, вы должны поместить курсор на первый символ конвейера и ввести **2r&**.

Вы можете использовать числовой префикс с командой **S** для замены нескольких строк. Однако быстрее и проще применить команду **s** с командой перемещения.

Хорошим примером использования команды **s** с числовым префиксом является случай, когда вы хотите преобразовать несколько символов в середине слова. Команда **r** не подойдет, **cw** изменит слишком много текста, а вот **s** с числовым параметром работает примерно так же, как команда **R**.

Существуют и другие комбинации команд, которые естественным образом работают вместе. Например, `ea` добавляет новый текст в конец слова. Полезно научиться распознавать такие комбинации, чтобы быстрее их запомнить.

Объединение двух строк с помощью команды J

Иногда при редактировании файла вы получаете серию коротких строк, которые трудно отредактировать. Предположим, что ваш файл `practice` выглядит следующим образом:

```
With a
screen editor
you can
scroll the page, move the cursor
```

Если вы хотите объединить две строки в одну, поместите курсор в любом месте в первой строке и нажмите сочетание клавиш **Shift+J**.

Команда	Результат
J	With a█screen editor you can scroll the page, move the cursor J соединяет строку, на которой находится курсор, со строкой ниже
.	With a screen editor█you can scroll the page, move the cursor Повторяя последнюю команду (J) с помощью команды ., вы соедините следующую строку с текущей

Числовой аргумент с J объединяет указанное количество последовательных строк. И 1J, и 2J соединяют текущую строку со следующей за ней строкой. Числовой аргумент из трех или более объединяет указанное количество строк, включая ту, на которой находится курсор. В приведенном здесь примере можно соединить первые три строки с помощью команды 3J.

Проблемы с командами в vi

- Когда вы вводите команды, текст прыгает по экрану и ничего не работает.

Убедитесь, что вы вводите команду `j`, а не `J`.

Возможно, вы нажали клавишу **Caps Lock**, не заметив этого; `vi` и `Vim` чувствительны к регистру, то есть команды в верхнем регистре (например, `I`, `A`, `J`) отличаются от команд в нижнем регистре (`i`, `a`, `j`). Если вы нажмете в таком режиме клавишу, все ваши команды интерпретируются как прописные буквы,

а не строчные. Снова нажмите клавишу **Caps Lock**, чтобы вернуться к строчным буквам, нажмите клавишу **Esc**, чтобы убедиться, что вы находитесь в командном режиме, а затем введите либо **U**, чтобы восстановить последнюю измененную строку, либо **u**, чтобы отменить последнюю команду. Вероятно, вам также придется совершить некоторые дополнительные операции, чтобы полностью восстановить искаженную часть вашего файла.

Индикаторы режимов

Как вы уже знаете, у редактора есть два основных режима — командный режим и режим ввода. Обычно, глядя на экран, нельзя с ходу определить, какой режим активирован. Кроме того, как узнать, где именно в файле вы находитесь, без использования команды `ex :.` или сочетания клавиш **Ctrl+G**?

Это решается двумя способами: параметры `showmode` и `ruler`. Vim имеет и то и другое, в то время как в версиях vi Heirloom и Solaris `/usr/xpg7/bin` есть только `showmode` (табл. 2.1).

Таблица 2.1. Индикаторы режимов и положения

Редактор	С параметром <code>ruler</code> отображает	С параметром <code>showmode</code> отображает
vi	Нет	Отдельные индикаторы для режимов открытия, ввода, вставки, добавления, изменения, замены, замены одного символа
Vim	Строка и столбец	Визуальные индикаторы режима ввода и замены

Перечень основных команд vi

В табл. 2.2 представлены команды, которые можно комбинировать с командами `c`, `d` и `u` с различными текстовыми объектами. Последние две строки показывают дополнительные команды для редактирования. В табл. 2.3 и 2.4 перечислены другие основные команды. В табл. 2.5 приведены остальные команды, описанные в этой главе.

Таблица 2.2. Команды редактирования

Текстовый объект	Изменение	Удаление	Копирование
Одно слово	<code>cw</code>	<code>dw</code>	<code>yw</code>
Два слова с пробелом	<code>2cw</code> или <code>c2w</code>	<code>2dw</code> или <code>d2w</code>	<code>2yw</code> или <code>y2w</code>

Продолжение ➞

Таблица 2.2 (продолжение)

Текстовый объект	Изменение	Удаление	Копирование
Три слова назад	Зсb или сЗb	Зdb или dЗb	Зyb или yЗb
Одна строка	сс	dd	yy или Y
К концу строки	с\$ или С	d\$ или D	y\$
К началу строки	с0	d0	y0
Один символ	г	x или X	y1 или yh
Пять символов	5s	5x	5y1

Таблица 2.3. Команды перемещения

Перемещение	Команды
←, ↓, ↑, →	h, j, k, l
←, ↓, ↑, →	Backspace, Ctrl+N и Enter, Ctrl+P, Пробел
К первому символу следующей строки	+
К первому символу предыдущей строки	-
До конца слова	e или E
Вперед на одно слово	w или W
Назад на одно слово	b или B
До конца строки	\$
К началу строки	0 (ноль)
К определенной строке	G

Таблица 2.4. Другие действия

Действие	Команды
Добавить текст из регистра	р или Р
Запустить vi, открыть файл, если указан	vi файл
Запустить Vim, открыть файл, если указан	vim файл
Сохранить изменения, закрыть файл	ZZ
Не сохранять изменения, закрыть файл	:q! Enter

Таблица 2.5. Команды создания, удаления и обработки текста

Действие	Команда
Вставить текст в текущую позицию	i
Вставить текст в начало строки	I
Добавить текст в текущую позицию	a
Добавить текст в конец строки	A
Создать новую строку под курсором для нового текста	o
Создать новую строку над курсором для нового текста	O
Поместить удаленный текст после курсора или под текущей строкой	p
Поместить удаленный текст перед курсором или над текущей строкой	P
Заменить символ в поле курсора	r
Заменить существующие символы новым текстом	R
Удалить текущий символ и войти в режим ввода	s
Удалить строку и заменить текст	S
Удалить символ в поле курсора	x
Удалить символ перед курсором	X
Объединить текущую и следующую строки	J
Переключить регистр	~
Повторить последнее действие	.
Отменить последнее изменение	u
Восстановить строку в исходное состояние	U

В целом вы можете обойтись командами, перечисленными в этих таблицах. Однако, чтобы задействовать реальную мощь редактора (и повысить свою собственную производительность), вам понадобится больше инструментов. В следующих главах мы их разберем.

Эффективная навигация

Конечно, вы не только будете создавать новые файлы, но и потратите много времени на редактирование уже существующих. Просто открыть файл на первой строке и перемещаться по нему построчно — неэффективно. Иногда требуется быстро добраться до определенного места в тексте и начать работу оттуда.

Все изменения начинаются с позиции, куда вы поместили курсор и откуда вы хотите начать редактирование (или с места, определяемого с помощью команд редактора `ex` через номера строк, подлежащих редактированию). В текущей главе мы расскажем, как перемещаться по содержимому файла различными способами (по экранам, тексту, поисковым запросам (шаблонам) и номерам строк). Существует множество способов перемещения в `vi` и `Vim`, поскольку скорость редактирования зависит от того, как быстро вы можете добраться до нужного места в файле всего за несколько нажатий клавиш.

Здесь мы изучим:

- перемещение по экранам;
- перемещение по текстовым фрагментам;
- перемещение путем поиска;
- перемещение по номерам строк.

Перемещение по экранам

В книгах позиция обозначается номером страницы: страница, на которой вы остановились, или ее номер в индексе. При редактировании файлов такой роскоши нет. Некоторые файлы занимают всего несколько строк, и вы можете

окинуть весь файл одним взглядом. Но многие из них содержат сотни (или тысячи!) строк.

Представьте, что файл — это текст на длинном рулоне бумаги. Экран представляет собой окно из 24 (чаще всего) строк текста на этом рулоне¹.

В режиме ввода вы фактически набираете текст в нижней строке экрана. Когда вы дойдете до конца и нажмете клавишу **Enter**, верхняя строка исчезнет из поля зрения, а внизу экрана появится пустая строка для нового текста. Это называется *прокруткой* документа.

В командном режиме вы можете перемещаться по файлу, чтобы просмотреть весь текст, прокручивая экран вперед или назад. Поскольку перемещение курсора допускается задавать числовыми параметрами, можно быстро перемещаться в любое место вашего файла.

Прокрутка экрана

В *vi* доступно несколько команд для прокрутки вперед и назад по файлу на полном и половинном экранах:

- **^F** — прокрутка вперед на один экран;
- **^B** — прокрутка назад на один экран;
- **^D** — прокрутка вперед на половину экрана (вниз);
- **^U** — прокрутка назад на половину экрана (вверх).



В этом списке команд символ **^** представляет клавишу **Ctrl**. То есть команда **^F** подразумевает, что нужно удерживать нажатой клавишу **Ctrl** и одновременно нажимать сочетание клавиш **Shift+F**.

Существуют также команды для прокрутки экрана вверх на одну строку (**^E**) и вниз на одну строку (**^Y**). Однако эти две команды не затрагивают курсор. Он остается в той же точке строки, в которой он находился в момент выполнения команды.

¹ Редактор всегда подстраивается под размер вашего экрана, даже если во время редактирования вы задали размер окна эмулятора терминала.

Изменение положения экрана с помощью команды z

Если вы хотите прокрутить экран вверх или вниз, но чтобы курсор оставался в той строке, на которой вы его оставили, используйте команду z:

- **z Enter** и **z+ Enter** — переместить текущую строку в верхнюю часть экрана и прокрутить;
- **z .** — переместить текущую строку в центр экрана и прокрутить;
- **z -** — переместить текущую строку в нижнюю часть экрана и прокрутить.

Бесполезно добавлять числовой параметр для команды z (в конце концов, переместить курсор в верхнюю часть экрана можно только один раз, повторение одной и той же команды z ничего не изменит). В свою очередь, команда z понимает числовой параметр как номер строки, к которой необходимо переместиться. Например, **z Enter** перемещает текущую строку в верхнюю часть экрана, но **200z Enter** переходит к строке 200, отображая ее в верхней части экрана.



Некоторые дистрибутивы GNU/Linux поставляются с файлом `/etc/vimrc`, который устанавливает для Vim параметр `scrolloff` (смещение прокрутки) с ненулевым значением (обычно пять)¹. Другие используют файл `/usr/share/vim/vimXX/defaults.vim`, где XX — версия Vim. Параметр `scrolloff` в ненулевом значении приводит к тому, что Vim всегда предоставляет соответствующее количество строк контекста выше и ниже курсора. Таким образом, вы поймете, почему при вводе команды **z Enter**, чтобы поместить текущую строку в верхнюю часть экрана, она помещается всего на несколько строк ниже верха экрана.

Этот параметр также влияет на команды H и L (см. подраздел «Перемещение по видимой части экрана» далее в данной главе) и, возможно, на другие.

Вы можете отменить действие такого параметра по умолчанию, присвоив параметру `scrolloff` значение ноль в файле `.vimrc` на вашем компьютере (для получения дополнительной информации об этом файле см. разделы «Настройка vi и Vim» в главе 7 и «Системные и пользовательские конфигурационные файлы» в главе 8).

Перерисовка экрана

Если вы используете vi или Vim в окне терминала, во время редактирования на экране могут отображаться системные сообщения (такое может происходить, если вы вошли в систему удаленного сервера). Эти сообщения не становятся частью вашего буфера редактирования, но мешают работе. При их возникновении вам необходимо повторно обновить или перерисовать изображение на экране.

¹ Спасибо Роберту П. Дж. Дэю за то, что он заметил это в своей системе и рассказал нам.

Всякий раз при прокрутке вы обновляете часть (или весь) экрана, поэтому от нежелательных сообщений можно избавиться, прокрутив экран, а затем вернувшись в предыдущее положение. Вы также можете перерисовать экран без прокрутки, нажав **Ctrl+L**.

Перемещение по видимой части экрана

Перемещаться по экрану можно, используя следующие команды:

- **H** — команда «домой» — перейти к первому символу в начале верхней строки экрана;
- **M** — перейти к первому символу в начале средней строки экрана;
- **L** — перейти к первому символу в начале последней строки экрана;
- **nH** — перейти к первому символу в начале *n*-й строки, считая от верхней;
- **nL** — перейти к первому символу в начале *n*-й строки, считая от нижней.

Команда **H** перемещает курсор из любого места на экране на первую или «начальную» строку. Команда **M** перемещает курсор к средней строке, а **L** — к последней. Чтобы перейти к строке под первой, используйте команду **2H**.

Команда	Результат
L	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of your edits as you make them. Screen editors are very popular, since they allow you to make changes as you read through a file. Перейдите к началу последней строки экрана с помощью команды L
2H	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of your edits as you make them. Screen editors are very popular, since they allow you to make changes as you read through a file. Перейдите к началу второй строки экрана с помощью команды 2H. (Команда H перемещает курсор в верхнюю строку экрана)

Перемещение по строкам

На текущем экране также есть команды для перемещения по строкам.

Вы уже знакомы с командами `j` и `k`. Кроме того, вы можете использовать:

- `Enter` — переход к первому непустому символу следующей строки;
- `+` — переход к первому непустому символу следующей строки (то же, что и `Enter`);
- `-` — переход к первому непустому символу предыдущей строки.

Эти три команды перемещают курсор вниз или вверх до первого символа строки, игнорируя любые пробелы или табуляции. Команды `j` и `k`, напротив, перемещают курсор вниз или вверх до первой позиции строки, даже если эта позиция пуста (и предполагается, что курсор начался с первой позиции).

Перемещение по строке

Не забывайте, что команды `h` и `l` перемещают курсор влево и вправо, а команды `0` (ноль) и `$` — в начало и конец строки соответственно. Также можно использовать следующие команды:

- `^` — переход к первому непустому символу текущей строки;
- `n|` — переход к символу в столбце `n` текущей строки или в конец строки, если значение `n` больше количества символов в строке¹.

Как и в случае с командами перемещения строки, показанными ранее, `^` перемещает к первому символу строки, игнорируя любые пробелы и отступы; `0`, напротив, перемещает в первую позицию строки, даже если эта позиция пуста.

Перемещение по текстовым фрагментам

Перемещаться в файле `vi` также можно по текстовым фрагментам — словам, предложениям, абзацам или разделам. Вы уже научились перемещаться вперед и назад по слову (`w`, `W`, `b` или `B`). Кроме того, вы можете использовать следующие команды:

- `e` — перейти в конец текущего слова (слова, разделяемые знаками препинания и пробелами);
- `E` — перейти в конец текущего слова (слова, разделяемые пробелом);
- `(` — перейти к началу текущего предложения;

¹ Почему применяется `|`? Потому что символ `|` использовался в качестве команды перемещения `troff`.

- `)` — перейти к началу следующего предложения;
- `{` — перейти к началу текущего абзаца;
- `}` — перейти к началу следующего абзаца;
- `[[` — перейти к началу текущего раздела;
- `]]` — перейти к началу следующего раздела.

Чтобы найти конец предложения, `vi` и `Vim` ищут один из этих знаков препинания: `?`, `.` или `!`. `vi` определяет конец предложения, когда за знаком препинания следует по крайней мере два пробела или когда он появляется как последний непустой символ в строке. Если вы оставили только один пробел после точки или если предложение заканчивается кавычками, то `vi` не распознает предложение. Программа `Vim` не настолько старомодна и требует только один пробел после завершающего знака препинания.

Абзац определяется как текст до следующей пустой строки или до одного из макросов абзаца по умолчанию (`.IP`, `.PP`, `.LP` или `.QP`) из пакета макросов `troff`¹. Аналогично раздел определяется как текст вплоть до следующего макроса раздела по умолчанию (`.NH`, `.SH`, `.H1` или `.HU`). Макросы, которые распознаются как разделители абзацев или разделов, можно настроить с помощью команды `:set`, как описано в главе 7.

Напомним, что вы можете комбинировать числа с командами перемещения. Например, `3)` перемещает вперед на три предложения. Также вы можете редактировать текст с помощью команд перемещения: `d)` удаляет текст до конца текущего предложения, `2u}` копирует (помещает в буфер) два абзаца вперед.

Помните также, что вы можете использовать команды движения с командами редактирования, такими как `sw` и `se`. Роберт П. Дж. Дэй с интересом отмечает, что «команды `w` и `e` — это немного разные команды перемещения, в то время как команды изменения `sw` и `se` делают одно и то же».

Перемещение путем поиска

Одним из наиболее полезных способов быстрого перемещения по большому файлу является поиск фрагмента текста или, точнее, символов по *шаблону* (pattern). Иногда поиск используется, чтобы найти слово с ошибкой или переменную в программе.

Команда поиска — это специальный символ `/` (слеш). При вводе слеша он появляется в нижней строке экрана. Затем наберите текст, который хотите найти: `/шаблон`.

¹ Сейчас команда `troff` все еще используется, но уже не так часто.

В качестве запроса (его шаблона) может выступать как целое слово, так и любая другая последовательность символов (которая называется символьной строкой). Например, если вы ищете символы *red*, вы найдете как слово *red* целиком, так и слова, где эти символы встречаются, например *occurred*. Если вы добавляете пробел до или после *шаблона*, то он также учитывается. Для завершения команды необходимо нажать клавишу **Enter**.

Программы vi и Vim, как и все другие редакторы Unix, имеют специальный язык сопоставления шаблонов, который позволяет выполнять сложные текстовые запросы: например, любое слово, начинающееся с заглавной буквы, или слово *The* в начале строки. Мы поговорим об этом мощном синтаксисе в главе 6. На данный момент считайте шаблон словом или фразой.

Редактор начинает поиск с позиции курсора и движется вперед, а по достижении конца документа переходит к его началу. Курсор при этом переместится к первому совпадению с шаблоном поиска. Если такового отсутствует, в строке состояния отобразится сообщение *Pattern not found*¹.

На примере файла *practice* посмотрим, как перемещать курсор с помощью поиска.

Команда	Результат
<code>/edits Enter</code>	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of your edits as you make them. Поиск по шаблону <code>edits</code> . Курсор переместится непосредственно к соответствующему слову. Обратите внимание, что не нужно вводить пробел после запроса перед нажатием клавиши Enter
<code>/scr Enter</code>	With a s creen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of your edits as you make them. Поиск по шаблону <code>scr</code> . Поиск завершается в начале файла

Обратите внимание, что вы можете ввести любую комбинацию символов, а не только слово целиком.

Чтобы искать в обратном направлении, введите `? вместо /`:

?шаблон

В обоих случаях при необходимости поиск охватывает начало или конец файла.

¹ Сообщения могут различаться в зависимости от версии редактора, но значение у них одно. Мы не будем отмечать, что текст сообщения может быть разным; во всех случаях смысл передаваемой информации один и тот же.

Повторный поиск

Последний шаблон, который использовался для поиска, остается доступным на протяжении всего сеанса редактирования. После выполнения поиска вместо того, чтобы повторно нажимать клавиши для данной операции, вы можете использовать следующие команды `vi`:

- `n` — повторить поиск в том же направлении;
- `N` — повторить поиск в противоположном направлении;
- `/Enter` — повторить поиск вперед;
- `?Enter` — повторить поиск назад.

Поскольку последний шаблон остается доступным, вы можете найти совпадения, поработать с ними, а затем снова осуществить поиск по тому же шаблону, используя команды `n`, `N`, `/` или `?`. Направление поиска (`/` — вперед, `?` — назад) отображается в левом нижнем углу экрана. Программа Vim предоставляет более широкие возможности, чем `vi`. Она помещает текст поиска в командную строку и позволяет прокручивать сохраненную историю команд поиска с помощью клавиш со стрелками `↑`, `↓`. В пункте «Знакомство с окнами истории» подраздела «Удвойте удовольствие» в главе 14 обсуждается, как в полной мере использовать сохраненную историю поисковых запросов.

Чтобы продолжить работу с предыдущим примером, когда шаблон `scr` еще доступен для поиска, вы можете проделать следующее.

Команда	Результат
<code>n</code>	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of your edits as you make them. Перейдите к следующему вхождению шаблона (от слова <code>screen</code> к <code>scrol</code>) с помощью команды <code>n</code> (<code>next</code>)
<code>?you Enter</code>	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of your edits as you make them. Выполните поиск в обратном направлении от курсора до первого появления слова <code>you</code> с помощью символа <code>?</code>. После ввода запроса нужно нажать клавишу <code>Enter</code>
<code>N</code>	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of your edits as you make them. Повторите предыдущий поиск слова <code>you</code>, но в противоположном направлении (вперед)

Иногда нужно найти слово, которое находится впереди от текущего положения курсора, без перехода в начало файла. Для этого существует параметр `wrapscan`, который определяет, когда завершается поиск. Вы можете ограничить поиск следующим образом:

```
:set nowrapscan
```

Когда задан параметр `nowrapscan` и прямой поиск завершается неудачей, в строке состояния vi отображается следующее сообщение:

```
Address search hit BOTTOM without matching pattern
```

В Vim оно выглядит так:

```
E385: search hit BOTTOM without match for: foo
```

Когда параметр `nowrapscan` установлен и обратный поиск завершается неудачей, в сообщениях отображается слово *TOP* вместо *BOTTOM*.

Редактирование с помощью функции поиска

Вы можете комбинировать символы / и ? с командами, которые изменяют текст, такими как `c` и `d`. В продолжение к предыдущему примеру...

Команда	Результат
<code>d?move Enter</code>	With a screen editor you can scroll the page, Y our edits as you make them. Удаляет символы перед курсором до слова <i>move</i>

Обратите внимание, что удаление происходит посимвольно, а не целыми строками.

В этом разделе приводится лишь базовая информация о поиске по шаблону. В главе 6 вы узнаете больше о шаблонах и их использовании при внесении крупных изменений в файл.

Поиск в текущей строке

Существуют также сокращенные версии команд поиска, выполняемых в пределах текущей строки. Команда `fx` перемещает курсор к следующему экземпляру символа *x* (где *x* — это любой символ). Команда `tx` перемещает курсор на символ *перед* следующим экземпляром *x*. (`f` — сокращенно от `find`; `t` — сокращенно от `to`, что

означает в данном случае «до».) Затем можно повторить поиск, нажав клавишу ; (точка с запятой).

Команды поиска внутри строки кратко описаны ниже. Ни одна из них не перемещает курсор на следующую строку:

- **fx** — найти (переместить курсор к) следующее вхождение *x* в строке, где *x* — любой символ;
- **Fx** — найти (переместить курсор к) предыдущее вхождение *x* в строке;
- **tx** — найти (переместить курсор к) символ *перед* следующим вхождением *x* в строке;
- **Tx** — найти (переместить курсор к) символ *после* предыдущего вхождения *x* в строке;
- **;** — повторить предыдущую команду поиска в том же направлении;
- **,** (запятая) — повторить предыдущую команду поиска в противоположном направлении.

Добавив к любой из этих команд числовой параметр *n*, вы сможете определить местонахождение *n*-го символа. Предположим, вы редактируете файл `practice` в этой строке:

With a screen editor you can scroll the

Есть несколько вариантов найти вхождения буквы *o*.

Команда	Результат
fo	With a screen edito <u>r</u> you can scroll the Найдите первое упоминание буквы <i>o</i> в текущей строке с помощью команды f
;	With a screen editor y <u>o</u> u can scroll the Перейдите к следующему вхождению буквы <i>o</i> с помощью команды ; (найдите следующую букву <i>o</i>)

Команда **dfx** удаляет все символы до именованного символа *x* включительно. Эта команда полезна при удалении или копировании фрагментов строк. Возможно, вам придется использовать команду **dfx** вместо **dw**, если в строке есть символы или знаки препинания, которые затрудняют подсчет слов. Команда **t** работает так же, как и **f**, за исключением того, что она помещает курсор перед искомым символом. Например, команда **ct.** может использоваться для изменения текста до конца предложения, оставляя точку.

Перемещение по номерам строк

Строки в файле нумеруются последовательно, и вы можете перемещаться по файлу, указывая их номера.

Номера строк помогают определить начало и конец больших фрагментов текста. Они также полезны программистам, поскольку сообщения об ошибках компилятора ссылаются на номера строк. Наконец, они используются командами редактора `ex`, о которых вы узнаете в следующих главах.

Если вы собираетесь перемещаться по номерам строк, у вас должен быть способ их идентификации. Они отображаются на экране с помощью параметра `:set nu`, описанного в главе 5. Вы также можете отобразить номер текущей строки внизу экрана.

Команда `Ctrl+G` в программе `vi` отображает в нижней части экрана текущий номер строки, общее количество строк в файле и какой процент от общего числа представляет текущий номер строки. Например, для файла `practice` команда `Ctrl+G` выдает следующее:

```
"practice" line 3 of 6 --50%--
```

Результат этой команды в Vim выглядит так:

```
"practice" 4 lines --75%--           3,23           All
```

Предпоследнее поле — это позиция курсора (строка 3, символ 23). В больших файлах последнее поле указывает, насколько далеко вы продвинулись в файле в процентном соотношении.

Комбинация `Ctrl+G` поможет найти номер строки, необходимый для выполнения команды, а также сориентироваться в тексте, если вы отвлеклись от редактирования.

Если вы изменили файл, но еще не успели записать его, в строке состояния после имени файла можете увидеть надпись `[Modified]`.

Команда G (перейти к)

Номера строк могут использоваться для перемещения курсора по файлу. Команда `G` (перейти к) принимает номер строки в качестве числового аргумента и переходит непосредственно к этой строке. Например, команда `44G` перемещает курсор в начало строки 44. А `G` без номера строки переместит курсор на последнюю строку файла.

Две обратные кавычки (``) возвратят вас в исходное положение (в котором была выполнена последняя команда G), если вы не внесли каких-либо изменений за это время. Если же правки были осуществлены, а курсор перемещен с помощью какой-либо команды, отличной от G, то команда `` вернет курсор в позицию последнего редактирования. Если вы ввели команду поиска (/ или ?), то `` вернет курсор туда, откуда вы начали операцию поиска. Два апострофа (') работают так же, как две обратные кавычки, за исключением того, что они возвращают курсор в начало строки, а не точно туда, где он находился.

Общее количество строк, отображаемых с помощью сочетания клавиш **Ctrl+G**, может дать вам приблизительное представление о том, сколько строк нужно переместить. Если вы находитесь на десятой строке 1000-строчного файла:

```
"practice" 1000 lines --1%-- 10,1 1%
```

и знаете, что хотите начать редактирование ближе к концу этого файла, вы можете приблизительно указать пункт назначения с помощью команды **800G**.

Перемещение по номерам строк — это инструмент, который поможет быстро перемещаться по большому файлу.

Перечень команд перемещения в vi

В табл. 3.1 приведены все команды, описанные в этой главе.

Таблица 3.1. Команды перемещения

Перемещение	Команда
Прокрутить вперед на один экран	^F
Прокрутить назад на один экран	^B
Прокрутить вперед на половину экрана	^D
Прокрутить назад на половину экрана	^U
Прокрутить вперед на одну строку	^E
Прокрутить назад на одну строку	^Y
Отобразить текущую строку в верхней части экрана и прокрутить	z Enter
Отобразить текущую строку в центре экрана и прокрутить	z .
Отобразить текущую строку в нижней части экрана и прокрутить	z -

Продолжение ➞

Таблица 3.1 (продолжение)

Перемещение	Команда
Перерисовать (обновить) экран	^L
Перейти к верхней строке экрана	H
Перейти к средней строке экрана	M
Перейти к нижней строке экрана	L
Перейти к первому символу следующей строки	Enter
Перейти к первому символу следующей строки	+
Перейти к первому символу предыдущей строки	-
Перейти к первому непустому символу текущей строки	^
Перейти к столбцу <i>n</i> текущей строки	<i>n</i>
Перейти к концу слова	e
Перейти к концу слова (игнорируя знаки препинания)	E
Перейти к началу текущего предложения	(
Перейти к началу следующего предложения	)
Перейти к началу текущего абзаца	{
Перейти к началу следующего абзаца	}
Перейти к началу текущего раздела	[[
Перейти к началу следующего раздела	]]
Поиск вперед по запросу	/шаблон Enter
Выполнить поиск по запросу в обратном направлении	?шаблон Enter
Повторить последний поиск	n
Повторить последний поиск в противоположном направлении	N
Повторить последний поиск вперед	/
Повторить последний поиск в обратном направлении	?
Перейти к следующему упоминанию <i>x</i> в текущей строке	f <i>x</i>
Перейти к предыдущему упоминанию <i>x</i> в текущей строке	F <i>x</i>
Переместиться прямо перед следующим упоминанием <i>x</i> в текущей строке	t <i>x</i>
Переместиться сразу после предыдущего упоминания <i>x</i> в текущей строке	T <i>x</i>
Повторить предыдущую команду поиска в том же направлении	;

Перемещение	Команда
Повторить предыдущую команду поиска в противоположном направлении	, (запятая)
Перейти к заданной строке <i>n</i>	<i>n</i> G
Перейти к концу файла	G
Вернуться к предыдущей метке или контексту	` `
Вернуться к началу строки, содержащей предыдущую метку	' '
Показать текущую строку (не команда перемещения)	^G

ГЛАВА 4

За пределами основ

Вы уже познакомились с основными командами редактирования: `i`, `a`, `c`, `d` и `y`. В этой главе подробно разбираются операции редактирования и функции, поверхностно рассмотренные ранее.

- Описание дополнительных возможностей редактирования с обзором основных команд.
- Консольные возможности программ `vi` и `Vim`, включая различные способы открытия файлов для редактирования.
- Регистры памяти, в которых хранятся вносимые в документ правки.
- Отметка позиций в файле.
- Другие дополнительные операции правки.

Дополнительные комбинации команд

В главе 2 вы узнали о командах редактирования `c`, `d` и `y`, а также о том, как комбинировать их с командами перемещения и числами (например, `2cw` или `4dd`). В главе 3 мы рассмотрели множество команд перемещения. Как выяснилось, вы можете комбинировать команды редактирования с командами перемещения (навигации). В табл. 4.1 представлены несколько дополнительных опций редактирования, которых вы раньше не видели.

Таблица 4.1. Дополнительные опции редактирования

Изменить	Удалить	Скопировать	От курсора к...
<code>cH</code>	<code>dH</code>	<code>yH</code>	Верхней части экрана
<code>cL</code>	<code>dL</code>	<code>yL</code>	Нижней части экрана

Изменить	Удалить	Скопировать	От курсора к...
c+	d+	y+	Следующей строке
c5	d5	y5	Столбцу 5 текущей строки
2c)	2d)	2y)	Второму предложению
c{	d{	y{	Предыдущему абзацу
c/запрос	d/запрос	y/запрос	Запросу
cn	dn	yn	Следующему запросу
cG	dG	yG	Концу файла
c13G	d13G	y13G	Номеру строки 13

Обратите внимание, что все последовательности в табл. 4.1 соответствуют одному из двух общих синтаксисов:

(команда)(число)(текстовый объект)

или:

(число)(команда)(текстовый объект)

Число — это необязательный числовой аргумент. *Команда* в этом случае является одной из c, d или y. *Текстовый объект* — это команда перемещения.

Общий синтаксис команд vi рассматривался в главе 2. Вы также можете вернуться к табл. 2.2 и 2.3.

Параметры загрузки vi и Vim

Ранее мы вызывали редактор из командной строки с помощью команды:

```
$ vi файл
```

или

```
$ vim файл
```

Существуют и другие полезные параметры команды vim. Вы можете открыть файл на определенной странице (по номеру) или с учетом результатов поискового запроса. Допускается также открыть файл в режиме чтения. Еще один параметр восстанавливает все изменения в файле, которые вы внесли до сбоя системы.

Параметры, описанные в следующем разделе, применимы как к vi, так и к Vim.

Перемещение в определенное место

Редактируя файл, вы можете открыть его, а затем сразу перейти к первому совпадению поискового запроса или к определенной строке согласно ее номеру. Вы также можете указать направление первого перемещения поиска или номера строки прямо в командной строке. Это выполняется с помощью параметра `-с команда` (для обратной совместимости с более ранними версиями `vi` вы также можете использовать команду `+команда`):

- `$ vim -с n файл` — открывает *файл* на строке *n*;
- `$ vim -с /запрос файл` — открывает *файл* на первом совпадении *запроса*;
- `$ vim + файл` — открывает *файл* на последней строке.

Чтобы открыть файл `practice` и перейти непосредственно к строке, содержащей слово `Screen`, введите следующее.

Команда	Результат
<code>\$ vim -с /Screen practice</code>	With a screen editor you can scroll the page, move the cursor, delete lines, and insert characters, while seeing the results of your edits as you make them. Screen editors are very popular, since they allow you to make changes as you read Введите команду <code>vim</code> с параметром <code>-с /запрос</code>, чтобы перейти непосредственно к строке, содержащей слово <code>Screen</code>

Как видно из примера, результат поиска не обязательно расположен в верхней части экрана. Интересно, что курсор помещается на первый символ строки, а не на первый символ совпадающего текста! Если вы добавляете пробелы в запрос, вы должны заключить весь запрос в одинарные или двойные кавычки¹:

```
-с /"you make"
```

или уберите пробел с помощью обратного слеша:

```
-с /you\ make
```

Кроме того, если вы хотите использовать общий синтаксис запросов, описанный в главе 6, вам может потребоваться экранировать специальные символы от интерпретации оболочкой с помощью одинарных кавычек или обратного слеша.

¹ Это требование оболочки, не программы.

Команда `-с /запрос` полезна, если вы вынуждены закончить сеанс редактирования до завершения работы. Вы можете пометить текущую позицию, вставив текст в духе `ZZZ` или `ОСТАНОВКА`. Затем, когда вы вернетесь к работе над файлом, вам понадобится лишь набрать команду `/ZZZ` или `/ОСТАНОВКА`.

После того как в редакторе откроется ваш файл и выполнит поиск по запросу, который вы ему задали с помощью параметра `-с`, вы можете перейти к следующему совпадению, используя команду `n`.



Обычно при редактировании файлов в `vi` и `Vim` параметр `wrapscan` включен. Если вы настроили свою среду таким образом, что `wrapscan` всегда отключен (см. подраздел «Повторный поиск» в главе 3), возможно, вы не сможете использовать команду `-с /запрос`. Если вы попытаетесь открыть файл таким образом, редактор откроет файл на последней строке и отобразит сообщение `Address search hit BOTTOM without matching pattern` (оно может отличаться в зависимости от версии `vi` и `Vim`).

Режим чтения

Существует способ просмотреть файл, защитив его от непреднамеренных изменений. (Возможно, вам захочется открыть объемный файл, чтобы попрактиковаться в навигации по нему, или вы хотите просмотреть служебный файл или исходный код программы.) Вы можете открыть файл в режиме чтения и использовать все обычные команды навигации, без возможности изменить файл. Чтобы просмотреть файл в данном режиме, введите либо:

```
$ vim -R файл
```

либо

```
$ view файл
```

(Команда `view` может включать любой из параметров командной строки для перехода к определенной позиции в файле¹.)

Если вам все же потребуется внести некоторые изменения в файл, то переопределите режим чтения, добавив восклицательный знак к команде записи:

```
:w!
```

или

```
:wq!
```

¹ Обычно команда `view` — это просто ссылка на программу `vi`. В некоторых системах команда `view` выполняет команду `vim -R`.

Обратите внимание, что, если вы редактируете файл, разрешения на запись которого у вас нет, вы также откроете его только в режиме чтения. В этом случае, если файл принадлежит вам, команды `:w!` или `:wq!` все равно будут работать; `vi` временно изменяет права доступа к файлу, чтобы вы могли его записать. В противном случае файл нельзя будет сохранить.

Если у вас возникли проблемы с записью файла, обратитесь к списку проблем в подразделе «Проблемы при сохранении файлов» в главе 1.

Восстановление содержимого из буфера

Иногда во время редактирования файла может произойти сбой системы. Обычно в этом случае все изменения, внесенные после вашей последней записи (сохранения), теряются. Однако существует параметр `-r`, который позволяет восстановить внесенные вами изменения текста из буфера в том виде, в каком он был во время сбоя системы.

Восстановление в vi

В классической системе Unix с оригинальным приложением `vi` при первом входе в систему после перезагрузки вы получите сообщение о том, что ваш буфер сохранен. Кроме того, если вы введете команду, показанную ниже, вы увидите список всех файлов, которые система сохранила:

```
$ ex -r
```

или

```
$ vi -r
```

Используйте параметр `-r` с именем файла, чтобы восстановить внесенные вами изменения текста из буфера. Например, чтобы восстановить отредактированное содержимое файла `practice` из буфера после сбоя системы, введите:

```
$ vi -r practice
```

Разумно немедленно восстановить файл после загрузки системы, чтобы вы случайно не внесли изменения в файл, а затем не пришлось устранять конфликты версий между содержимым буфера и заново отредактированным файлом.

Вы можете сохранить содержимое буфера, не дожидаясь сбоя, используя команду `:pre` (сокращение от `:preserve`). Она полезна, если вы внесли изменения, а затем обнаружили, что не можете их сохранить, потому что у вас нет разрешения на запись. (Допускается записать копию файла под другим именем или в каталог,

в котором у вас есть разрешение на запись. См. подраздел «Проблемы при сохранении файлов» в главе 1.)

Восстановление в Vim

Восстановление содержимого из буфера в программе Vim выполняется несколько иначе. Vim обычно хранит свой рабочий файл (называемый *файлом подкачки*) в том же каталоге, что и редактируемый файл. Для файла `practice` файл подкачки будет называться `.practice.swp`.

И если этот файл существует, когда вы в следующий раз откроете `practice`, в Vim появится запрос, хотите ли вы восстановить документ. Восстановите и пересохраните файл. Затем вы должны *немедленно выйти* и вручную удалить файл `.practice.swp`; Vim не сделает это автоматически. После этого вы можете вернуться в Vim и продолжить редактирование в обычном режиме.

Параметр `directory` команды `:set` позволяет управлять тем, куда Vim помещает файл подкачки. Для получения дополнительной информации см. табл. Б.2 в разделе «Параметры Vim 8.2» приложения Б.

Использование регистров

Во время редактирования документа каждый последний удаленный (`d` или `x`) или вырезанный (`y`) фрагмент текста сохраняется в безымянном регистре. Вы можете получить доступ к содержимому этого регистра и вернуть сохраненный текст в свой файл с помощью команды `p` или `P`.

Последние девять удаленных или вырезанных фрагментов текста сохраняются в нумерованных регистрах. Вы можете получить доступ к каждому из них, чтобы восстановить любую (или все) из последних девяти операций. (Однако небольшие удаленные фрагменты текста, части строк не сохраняются. Они могут быть восстановлены только с помощью команды `p` или `P` сразу после удаления.)

Вы также можете поместить извлеченный (скопированный) текст в регистры, обозначенные буквами. Вам доступно 26 регистров (с именами от `a` до `z`) для вырезанного текста, а чтобы восстановить его, воспользуйтесь командой `put` в любой момент.

Восстановление удаленного текста

Возможность удалять большие фрагменты текста одним махом — это очень хорошо, но как быть, если вы по ошибке удалите 53 строки? Вы можете восстановить любой из девяти последних удаленных фрагментов текста, поскольку они сохраняются

в нумерованных регистрах. Последний удаленный фрагмент текста сохраняется в регистре 1, предпоследний — в регистре 2 и т. д.

Чтобы восстановить удаленное, введите " (двойная кавычка), идентифицируйте текст по номеру, а затем введите команду `put`. Чтобы восстановить предпоследний удаленный фрагмент текста (из регистра 2), введите:

```
"2p
```

Текст из регистра 2 вставляется после курсора.

Если вы не знаете, в каком регистре содержится удаленный фрагмент, который вы хотите восстановить, вам не нужно снова и снова вводить команду `"np`. Достаточно набрать `"1p` для вставки первого удаленного фрагмента текста, затем, если он вставлен неправильно, используйте команду `u`, чтобы отменить действие. Затем вы можете использовать команду повтора `(.)`, чтобы поместить следующий фрагмент — `u`, чтобы отменить вставку и т. д. После этого программа автоматически увеличит номер регистра на единицу. Так вы можете искать нужный фрагмент текста по нумерованным регистрам, чтобы поместить содержимое каждого последующего регистра в файл один за другим, с помощью команды:

```
"1ru.u.u          u т.п.
```

Каждый раз, когда вы выполняете команду `u`, извлеченный из буфера текст удаляется, а когда вы вводите точку `(.)`, содержимое следующего регистра появляется в вашем файле. Продолжайте выполнять команды `u` и `.` до тех пор, пока вы не восстановите искомый текст.

Работа с именованными регистрами

Имейте в виду, что фрагмент текста из безымянного регистра необходимо вставить (`r` или `P`) в документ, прежде чем вносить какие-либо другие изменения, иначе регистр будет перезаписан. Для этого также подойдут команды `u` и `d` с символами 26 именованных регистров (от `a` до `z`), которые используются для копирования и перемещения текста. Если вы присвоите регистру имя для хранения удаленного текста, вы можете получить его содержимое в любое время во время сеанса редактирования.

Чтобы перейти в именованный регистр, введите перед командой перехода двойную кавычку `"` и имя регистра, который вы хотите использовать. Например:

```
"duy      Помещает текущую строку в регистр d  
"a7uy     Помещает следующие семь строк в регистр a
```

После загрузки содержимого в именованные регистры и перемещения в новую позицию документа используйте команду `r` или `R`, чтобы вставить текст:

<code>"dP</code>	<i>Помещает содержимое регистра <code>d</code> перед курсором</i>
<code>"aP</code>	<i>Помещает содержимое регистра <code>a</code> после курсора</i>

В документ можно поместить только все содержимое регистра, а не отдельную его часть.

В следующей главе вы узнаете, как редактировать сразу несколько файлов. Узнав способы переключения между файлами, не выходя из редактора, вы сможете использовать именованные регистры для выборочной передачи текста между файлами. При использовании многооконной функции Vim вам также станет доступен безымянный регистр для передачи данных между файлами.



Безымянный и именованный регистры буфера работают в рамках одного сеанса Vim. Это позволяет легко копировать и вставлять текст между файлами, которые вы редактируете в нескольких окнах во время одной рабочей сессии. Но эти буферы недоступны между различными сеансами в Vim! (К примеру, если у вас одновременно запущена программа `gvim` с другими открытыми файлами.) При этом `gvim` имеет доступ к системному буферу обмена так же, как и любое другое GUI-приложение. Таким образом, вы без проблем можете и копировать, и вставлять текст между файлами, используя средства операционной системы, а не Vim.

Вы также можете поместить удаленный текст в именованные регистры, используя практически те же команды:

`"a5dd` *Вырезает пять строк и помещает их в регистр `a`*

Если вы укажете имя регистра с прописной буквы, ваш извлеченный или удаленный текст *будет добавлен* к текущему содержимому соответствующего регистра в нижнем регистре. Так вы можете компоновать помещаемый в регистр текст из разных источников. Например:

- `"zd`) — удалить текст от курсора до конца текущего предложения и сохранить текст в регистре `z`;
- `2)` — переместиться на два предложения вперед;
- `"Zy`) — добавить следующее предложение в регистр `z`.

Вы можете продолжать добавлять дополнительный текст в именованный регистр столько, сколько захотите, но имейте в виду: если при помещении в буфер текста вы забудете указать идентификатор регистра с прописной буквы, вы перезапишете его, потеряв все содержимое.

Маркеры

В процессе редактирования вы можете пометить позицию в документе невидимой «закладкой», внести изменения в другом месте, а затем вернуться к отмеченной позиции. Зачем вам это нужно? По словам Уилла Гальего:

«Больше всего я люблю маркировать позиции в документе, выполняя удаление/извлечение/изменение больших фрагментов текста. Например, допустим, мне нужно удалить большое количество строк. Я не хочу считать все эти строки, а затем выполнять команду `номерpdd`, удобнее будет перейти к нижней части, пометив ее маркером с помощью команды типа `ma` (где буква `a` играет роль маркера позиции), затем перемещусь в позицию, откуда хочу начать удаление, и введу команду `d`a`, чтобы удалить текущую строку и все строки до маркера `a`. Команду `yy` и другие похожие команды можно использовать похожим образом».

Вот как можно установить маркеры позиции в командном режиме:

- `mx` — отметить текущую позицию символом `x` (вместо `x` можно использовать любую букву; `vi` допускает использование только строчных букв, `Vim` допускает и различает прописные и строчные буквы);
- `'x` (апостроф) — переместить курсор на первый символ строки, отмеченной символом `x`;
- ``x` (обратная кавычка) — переместить курсор на символ, отмеченный символом `x`;
- ```` (обратные кавычки) — вернуться к точному положению предыдущей метки или контекста после перемещения;
- `''` (апострофы) — вернуться к началу строки предыдущей метки или контекста.



Маркеры позиции устанавливаются только во время текущего сеанса, они не сохраняются в файле.

Другие расширенные возможности редактирования

Существуют и другие дополнительные функции редактирования, которые вы можете выполнить в `vi` и `Vim`, но, чтобы использовать их, вы должны сначала узнать чуть больше о редакторе `ex`, прочитав следующую главу.

Перечень регистров и маркировки

В табл. 4.2 приведены параметры командной строки, общие для всех версий программы *vi*. В табл. 4.3 и 4.4 обобщены команды регистров и маркировки.

Таблица 4.2. Параметры команд

Параметры	Значение
-с <i>n</i> файл	Открыть <i>файл</i> на строке <i>n</i> (стандартная POSIX-версия)
+ <i>n</i> файл	Открыть <i>файл</i> на строке <i>n</i> (традиционная версия <i>vi</i>)
+ файл	Открыть <i>файл</i> на последней строке
-с /запрос файл	Открыть <i>файл</i> на первом совпадении запроса (стандартная POSIX-версия)
+ /запрос файл	Открыть <i>файл</i> на первом совпадении запроса (традиционная версия <i>vi</i>)
-с команда файл	Выполнение <i>команды</i> после открытия <i>файла</i> ; обычно это номер строки или поисковый запрос
-r	Восстановление файлов после сбоя
-R	Работа в режиме чтения (аналогично команде <i>view</i>)

Таблица 4.3. Названия регистров

Имена регистров	Использование
1–9	Нумерованные регистры: последние девять удалений, от наиболее до наименее недавних
a–z	Именованные регистры от <i>a</i> до <i>z</i> , которые вы можете использовать по мере необходимости

Таблица 4.4. Команды регистров и маркирования

Команда	Значение
" <i>b</i> команда	Выполнить <i>команду</i> с регистром <i>b</i>
mx	Отметить текущую позицию символом <i>x</i>
' <i>x</i>	Переместить курсор на первый символ строки, отмеченной <i>x</i>
` <i>x</i>	Переместить курсор на символ, отмеченный <i>x</i>
``	Возврат к точному положению предыдущей метки или контекста
''	Возврат к началу строки предыдущей метки или контекста

ГЛАВА 5

Знакомство с редактором ex

Если эта книга посвящена редакторам `vi` и `Vim`, то зачем тогда включать в нее главу о другом редакторе? В общем-то, `ex` на самом деле — это не очередной редактор. `vi` — это визуальный режим более общего, базового линейного редактора, которым и является `ex`. Некоторые команды `ex` пригодятся вам во время работы с `vi`, поскольку они могут значительно сэкономить время редактирования. Большую их часть можно использовать, даже не покидая `vi`: воспринимайте командную строку как третий режим, наряду с командным и режимом ввода. Различные команды перемещения и изменения текста `vi`, с которыми вы познакомились в предыдущих главах, хороши, но, если это все, что вам нужно, вы можете с тем же успехом пользоваться Блокнотом или чем-то подобным. Причина, по которой `vi` обожают, — `ex`: именно в `ex` и заключена вся мощь!



`Vim` предоставляет базовый редактор `ex`, но с многочисленными усовершенствованиями по сравнению с исходной его версией. В системах, где `vi` — это `Vim`, `ex` обычно вызывает `Vim` в режиме `ex`.

В текущей и последующих главах первой части мы не проводим большого различия между `vi` и `Vim`, так как содержание этих глав применимо к обоим приложениям. Во время чтения вы можете спокойно воспринимать `vi` как «`vi` и `Vim`».

Вы уже знаете, как воспринимать файлы в качестве последовательности пронумерованных строк. `ex` предоставляет вам команды для редактирования с большей мобильностью и областью действия. С `ex` вы способны легко перемещаться между файлами и перемещать текст из одного файла в другой различными способами. Вы можете быстро отредактировать фрагменты текста, превышающего по объему один экран. А с помощью глобальной замены разрешается осуществлять замещения во всем файле в соответствии с заданным запросом.

Эта глава знакомит с `ex` и его командами. Вы научитесь:

- перемещаться по файлу, используя номера строк;
- использовать команды `ex` для копирования, перемещения и удаления фрагментов текста;

- сохранять файлы и фрагменты файлов;
- работать с несколькими файлами (читать тексты и команды, перемещаться между файлами).

Команды `ex`

Задолго до появления `vi` и других экранных редакторов люди взаимодействовали с компьютерами посредством печатающих терминалов, а не через современные растровые экраны с манипуляторами и программами эмуляции терминала. Номера строк были способом быстрого определения части файла, над которой нужно работать, а линейные редакторы эволюционировали для правки этих файлов. Программист или другой пользователь компьютера, как правило, выводил строку (или строки) на печатающий терминал, передавал команды редактирования, чтобы изменить только эту строку, а затем выводил заново на экран для проверки отредактированной строки¹.

Сегодня файлы больше не редактируются на печатающих терминалах, но некоторые команды линейного редактора `ex` все еще полезны для пользователей более сложных визуальных редакторов, построенных поверх `ex`. Хотя гораздо проще делать правки при помощи `vi`, линейная ориентация `ex` дает преимущества, когда необходимо внести широкомасштабные изменения для более чем одной части файла.



У многих команд, которые мы увидим в данной главе, есть аргументы имени файла. Хотя это и возможно, но, как правило, не рекомендуется оставлять пробелы в именах ваших файлов. `ex` рискует запутаться, а вы — потратить слишком много неоправданных усилий, добиваясь принятия имени файла. Используйте лучше нижнее подчеркивание, длинное тире или точки для разделения частей имен ваших файлов, и вам будет намного проще работать.

Перед тем как вы начнете просто заучивать команды `ex` (или, что еще хуже, игнорировать их), давайте приоткроем завесу тайны для некоторых линейных редакторов. Наблюдение за работой `ex` при прямом вызове поможет вам разобраться в иногда непонятном синтаксисе команд.

Откройте знакомый вам файл и попробуйте применить несколько команд `ex`. Так же как вы вызываете редактор `vi` для файла, вы можете вызвать линейный редактор `ex`, запустив команду `ex` в командной строке. При вызове `ex` вы увидите

¹ `ex` — потомок старинного Unix-редактора `ed`, который сам был основан на более раннем линейном редакторе, известном как `QED`. Версии этих редакторов до сих пор доступны для современных систем.

сообщение об общем числе строк в файле и приглашение на ввод команды в виде двоеточия. Например:

```
$ ex practice
"practice" 8L, 261B
Entering Ex mode. Type "visual" to go to Normal mode.
:
```

Вы не увидите в файле никаких строк, пока вы не зададите команду `ex`, которая отображает две и более строки.

Команды `ex` состоят из адресной строки (которая может быть просто номером строки) и команды, которая завершается нажатием клавиши **Enter**. Одной из самых базовых команд является `p` — вывод (на экран). То есть, к примеру, если вы введете `1p`, вы увидите первую строку файла:

```
:1p
With a screen editor you can
:
```

На самом деле вы можете опустить `p`, так как сам по себе номер строки эквивалентен команде вывода для этой строки. Чтобы вывести больше одной строки, вы можете указать диапазон номеров строк (в частности, `1,3` — два числа, разделенные запятой, с или без пробела между ними). Например:

```
:1,3
With a screen editor you can
scroll the page, move the cursor,
delete lines, insert characters, and more,
```

Предполагается, что команда без номера строки влияет на текущую строку. Так, к примеру, команду замены (`s`), которая позволяет вам заменить одно слово на другое, можно ввести следующим образом:

```
:1
With a screen editor you can
:s/screen/line/
With a line editor you can
```

Обратите внимание, что измененная строка выведена заново после запущенной команды. Вы также можете осуществить то же изменение немного по-другому:

```
:1s/screen/line/
With a line editor you can
```

Несмотря на то что вы будете запускать команды `ex` из `vi` и не будете их использовать напрямую, стоит провести некоторое время в самом редакторе `ex`.

Вы поймете, каким образом указать ему, с какой строкой (или строками) работать, а также какую команду выполнять.

После выполнения нескольких команд `ex` в файле `practice` вы должны вызвать `vi` для того же файла, чтобы увидеть его в более привычном визуальном режиме. При работе в `ex` команда `:vi` переводит вас из `ex` в `vi`.

Для вызова команды `ex` из `vi` нужно ввести специальный символ нижней строки : (двоеточие). Затем введите команду `ex` и нажмите клавишу `Enter` для ее выполнения. Так, к примеру, в редакторе `ex` вы переходите к строке, просто вводя ее номер в командную строку после двоеточия. Чтобы перейти к строке 6 в файле, используя эту команду из `vi`, введите:

```
:6
```

И нажмите `Enter`.

После следующего упражнения мы обсудим только те команды `ex`, которые выполняются из `vi`.

Упражнение: редактор `ex`

Это упражнение нужно запускать из окна эмулятора терминала.

1. В приглашении командной строки вызовите редактор `ex` для файла с именем `practice`:

```
ex practice
```

2. Появится сообщение:

```
"practice" 8L, 261B
Entering Ex mode. Type "visual" to go to Normal mode.
```

3. Перейдите к первой строке и выведите ее на экран:

```
:1
```

4. Выведите на экран строки с первой по третью:

```
:1,3
```

5. Замените `line` на `screen` в строке 1:

```
:1s/screen/line/
```

6. Вызовите редактор `vi` для файла:

```
:vi
```

7. Перейдите к первой строке:

```
:1
```

Проблема при переходе в визуальный режим

Когда вы редактируете в vi, вы внезапно оказываетесь в редакторе ex.

Команда Q в командном режиме vi вызывает ex. Каждый раз, когда вы находитесь в ex, команда vi возвращает вас в редактор vi.

Редактирование с ex

У многих команд ex, которые выполняют обычные операции редактирования, есть эквивалент в vi, упрощающий их работу. Очевидно, что вы будете использовать команды dw или dd для удаления одного слова или строки, а не команду delete в ex. Тем не менее, если вы хотите внести изменения, затрагивающие несколько строк, команды ex будут более полезны, так как они позволяют вам править большие фрагменты текста с помощью единственной команды.

Эти команды ex и их сокращенные версии приведены ниже. Помните, что в vi каждой команде ex должно предшествовать двоеточие. Допустимо использовать как полное имя команды, так и ее сокращенную версию, это зависит от того, что вам легче запомнить.

Полное имя	Сокращенная версия	Значение
delete	d	Удаление строк
move	m	Перемещение строк
copy	co	Копирование строк
	t	Копирование строк (синоним co; сокращение to)

Вы можете разделить различные элементы команды ex с помощью пробелов, если вам кажется, что это делает команду более читабельной. Например, вы можете таким образом разделить адреса строк, поисковые запросы и команды. Однако нельзя использовать пробел в качестве разделителя внутри запроса или в конце команды замены (некоторые примеры будут приведены в подразделе «Сочетание команд ex» далее).

Адреса строк

Для каждой команды редактирования ex вам необходимо указать, какой номер строки (или строк) редактировать. И для каждой команды ex move или copy вам также потребуется указать приложению ex, куда перемещать или копировать текст.

Указать адреса строк можно различными способами:

- с помощью точных номеров строк;
- с помощью символов, которые помогают вам указать номера строк относительно вашего текущего местоположения в файле;
- с помощью *адресных* шаблонов поиска, которые указывают необходимые строки.

Рассмотрим некоторые примеры.

Определение диапазона строк

Вы можете использовать номера строк для точного определения строки или диапазона строк. Адреса, которые используют точные номера, называются *абсолютными* адресами строк:

- `:3,18d` — удалить строки с 3-й по 18-ю включительно;
- `:160,224m23` — переместить строки с 160-й по 224-ю, чтобы они следовали после строки 23 (как команды `delete` и `put` в `vi`);
- `:23,29co100` — скопировать строки с 23-й по 29-ю и поместить их после строки 100 (как команды `yank` и `put` в `vi`).

Чтобы упростить редактирование номеров строк, вы можете также отобразить их все в левой части экрана. Следующая команда:

```
:set number
```

или ее сокращенная версия:

```
:set nu
```

отображает номера строк. Затем в файле `practice` появится следующее:

```
1 With a line editor                               "screen" ранее изменен на "Line"
2 you can scroll the page,
3 move the cursor, delete lines,
4 insert characters, and more
```

Эти номера строк не сохраняются как часть текста, когда вы записываете файл, и не отображаются, если вы распечатываете документ. Помните, что длинные строки могут занимать на экране два и более ряда, однако это все еще одна строка, и у нее есть свой номер. Номера строк отображаются до тех пор, пока вы не завершите сеанс редактирования либо пока не отключите команду `set`:

```
:set nonumber
```

или:

```
:set nonu
```

Vim позволяет вам переключать настройки с помощью:

```
:set nu!
```

Чтобы временно отобразить последовательность номеров определенных строк, вы можете использовать знак # в качестве команды. Например:

```
:1,10#
```

отображает номера строк с 1-й по 10-ю строки.

Как было описано в разделе «Перемещение по номерам строк» главы 3, вам доступна команда **Ctrl+G** для отображения текущего номера строки. Вы можете таким образом определить номера строк, соответствующие началу и концу фрагмента текста, и перемещаться между ними, используя сочетание клавиш **Ctrl+G**.

Кроме того, еще один способ определения номеров строк — это команда **=** в **ex**:

- **:=** — вывести общее количество строк;
- **:.=** — вывести номер текущей строки (точка является условным обозначением текущей строки, но об этом позднее);
- **:/шаблон/= :.,\$d** — вывести номер следующей строки, который соответствует *шаблону* (поиск начинается с текущей строки; использование шаблонов поиска кратко описано в подразделе «Шаблоны поиска» далее).

Символы адресов строк

Кроме номеров строк, вы также можете использовать символы для адресов строк. Точка (.) обозначает текущую строку, \$ — последнюю строку, а % — каждую строку. Это идентично команде **1,\$**. Эти символы можно также сочетать с абсолютным адресом строки. Например:

- **:.,\$d** — удалить строки, начиная с текущей, до конца файла;
- **:20,.\$m** — переместить строки с 20-й по текущую строку (включительно) в конец файла;
- **:%d** — удалить все строки в файле;
- **:%t\$** — скопировать все строки и поместить их в конец файла (сделав последовательный дубликат).

Помимо абсолютного адреса, вы можете указать текущий адрес. Символы **+** и **-** работают как арифметические операторы. Если их поместить перед числом, они прибавят или отнимут следующее значение. Например:

- `:. ,+20d` — удалить текущую строку и следующие 20 строк;
- `:226,$m.-2` — переместить строки с 226-й до конца файла на две строки выше текущей;
- `:. ,+20#` — отобразить номера строк, начиная с текущей, и 20 строк после нее.

На самом деле не обязательно вводить точку (`.`), когда вы используете `+` или `-`, потому что текущая строка и так считается исходным положением.

Если символы `+` и `-` не сопровождаются числами, то они считаются эквивалентными `+1` и `-1` соответственно¹. Аналогичным образом символы `++` и `--` расширяют диапазон на дополнительную строку и т. д. Соответственно, `+++` перемещает вас вперед на три строки. Символы `+` и `-` можно также использовать с шаблонами поиска, что будет рассмотрено в следующем разделе.

Число `0` обозначает верх файла (воображаемую нулевую строку) и эквивалентно `1-`. Оба эти символа позволяют вам перемещать или копировать строки в самое начало файла, перед первой строкой существующего текста. Например: `:- ,+t0` — скопировать три строки (на которой курсор, перед ним и после) и поместить их в начало файла.

Шаблоны поиска

Другим средством, с помощью которого `ex` может адресовать строку, являются шаблоны поиска. Например:

- `:/шаблон/d` — удалить следующую строку, содержащую *шаблон*;
- `:/шаблон/+d` — удалить строку *ниже* следующей строки, содержащей *шаблон* (допускается использовать `+1` вместо символа `+`);
- `:/шаблон1/,/шаблон2/d` — удалить с первой строки, содержащей *шаблон1*, по первую строку, содержащую *шаблон2*;
- `:. ,/шаблон/t23` — взять текст с текущей строки (`.`) по первую строку, содержащую *шаблон*, и поместить его после строки 23.

Обратите внимание, что тело шаблона обособлено слешем. Любые символы интервалов или табуляция между слешами воспринимаются как часть шаблона для поиска.

¹ В относительном адресе вам не стоит отделять символы плюс или минус от следующего за ними номера. Например, `+10` означает «следующие 10 строк», но `+ 10` означает «следующие 11 строк (`1 + 10`)», что, вероятно, не соответствует вашей цели (или желаниям).

Используйте в качестве разделителя ? вместо /, чтобы осуществить обратный (в другом направлении) поиск в файле. Если вы что-то удаляете с помощью шаблона в vi и ex, имейте в виду, что существует различие в работе этих двух редакторов. Предположим, ваш файл `practice` содержит следующие строки:

```
With a screen editor you can scroll the
page, move the cursor, delete lines, insert
characters, and more, while seeing results
of your edits as you make them.
```

Чтобы удалить текст до слова *while*, нужно сделать следующее.

Команда	Результат
d/while	<pre>With a screen editor you can scroll the page, move the cursor, while seeing results of your edits as you make them.</pre> Данная команда vi удаляет текст, начиная с местоположения курсора до слова <i>while</i>, но не трогает две оставшиеся строки
../while/d	<pre>With a screen editor you can scroll the of your edits as you make them.</pre> Команда ex удаляет весь диапазон адресных строк: в данном случае как текущую строку, так и содержащую шаблон. Все строки удаляются полностью

Переопределение местоположения текущей строки

Иногда использование относительного адреса строки в команде может привести к неожиданным результатам. Например, предположим, что курсор расположен на строке 1 и вы хотите вывести строку 100 плюс пять строк под ней. Если вы введете:

```
:100,+5 p
```

вы получите сообщение об ошибке E16: Invalid range или First address exceeds second от Vim и vi соответственно. Причина сбоя в команде заключается в том, что второй адрес вычисляется относительно текущего местоположения курсора (строка 1), поэтому ваша команда на самом деле передает следующее:

```
:100,6 p
```

Вам нужно каким-то способом заставить команду воспринимать строку 100 как текущую строку, несмотря на то что курсор находится на строке 1.

Редактор **ex** предоставляет такой способ. Когда вы используете точку с запятой вместо запятой, адрес первой строки пересчитывается как текущая строка. Например, команда:

```
:100;+5 p
```

выводит необходимые строки, а +5 теперь вычисляется относительно строки 100. Точка с запятой полезна при работе с шаблонами поиска, а также с абсолютными адресами. Например, чтобы вывести следующую строку, содержащую *шаблон* плюс 10 строк после него, введите команду:

```
:/шаблон/;+10 p
```

Глобальный поиск

Вы уже знаете, как использовать / (слеш) для поиска символов в ваших файлах. У **ex** есть глобальная команда **g**, которая позволяет выполнить поисковый запрос и отобразить все содержащие его строки, когда поиск завершится. Команда **:g!** выполняет действие, противоположное действию команды **:g**. Используйте **:g!** (или его аналог **:v**) для поиска всех строк, которые *не* содержат *искомый шаблон*.

Вы можете использовать глобальную команду для всех строк файла или использовать адреса строк, чтобы ограничить глобальный поиск до указанных параметров:

- **:g/шаблон** — найти (переместиться к) последнее упоминание *шаблона* в файле;
- **:g/шаблон/p** — найти и отобразить все строки файла, содержащие *шаблон*. Vim отображает строку и затем выводит подсказку: «Нажмите Enter или введите команду для продолжения»;
- **:g/шаблон/nu** — найти и отобразить все строки файла, которые не содержат *шаблон*; также отобразить номер каждой найденной строки;
- **:60,124g/шаблон/p** — найти и отобразить все строки между 60-й и 124-й, содержащие *шаблон*.

Имейте в виду, что команду **g** можно также использовать для глобальных замен. Мы обсудим это позднее в главе 6.

Сочетание команд **ex**

Не обязательно вводить двоеточие, чтобы начать новую команду **ex**. В **ex** вертикальная черта (|) является разделителем команд, позволяющим вам сочетать несколько команд за раз (очень похоже на то, как точка с запятой разделяет несколько команд в приглашении командной строки). Когда вы используете |, отслеживайте адреса

строк, которые вы указали. Если одна команда влияет на порядок строк в файле, следующая команда будет уже использовать новое местоположение строк. Например:

- `:1,3d | s/thier/their/` — удалить строки с 1-й по 3-ю (теперь вы остаетесь в верхней строке файла), а затем осуществить замену для текущей строки (которая была 4-й до того, как вы вызвали приглашение `ex`);
- `:1,5 m 10 | g/шаблон/nu` — переместить строки с 1-й по 5-ю после строки 10, а затем отобразить все строки (с номерами), содержащие *шаблон*.

Обратите внимание, что использование пробелов облегчает читабельность команд.

Сохранение файлов и завершение работы

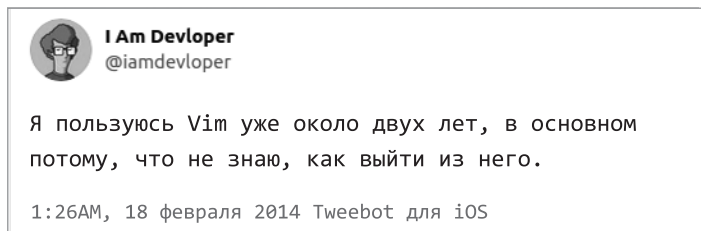


Рис. 5.1. Не все понимают `vi` (взято с <https://twitter.com/iamdeveloper/status/435555976687923200> с разрешения пользователя)

В отличие от IAmDeveloper (рис. 5.1) вы знаете команду `vi ZZ` для записи (сохранения) файла и закрытия его. Однако команды `ex` дают вам больше контроля. Мы уже мельком упоминали некоторые из них. Давайте теперь рассмотрим их более подробно:

- `:w` — записать (сохранить) буфер в файл, но не выходить из него. Вы можете (и должны) использовать `:w` на протяжении всего сеанса редактирования, чтобы защитить ваши правки от системных сбоев или серьезных ошибок редактирования;
- `:q` — выйти из редактора (и вернуться к приглашению командной строки);
- `:wq` — записать файл, а затем выйти из редактора. Запись произойдет безоговорочно, даже если файл не был изменен. Это обновит время изменения файла;
- `:x` — записать файл, а затем выйти из редактора. Файл сохраняется, только если он был изменен¹.

¹ Разница между командами `:wq` и `:x` важна при редактировании исходного кода и использовании команды `make`, которая осуществляет действия на основе времени изменения файла.

Редактор защищает существующие файлы и ваши правки в буфере. Например, если вы хотите записать ваш буфер в существующий файл, вы получите предупреждение. Также, если вы вызвали *vi* для файла, внесли правки и хотите выйти *без* сохранения, *vi* выдаст сообщение об ошибке, подобное следующему:

```
No write since last change.
```

Эти предупреждения могут предотвратить дорогостоящие ошибки, но иногда вам все равно необходимо продолжить выполнение команды. Восклицательный знак (!) после команды игнорирует предупреждение:

```
:w!  
:q!
```

Vim любезно сообщает вам следующее, когда вы пытаетесь выйти без сохранения файла:

```
E37: No write since last change (add ! to override)
```

Команду *:w!* можно также использовать для сохранения правок в файле, который был открыт в режиме чтения, с помощью *-R* или *view* в *vi* (предполагается, что у вас есть разрешение для записи файла).

Команда *:q!* является необходимой командой редактирования, позволяющей вам выходить, не затрагивая исходный файл, вне зависимости от любых изменений во время сеанса¹. Содержимое буфера отбрасывается.

Переименование буфера

Вы можете использовать команду *:w* для сохранения всего буфера (копии редактируемого вами файла) в новом файле.

Предположим, у вас есть файл *practice*, содержащий 600 строк. Вы открываете документ и вносите в него значительные правки. Вы хотите выйти, но также сохранить для сравнения *обе* версии *practice* — старую и новую. Чтобы сохранить отредактированный буфер в файле с именем *practice.new*, задайте команду:

```
:w practice.new
```

Ваша предыдущая версия файла *practice* остается неизменной (если вы ранее не использовали команду *:w*). Чтобы завершить редактирование новой версии, наберите *:q*.

¹ Отбрасываются только изменения, сделанные с момента последней записи.

Сохранение части файла

Во время редактирования вам может потребоваться сохранить только часть вашего файла в виде отдельного документа. Например, вы ввели форматирование кодов и текст, который вы хотите использовать в качестве заголовка для нескольких файлов.

Вы можете объединить адреса строк `ex` и команду записи `w`, чтобы сохранить часть файла. Например, если у вас открыт файл `practice` и вы хотите сохранить его часть как `новый_файл`, введите:

- `:230,$w новый_файл` — сохранить со строки 230 и до конца файла в `новый_файл`;
- `:600w новый_файл` — сохранить с текущей строки до строки 600 в `новый_файл`.

Добавление в сохраненный файл

Вы можете использовать переадресацию Unix и оператор присоединения (`>>`) с командой `w`, чтобы добавить все или часть содержимого буфера в существующий файл. Например, если вы ввели:

```
:1,10w новый_файл
```

и затем:

```
:340,$w >> новый_файл
```

`новый_файл` будет содержать строки с 1-й по 10-ю и с 340-й до конца буфера.

Копирование файла в другой файл

Иногда вам может понадобиться скопировать текст или данные из одного существующего файла в редактируемый вами файл. Чтение содержимого другого файла осуществляется с помощью команды `ex`:

```
:read имя_файла
```

или ее сокращенной версии:

```
:r имя_файла
```

С помощью этой команды вы вставляете содержимое считанного *файла*, начиная со строки после местоположения курсора. Если вы хотите указать иную строку, просто введите ее номер (или другой адрес строки) перед командой `:read` или `:r`.

Предположим, вы редактируете файл `practice` и хотите прочитать файл с именем `data` из другого каталога `/home/tim`. Поместите курсор на строку выше той, в которую вы хотите вставить новые данные, и введите:

```
:r /home/tim/data
```

Все содержимое `/home/tim/data` считывается в файл `practice`, начиная со строки, расположенной под курсором.

Чтобы считать тот же файл и поместить его после строки 185, вам нужно ввести:

```
:185r /home/tim/data
```

Ниже представлены другие способы чтения в файле:

- `:$r /home/tim/data` — поместить читаемый файл в конец текущего файла;
- `:0r /home/tim/data` — поместить читаемый файл в самое начало текущего файла;
- `:/шаблон/r /home/tim/data` — поместить читаемый файл в текущий файл после строки, содержащей *шаблон*.

Редактирование нескольких файлов одновременно

Команды `ex` позволяют вам переключаться между несколькими файлами. Преимущество редактирования нескольких файлов заключается в скорости, ведь требуется время, чтобы выйти и вновь войти в `vi` или `Vim` для каждого файла, который вы хотите править. Не завершая сеанс и перемещаясь между документами, вы не только ускоряете процесс, но также сохраняете определенные вами последовательности сокращений и команд (см. главу 7) и регистры для копирования текста из одного файла в другой.

Вызов Vim для нескольких файлов

Когда вы первый раз вызываете редактор, вы можете указать более одного файла для редактирования и затем использовать команды `ex` для перемещения между ними. Например:

```
$ vim файл1 файл2
```

работа начинается с файла *файл1*. После того как вы закончите редактирование, команда `ex :w` запишет (сохранит) *файл1*, а команда `:n` вызовет следующий файл (*файл2*).

Предположим, что вы хотите отредактировать два файла, `practice` и `note`.

Команда	Результат
\$ vim practice note	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, Открывает два файла: practice и note. Первый — practice — отображается на вашем экране. Вносите любые правки
:w	"practice" 8L, 261C 8,1 All Сохраняет отредактированный файл practice с помощью команды ex w
:n	Dear Mr. Henshaw: Thank you for the prompt . . . ex вызывает следующий файл note с помощью команды n. Вносите любые правки
:x	"note" 19L, 571C written 19,1 All Сохраняет второй файл note и завершает сеанс редактирования

Использование списка аргументов

На самом деле `ex` позволяет вам не только переходить к следующему файлу в списке аргументов с помощью `:n`. Команда `:args` (сокращенно `:ar`) создает список файлов, указанных в командной строке, заключая текущий файл в квадратные скобки:

Команда	Результат
\$ vim practice note	With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, Открывает два файла practice и note. Первый файл practice отобразится на вашем экране
:args	[practice] note 8,1 All Vim отображает список аргументов в строке состояния с помощью квадратных скобок вокруг текущего имени файла

Команда `vi :rewind (:rew)` сбрасывает текущий файл, чтобы он стал первым в командной строке. Vim предоставляет соответствующую команду `:last` для перемещения к последнему файлу в командной строке. Если вы просто хотите вернуться к предыдущему файлу, используйте `:prev` в Vim.

Вызов нового файла

Нет необходимости вызывать несколько файлов в начале сеанса редактирования. Вы можете переключиться на другой файл в любое время с помощью команды `ex :e`. Если вы хотите отредактировать другой файл, сохраните текущий (`:w`) и затем наберите команду:

```
:e имя_файла
```

Предположим, вы редактируете файл `practice` и хотите откорректировать файл `letter`, а затем вернуться к `practice`.

Команда	Результат
<code>:w</code>	"practice" 8L, 261C 8,1 All Сохраните <code>practice</code> с помощью команды <code>:w</code> . Файл <code>practice</code> сохранен и остается на экране. Теперь вы можете переключиться на другой файл, потому что правки сохранены
<code>:e letter</code>	"letter" 23L, 1344C 1,1 All Вызовите файл <code>letter</code> с помощью команды <code>:e</code> . Внесите любые правки

Сочетание клавиш для имени файла

Редактор запоминает два имени файлов за раз как текущий и альтернативный файлы, а ссылаться на них можно с помощью символов `%` и `#` соответственно. Символ `#` особенно полезен в сочетании с командой `:e`, поскольку позволяет вам легко переключаться между двумя файлами. В недавно приведенном примере вы можете вернуть первый файл `practice`, набрав `:e #`, или передать его в текущий файл с помощью `:r #`.

Если вы изначально не сохранили активный файл, редактор не разрешит вам переключаться между файлами с помощью команд `:e` или `:n`, пока вы не добавите восклицательный знак после команды.

Например, если после внесения правок в файл `letter` требуется сбросить правки и вернуться к файлу `practice`, наберите `:e! #`.

Следующая полезная команда сбрасывает ваши правки и возвращает последнюю сохраненную версию текущего файла:

```
:e!
```

В противоположность символу `#` символ `%` в основном чаще применяется при сохранении содержимого текущего буфера в новый файл. Например, в подразделе

«Переименование буфера» ранее мы продемонстрировали, как сохранить вторую версию файла `practice` с помощью команды:

```
:w practice.new
```

Поскольку `%` обозначает имя текущего файла, эту строку допускается набрать следующим образом:

```
:w %.new
```

Переключение файлов в командном режиме

Поскольку переключение на предыдущий файл довольно частая операция, в vi предусмотрена команда `Ctrl+^`, которая идентична `:e #`. Как и при применении команды `:e`, если текущий буфер не был сохранен, редактор не позволит вам переключиться на предыдущий файл.

Правки между файлами

Когда вы присваиваете регистру для «вырезанного» текста однобуквенное имя, у вас появляется удобный способ переместить текст из одного файла в другой. Именованные регистры не очищаются при загрузке нового файла в буфер редактирования с помощью команды `:e`. Следовательно, вырезая или удаляя текст из одного документа (при необходимости в нескольких именованных регистрах), вызывая новый файл с помощью команды `:e` и помещая в него именованный регистр (или регистры), вы можете перемещать данные между файлами.

Следующий пример демонстрирует, как перенести текст из одного файла в другой.

Команда	Результат
"f4yy	With a S creen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of the edits as you make them Помещает четыре строки в регистр f
:w	"practice" 8L, 261C 8,1 All Сохраняет файл.
:e letter	Dear Mr. Henshaw: I thought that you would b e interested to know that: Yours truly, Укажите файл letter с помощью команды <code>:e</code> . Переместите курсор туда, где необходимо разместить скопированный текст

Команда	Результат
"fр	Dear Mr. Henshaw: I thought that you would be interested to know that: With a screen editor you can scroll the page, move the cursor, delete lines, insert characters, and more, while seeing the results of the edits as you make them Yours truly, Поместите скопированный текст из именованного регистра f под курсор

Еще одним способом перемещения текста из одного файла в другой является использование команд :ya (yank) и :pu (put) в ex. Они работают так же, как и команды у и р в vi, но с использованием адресации строк ex и именованных регистров.

Например:

```
:160,224ya a
```

копирует строки с 160-й по 224-ю в регистр a. Далее с помощью :е вы переместитесь в нужный файл. Установите курсор на строку, после которой вы хотите вставить скопированные строки. Затем наберите:

```
:pu a
```

чтобы поместить содержимое регистра a после текущей строки.

Краткое описание команд ex

Ниже представлены сводные таблицы команд ex, рассмотренных в этой главе (табл. 5.1–5.7). В приложении А представлен более полный список наиболее полезных команд ex в vi и Vim.

Таблица 5.1. Команды вывода строк

Полное имя	Сокращенная версия	Значение
<i>адрес</i>		Вывести строку с <i>адресом</i>
<i>диапазон адресов</i>		Вывести строку с <i>диапазоном адресов</i>
print	р	Вывести строки
	#	Вывести строки с номерами строк

Таблица 5.2. Удаление, перемещение и копирование строк

Полное имя	Сокращенная версия	Значение
delete	d	Удалить строки
move	m	Переместить строки
copy	co	Скопировать строки
	t	Скопировать строки (синоним co; сокращение to)
yank	ya	Скопировать строки в именованный регистр
put	pu	Вставить строки из именованного регистра

Таблица 5.3. Символы адресов строк

Символ	Значение
<i>n</i>	Номер строки <i>n</i>
.	Текущая строка
\$	Последняя строка
%	Все строки файла
. + <i>n</i>	Текущая строка плюс <i>n</i>
. - <i>n</i>	Текущая строка минус <i>n</i>
/шаблон/	Поиск вперед до первого совпадения
?шаблон?	Поиск в обратную сторону до первого совпадения

Таблица 5.4. Глобальное управление

Полное имя	Сокращенная версия	Значение
global команда	g команда	Выполнить команду глобально (для всех строк)
global! шаблон команда	g! шаблон команда	Выполнить команду для всех строк, не соответствующих шаблону
	v шаблон команда	Выполнить команду для всех строк, не соответствующих шаблону

Таблица 5.5. Работа с буферами и файлами

Полное имя	Сокращенная версия	Значение
args	ar	Отобразить список аргументов с текущим файлом, заключенным в квадратные скобки
edit	e	Перейти к редактированию указанного файла
last	la	Перейти к последнему файлу в списке аргументов
next	n	Перейти к следующему файлу, указанному в командной строке
previous	prev	Вернуться к предыдущему файлу
read	r	Передать указанный файл в буфер редактирования
rewind	rew	Вернуться к первому файлу в списке аргументов
write	w	Сохранить буфер редактирования на диск
Ctrl-^		Вернуться к предыдущему файлу (команда vi)

Таблица 5.6. Выход из редактора

Полное имя	Сокращенная версия	Значение
quit	q	Завершить работу редактора
	wq	Записать файл принудительно, а затем завершить работу
xit	x	Записать файл, только если он был изменен, а затем завершить работу
Q		Переключиться на ex (команда vi)
visual	vi	Переключиться с ex на vi

Таблица 5.7. Условные обозначения имени файла

Символ	Значение
%	Текущее имя файла
#	Предыдущее имя файла

ГЛАВА 6

Глобальная замена

Иногда в середине или конце файла вы можете заметить, что некоторые вещи стали обозначаться по-другому. Или, например, в инструкции к какому-то продукту внезапно меняют (маркетинг!) его название, которое встречается в вашем документе. Достаточно часто у вас может возникнуть необходимость вернуться и изменить то, что вы уже написали, и вам требуется внести похожие или идентичные изменения в нескольких местах.

Для реализации подобных действий существует мощный инструмент под названием *глобальная замена*. С помощью одной команды можно автоматически заменить слово (или строку символов) везде, где оно встречается в файле.

При глобальной замене редактор *ex* проверяет каждую строку файла на наличие заданного набора символов. Во всех строках, где обнаружено точное совпадение, *ex* заменяет его на *новую строку* символов. На данный момент шаблон поиска будет рассматриваться как простая строка. Далее в главе мы изучим мощный язык сопоставления с шаблоном, известный как *регулярные выражения*.

По факту глобальная замена использует лишь две *ex*-команды: *:g* (*global*) и *:s* (*substitute*). Поскольку синтаксис команд глобальной замены может быть достаточно сложным, давайте рассмотрим его поэтапно.

Команда замены

Синтаксис команды замены следующий:

:s/старый/новый/

Это действие заменяет *первое* найденное совпадение *старого* символа на *новый* в текущей строке. Символ */* (слеш) является разделителем между различными частями команды и не обязателен, если речь идет о последнем символе строки (на самом деле вы можете использовать любой знак пунктуации для разделения, но об этом чуть позднее).

Команда `:s` разрешает параметры, следующие за строкой замены. Например, команда замены с синтаксисом:

```
:s/старый/новый/g
```

меняет *каждый* найденный *старый* символ на *новый* в текущей строке, а не только первый. Параметр `g` здесь также означает *global* (он влияет на каждый элемент в строке, не путайте его с командой `:g`, упомянутой ранее, которая воздействует на каждую строку файла).

Введя перед командой `:s` адрес, вы можете расширить ее диапазон, увеличив количество строк. Например, следующая команда меняет каждое упоминание *старого* набора символов на *новый* с 50-й по 100-ю строки:

```
:50,100s/старый/новый/g
```

А эта команда осуществляет замену во всем файле:

```
:1,$s/старый/новый/g
```

Допускается использовать `%` вместо `1,$`, чтобы указать каждую строку в файле. Следовательно, предыдущую команду можно также задать таким образом:

```
:%s/старый/новый/g
```

Глобальная замена работает гораздо быстрее и эффективнее, чем поиск и замена по отдельности. Поскольку команду можно использовать для внесения различных изменений, мы для начала продемонстрируем простые замены, а затем перейдем к более сложным — контекстно зависимым заменам.

Подтверждение замены

Имеет смысл быть очень осторожными, используя команды поиска и замены. Иногда полученный результат может не соответствовать вашим ожиданиям. Чтобы отменить любую команду поиска и замены, введите `u` при условии, что эта команда была последней внесенной вами правкой. Однако не всегда получается заметить нежелательные изменения вовремя.

Еще один способ защитить ваш отредактированный файл — сохранить его с помощью команды `:w` перед тем, как выполнить глобальную замену. После этого вы по крайней мере сможете закрыть файл без сохранения ваших правок и вернуться к его предыдущему варианту. Чтобы прочитать последнюю сохраненную версию файла, наберите команду `:e!` (в любом случае сохранить файл будет отличной идеей).

Разумно соблюдать осторожность и знать, что в точности будет изменено в вашем файле. Если вы хотите просмотреть результаты поиска и подтвердить каждую замену прежде, чем она будет выполнена, добавьте параметр `c` (сокращение от `confirm`) в конец команды замены:

```
:1,30s/his/the/gc
```

Редактор `ex` (в Vim) отображает всю строку там, где она была расположена, выделяет текст для замены и запрашивает подтверждение:

```
copyists at his school
~
~
~
replace with the (y/n/a/q/l/^E/^Y)?
```

Если вы хотите осуществить замену, введите `y` (от слова `yes`), если нет, просто нажмите `n` (от слова `no`).

Согласно документации Vim, ниже представлены значения возможных ответов:

- `y` — заменить это совпадение;
- `n` — пропустить это совпадение;
- `a` — заменить это и все остальные совпадения;
- `q` — завершить замену;
- `l` — заменить это совпадение и затем завершить (`l` от слова `last`);
- `Ctrl+E` — прокрутить экран вверх;
- `Ctrl+Y` — прокрутить экран вниз;
- `Esc` — завершить замену.

Vim предоставляет ряд дополнительных параметров, помимо `g` и `c`. Для получения дополнительной информации наберите `:help s_flags`.

В `vi` сочетание `n` (повторить последний поиск) и точки `(.)` (повторить последнюю команду) также является невероятно полезным и быстрым способом пролистывания файла и внесения повторяющихся изменений, которые вы не хотели бы применять глобально. Так, к примеру, если ваш редактор (человек) сказал вам, что вместо «что» лучше использовать «который», вы можете прицельно проверить каждое «что» в тексте, изменяя только неправильные:

- `/which` — искать *which*;
- `cwthat Esc` — поменять на *that*;

- `n` — повторить поиск;
- `n` — повторить поиск, пропустить изменение;
- `.` — повторить изменение (если необходимо).

Глобальные действия в файле

Редактор `ex` предоставляет мощную команду, которая позволяет применить вторую команду для всех соответствующих строк в файле. Это *глобальная* команда `:g`, которая имеет следующий вид:

`:g/шаблон/ команда`

Получив эту команду, `ex` проходит весь буфер редактирования, запоминая каждую строку, которая соответствует *шаблону* поиска. Затем для каждой соответствующей строки он выполняет заданную *команду*. Ниже приведены два примера:

- `g/# FIXME/d` — удалить все строки с комментарием `FIXME`;
- `g/# FIXME/s/FIXME/DONE/` — изменить все экземпляры с комментарием `FIXME` на `DONE`.

Как мы вскоре увидим, глобальная команда (`:g`) наиболее часто используется в сочетании с командой замены (`s`). Но она также сочетается и с другими командами `ex`, которые мы рассмотрим позднее в этой главе.

Контекстно зависимая замена

Самые простые глобальные замены замещают один набор символов на другой. Если вы сделали в файле орфографическую ошибку (*editer* вместо *editor*), то достаточно набрать:

`:%s/editer/editor/g`

Это замещает каждое слово *editer* на *editor* во всем файле.

С помощью глобальной команды `:g` можно искать строки согласно запросу, а затем менять их. Воспринимайте это как контекстно зависимую замену.

Синтаксис следующий:

`:g/шаблон/s/старое/новое/g`

Первый символ **g** указывает команде работать со всеми строками файла, соответствующими *шаблону* поиска. В строках, подходящих под *шаблон*, редактор **ex** должен заменить (**s**) *старые* символы на *новые*. Последний символ **g** указывает, что замена должна осуществляться глобально *для этой строки*. Это означает, что все соответствия *старому* заменяются на *новое*, а не только первые совпадения в каждой соответствующей строке.

Например, пока мы пишем эту книгу, инструкции HTML `<span class="keycap">` и `</span>`, встроенные в AsciiDoc, выделяют ESC, чтобы обозначить клавишу *Escape*. Иногда может потребоваться заменить *Esc* на *ESC*, но чтобы при этом изменение не затрагивало отдельные слова *Escape*, которые могут встретиться в тексте. Чтобы это сделать (когда *Esc* находится в строке, содержащей нотацию `class="keycap"`), введите:

```
:g/class="keycap"/s/ESC/ESC/g
```

Если набор символов, используемый для поиска строки, совпадает с тем, который вы хотите изменить, вам не нужно его повторять. Следующая команда:

```
:g/строка/s//новое/g
```

осуществляет поиск строк, содержащих *строку*, и производит замену для этой же строки.

Обратите внимание, что:

```
:g/едитер/s//едитер/g
```

действует так же, как и:

```
:%s/едитер/едитер/g
```

Вы можете сохранить напечатанное с помощью второй команды. Как мы упоминали ранее, допускается сочетать команду **:g** не только с командой **:s**, но и с **:d**, **:m**, **:co** и другими командами **ex**. Как вы увидите позднее, таким образом можно осуществлять глобальное удаление, перемещение и копирование.

Правила сопоставления с шаблоном

Осуществляя глобальные замены, такие редакторы Unix, как **vi** и **Vim**, позволяют искать не только фиксированные строки символов, но также изменяемые шаблоны слов, которые называются *регулярными выражениями*.

Когда вы указываете буквенную строку символов, поиск может выдать и те результаты, которые вам не нужны. Проблема поиска заключается в том, что слово

можно использовать разными способами или одно слово может быть включено в другое (например, `top` в `stopper`). Регулярные выражения помогают осуществлять поиск слов в контексте. Обратите внимание, что регулярные выражения можно использовать с командами `/` и `?`, а также с `:g` и `:s`.

В основном одни и те же регулярные выражения работают и с другими программами Unix, например `grep`, `sed` и `awk`¹.

Регулярные выражения составляются путем сочетания обычных и специальных символов, называемых *метасимволами*². Метасимволы и их использование будут рассмотрены далее.

Метасимволы в шаблонах поиска

Далее представлены метасимволы и их значения.

- `.` (*точка*) — выдает себя за любой *одионый* символ, за исключением символа новой строки. Помните, что пробел считается за символ. Например, `p.p` соответствует таким символам строк, как `per`, `pip` и `psr`.
- `*` — сопоставляет ноль и более (сколько есть) одионых символов, которые стоят непосредственно перед метасимволом. Например, `slow*w` соответствует `slow` (одно `o`) или `slw` (нет `o`). (Также соответствует `sloow`, `sloooow` и т. д.)
`*` — может следовать за метасимволом. Например, поскольку `.` (точка) соответствует любому символу, `.*` обозначает «сопоставить любое количество любого символа».

Например, команда `:s/End.* /End/` удаляет все символы после `End` (заменяет остаток строки пустотой).

- `^` — когда используется в начале регулярного выражения, необходимо, чтобы следующее за ним регулярное выражение находилось в начале строки. Например, `^Part` соответствует слову `Part`, когда оно находится в начале строки, а `^...` соответствует трем первым символам строки. Если (`^`) находится не в начале регулярного выражения, то данный символ означает сам себя.
- `$` — когда используется в конце регулярного выражения, необходимо, чтобы предшествующее ему регулярное выражение находилось в конце строки;

¹ Больше информации о регулярных выражениях можно найти в книге: Фридл Д. Регулярные выражения. 3-е изд. — СПб.: Питер, 2018.

² Строго говоря, мы должны были бы называть их метапоследовательностями, поскольку иногда два символа вместе, а не отдельный символ имеют определенное значение. Тем не менее понятие «метасимволы» является общепринятым в литературе, посвященной Unix, поэтому мы следуем этим нормам.

например, `here:$` соответствует *here*., только если оно встречается в конце строки. Если символ `$` находится не в конце регулярного выражения, то означает сам себя.

Метасимволы `^` и `$` называются *якорями*, поскольку они привязывают сопоставление к началу или к концу строки соответственно.

- `\` — относиться к этому специальному символу как к обычному символу. Например, `\.` соответствует обычной точке, а не обозначает «любой одиночный символ», а `\*` соответствует обычной звездочке вместо «любое количество символов». Обратный слеш (`\`) предотвращает интерпретацию специальных символов. Это называется экранированием. Используйте `\\`, чтобы получить просто слеш.
- `[]` — сопоставить любой *один* символ из тех, что заключены в квадратные скобки. Например, `[AB]` соответствует либо *A*, либо *B*, а `p[aeiou]t` соответствует *pat*, *pet*, *pit*, *pot* или *put*. Чтобы указать диапазон последовательных символов, разделите первый и последний дефисом. Например, диапазон `[A-Z]` соответствует заглавным буквам от *A* до *Z*, а `[0-9]` — числам от 0 до 9.

Допускается включать в квадратные скобки более одного диапазона, а также указывать совокупность диапазонов и отдельных символов. Например, `[ ;A-Za-z() ]` соответствует четырем разным знакам пунктуации, а также английским буквам.



Когда регулярные выражения и *vi* были впервые разработаны, они должны были работать только с набором символов ASCII. На сегодняшнем мировом рынке современные системы поддерживают и региональные, которые дают различные интерпретации символов, находящихся между *a* и *z*. Для получения точных результатов вам стоит использовать выражение в квадратных скобках стандарта POSIX (будет рассмотрено далее) в вашем регулярном выражении и избегать диапазонов вида *a-z*.

Большинство метасимволов утрачивают свое особое значение внутри квадратных скобок, поэтому не стоит их избегать, если требуется использовать их как обычные символы. Однако есть исключения: `\`, `-` и `]` — старайтесь избегать их внутри скобок. Дефис (`-`) приобретает значение указателя диапазона. Чтобы использовать обычный дефис, поставьте его на место первого символа внутри квадратных скобок (это также сработает и для `]`).

Каретка (`^`) приобретает специальное значение, только если она расположена первой внутри квадратных скобок, но в этом случае ее значение отличается от обычного значения метасимвола `^`. Будучи первым символом внутри квадратных скобок, `^` инвертирует их значение: скобки соответствуют любому символу, которого *нет* в списке. Например, `[^0-9]` соответствует любому символу, не являющемуся числом.

- `\( \)` — сохраняет строку, заключенную между `\(` и `\)`, в специальное хранилище, называемое *буфером хранения*¹. До девяти подстрок можно сохранить подобным образом в одиночной строке. Например, строка:

```
\(That\) or \(\this\)
```

сохраняет *That* и *this* в буфер хранения номер 1 и номер 2 соответственно. Текст, соответствующий удержанным строкам, можно «воспроизвести» в заменах с помощью последовательностей с `\1` по `\9`. Например, чтобы перефразировать *That or this*, заменив его на *this or That*, вы можете ввести:

```
:%s/\(That\) or \(\this\)/\2 or \1/
```

Допускается также использовать нотацию `\n` внутри поиска или строки замены. Например:

```
:s/\(abcd\)\1/alphabet-soup/
```

заменяет *abcdabcd* на *alphabet-soup*.

- `\< \>` — сопоставляет символы в начале (`\<`) или в конце (`\>`) слова. Начало или конец слова задается либо знаками пунктуации, либо пробелом. Например, выражение `\<ac` сопоставляет только слова, которые начинаются с *ac*, например *action*, в то время как `ac\>` — только слова, которые заканчиваются на *ac*, например *maniac*. Ни одно из этих выражений не соответствует слову *react*. Обратите внимание, что, в отличие от `\( . . \)`, данные метасимволы не обязательно использовать в сопоставимых парах.

В исходном *vi* есть дополнительные метасимволы.

`~` — сопоставляет любое регулярное выражение, которое использовалось в *последнем* поиске. Например, если вы ищете *The*, вы можете искать *Then* с `/~n`. Обратите внимание, что этот метасимвол допускается использовать только в регулярном поиске (с `/`). Он не будет работать как метасимвол в команде замены. Однако у него аналогичное значение в заменяющей части команды замены (это описано в разделе «Метасимволы в строках замены»).

Использование `~` довольно ненадежная особенность исходного *vi*. После его применения сохраненные шаблоны поиска устанавливаются на *новый* текст, введенный после `~`, а *не* на объединенный новый шаблон, как можно было бы ожидать. Несмотря на факт существования данной функции, ее навряд ли можно было бы рекомендовать к использованию. К тому же *Vim* не ведет себя подобным образом.

Обратите внимание, что *Vim* поддерживает расширенный синтаксис регулярных выражений. Для дополнительной информации ознакомьтесь с разделом «Расширенные регулярные выражения» главы 8.

¹ Буфер хранения отличается и от буфера редактирования файла, и от регистров удаления текста.

Выражения в квадратных скобках POSIX

Мы только что рассмотрели использование квадратных скобок для сопоставления любого из заключенных в них символов, например `[a-z]`. Стандарт POSIX предоставляет дополнительные средства для сопоставления символов, не являющихся буквами английского алфавита. Например, французское `è` — это алфавитный символ, но он несопоставим с типичным классом символов `[a-z]`. Кроме того, стандарт предусматривает последовательности символов, которые нужно рассматривать как одиночный структурный элемент при сопоставлении и объединении (сортировке) данных строки.

POSIX также формализует терминологию. Группы символов внутри квадратных скобок называются *выражением в квадратных скобках* стандарта POSIX. Внутри выражений в квадратных скобках наряду с такими символами, как `a`, `!` и т. д., у вас могут быть и дополнительные компоненты. К ним относятся следующие.

- *Классы символов.* Класс символа POSIX состоит из ключевого слова, заключенного в квадратные скобки с помощью `[ : и : ]`. Ключевые слова описывают различные классы символов, например алфавитные, управляющие и т. д. (табл. 6.1).
- *Объединяющие символы.* Объединяющий символ — это последовательность из нескольких символов, которую нужно воспринимать как одиночный элемент. Он состоит из символов, заключенных в скобки с помощью `[ . и . ]`.
- *Классы эквивалентности.* Класс эквивалентности содержит набор символов, которые следует рассматривать как эквивалентные, например `e` и `è`. Он состоит из именованного элемента из регионального кода, заключенного в скобки с помощью `[ = и = ]`.

Все эти компоненты *должны* находиться внутри квадратных скобок выражения в квадратных скобках. Например, `[[:alpha:]]` сопоставляет одиночный алфавитный символ или восклицательный знак, `[[:.ch.]]` сопоставляет объединяющий элемент `ch`, а не просто отдельные буквы `c` и `h`. Во французском региональном коде `[[:e=]]` может соответствовать любому из символов: `e`, `è` или `é`. Классы и сопоставляемые символы представлены в табл. 6.1.

Таблица 6.1. Классы символов POSIX

Класс	Сопоставляемые символы
<code>[[:alnum:]]</code>	Алфавитно-цифровые символы
<code>[[:alpha:]]</code>	Алфавитные символы
<code>[[:blank:]]</code>	Символы пробела и табуляции
<code>[[:cntrl:]]</code>	Управляющие символы

Класс	Сопоставляемые символы
[:digit:]	Цифровые символы
[:graph:]	Печатаемые и видимые (без пробелов) символы
[:lower:]	Символы нижнего регистра
[:print:]	Печатаемые символы (включая пробелы)
[:punct:]	Символы знаков пунктуации
[:space:]	Все символы пробелов (пробел, табуляция, новая строка, вертикальная табуляция и т. д.)
[:upper:]	Символы верхнего регистра
[:xdigit:]	Шестнадцатеричные цифры

Современные системы чувствительны к выбранному во время установки региональному коду; вы можете рассчитывать на приемлемые результаты, особенно при попытке сопоставить буквы нижнего и верхнего регистров, просто используя выражения в квадратных скобках POSIX¹.

КАК ВЫБРАТЬ СВОЙ РЕГИОНАЛЬНЫЙ КОД

Чтобы выбрать региональный код для используемых команд, необходимо установить определенные переменные окружения, чьи имена начинаются с символов LC_. В рамках этой книги мы можем лишь сказать, что самый простой способ установки регионального кода — установить переменную окружения LC_ALL. Региональный код устанавливается по умолчанию вместе с системой, если вы не заменили его.

Увидеть список доступных региональных кодов для вашей системы можно с помощью команды `locale`:

```
$ locale -a          B GNU/Linux
C
C.UTF-8
en_AG
en_AG.utf8
en_AU.utf8
...
```

Обратите внимание, что у файлов *нет* связанных с ними региональных кодов. Коды указывают, как команда интерпретирует данные, считываемые из файлов. Как правило, файлы с кодировкой UTF-8 должны правильно обрабатываться во всех основанных на Юникоде региональных кодах, но качество обработки может различаться.

¹ В операционной системе Solaris 10 `/usr/xpg4/bin/vi` and `/usr/xpg6/bin/vi` поддерживает выражения в квадратных скобках POSIX, но `/usr/bin/vi` — нет. В Solaris 11 все версии поддерживают выражения в квадратных скобках POSIX.

Метасимволы в строках замены

Когда вы осуществляете глобальные замены, метасимволы регулярных выражений, рассмотренные ранее, сохраняют свое специальное значение только в поисковой части (первая часть) команды.

Например, вводя:

```
:%s/1\. Start/2. Next, start with $100/
```

обратите внимание, что строка замены интерпретирует символы `.` и `$` буквально, не заставляя вас переключаться в другой режим. В подтверждение сказанного, допустим, вы ввели:

```
:%s/[ABC]/[abc]/g
```

Если вы рассчитываете заменить *A* на *a*, *B* на *b* и *C* на *c*, то вы будете удивлены. Поскольку квадратные скобки в строке замены действуют как обычные символы, эта команда заменяет каждое вхождение *A*, *B* или *C* на строку из пяти символов `[abc]`.

Чтобы решить подобную проблему, вам нужен способ указать переменные строк замены. К счастью, существуют дополнительные метасимволы со специальными значениями в строках *замены*.

- `\n` — заменить `\n` текстом, сопоставимым с *n*-й подстрокой, которая была сохранена ранее с помощью `\(` и `\)`, где *n* — это число от 1 до 9, а сохраненные ранее подстроки (удерживаемые в буфере хранения) отсчитываются, начиная с левой части строки. Более подробную информацию о `\(` и `\)` смотрите в подразделе «Метасимволы в шаблонах поиска» ранее.
- `\` — интерпретировать следующий специальный символ как обычный символ. Обратные слешы являются метасимволами в строке замены так же, как и в шаблонах поиска. Чтобы указать фактический обратный слеш, введите `(\\)`.
- `&` — заменить `&` целым текстом, сопоставимым с шаблоном поиска. Это нужно, чтобы не вводить текст заново:

```
:%s/Washington/&, George/
```

В замене будет *Washington, George*. Символ `&` может также заменить шаблон переменной (как указано в регулярном выражении). Например, чтобы заключить каждую строку с 1-й по 10-ю в круглые скобки, введите:

```
:1,10s/.*/(&)/
```

Шаблон поиска сопоставляет всю строку, а `&` воспроизводит строку, включенную в ваш текст.

- `~` — найденная строка заменяется текстом, указанным в *последней* команде замены. Этот символ полезен при повторе правки. Например, вы можете в одной

строке ввести `:s/thier/their/` и повторить изменение для другой строки с помощью `:s/thier/~/.` Тем не менее шаблон поиска не должен быть один и тот же. Например, допускается ввести в одной строке `:s/his/their/` и повторить замену в другой с помощью `:s/her/~/1.`

- `\u` или `\l` — вызвать следующий символ в строке замены для перевода в верхний или нижний регистр соответственно. Например, чтобы изменить *yes, doctor* на *Yes, Doctor*, вы можете ввести:

```
:%s/yes, doctor/\uyes, \udoctor/
```

Однако данный пример бесполезен, так как легче просто сразу ввести строку замены с заглавными буквами. Как и в случае с любым регулярным выражением, `\u` и `\l` наиболее полезны при работе с переменной строкой. К примеру, возьмите использованную ранее команду:

```
:%s/(That\) or \((this\)\/\2 or \1/
```

Результатом будет *this* или *That*, но требуется небольшая корректировка. Мы будем использовать `\u`, чтобы заменить первую букву в *this* на заглавную (сохранена в буфере хранения 2), и `\l`, чтобы заменить первую букву в *That* на строчную (сохранена в буфере хранения 1):

```
:s/(That\) or \((this\)\/\u\2 or \l\1/
```

Результатом будет *This* или *that*. Не перепутайте цифру 1 с нижним регистром буквы *l* — единица следует после нее.

- `\U` или `\L` и `\e` или `\E` — `\U` и `\L` идентичны `\u` и `\l`, но все следующие за ними символы преобразуются в верхний или нижний регистры до конца строки замены или до `\e` или `\E`. Если нет `\e` или `\E`, то на все символы замещающего текста воздействует `\U` или `\L`. Например, чтобы перевести в верхний регистр *Fortran*, введите:

```
:%s/Fortran/\UFortran/
```

или, используя символ `&`, повторите строку поиска:

```
:%s/Fortran/\U&/
```

Все строки поиска *чувствительны к регистру*. То есть поиск *the* не найдет *The*. Чтобы обойти эту проблему, укажите и верхний, и нижний регистры в строке:

```
/[Tt]he
```

Вы можете также указать редактору игнорировать регистр, набрав `:set ic`. Для дополнительной информации перейдите в подраздел «Команда `:set`» в главе 7.

¹ Современные версии стандартного редактора `ed` используют `%` как одиночный символ в замещаемом тексте, чтобы обозначить «замещаемый текст последней команды замены».

Дополнительные приемы замены

Вам следует знать некоторые дополнительные важные факты о команде замены.

- Просто `:s` идентично `:s//~/.` Другими словами, данная команда повторяет последнюю замену. Это позволит сэкономить огромное количество времени при работе с документом, когда вы неоднократно осуществляете одно и то же действие, но не хотите использовать глобальную замену.
- Если вы воспринимаете `&` в значении «то же самое» (как что-то только что сопоставленное), эта команда относительно проста. Вы можете использовать `g` после `&`, чтобы произвести глобальную замену для строки, и даже применять ее в диапазоне строк:

```
:%g                                      Повторяет последнюю замену везде
```

- Клавишу `&` можно использовать как команду режима `vi` для выполнения команды `:&`, то есть повторить предыдущую замену. Это может сэкономить еще больше времени, чем `:s + Enter`, — одно нажатие клавиши вместо трех.
- Команда `:~` похожа на `:&`, но с небольшим нюансом: применяемый шаблон поиска является последним регулярным выражением, использованным в *любой* команде, не обязательно в последней команде замены. Например, в последовательности:

```
:s/red/blue/
:/green
:~
```

сочетание `:~` эквивалентно `:s/green/blue/`¹.

- Кроме символа `/`, вы можете в качестве разделителя использовать любой не алфавитно-цифровой и не являющийся пробелом символ, кроме обратного слеша, двойных кавычек и вертикальной линии (`\`, `"` и `|`). Это особенно удобно, когда вам нужно изменить имя `path`. Например:

```
:%s;/user1/tim;/home/tim;g
```

- Когда включен параметр `edcompatible`, редактор запоминает флаги (`g` для глобального и `s` для подтверждения), использованные в последней замене, и применяет их к следующей.

Это особенно полезно, когда вы перемещаетесь по файлу и хотите осуществить глобальные замены. Вы можете произвести первую замену следующим образом:

```
:s/старое/новое/g
:set edcompatible
```

и после этого последующие команды замены становятся глобальными.

Обратите внимание, что, несмотря на имя, ни одна из известных версий редактора `ed` в Unix не работает подобным образом.

¹ За этот пример из документации `nvim` спасибо Киту Бостичу.

Примеры сопоставления с шаблоном

Если вы еще не знакомы с регулярными выражениями, предыдущее обсуждение специальных символов могло вам показаться невероятно сложным. Еще несколько примеров должны прояснить ситуацию. В последующих примерах квадрат (□) обозначает пробел, но он не является специальным символом.

Посмотрим, как можно использовать некоторые специальные символы при замене. Предположим, у вас есть длинный файл и вы хотите заменить слово *child* на слово *children* во всем файле. Сначала вы сохраняете отредактированный буфер с помощью :w, а затем делаете следующую глобальную замену:

```
:%s/child/children/g
```

Затем по ходу редактирования вам попадают такие слова, как *childrenish*. То есть вы непреднамеренно заменили слово *childish*. При попытке вернуть последний сохраненный буфер с помощью :e! вы вводите:

```
:%s/child□/children□/g
```

Обратите внимание, что после слова *child* есть пробел. Но эта команда пропускает вхождения *child.*, *child*, *child:* и т. д. Немного поразмыслив, вы вспоминаете, что квадратные скобки позволяют указать один символ из списка, и приходите к следующему решению:

```
:%s/child[□,.;:!?]/children[□,.;:!?]/g
```

Эта команда ищет слово *child*, после которого следует либо пробел (обозначенный □), либо любой знак пунктуации (, . ; : ! ?). Пытаясь заменить эти совпадения словом *children*, за которым следует соответствующий пробел или знак пунктуации, вы получаете кучу знаков пунктуации после каждого вхождения *children*. Вам нужно сохранить пробелы и знаки пунктуации внутри \ (и \). Затем вы можете их воспроизвести с помощью \1. Итак, следующая попытка:

```
:%s/child\([□,.;:!?]\)/children\1/g
```

Когда поиск сопоставляет символ внутри \ (и \), то \1 с правой стороны воспроизводит тот же символ. Синтаксис может показаться невероятно сложным, но эта последовательность команд сделает вас невероятно продуктивными. *Любое время, потраченное на изучение синтаксиса регулярных выражений, окупится тысячекратно!*

Однако команда не так совершенна. Вхождения *Fairchild* также изменятся, поэтому вам нужен способ сопоставить слово *child*, не являющееся частью другого слова.

Оказывается, в vi и Vim (но не во всех прочих программах, использующих регулярные выражения) есть специальный синтаксис для обозначения «только если

шаблон является целым словом». Последовательность символов \`<` требует соответствия шаблона в начале слова, в то время как \`>` — соответствия в конце слова. Использование обоих вариантов ограничивает совпадение целым словом. Таким образом, в примере последовательность \`<child\>` находит все экземпляры слова *child*, независимо от того, следуют ли за ним знаки пунктуации или пробелы. Ниже представлена команда замены, которую вам следует использовать:

```
:%s/\<child\>/children/g
```

И последний вариант:

```
:%s/\<child\>/&ren/g
```

Поиск общего класса слов

Предположим, имена вашей подпрограммы начинаются с префиксов *mg*, *mgr* и *mg*:

```
mgibox routine,  
mgrbox routine,  
mgabox routine,
```

Если вы хотите сохранить префиксы во время изменения имени *box* на *square*, любая из следующих команд замены сделает это. Первый пример демонстрирует, как можно использовать \`(` и \`)` для сохранения любого фактически сопоставленного шаблона. Второй пример показывает, как вы можете осуществить поиск одного шаблона, не изменив при этом другой:

```
:g/mg\([ira]\)box/s//mg\1square/g
```

```
mgisquare routine,  
mgrsquare routine,  
mgasquare routine,
```

Глобальная замена отслеживает, сохранены ли *i*, *r* или *a*. В таком случае *box* заменяется на *square*, только если *box* — часть имени подпрограммы:

```
:g/mg[ira]box/s/box/square/g
```

```
mgisquare routine,  
mgrsquare routine,  
mgasquare routine,
```

Результат тот же, что и при применении предыдущей команды, но он менее безопасен, так как может изменить другие экземпляры *box* в этой же строке, а не только те, что внутри имени подпрограммы.

Перемещение фрагментов с помощью шаблонов

Вы также можете перемещать фрагменты текста, ограниченные шаблонами. Например, предположим, что у вас есть 150-страничное справочное руководство, написанное в специализированной версии XML. Каждая страница разделена на три абзаца с одинаковыми тремя заголовками: `<syntax>`, `<description>` и `<parameters>`. Вот пример одной страницы:

```
<reference>
<description>Get status of named file</description>
<shortname>STAT</shortname>
<syntax>
int stat(const char *filename, struct stat *data);
...
retval = stat(filename, data);</syntax>
<description><para>
Writes the fields of a system data structure into the
structure pointed to by data.
These fields contain (among other
things) information about the file's access
privileges, owner, and time of last modification.
</para></description>
<parameters>
<param><name>filename</name>
<para>A character string variable or constant containing
the Unix pathname for the file whose status you want
to retrieve.
You can give the ...
</para></param></parameters>
</reference>
```

Предположим, вы решили переместить `<description>` над абзацем `<syntax>`. Путем сопоставления с шаблоном вы можете переместить фрагменты текста на всех 150 страницах с помощью лишь одной команды!

```
:g /<syntax>/./</description>/-1 move /<parameters>/-1
```

Эта команда работает следующим образом. Во-первых, `ex` находит и помечает каждую строку, которая соответствует первому шаблону (то есть содержащую слово `<syntax>`). Во-вторых, для каждой отмеченной строки он устанавливает `.` (точка, текущая строка) на эту строку и выполняет команду. Команда перемещает группу строк с текущей строки (точка) на строку перед той, которая содержит слово `<description>` (`/</description>/-1`), до строки, содержащей `<parameters>` (`/<parameters>/-1`)¹.

¹ Мы могли бы переместить ее в строку после `</description>` с помощью `</</description>/`. Однако нельзя с точностью сказать, является ли это более или менее читабельным.

Обратите внимание, что `ex` может поместить текст только под указанной строкой.

Чтобы указать `ex` разместить текст над строкой, вы сначала отнимаете один с помощью команды `-1`, и затем `ex` помещает ваш текст под предыдущей строкой.

В подобном случае одна команда экономит буквально часы работы. Вот реальный пример: мы однажды использовали подобное сопоставление с шаблоном, чтобы переделать справочное руководство, содержащее сотни страниц.

Определение фрагментов с помощью шаблонов можно использовать также и с другими командами `ex`. Например, если вы хотите удалить все абзацы `<description>` в справочной главе, введите:

```
:g/<description>/,/<parameters>/-1d
```

Данный мощный вид изменения подразумевается в синтаксисе адресации строк `ex`, но он неочевиден даже для опытных пользователей. По этой причине, когда вы сталкиваетесь со сложной повторяющейся задачей редактирования, потратьте время на анализ проблемы и определите, можете ли вы применить инструменты сопоставления с шаблоном, чтобы выполнить работу.

Еще примеры

Поскольку лучший способ изучения сопоставления с шаблоном — на практике, далее представлены еще примеры с пояснениями. Внимательно ознакомьтесь с синтаксисом, чтобы понять принципы работы. Потом вы сможете адаптировать эти примеры под свои задачи.

НЕМНОГО О TROFF

Стандартные инструменты форматирования текста в Unix — это `troff` для наборщиков и лазерных принтеров и его собственный брат-близнец `nroff` для терминалов и линейных принтеров. Они используют одинаковый входной язык.

Входные данные для `troff` состоят из формируемого текста вперемешку с командными строками и управляющими последовательностями (например, для выделения текста курсивом или жирным шрифтом).

Когда-то давно знание и умение работать с `troff` и `nroff` было обязательным условием, чтобы стать мастером Unix. Со временем необходимость в них отпала, но они продолжают использоваться для одной важной задачи: создания страниц руководства.

Таким образом, хотя мы и сократили количество примеров с `troff` в книге, мы не удалили их все. Мы надеемся, что оставшиеся из них будут вам полезны.

1. Заключить слово *Enter* в коды `troff` для курсива:

```
:%s/Enter/\\fI&\\fP/g
```

Обратите внимание, что два обратных слеша (\\) нужны в замене, потому что одна обратная косая черта в коде `troff` для курсива будет интерпретироваться как специальный символ: `\\fI` считывается программой как *fI*, введите `\\fI`, чтобы получить *fI*.

2. Изменить список путей имен в файле:

```
:%s/\\home\\tim/\\home\\linda/g
```

Слеш (используемый как разделитель в последовательности глобальной замены) должен экранироваться обратным слешем, если он выступает в составе пути или экранирования (используйте `\\` для получения `/`). Альтернативный способ достижения того же результата заключается в использовании другого символа в качестве разделителя. Например, вы можете осуществить предыдущую замену, используя двоеточие в качестве разделителей (разделяющие двоеточия и команда `:ex` — это разные команды). Таким образом:

```
:%s:/home/tim:/home/linda:g
```

Это более читабельно.

3. Заключить слово *Enter* в код HTML для курсива:

```
:%s:Enter:<I>&</I>:g
```

Обратите внимание на использование `&` для представления фактически сопоставленного текста и, как было описано в предыдущем пункте, на использование двоеточий в качестве разделителей вместо слешей.

4. Заменить все запятые на точку с запятой в строках с 1-й по 10-ю:

```
:1,10s/\\./;/g
```

У точки есть специальное значение в синтаксисе регулярного выражения, и его нужно экранировать с помощью обратного слеша (\\.).

5. Заменить все вхождения слова *help* (или *Help*) на *HELP*:

```
:%s/[Hh]elp/HELP/g
```

или

```
:%s/[Hh]elp/\\U&/g
```

`\\U` заменяет следующие за ним символы на все заглавные. Следующие символы — это воспроизведенный шаблон поиска, который представляет собой *help* или *Help*.

6. Заменить *один или более* пробелов на один пробел:

```
:%s/\\s+/* */g
```

Убедитесь, что вы понимаете, как работает специальный символ *. Звездочка, следующая за любым символом (или регулярным выражением, которое соответствует одному символу, например `.` или `[[:lower:]]`), соответствует *нулю или более* экземпляров этого символа. Следовательно, вы должны указать *два* пробела, за которыми следует звездочка, чтобы сопоставить один и более пробелов (один пробел плюс ноль или более пробелов).

7. Заменить один или несколько пробелов после двоеточия ровно двумя пробелами:

```
:%s/:□□*/:□□/g
```

8. Заменить один или несколько пробелов после точки *или* двоеточия ровно двумя пробелами:

```
:%s/\([[:.])□□*/\1□□/g
```

Один из двух символов внутри квадратных скобок может быть сопоставлен. Этот символ помещается в буфер хранения с помощью `\(` и `\)` и восстанавливается с правой стороны с помощью `\1`. Обратите внимание, что внутри квадратных скобок такие специальные символы, как точка, не нужно экранировать.

9. Стандартизировать различные варианты использования слова или заголовка:

```
:%s/^Note[□:s]*/Notes:□/g
```

В квадратные скобки заключено три символа: пробел, двоеточие и буква `s`. Следовательно, набор символов `Note[□s:]` соответствует `Note□`, `Notes` или `Note:.` Звездочка добавляется к набору символов, чтобы он также соответствовал `Note` (без пробелов после него) и `Notes:` (уже правильное написание). Без звездочки `Note` будет пропущен, а `Notes:` — неправильно заменен на `Notes:□:`. В то же время это сокращает множество пробелов до одного таким образом, что `Note:□□` становится `Notes.□`.

10. Удалить все пустые строки:

```
:g/^$/d
```

На самом деле вы здесь сопоставляете начало строки (^) и конец строки (\$), а между ними ничего.

11. Удалить все пустые строки плюс все строки, содержащие только пробел:

```
:g/^[□tab]*$/d
```

В примере табуляция обозначена как `tab`. Строка может казаться пустой, но по факту содержать пробелы или табуляцию. В предыдущем примере подобные строки не удалялись. В этом же, как и в предыдущем, осуществляется поиск начала и конца строки, но при этом шаблон пытается найти любое количество пробелов или табуляций. Если соответствие не найдено, то строка считается

пустой. Чтобы удалить строки, содержащие пробел, но *не являющиеся* пустыми, вам нужно сопоставить строки *как минимум* с одним пробелом или табуляцией:

```
:g/^[tab][tab]*$/d
```

12. Удалить все начальные пробелы в каждой строке:

```
:%s/^[tab]*\(.*\)/\1/
```

Используйте `^[tab]*` для поиска одного или более пробелов в начале каждой строки, а затем `\(.*)`, чтобы сохранить оставшуюся часть строки в первый буфер хранения. Восстановите строку без начальных пробелов с помощью `\1`. Это можно сделать просто с помощью `s/^[tab]*//`.

13. Удалить все пробелы в конце каждой строки:

```
:%s/[tab]*$/
```

Для каждой строки убираем один или более пробелов в конце строки.

Из-за якорей `^` и `$` замены в этом и предыдущем примерах случаются только раз для любой заданной строки, поэтому параметр `g` не должен предшествовать строке замены.

14. Вставить `//` в начало каждой строки, начиная с текущей и до следующей, начинающейся с `}`:

```
:.,/^}/s;^;//
```

На самом деле мы здесь «заменяем» начало строки на `//`. Естественно, начало строки (будучи логической структурой, а не фактическим символом) не заменяется по-настоящему!

Команда превращает в комментарий все строки, начиная с текущей (точка) и до следующей, которая начинается с правой (закрывающей) фигурной скобки, с помощью комментариев `C++ //`. Как правило, эту команду используют, помещая курсор на первую строку определения функции, чтобы закомментировать всю функцию.

Обратите внимание на использование точки с запятой в качестве разделителя для команды замены, когда замещаемый текст содержит один или несколько слешей.

15. Добавить точку в конец следующих шести строк:

```
:.,+5s/$/./
```

Адрес строки указывает текущую строку плюс пять строк. Символ `$` указывает на конец строки. Как и в предыдущем примере, `$` является логической структурой. Вы на самом деле не заменяете конец строки.

16. Инvertировать порядок всех элементов списка, разделенных дефисом:

```
:%s/\(.*)- \(.*)/\2- \1/
```

Используйте `\(.*\)`, чтобы сохранить текст строки в первый буфер хранения, но только пока не найдете `□-□`. Затем используйте `\(.*\)`, чтобы сохранить оставшуюся часть строки во второй буфер хранения. Восстановите сохраненные части строки, инвертируя порядок двух буферов хранения. Результат этой команды для нескольких элементов показан ниже:

```
more - display files
```

становится:

```
display files - more
```

и:

```
lp - print files
```

становится:

```
print files - lp
```

Существует даже более лаконичная команда:

```
:%s/\(.*\)\(□-□\)\(.*\)/\3\2\1/
```

17. Заменить все буквы в файле на заглавные:

```
:%s/.*/\U&/
```

или

```
:%s/.*/\U&/g
```

Флаг `\U` в начале строки замены говорит редактору преобразовать заменяемое в верхний регистр. Символ `&` воспроизводит текст, сопоставляемый с помощью шаблонов поиска, как замену.

Эти две команды эквивалентны; тем не менее первая форма значительно быстрее, поскольку в результате получается лишь одна замена на строку (`.*` сопоставляет всю строку, один раз для каждой строки), в то время как вторая форма приводит к повторяющимся заменам для каждой строки (`.` сопоставляет только одиночный символ, а замена повторяется из-за завершающей команды `g`).

18. Инвертировать порядок строк в файле¹:

```
:g/.*/m0
```

Шаблон поиска сопоставляет все строки (строку, содержащую ноль или несколько символов). Каждая из них перемещается, одна за одной, в верх файла (то есть помещается после воображаемой нулевой строки). Как только строка оказывается наверху, она проталкивает предыдущие строки вниз, каждую по

¹ Из статьи Уолтера Зинца в UnixWorld, май 1990.

очереди, пока последняя не окажется наверху. Поскольку у всех строк есть начало, того же результата можно достичь более лаконично:

```
:g/^/mo0
```

19. В базе данных текстовых файлов ко всем строкам, не отмеченным фразой *Paid in full*, необходимо добавить фразу *Overdue*:

```
:g!/Paid in full/s/$/ Overdue/
```

или эквивалент:

```
:v/Paid in full/s/$/ Overdue/
```

Чтобы воздействовать на все строки, *кроме* соответствующих вашему шаблону, добавьте ! к :g или просто используйте команду v.

20. Каждую строку, которая начинается не с числа, переместить в конец файла:

```
:g!/^[[[:digit:]]]/m$
```

или:

```
:g/^[^[:digit:]]/m$
```

Будучи первым символом внутри квадратных скобок, каретка инвертирует значение команды, поэтому две команды приводят к одному результату. Первая обозначает: «Не сопоставляйте строки, начинающиеся с числа», а вторая — «Сопоставьте строки, которые не начинаются с числа».

Обратите внимание, что *есть* небольшое различие между этими командами. Первая воздействует на пустые строки, вторая — нет. Как же так? Команда /[^][[[:digit:]]/ сопоставляет строки, которые начинаются с цифры. А ! после команды :g инвертирует это, сопоставляя строки, которые не начинаются с цифры. Сюда относятся и пустые строки. Однако /[^][[^][:digit:]]/ сопоставляет строки, которые начинаются с нецифрового символа (для сопоставления в строке должен быть символ).

21. Изменить пронумерованные вручную заголовки разделов (например, 1.1, 1.2 и т. д.) на маркеры заголовков HTML <h1>:

```
:%s;^[1-9]\.[1-9] \(.*\);<h1>\1</h1>;
```

Строка поиска сопоставляет цифру, отличную от нуля, за которой следует точка, сопровождаемая еще одной отличной от нуля цифрой, за которой следует пробел, сопровождаемый чем угодно. Только что показанная команда не найдет номера глав с двумя или более цифрами. Для этого измените ее следующим образом:

```
:%s;^[1-9][0-9]*\.[1-9] \(.*\);<h1>\1</h1>;
```

Теперь она сопоставляет главы с 10-й по 99-ю (цифры от 1 до 9, за которыми следует любая цифра), 100-й по 999-ю (цифры от 1 до 9, за которыми следуют две любые цифры) и т. д. Команда по-прежнему находит главы от 1 до 9 (цифры от 1 до 9, не сопровождаемые другими цифрами).

22. Удалить нумерацию из заголовков разделов в документе. Вам нужно заменить шаблонные строки:

```
2.1 Introduction
10.3.8 New Functions
```

на строки:

```
Introduction
New Functions
```

Это можно сделать с помощью следующей команды:

```
:%s/^[1-9][0-9]*\.[1-9][0-9.]*\s//
```

Шаблон поиска похож на приведенный в предыдущем примере, только теперь числа различаются по длине. Как минимум заголовок содержит *число*, *точку*, *число*, поэтому вы начинаете с шаблона поиска из предыдущего примера:

```
[1-9][0-9]*\.[1-9]
```

Но в данном примере заголовок может содержать любое количество цифр или точек:

```
[0-9.]*
```

23. Заменить слово *Fortran* на фразу *FORTRAN* (акроним для *FORmula TRANslation*):

```
:%s/\(For\)\(tran\)/\U1\2E\(\acronym\of\U1\Emu1\U2\Eslation)/g
```

Сначала, поскольку слова *FORmula* и *TRANslation* используют части изначальных слов, необходимо сохранить шаблон поиска в двух частях: `\(For\)` и `\(tran\)`. При первом его восстановлении мы используем обе части вместе, переводя все символы в верхний регистр: `\U1\2`. Далее мы отменяем верхний регистр с помощью `\E`, иначе весь оставшийся текст будет в верхнем регистре. Замена продолжается с помощью фактически введенных слов, а затем восстанавливается первый буфер хранения. Этот буфер все еще содержит *For*, поэтому сперва нужно вновь перевести в верхний регистр: `\U1`. Сразу же после этого мы переводим остаток слова в нижний регистр: `\Emu1`. И наконец, мы восстанавливаем второй буфер хранения. Он содержит *tran*, поэтому необходимо повторить все предыдущие действия: перевод в верхний регистр, затем в нижний и вывод оставшегося слова `\U2\Eslation`).

Заключительные примеры сопоставления с шаблоном

В завершение этой главы мы представим задачи, реализующие сложные принципы сопоставления с шаблоном. Вместо того чтобы дать готовый ответ сразу, мы проработаем решения шаг за шагом.

Удаление неизвестного фрагмента текста

Предположим, у вас есть несколько строк следующего содержания:

```
the best of times; the worst of times: moving
The coolest of times; the worst of times: moving
```

Интересующие вас строки всегда заканчиваются словом *moving*, но вы никогда не знаете, какими могут быть первые два слова. Вы хотите изменить каждую строку, заканчивающуюся словом *moving*, следующим образом:

```
The greatest of times; the worst of times: moving
```

Поскольку требуется изменить определенные строки, вам нужно указать контекстно зависимую глобальную замену. Команда `:g/moving$/` сопоставляет строки, заканчивающиеся словом *moving*. Ваш шаблон поиска может содержать любое количество любого символа, поэтому на ум приходят метасимволы `.*`. Но они сопоставляют всю строку, если не ограничены каким-то образом. Вот какой будет ваша первая попытка:

```
:g/moving$/s/.*of/The□greatest□of/
```

Вы решаете, что эта строка поиска будет искать совпадения с начала строки до первого вхождения *of*. Поскольку вам нужно было указать слово *of*, чтобы ограничить поиск, вы просто повторяете его в замене. Вот как выглядит итоговая строка:

```
The greatest of times: moving
```

Но что-то пошло не так! Замена поглотила строку до второго упоминания *of* вместо первого. И вот почему: если дается выбор, то действие «сопоставить любое количество любого символа» сопоставляет *как можно больше текста*¹. В данном случае, поскольку слово *of* появляется дважды, ваша строка поиска находит следующее:

```
the best of times; the worst of
```

а не:

```
the best of
```

Ваш шаблон поиска должен быть более строгим:

```
:g/moving$/s/.*of□times;/The□greatest□of□times;/
```

Теперь метасимволы `.*` сопоставляют все символы до экземпляра фразы *of times*. Поскольку в тексте только один экземпляр, он должен быть первым.

¹ Если более формально, то сопоставляется самый длинный крайний слева текст.

Тем не менее бывают случаи, когда использовать метасимволы `.` и `*` неудобно или даже неправильно. Например, чтобы ограничить ваш шаблон поиска, придется ввести много слов, иначе будет невозможно установить границы поиска специальными словами (если текст в строках сильно отличается). В следующем разделе рассмотрен как раз такой случай.

Переключение элементов в текстовой базе данных

Предположим, вы хотите поменять местами фамилии и имена в (текстовой) базе данных. Строки выглядят следующим образом:

```
Name: Feld, Ray; Areas: PC, Unix; Phone: 765-4321
Name: Joy, Susan S.; Areas: Graphics; Phone: 999-3333
```

Каждое имя поля заканчивается запятой, а каждое поле разделено точкой с запятой. Взяв в качестве примера первую строку, вы меняете *Feld, Ray* на *Ray Feld*. Мы рассмотрим несколько команд, которые выглядят многообещающе, но не работают. После каждой команды мы продемонстрируем, как выглядит строка до и после изменений:

```
:%s/: \(.*\), \(.*\);/: \2 \1;/
```

```
Name: Feld, Ray; Areas: PC, Unix; Phone: 765-4321      До
Name: Unix Feld, Ray; Areas: PC; Phone: 765-4321      После
```

Мы выделили содержимое первого буфера хранения **полужирным шрифтом**, а второго — *курсивом*. Обратите внимание, что первый буфер хранения содержит больше, чем вам необходимо. Поскольку он не был строго ограничен следующими за ним метасимволами, то буфер хранения смог сохранить текст до второй запятой. Попробуйте теперь ограничить содержимое первого буфера хранения:

```
:%s/: \(....\), \(.*\);/: \2 \1;/
```

```
Name: Feld, Ray; Areas: PC, Unix; Phone: 765-4321      До
Name: Ray; Areas: PC, Unix Feld; Phone: 765-4321      После
```

Здесь вы попытались сохранить фамилию в первом буфере, но теперь второй сохраняет все вплоть до последней точки с запятой в строке. Теперь ограничьте также второй буфер хранения:

```
:%s/: \(....\), \(...\);/: \2 \1;/
```

```
Name: Feld, Ray; Areas: PC, Unix; Phone: 765-4321      До
Name: Ray Feld; Areas: PC, Unix; Phone: 765-4321      После
```

Вы получили желаемый результат, но только в особом случае для четырехбуквенной фамилии и трехбуквенного имени (предыдущая попытка содержала ту же ошибку). Почему же просто не вернуться к первой попытке, но на этот раз задать более строгие ограничения относительно конца шаблона поиска?

```
%s/: \(.*\), \(.*\); Area/: \2 \1; Area/
```

```
Name: Feld, Ray; Areas: PC, Unix; Phone: 765-4321    До
Name: Ray Feld; Areas: PC, Unix; Phone: 765-4321    После
```

Сработало! Однако есть другая загвоздка. Предположим, что поле *Area* не всегда представлено в строке или не всегда второе. Только что показанная команда не будет работать для таких строк.

Мы приводим эту проблему, чтобы пояснить суть. Каждый раз, когда вы рассматриваете сопоставление элементов, лучше работать над совершенствованием переменных (метасимволов), а не подбирать специальные символы для ограничения поиска. Чем больше переменных вы используете в вашем шаблоне, тем эффективнее будет команда.

Взгляните на приведенный пример еще раз и подумайте об элементах, которые хотите поменять местами. Каждое слово начинается с заглавной буквы и сопровождается некоторым количеством строчных букв, поэтому вы можете сопоставить имена следующим образом:

```
[[:upper:]][[:lower:]]*
```

В фамилии также может быть более одной заглавной буквы (например, *McFly*), поэтому вам следует поискать такие случаи во второй и последующих буквах:

```
[[:upper:]][[:alpha:]]*
```

Не помешает использовать это также и для имени (никогда не угадаешь, когда появится *McGeorge Bundy*). Ваша команда теперь выглядит следующим образом:

```
%s/: \([[:upper:]][[:alpha:]]*\), \([[:upper:]][[:alpha:]]*\);/: \2 \1;/
```

Подобная запись довольно тяжело воспринимается, не правда ли? И она все еще не охватывает такие случаи, как *Joy*, *Susan S.* Поскольку поле имени может включать инициалы второго имени, вам нужно добавить пробел и точку внутри второй пары скобок. Но и этого хватит. Иногда точно указать, что вы хотите, оказывается сложнее, чем указать, чего вы *не* хотите. В вашем примере базы данных фамилия заканчивается запятой, поэтому поле фамилии можно рассматривать как строку символов, которые *не* являются запятыми:

```
[^,]*
```

Данная команда сопоставляет символы до первой запятой. Аналогично поле имени является строкой символов, которые *не* являются точкой с запятой:

```
[^;]*
```

Поместив эти более эффективные команды в вашу предыдущую команду, вы получите:

```
:%s/: \([^,]*\), \([^;]*\);/: \2 \1;/
```

Ту же команду можно ввести как контекстно зависимую замену. Если все строки начинаются с *Name*, допускается ввести:

```
:g/^Name/s/: \([^,]*\), \([^;]*\);/: \2 \1;/
```

Можно также добавить звездочку после первого пробела, чтобы сопоставить двоеточие, после которого есть лишние пробелы (или нет пробелов):

```
:g/^Name/s/: *\([^,]*\), \([^;]*\);/: \2 \1;/
```

Использование :g для повтора команды

Когда команда :g используется обычным способом, с которым мы уже знакомы, она выбирает строки, которые, как правило, потом редактируются последующими командами в той же командной строке *ex*: например, мы выбираем строки с помощью :g и затем осуществляем в них замены или выбираем их и удаляем:

```
:g/mg[ira]box/s/box/square/g
:g/^$/d
```

Однако в своем двухчастном обучающем руководстве в *UnixWorld*¹ Уолтер Зинц высказывает интересные соображения по поводу команды :g. Эта команда выбирает строки, но ассоциированные команды редактирования не обязательно фактически воздействуют на них.

Вместо этого он демонстрирует метод, с помощью которого вы можете повторить команды *ex* некоторое произвольное количество раз. Например, предположим, что вы хотите поместить десять копий строк с 12-й по 17-ю вашего файла в конец вашего текущего файла. Вы можете ввести:

```
:1,10g/^/ 12,17t$
```

¹ Часть 1 «Советы по vi для опытных пользователей» вышла в апрельском выпуске *UnixWorld* 1990 года; часть 2 «Использование vi для автоматизации сложных правок» вышла в майском выпуске 1990 года. Представленные примеры взяты из части 2. Обучающее руководство доступно в хранилище GitHub (<https://www.github.com/learning-vi/vi-files>).

Это очень неожиданное использование команды `:g`, но оно работает! Команда `:g` выбирает строку 1, выполняет указанную команду `t`, а затем переходит к строке 2, чтобы выполнить следующую команду копирования. Когда будет достигнута строка 10, `ex` сделает десять копий.

Сбор строк

Рассмотрим еще один расширенный пример использования `:g`, основанный на предложениях из обучающего руководства Зинца. Предположим, что вы редактируете документ, состоящий из нескольких частей. Ниже приведена часть 2 этого файла, в котором многоточия обозначают пропущенный текст и номера строк для справки:

```
301 Part 2
302 Capability Reference
303 .LP
304 Chapter 7
305 Introduction to the Capabilities
306 This and the next three chapters ...

400 ... and a complete index at the end.
401 .LP
402 Chapter 8
403 Screen Dimensions
404 Before you can do anything useful
405 on the screen, you need to know ...

555 .LP
556 Chapter 9
557 Editing the Screen
558 This chapter discusses ...

821 .LP
822 Part 3
823 Advanced Features
824 .LP
825 Chapter 10
826 ....
```

Номер каждой главы отображается в одной строке, название главы — в строке ниже, а текст главы (выделен **полужирным шрифтом**) начинается в строке под ней. Первое, что вы хотели бы сделать, — скопировать начальную строку каждой главы, отправив ее в уже существующий файл с именем *begin*.

Сделать это можно с помощью следующей команды:

```
:g /^Chapter/ .+2w >> begin
```

Перед тем как использовать эту команду, необходимо переместиться на верх вашего файла. Сначала вы ищете *Chapter* в начале строки, но затем запускаете команду

в начальной строке каждой главы — во второй строке под *Chapter*. Поскольку строка, начинающаяся с *Chapter*, теперь выделена как текущая, адрес строки `.+2` указывает на вторую строку под ней (допускается использовать эквивалентные адреса строки `+2` или `++`). Вам нужно записать эти строки в существующий файл с именем *begin*, поэтому вы используете команду `w` с оператором присоединения `>>`.

Предположим, вы хотите отправить начало глав только из части II. Для этого необходимо ограничить строки, выбранные с помощью команды `:g`, поэтому нужно изменить команду, чтобы она приняла следующий вид:

```
:/^Part 2/,/^Part 3/g /^Chapter/ .+2w >> begin
```

Здесь команда `:g` выбирает строки, начинающиеся с *Chapter*, и осуществляет поиск в файле, начиная с первой строки в части II по первую строку в части III. Если вы запустите только что продемонстрированную команду, последние строки файла *begin* будут читаться следующим образом:

```
This and the next three chapters ...  
Before you can do anything useful  
This chapter discusses ...
```

С этих строк начинаются главы 7, 8 и 9.

В дополнение к только что отправленным строкам вам нужно скопировать названия глав в конец документа, чтобы сформировать оглавление. Вы можете использовать вертикальную черту, чтобы добавить вторую команду после вашей первой команды, следующим образом:

```
:/^Part 2/,/^Part 3/g /^Chapter/ .+2w >> begin | +t$
```

Помните, что в каждой последующей команде адреса строк задаются относительно предыдущей команды. Первая отметила строки (в части II), которые начинаются с *Chapter*, а названия глав появляются в строке под этими строками. Следовательно, чтобы получить доступ к названиям глав во второй команде, нужно ввести адрес строки `+` (или его эквиваленты `+1` или `.+1`). Затем используйте `t$` для копирования названий глав в конец файла.

Как видите, размышления и эксперименты могут привести к необычным решениям в редактировании. Не бойтесь пробовать. Просто для начала проверьте, сделали ли вы резервную копию файла! Конечно, поскольку в Vim есть возможность бесконечной «отмены», вам может даже и не понадобится дубликат файла. Для дополнительной информации перейдите в раздел «Расширенный откат изменений» главы 8.

Продвинутое редактирование

В текущей главе вы познакомитесь с некоторыми более расширенными возможностями `vi` и `Vim`, а также лежащего в их основе редактора `ex`. Вы должны основательно ознакомиться с материалом предыдущих глав, прежде чем начать работу с понятиями, представленными здесь.

Как и предыдущие главы, эта рассматривает общие для всех версий `vi` возможности, но в контексте `Vim`. Когда вы встречаете в тексте просто `vi`, как правило, имеется в виду `vi` и `Vim`.

Данная глава разделена на пять частей. В первой обсуждается ряд способов установки параметров, которые позволяют настроить ваше окружение редактирования. Вы научитесь использовать команду `set` и создавать некоторые виды окружений редактирования с помощью файлов `.exrc`.

Во второй части мы расскажем, каким образом можно выполнять команды `Unix` из редактора, а также рассмотрим варианты использования редактора для фильтрации текста при выполнении команд `Unix`.

Из третьей части вы узнаете о различных способах привязки длинных последовательностей команд к нажатию одной или нескольких клавиш (это называется *переназначением* клавиш). Сюда также включен раздел, посвященный `@`-функциям, которые позволяют сохранять последовательность команд в регистре.

В четвертой части обсуждается использование сценариев (скриптов) `ex` из командной строки `Unix` или из сценариев оболочки. Сценарии — это эффективный способ осуществления повторяющихся правок.

В пятой части рассматриваются некоторые функции, которые особенно полезны для программистов. Это параметры для контроля отступов строк и параметр для отображения невидимых символов (в частности, табуляции и новых строк). А также команды поиска, которые удобны при работе с блоками программного кода или с функциями `C` и `C++`.

Настройка vi и Vim

vi и Vim работают по-разному в зависимости от терминала, на котором они установлены.

В современных системах Unix редакторы получают инструкции по работе вашего типа терминала из базы данных терминала `terminfo`¹.

Существует также ряд параметров, которые вы можете установить внутри редактора и которые влияют на его работу. Например, можно задать правое поле, в результате чего **vi** будет переносить строки автоматически и вам не придется нажимать клавишу `Enter`.

Параметры внутри редактора изменяются с помощью команды `ex :set`. Кроме того, при каждом запуске **vi** и **Vim** считывают файл `.exrc` в вашем домашнем каталоге для получения дальнейших инструкций для работы. Поместив в этот файл команду `:set`, вы можете изменить способ действия редактора всякий раз, когда вы его используете.

Есть также возможность установить файлы `.exrc` в локальных каталогах, чтобы инициализировать разнообразные параметры, которые вы хотите использовать в различных окружениях. Например, вы могли бы задать один набор параметров для редактирования английского текста, а другой набор для редактирования исходной программы. Первым выполняется файл `.exrc` в вашем домашнем каталоге, а затем — находящийся в вашем текущем каталоге.

И наконец, любые команды, хранящиеся в переменной окружения `EXINIT`, выполняются при запуске. Настройки в `EXINIT` имеют приоритет над настройками в файле домашнего каталога `.exrc`.

Команда :set

Существует два типа параметров, которые можно изменить с помощью команды `:set`: параметры переключателя, которые либо включены, либо выключены, и параметры, принимающие числовое или строковое значение (такие как местоположение поля или имя файла).

Параметры переключателя могут быть включены или выключены по умолчанию. Команда для включения переключателя:

```
:set параметр
```

¹ Местоположение этой базы данных свое для каждого поставщика. Чтобы получить больше информации именно о вашей системе, попробуйте выполнить команду `man terminfo`.

Команда для выключения переключателя:

```
:set nonапараметр
```

Например, чтобы указать шаблонам поиска игнорировать регистр, введите:

```
:set ic
```

Если вы хотите, чтобы `vi` вернулся к режиму чувствительности к регистру при поиске, наберите:

```
:set noic
```

У многих параметров есть как полные названия, так и сокращения. Например, команда `ic` — это сокращение от `ignorecase`. Вы также могли бы ввести `set ignorecase` для игнорирования регистра и `noignorecase` для восстановления поведения по умолчанию.

Vim позволяет вам переключать значение параметра следующим образом:

```
:set параметр!
```

У некоторых параметров есть присвоенные им значения. Например, параметр `window` устанавливает число строк, показываемых в «окне» экрана. Чтобы установить значения для этих параметров, используйте знак равенства (=):

```
:set window=20
```

Во время сеанса редактирования вы можете проверить, какие параметры используются:

```
:set all
```

Команда отображает полный список параметров, включая установленные вами и заданные по умолчанию.

Дисплей должен выглядеть примерно так¹:

<code>autoindent</code>	<code>nomodelines</code>	<code>noshowmode</code>
<code>autoprint</code>	<code>nonumber</code>	<code>noslowopen</code>
<code>noautowrite</code>	<code>open</code>	<code>nosourceany</code>
<code>nobeautify</code>	<code>nooptimize</code>	<code>tabstop=8</code>
<code>directory=/var/tmp</code>	<code>paragraphs=IPLPPPQPP LIPplpipbp</code>	<code>taglength=0</code>

¹ Результат выполнения `:set` сильно зависит от вашей версии `vi`. Этот конкретный дисплей типичен для Unix `vi`. Параметры представлены в алфавитном порядке сверху вниз в столбцах, при этом игнорируется любое стоящее в начале `no`. В Vim есть гораздо больше параметров, чем показано здесь.

noedcompatible	prompt	tags=tags /usr/lib/tags
noerrorbells	noreadonly	term=xterm
noexrc	redraw	noterse
flash	remap	timeout
hardtabs=8	report=5	ttytype=xterm
noignorecase	scroll=11	warn
nolisp	sections=NHSHH HUnhsh	window=23
nolist	shell=/bin/bash	wrapscan
magic	shiftwidth=8	wrapmargin=0
mesg	showmatch	nowriteany

Вы можете найти текущее значение любого отдельного параметра по имени с помощью команды:

```
:set параметр?
```

Команда:

```
:set
```

показывает параметры, которые вы специально поменяли или установили либо в вашем файле `.exrc`, либо во время текущего сеанса. Например, изображение на экране может выглядеть следующим образом:

```
number sect=AhBhChDh window=20 wrapmargin=10
```

Файл `.exrc`

Файл `.exrc`, управляющий вашим окружением редактирования, находится в домашнем каталоге. Вы можете изменить файл `.exrc` с помощью Vim, как и любой другой текстовый файл (для применения настроек необходимо перезапустить Vim или явно прочитать файл заново с помощью команды `:source`).

Если у вас нет файла `.exrc`, просто создайте его. Введите в этот файл команды `set`, `ab` и `map`, которые вы хотите использовать при каждом редактировании (`ab` и `map` обсуждаются далее в этой главе). Файл `.exrc` может выглядеть следующим образом:

```
set nowrapscan wrapmargin=7
set sections=SeAhBhChDh nomesg
map q :w^M:n^M
ab ORA O'Reilly Media, Inc.
```

Поскольку файл фактически считывается `ex` до перехода в визуальный режим (`vi`), перед командами `.exrc` не обязательно добавлять двоеточие.

Переменные окружения

Помимо считывания файла `.exrc` в вашем домашнем каталоге, редактор будет читать файл с именем `.exrc` в текущем каталоге. Это позволяет вам устанавливать параметры, подходящие для конкретного проекта.

Во всех современных версиях `vi`, включая `Vim`, необходимо сначала установить параметр `exrc` в файле `.exrc` вашего домашнего каталога, прежде чем редактор будет читать файл `.exrc` в текущем каталоге:

```
set exrc
```

Эта опция не позволяет помещать файл `.exrc` в ваш рабочий каталог другим людям, чьи команды могут угрожать безопасности вашей системы¹.

Например, вам может понадобиться один набор параметров в каталоге, который в основном используется для программирования:

```
set number autoindent sw=4 terse
set tags=/usr/lib/tags
```

и другой набор параметров в каталоге, используемом для редактирования текста:

```
set wrapmargin=15 ignorecase
```

Обратите внимание, что вы можете установить определенные параметры в файле `.exrc` в вашем домашнем каталоге и сбросить их в локальном.

Допускается также задать переменные окружения редактирования, сохранив настройки параметра в файле, отличном от `.exrc`, и читая этот файл с помощью команды `:so` (`so` — сокращение от `source`).

Например:

```
:so .progoptions
```

Редактор не использует путь поиска, чтобы найти файлы для `:so`. Следовательно, считается, что имена файлов, которые не начинаются с `/`, относятся к текущему каталогу.

Локальные файлы `.exrc` также полезны для определения сокращений и переназначений клавиш (описаны далее в этой главе). Авторы, использующие язык разметки при написании книги или другого документа, могут с легкостью сохранить

¹ Исходные версии `vi` автоматически считывали оба файла, если они существовали. Параметр `exrc` закрывает потенциальную дыру в безопасности.

сокращения для последующего применения в этой книге, в файле `.exrc` в каталоге, в котором расположена книга.

Имейте в виду, что все файлы книги должны находиться в одном месте. Если они разнесены по разным каталогам, вам придется скопировать файл `.exrc` в каждый из них или, например, использовать функцию Vim `autocmd`, которая позволяет устанавливать параметры и осуществлять действия на основе расширения имени редактируемого вами файла. Это упрощает настройку редактирования одним способом, скажем, для DocBook XML и другим способом, например, для AsciiDoc или LaTeX. См. подраздел «Автокоманды» в главе 12.

Некоторые полезные параметры

Набрав команду `:set all`, вы обнаружите, что существует невероятное множество параметров, которые можно установить. Многие из них используются внутри редактора и, как правило, не изменяются. Другие важны лишь в определенных случаях (например, `noredraw` и `window` могут быть полезны во время межконтинентального сеанса `ssh`). В табл. Б.1 в разделе «Параметры Heirloom и Solaris vi» приложения Б вы найдете краткое описание каждого параметра. Не жалейте времени на то, чтобы протестировать параметры. Если параметр привлекателен для вас, попробуйте установить (или сбросить) его и посмотрите, что произойдет во время редактирования. Вы можете найти некоторые неожиданно полезные инструменты. Как уже обсуждалось ранее в подразделе «Перемещение по строке» в главе 2, параметр `wrapmargin` важен при редактировании непрограммного текста. `wrapmargin` задает размер правого поля, которое используется для автоматического переноса текста на новую строку при вводе (это позволяет избежать ручного ввода возврата каретки)¹. Обычное значение поля — от 7 до 15:

```
:set wrapmargin=10
```

Остальные три параметра управляют действиями редактора при осуществлении поиска. Обычно поиск различает верхний и нижний регистры (*foo* не сопоставляется с *Foo*), переходит в начало файла (означает, что вы можете начать поиск в любой точке файла и все равно найти все вхождения) и распознает подстановочные знаки при совпадении. К настройкам по умолчанию, которые управляют этими параметрами, относятся `ignorecase`, `wrapscan` и `magic` соответственно. Чтобы их изменить, необходимо установить противоположные параметры переключения:

¹ На компьютере ввод символа возврата каретки означает нажатие клавиши `Enter`. Термин произошел от пишущих машинок, на которых после окончания строки использовался выравниватель для прокрутки бумаги вверх на одну строку и возврата каретки (часть, удерживающая бумагу) обратно в начало строки. Это объясняет и происхождение символов ASCII LF и CR (linefeed (перевод строки) и carriage return (возврат каретки)).

`noignorecase`, `nowrapscan` и `nomagic`. Особенно интересными для программиста параметрами могут быть `autoindent`, `expandtab`, `list`, `number`, `shiftwidth`, `showmatch` и `tabstop`, а также их противоположности.

И наконец, немного о параметре `autowrite`: когда он задан, редактор автоматически осуществляет контрольное считывание всего содержимого измененного буфера при запуске команды `:n` (`next`), чтобы перейти к следующему файлу для редактирования и перед выполнением консольной команды с помощью `!`.

Выполнение команд Unix

Во время редактирования вы можете отображать или читать результаты любой команды Unix. Восклицательный знак (`!`) указывает `ex` создать оболочку и интерпретировать следующий за ним текст как команду Unix:

```
:!команда
```

Поэтому, если вы редактируете и хотите перепроверить текущий каталог, не выходя из `vi`, введите:

```
:!pwd
```

На вашем экране появится полный путь текущего каталога. Нажмите `Enter`, чтобы продолжить редактирование с того же места, на котором закончили.

Если вы хотите задать несколько команд Unix подряд, не возвращаясь между ними к сеансу редактирования, вы можете создать оболочку со следующей командой `ex`:

```
:sh
```

Для выхода из оболочки и возврата в `vi` нажмите `Ctrl+D` (это работает даже из `gvim`, версия Vim GUI).

Допускается сочетание `:read` с вызовом оболочки для считывания результатов команды Unix в файл. Очень простой пример:

```
:read !date
```

или покороче:

```
:r !date
```

считывает информацию о дате, установленной в системе, в текст вашего файла. Передавая команде `:r` адрес строки, вы можете считать результат команды с любой точки вашего файла. По умолчанию он вводится после текущей строки.

Предположим, вы редактируете файл и хотите считать четыре телефонных номера из файла с именем `phone`, но в алфавитном порядке. `phone` считывает:

```
Willing, Sue 333-4444
Walsh, Linda 555-6666
Quercia, Valerie 777-8888
Dougherty, Nancy 999-0000
```

Команда:

```
:r !sort phone
```

считывает содержимое файла `phone` после того, как оно было передано с помощью фильтра `sort`:

```
Dougherty, Nancy 999-0000
Quercia, Valerie 777-8888
Walsh, Linda 555-6666
Willing, Sue 333-4444
```

Допустим, вы редактируете файл и хотите вставить текст из другого файла в каталоге, но не можете вспомнить имя нового файла. Вы *могли бы* пойти длинным путем: выйти из файла, задать команду `ls`, записать правильное имя файла, вновь открыть файл и найти нужное место.

Или вы могли бы решить эту задачу за меньшее количество шагов.

Команда	Результат
<code>:!ls</code>	<pre>file1 file2 letter newfile practice</pre> Отобразить список файлов в текущем каталоге. Записать правильное имя файла. Нажать <code>Enter</code> для продолжения редактирования
<code>:r newfile</code>	<pre>"newfile" 35L, 1569C 2,1 Top</pre> Читать в новом файле



Один из авторов очень часто сочетает команду `г` с `%` в качестве имени файла, чтобы упростить исправление орфографических ошибок в документе:

```
:w
:$_r !spell %
```

Эта команда сохраняет файл и затем считывает выходные данные команды `spell` файла в буфер в конце (в некоторых системах вы можете использовать `:r !spell % | sort -u`, чтобы получить отсортированный список неправильно написанных слов).

Имея список слов с орфографическими ошибками в буфере, наш автор затем просматривает каждое из них по очереди, осуществляя поиск в файле и исправляя ошибки, удаляя каждое слово из списка по завершении работы с ним.

Фильтрация текста с помощью команды

Вы можете отправить фрагмент текста в качестве стандартных входных данных в команду Unix. Они заменяют фрагмент текста в буфере. Имеется возможность фильтровать текст с помощью команды либо из `ex`, либо из `vi`. Основное различие между этими двумя методами заключается в том, что в `ex` вы указываете фрагмент текста с помощью адресов строк, а в `vi` — с помощью текстовых объектов (команды перемещения).

Фильтрация текста в `ex`

Первый пример демонстрирует, как фильтровать текст с помощью `ex`. Предположим, что список имен из предыдущего примера расположен не в отдельном файле с именем `phone`, а уже содержится в текущем файле с 96-й по 99-ю строку. Необходимо просто ввести адрес строк, которые вы хотите отфильтровать, сопроводив его восклицательным знаком и консольной командой для выполнения. Например, команда:

```
:96,99!sort
```

передает строки с 96-й по 99-ю с помощью фильтра `sort` и заменяет их выходными данными `sort`.

Фильтрация текста с помощью команд перемещения `vi`

В режиме `vi` есть возможность отфильтровать текст с помощью команды Unix, введя восклицательный знак, сопровождаемый любой командой перемещения `vi`, которая указывает на фрагмент текста, а затем запустив командную строку оболочки для выполнения. Например:

```
!)команда
```

передает следующее предложение с помощью *команды*.

Существует несколько нюансов в действиях `vi` при использовании этой функции.

- Восклицательный знак не появляется сразу на экране. Когда вы вводите команду для текстового объекта, который хотите отфильтровать, восклицательный знак появляется внизу экрана, *но символ, который вы вводите для ссылки на объект, не отображается*.
- Фрагменты текста должны содержать минимум две строки, чтобы вы могли использовать только команды, которые перемещают несколько строк (`G`, `{` },

(), [[]], +, -). Для повторения операции можно задать число перед восклицательным знаком или текстовым объектом (например, и !10+, и 10!+ указывают на следующие десять строк). Такие объекты, как w, не работают, если их количество не превышает одну строку. Чтобы указать объект, вы также можете использовать слеш (/), сопровождаемый шаблоном и возвратом каретки. Это действие переносит текст в шаблон в качестве входных данных для команды.

- Затрагиваются все строки. Например, если ваш курсор расположен посередине строки и вы запускаете команду, чтобы перейти в конец следующего предложения, изменяются все строки, содержащие конец и начало предложения, а не только само предложение¹.
- Существуют специальные текстовые объекты, которые допускается использовать только с этим синтаксисом команды. Вы можете указать текущую строку, введя второй восклицательный знак:

!!команда

Не забывайте, что для повторения операции число может предшествовать либо всему предложению, либо текстовому объекту. К примеру, чтобы изменить строки с 96-й по 99-ю, как в предыдущем примере, достаточно поместить курсор на строку 96 и ввести либо 4!!sort, либо !4!sort. Или, предположим, у вас есть часть текста в файле, в которой вы хотите преобразовать строчные буквы в заглавные. Вы могли бы обработать эту часть с помощью команды tr, чтобы изменить регистр. Здесь второе предложение является фрагментом текста, который фильтруется с помощью команды:

4!!sort

или

!4!sort

В качестве другого примера предположим, что у вас есть некоторый текст в файле, буквы которого вы хотите изменить со строчных на прописные. Вы можете выполнить эту задачу с помощью команды tr, чтобы сменить регистр. В примере ниже второе предложение — это фрагмент текста, который нужно отфильтровать следующим образом:

Предыдущее предложение.

With a screen editor you can scroll the page
move the cursor, delete lines, insert characters,

¹ Конечно же, всегда есть исключение. В этом примере Vim меняет только текущую строку.

and more, while seeing the results of your edits
as you make them.
Следующее предложение.

Команда	Результат
!)	One sentence after. ~ ~ ~ .,.+4!■ Номера строк и восклицательный знак появляются в последней строке, чтобы пригласить вас в консольную команду. Символ) указывает, что предложение является фрагментом текста для фильтрации
tr '[:lower:]' '[:upper:]'	Предыдущее предложение. WITH A SCREEN EDITOR YOU CAN SCROLL THE PAGE MOVE THE CURSOR, Delete LINES, Insert CHARACTERS, AND MORE, WHILE SEEING THE RESULTS OF YOUR EDITS AS YOU MAKE THEM. Следующее предложение. Введите консольную команду и нажмите Enter. Входные данные замещаются выходными

Синтаксис для повтора предыдущей команды:

! *объект* !

При редактировании текста для электронного письма может оказаться полезным отфильтровать ваше сообщение с помощью программы `fmt`, чтобы «украсить» его перед отправкой. Помните, что «исходные» входные данные замещаются выходными. К счастью, если произойдет ошибка, например, отправится ошибочное сообщение вместо ожидаемых выходных данных, вы можете отменить команду и восстановить строки.

Сохранение команд

Бывает, что приходится печатать в файле одни и те же длинные фразы снова и снова. Существует несколько различных способов сохранения длинных последовательностей команд как в командном режиме, так и в режиме ввода текста. Когда вы вызываете одну из этих сохраненных последовательностей для ее выполнения, вам нужно лишь ввести несколько символов (или даже один), и вся последовательность будет выполнена, как если бы вы ввели всю команду целиком.

Сокращение слова

Вы можете опередить аббревиатуры, которые редактор автоматически расширит до целого текста, когда вы введете их в режиме ввода. Чтобы определить сокращение, используйте команду `ex`:

```
:ab сокращение фраза
```

сокращение является условным обозначением для указанной *фразы*. Последовательность символов, создающая сокращение, разворачивается в режиме ввода, только если вы вводите ее как целое слово. Сокращение не разворачивается, если расположено внутри слова.

Предположим, в файле `practice` требуется ввести текст, содержащий часто повторяющуюся фразу, например сложное имя продукта или компании. Команда:

```
:ab imrc International Materials Research Center
```

сокращает *International Materials Research Center* до *imrc*. Теперь каждый раз, когда вы вводите `imrc` в режиме ввода, он разворачивается до целого выражения.

Команда	Результат
<code>ithe imrc</code>	the International Materials Research Center

Сокращения разворачиваются, как только вы нажмете на алфавитно-цифровой символ (например, знак пунктуации), пробел, возврат каретки или `Esc` (возвращение в командный режим). При выборе сокращения отдавайте предпочтение тем сочетаниям символов, которые обычно не встречаются, когда вы вводите текст. Если вы создали сокращение, которое разворачивается в неожиданных местах, отключите его, набрав:

```
:unab сокращение
```

(Сокращенная версия разворачивается при нажатии клавиши `Enter` для команды `:unab`, но не переживайте, она по-прежнему отключена правильно.) Чтобы отобразить список ваших текущих сокращений, введите:

```
:ab
```

Символы, составляющие ваше сокращение, не могут также отображаться в конце вашей фразы. Например, если вы запустите команду:

```
:ab PG This movie is rated PG
```

`vi` выдаст сигнал `No tail recursion` и условное обозначение не будет установлено. Это сообщение обозначает, что вы задали нечто, что будет разворачиваться многократно, создавая бесконечный цикл. Если вы запустите команду:

```
:ab PG the PG rating system
```

предупреждение не появится.

При проверке мы получили следующие результаты этих версий `vi`:

- *Solaris /usr/xpg7/bin/vi u Heirloom vi* — хвостовая рекурсивная версия не допускается, а версия с именем в середине разворачивания разворачивается только раз;
- *Vim* — обе формы выявляются и разворачиваются только раз.

Если вы используете `vi` Unix, проверьте текущую версию, прежде чем повторять ваше сокращение как часть определяемой фразы.

Команда `map`

Если во время редактирования вы часто используете определенную (или очень сложную) последовательность команд, то отличным решением будет присвоить ей неиспользуемую клавишу с помощью команды `map`, чтобы сократить количество нажимаемых клавиш или время, затрачиваемое на запоминание последовательности.

Команда `map` действует почти так же, как `ab`, за исключением того, что вы определяете макрос для командного режима вместо режима ввода.

- `:map x последовательность` — определяет символ `x` как *последовательность* команд редактирования.
- `:unmap x` — отключает *последовательность*, определенную для `x`.
- `:map` — составляет список текущих переназначаемых символов.

Перед тем как вы начнете создавать свои собственные переназначения, необходимо познакомиться с клавишами, которые `vi` не использует в командном режиме и которые доступны для пользовательских команд:

- *буквы* — `g`, `K`, `q`, `V` и `v`;
- *клавиши управления* — `^A`, `^K`, `^O`, `^W` и `^X`;
- *символы* — `_`, `*`, `\` и `=`.

`Vim` использует все эти символы, за исключением `^K`, `^_` и `\`.



Символ `=` используется `vi`, если установлен режим `Lisp`, а также для форматирования текста с помощью `Vim`. Во многих современных версиях `vi` символ `_` эквивалентен команде `^`, и у `Vim` есть «визуальный режим», который использует ключи `v`, `V` и `^V`. (См. подраздел «Перемещение в визуальном режиме» главы 8.) Мораль такова — тщательно проверяйте свою версию.

В зависимости от вашего терминала вы также можете связать переназначенные последовательности с помощью специальных функциональных клавиш. Допускается также переназначить клавиши, которые уже используются в командном режиме, но в этом случае вы утратите доступ к их первоначальным функциям (мы приведем пример позднее, в подразделе «Дополнительные примеры переназначения клавиш»). В разделе «Несколько удобных переназначений» главы 14 представлены некоторые дополнительные, более сложные примеры переназначения.

С помощью переназначений можно создавать простые или сложные последовательности команд. В качестве простого примера зададим команду для инверсии порядка слов. В `vi` с курсором, расположенным следующим образом:

```
you can the scroll page
```

последовательностью команд, которая переместит *the* после *scroll*, будет `dwelp`: удалить слово — `dw`, перейти в конец следующего слова — `e`, переместиться на один пробел вправо — `l`, поместить туда удаленное слово — `p`. Сохранение этой последовательности:

```
:map v dwelp
```

позволит поменять местами два слова в любой момент сеанса редактирования с помощью одного нажатия клавиши `V`.

Переназначение с помощью выноски

В `Vim` практически каждая клавиша задействована для чего-то. Эта особенность может вызвать трудности при принятии решения о переназначении клавиш. Поэтому `Vim` предоставляет альтернативный способ, позволяя задавать переназначение с помощью специальной переменной `mapleader`. По умолчанию значением `mapleader` является `\` (обратный слеш).

Теперь вы можете задать переназначение клавиш, не выбирая неясные неиспользуемые клавиши `Vim` или не жертвуя существующей функциональностью `Vim`. При определении с помощью `mapleader` вы просто вводите символ выноски, а затем клавишу, которую хотите задать.

Например, вы хотите создать мнемосхему для *закрытия* Vim и выбираете *q* (от слова *quit*), как легко запоминаемую клавишу, но при этом вы хотите сохранить *q* в качестве начального символа для определенных многосимвольных команд Vim (например, команда *qq* для начала записи макроса). Чтобы это сделать, переназначьте *q* с помощью выноски:

```
:map <leader>q :q<cr>
```

Теперь команду *ex :quit* можно выполнить с помощью клавиш *\q*.

Установите любое значение *mapleader*, если вам не нравится использовать символ **. Поскольку *mapleader* является переменной Vim, синтаксис будет следующий:

```
:let mapleader="X"
```

где *X* — выбранный вами символ выноски.

Защита клавиш от интерпретации ex

Обратите внимание, что при определении переназначения нельзя просто ввести определенные клавиши, такие как **Enter**, **Esc**, **Backspace** и **Delete**, как часть команды для переназначения, поскольку они уже имеют значение в *ex*. Если вы хотите включить какую-либо из этих клавиш в последовательность команд, необходимо экранировать обычное значение клавиши, добавив перед ней **Ctrl+V**. Клавиша **^V** отобразится в схеме как символ **^**. Следующие за **^V** символы также не будут отображаться ожидаемым образом. Например, возврат каретки отображается как **^M**, выход — как **^I**, пробел — как **^N** и т. д.

С другой стороны, если вы хотите использовать управляющий символ в качестве переназначаемого, то в большинстве случаев вам для этого потребуется, удерживая **Ctrl**, нажать буквенную клавишу. Так, к примеру, для переназначения **^A** нужно всего лишь ввести:

```
:map Ctrl-A последовательность
```

Однако существует три управляющих символа, которые необходимо экранировать с помощью **^V**., **^T**, **^W** и **^X**. Так, если вы, например, хотите переназначить **^T**, вы должны ввести:

```
:map Ctrl-V Ctrl-T последовательность
```

Использовать **Ctrl+V** допускается в любой команде **ex**, а не только в **map**. Это значит, что вы можете ввести возврат каретки в сокращении или команде замены. Например, сокращение:

```
:ab 123 one^Mtwo^Mthree
```

разворачивается до:

```
one
two
three
```

Здесь мы продемонстрировали последовательность **Ctrl+V Enter** в виде **^M**, как она отображалась бы на вашем экране (Vim выделяет **^M** другим цветом, так что вы сможете понять, что это на самом деле управляющий символ).

Вы также можете глобально добавить строки в определенные места. Команда:

```
:g/^Section/s//As you recall, in^M&/
```

вставляет перед всеми строками, начинающимися со слова *Section*, фразу на отдельной строке. Символ **&** восстанавливает шаблон поиска.

К сожалению, в командах **ex** у одного символа всегда есть специальное значение, даже если вы попытаетесь заключить его в кавычки с помощью **Ctrl+V**. Напомним: вертикальная черта (**|**) играет роль разделителя нескольких команд **ex**. Вы не можете использовать ее в переназначениях режима ввода.

Теперь, познакомившись с использованием **Ctrl+V** для защиты определенных клавиш внутри команд **ex**, вы готовы задавать некоторые сложные последовательности переназначений.

Пример сложного переназначения

Предположим, что у вас есть глоссарий со следующими записями:

```
map - an ex command which allows you to associate
a complex command sequence with a single key.
```

И перед вами стоит задача преобразовать этот список в пользовательский формат XML, чтобы он выглядел следующим образом:

```
<glossaryitem>
<name>map</name>
<para>An ex command...
```

Лучший способ определения сложного переназначения — осуществить правку вручную один раз, записав каждое нажатие клавиши, а затем воссоздать эти комбинации в виде схемы. Вам потребуется выполнить такую последовательность действий.

1. Вставьте тег `<glossaryitem>`, новую строку и тег `<name>`.
2. Нажмите **Esc**, чтобы выйти из режима ввода.
3. Перейдите в конец первого слова (**e**) и добавьте тег `</name>`, новую строку и тег `<para>`.
4. Нажмите **Esc**, чтобы выйти из режима ввода.
5. Переместитесь вперед на один символ, подальше от закрывающего символа `>` (1).
6. Удалите пробел, дефис и следующие пробелы (3x) и преобразуйте символы следующего слова из строчных в заглавные (~).

Если вы будете повторять эту последовательность не один раз, то потратите довольно много времени и усилий.

С помощью команды `:map` можно сохранить всю последовательность таким образом, чтобы в дальнейшем выполнять ее лишь одним нажатием клавиши:

```
:map g I<glossaryitem>^M<name>^[ea</name>^M<para>^[13x~
```

Обратите внимание, что необходимо «заключить в кавычки» символы **Esc** и **Enter** с помощью **Ctrl+V**. `^[` — это последовательность, которая появляется при вводе **Ctrl+V**, за которым следует **Enter**. `^M` — это последовательность, отображаемая при вводе **Ctrl+V Enter**.

Теперь вся серия правок выполняется просто нажатием клавиши **g**. При медленном соединении вы можете фактически увидеть, как каждая правка осуществляется отдельно. При быстром это выглядит как волшебство.

Не отчаивайтесь, если ваша первая попытка переназначения клавиш окажется неудачной. Незначительная ошибка при определении переназначения может очень сильно повлиять на ожидаемые результаты. Чтобы отменить правку, введите **u** и затем попытайтесь снова. (Это именно то, что нам пришлось делать во время разработки данной схемы.)

Некоторые команды, для которых ввод схемы упрощает правки XML, рассмотрены в подразделе «Переназначение нескольких клавиш ввода» далее.

Дополнительные примеры переназначения клавиш

В следующих примерах рассмотрены возможные варианты сочетаний клавиш при определении переназначений клавиатуры.

1. Добавлять текст при каждом перемещении в конец слова:

```
:map e ea
```

В большинстве случаев единственная причина, по которой вы вынуждены переместиться в конец слова, — добавление текста. Эта последовательность переназначения автоматически переводит вас в режим ввода. Обратите внимание, что у переназначенной клавиши **e** есть значение в **vi**. Вы можете задать для нее другую команду, но учтите, что обычный функционал клавиши будет недоступен, пока действует переназначение. В данном случае это не так плохо, поскольку команда **E** часто идентична **e**.

2. Поменять местами два слова:

```
:map K dwElp
```

Мы уже немного обсуждали данную последовательность ранее, только теперь вам нужно использовать **E** (предположим, что для этого и оставшихся примеров команда **e** переназначена на **ea**). Помните, что курсор размещен на первом из двух слов. К сожалению, из-за команды **1** эта последовательность (и предыдущая версия) не срабатывает, если два слова находятся в конце строки: при работе с последовательностью курсор оказывается в конце строки, а **1** не может двигаться дальше вправо.

3. Сохранить файл и отредактировать следующий в очереди:

```
:map q :w^M:n^M
```

(Используйте **Ctrl+V Enter**, чтобы получить **^M** в схеме.) Обратите внимание, что вы можете переназначить клавиши на команды **ex**, но обязательно завершайте каждую команду **ex** возвратом каретки. Эта последовательность упрощает переход от одного файла к другому и полезна, если вы открыли много коротких файлов одной командой **vi**. Переназначение буквы **q** поможет вам запомнить, что последовательность схожа с *quit*¹.

4. Выделить слово с помощью кодов **troff**:

```
:map v i\fb^[e\fbP^[
```

Убедитесь, что курсор находится в начале слова. Сначала вы входите в режим ввода, а затем набираете код для полужирного шрифта. В командах переназначения вам не нужно вводить два обратных следа, чтобы получить одну. Затем

¹ Для осуществления этой комбинированной операции Vim предоставляет команду **:wn**, но **:map q :wn^M** все равно будет полезна.

вы возвращаетесь в командный режим, введя «закавыченный» **Esc**. И наконец, добавляете закрывающий код **troff** в конец слова и возвращаетесь в командный режим.

Обратите внимание, что при добавлении в конец слова мы не использовали **ea**, поскольку для этой последовательности уже переназначена буква **e**. Это демонстрирует, что переназначенные последовательности могут содержать другие переназначенные команды. Данная возможность контролируется опцией **remap**, которая обычно включена.

5. Выделить слово с помощью кодов HTML, даже если курсор не стоит в начале слова:

```
:map V lbi<B>^[e</B>^[
```

Эта последовательность схожа с предыдущей. Помимо HTML, вместо **troff** в ней также используется **lb** для обработки дополнительных задач размещения курсора в начало слова. Курсор может быть расположен в середине слова, поэтому следует переместить его в начало командой **b**. Но если курсор находится уже в начале слова, **b** переместит его к предыдущему. Чтобы предотвратить это, введите **l**, перед тем как перейти назад с помощью **b**, чтобы курсор никогда не располагался на первой букве слова. Вы можете задать варианты данной последовательности, заменив **b** на **B**, а **e** на **Ea**. Тем не менее во всех случаях команда **l** не позволяет последовательности сработать, если курсор находится в конце строки. (Добавьте пробел, чтобы обойти это.)

6. Находить и удалять круглые скобки, в которые заключены слово или фраза¹:

```
:map = xf)xp
```

Данная последовательность предполагает, что вы сперва находите открытые круглые скобки, набрав **/** (с последующим **Enter**).

Чтобы убрать круглые скобки, используйте команду **map**: удаляем открытые круглые скобки с помощью **x**, находим закрывающие с помощью **f**), снова удаляем (**x**), а затем повторяем наш поиск для открытых круглых скобок с помощью **p**.

Если нет необходимости удалять круглые скобки (например, они нужны в конкретном месте), не используйте команду переназначения: просто нажмите **p**, чтобы найти следующие открывающие круглые скобки.

Вы можете также изменить переназначенную последовательность в этом примере, чтобы обработать сопоставимые пары кавычек.

7. Закомментируйте строку в стиле C/C++:

```
:map g I/* ^[A */^[
```

¹ Из статьи Уолтера Зинца «Советы по vi для опытных пользователей» в *Unix World*, апрель 1990. Это предназначено для заключенного в круглые скобки текста, а не для вложенных, заключенных в круглые скобки равенств.

Последовательность вставляет /* в начало строки и добавляет */ в конец строки. Вы можете также переназначить команду замены, чтобы она выполняла то же самое действие:

```
:map g :s;.*;/* & */;^M
```

Здесь вы переназначаете всю строку (с помощью .*), и когда вы воспроизводите ее (с помощью &), то заключаете строку в символы комментария. Обратите внимание, что использование точки с запятой в качестве разделителя позволяет избежать экранирования символов / в комментариях.

В заключение вам следует знать, что существует много клавиш, которые либо выполняют те же задачи, что и другие, либо редко используются. (Например, ^J действует так же, как и j.) Однако, прежде чем смело отключать обычное использование команд vi в определении переназначения, вам следует хорошо их изучить.

Переназначение клавиш для режима ввода текста

Как правило, переназначение применяется только в командном режиме. В конце концов, в режиме ввода клавиши означают сами себя и их не стоит переназначать в качестве команд. Однако, добавив восклицательный знак (!) в команду map, вы можете заставить ее обойти значение клавиши по умолчанию и осуществить переназначение в режиме ввода. Данная функция полезна в случае, когда вы находитесь в режиме редактирования текста, но вам нужно ненадолго перейти в командный режим, запустить команду, а затем вернуться в режим ввода. Например, предположим, что вы только что напечатали слово, но забыли выделить его курсивом (или поместить его в кавычки и т. д.). Вы можете задать следующее переназначение:

```
:map! + ^[bi<I>^[ea</I>
```

Теперь, когда вы вводите + в конце слова, вы заключаете слово в коды HTML для курсива. Символ + не появится в тексте.

Только что показанная последовательность переходит в командный режим (^[), выполняет резервное копирование для вставки первого кода (bi<I>), снова переходит в командный режим (^[) и перемещается дальше, чтобы вставить второй код (ea<I>). Поскольку последовательность переназначения начинается и заканчивается в режиме ввода, вы можете продолжить печатать текст после выделения слова.

Рассмотрим еще один пример: предположим, что во время ввода текста вы обнаружили, что предыдущая строка должна была закончиться двоеточием. Чтобы исправить это, задайте следующую последовательность переназначения¹:

```
:map! % ^[kA:^[jA
```

¹ Из «Советов по vi для опытных пользователей».

Теперь, если ввести % в любом месте текущей строки, вы добавите двоеточие в конец предыдущей. Эта команда включает командный режим, поднимается на строку выше и добавляет двоеточие (^[kA:). Затем снова переходит в командный режим, спускается на первоначальную строку и оставляет вас в конце строки в режиме редактирования (^[jA).

Обратите внимание, что для предыдущих команд переназначения мы использовали обычные символы (+ и %). Когда символ переназначается для режима ввода, вы больше не можете печатать его как текст (если только перед ним нет Ctrl+V).

Чтобы восстановить обычный ввод символа, используйте команду:

```
:unmap! x
```

где x — это символ, который был ранее переназначен для режима редактирования текста. (Хотя редактор разворачивает x в командной строке по мере ввода, создавая впечатление, что вы отменяете переназначение развернутого текста, он отменяет его правильно.)

Переназначение в режиме ввода больше подходит для привязки символьных строк к специальным клавишам, которые вы не используете другим образом. Оно особенно полезно для программируемой клавиатуры, о чем мы поговорим далее.

Переназначение функциональных клавиш программируемой клавиатуры

Раньше терминалы снабжались программируемой клавиатурой. С помощью специального режима вы могли настроить функциональные клавиши для отправки любого необходимого символа или символов. Прикладные программы затем могли воспользоваться этими клавишами, применяя их в качестве «ярлыков» для общих или специальных действий.

У современных компьютеров и ноутбуков тоже есть функциональные клавиши, как правило 10 или 12 клавиш в верхнем ряду, обозначенные с F1 по F12. Вместо того чтобы настраивать их поведение с помощью специального аппаратного режима, они задаются эмуляторами терминала и другими программами, запущенными в вашей системе.

Поскольку у эмуляторов терминала есть записи в базе данных `terminfo`, редактор способен распознать эскап-последовательности, созданные функциональными клавишами, позволяя вам переназначать их для специальных действий, если необходимо. Это осуществляется в `ex` с помощью синтаксиса:

```
:map #1 команды
```

для функциональной клавиши номер 1 и т. д.

Как и в случае с другими клавишами, переназначение применяется по умолчанию в командном режиме, но с помощью команды `map!` вы можете также задать два различных значения для функциональной клавиши: одно для использования в командном режиме, а другое — для режима ввода. Например, работая с HTML, вам может понадобиться поместить коды переключения шрифтов для функциональных клавиш, например¹:

```
:map #2 i<I>^[
:map! #2 <I>
```

Если вы находитесь в командном режиме, первая функциональная клавиша переводит в режим ввода, печатает три символа `<I>` и снова включает командный режим. Если вы уже находитесь в режиме редактирования текста, клавиша просто вводит три символа HTML-кода. Если последовательность содержит `^M`, что является возвратом каретки, нажмите `Ctrl+V Ctrl+M`.

К примеру, чтобы клавиша `F2` стала доступна для переназначения, в базе данных вашего терминала должно быть определение `k2`, такое как:

```
k2=^A@^M
```

В свою очередь, символы из определения:

```
^A@^M
```

должны выводиться при нажатии этой клавиши.

Просмотр списка функциональных клавиш и закрепленных за ними действий

Чтоб просмотреть назначение функциональной клавиши, используйте команду `od` (octal dump) с параметром `-c` (отобразить каждый символ). Для этого нажмите `Enter` после функциональной клавиши, затем `Ctrl+D`, чтобы `od` вывел информацию на экран. Например:

```
$ od -c          od читает со стандартного устройства ввода
^[[[A           нажата функциональная клавиша
^D              Ctrl + D, EOF
0000000 033    [    [    A  \n
0000005
```

Здесь функциональная клавиша отправляет `Escape`, две левые квадратные скобки и `A`.

¹ За функциональной клавишей `F1` часто скрывается функция «справка». Данная клавиша зарезервирована для использования эмулятором терминала, соответственно, в нашем примере мы используем `F2`.

Переназначение других специальных клавиш

У многих клавиатур есть специальные клавиши, такие как **Home**, **End**, **Page Up** и **Page Down**, которые воспроизводят команды **vi**. Если описание **terminfo** вашего эмулятора терминала полное, редактор распознает эти клавиши. Но если оно не завершено, используйте команду **map**, чтобы сделать клавиши доступными. Обычно эти клавиши отправляют компьютеру **escape**-последовательность: символ **Escape**, за которым следует строка из одного и более символов. Чтобы прервать **Escape**, вам нужно нажать **^V** перед нажатием специальной клавиши в переназначении. Например, чтобы переназначить **Home** на подходящий эквивалент **vi**, вы можете задать следующее переназначение:

```
:map Ctrl-V Home 1G
```

Это отображается на вашем экране как:

```
:map ^[[H 1G
```

Похожие команды переназначения отображаются следующим образом¹:

:map Ctrl-V End G	отображает	:map ^[[Y G
:map Ctrl-V Page Up ^F	отображает	:map ^[[V ^F
:map Ctrl-V Page Down ^B	отображает	:map ^[[U ^B

Возможно, вы захотите поместить эти переназначения в ваш файл **.exrc**. Обратите внимание, что если специальная клавиша создает длинную **escape**-последовательность (содержащую множество непечатаемых символов), то **^V** берет в кавычки только исходный управляющий символ и переназначение не работает. Вам придется найти всю **escape**-последовательность (используя **od**, как было показано ранее) и ввести ее вручную, ставя кавычки в нужных местах, вместо того чтобы просто нажать **^V** и потом клавишу.

Если вы используете разные виды терминалов (например, командное окно в системе **Windows** и **xterm**), то, скорее всего, представленные выше переназначения не всегда будут работать. По этой причине **Vim** предоставляет компактный способ описания подобных переназначений клавиш:

```
:map <Home> 1G          Введите 6 символов: < H o m e > (Vim)
```

¹ То, что вы увидите на своем экране, скорее всего, будет отличаться от показанных здесь **escape**-последовательностей.



vi и Vim обычно предоставляют эти переназначения, как описано, а если нет, вы можете переназначить их указанным способом. Тем не менее нам кажется, что подобные переназначения противоречат философии vi «никогда не отходи от клавиатуры». Показывая пользователям, как использовать Vim, одной из первых вещей, которую мы делаем (или рекомендуем), является переназначение клавиш **Home**, **End**, **Page Up**, **Page Down**, **Insert** и **Delete**, а также всех клавиш с изображением стрелки на «без операции», чтобы побудить к изучению собственных команд vi. Конечным результатом является более эффективное редактирование и лучшая мышечная память реальных команд vi, а также набор клавиш, доступных для переназначения в других трудных задачах.

Например, один из нас регулярно редактирует файлы данных четырех магазинов мороженого, которыми он управляет. Данные содержат случайные строки с информацией по продажам, разделенные пробелами, для магазинов за день. Он использует функцию Vim **autocommand**, чтобы определить, редактирует ли он файл, соответствующий названию магазина. (Подробнее об автокомандах читайте в подразделе «Автокоманды» в главе 12.) И, в свою очередь, задает клавишу **End** для суммирования четырех значений и изменения строки, чтобы отобразить значения и итоги. Переназначение будет выглядеть следующим образом¹:

```
:noremap <end> !!awk 'NF == 4 && $1 + $2 + $3 + $4 > 0 {
    printf "%s total: $%.2f\n", $0, $1 + $2 + $3 + $4;
    exit }; { print $0 }'<cr>
```

Это одна длинная строка в файле `.vimrc` (мы разбили ее на несколько строк, чтобы она поместилась на странице). Итак, «тяжелая задача» заключается в использовании клавиши **End** в строке файла данных, которая выглядит следующим образом:

```
450 235 1002 499
```

и которая преобразуется в:

```
450 235 1002 499 total: $2186.00
```

при нажатии клавиши **End**. Обратите внимание, что в переназначении присутствует двойная проверка на достоверность с помощью команды **awk** для выполнения вычислений, но только при наличии четырех полей (`NF == 4`) и только если их сумма приводит к значению больше нуля. (Для получения информации о команде `:noremap` смотрите `:help :map-modes`.)

¹ Знак % нужно экранировать в переназначении, чтобы Vim не замещал его текущим именем файла.

Переназначение нескольких клавиш ввода

Переназначить можно не только функциональные клавиши. Допускается также переназначать последовательности обычных клавиш. Это поможет упростить введение некоторых типов текста, таких как DocBook XML и HTML.

Ниже представлены некоторые команды `:map`, которые упрощают ввод XML разметки DocBook (спасибо за них Джерри Пику). Строки, начинающиеся с двойных кавычек, являются комментариями (подробнее об этом — в подразделе «Комментарии в сценариях `ex`» далее):

```
:set noremap
" bold:
map! =b </emphasis>^[F<i<emphasis role="bold">
map =B i<emphasis role="bold">^[
map =b a</emphasis>^[
" Move to end of next tag:
map! =e ^[f>a
map =e f>
" footnote (tacks opening tag directly after cursor in text-input mode):
map! =f <footnote>^M<para>^M</para>^M</footnote>^[kO
" Italics ("emphasis"):
map! =i </emphasis>^[F<i<emphasis>
map =I i<emphasis>^[
map =i a</emphasis>^[
" paragraphs:
map! =p ^[j<para>^M</para>^[O
map =P O<para>^[
map =p o</para>^[
" less-than:
map! *l &lt;
...
```

Когда вы будете использовать эти команды, для добавления сноски вам нужно будет зайти в режим редактирования текста и ввести `=f`. Затем редактор вставит открывающие и закрывающие теги и оставит курсор между ними, включив режим ввода:

```
All the world's a stage.<footnote>
<para>█
</para>
</footnote>
```

Эти макросы доказали свою эффективность во время работы над прошлыми изданиями книги: они могут с легкостью подстроиться под различные языки разметки, такие как AsciiDoc, LaTeX, Texinfo или Sphinx.

@-функции

Именованные регистры предоставляют еще один способ создания *макросов* — сложных последовательностей команд, которые вы повторяете с помощью нажатия лишь нескольких клавиш.

Если ввести в текст командную строку (последовательность **vi** или команду **ex**, которой предшествует двоеточие) и затем удалить ее в именованный регистр, появится возможность выполнить содержимое этого регистра с помощью команды **@**. Например, откройте новую строку и введите:

```
cwgadfly Ctrl-V Esc
```

На вашем экране это отобразится как:

```
cgadfly^[
```

Снова нажмите **Esc**, чтобы выйти из режима ввода, а затем удалите строку в регистр **g**, набрав "**gdd**". Теперь каждый раз, когда вы будете помещать курсор в начало слова и вводить **@g**, это слово в вашем тексте будет меняться на *gadfly*¹.

Поскольку **@** интерпретируется как команда **vi**, то точка (.) повторяет всю последовательность, даже если регистр содержит команду **ex**. **@@** повторяет последнюю **@**, а **u** или **U** можно использовать для отмены результата **@**.

Vim упрощает запись текста в именованный регистр. В командном режиме **vi** команда **q**, сопровождаемая именем регистра, начинает запоминать, что вы вводите в именованный регистр. Используйте **q** саму по себе, чтобы завершить процедуру. Vim выводит сообщение в строку состояния о том, что он находится в режиме записи, чтобы вы об этом не забыли. Используя регистр **a** из предыдущего примера в Vim, вы бы ввели строку **qacwgadfly^[q**, которую потом можно выполнить с помощью **@a**.

Это простой пример; @-функции эффективны, потому что они могут быть адаптированы к очень специфическим командам. Они особенно полезны при редактировании нескольких файлов, потому что вы можете сохранить команды в их именованных регистрах и получить доступ к ним из любого редактируемого файла. @-функции также отлично сочетаются с командами глобальной замены, которые обсуждались в главе 6.

¹ Это немного сложно. **dd** также получает новую строку в конце строки, заставляя **@g** перемещать курсор на одну строку вниз после изменения текущего слова. Чтобы сделать это правильно, необходимо ввести "**gdf^V^[**". Фух!

Естественно, если вы используете именованные регистры и для @-функций, и для хранения извлеченного или удаленного текста, убедитесь, что разделили их, например используя первые буквы алфавита для хранения, а последние буквы алфавита — для @-функций.

Запуск регистров из ex

Вы также можете вызвать текст, сохраненный в регистре, из режима ex. В этом случае потребуется ввести команду ex, удалить ее в именованный регистр, а затем использовать команду @ из командной строки ex, начинающейся с двоеточия. Например, введите следующий текст:

```
ORA publishes great books.  
ORA is my favorite publisher.  
1,$s/ORA/O'Reilly Media/g
```

Поместив курсор на последнюю строку, удалите команду в регистр g: "gdd. Переместите курсор на первую строку: kk. Затем запустите регистр из командной строки с двоеточием: :@g Enter. Ваш экран должен выглядеть следующим образом:

```
O'Reilly Media publishes great books.  
O'Reilly Media is my favorite publisher.
```

Некоторые версии vi интерпретируют * как @ при использовании из командной строки ex. Vim также делает это, но только если установлен параметр compatible. Кроме того, если после команды @ или * указан символ регистра *, то команда будет взята из регистра по умолчанию (неименованного).

Использование сценариев ex

Определенные команды ex вы используете только из vi, такие как переназначения, сокращения и т. д. Если вы сохраните эти команды в вашем файле .exrc, они будут автоматически выполняться при запуске vi или Vim. Любой файл, содержащий команды для выполнения, называется *сценарием (скриптом)*.

Команды в типовом сценарии .exrc нельзя использовать за пределами vi. Однако можно сохранить другие команды ex в сценарий, а затем выполнить его для одного или нескольких файлов. По большей части в этих внешних скриптах используются команды замены.

Составителю технической документации сценарии ex нужны для обеспечения согласованности терминологии или даже орфографии во всем наборе документов.

Например, предположим, что вы запустили команду Unix `spell` для двух файлов и что эта команда вывела следующий список орфографических ошибок:

```
$ spell sect1 sect2
chmod
ditroff
myfile
thier
writeable
```

Как это часто бывает, `spell` пометил несколько технических терминов и частных примеров, которые он не распознал, но он также выявил две настоящие орфографические ошибки.

Поскольку проверилось сразу два файла, мы не знаем, в каком из них выявлены ошибки и где конкретно они находятся. Хотя для данного конкретного примера существуют несложные способы это выяснить, представьте, насколько трудоемкой будет подобная работа для человека, который плохо знает орфографию или проверяет много файлов за раз.

Чтобы упростить работу, вы можете записать сценарий `ex`, содержащий следующие команды:

```
%s/thier/their/g
%s/writeable/writable/g
wq
```

Предположим, что вы сохранили эти строки в файле с именем `exscript`. Скрипт можно выполнить из `vi` с помощью команды:

```
:so exscript
```

или применить его к файлу непосредственно из командной строки. Затем вы можете отредактировать файлы `sect1` и `sect2` следующим образом:

```
$ ex -s sect1 < exscript
$ ex -s sect2 < exscript
```

Символ `-s` (либо для `script mode`, либо для `silent mode`), следующий за вызовом `ex`, — это способ POSIX сообщить редактору о необходимости подавить обычные сообщения терминала¹.

Если бы сценарий оказался длиннее приведенного в нашем примере, мы сэкономили бы достаточное количество времени. Тем не менее возникает вопрос, нет ли способа избежать повторения процесса для каждого редактируемого файла. Конечно, мы можем написать скрипт оболочки, который включает (хоть и обобщенно) вызов `ex`, так что его можно использовать для любого количества файлов.

¹ Традиционно `ex` использовал один знак минус для этой цели. Как правило, для обеспечения обратной совместимости принимаются оба знака.

Зацикливание сценария оболочки

Возможно, вы знаете, что оболочка является не только языком программирования, но и интерпретатором командной строки. Чтобы вызвать `ex` для нескольких файлов, мы используем простой тип команды сценария оболочки под названием `for`. Цикл `for` позволяет применить последовательность команд для каждого аргумента, представленного в скрипте. Цикл `for`, вероятно, является наиболее полезной частью программирования оболочки для начинающих. Вам стоит его запомнить, даже если вы не собираетесь писать никакие другие программы оболочки.

Вот как выглядит синтаксис цикла `for`:

```
for переменная in список
do
    команда(команды)
done
```

Например:

```
for file in "$@"
do
    ex -s "$file" < exscript
done
```

(Для команды `ex` нет необходимости делать отступ, мы его сделали для лучшей читабельности.) Использование кавычек для расширения переменной `file` (`$file`) позволяет сценарию работать, даже если в именах файлов есть пробелы¹.

Создав сценарий оболочки, мы сохраняем его в файл с именем `correct` и делаем его исполняемым с помощью команды `chmod 755 correct`. Теперь введите следующее:

```
$ ./correct sect1 sect2
```

Цикл `for` в `correct` присваивает каждый аргумент (каждый файл в списке указан с помощью `"$@"`, то есть «все аргументы») переменной `file` (`$file`) и выполняет сценарий `ex` для содержимого данной переменной.

Возможно, будет проще понять, как работает цикл `for`, на примере, выходные данные которого более заметны. Давайте посмотрим на скрипт для переименования файлов:

```
for file in "$@"
do
    mv "$file" "$file.x"
done
```

¹ Пробелы в именах файлов не являются хорошей практикой, но и нередки. Хорошо написанные сценарии должны работать и для таких файлов.

Предполагая, что этот сценарий находится в исполняемом файле с именем *move*, мы можем сделать следующее:

```
$ ls
ch01 ch02 ch03 move
$ ./move ch??           Только файлы из главы
$ ls                   Проверяем результат
ch01.x ch02.x ch03.x move
```

Проявив творческий подход, вы могли бы переписать сценарий, чтобы переименовать файлы более точно:

```
for nn in "$@"
do
    mv "ch$nn" "sect$nn"
done
```

С написанным подобным образом скриптом вы указываете в командной строке числа вместо имен файлов:

```
$ ls
ch01 ch02 ch03 move
$ ./move 01 02 03
$ ls
sect01 sect02 sect03 move
```

Циклу *for* не нужно принимать "\$@" («все аргументы») в качестве списка значений для замены. Допускается также указать явный список. Например:

```
for переменная in a b c d
```

присваивает *переменные* последовательно: *a*, *b*, *c* и *d*. Вы также можете заменить выходные данные команды. Например:

```
for переменная in $(grep -l "Alcuin" *)
```

присваивает *переменную* имени каждого файла по очереди, где *grep* находит строку *Alcuin*. (*grep -l* выводит имена файлов, содержимое которых соответствует шаблону, не выводя при этом фактические сопоставимые строки.)

Если список не указан:

```
for переменная
```

переменная присваивается последовательно каждому аргументу командной строки, почти так же, как было в нашем изначальном примере. Последовательность из четырех символов "\$@" разворачивается до "\$1", "\$2", "\$3" и т. д. Знаки кавычек предотвращают дальнейшую интерпретацию специальных символов и сохраняют имена файлов с пробелами в виде одного элемента.

Вернемся к нашей основной цели и нашему исходному сценарию:

```
for file in "$@"
do
    ex -s "$file" < exscript
done
```

Может показаться не таким изысканным использовать два скрипта: сценарий оболочки и сценарий `ex`. На самом деле оболочка предоставляет способ включить редактируемый сценарий в сценарий оболочки, что мы сейчас и рассмотрим.

Встроенные документы

В сценарии оболочки оператор `<<` указывает принять следующие строки, вплоть до указанной строки, в качестве входных данных команды (это часто называется *встроенным документом*). С помощью следующего синтаксиса мы можем включить наши команды редактирования в файл `correct` таким образом:

```
for file in "$@"
do

    ex -s "$file" << end-of-script
    g/thier/s//their/g
    g/writeable/s//writable/g
    wq
end-of-script

done
```

Строка `end-of-script` является полностью произвольной: она должна быть строкой, которая не появится во входных данных и может быть использована оболочкой для распознавания завершения работы с данным документом. Она также *должна* быть помещена в начало строки. Согласно общепринятым правилам многие пользователи обозначают конец встроенного документа с помощью строки `EOF` или `E_O_F`, чтобы обозначить конец файла.

У каждого из приведенных подходов есть свои преимущества и недостатки. Если вы хотите внести разовую серию правок и не против каждый раз переписывать сценарий, то встроенный документ является эффективным инструментом для выполнения работы.

Однако более гибким является написание команд редактирования в отдельном от сценария оболочки файле. Например, вы могли бы установить соглашение о том, что команды редактирования всегда будут помещаться в файл с именем `exscript`. Тогда вам потребуется записать `correct` лишь раз. Вы можете хранить его в своем личном каталоге «Инструменты» (который вы добавили в ваш путь поиска) и использовать его по необходимости.

Сортировка фрагментов текста: пример сценария ex

Предположим, вы хотите отсортировать в алфавитном порядке содержимое файла глоссария с определениями, закодированными в пользовательской версии XML. Каждое определение содержится в элементе `<glossaryitem></glossaryitem>`. Название каждой записи хранится в элементе `<name></name>`. Файл глоссария выглядит примерно следующим образом:

```
<glossaryitem>
<name>TTY_ARGV</name>
<para>The command, specified as an argument vector,
that the TTY subwindow executes.</para>
</glossaryitem>
<glossaryitem>
<name>ICON_IMAGE</name>
<para>Sets or gets the remote image for icon's image.</para>
</glossaryitem>
<glossaryitem>
<name>XV_LABEL</name>
<para>Specifies a frame's header or an icon's label.</para>
</glossaryitem>
<glossaryitem>
<name>SERVER_SYNC</name>
<para>Synchronizes with the server once.
Does not set synchronous mode.</para>
</glossaryitem>
```

Вы можете расположить в алфавитном порядке содержимое файла, запустив строки из команды Unix `sort`, но построчное упорядочение не та задача. Вам требуется отсортировать только термины глоссария, перемещая каждое определение (нетронутым) вместе с соответствующим термином. Допускается объединять целый фрагмент текста в одну строку. Ниже представлена первая версия вашего сценария ex:

```
g/^<glossaryitem>/,/^</glossaryitem>/j
%!sort
wq
```

Каждая запись глоссария находится между тегами `<glossaryitem>` и `</glossaryitem>` (обратите внимание на использование `\` для экранирования слеша в закрывающей метке). `j` является командой ex для объединения строки (в режиме `vi` эквивалент `J`). Таким образом, первая команда объединяет все записи глоссария в одну «строку». Вторая команда затем сортирует файл, выводя следующие строки:

```
<glossaryitem> <name>ICON_IMAGE</name> <para>Sets ... </glossaryitem>
<glossaryitem> <name>SERVER_SYNC</name> <para>Synchronizes ... </glossaryitem>
<glossaryitem> <name>TTY_ARGV</name> <para>The command, ... </glossaryitem>
<glossaryitem> <name>XV_LABEL</name> <para>Specifies ... </glossaryitem>
```

Теперь строки отсортированы по пунктам глоссария. К сожалению, в каждой строке есть XML-теги вперемешку с текстом (мы использовали многоточия [...], чтобы обозначить пропущенный текст). Вам нужно каким-то образом вставить новые строки, чтобы они были разъединены. Чтобы сделать это, измените ваш сценарий `ex`: отметьте точки соединения текстовых фрагментов *перед* их объединением, а затем замените маркеры новыми строками. Ниже представлен расширенный сценарий `ex`:

<code>g/^<glossaryitem>/,/^<\glossaryitem>/-1s/\$/@@/</code>	<i>Добавить @@ в конец каждой строки</i>
<code>g/^<glossaryitem>/,/^<\glossaryitem>/j</code>	<i>Объединить записи</i>
<code>%!sort</code>	<i>Отсортировать их</i>
<code>%s/@@ /^@/g</code>	<i>Разбить строки</i>
<code>wq</code>	<i>Сохранить файл</i>

Первые три команды выводят следующее:

```
<glossaryitem>@@ <name>ICON_IMAGE</name>@@ <para>Sets ...</para>@@ </glossaryitem>
<glossaryitem>@@ <name>SERVER_SYNC</name>@@ <para>Synchronizes ...</para>@@ </
glossaryitem>
<glossaryitem>@@ <name>TTY_ARGV</name>@@ <para>The command, ...</para>@@ </
glossaryitem>
<glossaryitem>@@ <name>XV_LABEL</name>@@ <para>Specifies ...</para>@@ </
glossaryitem>
```

Обратите внимание на дополнительный пробел после каждого `@@`. Пробелы являются результатом выполнения команды `j`, потому что она преобразует каждый символ новой строки в пробел.

Первая команда помечает изначальные переносы строк с помощью `@@`. Нет необходимости отмечать конец фрагмента (после `</glossaryitem>`), поэтому первая команда использует `-1` для возврата вверх на одну строку в конце каждого фрагмента. Четвертая команда восстанавливает переносы строк, заменяя маркеры (плюс один дополнительный пробел) на новые строки. (Вы вводите новую строку с помощью `Ctrl+V Ctrl+J`. Это мы скоро рассмотрим.) Теперь ваш файл отсортирован по фрагментам.

Тонкое различие между `vi` и `Vim`

В только что законченном сценарии для использования в `Vim` версии редактора `ex` мы ввели новую строку с помощью `Ctrl+V Ctrl+J`, и она отобразилась как `^@`. Однако при интерактивном редактировании в `Vim` вы должны делать это по-другому. Вы вводите новую строку с помощью `Ctrl+V Enter`, и это отображается как `^M`.

В исходном `vi` различий нет. Если вы используете оригинальный `vi`, то и в интерактивном режиме, и в сценарий вы вводите новую строку с помощью `Ctrl+V Enter`.

Комментарии в сценариях ex

Возможно, вы захотите вновь использовать скрипт, который мы только что разработали, адаптировав его к новой задаче. В сложном сценарии, подобном этому, разумным будет добавить комментарии, чтобы кому-то другому (или даже вам) было легче понять, как он работает. В сценариях ex все, что следует за двойными кавычками, игнорируется во время выполнения, поэтому двойными кавычками можно обозначить начало комментария. Комментарии могут располагаться на отдельной строке. Они также могут быть в конце любой команды, которая не интерпретирует кавычки как часть команды. Например, у кавычки есть значение сопоставления команд и экранирования оболочки, поэтому вы не сможете завершить такие строки комментарием.

Кроме использования комментариев, вы можете указать команду по ее полному имени, что, как правило, отнимает очень много времени при работе с vi. И наконец, если вы добавите пробелы, показанный ранее сценарий ex примет более читабельный вид:

```
" Mark lines between each <glossaryitem>...</glossaryitem> block
global /<glossaryitem>/,/^<\glossaryitem>/-1 substitute /$/@@/
" Now join the blocks into one line
global /<glossaryitem>/,/^<\glossaryitem>/ join
" Sort each block--now really one line each
%!sort
" Restore the joined lines to original blocks
%substitute /@@ /^@/g
" Write the file back out and exit
wq
```



В предыдущих изданиях книги мы писали: удивительно, но команда substitute не работает с ex, хотя полные имена других команд работают.

На тот момент такое утверждение было правильным, по крайней мере для версии Solaris vi.

Однако, протестировав версии vi Heirloom и Solaris 11, мы обнаружили, что команда substitute прекрасно работает в качестве команды ex. Тем не менее перед использованием полной команды в сценарии вам стоит проверить вашу локальную версию и убедиться, что она работает.

За пределами ex

Если вы заинтересованы в расширении своих возможностей редактирования, вам стоит знать, что в системах Unix есть даже более мощные редакторы, чем ex: редактор потоков sed и язык управления данными awk. Существует также невероятно популярный язык программирования perl. Информацию об этих программах вы

можете найти в книгах «sed & awk» Дейла Догерти и Арнольда Роббинса; «Эффективное программирование на awk» Арнольда Роббинса; «Изучаем Perl» Рэндала Л. Шварца, Тома Феникса, Брайана Де Фоя и «Программирование на Perl» Ларри Уолла, Тома Кристиансена, Джона Орванта.

Редактирование исходного кода программы

Все функции, которые мы обсуждали до этого момента, больше подходят для редактирования обычного текста или исходного кода программы. Однако есть ряд дополнительных функций, которые интересны в основном программистам. Они включают управление отступами, поиск начала и конца алгоритмов, использование `ctags`.

Дальнейшее обсуждение взято из документации, предоставленной MKS, Inc. (бывшая Mortice Kern Systems) с ее превосходной реализацией `vi` для систем на базе MS-DOS и Windows, доступной как часть инструментария MKS (<https://www.ptc.com/en/products/developer-tools/mks-toolkit>). Документация использована с разрешения MKS, Inc. (<https://www.mkssoftware.com>).

Управление отступом

Исходный код программы отличается от обычного текста рядом особенностей. Одной из наиболее важных из них являются отступы. Они показывают логическую структуру программы: способ, с помощью которого операторы группируются в блоки. `vi` предоставляет автоматическое управление отступом. Чтобы использовать его, запустите команду:

```
:set autoindent
```

Теперь, когда вы делаете отступ строки с помощью пробелов или табуляции, следующие строки автоматически отступают на то же расстояние. Если вы нажмете **Enter** после введения первой строки с отступом, курсор перейдет на следующую строку и автоматически сделает отступ на то же расстояние, что и в предыдущей строке.

Будучи программистом, вы увидите, что это экономит немало времени по сравнению с ручной расстановкой, особенно если у вас несколько уровней отступов.

Когда вы введете код при включенном автоматическом отступе, набрав в начале строки **Ctrl+T**, вы получите следующий уровень отступа, а набрав **Ctrl+D**, уберете один.

Обратите внимание, что ввод **Ctrl+T** и **Ctrl+D** осуществляется в режиме редактирования, в отличие от большинства команд, которые вводятся в командном режиме.

Существует два дополнительных варианта команды **Ctrl+D**.

- **^ ^D** — когда вы вводите **^ ^D** (**^ Ctrl+D**), редактор перемещает курсор обратно в начало строки, но только для текущей строки. Следующая введенная вами строка будет начинаться с текущего уровня автоматического отступа. Это особенно полезно для ввода команд препроцессора **C** при наборе исходного кода **C/C++**.
- **0 ^D** — когда вы вводите **0 ^D**, редактор перемещает курсор обратно в начало строки. Кроме того, текущий уровень автоматического отступа обнуляется, а у следующей введенной вами строки не будет автоматического отступа.

И наконец, в вашем терминале есть символ *стирания строки*, обычно **Ctrl+U**, который стирает все напечатанные входные данные строки. Эта команда также работает в версии GUI Vim.

Попробуйте использовать параметр **autoindent** при вводе исходного кода. Он упрощает работу с отступами. Иногда он даже может помочь избежать ошибок, например, в исходном коде на **C**, где вам обычно требуется одна закрывающая фигурная скобка **}** для каждого уровня отступа, к которому вы возвращаетесь.

Команды **<<** и **>>** также полезны при отступах в исходном коде. По умолчанию **>>** сдвигает строку на восемь пробелов вправо (то есть добавляет восемь пробелов отступа), а **<<** — влево. Например, переместите курсор в начало строки и нажмите **>** дважды (**>>**). Вы увидите, как строка перемещается вправо. Если теперь вы дважды нажмете **<** (**<<**), строка переместится обратно.

Чтобы сдвинуть несколько строк, наберите число и **>>** или **<<**. Например, расположите курсор на первой строке объемного параграфа и введите **5>>**. Эта команда сдвинет первые пять строк параграфа.

По умолчанию сдвиг происходит на восемь пробелов (вправо или влево). Вы можете изменить это с помощью команды:

```
:set shiftwidth=4
```

Удобно, когда **shiftwidth** того же размера, что и расстояние между остановками табуляции. Ширина табуляции по умолчанию также равна восьми символам.

Редактор старается быть умным, когда делает отступы. Если вы видите текст с отступом в восемь пробелов за раз, по факту это символы табуляции, которые редактор вставляет в файл (они обычно расширяются до восьми пробелов). Это стандартная настройка Unix. Она наиболее заметна, когда вы вводите символ

табуляции при обычном вводе и когда файлы отправляются на печать — Unix расширяет их до восьми пробелов.

Чтобы изменить способ отображения табуляции на экране, необходимо преобразовать параметр `tabstop`. Например, если у вас есть что-то с большим отступом, вы можете настроить остановку табуляции через каждые четыре символа, чтобы строки не переносились:

```
:set tabstop=4
```

Вы также должны изменить `shiftwidth` в тот же момент и на тот же размер отступа, что и у табуляции.



Меняйте ширину табуляции обдуманно. Хотя vi и Vim могут отображать файл с произвольной настройкой ширины табуляции, символы табуляции в ваших файлах все равно расширяются на восемь символов во многих программах Unix.

И что еще хуже: смешивание табуляции, пробелов и необычных остановок табуляции сделает ваш файл абсолютно нечитабельным при отображении за пределами редактора, с помощью такой утилиты, как `more`, или при печати.

При программировании и работе с табуляцией и остановками табуляции у вас есть два варианта.

- Принять восьмисимвольную ширину табуляции как факт взаимодействия с Unix и приспособиться к этому.
- Настроить редактор, чтобы он расширял табуляции до пробелов по мере их ввода. Сделать это можно с помощью следующей команды:

```
:set expandtab
```

Когда `expandtab` установлен, каждый раз при нажатии **Tab** редактор вводит достаточное количество пробелов, чтобы переместить курсор на следующую остановку табуляции.

Если все члены вашей команды будут работать по данному принципу, то весь код будет отформатирован в едином согласованном стиле. Это особенно важно для такого языка, как Python, в котором отступ кода указывает на группировку операторов, а несоответствующие пробелы и табуляции могут стать причиной ошибок¹. Дополнительно вы можете использовать параметр `expandtab` для преобразования ранее существующих табуляций в пробелы.

¹ Тема пробелов и табуляций может вызвать серьезные споры. Посмотрите этот замечательный фрагмент из сериала «Кремниевая долина»: <https://www.youtube.com/watch?v=SsoOG6ZeyUI>.

Иногда отступ может не работать ожидаемым образом, потому что то, что вы считаете символом табуляции, на самом деле является одним пробелом или более. Обычно ваш экран отображает и табуляцию, и пробелы как пустое пространство, что делает их неразличимыми. Попробуйте запустить команду:

```
:set list
```

Это изменит ваш экран так, что табуляция будет отображаться как символ управления ^I, а конец строки — как \$. Таким образом вы сможете увидеть истинный пробел, а также заметить лишние пробелы в конце строки. Временным эквивалентом является команда :l. Например, команда:

```
:5,20 l
```

отображает строки с 5-й по 20-ю, показывая символы табуляции и символы конца строки.

Специальная команда поиска

Символы (, [и { можно назвать открывающими скобками. Когда курсор располагается на одном из них, нажатие клавиши % переместит курсор с открывающей скобки на соответствующую закрывающую —),] или } — с учетом обычных правил вложенных скобок¹. Например, если бы вы поместили курсор на первую (в:

```
if [ cos(a[i]) == sin(b[i]+c[i]) )  
{  
    printf("cos and sin equal!\n");  
}
```

и нажали %, вы бы увидели, как курсор перемещается к соответствующей закрывающей круглой скобке в конце строки.

Аналогично, если курсор расположен на одном из символов закрывающей скобки, нажатие % переместит курсор обратно к соответствующему открывающему символу скобки. Например, поместите курсор на закрывающую фигурную скобку после строки printf и нажмите %.

Редактор также способен найти символ скобки за вас. Если курсор расположен не на символе скобки, то при нажатии % он выполнит поиск в текущей строке до первой открывающей или закрывающей скобки и затем перейдет к соответствующему ей символу! Например, если курсор расположен на символе = первой строки из примера выше, то % найдет открытую круглую скобку и затем перейдет к закрытой круглой скобке.

¹ Некоторые версии также сопоставляют < и > с помощью %.

Этот символ поиска не только помогает вам перемещаться вперед и назад в программе, осуществляя длинные переходы, но также позволяет проверить вложенность фигурных и круглых скобок в исходном коде. Например, если вы установите курсор на первый символ { в начале функции С и нажмете %, курсор должен будет перейти к }, что (как вы думаете) заканчивает функцию. Если символ не тот, значит, что-то пошло не так. Если в файле не найдено совпадения с символом }, то редактор оповестит вас¹.

Вот еще один способ поиска сопоставимых скобок:

```
:set showmatch
```

В отличие от % установка `showmatch` (или его сокращения `sm`) поможет вам в режиме редактирования. Когда вы вводите `)` или `}`², курсор ненадолго перемещается к соответствующим символам `(` или `{`, прежде чем занять текущую позицию. Если совпадение не найдено, терминал подает сигнал. Если совпадение просто «за кадром», редактор молча продолжает работу. Используя дополнительный программный модуль `matchparen`, который загружается по умолчанию, Vim способен выделить соответствующую круглую или фигурную скобку.

Использование тегов

Исходный код для больших программ С или С++ обычно распределяется на несколько файлов. Иногда бывает сложно отследить, какой файл какие определения функций содержит. Чтобы упростить задачу, используйте команду Unix `ctags` вместе с `ex`-командой `:tag`.

Вы запускаете команду `ctags` в командной строке оболочки. Ее задача — создать информационный документ, который редактор будет использовать в дальнейшем, чтобы определить, какие файлы какие функции определяют. По умолчанию данный файл называется `tags`. Во время сеанса редактирования следующая команда:

```
:!ctags file.c
```

создает файл с именем `tags` в вашем текущем каталоге, который содержит информацию по функциям, определенным в `file.c`. А команда:

```
:!ctags *.c
```

создает файл `tags`, описывающий все исходные файлы С в каталоге.

¹ Обратите внимание, что редактор также учитывает скобки внутри закомментированных строк и комментариев, поэтому % ненадежен.

² В Vim `showmatch` также показывает вам сопоставимые квадратные скобки (`[` и `]`).



Существующие версии `ctags` в Unix работают с языком C и часто с Pascal и Fortran 77. Иногда они даже могут обрабатывать язык ассемблера. Но практически все они не работают с C++. Доступны другие версии, которые могут создавать файлы `tags` для C++ и других языков и типов файлов. Для получения дополнительной информации перейдите в подраздел «Расширенные теги» далее.

Теперь предположим, что файл `tags` содержит информацию обо всех исходных файлах, которые составляют программу C. Предположим также, что вы хотите просмотреть или отредактировать функцию программы, но не знаете, где она расположена. Из режима `vi` команда:

```
:tag name
```

просматривает файл `tags`, чтобы найти, какой файл содержит определение функции `name`. Затем она считывает этот файл и помещает курсор на строку, в которой определено имя. Таким образом, вам не нужно знать, какой файл необходимо редактировать. Остается только решить, какую функцию вы хотите отредактировать.

Допускается также использовать функцию тегов в командном режиме `vi`. Поместите курсор на искомый идентификатор, а затем наберите `Ctrl+]`. Редактор выполнит поиск тега и перейдет к файлу, который определяет идентификатор. Будьте внимательны при размещении курсора: редактор использует «слово», начиная с текущей позиции курсора, а не все слово целиком.



Если вы попытаетесь использовать команду `:tag` для чтения нового файла, но вы не сохранили последние изменения текущего текста, редактор не позволит вам перейти к новому файлу. Вы должны либо записать ваш текущий файл с помощью команды `:w` и затем запустить `:tag`, либо ввести:

```
:tag! имя
```

чтобы редактор продолжил работу без учета правок.

Расширенные теги

Работа команды `ctags` в Unix имеет свои ограничения. Программа Exuberant `ctags`, написанная Дарреном Хиббертом, — это клон `ctags`, но гораздо функциональнее, чем `ctags` Unix. Она создает расширенный формат файла `tags`, который делает поиск тегов и процесс сопоставления более гибким и мощным.

К сожалению, последнее обновление Exuberant `ctags` было в 2009 году. Проект Universal `ctags` предоставляет поддерживаемую версию `ctags`, начиная с версии Exuberant `ctags`.

В этом разделе мы сначала опишем программу `Universal ctags`, а затем расширенный формат файла тегов.

Здесь также будут описаны стеки тега: способность сохранять несколько местоположений, посещенных с помощью команд `:tag` или `^J`. Vim и (как ни удивительно) версия vi Solaris поддерживают стеки тегов.

Universal ctags

Официальная страница `Universal ctags` находится по адресу <https://ctags.io/>. Исходный код вы найдете по адресу <https://github.com/universal-ctags/ctags>. Поиск информации в Интернете, позволяет ли пакетный менеджер вашей системы устанавливать предварительно скомпилированную версию или вам потребуется создавать ее из исходного кода.

Следующий список программных функций написан на основе файла `old-docs/README.exuberant` в дистрибутиве `Universal ctags`.

- Способность создавать теги для *всех* типов тегов языков C и C++, включая имена классов, макроопределения, имена enum, перечислители (значения внутри перечисления), определения функций (методов), прототипы/описания функций (методов), элементы структуры и элементы данных класса, имена struct, typedef, имена union и переменные. (Bay!)
- Поддерживается код и C, и C++.
- Поддерживается 41 язык, включая C# и Java.
- Надежность при анализе кода и устойчивость к «обману» с помощью кода, содержащего условные конструкции препроцессора `#if`.
- Можно использовать для вывода удобного для восприятия списка выбранных объектов, найденных в исходных файлах.
- Поддерживается создание файлов `tags` в стиле GNU Emacs (*etags*).
- Поддержка различных операционных систем, включая Unix, OpenVMS, и MS-Windows.

`Universal ctags` создает файлы `tags` по описанной далее форме.

Новый формат тегов

Обычно в файле `tags` есть три разделенных табуляцией поля: имя тега (обычно идентификатор), исходный файл, содержащий тег, и указатель, где найти идентификатор. Указатель представляет собой простой номер строки или шаблон поиска `nomagic`, обособленный либо слешами, либо знаками вопроса. Кроме того, файл `tags` всегда отсортирован.

Данный формат создан программой `ctags` в Unix. На самом деле большинство версий `vi` позволяли вводить *любую* команду в поле шаблона поиска (довольно серьезная брешь в безопасности). Более того, из-за незадокументированной особенности реализации, если строка заканчивалась точкой с запятой, а затем двойными кавычками (`;`), все, что следовало за этими двумя символами, игнорировалось (двойные кавычки начинают комментарий, как это происходит в файлах `.exrc`).

Новый формат также совместим с традиционным форматом. Первые три поля идентичны: тег, имя файла и шаблон поиска. `Universal ctags` генерирует только шаблоны поиска, а не произвольные команды. Специальные атрибуты помещаются после разделяющих символов `;`. Каждый атрибут отделен от следующего с помощью символа табуляции и состоит из двух разделенных двоеточием подполей. Первое представляет собой ключевое слово, описывающее атрибут, второе — фактическое значение. В табл. 7.1 представлен список поддерживаемых ключевых слов.

Таблица 7.1. Расширенные ключевые слова `ctags`

Ключевое слово	Значение
<code>arity</code>	Для функций. Определяет количество аргументов
<code>class</code>	Для компонентных функций C++ и переменных. Значением является имя класса
<code>enum</code>	Для значений типа данных <code>enum</code> . Значением является имя типа <code>enum</code>
<code>file</code>	Для «статических» тегов, то есть локальных для файла. Значением должно быть имя класса. Если значение задано в виде пустой строки (просто <code>file:</code>), оно считается идентичным полю имени файла. Этот особый случай был добавлен отчасти ради компактности, а отчасти для обеспечения простого способа обработки файлов <code>tags</code> , которых нет в текущем каталоге. Значение поля имени файла всегда относится к каталогу, где расположен сам файл <code>tags</code>
<code>function</code>	Для локальных тегов. Значением является имя функции, в которой они определены
<code>kind</code>	Значение — это одна буква, которая указывает на лексический тип тега. Это может быть <code>f</code> для функции, <code>v</code> для переменной и т. д. Поскольку имя атрибута по умолчанию — <code>kind</code> , одиночная буква может обозначать тип тега (например, <code>f</code> для функции)
<code>scope</code>	Предназначено в основном для функций членов класса C++. Обычно оно принимает значение <code>private</code> для частных членов или не включает открытых членов, поэтому пользователи могут ограничить поиск тегов только для открытых членов
<code>struct</code>	Для полей в <code>struct</code> . Значением является имя структуры
<code>union</code>	Для полей в <code>union</code> . Значением является имя объединения

Если поле не содержит двоеточие, оно считается относящимся к типу `kind`. Ниже приведено несколько примеров:

```
ALREADY_MALLOCED awk.h    /^#define ALREADY_MALLOCED /;" d
ARRAYMAXED      awk.h    /^ ARRAYMAXED = 0x4000;"      e      enum:exp_
node::flagvals
array.c    array.c 1;" F
```

`ALREADY_MALLOCED` — это макрос языка C. `ARRAYMAXED` — это перечисление `enum C`, определенное в `awk.h`. Третья строка немного отличается: это тег для фактического исходного файла! Она сформирована с помощью параметра `--extras=f` в `Universal ctags` и позволяет задать команду `:tag array.c`. Для большего удобства вы можете поместить курсор на имя файла и использовать команду `^]` для перехода к этому файлу (например, если вы редактируете *Makefile* и хотите перейти в конкретный исходный файл).

В части значения каждого атрибута символы обратного слеша, табуляции, возврата каретки и новой строки кодируются как `\\`, `\t`, `\r` и `\n` соответственно.

У универсальных файлов `tags` могут быть номера исходных тегов, которые начинаются с `!_TAG_`. Эти теги обычно сортируются в начале файла и полезны для определения того, какая программа сгенерировала файл. Вот что создает `Universal ctags`:

```
!_TAG_FILE_FORMAT      2 /extended format; --format=1 will not append ;" to lines/
!_TAG_FILE_SORTED      1 /0=unsorted, 1=sorted, 2=foldcase/
!_TAG_OUTPUT_EXCMD     mixed /number, pattern, mixed, or combineV2/
!_TAG_OUTPUT_FILESEP    slash /slash or backslash/
!_TAG_OUTPUT_MODE      u-ctags /u-ctags or e-ctags/
!_TAG_PATTERN_LENGTH_LIMIT 96 /0 for no limit/
!_TAG_PROC_CWD         /home/arnold/Gnu/gawk/gawk.git/ //
!_TAG_PROGRAM_AUTHOR   Universal Ctags Team //
!_TAG_PROGRAM_NAME     Universal Ctags /Derived from Exuberant Ctags/
!_TAG_PROGRAM_URL      https://ctags.io/ /official site/
!_TAG_PROGRAM_VERSION  5.9.0 /p5.9.20201206.0/
```

Редакторы могут использовать эти специальные теги для реализации специальных функций. Например, Vim обращает внимание на тег `!_TAG_FILE_SORTED` и использует двоичный поиск вместо линейного для поиска файла `tags`, если файл действительно отсортирован.

Если вы работаете с файлами `tags`, мы рекомендуем вам найти и установить программу `Universal ctags`.

Стеки тегов

Команда `:tag` в `ex` и команда командного режима `vi ^]` предоставляют ограниченные средства для поиска идентификаторов, основанные на информации из файла `tags`. Vim и Solaris `vi` расширяют эту возможность, поддерживая *стек* местоположений тегов. Каждый раз, когда вы запускаете команду `ex :tag` или используете `^]` в режиме `vi`, редактор запоминает текущее местоположение перед поиском указанного тега. Затем вы можете вернуться в сохраненное местоположение с помощью (как правило) команды `vi ^T` или команды `ex`.

Ниже приведен пример стеков тегов в Solaris `vi`. Стеки тегов в Vim описаны в разделе «Стеки тегов».

Solaris vi

Как ни странно, но версия Solaris `vi` поддерживает создание стеков тегов. Возможно, не так уж и удивительно, что эта функция полностью не задокументирована на страницах руководств Solaris `ex(1)` и `vi(1)`. Для полноты картины мы обобщили стеки тегов Solaris `vi` в табл. 7.2, 7.3 и 7.4. Стеки тегов Solaris `vi` достаточно просты¹.

Таблица 7.2. Команды тега Solaris `vi`: команды `ex`

Команда	Функция
<code>ta[g][!] строка_тега</code>	Отредактировать файл, содержащий <i>строку_тега</i> , как определено в файле <code>tags</code> . Знак <code>!</code> заставляет <code>vi</code> переключиться на новый файл, если текущий буфер был изменен, но не сохранен
<code>po[p][!]</code>	Выталкивать из стека тега по одному элементу

Таблица 7.3. Команды тега командного режима Solaris `vi`

Команда	Функция
<code>^]</code>	Найти местоположение идентификатора под курсором в файле <code>tags</code> и перейти туда. Если стек тегов включен, текущее местоположение автоматически заносится в стек тегов
<code>^T</code>	Вернуться к предыдущему местоположению в стеке тегов, то есть вытолкнуть один элемент

¹ Эта информация была получена в ходе экспериментов. Ваши результаты могут отличаться.

Таблица 7.4. Параметры Solaris vi для управления тегами

Параметр	Функция
taglength, t1	Управляет числом важных символов в теге, по которому будет производиться поиск. Значение по умолчанию, равное нулю, означает, что все символы важные
tags	Значение — это список имен файлов, в которых нужно искать теги. Значение по умолчанию равно "tags /usr/lib/ tags"
tagstack	Когда установлено значение true, vi помещает в стек тегов каждое местоположение. Используйте команду :set notagstack, чтобы отключить стек тегов

Universal ctags и Vim

Чтобы дать вам представление об использовании стека тегов, мы рассмотрим небольшой пример, в котором применяются Universal ctags и Vim.

Предположим, вы работаете с программой, которая использует функцию GNU getopt_long, и вам нужно больше узнать о ней.

GNU getopt состоит из трех файлов: getopt.h, getopt.c и getopt1.c.

Сначала вы создаете файл tags, а затем начинаете редактирование основной программы, расположенной в main.c:

```
$ ctags *.c[h]
$ ls
getopt1.c getopt.c getopt.h getopt_int.h main.c Makefile tags
$ vim main.c
```

Команда	Результат
/getopt_	/* option processing. ready, set, go! */ while ((c = getopt_long(argc, argv, optlist, optab, NULL)) != EOF) { switch (c) { case 'l': Отредактировать main.c и перейти к вызову getopt_long
^]	int getopt_long (int argc, char *const *argv, const char *options, const struct option *long_options, int *opt_index) { return _getopt_internal (argc, argv, options, long_options, opt_index, 0); } "getopt1.c" 195L, 4750B Осуществить поиск тега по getopt_long. Vim переходит к getopt1.c, поместив курсор на определение getopt_long

Получается, что `getopt_long` — «функция-обертка» для `_getopt_internal`. Вы помещаете курсор на `_getopt_internal` и осуществляете еще один поиск тега.

Команда	Результат
3jf_ ^]	<pre>int █getopt_internal (int argc, char *const *argv, const char *optstring, const struct option *longopts, int *longind, int long_only) { int result; getopt_data.optind = optind; getopt_data.opterr = opterr; result = _getopt_internal_r (argc, argv, optstring, longopts, longind, long_only, &getopt_data); optind = getopt_data.optind;</pre> "getopt.c" 1294L, 34298B Теперь вы перешли к <code>getopt.c</code>. Чтобы узнать больше о <code>struct option</code>, переместите курсор в <code>option</code> и осуществите очередной поиск тега
jfo;	<pre>one). For long options that have a zero `flag' field, `getopt' returns the contents of the `val' field. */ struct option {     const char *name;     /* has_arg can't be an enum because some compilers complain about        type mismatches in all the code that assumes it is an int. */     int has_arg;     int *flag;     int val; };  /* Names for the values of the `has_arg' field of `struct option'. */</pre> "getopt.h" 191L, 6644B Редактор переходит к определению <code>struct option</code> в <code>getopt.h</code>. Теперь вы можете просмотреть комментарии, поясняющие, как он используется
:tags	<pre># TO tag FROM line in file/text 1 1 getopt_long 29 main.c 2 1 _getopt_internal 70 getopt1.c 3 1 option 1185 getopt.c</pre> Команда <code>:tags</code> в Vim отображает стек тега

Набрав `^T` три раза, вы переместитесь обратно в `main.c`, откуда вы и начали. Теги позволяют легко перемещаться по тексту при редактировании исходного кода.

ЧАСТЬ II

Vim

В части II описывается один из наиболее популярных клонов `vi` под названием Vim. Эта часть содержит следующие главы.

- Глава 8 «Vim (улучшенный `vi`): обзор и отличия от `vi`».
- Глава 9 «Графический Vim (`gvim`)».
- Глава 10 «Многооконный режим в Vim».
- Глава 11 «Расширенные возможности Vim для программистов».
- Глава 12 «Сценарии Vim».
- Глава 13 «Прочие полезные возможности Vim».
- Глава 14 «Несколько продвинутых приемов работы с Vim».

ГЛАВА 8

Vim (улучшенный vi): обзор и отличия от vi

«Смотрите! Там, в небе! Это птица!»

«Это самолет!»

«Это Супермен!»

Да, это Супермен! Незнакомец с другой планеты, прибывший на Землю и обладающий силами и способностями, намного превосходящими возможности смертного человека.

Сериал 1950-х «Супермен»

Несмотря на то что Vim не является ни знакомцем, ни пришельцем, у него *есть* силы и способности, превосходящие обычные текстовые редакторы!

В этой главе вы познакомитесь с самыми примечательными среди всех технических усовершенствований Vim по сравнению с vi, а также немного с его историей. Далее мы рассмотрим некоторые специальные режимы Vim и инструменты для обучения новых пользователей. Затем расскажем о некоторых улучшениях Vim по сравнению с vi, начиная с синтаксических определений нескольких цветов и заканчивая полномасштабными сценариями.

Если vi прекрасен (а это так и есть), то Vim великолепен. В первой главе части II мы обсудим, как Vim восполняет многие функции, на нехватку которых жаловались пользователи vi. В этой главе рассматриваются:

- улучшенный процесс редактирования по сравнению с vi;
- встроенная справка;

- параметры запуска и инициализации;
- новые команды перемещения;
- расширенные регулярные выражения;
- расширенный откат изменений;
- инкрементный поиск;
- прокрутка слева направо.

Остальные главы части II охватывают:

- графический пользовательский интерфейс (GUI) Vim;
- многооконное редактирование;
- улучшенные средства программирования;
- сценарии Vim;
- прочие полезные возможности;
- несколько продвинутых приемов работы с Vim.

Немного о Vim

Vim (<https://www.vim.org>) — это модифицированный vi¹. Он написан и поддерживается Брэмом Моленааром. На сегодняшний день Vim, возможно, самая распространенная реализация vi. На момент написания книги текущая версия редактора 8.2.

Возможности компьютеров резко возросли за период между седьмым изданием книги и этим. В 2008 году 1 гигабайт считался большим объемом памяти. И хотя память становилась все более доступной и объемной (то есть больше гигабайт), пользователям все еще приходилось настраивать свои приложения и инструменты для рационального использования доступных вычислительных ресурсов.

Сегодня 16 гигабайт памяти стало нормой, а многие компьютеры, как правило, поставляются со сверхбыстрыми твердотельными накопителями (SSD). Подобные технологические усовершенствования избавляют от некоторых старых привычек. Следовательно, большинство советов по настройке Vim задают более высокую планку для возможностей редактирования. Позднее мы обсудим такие вещи, как история команд и поиска, а также историю изменений для их отмены.

¹ Мы обнаружили, что «Википедия» ([https://en.wikipedia.org/wiki/Vim_\(text_editor\)#History](https://en.wikipedia.org/wiki/Vim_(text_editor)#History)) приписывает Vim такое определение: «При первом выпуске Vim его название было акронимом для Vi Imitation».

Свободный от стандартов и комитетов, Vim продолжает наращивать свои функциональные возможности. Вокруг него выросло целое сообщество, члены которого во время циклов разработки совместно решают путем голосования, какие новые функции добавить, а какие изменить.

Благодаря самоотверженной энергии Брэма и такой системе выбора Vim влюбляет в себя все больше новых пользователей. Он не изменяет себе и при этом развивается вместе с компьютерной индустрией и, соответственно, с потребностями людей в области редактирования. К примеру, его контекстно зависимое редактирование началось с языка C и расширилось до C++, Java, а к настоящему моменту и до C#.

Vim идет в ногу со временем и включает в себя много функций, облегчающих правку кода на многих новых языках.

Сегодня Vim настолько распространен, особенно для Unix и его вариаций (например, BSD и GNU/Linux), что для многих (если не для большинства) пользователей он является синонимом vi. И действительно, основная масса (если не все) дистрибутивов GNU/Linux поставляются вместе с установленным по умолчанию Vim в виде двоичного файла `/usr/bin/vi`!

Vim предоставляет функции, которых нет в vi и которые считаются необходимыми для современного редактирования текстов, такие как простота использования¹, графическая поддержка терминала, цвета, подсветка синтаксиса и форматирования, а также расширенные настройки.

Обзор

Здесь приведен обзор Vim и многих его улучшений со ссылками на главы и разделы книги, где эти улучшения описаны.

Автор и история

Брэм начал работу над Vim после приобретения компьютера Amiga во второй половине 1988 года². Будучи пользователем Unix, он работал в покоем на vi редакторе **stevie**, который считал далеким от совершенства. К счастью, он поставлялся с ис-

¹ Простота использования субъективна, но мы убеждены, что пользователи, потратившие время на изучение расширенных функций Vim, согласятся с подобной оценкой.

² Спасибо Брэму Моленару за предоставленный материал, на основе которого написан данный раздел. Больше информации по истории Vim вы можете найти в презентации Брэма «Vim 25» (https://www.youtube.com/watch?v=ayc_qrB-93o).

ходным кодом, и Брэм начал создавать более совместимый с vi редактор, устраняя при этом ошибки. Через некоторое время программа стала вполне пригодной для использования. Первая версия Vim была создана 2 ноября 1991 года, а выпущена в январе 1992-го. Vim 1.14 был выпущен на диске № 591 Фреда Фиша (набор бесплатного ПО для Amiga).

Другие люди начали использовать программу, она им понравилась, и они решили помочь с ее разработкой. За переносом на Unix последовали переносы на MS-DOS и другие системы, и впоследствии Vim стал одним из самых доступных клонов vi. Постепенно добавлялись новые функции: многоуровневая отмена, работа в нескольких окнах и многие другие. Некоторые функции были уникальными для Vim, но многие были созданы на основе других аналогов vi. Целью было и остается обеспечить наилучшее взаимодействие с пользователем.

Сегодня Vim, возможно, самый полнофункциональный из всех редакторов типа vi, а онлайн-справка по нему поражает своей обширностью.

Одно из самых малоизвестных свойств Vim — это его поддержка набора текста справа налево, что полезно для таких языков, как иврит и фарси, и иллюстрирует универсальность Vim.

Еще одна цель создания Vim — быть надежным редактором, на который могут положиться профессиональные разработчики ПО. Сбои Vim редки, а если они и случаются, вы всегда сможете восстановить свои правки.

Разработка Vim продолжается. Группа людей, помогающих добавлять функции и переносить Vim на большее количество платформ, растет, а качество адаптации под различные компьютерные системы улучшается. В версии Microsoft Windows есть диалоговые окна и селектор файлов, которые делают доступными для большого количества пользователей трудные для изучения команды vi.

Почему Vim?

Vim так сильно расширил функции vi, что проще ответить на вопрос «Почему не Vim?». vi ввел стандарт, который другие клоны заимствовали, в то время как для Vim он стал фундаментом. Разработчики Vim так кардинально переработали способности редактора, что иногда он нагружал процессоры до предела, чтобы те могли выполнить задачи Vim за приемлемое время. Мы не знаем, была ли это слепая вера Брэма в то, что возможности процессора и памяти возрастут настолько, чтобы соответствовать запросам Vim, но, к счастью, современные компьютеры способны справиться с Vim.

Отличие от vi

Vim более универсален, чем vi. Существуют по крайней мере несколько версий Vim, которые доступны практически во всех операционных системах, в то время как vi работает только в Unix или Unix-подобных системах.

vi — это исходная программа, которая мало изменилась за эти годы. Она является носителем стандарта POSIX и отлично выполняет свою роль. Vim начинается там, где кончается vi. Новый редактор не только обеспечивает весь функционал vi, но и расширяет его, добавляя графический интерфейс и такие функции, как сложные параметры и сценарии, выходя далеко за пределы изначальных возможностей vi.

Vim поставляется со своей собственной встроенной документацией в виде каталога специализированных текстовых файлов. Беглый осмотр данного каталога (с помощью стандартного инструмента Unix для подсчета слов `wc -l *.txt`) показывает 140 файлов, содержащих почти 200 000 строк документации!¹ Это первый намек на широту возможностей Vim. Vim получает доступ к этим файлам с помощью своей внутренней команды `help`, которая также недоступна в vi. Позднее мы рассмотрим систему справки Vim более подробно и дадим вам советы и рекомендации, которые помогут извлечь максимальную пользу из полученного опыта.

В текущей версии Vim в справочном файле `vi_diff` описывается, чем отличается Vim от исходного vi. Деталей было так много, что примечания к справочным файлам создавали слишком много путаницы и были удалены!

Эта и последующие главы охватывают несколько наиболее интересных функций Vim, начиная с расширений оригинального vi и заканчивая новыми «фишками». Здесь описываются лучшие и наиболее популярные способы повышения вашей производительности. Мы также рассмотрим как общепризнанные полезные улучшения, например подсветку синтаксиса, так и некоторые малоизвестные функции, которые полезны для достижения еще большей продуктивности. Например, в подразделе «Автокоманды» в главе 12 мы показываем способ настройки строки состояния Vim для отображения текущих даты и времени при каждом перемещении курсора.

Категории функций

Vim охватывает широкий спектр функций, характерных практически для любой задачи редактирования текста. Некоторые функции просто расширили то, что пользователи хотели от исходного vi, другие — полностью новые, и их нет в vi. И если вам нужно что-то, чего там *нет*, Vim предлагает встроенный сценарий для

¹ Вы можете найти эти файлы из вашего сеанса Vim. Справка находится в папке `doc` в каталоге `$VIMRUNTIME`. Введите команду `ex :!ls $VIMRUNTIME/doc`.

неограниченных расширений и настроек. Ниже представлены некоторые категории функций Vim.

- *Инициализация.* Vim, как и vi, использует конфигурационные файлы для определения сеансов во время запуска, однако в Vim репертуар определяемого поведения значительно шире. Вы можете просто задать несколько параметров, как в vi, либо написать целый комплекс настроек, который определит ваш сеанс работы на основе любого заданного контекста. Например, можно написать сценарий для ваших файлов инициализации, чтобы предварительно скомпилировать код в зависимости от того, в каком каталоге вы редактируете файлы, или вы можете получить информацию из какого-либо источника в реальном времени и включить его в свой текст при запуске. См. раздел «Параметры запуска и инициализации» в данной главе.
- *Возможность бесконечно откатывать изменения.* Редактор vi в Unix позволяет вам отменить ваше последнее действие или восстановить текущую строку до состояния, в котором она была до изменений. Vim предоставляет «бесконечный откат изменений» — возможность продолжать отменять *любые* ваши правки, вплоть до первоначального состояния файла. Для получения дополнительной информации смотрите далее в этой главе раздел «Расширенный откат изменений».
- *Функции графического пользовательского интерфейса (GUI) Vim.* Vim расширяет удобство работы для большинства пользователей, позволяя редактировать с помощью указателя мыши, как и многие другие, простые в использовании редакторы. Все возможности опытного пользователя улучшаются благодаря простому доступу к GUI для облегчения редактирования. Для получения дополнительной информации см. главу 9.
- *Многооконный режим.* Как уже упоминалось, в Vim есть возможность одновременного открытия нескольких окон для одного файла или нескольких файлов. Полный обзор данной функции дается в главе 10.
- *Помощь программисту.* Хотя Vim не пытается удовлетворить все потребности программирования, он предлагает много функций, которые обычно встречаются в интегрированных средах разработки (IDE). От быстрых циклов «редактирование — компиляция — отладка» до автозаполнения ключевых слов — для всего в Vim есть специальные функции, позволяющие вам не только быстро вносить правки, но и помогающие программировать. Дополнительную информацию вы найдете в главе 11.

Естественно, при желании вы *можете* превратить Vim в IDE (подробнее об этом — в главе 15).

- *Завершение.* Vim позволяет завершать частично напечатанные слова с помощью правил контекстно зависимого завершения. Например, Vim способен искать слова в словаре или файле, содержащем ключевые слова, характерные для того или иного языка. См. раздел «Завершение по ключевым словам и словарю» главы 11.

- *Синтаксические расширения.* Vim позволяет управлять отступами и выделением текста цветом на основе синтаксиса. И для определения подобного автоматического форматирования у Vim есть много параметров. Если вам не нравится цветовая подсветка, вы можете ее поменять. Если вам нужен определенный стиль отступа, Vim предоставит его, а если у вас особые потребности, настройте ваше окружение. Для получения дополнительной информации перейдите в раздел «Подсветка синтаксиса» главы 11.
- *Сценарии и плагины.* Вы можете написать свои собственные расширения Vim или загрузить плагины из Интернета. Есть даже возможность внести свой вклад в сообщество Vim, опубликовав свои расширения, чтобы другие смогли ими воспользоваться. Дополнительную информацию вы найдете в главе 12.
- *Постобработка.* В дополнение к функциям инициализации Vim позволяет вам определять, что делать *после* завершения редактирования файла. Вы можете написать процедуры очистки, чтобы удалить временные файлы, накопленные в результате компиляции, или вносить правки в файл в реальном времени, прежде чем он будет записан обратно в хранилище. Например, у вас есть возможность проверить компоновку кода Python на предмет последовательного соблюдения локальных правил форматирования. Вы обладаете полным контролем над настройкой любых действий постредактирования.
- *Строки произвольной длины и двоичные данные.* Исторические версии vi часто были ограничены примерно тысячью символами на строку. Более длинные строки укорачивались. Vim обрабатывает строки любой длины¹.

Vim поддерживает Юникод и отображает многобайтовые символы Юникода в виде одного глифа, когда это возможно. Vim способен редактировать файлы, содержащие любой восьмибитный символ, а также двоичные и исполняемые файлы. Иногда это может быть очень полезно. Редактирование двоичных файлов рассматривается в разделе «Редактирование двоичных файлов» главы 13. Для получения дополнительной информации о параметрах, особенно о `file format` и `filetype`, см. раздел «Параметры Vim 8.2» приложения Б.

Есть любопытная деталь. Традиционно vi всегда записывает файл с добавлением в конце символа новой строки. При редактировании двоичного файла эта операция может добавить один символ к файлу и создать проблемы. Vim совместим с vi по умолчанию и добавляет эту новую строку. Чтобы этого не происходило, настройте параметр `binary`.

- *Контекст сеанса.* Vim хранит информацию о сеансе в файле `.viminfo`. Вы когда-нибудь задавались вопросом «На чем это я остановился?» во время проверки и редактирования файла? Конфигурационный файл `.viminfo` решает эту про-

¹ В действительности вплоть до максимального значения типа данных языка C `long` — 2 147 483 647 на 32-битном компьютере и значительно больше в 64-битной системе.

блему, позволяя определять количество и тип информации, которую следует сохранять во время сеансов. Например, вы можете определить, сколько «недавних документов» или последних отредактированных файлов отслеживать, сколько правок (удалений, изменений) запоминать для каждого файла, сколько запоминать команд из истории команд и сколько регистров и строк сохранять из предыдущих действий редактирования (вставок, удалений и т. д.).

Vim также запоминает, на какой строке вы находились в каждом из недавно отредактированных файлов. Если при завершении сеанса редактирования ваш курсор был расположен на строке 25, то при следующем открытии файла курсор опять окажется на этой строке. Для получения дополнительной информации перейдите в раздел «viminfo: итак, где же я остановился?» главы 13.

- *Переходы.* Vim управляет переходами из одного состояния в другое. Когда во время сеанса вы перемещаетесь от буфера к буферу или от окна к окну (обычно это одно и то же), Vim автоматически выполняет служебные действия до и после операции.
- *Прозрачное редактирование.* Vim выявляет и автоматически разделяет архивированные и сжатые файлы. Например, вы можете напрямую отредактировать такой сжатый файл, как `myfile.txt.gz`, или даже каталоги. Vim позволяет перемещаться по каталогу и выбирать файлы для редактирования с помощью знакомых команд навигации в стиле `vi`.
- *Метаинформация.* Vim предлагает четыре удобных неизменяемых регистра, из которых пользователи могут извлечь метаинформацию для «вставок»: текущее имя файла (%), альтернативное имя файла (#), последнюю выполненную команду командной строки (:) и последний вставленный текст (., точка).
- *Регистр «черная дыра».* Это малоизвестное, но очень полезное расширение для регистров редактирования. Обычно при удалении текста он помещается в регистры с помощью схемы чередования (см. раздел «Использование регистров» главы 4), которая полезна при циклическом переборе предыдущих удалений для возврата ранее удаленного текста. В Vim есть регистр «черная дыра» как место, куда можно выбросить удаленный текст, не нарушая его чередование в обычных регистрах. Если вы пользователь Unix, рассматривайте данный регистр как версию Vim `/dev/null`. Из файла справки Vim `change.txt`: «При записи в этот регистр ничего не происходит. Это можно использовать, чтобы удалить текст, не затрагивая обычные регистры. При считывании из данного регистра ничего не возвращается».

Vim также позволяет вам вернуться в совместимый с `vi` режим с помощью параметра `compatible (:set compatible)`. Большую часть времени вы, вероятно, захотите пользоваться дополнительными функциями Vim, но обратная совместимость лишней не будет.

Философия

Философия Vim тесно связана с философией vi. Оба обеспечивают мощное и изысканное редактирование, зависят от модальности (командный режим против режима ввода) и заточены под «клавиатурное» редактирование: вы можете осуществлять все свои правки быстро и эффективно, ни разу не прикоснувшись к мыши. Нам нравится называть это методом слепого редактирования по аналогии с методом слепой печати, что отражает соответствующее увеличение скорости и эффективности, приносимые обоими редакторами в их задачи.

Vim расширяет эту философию, предоставляя функции для менее опытных пользователей (графический интерфейс, визуальный режим подсветки) и мощные параметры для опытных пользователей (сценарии, расширенные регулярные выражения, реконфигурируемые синтаксис и отступы).

А для еще более опытных пользователей, которые любят писать код, Vim поставляется с исходным кодом. Пользователи могут свободно (и это даже поощряется) совершенствовать приложение. С точки зрения философии Vim обеспечивает баланс потребностей *всех* пользователей.

Вспомогательные средства и простые режимы для новых пользователей

Учитывая, что и vi, и Vim предъявляют определенные требования к знаниям новых пользователей, Vim все же предоставляет несколько функций, которые упрощают процесс работы и обучения.

- *Графический Vim (gvim)*. Когда вы вызываете команду `gvim`, Vim отображает графическое окно с широкими возможностями, предлагая полную функциональность Vim с добавлением функций «укажи и щелкни», ставших популярными благодаря современным программам графического интерфейса. Во многих окружениях `gvim` представляет собой другой двоичный файл, созданный путем компиляции Vim со всеми включенными параметрами графического интерфейса. Он также может быть вызван с помощью команды `vim -g`.
- *«Простой» Vim (evim)*. Команда `evim` заменяет некоторые простые действия на стандартные функции vi, которые могут показаться новичкам более интуитивным способом редактирования файлов. Опытным пользователям этот режим может показаться не таким простым, так как они уже привыкли к стандартному поведению vi. Его также можно вызвать с помощью команды `vim -y`.
- *vimtutor*. Это отдельная команда, которая, по сути, запускает Vim со специальным файлом справки. Подобный вызов Vim предоставляет пользователю отправную точку для изучения редактора. Для завершения *vimtutor* требуется 30 минут.

Вы также можете найти множество интерактивных обучающих руководств по Vim в Интернете. Одним из них является OpenVim (<https://www.openvim.com>). Другое называется VIM Adventures (<https://vim-adventures.com>), оно рассматривает изучение Vim как приключенческую игру.

Встроенная справка

Как уже упоминалось ранее, Vim содержит около 200 000 строк документации. Практически вся она доступна во встроенной справке Vim. Самый простой способ открыть ее — использовать команду `:help`. Это интересно, потому что она показывает пользователю первый пример многооконного редактирования Vim.

Поскольку встроенная справка требует минимальных навыков навигации vi, то перед пользователями встает дилемма из разряда «курица и яйцо». Чтобы получить от нее максимальную пользу, они должны знать, как переходить туда и обратно по тегам. Мы представим здесь обзор перемещения по экрану справки.

Команда `:help` вызывает что-то похожее на:

```
*help.txt*      For Vim version 8.2.  Last change: 2020 Aug 15

                VIM - main help file

Move around:   Use the cursor keys, or "h" to go left,      k
               "j" to go down, "k" to go up, "l" to go right.  j
Close this window: Use ":q<Enter>".
Get out of Vim:  Use ":qa!<Enter>" (careful, all changes are lost!).

Jump to a subject: Position the cursor on a tag (e.g. |bars|) and hit Ctrl-].
With the mouse:   ":set mouse=a" to enable the mouse (in xterm or GUI).
                  Double-click the left mouse button on a tag, e.g. |bars|.

Jump back:       Type Ctrl-O. Repeat to go further back.
Get specific help: It is possible to go directly to whatever you want help
                  on, by giving an argument to the |:help| command.
                  Prepend something to specify the context: *help-context*

                WHAT PREPEND EXAMPLE ~
Normal mode command          :help x
Visual mode command          v_  :help v_u
Insert mode command          i_  :help i_<Esc>
Command-line command         :   :help :quit
Command-line editing         c_  :help c_<Del>
Vim command argument         -   :help -r
Option                       '   :help 'textwidth'
Regular expression           /   :help /[
See |help-summary| for more contexts and an explanation.
```

Search for help: Type `":help word"`, then hit `Ctrl-D` to see matching help entries for "word".
Or use `":helpgrep word"`. `|:helpgrep|`

Getting started: Do the Vim tutor, a 30-minute interactive course for the basic commands, see `|vimtutor|`.
Read the user manual from start to end: `|usr_01.txt|`

Vim stands for Vi IMproved. Most of Vim was made by Bram Moolenaar, but only through the help of many others. See `|credits|`.

К счастью, Vim учитывает потенциальные проблемы навигации для начинающих и предусмотрительно открывает основные рекомендации по перемещению, и даже сообщает вам, как закрыть окно справки. Мы настоятельно рекомендуем вам изучить справку.

Как только вы познакомитесь с командой `:help`, попробуйте воспользоваться табуляцией в командной строке Vim. Для любой команды в приглашении (`:`) нажатие клавиши `Tab` приводит к автоматическому завершению наименования команды в зависимости от контекста. Например, следующая строка:

```
:e /etc/pas Tab
```

в любой системе Unix расширится до:

```
:e /etc/passwd
```

Команда `:e` подразумевает, что аргументом команды является файл, поэтому Vim ищет частично совпадающие по имени файлы, чтобы завершить ввод.

Но у команды `:help` есть свой собственный контекст, охватывающий тематические разделы справки. Фрагмент строки раздела, который вы вводите, сопоставляется с подстрокой в любом доступном разделе справки Vim. Мы настоятельно рекомендуем вам изучить и использовать эту функцию. Она сэкономит время и откроет новые и интересные функции, о которых вы, возможно, не знали.

Например, предположим, что вы хотите узнать, как разделить экран. Начните с:

```
:help split
```

и нажмите клавишу `Tab`. Команда справки циклически выполнит: `split()`; `:split`; `:split_f`; `splitfind`; `splitview`; `g:netrw_browse_split`; `:diff split`; `:dsplit`; `:isplit`; `:vsplit`; `+vertspl`; `'splitright'`; `'splitbelow'` и т. д. Чтобы увидеть справку по какой-либо теме, нажмите `Enter`, когда подсвечен необходимый раздел. Вы увидите не только то, что ищете (`:split`), но также что-то, что даже не предполагали возможным сделать, например `:vsplit` — команду «вертикальное разделение».

Параметры запуска и инициализации

Vim использует различные алгоритмы для настройки своего окружения при запуске. Он проверяет параметры командной строки. Он также проводит самоконтроль (как он был вызван и под каким именем?). Существуют различные скомпилированные двоичные коды для удовлетворения различных потребностей (графический интерфейс против текстового окна). Vim также использует последовательность инициализированных файлов, в которой может быть определено и изменено бесчисленное множество комбинаций поведений. Параметров настолько много, что не хватит целой книги, чтобы рассмотреть их все. Мы коснемся некоторых наиболее интересных из них. Далее мы обсудим последовательности запуска Vim в таком порядке:

- параметры командной строки;
- поведения, связанные с именем команды;
- конфигурационные файлы (для всей системы и для пользователя);
- переменные окружения.

Данный раздел познакомит вас с *некоторыми* способами запуска Vim. Для получения более подробной информации о множестве других параметров используйте команду справки:

```
:help startup
```

Параметры командной строки

Параметры командной строки Vim обеспечивают гибкость и мощь. Некоторые параметры вызывают дополнительные функции, в то время как другие отменяют и подавляют поведение по умолчанию. Мы рассмотрим синтаксис командной строки, который использовался бы в типичном окружении Unix. Однобуквенные параметры начинаются с - (один дефис), как в -b, что позволяет править двоичные файлы. Параметры длиной в слово начинаются с -- (два дефиса), как в --noplugin, что отменяет поведение по умолчанию для запуска плагинов. Аргумент командной строки, состоящий лишь из двух дефисов, сообщает Vim, что остальная часть командной строки не содержит параметров. Это стандартное поведение Unix.

Благодаря параметрам командной строки вы можете выборочно указать одно или несколько имен файлов для редактирования. На самом деле существует интересная ситуация, в которой имя файла может быть одним дефисом, сообщаящим Vim, что входные данные поступают из стандартного ввода оболочки — *stdin*, но это уже расширенное использование.

Ниже представлен частичный список параметров командной строки Vim, которые недоступны в `vi` (все параметры `vi` доступны в Vim).

- `-b` — редактировать в двоичном режиме. Описание говорит само за себя. Правка двоичных файлов — дело вкуса, но это мощный способ редактирования файлов, невозможный в большинстве других инструментов. Если вас заинтересовал данный способ, прочитайте раздел справки Vim, касающийся редактирования двоичных файлов.
- `-с команда` — выполнить команду как команду `ex`. В `vi` есть такой же параметр, но Vim допускает до десяти экземпляров `-с` в одной команде. Каждый экземпляр команды должен использовать свою собственную метку `-с`.
- `-C` — запустить Vim в совместимом с `vi` режиме. По очевидным причинам этого параметра никогда не будет в `vi`.
- `--cmd команда` — выполнить команду до `vimrc`-файлов. Это длинная форма параметра `-с`.
- `-d` — запустить в режиме сравнения. Vim сравнивает два, три или четыре файла и устанавливает параметры, упрощая выявление различий в файлах (`scrollbind`, `foldcolumn` и т. д.).

Vim использует собственную программу для выявления различий в файлах, которая называется `diff` в системах Unix. Версия Windows предлагает загрузаемый исполняемый файл, с помощью которого Vim может выполнить сравнение.

- `-E` — запустить в улучшенном режиме `ex`. Например, улучшенный режим `ex` будет использовать расширенные регулярные выражения.
- `-g` — запустить `gvim` (графический интерфейс).
- `-M` — отключает опцию записи. Буферы будут неизменны. Поскольку вы не можете изменить буфер, Vim обеспечивает отсутствие изменений, отключив обе команды `ex` — `:w` и `:w!`.
- `-o[n]` — открыть все файлы в отдельных окнах. Вы можете ввести целое число, чтобы указать необходимое количество открываемых окон. Файлы, упомянутые в командной строке, заполняют лишь указанное число окон (остальные остаются в буфере Vim). Если указанное число окон превышает количество файлов, Vim открывает пустые окна, чтобы удовлетворить запрос.
- `-O[n]` — как `-o`, но открывает вертикально разделенные окна.
- `-u` — запустить Vim в «простом» режиме. Задает параметры на более интуитивное поведение для новичков. В то время как неопытным пользователям он может помочь, бывалым программистам, вероятно, покажется сбивающим с толку и раздражающим.

- **-Z** — запустить с ограниченным доступом. Это, по сути, отключает все внешние интерфейсы и предотвращает доступ к системным функциям. Например, пользователи не смогут использовать **!G!sort** для сортировки от текущей строки в буфере до конца файла. Фильтр **sort** не будет доступен.

Следующий список является последовательностью параметров для использования удаленного экземпляра сервера Vim. Параметры **--remote** указывают удаленному Vim (который может выполняться на одном и том же компьютере или на разных) редактировать файл или оценить выражение на этом удаленном сервере. Параметры **--server** сообщают Vim, на какой сервер отправлять данные, или указывают, что Vim может сам объявить себя сервером. Параметр **--serverlist** просто выводит список доступных серверов:

```
--remote файл
--remote-expr выражение
--remote-send клавиши
--remote-silent файл
--remote-tab
--remote-tab-silent
--remote-tab-wait
--remote-tab-wait-silent
--remote-wait файл ...
--remote-wait-silent файл ...
--serverlist
--servername имя
```

Чтобы получить подробную информацию обо всех параметрах командной строки, включая полный набор **vi**, обратитесь к разделу «Синтаксис командной строки». Для получения дополнительной информации о параметрах **--remote** запустите команду Vim **:help remote**.

Поведения, связанные с именем команды

У Vim есть две основные формы: графическая (использование X Window System в вариантах Unix и собственных графических интерфейсов в других операционных системах) и текстовая, каждую из которых можно запустить с подмножеством параметров. Пользователям Unix достаточно набрать одну из команд из следующего списка, чтобы получить необходимое поведение.

- **vim** — запустить Vim с текстовым интерфейсом.
- **gvim** — запустить Vim в графическом режиме. Во многих окружениях **gvim** является другим двоичным файлом Vim со всеми параметрами графического интерфейса, включенными во время компиляции. Результат тот же, что и при запуске Vim с помощью команды **vim -g**.

- **view, gview** — запустить Vim или **gvim** в режиме «только для чтения». Результат тот же, что и при запуске Vim с помощью команды **vim -R**.
- **rvim** — запустить Vim в ограниченном режиме. Весь внешний доступ к командам оболочки и возможность приостановить сеанс редактирования с помощью команды **^Z** отключены.
- **rgvim** — результат будет тот же, что и при запуске Vim с помощью команды **rvim**, но для графической версии.
- **rview** — аналогична команде **view**, но запускает Vim в ограниченном режиме. В этом режиме у пользователей нет доступа к фильтрам, внешним окружениям или функциям операционной системы. Результат тот же, что и при запуске Vim с помощью команды **vim -Z** (параметр **-R** вызывает описанный выше режим «только для чтения»).
- **rgview** — результат тот же, что и при запуске Vim с помощью команды **rview**, но для графической версии.
- **evim, eview** — использовать «простой» режим для редактирования или просмотра в режиме «только для чтения». Vim устанавливает параметры и функции, поэтому он ведет себя более предсказуемо для тех, кто не знаком с парадигмой Vim. Результат будет тот же, что и при запуске Vim с помощью команды **vim -y**. Опытным пользователям, вероятно, этот режим не покажется простым, поскольку они уже привыкли к стандартному поведению **vi**.
Обратите внимание, что не существует аналогов версии **gXXX** этих команд, потому что **gvim** как будто бы уже считается простым или по крайней мере интуитивно понятным в изучении, с предсказуемым поведением «укажи и щелкни».
- **vimdiff, gvimdiff** — запустить в режиме **diff** и выполнить сравнение входных файлов. Этот момент более подробно описан далее в разделе «В чем же разница?» главы 13.
- **ex, gex** — использовать режим редактирования строки **ex**. Полезна в сценариях. Результат будет тот же, что и при запуске Vim с помощью команды **vim -e**.

Пользователи MS-Windows могут получить доступ к аналогичному выбору версий Vim в списке программ (меню Пуск).

Системные и пользовательские конфигурационные файлы

Vim ищет признаки инициализации в определенной последовательности. Он выполняет первый набор найденных инструкций (либо в виде переменной окружения, либо в файле) и начинает редактирование. Таким образом, первый элемент следующего списка, который встречается, является единственным элементом, который выполняется. Последовательность такая.

1. `VIMINIT` — это переменная окружения. Если она непустая, Vim выполняет ее содержимое как команду `ex`.
2. Пользовательские файлы `vimrc`. Файл инициализации `vimrc` (ресурс Vim) — это межплатформенное понятие, но из-за едва уловимых различий операционной системы и платформы Vim ищет его в других местах в следующем порядке:

<code>\$HOME/.vimrc</code>	Unix, OS/2 ¹ , and Mac OS X
<code>\$HOME/_vimrc</code>	MS-Windows and MS-DOS
<code>\$VIM/_vimrc</code>	MS-Windows and MS-DOS
<code>s:.vimrc</code>	Amiga
<code>home:.vimrc</code>	Amiga
<code>\$VIM/.vimrc</code>	OS/2 and Amiga

3. Локальные файлы `.exrc` и `vimrc`. Если установлен параметр Vim `exrc`, Vim ищет три дополнительных конфигурационных файла: `.vimrc`, `.gvimrc` и `.exrc`. Для систем, не поддерживающих POSIX, имя файла может начинаться с отличного от точки символа.

Файл `.vimrc` прекрасно подходит для настройки параметров редактирования Vim. В этом файле можно установить и сбросить практически любой параметр Vim, и он особенно хорошо подходит для установки глобальных переменных и определения функций, сокращений, переназначения клавиш и т. д. Вот несколько вещей, которые стоит знать о файле `.vimrc`.

- Комментарии начинаются с двойных кавычек (`"`), которые могут быть в любом месте строки. Весь текст после двойных кавычек игнорируется.
- Команды `ex` можно задавать с помощью двоеточия или без него. Например, команда `set autoindent` идентична команде `:set autoindent`.
- С файлом гораздо проще работать, если разбить большие наборы определений параметров на отдельные строки. Например:

```
set terse sw=1 ai ic wm=15 sm nows ruler wc=<Tab> more
```

эквивалентно:

```
set terse      " short error and info messages
set shiftwidth=1
set autoindent
set ignorecase
set wrapmargin=15
set nowrapscan " don't scan past end or top of file in searches
set ruler
set wildchar=<TAB>
set more
```

¹ Мы не знаем, сколько людей все еще используют системы OS/2 или Amiga. Но если вы среди них, вам будет приятно узнать, что Vim поддерживает вас!

Обратите внимание на вторую последовательность команд, она более читабельная. С помощью второго метода также намного проще осуществлять удаления, вставки и временное комментирование строк при отладке настроек в конфигурационном файле. Например, если вы хотите временно отключить нумерацию строк в конфигурации запуска, вы просто вставляете двойные кавычки (") в начало строки `set number` вашего конфигурационного файла.

Переменные окружения

Многие переменные окружения влияют на поведение Vim при запуске и даже на некоторые действия во время сеанса редактирования. Они в основном прозрачны и обрабатываются по умолчанию, если не были настроены.

Как установить переменные окружения

Командное окружение, которое вы используете при входе в систему (в Unix она называется оболочкой), устанавливает переменные, отображающие или контролирующие ее поведение. Переменные окружения особенно эффективны, потому что влияют на программы, вызываемые в командном окружении. Следующие инструкции не являются специфичными для Vim, используйте их для установки любых переменных окружения, которые вы хотите задать в командном окружении.

MS-Windows

Чтобы установить переменную окружения, сделайте следующее.

1. Откройте панель управления.
2. Дважды щелкните левой кнопкой мыши на **System**.
3. Щелкните на вкладке **Advanced**.
4. Нажмите кнопку **Environment Variables**.

В результате вы получите окно, разделенное на две области — **User** и **System**. Новичкам не стоит изменять переменные окружения **System**. В области **User** вы можете установить переменные окружения, относящиеся к Vim, и сделать так, чтобы они сохранялись во время всех сеансов входа.

Unix/Linux Bash и другие оболочки Bourne

Отредактируйте подходящий конфигурационный файл оболочки (например, `.bashrc` для пользователей командной оболочки Bash) и вставьте строки примерно следующего содержания:

```
VARABC=какое-то_значение
VARXYZ=какое-то_другое_значение MYVIMRC=/path/to/my/vimrc/file
export VARABC VARXYZ MYVIMRC
```

Порядок этих строк не имеет значения. Оператор `export` просто делает переменные видимыми для программ, выполняемых в оболочке, и таким образом превращает их в переменные окружения. Значение экспортированных переменных можно установить до и после их экспорта.

Оболочки Unix/Linux C

Отредактируйте подходящий конфигурационный файл оболочки (например, `.cshrc`) и вставьте строки примерно следующего содержания:

```
setenv VARABC какое-то_значение
setenv VARXYZ какое-то_другое_значение
setenv MYVIMRC /path/to/my/vimrc/файл
```

Переменные окружения, относящиеся к Vim

В следующем списке показано большинство переменных окружения и их действия.

Параметр командной строки Vim `-u` отменяет переменные окружения и переходит непосредственно в указанный файл инициализации. Параметр `-u` *не* отменяет переменные окружения, которые не относятся к Vim.

- **EXINIT** — то же самое, что и **VIMINIT**. Используется, если **VIMINIT** не определен.
- **MYVIMRC** — отменяет Vim-поиск файлов инициализации. Если у **MYVIMRC** есть значение при запуске, то Vim определяет, является ли значение именем файла инициализации, и, если файл существует, использует его исходные настройки. Ни к какому другому файлу он не обращается (смотрите последовательность поиска в предыдущем разделе).
- **SHELL** — уточняет, какую оболочку или внешний интерпретатор команд использует Vim для команд оболочки (`!!`, `:!` и т. д.). В командном окне MS-Windows, если **SHELL** не установлена, вместо нее используется переменная окружения **COMSPEC**.
- **TERM** — устанавливает внутренний параметр Vim `term`. В каком-то смысле в этом нет необходимости, поскольку редактор сам настраивает свой терминал. Другими словами, Vim, вероятно, знает, что такое терминал, лучше, чем предопределенная переменная.
- **VIM** — содержит путь к системному каталогу, в котором находится информация по стандартной установке Vim (только для информативных целей и не используется Vim).
- **VIMINIT** — указывает команды `ex` для выполнения при запуске Vim. Определяет несколько команд, разделяя их вертикальными чертами (`|`).



Если на компьютере существует более одной версии Vim, VIM, скорее всего, будет отображать разные значения, в зависимости от версии, которую вы запустили. Например, на компьютере одного из авторов версия Cygwin устанавливает переменную среды VIM в /usr/share/vim, в то время как пакет vim.org устанавливает ее в C:\Program Files\Vim.

Это важно знать, если вы изменяете файлы Vim, поскольку эти правки могут не подействовать, если вы отредактируете не те файлы!

- **VIMRUNTIME** — указывает на файлы поддержки Vim, такие как онлайн-документация, определения синтаксиса и каталоги плагинов. Vim, как правило, делает это сам. Если вы зададите переменную, например, в файле .bashrc, она, скорее всего, приведет к ошибкам при установке более новой версии Vim, поскольку ваша персональная переменная VIMRUNTIME может указывать на старое, несуществующее или неверное местоположение.

Новые команды перемещения

Vim предоставляет все команды перемещения **vi**, большая часть из которых перечислена в главе 3, и добавляет несколько других, представленных в табл. 8.1.

Табл. 8.1. Команды перемещения в Vim

Команда	Описание
<code>n Ctrl+End</code>	Перемещение в конец файла, например, к последнему символу последней строки. Если <code>n</code> было указано, переходит к последнему символу строки <code>n</code>
<code>Ctrl+Home</code>	Перемещение к первому непробельному символу в первой строке файла. Отличается от <code>Ctrl+End</code> тем, что <code>Ctrl+Home</code> не перемещает курсор на символ пробела
<code>число%</code>	Перемещение к строке, заданной указанным <i>числом</i> процентов от общего количества строк в файле, к первому непробельному символу. Важно отметить, что вычисления Vim основываются на количестве строк в файле, а не на общем количестве символов. Это может показаться неважным, но представьте файл, содержащий 200 строк, первые 195 из которых содержат 5 символов (например, цены, такие как \$4.98), а последние четыре строки — по 1000 символов. В Unix (с учетом символа новой строки) файл будет содержать примерно: $(195 * (5 + 1))$ (количество символов в первых пятидесятистрочных строках) $+ 2 + (4 * (1000 + 1))$ (количество символов в тысячестрочных строках) или 5200 символов. При посимвольном подсчете 50 % курсор был бы помещен в строку 96, однако Vim поместит курсор на строку 100
<code>:go n</code>	Перемещение к <code>n</code> -му байту в буфере. Все символы, включая символы конца строки, считаются
<code>:n go</code>	

Обозначение <C-xxx> — это способ Vim описывать комбинации клавиш независимым от системы образом. В данном случае C- обозначает удерживать клавишу **Ctrl** при одновременном нажатии другой клавиши для xxx. Например, <C-End> означает «нажать **Ctrl** и **End**».

Перемещение в визуальном режиме

С помощью Vim вы можете визуально определять выделение и осуществлять команды редактирования. Это напоминает графические редакторы, в которых области подсвечиваются по щелчку кнопкой мыши. Визуальный режим Vim удобен для редактирования, так как он способен отображать выделенный фрагмент текста, над которым проводится работа, *а также* действие всех мощных команд Vim в визуальном выделенном тексте. Это позволит вам выполнять гораздо более сложную работу над подсвеченным текстом, чем традиционные действия удаления и вставки в менее продвинутых редакторах.

Вы можете выделить видимую область в Vim не только указателем мыши, как в других редакторах, но еще и с помощью полезных команд перемещения и некоторых специальных команд визуального режима.

Например, если ввести **v** в командном режиме, запустится визуальный режим. При этом любые команды перемещения будут передвигать курсор и подсвечивать текст по мере того, как курсор переходит на новую позицию. Таким образом, в визуальном режиме команда «следующее слово» (**w**) перемещает курсор к следующему слову и подсвечивает выделенный текст. Другие перемещения соответствующим образом расширят выделенную область.

В визуальном режиме Vim использует некоторые специальные команды, благодаря которым удобно расширять выделенный текст, выбирая текстовый объект вокруг курсора. Например, курсор может находиться внутри «слова», одновременно внутри «предложения» и «абзаца». С помощью команд, расширяющих подсвеченный текст до текстового объекта, Vim позволяет увеличить визуальное выделение. Чтобы визуально выделить слово, используйте **aw** (в визуальном режиме).

Подсвечивать визуальные области буфера можно несколькими способами. В текстовом режиме просто введите **v**, чтобы включить или выключить визуальный режим. Если визуальный режим включен, то он выделяет и подсвечивает буфер, когда вы двигаете курсор. В **gvim** можно просто выделять текст мышью. Это установит визуальный флаг Vim. В табл. 8.2 показаны некоторые команды перемещений в визуальном режиме Vim.

Таблица 8.2. Команды перемещения в визуальном режиме Vim

Команда	Описание
<i>n</i> aw, <i>n</i> aW	Выделяет <i>n</i> слов, включая пробелы. Эта команда немного отличается от iw (см. ниже). Команда w ищет слова, разделенные знаками пунктуации, а W — пробелами
<i>n</i> iw, <i>n</i> iW	Выделяет <i>n</i> слов. Добавляет слова, но не пробелы. Команда w ищет слова, разделенные знаками пунктуации, а W — пробелами
as, is	Добавляет предложение или внутреннее предложение
ap, ip	Добавляет абзац или внутренний абзац

Для получения дополнительной информации о текстовых объектах и их использовании в визуальном режиме наберите:

```
:help text-objects
```

Мы рекомендуем вам поэкспериментировать с визуальным режимом, чтобы привыкнуть к нему. В частности, это отличный способ выбрать текст, к которому вы хотите применить команду замены, или когда требуется отфильтровать текст.

Расширенные регулярные выражения

Метасимволы, доступные при поиске и замене регулярных выражений **vi**, описаны в главе 6, в подразделе «Метасимволы в шаблонах поиска».

Vim предоставляет расширенные регулярные выражения, которые всегда доступны. Эффективность некоторых из них эквивалентна **egrep** (или **grep-E** в полностью совместимых с POSIX системах). Часть описаний в нижеприведенном списке взята из документации Vim.

- **\|** — указывает на варианты. Например, **a\|b** выбирает либо *a*, либо *b*. Тем не менее эта структура не ограничивается одиночными символами: **house\|home** выбирает либо *house*, либо *home*.
- **\&** — указывает конкатенацию. Конкатенация ищет соответствия части после последнего **\&**, но только если все предыдущие части совпадают. В справке Vim приведены такие примеры: **foobeep\&...** соответствует *foo* в *foobeep*, а **.*Peter\&.*Bob** соответствует строке, содержащей и *Peter*, и *Bob*.
- **\+** — сопоставляет одно или несколько предыдущих регулярных выражений. Это одиночный символ или группа символов, заключенных в круглые скобки. Обратите внимание на разницу между **\+** и *****. Символ ***** может означать пустое место, но для **\+** должно быть хотя бы одно совпадение. Например, **ho(use\|me)*** сопоставляет как *ho*, так и *home* и *house*, **ho(use\|me)+** не сопоставляет *ho*.

- `\=` — сопоставляет ноль или одно из предыдущих регулярных выражений. Это то же самое, что и оператор `?` в `egrep`.
- `\?` — сопоставляет ноль или одно из предыдущих регулярных выражений. Это то же самое, что и оператор `?` в `egrep`, только более привычно для людей, знакомых с `egrep` и `awk`.
- `\{...\}` — определяет *интервальное выражение*. Интервальные выражения описывают подсчитанное количество повторений. Для Vim достаточно, чтобы слеш предшествовал лишь левой фигурной скобке, а не правой. Далее *n* и *m* — целочисленные константы:
 - `\{n,m\}` — сопоставляет *n* с *m* в предыдущем регулярном выражении, сколько возможно. Важно ограничение, поскольку оно контролирует, сколько текста будет заменено во время команды замены¹. *n* и *m* являются неотрицательными числами (это включает ноль);
 - `\{n\}` — ищет ровно *n* повторений предыдущего регулярного выражения. Например, `(home\|house){2}` соответствует только *homehome*, *homehouse*, *househome* и *househouse*, но не более того;
 - `\{n, \}` — ищет по крайней мере *n* из предыдущего регулярного выражения, сколько возможно. Воспринимайте это как «как минимум *n*» повторений;
 - `\{, m\}` — ищет от нуля до *m* повторений в предыдущем регулярном выражении, сколько возможно;
 - `\{\}` — ищет ноль и более повторений в предыдущем регулярном выражении, сколько возможно (то же самое, что и `*`);
 - `\{-n, m\}` — ищет от *n* до *m* повторений в предыдущем регулярном выражении, как можно меньше;
 - `\{-n\}` — ищет *n* повторений из предыдущего регулярного выражения;
 - `\{-n, \}` — ищет как минимум *n* повторений из предыдущего регулярного выражения, как можно меньше;
 - `\{-, m\}` — ищет от нуля до *m* повторений в предыдущем регулярном выражении, как можно меньше.
- `~` — ищет последнюю заданную строку замены.
- `\(...\)` — обеспечивает группировку для `*`, `\+`, `\?` и `\=`, а также создает соответствующие подтексты, доступные в замещающей части команды замены (`\1`, `\2` и т. д.).

¹ Операторы `*`, `\+` и `\=` могут быть сведены к `\{0,\}`, `\{1,\}` и `\{0,1\}` соответственно, но первый вариант гораздо удобнее в использовании. Интервальные выражения были разработаны в Unix позднее, чем регулярные выражения.

- `\1` — сопоставляет ту же строку, что была сопоставлена первым подвыражением в `\( и \)`. Например, `\([a-z]\)`. `\1` сопоставляет *ata*, *ehe*, *tot* и т. п. `\2`, `\3` и т. д. можно использовать для представления второго, третьего и т. д. подвыражений.

Параметры `isident`, `iskeyword`, `isfname` и `isprint` определяют символы, которые появляются в идентификаторах, ключевых словах и именах файлов и выводятся на экран. Использование этих опций придает регулярным выражениям больше гибкости.

Vim предоставляет ряд дополнительных специальных последовательностей, которые служат условными обозначениями для некоторых непечатаемых символов, а также для часто используемых выражений в квадратных скобках. Vim называет их *классами символов* в соответствии с более ранним, оригинальным использованием в документации Unix. Они представлены в табл. 8.3.

Таблица 8.3. Символы регулярных выражений Vim и классы символов

Последовательность	Значение
<code>\a</code>	Алфавитный символ: то же, что и <code>[A-Za-z]</code>
<code>\A</code>	Неалфавитный символ: то же, что и <code>[^A-Za-z]</code>
<code>\b</code>	Backspace
<code>\d</code>	Цифра: то же, что и <code>[0-9]</code>
<code>\D</code>	Не цифра: то же, что и <code>[^0-9]</code>
<code>\e</code>	Escape
<code>\f</code>	Сопоставляет символ имени файла, как определено параметром <code>isfname</code>
<code>\F</code>	Как <code>\f</code> , только без цифр
<code>\h</code>	Первый символ слова: <code>[A-Za-z_]</code>
<code>\H</code>	Не первый символ слова: <code>[^A-Za-z_]</code>
<code>\i</code>	Сопоставляет любой символ идентификатора, как определено параметром <code>isident</code>
<code>\I</code>	Как <code>\i</code> , только без цифр
<code>\k</code>	Сопоставляет любой символ ключевого слова, как определено параметром <code>iskeyword</code>
<code>\K</code>	Как <code>\k</code> , только без цифр
<code>\l</code>	Символ нижнего регистра: то же, что и <code>[a-z]</code>
<code>\L</code>	Символ не нижнего регистра: то же, что и <code>[^a-z]</code>

Последовательность	Значение
\n	Сопоставляет новую строку. Можно использовать для сопоставления многострочных шаблонов
\o	Восьмеричная цифра: то же, что и [0-7]
\O	Не восьмеричная цифра: то же, что и [^0-7]
\p	Сопоставляет любой печатаемый символ, как определено параметром isprint
\P	Как \p, только без цифр
\r	Возврат каретки
\s	Сопоставляет символ пробела (в точности пробел или табуляцию)
\S	Сопоставляет все не являющееся пробелом или табуляцией
\t	Сопоставляет табуляцию
\u	Символ верхнего регистра: то же, что и [A-Z]
\U	Символ не верхнего регистра: то же, что и [^A-Z]
\w	Символ слова: то же, что и [0-9A-Za-z_]
\W	Символ, не являющийся словом: то же, что и [^0-9A-Za-z_]
\x	Шестнадцатеричная цифра: то же, что и [0-9A-Fa-f]
\X	Не шестнадцатеричная цифра: то же, что и [^0-9A-Fa-f]
_x	Где x является любым из вышеприведенных символов: сопоставляет тот же класс символа, но включая новую строку

И наконец, Vim предоставляет целый ряд дополнительных, более специфичных способов сопоставления регулярных выражений. Дополнительную информацию вы можете найти в `:help regexp`. Тем не менее мы считаем, что списка и таблицы, приведенных в этом разделе, вполне достаточно, чтобы занять вас на какое-то время.

Расширенный откат изменений

Помимо удобной функции отмены произвольного количества правок, Vim предлагает интересное дополнение под названием «*ветвление отмен*».

Чтобы использовать эту функцию, для начала решите, в каком объеме вы хотите контролировать отмену правок. Используйте параметр `undolevels` для определения количества отменяемых изменений, которые можно осуществлять во время

сеанса редактирования. По умолчанию это одна тысяча, что, вероятно, более чем достаточно для большинства пользователей. Если вам нужна совместимость с `vi`, установите значение параметра `undolevels` равным нулю:

```
:set undolevels=0
```

В `vi` команда отмены (`u`) в основном представляет собой переключение между текущим состоянием файла и его самым последним изменением. Первая *отмена* возвращает к состоянию до последнего изменения. Следующая *отмена* восстанавливает отмененное изменение. Vim ведет себя по-другому, а значит и команды применяются тоже иначе.

Вместо того чтобы переходить к самому последнему изменению, повторные вызовы команды `u` в Vim откатывают состояние файла через самые последние изменения по порядку на столько изменений, сколько определено параметром `undolevels`. Поскольку команда `u` перемещает только назад, нам потребуется команда для отмены отката и повторного применения изменений. Vim выполняет это действие с помощью команды `:redo` или `Ctrl+R`. Комбинация `Ctrl+R` может принимать числовой префикс для повторения нескольких изменений за раз.

Когда вы откатываете изменения и возвращаете их с помощью команд отмены и повторного выполнения, Vim сохраняет карту состояния файла и знает, когда был выполнен последний возможный откат. Когда все возможные отмены исчерпаны, Vim сбрасывает *измененное* состояние файла, что позволяет завершить работу без суффикса `!`. Хотя для обычного пользователя это скромное преимущество, оно более полезно для скриптов, в которых важно измененное состояние файла.

Для большинства пользователей достаточно простой отмены и повторного выполнения изменений. Но давайте рассмотрим более сложный сценарий. Что, если вы осуществили семь правок в файле и отменили три? Пока все в порядке, ничего необычного. А теперь предположим, что после отмены трех из семи действий вы внесли изменение, отличное от следующего в коллекции изменений Vim. Редактор определяет этот момент в истории изменений как *ветвь*, из которой выходят другие события. По этому пути вы теперь можете перемещаться вперед и назад в хронологическом порядке, но из точки ветвления допускается перемещаться вперед по любому пути записанных изменений.

Чтобы получить полное описание того, как перемещаться по изменениям в виде дерева, воспользуйтесь справкой Vim:

```
:help usr_32.txt
```

Мы также рекомендуем ознакомиться с некоторыми плагинами `undo` по адресу <https://vimawesome.com/?q=undo>.

Инкрементный поиск

При *инкрементном поиске* редактор перемещает курсор по файлу, подбирая текст, пока вы вводите шаблон поиска. После нажатия **Enter** поиск завершается¹. Данная функция Vim включается с помощью параметра `incsearch`. Когда эта опция активна, Vim подсвечивает текст, который совпадает с введенным вами ранее, изменяя его по мере ввода вами дополнительных символов в шаблон поиска.

Подобное поведение программы поначалу может сбить вас с толку. Однако через какое-то время вы привыкнете и в конце концов будете недоумевать, как вообще обходились без этой функции. Мы настоятельно рекомендуем добавить `set incsearch` в ваш файл `.vimrc`.

Прокрутка слева направо

По умолчанию `vi` и Vim переносят длинные строки, которые не помещаются на экране. Таким образом, одна логическая строка файла может занимать несколько физических на вашем мониторе.

Бывают случаи, когда предпочтительнее, чтобы длинная строка просто исчезла за правым краем экрана, а не была перенесена. Переход на эту строку, а затем перемещение вправо «прокрутит» экран в сторону.

В Vim числовой параметр (`sidescroll`, по умолчанию равен нулю) определяет, насколько «далеко» прокручивать экран, а булева опция (`wrap`, по умолчанию `true`) определяет, переносятся ли строки или исчезают за краем экрана. Таким образом, чтобы добиться боковой прокрутки, следует использовать команду `:set nowrap` и задать параметру `sidescroll` подходящее значение, например 8 или 16.

Резюме

На протяжении многих лет `vi` оставался стандартным инструментом редактирования текста в Unix. В свое время `vi` был практически революционным со своей ориентацией на два режима и философией сенсорного редактирования. Vim — это естественное продолжение `vi`, следующий эволюционный шаг для мощного редактирования и управления текстом.

- Vim расширяет `vi`, поднимая планку, заданную более старым редактором. Хотя другие редакторы также основаны на `vi`, они не стали такими же популярными, как Vim.

¹ В редакторе Emacs всегда был инкрементный поиск.

- Vim предлагает намного (намного!) больше, чем `vi`, настолько, что теперь он стал новым стандартом. Большинство Unix-подобных систем связывают команду `vi` с Vim.
- Vim подходит как начинающим, так *и* опытным пользователям. Новичкам он предлагает разнообразные обучающие инструменты и простые модели, в то время как экспертам — мощные расширения `vi` вместе с платформой, на которой можно улучшить и настроить Vim под свои конкретные нужды.
- Vim работает везде. Он доступен для большинства операционных систем. Возможно, Vim есть не буквально везде, но он очень близок к этому!
- Vim бесплатен. Более того, Vim — это пожертвование. Работа, которую проделал Брэм Моленар по созданию, усовершенствованию, обслуживанию и поддержке Vim, является одним из самых значимых подвигов на рынке бесплатного ПО. Если вам нравится *его* работа, Брэм приглашает вас познакомиться с его любимым делом — помощью детям в Уганде. Больше информации доступно на сайте ICCF Holland (<http://iccf-holland.org>) или в справке Vim `:help uganda`.

Графический Vim (gvim)

Vim стартовал как расширение `vi`, в которое добавлялись функции, недоступные в оригинальном `vi`. В итоге Vim стал самостоятельным приложением, дополненной и улучшенной версией и без того великолепного `vi`. Vim, не обремененный требованиями POSIX, смог сделать это быстро благодаря обратной связи пользователей.

Уже к моменту выпуска седьмого издания данной книги Vim предлагал сформированные и всеобъемлющие функции графического пользовательского интерфейса (GUI), которые мы рассмотрим в текущей главе. На протяжении последующих лет Vim продолжил улучшать GUI, и на сегодняшний день он лучше, чем когда-либо.

Довольно долго пользователи жаловались, что у `vi` и его клонов отсутствовал GUI. Во времена религиозных войн между фанатами Emacs и `vi` главным аргументом в пользу того, что использовать `vi` — дохлый номер, было отсутствие в нем GUI. На этот тезис уже давно дан ответ.

Клоны и другие похожие на `vi` программы создали свои собственные версии GUI. Графический Vim называется `gvim`. Он предлагает надежные и расширяемые функции и средства графического пользовательского интерфейса. В этой главе мы рассмотрим наиболее полезные из них.

Некоторые графические возможности `gvim` охватывают обычные функции Vim, в то время как другие обеспечивают удобство управления мышью, к чему привыкло большинство современных пользователей. Хотя некоторые опытные пользователи Vim могут съехидиться при мысли о внедрении GUI в работу их основного редактора, `gvim` основательно продуман и реализован. Он предлагает функции и свойства, охватывающие широкий спектр возможностей пользователей, облегчая обучение для новичков и предоставляя пользователям со стажем дополнительные инструменты редактирования. Это хороший компромисс.



В `gvim` для MS-Windows есть пункт меню, называемый `easy gvim` (простой `gvim`). Это действительно ценно для людей, которые никогда не работали с Vim, но, по иронии судьбы, совсем не просто для опытных пользователей.

В этой главе мы сначала обсудим общие понятия и функции GUI `gvim` и дадим краткий обзор взаимодействий с мышью. Кроме того, мы подробно рассмотрим отличия между различными окружениями `gvim`, а также моменты, которые вы должны знать о них. В частности, мы сфокусируемся на системах MS-Windows и X Window System, двух основных графических платформах¹. Мы также предоставим краткий список параметров GUI с описанием.

Знакомство с `gvim`

`gvim` обладает всеми возможностями, мощностью и свойствами Vim, привнося удобство и интуитивность в окружение GUI. `gvim` предоставляет современный опыт использования GUI от традиционных пунктов меню до визуальной подсветки. Для опытных пользователей `vi`, работающих через консоль, `gvim` по-прежнему предоставляет привычные функции и не разрушает парадигму, которая обеспечила `vi` репутацию мощного редактора.

Запуск `gvim`

Графический интерфейс Vim можно запустить с помощью команды `gvim` или `vim -g`. В MS-Windows после установки программы в контекстное меню добавляется пункт «редактировать с помощью Vim». Это обеспечивает быстрый и простой доступ к `gvim` с помощью интеграции его в окружение Windows.

Конфигурационные файлы и параметры воспринимаются `gvim` немного по-другому, чем в Vim. `gvim` считывает и выполняет два загрузочных файла: `.vimrc` и `.gvimrc`. Несмотря на то что вы можете поместить в `.vimrc` специфические параметры `gvim`, лучше определить их в `.gvimrc`. Это обеспечивает разделение настроек обычного Vim и `gvim`, а также гарантирует правильное поведение при запуске. Например, команда `:set columns=100` не работает в оригинальном Vim и выдаст ошибку при запуске².

¹ Особенно сейчас, когда MS-Windows предлагает подсистему Windows для Linux (WSL), полноценную подсистему GNU/Linux с возможностью для пользователей устанавливать их любимый дистрибутив GNU/Linux. Мы описываем один из способов запуска собственного для Linux `gvim` в WSL и отображаем сеанс изначально в Windows.

² Мы обнаружили, что Vim не всегда выдает ошибку и на самом деле запускается без сообщения об ошибке. Предупреждение появляется, потому что, даже если ошибки нет, Vim пытается соответствующим образом перенастроить определение вашей консоли/экрана/терминала. Некоторые терминалы или консоли адаптируются корректно, но наиболее вероятно, что в итоге поведение вашего терминала окажется наполовину правильным. Это может повлиять на поведение других приложений, которые зависят от определений терминала и используют их.

Если системный файл `gvimrc` существует (обычно в `$VIM/gvimrc`), то он выполняется. Администраторы могут использовать эту общесистемную конфигурацию для установки обычных параметров для всех пользователей. Это обеспечивает базовую конфигурацию, чтобы у пользователей был общий опыт редактирования.

Более опытные пользователи Vim могут добавить свои собственные пользовательские настройки и функции. После того как `gvim` считывает дополнительную конфигурацию системы, он ищет вспомогательную информацию о конфигурации в четырех местах в приведенном ниже порядке и прекращает поиск, когда найдет хотя бы одно из искомого.

- Команда `exrc`, хранящаяся в переменной окружения `$GVIMINIT`.
- Пользовательский файл `gvimrc`, обычно хранящийся в `$HOME/.gvimrc`. Он становится исходным, если был найден.
- Если в окружении Windows `$HOME` не установлен, то `gvim` ищет в `$VIM/_gvimrc`. Это обычная ситуация для пользователей Windows, однако для пользователей, у которых установлены Unix-подобные системы и у которых, скорее всего, установлена переменная `$HOME`, она сильно отличается. Одним из примеров является популярный набор инструментов Unix Cygwin.
- Если `_gvimrc` не найден, `gvim` в итоге ищет `.gvimrc`.

Если `gvim` находит непустой файл для выполнения, имя этого файла сохраняется в переменной `$MYGVIMRC` и дальнейшая инициализация прекращается.

Существует еще одна опция настройки. Если в только что описанной каскадной последовательности инициализации задан параметр `exrc`:

```
:set exrc
```

`gvim` дополнительно ищет в текущем каталоге один из файлов: `.gvimrc`, `.exrc` или `.vimrc` и делает его исходным, если до этого он не был обнаружен как файл инициализации и не был выполнен.



В окружении Unix существуют проблемы безопасности, касающиеся локальных каталогов, содержащих конфигурационные файлы (как `.gvimrc`, так и `.vimrc`). По умолчанию `gvim` накладывает некоторые ограничения на выполнение этих файлов, устанавливая параметр `secure`, если файл не принадлежит пользователю. Это помогает защититься от вредоносного кода. Если вы хотите быть уверены, установите явно параметр `secure` в вашем файле `.vimrc` или `.gvimrc`. Чтобы получить дополнительную информацию о параметре `secure`, см. раздел «Параметры Vim 8.2» приложения Б.

Использование мыши

Мышь в `gvim` полезна в любом режиме редактирования. Рассмотрим стандартные режимы Vim и разберем, каким образом `gvim` использует мышь в каждом из них.

- *Командный режим ex* — вы переходите в этот режим, когда открываете командный буфер в нижней части окна, вводя двоеточие (:). Если окно находится в командном режиме, можно использовать мышь для перемещения курсора в любом месте командной строки. Этот режим включен по умолчанию или при активации флага `c` в параметре `mouse`.
- *Режим ввода* — это режим для набора текста. Если вы щелкнете на буфере, который находится в режиме ввода, мышь переместит курсор и позволит вам сразу же начать вводить текст в этом месте. Данный режим включен по умолчанию или при активации флага `i` в параметре `mouse`.

Поведение мыши в режиме ввода обеспечивает более простое и интуитивное позиционирование в стиле «укажи и щелкни». В частности, оно позволяет избежать необходимости выходить из режима набора текста, перемещаться с помощью мыши, команд перемещения или других методов, а затем заново активировать режим ввода.

На первый взгляд это кажется прекрасной идеей, но на практике будет привлекательно только для определенного круга пользователей. Ветеранам Vim этот режим может показаться скорее раздражающим, чем помогающим.

Давайте представим, что может случиться, если вы, находясь в режиме ввода, попытаетесь переключиться из `gvim` в какое-то другое приложение. Когда вы возвращаетесь в окно `gvim` по щелчку кнопкой мыши, точка, на которую вы нажимаете, теперь является точкой вставки текста, и, вероятно, не той, которая вам нужна. В однооконном сеансе `gvim` вы можете оказаться не в том месте, где вы изначально работали, а в многооконном режиме указатель мыши может оказаться в совсем другом окне. В конечном итоге появляется риск ввода текста не в тот файл!

- *Командный режим vi* используется каждый раз, когда вы не находитесь в режиме ввода или в командной строке. Щелчок кнопкой мыши на экране просто оставляет курсор на символе, на который вы нажали. Данный режим включен по умолчанию или при активации флага `n` в параметре `mouse`.

Командный режим `vi` предоставляет прямой и простой метод смены позиции курсора, но предлагает довольно неуклюжую поддержку перемещения за пределы верхней и нижней части видимого окна. Удерживая кнопку мыши, плавно перемещайтесь вверх или вниз экрана; `gvim` совершит прокрутку вверх и вниз соответственно. Если прокрутка остановилась, подвигайте мышью

влево и вправо. Непонятно, почему командный режим `vi` работает подобным образом.

Еще одним недостатком данного режима, особенно для новичков, является то, что пользователи могут привыкнуть к управлению мышью и использовать его как основной метод позиционирования. Это может снизить мотивацию изучать команды навигации Vim и, следовательно, его мощные методы редактирования. И наконец, он создает ту же потенциальную путаницу, что и режим ввода.

Кроме того, `gvim` предлагает *визуальный* режим, также известный как режим *выбора*. Он включен по умолчанию или при активации флага `v` в параметре `mouse`. Визуальный режим наиболее универсален, потому что позволяет вам выбрать текст перетаскиванием мыши, при этом подсвечивая выделенный элемент. Его можно использовать в сочетании с командным режимом, режимом ввода и командным режимом `vi`.

В параметре `mouse` можно указать любое сочетание флагов. Данный синтаксис продемонстрирован в следующих командах:

- `:set mouse=""` — отключить все действия мыши;
- `:set mouse=a` — включить все действия мыши (по умолчанию);
- `:set mouse+=v` — включить визуальный режим (`v`). В этом примере использован синтаксис `+=` для добавления флага в текущие настройки параметра `mouse`;
- `:set mouse-=c` — отключить действия мыши в командном режиме `vi` (`c`). В этом примере использован синтаксис `-=` для удаления флага из текущих настроек параметра `mouse`.

Новички могут предпочесть включить все настройки, в то время как опытные пользователи — полностью выключить мышь (как делает это один из нас).

Если вы используете мышь, мы рекомендуем вам выбирать знакомое поведение с помощью команды `gvim :behave`, которая в качестве аргумента принимает либо `mswin`, либо `xterm`. Как понятно из названий аргументов, `mswin` устанавливает параметры, чтобы как можно более точно воспроизвести поведение Windows, в то время как `xterm` — X Window System.

В Vim есть несколько других опций, включая `mousefocus`, `mousehide`, `mouse model` и `selectmode`. Информацию об этих параметрах вы найдете во встроенной в Vim документации.

По умолчанию `gvim` поддерживает и мышь с колесиком прокрутки, предсказуемо прокручивая экран или окно вверх и вниз, вне зависимости от настроек параметра `mouse`.

Полезные пункты меню

gvim упрощает некоторые из наиболее скрытых команд Vim путем действий, доступных из меню. Стоит упомянуть два из них.

Меню Window в gvim

Меню *Window* в gvim содержит множество полезных и распространенных команд управления окнами Vim: команды, которые разделяют одно GUI-окно на несколько областей. Возможно, вам будет удобно «оторвать» это меню, как показано на рис. 9.1, чтобы оно было всегда под рукой. Результат показан на рис. 9.2. (Вскоре мы обсудим подобные меню.)



Рис. 9.1. Меню Window в gvim

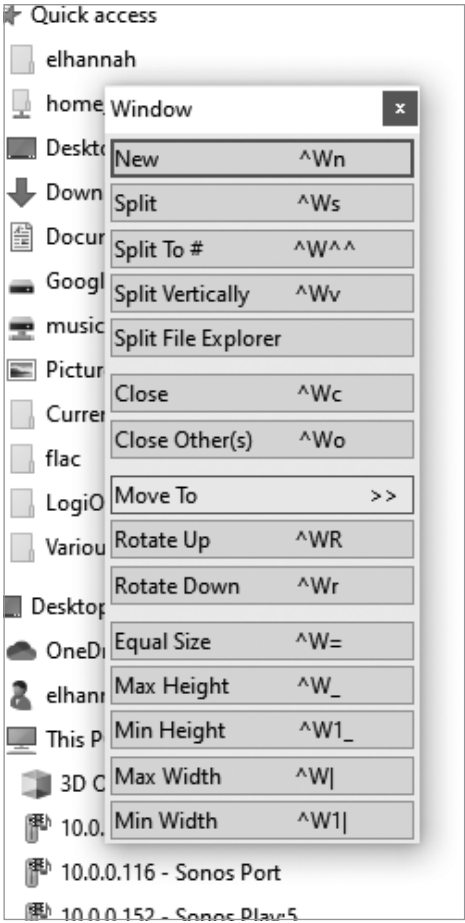


Рис. 9.2. Плавающее меню Window в gvim

Обратите внимание, как меню на рис. 9.2 перемещается и всплывает поверх другого приложения. Это хороший способ сделать часто используемые функции доступными, но не мешающими редактированию. Оба способа удобны для обычных операций выбора, вырезания, копирования, удаления и вставки. Пользователи других GUI-редакторов постоянно используют подобную функцию, но она будет полезна и для старожилов Vim, а особенно для тех, кто взаимодействует с буфером обмена Windows предсказуемым образом.

Всплывающее меню gvim, вызываемое щелчком правой кнопкой мыши

Когда вы щелкаете правой кнопкой мыши внутри редактируемого буфера, на экране появляется выпадающее меню (рис. 9.3).

Если выделен любой текст (подсвечен), при щелчке правой кнопкой мыши всплывает другое меню (рис. 9.4).

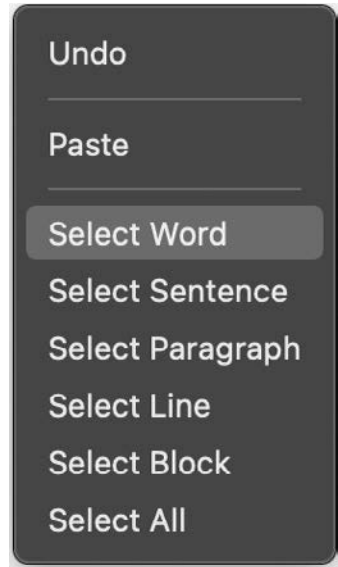


Рис. 9.3. Общее меню редактирования gvim

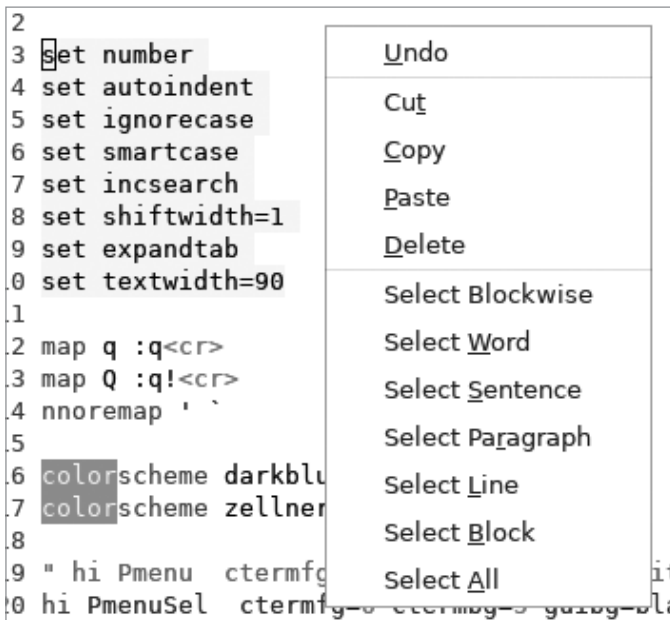


Рис. 9.4. Меню редактирования gvim при выбранном тексте

Настройка полос прокрутки, меню и панелей инструментов

`gvim` предоставляет обычные элементы управления GUI, такие как полосы прокрутки, меню и панели инструментов. Как и большинство современных GUI-приложений, эти элементы управления можно настроить.

Окно `gvim` по умолчанию отображает несколько меню и панель инструментов наверху, как показано на рис. 9.5.

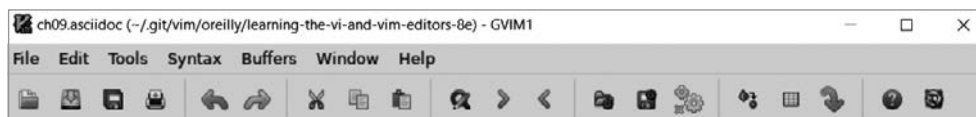


Рис. 9.5. Верхняя часть окна `gvim` (версия Linux)

Полосы прокрутки

Полосы прокрутки, позволяющие вам быстро перемещаться вверх и вниз или вправо и влево по файлу, в `gvim` опциональны. Их можно отобразить или скрыть с помощью параметра `guioptions`, описанного в конце главы, в разделе «Параметры GUI и краткое описание команд».

Поскольку обычное поведение Vim заключается в отображении всего текста в файле (с переносом строк в окне при необходимости), стоит отметить, что у горизонтальной полосы прокрутки нет никакой цели в стандартно настроенных сеансах `gvim`.

Включить и выключить левую и правую полосы прокрутки вы можете, изменяя параметр `r` или `l` в `guioptions`. `l` отвечает за отображение левой полосы прокрутки, а `r` — правой. Верхние регистры `L` и `R` указывают `gvim` показать левую или правую полосу прокрутки только при вертикальном разделении окна.

Горизонтальная полоса прокрутки управляется с помощью включения или исключения `b` из `guioptions`.

И да, вы *можете* прокручивать одновременно вправо и влево! Точнее, прокрутка одной из полос заставляет другую переместиться в том же направлении. Может оказаться достаточно удобным настроить полосы прокрутки с обеих сторон. В зависимости от местоположения указателя мыши вы просто щелкаете и перетаскиваете ближайшую полосу прокрутки.



Многие параметры, включая `guioptions`, управляют несколькими вариантами поведения и поэтому могут включать много флагов по умолчанию. Новые флаги даже могут быть добавлены в будущих версиях `gvim`. Поэтому важно использовать синтаксис `+=` и `-=` в команде `:set guioptions`, чтобы избежать удаления нужного поведения. К примеру, `:set guioptions+=l` добавляет в `gvim` параметр «полоса прокрутки всегда слева», оставляя другие компоненты в строке `guioptions` нетронутыми.

Меню

В `gvim` есть полностью настраиваемое меню. В этом разделе мы опишем параметры меню по умолчанию, которые вы видели ранее на рис. 9.5, и покажем, как вы можете управлять его макетом.

На рис. 9.6 показан один пример использования меню: здесь мы выбираем `Global Settings` в меню `Edit`.

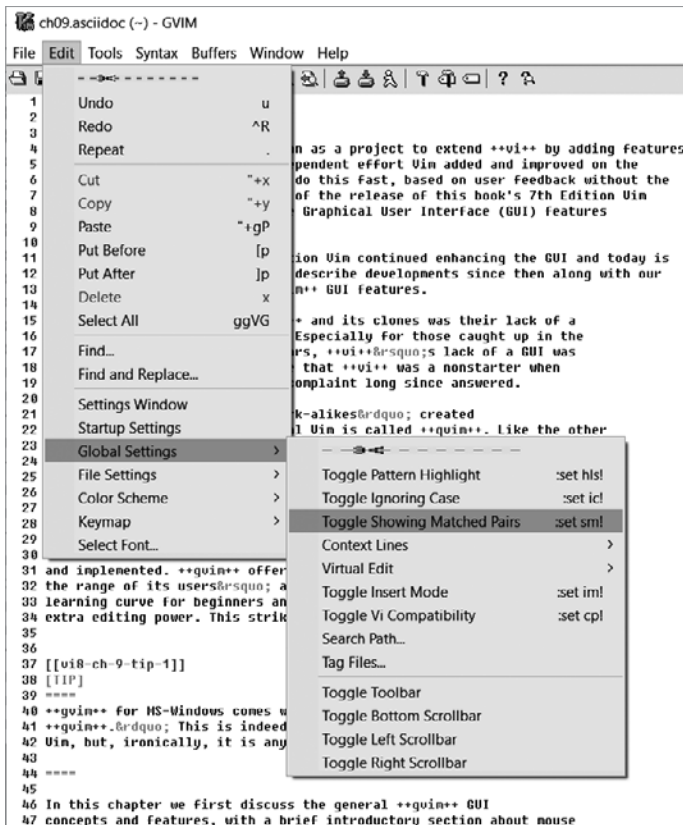


Рис. 9.6. Каскадное меню редактирования (версия Windows)

Имейте в виду, что эти параметры меню являются просто оболочками команд Vim. Фактически это возможность создавать и настраивать свои собственные пункты меню, что мы вскоре и обсудим.



Если вы обратите внимание на меню, содержащее клавиши и команды, показанные с правой стороны, вы сможете со временем выучить команды Vim. Например, на рис. 9.6, несмотря на то что новичкам привычнее найти знакомую команду Undo в меню Edit, где она находится в других популярных приложениях, гораздо быстрее и проще использовать клавишу `u`, которая показана в меню.

Как показано на рис. 9.6, каждое меню начинается с пунктирной линии, содержащей изображение ножниц. Щелчок в этом месте «отрезает» меню, чтобы создать плавающее окно, в котором все его параметры доступны без перехода в строку меню. Если вы нажмете на пунктирную линию над пунктом меню **Toggle Pattern Highlight** из рис. 9.6, вы увидите нечто похожее на рис. 9.7. Вы можете расположить свободно плавающее меню в любом месте вашего экрана.

Все команды этого подменю доступны сразу же после однократного нажатия на окно подменю. Каждому пункту меню соответствует определенная клавиша. Если пункт меню сам по себе является подменю, он сопровождается кнопкой со знаками «больше» (выглядят как указывающие направо стрелки) в правой части кнопки. Щелчок на этих стрелках раскроет подменю.

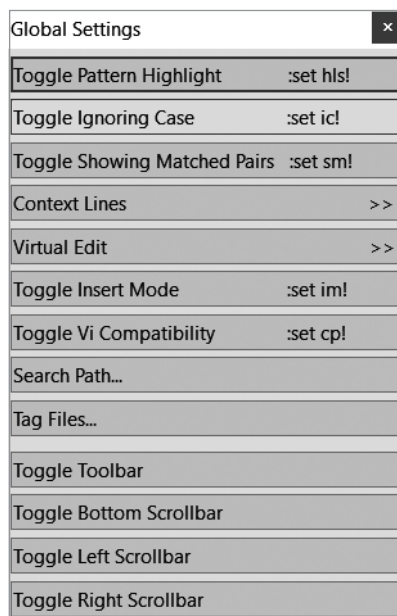


Рис. 9.7. Плавающее меню

Настройка основного меню

`gvim` хранит определения меню в файле с именем `$VIMRUNTIME/menu.vim`.

Определение пунктов меню похоже на переназначение. Как вы уже знаете из подраздела «Команда `map`» главы 7, переназначить клавишу можно следующим образом:

```
:map <F12> :set syntax=html<CR>
```

Манипуляции с меню происходят похожим образом.

Предположим, что вместо переназначения клавиши F12 для установки синтаксиса `html` вам нужен для этого специальный пункт HTML в вашем меню File. Используйте команду `:amenu`:

```
:amenu File.HTML :set filetype=html<CR>
```

Четыре символа `<CR>` вводятся, как показано в примере, и являются частью команды.

Теперь взгляните на ваше меню File. Вы должны в нем увидеть новый пункт HTML, как показано на рис. 9.8. Используя `amenu` вместо `menu`, вы обеспечиваете доступность этого пункта во всех режимах (командном, режиме ввода или обычном).



Команда `menu` добавляет пункт в меню только в командном режиме; пункт не появится в режиме ввода и обычном режиме.

Расположение пункта меню задается серией каскадных меню, чьи имена разделены точками (.). В нашем примере `File.HTML` добавил пункт меню HTML в меню File. Последняя запись в ряду является той, которую вы хотите добавить. Здесь мы добавили пункт в существующее меню, но скоро вы увидите, что можно также с легкостью создать целую каскадную серию подобных элементов.

Обязательно проверьте работоспособность вашего нового пункта меню. Например, мы начали редактировать файл, который Vim воспринимает как XML-файл, что видно в строке состояния на рис. 9.9 (чтобы узнать, как установить строку состояния, перейдите в подраздел «Хитрый трюк» в главе 12). Мы настроили строку состояния таким образом, что Vim и gvim отображают текущий активный тип файла по правому краю.



Рис. 9.8. Элемент меню HTML в меню File

0x6F line:1, col:2 All [xml]

Рис. 9.9. Строка состояния, показывающая XML-файл перед новым действием меню

После вызова нашего нового пункта меню HTML строка состояния Vim проверяет, работает ли данный пункт, и то, что тип файла теперь HTML (рис. 9.10).

0x6F line:1, col:2 All [html]

Рис. 9.10. Строка состояния, показывающая HTML-файл после нового действия меню

Обратите внимание, что у добавленного нами пункта меню HTML нет горячей клавиши или команды с правой стороны. Давайте это исправим.

Сначала удалим существующую запись:

```
:aunmenu File.HTML
```



Если вы добавляли пункт меню для командного режима `ex`, используя только команду `menu`, то его можно удалить с помощью `unmenu`.

Затем добавим новый пункт меню HTML, который отображает команду, связанную с этим пунктом:

```
:amenu File.HTML<TAB>filetype=html<CR> :set  
filetype=html<CR>
```

Теперь спецификация пункта меню сопровождается `<TAB>` (вводится буквально) и `filetype=html<CR>`. В целом, чтобы отобразить текст с правой стороны меню, поместите его после строки `<TAB>` и завершите символами `<CR>`. На рис. 9.11 показано итоговое меню `File`.



Если вам нужны пробелы в тексте описания пункта меню (или в самом имени меню), обособьте их обратным слешем (`\`). Иначе Vim интерпретирует все, что после первого пробела, как определение действия меню. В предыдущем примере, если бы вы захотели в качестве текста описания ввести `:set filetype=html` вместо `filetype=html`, команда `:amenu` имела бы следующий вид:

```
:amenu File.HTML<TAB>set\ filetype=html<CR> :set filetype=html<CR>
```

ch09.asciidoc (~/.git/vim/oreilly/l

File	Edit	Tools	Syntax	B
Open...				:e
Split-Open...				:sp
Open Tab...				:tabnew
New				:enew
Close				:close
Save				:w
Save As...				:sav
Split Diff With...				
Split Patched By...				
HTML			filetype=html	
Print				
Save-Exit				:wqa
Exit				:qa

Рис. 9.11. Пункт меню HTML, отображающий связанную команду

Мы не рекомендуем изменять определения меню по умолчанию, вместо этого лучше создать отдельные независимые записи. Вам потребуется определить новое меню на корневом уровне, но это так же просто, как добавить запись в существующее меню.

Продолжая работу над нашим примером, давайте создадим новое дерево меню под названием **МоеМеню** (MyMenu) и добавим в него пункт HTML. Для начала удалите пункт HTML из меню File:

```
:aunmenu File.HTML
```

Затем введите команду:

```
:amenu MyMenu.HTML<TAB>filetype=html :set filetype=html<CR>
```

На рис. 9.12 показано, как может выглядеть ваша строка меню.

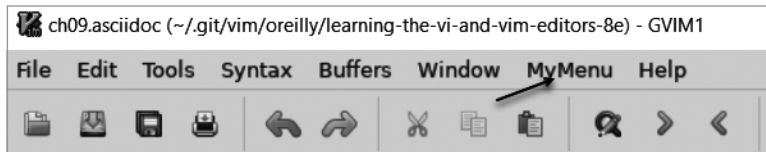


Рис. 9.12. Строка меню с добавленным меню МоеМеню

Команды меню предлагают более изысканный контроль над поведением меню, например позволяют узнать, указывает ли команда какое-либо действие, или даже сделать пункт меню невидимым. Мы обсудим эти возможности в следующем разделе.

Дополнительные настройки меню

Теперь, когда мы узнали, насколько легко изменять и расширять меню **gvim**, давайте рассмотрим другие примеры настройки и управления.

В предыдущем примере мы не указывали, где разместить **МоеМеню**, поэтому **gvim** произвольно поместил его между меню **Window** и **Help**. **gvim** позволяет нам управлять позицией с помощью так называемого *приоритета*, который представляет собой числовое значение, присвоенное каждому меню, чтобы определить его местоположение в строке меню. Чем больше это значение, тем правее расположено меню. К сожалению, понимание приоритета пользователями прямо противоположно тому, как оно определяется **gvim**. Чтобы правильно его определить, посмотрите на порядок расположения меню на рис. 9.5 и сравните его с приоритетами меню **gvim** по умолчанию, список которых представлен в табл. 9.1.

Таблица 9.1. Приоритеты меню gvim по умолчанию

Меню	Приоритет
File	10
Edit	20
Tools	40
Syntax	50
Buffers	60
Window	70
Help	9999

Большинство пользователей считают приоритет меню File выше Help (поэтому File расположено слева, а Help справа), но на самом деле все наоборот. Поэтому просто рассматривайте значение приоритета как показатель того, насколько далеко вправо отображается меню.

Вы можете определить приоритет меню, присоединив его числовое значение к команде меню. Если значение не указано, присваивается значение по умолчанию 500, что объясняет местоположение МоеМеню в предыдущем примере: оно оказалось между Window (приоритет 70) и Help (приоритет 9999).

Предположим, мы хотим, чтобы наше новое меню располагалось между меню File и Edit. Нам нужно присвоить МоеМеню числовой приоритет больше 10 и меньше 20. Следующая команда приведет к желаемому результату:

```
:15amenu MyMenu.HTML<TAB>filetype=html :set filetype=html<CR>
```



Как только меню создано, его позиция фиксируется для всего сеанса редактирования и не меняется в ответ на дополнительные команды, воздействующие на это меню. Например, вы не можете изменить позицию меню, добавив в него новый пункт и добавив в команду префикс с другим значением приоритета.

Еще больше путаницы в приоритеты и позиции меню вносит возможность управлять размещением пунктов меню *внутри* меню. Пункты меню с высоким приоритетом появляются в меню ниже, чем пункты с более низким приоритетом, однако синтаксис в этом случае используется другой.

Мы расширим один из предыдущих примеров меню, присвоив пункту HTML очень высокий приоритет (9999), в результате чего он появится в нижней части меню File:

```
:amenu .9999 File.HTML <TAB>filetype=html<CR> :set filetype=html<CR>
```

Почему перед 9999 стоит точка? Вам необходимо указать здесь два приоритета, разделенные точкой: один для File и другой для HTML. Мы оставляем приоритет File пустым, потому что это уже существующее меню и его нельзя изменить.

В целом приоритеты для пунктов меню отображаются между местоположением пункта меню и его определением. Для каждого уровня в иерархии меню вы должны указать приоритет или использовать точку, чтобы обозначить, что вы оставляете его пустым. Таким образом, если вы добавляете пункт глубоко в иерархию меню, например после Edit ► Global Settings ► Context lines ► Display, и хотите присвоить Display приоритет 30, то укажите его как ...30. Размещение вместе с приоритетом будет выглядеть следующим образом:

```
Edit.Global\ Settings.Context\ lines.Display ...30
```

Как и в случае с приоритетами меню, приоритеты пунктов меню фиксируются сразу после присвоения.

И наконец, вы можете управлять «пробелами» в меню с помощью разделителей меню gvim. Используйте то же определение, которое вы бы использовали для добавления пункта меню, но вместо имени команды введите дефис (-) перед и после него. Посмотрите строку в следующем примере с идентификаторами 2 и 3.

Собираем все вместе

Теперь мы знаем, как создавать, размещать и настраивать меню. Давайте сделаем наш пример постоянной частью окружения gvim, добавив рассмотренные команды в файл .gvimrc. Последовательность строк должна выглядеть примерно следующим образом:

```
" add XML/HTML/XHTML menu between File and Edit menus
❶ 15amenu MyMenu.XML<TAB>filetype=xml :set filetype=xml<CR>
❷ amenu ❸.600 MyMenu.-Sep- :
❹ amenu ❺.650 MyMenu.HTML<TAB>filetype=html :set filetype=html<CR>
❻ amenu ❼.700 MyMenu.XHTML<TAB>filetype=xhtml :set filetype=xhtml<CR>
```

Теперь у нас есть высококлассное персонализированное меню с быстрым доступом к трем часто используемым командам. В этом примере стоит отметить несколько важных моментов.

- Первая команда (❶) использует префикс 15, указывая gvim использовать приоритет 15. Для ненастроенного окружения эта команда помещает новое меню между File и Edit.
- Следующие команды (❷, ❹ и ❼) не указывают приоритет, потому что, как только приоритет определен, другие значения не учитываются.

- Мы использовали синтаксис приоритета подменю (**3**, **5** и **7**) после первой команды, чтобы обеспечить правильный порядок для каждого нового пункта. Обратите внимание, что мы начали с первого определения `.600`. Это гарантирует, что пункт подменю будет помещен за первым определенным нами пунктом, потому что мы не присвоили *этой* приоритет и он по умолчанию равен 500.

Для еще более удобного доступа нажмите на линию «ножницы», чтобы у вас было ваше персонализированное плавающее меню, как показано на рис. 9.13.

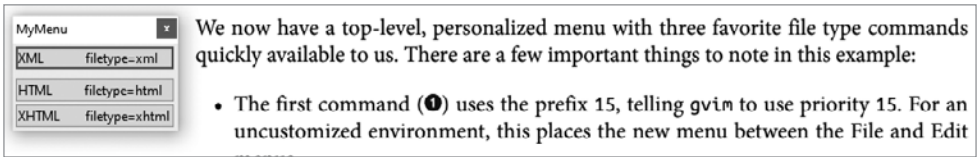


Рис. 9.13. Персонализированное плавающее «оторванное» меню

Панели инструментов

Панели инструментов представляют собой длинные ленты значков, которые обеспечивают быстрый доступ к функциям программы. Например, в GNU/Linux `gvim` отображает панель инструментов в верхней части окна (рис. 9.14).



Рис. 9.14. Панель инструментов `gvim` (версия Linux)

В табл. 9.2 показаны пиктограммы панели инструментов и их значение.

Таблица 9.2. Пиктограммы панели инструментов `gvim` и их значение

Значок	Описание	Значок	Описание
	Открыть диалоговое окно		Найти следующее вхождение шаблона поиска
	Сохранить текущий файл		Найти предыдущее вхождение шаблона поиска
	Сохранить все файлы		Загрузить ранее сохраненный сеанс редактирования

Значок	Описание	Значок	Описание
	Печать буфера		Сохранить текущий сеанс редактирования
	Отменить последнее изменение		Выбрать сценарий Vim для запуска
	Повторить последнее действие		Создать текущий проект с помощью команды make
	Вырезать выделенную часть текста в буфер обмена		Создать теги для текущего дерева каталогов
	Копировать выделенную часть текста в буфер обмена		Перейти к тегу под курсором
	Вставить буфер обмена в буфер		Открыть справку
	Найти и заменить		Искать в справке

Если эти значки вам незнакомы или непонятны, вы можете сделать так, чтобы панель инструментов отображала и текст, и значок:

```
:set toolbar="text,icons"
```



Как и в случае большинства расширенных функциональных возможностей, в Vim панель инструментов должна быть включена на этапе компиляции, чтобы пользователи, которым она не нужна, могли выключить ее, чтобы сэкономить память. Панель инструментов не появится, если в вашу версию gvim не скомпилирована одна из функций +GUI_GTK, +GUI_Athena, +GUI_Motif или +GUI_Photon. В приложении Г объяснено, как перекомпилировать Vim и создать ссылку на исполняемый файл gvim.

Процесс изменения панели инструментов почти такой же, как и меню. Фактически мы используем ту же команду `:menu`, но с дополнительным синтаксисом для указания графики. Хотя существует алгоритм, чтобы помочь gvim найти значок, связанный с каждой командой, мы рекомендуем явно указывать его.

gvim воспринимает панель инструментов как одномерное меню. И точно так же, как вы управляете расположением новых разделов меню справа налево, вы можете управлять расположением новых элементов панели инструментов, добавив

префикс в виде номера в команду `menu`, который определяет его позиционный *приоритет*. В отличие от меню создать новую панель инструментов нельзя. Все новые определения панели инструментов отображаются на единой панели. Ниже представлен синтаксис добавления нового элемента на панель инструментов:

```
:amenu icon=/some/icon/image.bmp ToolBar.NewToolBarSelection Action
```

где `/some/icon/image.bmp` — это путь к файлу, содержащему изображение (обычно значок) будущей кнопки на панели инструментов, `NewToolBarSelection` — это новый элемент для кнопки, а `Action` определяет действие, выполняемое при нажатии кнопки.

Например, давайте определим новый элемент панели инструментов, который при нажатии вызывает окно DOS в Windows. Предполагая, что путь Windows указан верно, мы определим наш элемент панели инструментов для запуска окна DOS из `gvim`, выполнив следующее (то есть *Action*):

```
:!cmd
```

Для кнопки или изображения нового элемента панели инструментов мы используем значок, показывающий приглашение на ввод команд DOS, как на рис. 9.15, который в нашей системе хранится в `$HOME/dos.bmp`.

Выполните команду:

```
:amenu icon="c:$HOME/dos.bmp" ToolBar.DOSWindow :!cmd<CR>
```



Рис. 9.15. Иконка DOS

Это создаст новый элемент панели инструментов и добавит наш значок в ее конец. Панель инструментов теперь должна выглядеть как на рис. 9.16. Новый значок отобразится справа с краю.



Рис. 9.16. Панель инструментов с добавленной кнопкой команды DOS

Всплывающие подсказки

`gvim` позволяет определять всплывающие подсказки как для пунктов меню, так и для значков панели инструментов. Подсказки меню отображаются в области командной строки `gvim` при наведении указателя мыши на этот пункт меню. Подсказки панели инструментов всплывают, когда указатель мыши находится на значке панели инструментов. Например, на рис. 9.17 показана подсказка, которая всплывает при наведении указателя мыши на кнопку `Find Previous` панели инструментов.

Команда `:tmenu` определяет подсказки как для меню, так и для значков панели инструментов. Синтаксис следующий:

```
:tmenu TopMenu.NextLevelMenu.MenuItem tool tip text
```

где `TopMenu.NextLevelMenu.MenuItem` определяет меню каскадом от верхнего уровня до пункта меню, для которого вы хотите определить всплывающую подсказку. Так, к примеру, для пункта `Open` из меню `File` она будет определена с помощью следующей команды:

```
:tmenu File.Open Open a file
```

Используйте `ToolBar` для «меню» высшего уровня, если вы определяете элемент панели инструментов (на самом деле не существует меню верхнего уровня для панели инструментов).

Определим всплывающую подсказку для значка панели инструментов DOS, которую мы создали в предыдущем разделе. Введите команду:

```
:tmenu ToolBar.DOSWindow Open up a DOS window
```

Теперь, когда вы наведете указатель мыши на добавленную кнопку панели инструментов, вы увидите всплывающую подсказку, как показано на рис. 9.18.



Рис. 9.17. Подсказка для кнопки Find Previous

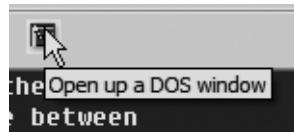


Рис. 9.18. Панель инструментов с добавленной командой DOS и ее новая всплывающая подсказка

gvim в Microsoft Windows

gvim невероятно популярен среди пользователей MS Windows. Давним пользователям `vi` и Vim версия Windows покажется великолепной, и, вероятно, это самая современная версия среди всех операционных систем.



Исполняемый файл должен автоматически и аккуратно интегрировать Vim в окружение Windows. Если этого не произошло, обратитесь к файлу справки `gui-w32.txt` в каталоге с установщиком Vim для получения инструкций для `regedit`. Поскольку это действие включает редактирование системного реестра Windows, не совершайте его, если оно вызывает у вас хотя бы малейший дискомфорт. Найдите кого-то более опытного, кто мог бы помочь вам в решении этой проблемы. Это обычная, но не тривиальная задача.

Пользователи Windows знакомы с *буфером обмена* — областью памяти, в которой текст и другая информация хранятся для упрощения операций копирования, вырезания и вставки. Vim поддерживает взаимодействие с буфером обмена Windows. Просто выделите текст в визуальном режиме и нажмите на пункты меню *Copy* или *Cut*, чтобы сохранить текст Vim в буфере обмена Windows. Затем вы сможете вставить данный текст в другие приложения Windows.

gvim в X Window System

Пользователи, знакомые с окружением X, могут определять и использовать многие из настраиваемых функций X. Например, можно задать множество ресурсов с помощью стандартных определений класса, обычно содержащихся в файле `.Xdefaults`.



Обратите внимание, что эти стандартные ресурсы X применимы только для GUI-версий Motif или Athena. Очевидно, что у версии Windows нет понимания ресурсов X. Не так очевидно, что ресурсы X не используются ни в KDE, ни в Gnome, а в современной системе gvim, скорее всего, будет основываться на одном из этих двух инструментариев.

Запуск gvim в Microsoft Windows WSL

На момент написания книги компания Microsoft выпустила две основные версии поддержки виртуализации дистрибутивов GNU/Linux — *Windows Subsystem for Linux* (WSL). Обычно их называют WSL и WSL2.

WSL предоставляет совместимые интерфейсы, которые позволяют запускать приложения GNU и, таким образом, обеспечивают работу дистрибутивов GNU/Linux в Windows. WSL2 же предоставляет собственное ядро Linux, исполняемое в виртуальной среде. Изучение данных архитектур выходит за рамки нашей книги, но стоит упомянуть, что один из авторов часто успешно и эффективно использовал WSL и подтверждает полную функциональность Vim в приложении *Microsoft Terminal* (это консольное приложение). Хотя это оказалось приятным сюрпризом, еще более захватывающим было узнать, что Microsoft предоставляет поддержку GUI для WSL Linux.

Информация из текущего раздела поможет вам запустить gvim из WSL2, отображаемого в среде Windows. Для тех, кто знаком с X11, этот подход будет довольно

стандартным, но для Windows необходимы некоторые настройки конфигурации, а для целевого дистрибутива GNU/Linux должен быть установлен соответствующий пакет `gvim`.

Установка `gvim` в WSL2

Здесь описывается установка и настройка `gvim` в дистрибутиве WSL2 Ubuntu. Сначала, чтобы проверить, существует ли `gvim`, воспользуйтесь `dpkg` для поиска двоичного файла `gvim`. Для исключения ложноположительных результатов (страницы руководства, конфигурационные файлы и т. д.) ищите `bin/gvim`:

```
$ dpkg -S bin/gvim
vim-gui-common: /usr/bin/gvimtutor
```

Нет, `gvim`! `gvimtutor` — это не `gvim`. Просто происходит возврат к Vim, основанному на терминале, когда `gvim` не установлен.

Теперь установите пакет `gvim` как корневой каталог (через `sudo`). Хотя мы знаем, что пакетом является `vim-gtk3`, иногда полезно позволить менеджеру пакетов Ubuntu `apt` дать подсказки. Просто попросите `apt` установить `gvim`, и вы увидите три варианта, из которых необходимо выбрать `vim-gtk3`:

```
vim@office-win10:~$ sudo apt install gvim
[sudo] password for vim:
Reading package lists... Done
Building dependency tree
Reading state information... Done
Package gvim is a virtual package provided by:
  vim-gtk 2:8.0.1453-1ubuntu1.4
  vim-athena 2:8.0.1453-1ubuntu1.4
  vim-gtk3 2:8.0.1453-1ubuntu1.4
You should explicitly select one to install.
```

Теперь, зная необходимый нам пакет, устанавливаем его с помощью `apt`:

```
vim@office-win10:~$ sudo apt install vim-gtk3
...
```

При прочих равных условиях `gvim` должен быть теперь доступен в вашей подсистеме Linux. Соответственно после установки обновится ваш `PATH`, а `gvim` добавится в доступные вам команды. Проверьте это с помощью команды `type`:

```
$ type gvim
gvim is /usr/bin/gvim
```

Итак, мы почти закончили. Вы можете запустить `gvim`, но это все еще не совсем то, что нам нужно. Сейчас вы увидите что-то вроде:

```
$ gvim
E233: cannot open display
Press Enter or type command to continue
```

а когда вы нажмете **Enter**, чтобы продолжить, `gvim` вернется к текстовому терминалу Vim. Нам нужно завершить настройку X Window, установив сервер X Windows в окружении Windows и попросив Linux `gvim` отобразить его графически¹.

Установка сервера X для Windows

Как только что было упомянуто, мы должны установить сервер из-под Microsoft Windows для получения графических запросов, что позволит нашему экземпляру WSL Linux `gvim` отображаться на Рабочем столе Windows. В нашем примере мы будем использовать свободно распространяемый сервер Windows Xming.

Загрузите последнюю версию программы (<https://sourceforge.net/projects/xming>) и запустите ее. Окно установщика показано на рис. 9.19.



Рис. 9.19. Программа установки Xming

¹ Это выглядит немного запутанно. Но смиритесь с этим. Windows и X Windows — не одно и то же, но оба важны. Windows — это Windows, знакомый вам Рабочий стол Microsoft. X Windows — это графический сервер, запущенный в вашей среде Microsoft Windows и знающий, как отображать графику из удаленных систем.

Настройка сервера X для Windows

Xming устанавливает два исполняемых файла:

- XLaunch — мастер настройки, упрощающий обычные экземпляры Xming;
- Xming — фактический сервер X.

Откройте меню Windows Applications и найдите папку Xming. Вы увидите нечто похожее на рис. 9.20.

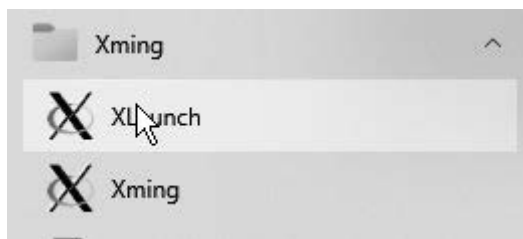


Рис. 9.20. Приложения Xming, установленные из Xming Setup

Выполните XLaunch для конфигурации Xming. XLaunch проведет вас по стандартным настройкам X. На рис. 9.21 изображено первое окно XLaunch. Выберите те же пункты, что и на рисунке.

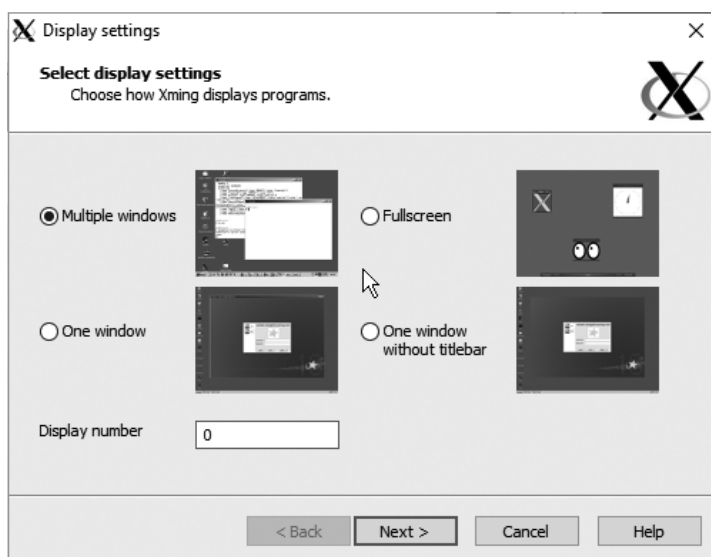


Рис. 9.21. Диалоговое окно XLaunch Display settings с выбранным пунктом Multiple windows

Выберите **Multiple windows** и в графе **Display number** введите число 0¹.

Следующее диалоговое окно определяет Session type X, как представлено на рис. 9.22.

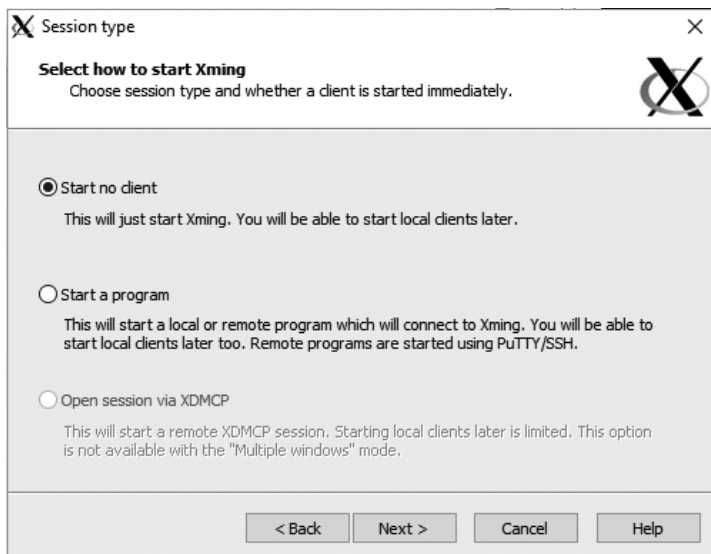


Рис. 9.22. Диалоговое окно XLaunch Session type с выбранным пунктом **Start no client**

Выберите пункт **Start no client**. XLaunch запустит сервер Xming X Windows самостоятельно. В этой роли Xming будет ждать и отображать приложения, запрашивающие отображение из удаленного хоста, которым в данном случае является наш хост WSL Linux.

Далее XLaunch записывает прочие разнообразные параметры X Windows (рис. 9.23).

Выберите **Clipboard** и **No Access Control**. Обратите внимание, что по умолчанию пункт **No Access Control** не активирован.



Мы выбираем параметр **No Access Control** для удобства, чтобы не связываться с механизмами безопасности X Windows. Мы делаем это из расчета, что компьютер находится в «безопасном» окружении, например в домашней сети, в которой не могут произойти атаки на сервер Xming X. Для любой общедоступной сети или офиса это неудачный выбор.

¹ Расшифровка различных настроек вне компетенции книги. Если коротко, мы выбираем **Multiple windows**, потому что данный параметр позволяет графически отображать **gvim** в окне **Windows**.

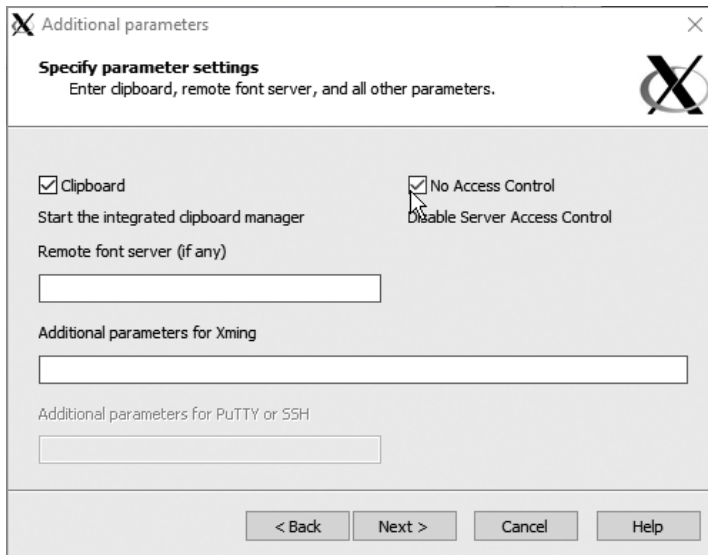


Рис. 9.23. Диалоговое окно XLaunch Additional parameters; мы выбираем Clipboard и No Access Control

Теперь мы настроили Xming и готовы к работе. В последнем диалоговом окне установщика XLaunch на рис. 9.24 отображены параметры для сохранения вашей новой конфигурации (мы не будем этого делать) и запуска Xming.

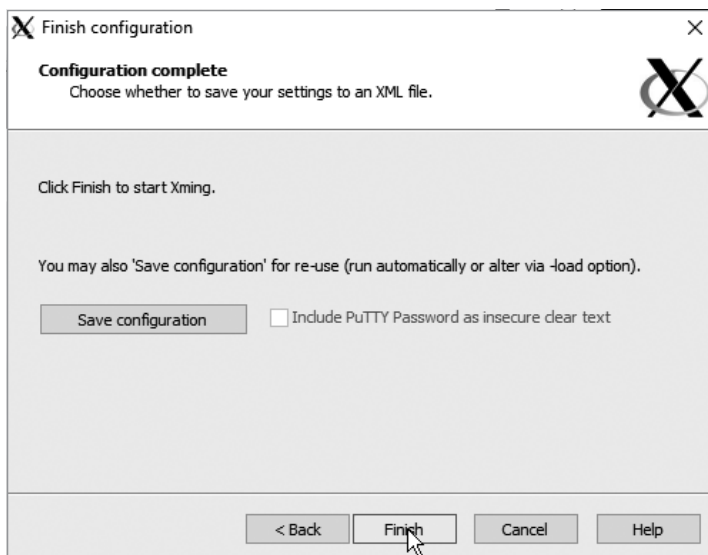


Рис. 9.24. Финальное диалоговое окно установщика XLaunch

Если при запуске Xming вы видите его значок в области уведомлений (рис. 9.25), это значит, что Xming работает корректно и готов отображать приложения Ubuntu X Windows.



Рис. 9.25. Значок Xming в области уведомлений Windows

Итак, у нас почти все готово для начала работы нашего экземпляра `gvim` в Ubuntu, осталась всего *одна* маленькая деталь. Если вы запустите `gvim`, он все еще будет выдавать сообщение об ошибке и возвращаться к терминалу Vim.

Это происходит потому, что нам необходимо сообщить нашей системе Ubuntu, *где* отображать `gvim`. В данном случае мы должны указать Ubuntu на наш Рабочий стол Microsoft Windows. Хотя для нас это *очевидно*, приложения X Windows по определению должны запрашивать сервер X, чтобы отобразить их содержимое, а мы этого не сделали.

Мы определяем в Ubuntu, *куда* мы хотим вывести `gvim` с нашим сетевым адресом Microsoft Windows *и* связанным с ним отображением сервера X¹. Формат записи: `имя_хоста:DISPLAY`, где `имя_хоста` — наш IP-адрес Windows, а `DISPLAY` — `0`.

Самый простой способ найти IP-адрес Microsoft Windows — это посмотреть в конфигурационном файле Ubuntu `/etc/resolv.conf`, в котором строка `nameserver` является адресом Microsoft Windows:

```
$ cat /etc/resolv.conf
# This file was automatically generated by WSL. To stop automatic generation of
# this file, add the following entry to /etc/wsl.conf:
# [network]
# generateResolvConf = false
nameserver 172.17.224.1
```

или:

```
$ grep nameserver /etc/resolv.conf
nameserver 172.17.224.1
```

¹ Да, у вашего компьютера на MS Windows теперь есть сетевой интерфейс, видимый для подсистемы Linux. WSL работает именно так, поскольку это виртуальный компьютер со своим собственным сетевым адресом, что требует семантики настоящей сети, чтобы Linux и MS Windows могли общаться друг с другом.

В нашем примере мы видим, что имя сервера Ubuntu — 172.17.224.1. Поэтому определение для нашего сервера X будет 172.17.224.1:0.0¹.

Существует несколько способов передать gvim информацию о том, на каком дисплее он должен быть представлен. Наиболее распространенным способом является переменная окружения DISPLAY, которую мы устанавливаем с помощью команды оболочки export:

```
$ export DISPLAY=172.17.224.1:0.0
```

Теперь мы готовы. Запустите gvim. Вы должны увидеть GUI-версию, отображаемую в виде окна на вашем Рабочем столе. Приглядитесь к рис. 9.26: он сделан с помощью gvim во время редактирования *этой* главы. Под терминалом и командной строкой заметно, где мы редактировали реальный файл главы. Круто. 😊



Рис. 9.26. Ubuntu gvim, отображенное на Рабочем столе Microsoft Windows

Обратите внимание на пронумерованные выноски.

- ❶ Это фактическое окно gvim, прозрачно работающее как приложение на Рабочем столе Microsoft Windows.
- ❷ Это базовый терминал Microsoft, в котором работает экземпляр WSL Ubuntu. Вы можете видеть команды, о которых мы только что упоминали в тексте.

¹ Обратите внимание на форму 0.0 для номера дисплея. Это связано с дисплеем и возможными несколькими экранами. Для большинства случаев (простых) дисплей должен быть равен 0.0.

❸ Это `xeeyes` — незначимое, но очень популярное приложение X Windows, отслеживающее нашу работу¹.

Мы показали вам, как настроить и использовать `gvim` для экземпляра Microsoft Windows WSL Linux и прозрачно отображать сеанс. Существует множество способов установки и конфигурации серверов и клиентов X Windows. То, что вы узнали здесь, является большим шагом к использованию `gvim` и пониманию основ X Windows. Полное описание X выходит за рамки книги. Для краткого (или не такого уж краткого) знакомства с X мы предлагаем документацию X.

Параметры GUI и краткое описание команд

В табл. 9.3 представлены команды и параметры, связанные с `gvim`. Они добавляются в Vim при компиляции с поддержкой GUI и активизируются при вызове `gvim` или `vim -g`.

Таблица 9.3. Параметры, предназначенные для `gvim`

Команда, параметр или флаг	Тип	Описание
<code>guicursor</code>	Параметр	Настройка формы и мерцания курсора
<code>guifont</code>	Параметр	Имена используемых однобайтовых шрифтов
<code>guifontset</code>	Параметр	Имена используемых многобайтовых шрифтов
<code>guifontwide</code>	Параметр	Список имен шрифтов для символов двойной ширины
<code>guiheadroom</code>	Параметр	Количество пикселей, оставляемых для оформления окна
<code>guioptions</code>	Параметр	Перечень используемых параметров и компонентов
<code>guipty</code>	Параметр	Использование псевдотерминала для команд : !
<code>guitablel</code>	Параметр	Пользовательская метка для вкладки страницы
<code>guitabletooltip</code>	Параметр	Пользовательская всплывающая подсказка для вкладки страницы
<code>toolbar</code>	Параметр	Элементы для отображения на панели инструментов
<code>-g</code>	Флаг	Запустить GUI (что также разрешает другие параметры)

¹ Вы можете этого не осознавать, но таким образом можно использовать любое приложение X Windows, доступное в экземпляре Ubuntu, а `xeeyes` — лишь одно из многих. Как только вы настроили сервер X, определили сеть и `DISPLAY`, все приложения X Windows будут отображаться на одном экране. Поздравляем! Вы только что завершили полезный урок по X Windows. Для дальнейшей проверки попробуйте запустить приложение `xterm`.

Команда, параметр или флаг	Тип	Описание
<code>-U gvimrc</code>	Флаг	Использовать загрузочный файл <code>gvim</code> с именем <code>gvimrc</code> или нечто подобное при запуске GUI
<code>:gui</code>	Команда	Запустить GUI (только для Unix-подобных систем)
<code>:gui имя_файла</code>	Команда	Запустить GUI и редактировать указанные файлы
<code>:menu</code>	Команда	Список всех меню
<code>:menu путь_меню</code>	Команда	Список меню, начинающихся с <code>путь_меню</code>
<code>:menu путь_меню действие</code>	Команда	Добавить меню <code>путь_меню</code> , чтобы выполнить <code>действие</code>
<code>:menu n путь_меню действие</code>	Команда	Добавить меню <code>путь_меню</code> с позиционным приоритетом <code>n</code>
<code>:menu Toolbar.имя_панели_инструментов действие</code>	Команда	Добавить элемент панели инструментов <code>имя_панели_инструментов</code> , чтобы выполнить <code>действие</code>
<code>:tmenu путь_меню текст</code>	Команда	Создать подсказку для пункта меню <code>путь_меню</code> с текстом из <code>текст</code>
<code>:unmenu путь_меню</code>	Команда	Удалить меню <code>путь_меню</code>

Многооконный режим в Vim

По умолчанию редактирование файлов в Vim происходит в одном окне, в котором отображается лишь один буфер за раз при перемещении между несколькими файлами или частями одного файла. Но Vim также предлагает многооконный режим, который призван упростить сложные задачи редактирования. Этот режим отличается от запуска нескольких экземпляров Vim в графическом терминале. В данной главе мы рассмотрим использование нескольких окон за один сеанс Vim.

Вы можете начать редактировать в нескольких окнах как при первичном запуске Vim, так и создать их после начала сеанса. Добавлять окна в ваш сеанс редактирования можно до бесконечности, а чтобы получить обратно одно окно, просто удалите лишние.

На сегодняшний день использование нескольких окон имеет больше смысла, чем когда-либо, поскольку мониторы высокого разрешения стали нормой. К моменту выхода седьмого издания хорошим разрешением считалось WXGA (1280 × 800). Сегодня (конец 2021 года) примерно за ту же цену можно легко найти мониторы с разрешением 4K (Ultra HD: 3840 × 2160). Оно примерно в *девять* раз превосходит предыдущее!

Далее многооконный режим Vim расширил возможности редактирования для пользователей, предложив несколько окон просмотра и беглое знакомство с одним или несколькими файлами одновременно. Это был огромный шаг в направлении к эффективному редактированию, но часто приходилось жертвовать размерами, которые достигались либо с помощью установки параметров переноса строки, чтобы вся строка оставалась видимой, либо с помощью установки параметров сдвига строки для прокрутки влево и вправо, когда строки обрезаются по краям окон.

Сейчас при наличии дисплеев с высоким разрешением многооконный режим Vim предоставляет те же самые мощные функции, а пользователи могут с легкостью разделять окна и все еще получать полномасштабное отображение текста для каждого окна¹.

¹ Конечно же, как описано в этой главе, у вас все еще есть возможность нарезать и разделять окна на самые маленькие размеры, в зависимости от ваших потребностей редактирования.

Ниже приведено несколько примеров того, как многооконность может облегчить вашу жизнь.

- Редактирование нескольких файлов, которые нужно отформатировать одинаково и которые можно сравнивать визуально по ходу работы.
- Вырезать и вставлять текст быстро и многократно между несколькими файлами или частями одного файла.
- Отображение одной части файла в качестве образца, чтобы упростить работу в другом месте того же файла.
- Сравнение двух версий файла.

Vim предлагает много удобных функций управления окнами, включая возможность:

- разделять окна горизонтально и вертикально;
- быстро перемещаться от одного окна к другому и обратно;
- копировать и перемещать текст из одних окон в другие;
- перемещать и переставлять окна;
- работать с буферами, включая скрытые буферы (будут описаны позднее);
- использовать для нескольких окон такие внешние инструменты, как команда `diff`.

В этой главе мы познакомим вас с работой в многооконном режиме и покажем, как запустить такой сеанс. Обсудим функции и рекомендации для сеанса редактирования и опишем, как завершить работу и убедиться, что вся она надежно сохранена (или при желании отменена!). Рассматриваются следующие темы:

- инициализация или запуск сеанса многооконного редактирования;
- команды `:ex` для нескольких окон;
- перемещение курсора из одного окна в другое;
- перемещение окон на дисплее;
- изменение размера окон;
- буферы и их взаимодействие с окнами;
- управление тегами с помощью окон;
- редактирование с вкладками (подобно вкладкам в современных браузерах и диалоговых окнах);
- закрытие и выход из окон.

Инициация многооконного редактирования

Вы можете инициировать сеанс многооконного редактирования при запуске Vim или разделить существующие окна в процессе редактирования. Многооконное редактирование динамично и позволяет вам открывать, закрывать и перемещаться между несколькими окнами в любой момент в большинстве случаев.

Инициация многооконности из командной строки

По умолчанию Vim открывает только одно окно для сеанса, даже если вы указали более одного файла. Мы точно не знаем, почему Vim сразу не открывает несколько окон для различных файлов, но это может происходить просто потому, что использование одного окна — типичное поведение *vi*. Несколько файлов занимают не один буфер, и у каждого файла он свой. (Буферы мы скоро обсудим.)

Чтобы открыть несколько окон из командной строки, используйте параметр Vim `-o`. Например:

```
$ gvim -o file1.txt file2.txt
```

Команда откроет сеанс редактирования с отображением окна, разделенного пополам горизонтально, по одному для каждого файла (рис. 10.1). Для каждого указанного в командной строке файла Vim попытается открыть свое окно. Если Vim не может разделить экран на достаточное количество окон, то первые файлы, перечисленные в командной строке, откроются, а остальные будут загружены в буферы, невидимые (но все еще доступные) для вас.

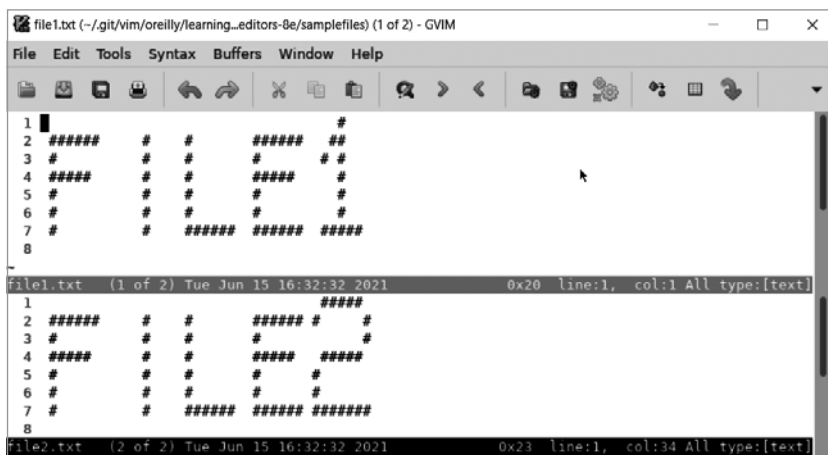


Рис. 10.1. Результат выполнения команды `gvim -o file1.txt file2.txt` (Linux gvim)

Ниже представлен другой синтаксис команды, который предварительно распределяет окна. Достаточно добавить число *n* в `-o`:

```
$ gvim -o5 file1.txt file2.txt
```

Это откроет сеанс, при котором дисплей будет горизонтально разделен на пять окон одинакового размера, самое верхнее из которых содержит `file1.txt`, а второе — `file2.txt` (рис. 10.2).

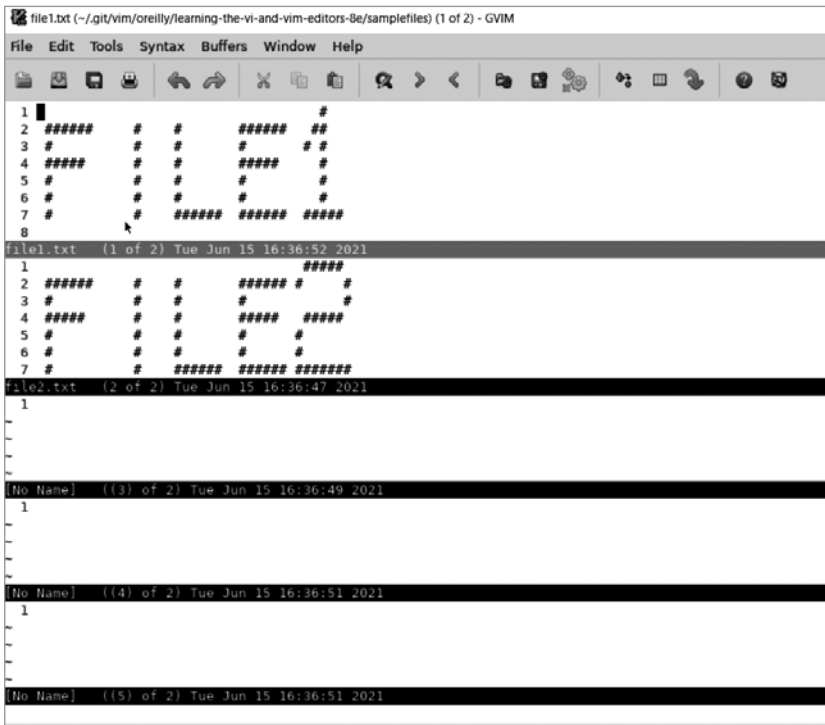


Рис. 10.2. Результат `gvim-o5 file1.txt file2.txt` (Linux gvim)



Когда Vim создает более одного окна, по умолчанию он добавляет строки состояния для каждого из них (в то время как для одного окна не отображает никаких строк). Вы можете управлять этим поведением с помощью параметра Vim `laststatus`, например:

```
:set laststatus=1
```

Установите значение `laststatus` равным 2, чтобы все время видеть строку состояния для каждого окна даже в однооконном режиме. Лучше всего сделать это в вашем файле `.vimrc`.

Поскольку размер окна влияет на читабельность и удобство работы, вам может понадобиться настроить размеры окон Vim. Используйте параметры Vim `winheight` и `winwidth` для определения разумных пределов для текущего окна (размеры других окон могут быть изменены, чтобы подстроиться под текущее).

Многооконное редактирование внутри Vim

Вы можете инициировать и изменить конфигурацию окна из Vim. Создайте новое окно с помощью команды `:split`. Это разделит текущее окно пополам, отображая один и тот же буфер в обеих половинах. Теперь вы сможете независимо перемещаться в каждом окне по одному и тому же файлу.



Существуют удобные сочетания клавиш для многих команд из этой главы. Например, в данном случае `Ctrl+W S` разделяет окно. Все относящиеся к окнам команды Vim начинаются с `Ctrl+W`, где `W` означает `window`. С целью обсуждения мы показываем только методы командной строки, потому что они предоставляют дополнительные возможности необязательных параметров, настраивающих поведение по умолчанию. Если вы регулярно используете команды, то можете с легкостью найти соответствующую последовательность клавиш в документации Vim, как описано в разделе «Встроенная справка» главы 8.

Аналогичным образом можно создать новое вертикально разделенное окно редактирования с помощью команды `:vsplit` (рис. 10.3).

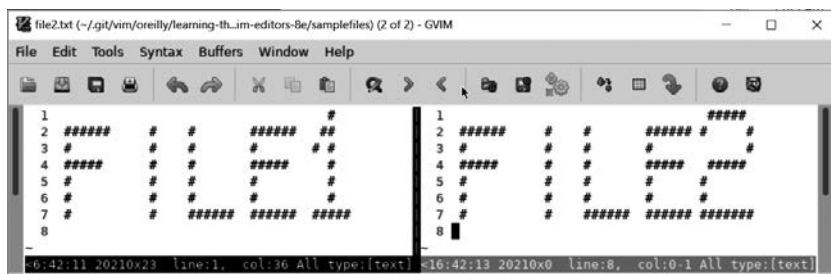


Рис. 10.3. Вертикально разделенное окно (Linux gvim)

В каждом из этих методов Vim разделяет окно (горизонтально или вертикально), и, поскольку в командной строке `:split` не был указан никакой файл, в конечном итоге вы будете редактировать один и тот же файл в двух окнах.



Если вы думаете, что нельзя одновременно редактировать один и тот же файл, разделите окно редактирования и прокрутите каждое окно, чтобы они отображали одну и ту же область файла. Внесите правку. Посмотрите на другое окно. Какая-то магия!

Как и зачем это использовать? При написании сценариев оболочки или программ на языке С часто используется блок текста, который описывает использование программы. Как правило, программа отображает этот фрагмент при вызове параметра `--help`. Дисплей разделяется таким образом, что одно окно отображает текст в качестве образца для редактирования кода в другом окне, которое анализирует все параметры и аргументы командной строки, описанные в тексте. Часто (практически всегда) этот код сложен и достаточно далек от используемого текста, и мы не можем отобразить все в одном окне.

Если вы хотите редактировать и просматривать другой файл, не сбрасывая свою позицию в текущем файле, задайте новый файл в качестве аргумента для вашей команды `:split`. Например:

```
:split otherfile
```

В следующем разделе более подробно описано разделение и объединение окон.

Открытие окон

Здесь мы детальнее расскажем, как добиться конкретного желаемого поведения при разделении окна.

Новые окна

Как обсуждалось ранее, самый простой способ открыть новое окно — выполнить команду `:split` (для горизонтального разделения) или `:vsplit` (для вертикального разделения). Далее мы более подробно рассмотрим множество команд и вариаций. Мы также приведем краткий обзор команд для получения быстрой справки.

Параметры во время разделения

Полная команда `:split` для открытия нового горизонтального окна выглядит следующим образом:

```
:[n]split [++opt] [+cmd] [файл]
```

где:

- *n* — сообщает Vim, сколько строк отображать в новом окне, которое располагается наверху;
- *opt* — передает информацию о параметрах Vim в сеанс работы с новым окном (обратите внимание, что ему должны предшествовать два знака плюс);

- *cmd* — передает команду для выполнения в новом окне (обратите внимание, что ему должен предшествовать один знак плюс);
- *файл* — указывает файл для редактирования в новом окне.

Например, предположим, что вы редактируете файл и хотите разделить окно для редактирования другого файла с именем *другой_файл*. При этом необходимо, чтобы в сеансе использовался *формат_файла* `unix` (что обеспечивает использование перевода строки в конце каждой строки вместо комбинации возврата каретки и перевода строки), а высота окна была 15 строк. Введите:

```
:15split ++формат_файла=unix другой_файл
```

Чтобы просто разделить экран, отображая один и тот же файл в обоих окнах и используя все текущие настройки по умолчанию, применяйте команды режима `vi` `Ctrl+W S`, `Ctrl+W Shift+S` или `Ctrl+W Ctrl+S`.



Если вы хотите, чтобы окна всегда разделялись поровну, установите параметр `equalalways`. А чтобы он использовался и в других сеансах, поместите его в ваш файл `.vimrc`. По умолчанию установка параметра `equalalways` разделяет и горизонтальные, и вертикальные окна на равные части. Добавьте параметр `eadirection` (`hor`, `ver`, `both` для горизонтальных, вертикальных или обоих соответственно), чтобы контролировать, какое направление разделяется поровну.

Следующий синтаксис команды `:split` открывает новое горизонтальное окно, как и раньше, но с некоторым отличием:

```
:[n]new [++opt] [+cmd] [файл]
```

Помимо создания нового окна, будут выполнены автокоманды `WinLeave`, `WinEnter`, `BufLeave` и `BufEnter` (для получения дополнительной информации об автокомандах смотрите подраздел «Автокоманды» в главе 12).

Наряду с командами горизонтального разделения Vim предлагает аналогичные вертикальные. Так, к примеру, чтобы разделить окно по вертикали, вместо команды `:split` или `:new` используйте `:vsplit` или `:vnew` соответственно. Доступны те же дополнительные параметры, что и для команд горизонтального разделения.

Существует две команды горизонтального разделения, не имеющие вертикальных аналогов:

- `:sview имя_файла` — разделяет экран горизонтально, чтобы открыть новое окно, и устанавливает параметр `readonly` для этого буфера. Команде `:sview` требуется имя файла в качестве аргумента;
- `:sfind [++ opt] [+ cmd] имя_файла` — работает как `:split`, но ищет *имя_файла* в `path`. Если Vim не находит файл, окно не разделяется.

Команды условного разделения

Vim позволяет вам указать команду, которая открывает окно, если найден новый файл. Команда `:topleft cmd` обязывает Vim выполнить `cmd` и отобразить новое окно с курсором в верхнем левом углу, если `cmd` открывает новый файл. Команда может привести к трем разным результатам:

- `cmd` разделит окно горизонтально, а новое окно займет верхнюю часть окна Vim;
- `cmd` разделит окно вертикально, а новое окно займет левую часть окна Vim;
- `cmd` не приведет к разделению, а вместо этого поместит курсор в верхний левый угол текущего окна.

В дополнение к команде условного разделения `:topleft` Vim предлагает аналогичные команды `:vertical`, `:leftabove`, `:aboveleft`, `:rightbelow`, `:belowright`, `:botright`. Их подробное описание вы найдете с помощью команды Vim `:help`.

Краткое описание команд для окон

В табл. 10.1 приведены команды для разделения окон.

Таблица 10.1. Краткое описание команд разделения окон

Команда ex	Команда vi	Описание
<code>:[n]split [++opt] [+cmd]</code> [файл]	Ctrl+W S Ctrl+W Shift+S Ctrl+W Ctrl+S	Разделить текущее окно на две части, помещая курсор в новое окно. Дополнительный аргумент <i>файл</i> открывает этот файл в новом окне. Окна создаются одинаковыми по размеру, насколько это возможно. Размер зависит от свободного пространства на экране
<code>:[n]new [++opt] [+cmd]</code>	Ctrl+W N Ctrl+W Ctrl+N	То же, что и <code>:split</code> , но в новом окне открывается пустой файл. Обратите внимание, что у буфера не будет имени, пока вы его не присвоите
<code>:[n]sview [++opt] [+cmd]</code> [файл]		Версия <code>:split</code> в режиме только для чтения
<code>:[n]sfind [++opt] [+cmd]</code> [файл]		Разделить окно и открыть <i>файл</i> (если указан) в новом окне. Искать <i>файл</i> в <i>path</i>
<code>:[n]vsplit [++opt]</code> [+cmd] [файл]	Ctrl+W V Ctrl+W Ctrl+V	Разделить текущее окно на два сверху вниз и открыть <i>файл</i> (если указан) в новом окне
<code>:[n]vnew [++opt] [+cmd]</code>		Вертикальная версия <code>:new</code>

Перемещение по окнам (перемещение вашего курсора от окна к окну)

С помощью мыши перемещаться по окнам легко как в `gvim`, так и в `Vim`. `gvim` поддерживает щелчок кнопкой мыши по умолчанию, в то время как в `Vim` его необходимо настроить с помощью параметра `mouse`. Хорошей настройкой по умолчанию для `Vim` является `:set mouse=a`, которая активизирует мышь для всех целей: командной строки, ввода и перемещения.

Если у вас нет мыши или вы предпочитаете клавиатуру, `Vim` предоставляет полный набор команд для быстрого и точного перемещения между окнами. К счастью, `Vim` использует сочетание клавиш `Ctrl+W` для навигации по окну. Следующая за этой комбинацией клавиша определяет перемещение или другое действие, а команды навигации по окну должны быть знакомы опытным пользователям `vi` и `Vim`, потому что они тесно связаны с теми же командами перемещения при редактировании.

Вместо того чтобы описывать каждую команду и ее поведение, мы рассмотрим пример, после которого таблица с кратким обзором команд не потребует дополнительных объяснений.

Чтобы переместиться из текущего окна `Vim` к следующему, наберите `Ctrl+W J` (или `Ctrl+W ↓` или `Ctrl+W Ctrl+J`). `Ctrl+W` — сокращение для команды `window`, а `j` аналогична команде `Vim j`, которая перемещает курсор на следующую строку.

В табл. 10.2 кратко описаны команды навигации по окну.



Как и многие команды `Vim` и `vi`, эти команды можно выполнить многократно, добавив перед ними префикс числа. Например, `3 Ctrl+W J` говорит `Vim` перейти к третьему окну вниз от текущего окна.

Таблица 10.2. Команды навигации по окну

Команда	Описание
<code>Ctrl+W W</code>	Перейти к следующему окну внизу или справа. Обратите внимание, что эта команда, в отличие от <code>Ctrl+W J</code> , циклически перебирает все окна <code>Vim</code> . Когда достигнуто самое нижнее окно, <code>Vim</code> начинает заново цикл и переходит к самому левому верхнему окну
<code>Ctrl+W ↓</code>	Перейти к следующему окну внизу.
<code>Ctrl+W Ctrl+J</code>	Обратите внимание, что эта команда не перебирает все окна циклически.
<code>Ctrl+W J</code>	Она просто перемещается вниз к следующему окну под текущим. Если

Команда	Описание
	курсор находится в самом нижнем окне экрана, команда не сработает. Также она обходит смежные окна по «пути вниз»: например, если справа от текущего окна расположено другое, команда не перейдет в него. (Используйте <code>Ctrl+W Ctrl+W</code> для циклического перемещения по окнам)
<code>Ctrl+W ↑</code>	Перейти к следующему окну наверху. Это противоположное действие команды <code>Ctrl+W J</code>
<code>Ctrl+W Ctrl+K</code>	
<code>Ctrl+W K</code>	
<code>Ctrl+W ←</code>	Перейти к окну слева от текущего
<code>Ctrl+W H</code>	
<code>Ctrl+W Backspace</code>	
<code>Ctrl+W →</code>	Перейти к окну справа от текущего
<code>Ctrl+W Ctrl+L</code>	
<code>Ctrl+W L</code>	
<code>Ctrl+W Shift+W</code>	Перейти к следующему окну наверху или слева. Это восходящий аналог команды <code>Ctrl+W W</code> (обратите внимание на разницу в регистре)
<code>Ctrl+W T</code>	Перейти к самому левому верхнему окну
<code>Ctrl+W Ctrl+T</code>	
<code>Ctrl+W B</code>	Перейти к самому правому нижнему окну
<code>Ctrl+W Ctrl+B</code>	
<code>Ctrl+W P</code>	Перейти к предыдущему (последнему доступному) окну
<code>Ctrl+W Ctrl+P</code>	

МНЕМОНИЧЕСКИЕ СОВЕТЫ

Буквы `t` и `b` являются мнемоническими символами для *верхнего* (*top*) и *нижнего* (*bottom*) окон.

В соответствии с правилом, согласно которому строчные и прописные буквы являются противоположностями, действие, вызываемое с помощью `Ctrl+W W`, противоположно `Ctrl+W Shift+W`.

Управляющие символы не различают верхний и нижний регистры. Другими словами, нажатие клавиши `Shift` вместе с клавишей `Ctrl` само по себе не приведет ни к какому результату. Однако различие между верхним и нижним регистром *распознается* для обычной клавиши клавиатуры, которую вы нажимаете после этого.

Перемещение окон

В Vim вы можете перемещать окна двумя способами: просто переставлять окна на экране или менять их фактическую компоновку. В первом случае размеры окна остаются неизменными во время смены позиции на экране. Во втором окна не только перемещаются, но и меняют размер, чтобы заполнить пространство, в которое их поместили.

Перемещение окон (поворот или перестановка)

Три команды перемещают окна, не меняя их компоновку. Две из них сдвигают окна позиционно в одном направлении (вправо или вниз) или в другом (влево или вверх), а третья меняет местами два, вероятно, несмежных окна. Эти команды работают *только* с той строкой или столбцом, в которых находится текущее окно.

Ctrl+W R поворачивает окна направо или вниз. Ее дополнением является команда **Ctrl+W Shift+R**, которая поворачивает окна в противоположном направлении.

Самый простой способ понять, как это работает, — представить строку или столбец окон Vim в виде одномерного массива. **Ctrl+W R** смещает каждый элемент массива на одну позицию вправо и перемещает последний элемент на первую освободившуюся позицию. **Ctrl+W Shift+R** просто перемещает элементы в другом направлении.

Если в столбце или строке нет окон, которые совпадают с текущим, то эта команда ничего не делает.

После того как Vim повернул окна, курсор остается в окне, из которого была выполнена команда поворота. Таким образом, курсор перемещается вместе с окном.

Ctrl+W X и **Ctrl+W Ctrl+X** позволяют вам поменять местами два окна в строке или столбце окон. По умолчанию Vim меняет местами текущее окно со следующим, а если следующего нет, то с предыдущим. Можно менять местами *n*-е окно, добавив перед командой число. Например, чтобы поменять местами текущее окно и третье после него, используйте команду **3 Ctrl+W X**.

Как и в двух предыдущих случаях, курсор остается в окне, из которого выполнялась команда.

Перемещение окон и их переконпоновка

Пять команд перемещают и изменяют компоновку окон: две перемещают текущее окно во всю ширину нижнего или верхнего окна, еще две — во всю высоту правого или левого окна, а пятая — в другую существующую вкладку. Для получения дополнительной информации о редактировании вкладок перейдите в раздел

«Редактирование с вкладками» далее. Первые четыре команды содержат знаковые мнемонические связи с другими командами Vim: к примеру, `Ctrl+W Shift+K` соответствует традиционному понятию `k` как «вверх». В табл. 10.3 приведен краткий перечень этих команд¹.

Таблица 10.3. Команды для перемещения и изменения компоновки окон

Команда	Описание
<code>Ctrl+W Shift+K</code>	Переместить текущее окно в верхнюю часть экрана, используя всю ширину экрана
<code>Ctrl+W Shift+J</code>	Переместить текущее окно в нижнюю часть экрана, используя всю ширину экрана
<code>Ctrl+W Shift+H</code>	Переместить текущее окно в левую часть экрана, используя всю высоту экрана
<code>Ctrl+W Shift+L</code>	Переместить текущее окно в правую часть экрана, используя всю высоту экрана
<code>Ctrl+W Shift+T</code>	Переместить текущее окно в новую вкладку

Сложно описать точное поведение этих команд. После перемещения и изменения размера окна Vim переконфигурирует окно соответствующим образом, до полной высоты или ширины экрана. На поведение компоновки также могут влиять некоторые настройки параметров окон.

Команды перемещения окон: краткое описание

В табл. 10.4 и 10.5 дано краткое описание команд, рассмотренных в этом разделе.

Таблица 10.4. Команды для поворота окон

Команда	Описание
<code>Ctrl+W R</code>	Повернуть окна вниз или вправо
<code>Ctrl+W Ctrl+R</code>	Повернуть окна вверх или влево
<code>Ctrl+W Shift+R</code>	Поменять местами со следующим окном или, если задано количество <i>n</i> , поменять местами с <i>n</i> -м окном

¹ Заглавные буквы или буквы с измененным регистром здесь являются своего рода расширением управления окнами. Помните, что с помощью этих команд вы перемещаете окна, а не курсор.

Таблица 10.5. Команды для изменения позиции и компоновки

Команда	Описание
<code>Ctrl+W Shift+K</code>	Переместить текущее окно в верхнюю часть экрана и использовать всю его ширину. Курсор остается в перемещаемом окне
<code>Ctrl+W Shift+J</code>	Переместить текущее окно в нижнюю часть экрана и использовать всю его ширину. Курсор остается в перемещаемом окне
<code>Ctrl+W Shift+H</code>	Переместить текущее окно в левую часть экрана и использовать всю его высоту. Курсор остается в перемещаемом окне
<code>Ctrl+W Shift+L</code>	Переместить текущее окно в правую часть экрана и использовать всю его высоту. Курсор остается в перемещаемом окне
<code>Ctrl+W Shift+T</code>	Переместить текущее окно в новую вкладку. Курсор остается в перемещаемом окне. Если текущее окно — единственное окно в текущей вкладке, никаких действий не предпринимается

Изменение размеров окон

Теперь, когда вы познакомились с функциями многооконности Vim, вам нужно научиться лучше ими управлять. В этом разделе вы узнаете, как изменить размер текущего окна, разумеется, с учетом влияния на другие окна на экране. Vim предоставляет параметры для управления размером окна и его поведением при открытии окон с помощью команд разделения.

Если вы предпочитаете управлять размером окон *без* команд, используйте `gvim` и работайте с помощью мыши. Просто перетаскивайте границы окна с ее помощью, чтобы изменить его размер. Для вертикально разделенных окон щелкните кнопкой мыши на вертикальном разделителе символа конвейеризации (`|`). Горизонтальные окна разделены строками состояния.

Команды изменения размера окна

Как и следовало ожидать, в Vim есть команды вертикального и горизонтального изменения размера. Как и другие команды для окон, они начинаются с `Ctrl+W` и прекрасно соотносятся с мнемоническими схемами, что упрощает их изучение и понимание.

Сочетание клавиш `Ctrl+W =` по возможности приводит все окна к единому размеру. На это также влияют текущие значения `winheight` и `winwidth`, которые мы рассмотрим в следующем разделе. Если доступное пространство экрана не делится

поровну, Vim изменяет размеры окон так, чтобы они были как можно более похожи по размеру.

Сочетание клавиш **Ctrl+W** — уменьшает высоту текущего окна на одну строку. В Vim также есть команда **ex**, позволяющая вам точно уменьшать размер окна. Например, команда **:resize -4** уменьшает высоту окна на четыре строки, пропорционально увеличивая окно под ним.



Существует еще один алгоритм, изменяющий размер окна: параметры строк. Обычно Vim управляет значением **lines**, и вы можете увидеть (и использовать) значение, ссылаясь на **lines** с помощью **ex**-команды **set**:

```
:set lines
```

Однако и для **gvim** допускается менять размер графического окна, задав **lines**. Побочным эффектом всего этого является то, что, если в терминале с одним окном вы установите значение **lines** меньше, чем количество строк в окне буфера **vim**, Vim изменяет размер области изображения в меньшую сторону. Соответственно Vim увеличивает строки, размещенные в командной строке **ex**, на количество строк, «потерянных» в окне буфера. Например, если в вашем файле **.vimrc** вы установите **lines**, равное 15, и запустите **vim** в 30-строковом окне, Vim выделит 15 строк в качестве буфера редактирования. Остальные строки будут размещены в строке состояния (одна строка) и в командном буфере **ex** (14 строк), что может привести в замешательство, поскольку командный буфер **ex** обычно занимает одну строку. Чтобы избежать подобного, мы рекомендуем использовать команду **:set lines=xx** только в вашем конфигурационном файле **.gvimrc**¹.

Сочетание клавиш **Ctrl+W** + увеличивает высоту текущего окна на одну строку, а команда **:resize +n** увеличивает ее на **n** строк. Как только достигнута максимальная высота окна, дальнейшее использование этой команды не дает результата.



Пожалуйста, обратитесь к подразделу «Изменение размера вашего окна» в главе 14, чтобы узнать о паре полезных переназначений клавиш, которые упростят изменение размера окна.

Команда **:resize n** устанавливает горизонтальный размер текущего окна равным **n** строк. Она задает абсолютный размер, в противоположность ранее описанным командам, которые вносят соответствующие изменения.

Команда **zn** устанавливает высоту текущего окна, равную **n** строк. Обратите внимание, что значение **n** — обязательно! Если его опустить, вы получите команду **vi/Vim z**, которая перемещает курсор в верхнюю часть экрана.

¹ Вы можете изменить высоту пространства команды **ex** с помощью параметра **cmdheight** (не путайте с **cmdwinheight**).

Сочетания клавиш `Ctrl+W <` и `Ctrl+W >` уменьшают и увеличивают ширину окна соответственно. Чтобы связать эти команды с их функциями, вспомните о мнемонической схеме «сдвиг влево» (`<<`) и «сдвиг вправо» (`>>`).

И наконец, сочетание клавиш `Ctrl+W |` изменяет ширину текущего окна на максимально возможную (по умолчанию). Вы также можете точно указать ширину окна с помощью команды `:verticalresize n`, где n — новая ширина окна.

Параметры изменения размера окна

Некоторые параметры Vim влияют на поведение команд изменения размера, описанных в предыдущем разделе.

- `winheight` и `winwidth` определяют минимальную высоту и ширину окна соответственно, когда окно становится активным. Например, если на экране размещены два окна размером по 45 строк, то по умолчанию Vim разделит их поровну. Если бы вы установили значение `winheight` больше 45, допустим 60, то всякий раз при переходе в новое окно Vim будет устанавливать высоту этого окна равной 60 строкам, а размер другого — 30. Это удобно при одновременном редактировании двух файлов. Вы автоматически увеличиваете размер выделенного окна при переключении от одного окна к другому и с одного файла на другой.
- `equalalways` говорит Vim всегда изменять размер окон поровну после разделения или закрытия окна. Этот параметр стоит устанавливать, чтобы обеспечить равнозначное распределение окон, когда вы добавляете и удаляете их.
- `eadirection` определяет полномочия для `equalalways`. Возможные значения `hor`, `ver` и `both` говорят Vim сделать окна равного размера по *горизонтали*, *вертикали* или *и так и так* соответственно. Изменение размера происходит каждый раз, когда вы разделяете или добавляете окно.
- `cmdheight` устанавливает высоту командной строки. Как было описано ранее, уменьшение высоты единственного окна увеличивает высоту командной строки. С помощью данного параметра можно сохранять желаемую высоту командной строки.
- `winminwidth` и `winminheight` определяют *минимальные* ширину и высоту для изменения размера окна. Vim воспринимает их как константы: окна не смогут стать меньше этих значений.

Краткое описание команд изменения размера окон

В табл. 10.6 приведены способы изменения размера окон. Параметры устанавливаются с помощью команды `:set`.

Таблица 10.6. Команды изменения размера окон

Команда или параметр	Тип	Описание
<code>Ctrl+W =</code>	Команда vi	Выровнять размер всех окон. Текущее окно принимает настройки параметров <code>winheight</code> и <code>winwidth</code>
<code>:resize -n</code>	Команда ex	Уменьшить размер текущего окна. Величина по умолчанию — одна строка
<code>Ctrl+W -</code>	Команда vi	То же, что и <code>:resize -n</code>
<code>:resize +n</code>	Команда ex	Увеличить размер текущего окна. Величина по умолчанию — одна строка
<code>Ctrl+W +</code>	Команда vi	То же, что и <code>:resize +n</code>
<code>:resize n</code>	Команда ex	Задать высоту текущего окна. По умолчанию устанавливается максимальная величина окна (если <i>n</i> не указано)
<code>Ctrl+W Ctrl+_</code> <code>Ctrl+W _</code>	Команда vi	То же, что и <code>:resize n</code>
<code>z n Enter</code>	Команда vi	Установить для текущего окна высоту <i>n</i>
<code>Ctrl+W <</code>	Команда vi	Уменьшить ширину текущего окна. Величина по умолчанию — один столбец
<code>Ctrl+W ></code>	Команда vi	Увеличить ширину текущего окна. Величина по умолчанию — один столбец
<code>:vertical resize n</code>	Команда ex	Установить для текущего окна ширину <i>n</i> . По умолчанию окно максимально широкое
<code>Ctrl+W </code>	Команда vi	То же, что и <code>:vertical resize n</code>
<code>cmdheight</code>	Параметр	Установить высоту командной строки
<code>eadirection</code>	Параметр	Определить способ выравнивания размера окна: вертикально, горизонтально или и так и так
<code>equalalways</code>	Параметр	Выровнять размер окон после разделения или закрытия окна
<code>winheight</code>	Параметр	При открытии окна или его создании установить его высоту не менее указанного значения
<code>winwidth</code>	Параметр	При открытии окна или его создании установить его ширину не менее указанного значения
<code>winminheight</code>	Параметр	Определить минимальную высоту окна и применить затем ко всем создаваемым окнам
<code>winminwidth</code>	Параметр	Определить минимальную ширину окна и применить затем ко всем создаваемым окнам

Буферы и их взаимодействие с окнами

Vim использует *буферы* в качестве контейнеров во время работы. Чтобы понять работу буферов, необходимо много практиковаться. Существует большое количество команд для управления ими и навигации по ним. Тем не менее стоит познакомиться с некоторыми основами буферов и задуматься, как и почему они существуют во время сеанса Vim.

Хорошим началом будет открыть несколько окон с разными файлами. Например, запустите Vim, открыв *файл1*. Затем внутри сеанса запустите команду `:set файл2`, а затем `:set файл3`. Теперь у вас должно быть открыто три файла в трех отдельных окнах Vim.

Теперь воспользуйтесь командами `:ls`, `:files` или `:buffers`, чтобы получить список буферов. Вы должны увидеть три пронумерованные строки, содержащие имена файлов вместе с дополнительной информацией. Это буферы Vim для текущего сеанса. Для каждого файла существует один буфер, а у каждого буфера есть уникальный неизменяемый связанный с ним номер. В нашем случае *файл1* находится в буфере один, *файл2* — в буфере два, а *файл3* — в буфере три.

Дополнительная информация о каждом буфере может быть отображена, если вы добавите восклицательный знак (!) после любой из перечисленных выше команд.

С правой стороны от номера каждого буфера расположены флаги состояния. Эти флаги описывают буферы, как показано в табл. 10.7.

Таблица 10.7. Флаги состояния, описывающие буферы

Код	Описание
и	Неперечисленный буфер. Буфер не отображается в списке, пока вы не используете модификатор !. Чтобы увидеть пример подобного буфера, введите <code>:help</code> . Vim разделяет текущее окно, чтобы открыть новое, в котором появляется встроенная справка. Обычная команда <code>:ls</code> не покажет буфер справки, но <code>:ls!</code> выведет его
% или #	% — буфер текущего окна. # — буфер, на который вы переключитесь с помощью команды <code>:edit #</code> . Оба являются взаимоисключающими
а или h	а обозначает активный буфер. То есть тот, который загружен и видим. h указывает на скрытый буфер. Такой буфер существует, но он не отображается ни в одном окне. Оба являются взаимоисключающими
– или =	– обозначает буфер с выключенным параметром <code>modifiable</code> . Файл доступен только для чтения. = является буфером, доступным только для чтения, который нельзя сделать изменяемым (к примеру, потому, что у вас нет прав для записи файла). Оба являются взаимоисключающими
+ или x	+ обозначает, что буфер изменен. x является буфером с ошибками считывания. Оба являются взаимоисключающими



Флаг `u` представляет интересный способ узнать, какой файл справки вы просматриваете в Vim. Например, если бы вы запустили команду `:help split`, за которой следует `:ls!`, вы бы увидели, что неперечисленный буфер ссылается на встроенный файл справки Vim `windows.txt`.

Теперь, когда вы знаете, как вывести список буферов Vim, мы можем поговорить о разнообразных способах их применения.

Специальные буферы Vim

Vim использует некоторые буферы, называемые *специальными*, для своих собственных целей. К примеру, описанные в предыдущем разделе буферы справки являются специальными. Обычно их нельзя редактировать или изменять.

Ниже приведено четыре примера специальных буферов Vim.

- *directory* — содержимое каталога, то есть список файлов в каталоге (и некоторые полезные дополнительные подсказки по командам). Это удобный инструмент в Vim, который позволяет вам перемещаться по буферу как по обычному текстовому файлу и выбирать файлы под курсором для редактирования, нажав клавишу `Enter`.
- *help* — содержит файлы справки Vim, описанные ранее в разделе «Встроенная справка» главы 8. Команда `:help` загружает эти файлы в специальный буфер.
- *QuickFix* — содержит список ошибок, созданных вашими командами (которые можно просмотреть с помощью `:cwindow`), или список местоположений (его можно увидеть, вызвав команду `:lwindow`). Не редактируйте содержимое этого буфера! Он помогает программистам повторять цикл «редактирование-компиляция-отладка». (См. главу 11.)
- *scratch* — эти буферы содержат текст для общих целей. Этот текст не сохраняется и может быть удален в любое время.

Скрытые буферы

Скрытые буферы представляют собой буферы Vim, которые в данный момент не отображаются ни в одном окне. Это упрощает редактирование нескольких файлов, учитывая ограниченное пространство экрана для нескольких окон, позволяя избежать постоянного считывания и перезаписывания файлов. Например, представьте, что вы редактируете файл *мой_файл*, но хотите ненадолго перейти к редактированию другого файла *мой_другой_файл*. Если установлен параметр `hidden`, то можно редактировать *мой_другой_файл* с помощью `:edit мой_другой_файл`, в результате чего Vim скроет буфер *мой_файл* и отобразит вместо него *мой_другой_файл*. Проверьте это с помощью `:ls` и обратите внимание, что оба буфера в списке, где *мой_файл* помечен как скрытый.

Команды буфера

Существует почти 50 команд, которые специально предназначены для буферов. Многие из них полезны, но большая часть находится за рамками данной книги. Vim управляет буферами автоматически, когда вы открываете и закрываете несколько файлов и окон. Набор команд для работы с буферами позволяет вам делать с ними практически все что угодно. Часто они используются внутри сценариев, чтобы обрабатывать такие задачи, как выгрузка, удаление или изменение буферов.

Мы познакомим вас с двумя командами буфера, которые способны выполнять большую работу со многими файлами.

- **windo cmd** — сокращение от **window do** (по крайней мере, мы думаем, что это подходящая мнемоническая схема). Эта команда псевдобуфера (на самом деле это команда окна) выполняет **cmd** в каждом окне. Она действует, если вы перейдете в верхнюю часть экрана (**Ctrl+W T**) и будете циклически просматривать каждое окно, чтобы выполнить указанную команду как **:cmd** в этом окне. Она действует только внутри текущей вкладки и останавливается в каждом окне, где **:cmd** выдает ошибку. Окно, в котором произошла ошибка, становится новым текущим окном. Для знакомства с вкладками Vim перейдите в раздел «Редактирование с вкладками» далее.

cmd не разрешено изменять состояние окон: она не может удалять, добавлять или менять их порядок.



cmd может соединять несколько команд **ex** с помощью символа вертикальной черты (**|**). Команды выполняются последовательно, при этом первая выполняется последовательно во всех окнах, затем — вторая и т. д.

В качестве примера команды **:windo** в действии предположим, что вы редактируете набор файлов Java и по какой-то причине у вас есть имя класса, в котором ошибочно использованы заглавные буквы. Вам нужно исправить это, изменив каждое вхождение **myPoorlyCapitalizedClass** на **MyPoorlyCapitalizedClass**. Это возможно с помощью **:windo**:

```
:windo %s/myPoorlyCapitalizedClass/MyPoorlyCapitalizedClass/g
```

Очень неплохо!

- **bufdo[!] cmd** — это аналог **windo**, но работает во всех буферах вашего сеанса редактирования, а не только с видимыми буферами в текущей вкладке. **bufdo**, как и **windo**, прекращает работу при первой ошибке и оставляет курсор в том буфере, где команда не была выполнена.

В следующем примере все буферы преобразуются в формат файла Unix:

```
:bufdo set fileformat=unix
```

Краткое описание команд буфера

В табл. 10.8 описаны не все команды, относящиеся к буферам. Здесь обобщены описанные в этом разделе и некоторые другие широко распространенные команды.

Таблица 10.8. Краткое описание команд буфера

Команда	Описание
<code>:ls[!]</code>	Вывести все буферы и имена файлов, включая неперечисленные буферы
<code>:files[!]</code>	(если модификатор <code>!</code> включен)
<code>:buffers[!]</code>	
<code>:ball</code>	Редактировать все аргументы или буферы (<code>sball</code> открывает их в новых окнах)
<code>:sball</code>	
<code>:unhide</code>	Редактировать все <i>загруженные</i> буферы (<code>sunhide</code> открывает их в новых окнах)
<code>:sunhide</code>	
<code>:badd <i>файл</i></code>	Добавить <i>файл</i> в список
<code>:bunload[!]</code>	Выгрузить текущий буфер из памяти. Модификатор <code>!</code> выгружает измененный буфер без сохранения
<code>:bdelete[!]</code>	Выгрузить текущий буфер и удалить его из списка буферов. Модификатор <code>!</code> выгружает измененный буфер без сохранения
<code>:buffer [<i>n</i>]</code>	Перейти к буферу <i>n</i> . <code>sbuffer</code> открывает новое окно
<code>:sbuffer [<i>n</i>]</code>	
<code>:bnext [<i>n</i>]</code>	Перейти к следующему <i>n</i> -му буферу. <code>sbnext</code> открывает новое окно
<code>:sbnext [<i>n</i>]</code>	
<code>:bNext [<i>n</i>]</code>	Перейти к следующему или предыдущему <i>n</i> -му буферу. <code>sbNext</code> и <code>sbprevious</code>
<code>:sbNext [<i>n</i>]</code>	открывают новое окно
<code>:bprevious [<i>n</i>]</code>	
<code>:sbprevious [<i>n</i>]</code>	
<code>:bfirst</code>	Перейти к первому буферу. <code>sbfirst</code> открывает новое окно
<code>:sbfirst</code>	
<code>:blast</code>	Перейти к последнему буферу. <code>sblast</code> открывает новое окно
<code>:sblast</code>	
<code>:bmod [<i>n</i>]</code>	Перейти к <i>n</i> -му измененному буферу. <code>sbmod</code> открывает новое окно
<code>:sbmod [<i>n</i>]</code>	

Управление тегами с помощью окон

Vim расширяет функциональные возможности тегов `vi` на окна, предлагая те же механизмы обхода тегов, но для нескольких окон. (Обсуждение тегов `vi` смотрите в подразделе «Использование тегов» в главе 7.) С помощью тега вы также можете открыть файл в соответствующем месте нового окна.

Команды окон для работы с тегами разделяют текущее окно и переходят либо к файлу, который соответствует тегу, либо к файлу, который соответствует имени файла под курсором.

- `:stag[!] tag` — эта команда разделяет окно для отображения местоположения найденного тега. Новый файл, содержащий соответствующий тег, становится текущим окном, а курсор помещается на соответствующий тег. Если тег не найден, команда завершается, а новое окно не создается.



Когда вы лучше познакомитесь с системой справки Vim, то сможете использовать команду `:stag`, чтобы открывать каждый новый термин справки в отдельном окне, вместо того чтобы переходить от файла к файлу в одном и том же окне.

- `Ctrl+W J` или `Ctrl+W ^` — эти сочетания клавиш разделяют окно и открывают новое над текущим. Новое окно становится текущим, а курсор помещается на соответствующий тег. Если нет такого тега, команда не выполняется.
- `Ctrl+W G` — такое сочетание клавиш разделяет окно и создает новое над текущим. В новом окне Vim выполняет команду `:tselect tag`, где `tag` — идентификатор тега под курсором. Курсор помещается в новое окно, которое становится текущим. Если нет сопоставимых тегов, команда не выполняется.
- `Ctrl+W G Ctrl+]` — данное сочетание клавиш работает как и `Ctrl+W G`, только вместо команды `:tselect` оно выполняет `:tjump`.
- `Ctrl+W F` или `Ctrl+W Ctrl+F` — эти сочетания клавиш разделяют окно и редактируют имя файла под курсором. Vim последовательно просматривает набор файлов в переменной параметра `path`, чтобы найти файл. Если файл не существует ни в одном каталоге `path`, команда не выполняется и не создает новое окно.
- `Ctrl+W Shift+F` — это сочетание клавиш разделяет окно и редактирует имя файла под курсором. Курсор помещается в новое окно этого файла и устанавливается на ту строку, которая соответствует номеру, следующему за именем файла в первом окне.
- `Ctrl+W G F` — открывает файл под курсором в новой вкладке. Если файл не существует, новая вкладка не создается.

- **Ctrl+W G Shift+F** — такое сочетание клавиш открывает файл под курсором в новой вкладке и устанавливает курсор на строку, указанную номером, следующим за именем файла в первом окне. Если файл не существует, новая вкладка не создается.

Редактирование с вкладками

Знали ли вы, что, кроме редактирования в нескольких окнах, вы можете создавать несколько *вкладок*? Vim позволяет создавать новые вкладки, каждая из которых действует независимо. В каждой из них вы можете разделить экран, редактировать несколько файлов, то есть делать практически все, что вы обычно делаете в одиночном окне, только теперь всей вашей работой легко управлять в одном месте.

Многие пользователи Chrome и Firefox очень хорошо знакомы с вкладками и признают ценность этой функции для редактирования¹. Непосвященным следует попробовать.

Вы можете использовать вкладки как в обычном Vim, так и в **gvim**, но в **gvim** это делать гораздо приятнее и удобнее. К наиболее важным способам создания и управления вкладками относятся:

- **:tabnew имя_файла или :tabedit имя_файла** — открывает новую вкладку и редактирует в ней файл (опционально). Если файл не указан, Vim откроет новую вкладку с пустыми буфером;
- **:tabclose** — закрывает текущую вкладку;
- **:tabonly** — закрывает все другие вкладки. Если в других вкладках есть измененные файлы, они не удаляются, если только не установлен параметр **autowrite**, в этом случае все измененные файлы записываются до закрытия других вкладок.

В **gvim** вы можете активировать любую вкладку, просто щелкнув на ней в верхней части экрана. Допускается также активировать вкладки в символьных терминалах с помощью мыши, если мышь настроена (см. параметр **mouse**). Также можно легко перемещаться справа налево от вкладки к вкладке, используя сочетания клавиш **Ctrl Page Down** (перейти на одну вкладку вправо) и **Ctrl Page Up** (перейти на одну вкладку влево). Если вы находитесь на крайней слева или крайней справа вкладке и пытаетесь перейти влево или вправо, Vim перейдет на крайнюю правую или на крайнюю левую вкладку соответственно.

¹ Круто, ведь, когда вышло седьмое издание, Chrome даже не существовал! А сегодня во всех браузерах есть вкладки, и все пользователи должны быть знакомы с ними.

gvim предоставляет всплывающее меню для вкладки, вызываемое правым щелчком кнопкой мыши. Из этого меню также можно открыть новую вкладку (с новым файлом для редактирования или без такового) и закрыть вкладку.

На рис. 10.4 приведен пример набора вкладок (обратите внимание на всплывающее меню вкладки). На рис. 10.5 представлен тот же пример в эмуляторе терминала.

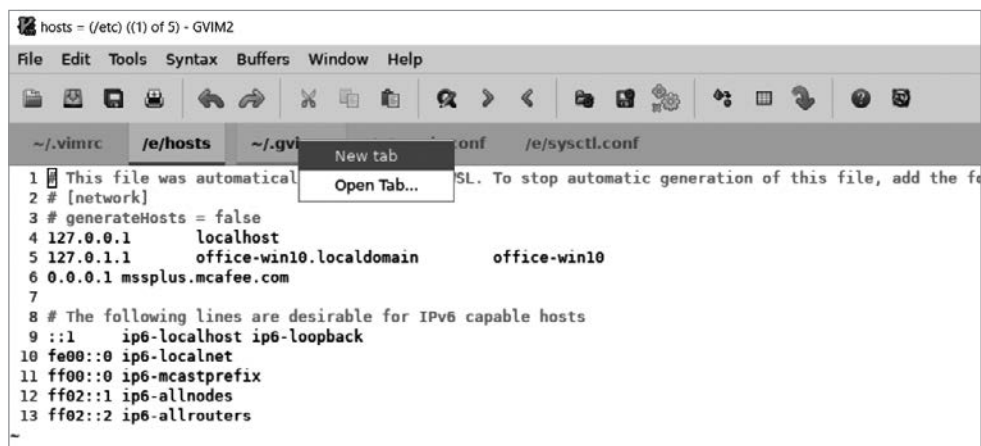


Рис. 10.4. Пример вкладок gvim и редактирования с вкладками

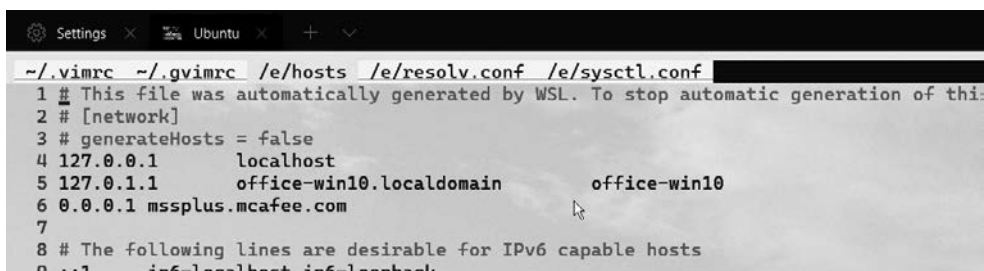


Рис. 10.5. Тот же пример вкладок в эмуляторе терминала (Linux Vim)



Параметр командной строки Vim `-p` открывает несколько файлов, в результате чего каждый из них занимает отдельную вкладку. Наши примеры на рис. 10.4 и 10.5 были вызваны следующим образом:

```
gvim -p ~/.vimrc ~/.gvimrc /etc/hosts /etc/resolv.conf/etc/sysctl.conf
vim -p ~/.vimrc ~/.gvimrc /etc/hosts /etc/resolv.conf/etc/sysctl.conf
```


Заккрытие и выход из окон

Существует четыре различных способа закрыть окно, специфичных для редактирования окон: *выйти*, *закрыть*, *скрыть* и *закрыть все остальные*.

- **Ctrl+W Q** или **Ctrl+W Ctrl+Q** — на самом деле это оконные версии команды `:quit`. В самом простом своем виде (например, в одиночном сеансе для одного окна) они ведут себя точно так же, как команда `vi :quit`. Если установлен параметр `hidden` и текущее окно является последним на экране, ссылающимся на этот файл, окно закрывается, но буфер файла сохраняется и скрывается (его можно извлечь). Другими словами, Vim все еще хранит файл, и вы можете потом вернуться к его редактированию. Если параметр `hidden` не установлен, а окно является последним ссылающимся на этот файл и есть несохраненные изменения, команда не выполняется, чтобы избежать потери ваших изменений. Но если файл отображается в каком-нибудь другом окне, то текущее закрывается.
- **Ctrl+W C** или `:close[!]` — эти команды закрывают текущее окно. Если установлен параметр `hidden` и это окно является последним ссылающимся на данный файл, то Vim закрывает окно, а буфер скрывает. Если это окно находится во вкладке и является последним для нее, окно *и* вкладка закрываются. Пока вы не используете модификатор `!`, команда не оставит ни одного файла с несохраненными изменениями. Модификатор `!` указывает Vim безоговорочно закрыть текущее окно.



Обратите внимание, что эта команда не использует **Ctrl+W Ctrl+C**, потому что Vim прибегает к **Ctrl+C** для отмены команд. Поэтому, если вы попытаетесь применить **Ctrl+W Ctrl+C**, то **Ctrl+C** просто отменит команду.

Аналогично, если команды **Ctrl+W** используются в сочетании с **Ctrl+S** и **Ctrl+Q**, некоторые пользователи могут обнаружить, что их эмуляторы терминалов зависли, потому что часть эмуляторов терминалов интерпретируют **Ctrl+S** и **Ctrl+Q** как управляющие символы для остановки и начала отображения информации на экране. Если ваш экран загадочным образом завис при использовании этого сочетания, попробуйте вместо него другие комбинации.

- **Ctrl+W O**, **Ctrl+W Ctrl+O** и `:only[!]` — эти команды закрывают все окна, за исключением текущего. Если задан параметр `hidden`, то все закрываемые окна скрывают свои буферы. Если он не установлен, любое окно, ссылающееся на файл с несохраненными изменениями, остается на экране, пока вы не задействуете модификатор `!`. В этом случае все окна закрываются и файлы не перезаписываются. На поведение этой команды может повлиять параметр `autowrite`: если он установлен, все окна закрываются, но окна, содержащие

несохраненные изменения, перед закрытием сохраняют содержимое своих файлов на диск.

- `:hide [cmd]` — эта команда завершает работу текущего окна и скрывает буфер, если другие окна не ссылаются на него. Если дополнительно указан *cmd*, буфер будет скрыт, а команда выполнится.

В табл. 10.9 кратко описаны эти команды.

Табл. 10.9. Команды для закрытия и выхода из окон

Команда	Сочетание клавиш	Описание
<code>:quit[!]</code>	<code>Ctrl+W Q</code> <code>Ctrl+W Ctrl+Q</code>	Выйти из текущего окна
<code>:close[!]</code>	<code>Ctrl+W C</code>	Закрыть текущее окно
<code>:only[!]</code>	<code>Ctrl+W O</code> <code>Ctrl+W Ctrl+O</code>	Сделать текущее окно единственным
<code>:hide[cmd]</code>		Закрыть текущее окно и скрыть буфер. Выполнить <i>cmd</i> , если задано

Резюме

Как вы теперь понимаете, Vim наращивает мощности редактирования с помощью своих многочисленных функций для окон. Vim позволяет вам создавать и удалять окна быстро и на лету. Кроме того, Vim обеспечивает скрытую производительность команд необработанного буфера, а буферы являются базовой инфраструктурой управления файлами, с помощью которой Vim управляет редактированием окон. Это еще один прекрасный пример того, как Vim преподносит многооконное редактирование новичкам, одновременно предоставляя опытным пользователям инструменты, необходимые им для настройки работы с окнами.

Расширенные возможности Vim для программистов

Редактирование текста — это всего лишь одна из функций Vim. Хорошим программистам необходимы мощные инструменты для обеспечения эффективной и профессиональной работы. Хороший редактор — это только начало, и одного его недостаточно. Многие современные среды программирования пытаются предоставить комплексные решения, однако все, что действительно нужно, — это мощный и эффективный редактор с некоторыми дополнительными возможностями.

Инструменты программирования предлагают дополнительные функции, начиная с подсветки синтаксиса, автоматических отступов, форматирования, автозавершения ключевых слов и т. д. и заканчивая полноценными интегрированными средами разработки (IDE) со сложной интеграцией, которые создают полноценные экосистемы разработки. Эти IDE могут быть как дорогими (например, Microsoft Visual Studio¹), так и бесплатными (Eclipse). И даже если требования к ресурсам компьютера невысоки, часто бывает достаточно чего-то облегченного. Vim удовлетворяет требованиям *легковесности*, не только предоставляя некоторые возможности IDE, но и с помощью распространяемых сообществом плагинов приближаясь к статусу IDE. (Для более глубокого погружения в тему разработки с помощью плагинов Vim IDE см. главу 15.)

Цели программистов разнообразны, как и технические требования. Небольшой объем работы можно легко выполнить с помощью простых редакторов, возможности которых чуть больше, чем простая правка текста. Значительные, многокомпонентные, мультиплатформенные и требующие целого штата программистов задачи *почти наверняка* потребуют уровня работы, который обеспечивают IDE. Но опыт показывает, что многие искушенные программисты чувствуют, что IDE предлагает едва ли больше, чем дополнительную сложность без повышения вероятности успеха.

¹ Не путайте с бесплатным и великолепным Microsoft Visual Studio Code.

Vim предлагает прекрасный компромисс между простыми редакторами и монолитным IDE. В нем есть функции, которые до недавнего времени были лишь в дорогих средах разработки. Он позволяет выполнять задачи быстро и просто без издержек и сложности обучения, характерных для IDE.

Множество параметров, средств, команд и функций специально разработаны для облегчения жизни программиста: от свертывания строк кода в одну строку до подсветки синтаксиса и автоматического форматирования. Vim предоставляет программистам много инструментов, которые можно полноценно оценить только при их использовании. На самом высоком уровне он предлагает своего рода мини-IDE под названием QuickFix, но у него есть также удобные функции, специфичные для различных задач программирования. В данной главе мы рассмотрим следующие темы.

- Свертывание.
- Автоматические и умные отступы.
- Автоматическое завершение по ключевым словам и словарю.
- Теги и расширенные теги.
- Подсветка синтаксиса и авторская подсветка (создание своей собственной).
- QuickFix, мини-IDE в Vim.

Свертывание и создание структуры (режим структуры)

Свертывание позволяет вам определять, какие части файла вы видите. Например, во фрагменте кода вы можете скрыть все что угодно с помощью фигурных скобок или скрыть все комментарии. Свертывание представляет собой двухэтапный процесс. Сначала с помощью любого из методов свертывания (которые мы скоро опишем) вы определяете содержимое фрагмента текста для данного действия. Затем, когда вы используете команду свертывания, Vim скрывает обозначенный текст и оставляет на его месте однострочный заполнитель. На рис. 11.1 показано, как выглядит свертывание в Vim. Вы можете управлять скрытыми строками с помощью заполнителя свертывания.

В данном примере строка 11 скрыта с помощью двухстрочного свертывания, начинающегося со строки 10. Восемистрочное свертывание начинается на строке 15 и скрывает строки с 15-й по 22-ю. А четырехстрочное свертывание начинается на строке 26 и скрывает строки с 26-й по 29-ю.

```
4
5 int fcn (int v1, int v2)
6 {
7
8     printf ("02 some line\n");
9     printf ("03 some line\n");
10 +-- 2 lines: printf ("04 some line\n");-----
12     printf ("06 some line\n");
13
14     if (thiscode == anysense)
15 +-- 8 lines: { -----
23
24     printf ("06 some line\n");
25     printf ("06 some line\n");
26 +-- 4 lines: printf ("06 some line\n");-----
30
31 }
```

Рис. 11.1. Пример свертывания Vim (MacVim, цветовая схема: zellner)

Практически не существует пределов для количества создаваемых свертываний. Вы даже можете создавать вложенные свертывания (свертывания внутри свертываний).

Несколько параметров управляют тем, как Vim создает и отображает свертывания. Кроме того, если вы потратили время на создание множества свертываний, Vim предоставляет удобные команды `:mkview` и `:loadview` для сохранения свертываний между сеансами, чтобы вам не пришлось создавать их снова.

Чтобы изучить свертывания, потребуется приложить некоторые усилия, но, освоив их, вы получите мощный способ управления отображением. Надежные и удобные в обслуживании программы требуют грамотного проектирования на нескольких уровнях, поэтому для хорошего программирования часто есть необходимость «увидеть лес за деревьями», другими словами, игнорировать детали реализации, чтобы увидеть общую структуру файла.

Опытным пользователям Vim предлагает шесть различных способов определения, создания и управления свертываниями. Эта гибкость позволяет создавать и управлять свертываниями в различных контекстах. В конечном счете после создания свертывание открывает, закрывает и реагирует похожим образом в отношении всего набора команд свертывания.

Шесть методов создания свертывания:

- `diff` — различия между двумя файлами определяют свертывания;
- `expr` — регулярные выражения определяют свертывания;

- `indent` — свертывания и уровни свертываний соответствуют отступу текста и значению параметра `shiftwidth`;
- `manual` — свертывания и уровни свертываний являются результатом команд пользователя Vim (например, свертывание абзаца);
- `marker` — предопределенные (но также определяемые пользователем) маркеры в файле указывают границы свертываний;
- `syntax` — свертывание соответствует семантике языка файла (например, функциональные блоки программы на языке C могут свертываться).

Все эти элементы — значения параметра `foldmethod`. Управление свертываниями (открытие и закрытие, удаление и т. д.) одинаково для всех методов. Мы расскажем о ручном свертывании и обсудим подробно команды свертывания Vim, а также немного затронем некоторые детали других более сложных методов, хотя они выходят за рамки данного знакомства. Мы надеемся, что наш обзор подтолкнет вас к более детальному изучению остальных методов.

Итак, давайте кратко рассмотрим важные команды свертывания и на практике разберем, что же это такое.

Команды свертывания

Все команды свертывания начинаются с символа `z`. Чтобы запомнить это, представьте сложенный (свернутый) в форме буквы `z` лист бумаги.

Существует около 20 команд свертывания, начинающихся с `z`. С их помощью вы создаете и удаляете свертывание, открываете и закрываете свертывание (скрываете или разворачиваете текст, принадлежащий свертыванию), переключаете состояние свертывания «развернуть/скрыть». Ниже представлены краткие описания команд¹:

- `zA` — рекурсивно переключает состояние свертываний;
- `zC` — рекурсивно закрывает свертывания;
- `zD` — рекурсивно удаляет свертывания;
- `zE` — удаляет *все* свертывания;
- `zf` — создает свертывания, начиная с текущей строки до той, куда следующая команда перемещения поместит курсор;

¹ Пожалуйста, обратите внимание и не перепутайте удаление свертываний с командой Vim `delete`. Удаление свертывания удаляет визуальную семантику скрытых строк. Удаление содержимого свертывания оказывает влияние на текст!

- [число] zF — создает свертывание, охватывающее *число* строк, начиная с текущей;
- zM — устанавливает значение параметра `foldlevel` равным нулю;
- zN, zn — устанавливает (zN) или сбрасывает (zn) параметр `foldenable`;
- zO — рекурсивно открывает свертывания;
- zA — переключает состояние одного свертывания;
- zC — закрывает одно свертывание;
- zD — удаляет одно свертывание;
- zi — переключает значение параметра `foldenable`;
- zj, zk — перемещает курсор в начало (zj) следующего свертывания или в конец (zk) предыдущего свертывания. Обратите внимание на мнемонические схемы команд перемещения j и k и на то, что они аналогичны перемещениям внутри контекста свертываний;
- zm, zr — уменьшает (zm) или увеличивает (zr) значение параметра `foldlevel` на единицу;
- zo — открывает одно свертывание.



Не перепутайте удаление свертывания и удаление текста. Используйте команду `zd`, чтобы удалить или снять определение свертывания. Удаленное свертывание никак не влияет на содержащийся в нем текст. Да, мы упоминаем об этом здесь уже не первый раз. Это важно знать. Один из нас потерял работу, думая, что он просто удаляет свертывания, а на самом деле — содержимое. Естественно, осознание пришло слишком поздно.

Команды `zA`, `zC`, `zD` и `zO` называются рекурсивными, потому что они работают со всеми свертываниями, вложенными в другое, в котором вы выполняете команды.

Ручное свертывание

Если вы знакомы с командами перемещения Vim, вы уже знаете половину того, что вы должны, чтобы искусно управлять командами ручного свертывания.

Например, чтобы свернуть три строки, введите одно из следующих сочетаний:

```
3zF
2zfj
```

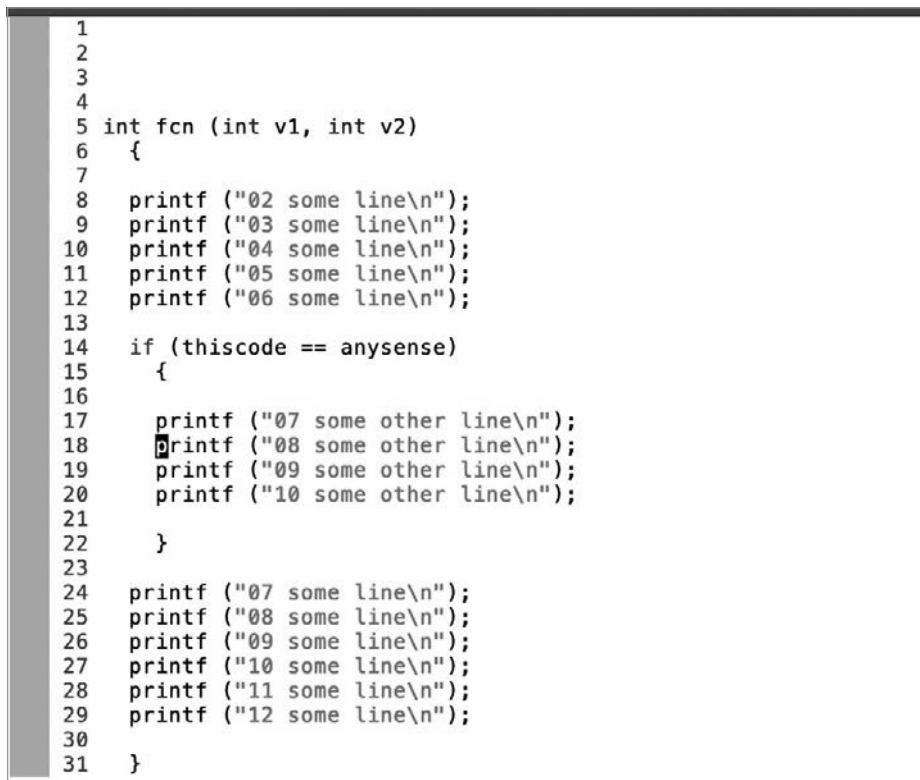
`3zF` выполняет команду свертывания `zF` для трех строк, начиная с текущей, а `2zfj` — команду `zf`, начиная с текущей строки до строки, куда `j` переместит курсор (на две строки вниз).

Попробуем использовать более сложную команду для программистов на языке C. Чтобы свернуть фрагмент кода на языке C, поместите курсор на открывающую или закрывающую фигурную скобку (`{` или `}`) фрагмента текста и введите `zf%`. (Помните, что `%` перемещает к соответствующей фигурной скобке.)

Создайте свертывание от курсора и до начала файла, введя `zfgg`. (`gg` переходит в начало файла).

Понять свертывание проще всего на примере. Возьмем простой файл, создадим свертывания, поуправляем ими и посмотрим за поведением. Мы также увидим некоторые улучшенные визуальные подсказки свертывания, предоставляемые Vim.

Для начала рассмотрим пример файла, представленного на рис. 11.2, который содержит некоторые (бессмысленные) строки кода на языке C. Изначально свертываний нет.



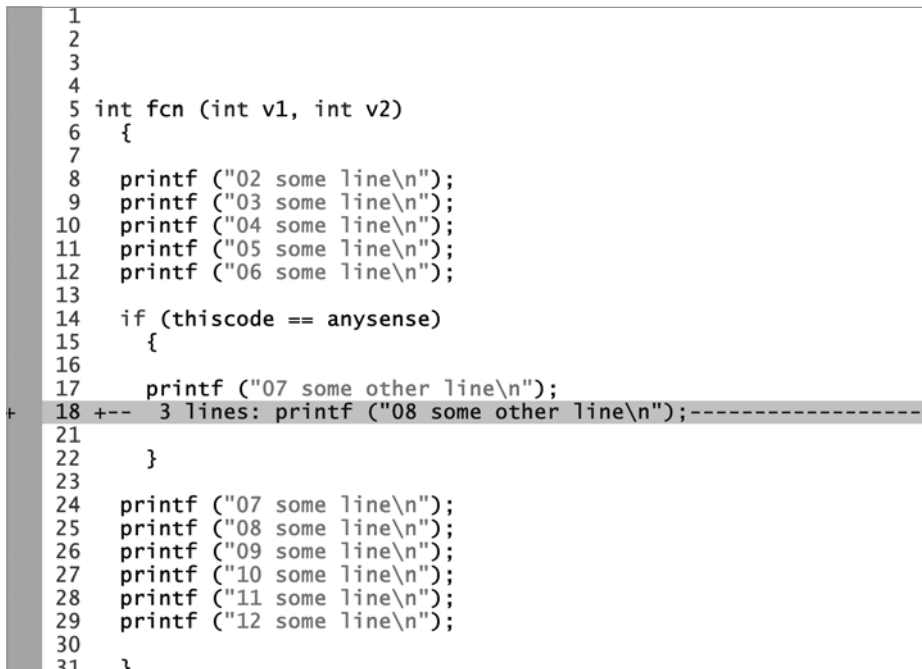
```
1
2
3
4
5 int fcn (int v1, int v2)
6 {
7
8     printf ("02 some line\n");
9     printf ("03 some line\n");
10    printf ("04 some line\n");
11    printf ("05 some line\n");
12    printf ("06 some line\n");
13
14    if (thiscode == anysense)
15    {
16
17        printf ("07 some other line\n");
18        printf ("08 some other line\n");
19        printf ("09 some other line\n");
20        printf ("10 some other line\n");
21
22    }
23
24    printf ("07 some line\n");
25    printf ("08 some line\n");
26    printf ("09 some line\n");
27    printf ("10 some line\n");
28    printf ("11 some line\n");
29    printf ("12 some line\n");
30
31 }
```

Рис. 11.2. Пример файла без свертываний (MacVim, цветовая схема: zellner)

Обратите внимание на несколько вещей на данном рисунке. Во-первых, Vim отображает номера строк на левой стороне экрана. Мы рекомендуем всегда их включать (с помощью параметра `number`) для дополнительной визуальной информации о местоположении в файле, а когда вы сворачиваете строки и они пропадают из поля зрения, эта информация становится более ценной. Vim показывает вам, сколько строк не отображается, а номера подтверждают и подкрепляют эти сведения.

Также обратите внимание на серые столбцы слева от номеров строк. Они зарезервированы для дополнительных визуальных подсказок свертывания. По мере создания и использования свертываний мы увидим визуальные подсказки, которые Vim вставляет в эти столбцы.

На рис. 11.2 курсор расположен на строке 18. Свернем эту и две последующие строки в одно свертывание, введя `zf2j`. На рис. 11.3 показан результат.



```
1
2
3
4
5 int fcn (int v1, int v2)
6 {
7
8     printf ("02 some line\n");
9     printf ("03 some line\n");
10    printf ("04 some line\n");
11    printf ("05 some line\n");
12    printf ("06 some line\n");
13
14    if (thiscode == anysense)
15    {
16
17        printf ("07 some other line\n");
18 +-- 3 lines: printf ("08 some other line\n");-----
19
20
21
22    }
23
24    printf ("07 some line\n");
25    printf ("08 some line\n");
26    printf ("09 some line\n");
27    printf ("10 some line\n");
28    printf ("11 some line\n");
29    printf ("12 some line\n");
30
31 }
```

Рис. 11.3. Три строки, свернутые в строке 18 (MacVim, цветовая схема: zellner)

Обратите внимание, как Vim создает легко идентифицируемый маркер с помощью `+-` в качестве префикса и как он отображает текст, начиная с первой свернутой строки в заполнителе *свертывания*. А теперь взгляните на еще одну визуальную подсказку: символ `+` в крайней левой части экрана.

В том же файле мы свернем еще один фрагмент кода после оператора `if` между фигурными скобками, включая сами скобки. Поместите курсор на одну из фигурных скобок и введите `zf%`¹. Теперь файл выглядит так, как показано на рис. 11.4.

```

10 printf ("04 some line\n");
11 printf ("05 some line\n");
12 printf ("06 some line\n");
13
14 if (thiscode == anysense)
15  +-- 8 lines: { -----
23
24 printf ("07 some line\n");
25 printf ("08 some line\n");
26 printf ("09 some line\n");
27 printf ("10 some line\n");
28 printf ("11 some line\n");
29 printf ("12 some line\n");
30
31

```

Рис. 11.4. Фрагмент кода, свернутый после оператора `if` (MacVim, цветовая схема: `zellner`)

Теперь свернуто восемь строк кода, три из которых содержатся в созданном ранее свертывании. Это называется *вложенным* свертыванием. Обратите внимание, что ничто не указывает на вложенное свертывание.

Наш следующий эксперимент будет состоять в помещении курсора на строку 25 и свертывании всех строк вплоть до объявления функции `fcn`, включая ее саму. На этот раз мы используем *поиск* в движении. Выполните свертывание с помощью `zf`, а затем обратный поиск до начала функции `fcn` с помощью `?int fcn` (команда обратного поиска в Vim). Нажмите клавишу `Enter`. Экран теперь выглядит как на рис. 11.5.

```

3
4
+ 5 +-- 21 lines: int fcn (int v1, int v2)---
26 printf ("09 some line\n");
27 printf ("10 some line\n");
28 printf ("11 some line\n");
29 printf ("12 some line\n");

```

Рис. 11.5. Свертывание до начала функции (MacVim, цветовая схема: `zellner`)

¹ Свертывание является общим; вы можете использовать концепцию текстовых объектов, представленных в части I. Таким образом можно свертывать в любом месте внутри объекта, в данном примере — везде внутри фигурных скобок. Команда `vi` будет `zf`.



Если вы считаете строки и создаете свертывание, которое охватывает другое свертывание (например, 3zf), то все содержащиеся в захваченном свертывании строки считаются за одну строку. Например, если курсор находится на строке 30, а строки 31–35 скрыты в свертывании на следующей строке экрана, то есть следующая строка на экране отображает строку 36, то 3zf создаст новое свертывание, содержащее эти строки, как они показаны на экране: текстовая строка 30, пять строк, содержащихся в свертывании со строками 31–35, и текстовая строка 36, отображаемая в следующей строке на экране. Запутанно? Немного. Можно сказать, что команда zf считает строки по принципу «что вижу, то и сворачиваю».

Попробуем другие функции. Сначала откройте все свертывания с помощью команды **z0** (после **z** буква **0**, а не ноль). Теперь вы можете увидеть некоторые визуальные подсказки в левом поле, как показано на рис. 11.6. Каждый столбец здесь называется столбцом свертывания.

```

4
5 int fcn (int v1, int v2)
6 {
7
8     printf ("02 some line\n");
9     printf ("03 some line\n");
10    printf ("04 some line\n");
11    printf ("05 some line\n");
12    printf ("06 some line\n");
13
14    if (thiscode == anysense)
15    {
16
17        printf ("07 some other line\n");
18        printf ("08 some other line\n");
19        printf ("09 some other line\n");
20        printf ("10 some other line\n");
21
22    }
23
24    printf ("07 some line\n");
25    printf ("08 some line\n");
26    printf ("09 some line\n");
27    printf ("10 some line\n");
28    printf ("11 some line\n");
29    printf ("12 some line\n");
30
31 }
```

Рис. 11.6. Все открытые свертывания (MacVim, цветовая схема: zellner)

На этом рисунке первая строка каждого свертывания отмечена знаком минус (–), а все остальные строки свертывания — вертикальной чертой (|).

Самое большое (ближайшее внешнее) свертывание находится в крайнем левом столбце, а самое дальнее внутреннее свертывание — в крайнем правом. Как видно на рисунке, строки 5–25 представляют самый низкий уровень свертывания (в данном случае первый), строки 15–22 — следующий уровень свертывания (второй), а строки 18–20 — самый высокий уровень.



По умолчанию этот полезный визуальный образ выключен (мы не знаем почему, может быть, потому что он занимает слишком много места на экране). Включите его и задайте ему ширину с помощью следующей команды:

```
:set foldcolumn=n
```

где *n* — количество используемых столбцов (максимум 12, по умолчанию 0). На рисунке мы использовали `foldcolumn=5`. Самые внимательные заметят, что на предыдущих рисунках значение `foldcolumn` было равно 3. Мы изменили значение для более удобного восприятия.

Теперь создадим больше свертываний, чтобы понаблюдать, как они действуют.

Для начала сверните заново самое глубокое свертывание, которое охватывает строки 18–20, поместив курсор на любую строку внутри диапазона этого свертывания и введя `zc` (закрыть свертывание). На рис. 11.7 показан результат.

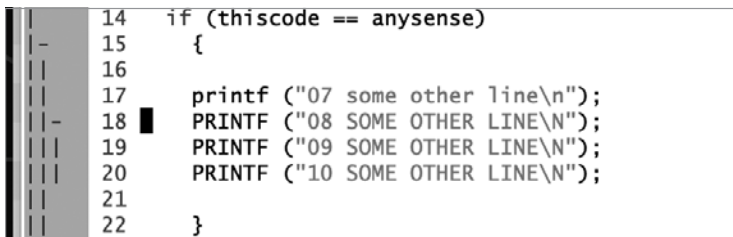
```

5 int fcn (int v1, int v2)
6 {
7
8     printf ("02 some line\n");
9     printf ("03 some line\n");
10    printf ("04 some line\n");
11    printf ("05 some line\n");
12    printf ("06 some line\n");
13
14    if (thiscode == anysense)
15    {
16
17        printf ("07 some other line\n");
18 +---- 3 lines: printf ("08 some other line\n");-----
19
20    }
21
22    }
23
24    printf ("07 some line\n");
25    printf ("08 some line\n");
26    printf ("09 some line\n");
27    printf ("10 some line\n");
28    printf ("11 some line\n");
29    printf ("12 some line\n");
30
31 }
```

Рис. 11.7. После повторного свертывания строк 18–20 (MacVim, цветовая схема: zellner)

Видите изменения на сером поле? Vim поддерживает визуальные подсказки, упрощая визуализацию и управление вашими свертываниями.

Теперь посмотрим, что делает со свертыванием типичная команда «одной строки». Поместите курсор на свернутую строку (18) и введите `~` (смена регистра для всех символов в текущей строке). Обратите внимание, что это используется для изменения регистра в строке, когда установлен параметр Vim `tildeop`; в противном случае будет команда `g~`. Помните, что в Vim `~` является объектным оператором (если не установлен параметр `compatible`) и поэтому должен переключать регистр всех символов в строке. Далее откройте свертывание, введя `zo` (открыть свертывание). Код должен выглядеть как на рис. 11.8.



```
14  if (thiscode == anysense)
15  {
16
17      printf ("07 some other line\n");
18      PRINTF ("08 SOME OTHER LINE\n");
19      PRINTF ("09 SOME OTHER LINE\n");
20      PRINTF ("10 SOME OTHER LINE\n");
21
22  }
```

Рис. 11.8. Изменение регистра, примененное к свертыванию
(MacVim, цветовая схема: zellner)

Это очень мощная функция. Команды строк или операторы воздействуют на весь текст, подставленный в строке свертывания! Правда, данный пример может показаться надуманным, но он прекрасно иллюстрирует потенциал этой техники.



Как мы только что увидели, любое действие со свертыванием воздействует на все свертывание. Например, если вы поместите курсор (см. рис. 11.7) на строку 18, то есть свертывание, скрывающее строки с 18-й по 20-ю, и введете `dd` (удалить строку) — все три строки и свертывание будут удалены.

Важно также отметить, что Vim управляет всеми действиями редактирования, как если бы свертываний не существовало, поэтому любые отмены аннулируют все действия редактирования. Так, если вы введете `u` (отменить) после предыдущего удаления, все три строки будут восстановлены. Функция отмены отличается от рассмотренных в этом разделе «однострочных» действий, хотя и может показаться, что они действуют похожим образом.

Теперь самое время, чтобы познакомиться с визуальными подсказками в поле столбца свертывания. С их помощью проще увидеть, как будет действовать свертывание. Например, команда `zc` (закреть свертывание) закрывает самое дальнейшее внутреннее свертывание, содержащее строку, на которой расположен курсор.

Размер этого свертывания можно узнать по вертикальным чертам в столбцах свертывания. Как только вы освоитесь, такие действия, как открытие, закрытие и удаление свертываний, станут привычными.

Создание структуры

Рассмотрим следующий простой (и придуманный) файл, в котором для отступов используется табуляция:

1. This is Headline ONE with NO indentation and NO fold level.
 - 1.1 This is sub-headline ONE under headline ONE
This is a paragraph under the headline. Its fold level is 2.
 - 1.2 This is sub-headline TWO under headline ONE.
2. This is Headline TWO. No indentation, so no folds!
 - 2.1 This is sub-headline ONE under headline TWO.
Like the indented paragraph above, this has fold level 2.
 - Here is a bullet at fold level 3.
A paragraph at fold level 4.
 - Here is the next bullet, again back at fold level 3.
 - And, another set of bullets:
 - Bullet one.
 - Bullet two.
 - 2.2 This is sub-heading TWO under Headline TWO.
3. This is Headline THREE.

Вы можете использовать свертывания Vim, чтобы посмотреть свой файл как псевдоструктуру. Определите метод свертывания как `indent`:

```
:set foldmethod=indent
```

В нашем файле мы установим для `shiftwidth` (уровень отступа для табуляций) значение 4. Теперь можно открывать и закрывать свертывания на основании отступов строк. Для каждой ширины сдвига (в данном случае кратной четырем столбцам) в строке с отступом ее уровень свертывания увеличивается на единицу. Например, подзаголовки в вашем файле имеют отступ, равный одной ширине сдвига или четырем столбцам. Следовательно, уровень свертывания равен единице. У строк с отступом в восемь столбцов (две ширины сдвига) уровень свертывания равен двум и т. д.

Чтобы управлять уровнем видимых свертываний, используйте команду `foldlevel`. Она принимает целое число в качестве аргумента и отображает только строки, чьи уровни свертываний *меньше или равны* данному аргументу. В текущем файле мы можем запросить отображать только заголовки самого высокого уровня:

```
:set foldlevel=0
```

Теперь наш экран должен выглядеть как на рис. 11.9.

```

+ s/foldfileC_2.c  s/foldfile.txt
1 1. This is Headline ONE with NO indentation and NO fold level.
2 +-- 4 lines: 1.1 This is sub-headline ONE under headline ONE-----
6 2. This is Headline TWO. No indentation, so no folds!
7 +-- 9 lines: 2.1 This is sub-headline ONE under headline TWO-----
16 3. This is Headline THREE.

```

Рис. 11.9. Результат команды `:set foldlevel=0`
(Linux gvim, цветовая схема: zellner)

Чтобы отобразить все вплоть до маркеров (включительно), установите значение `foldlevel` равным 2. Тогда все свертывания с уровнем *больше одного* будут выглядеть как на рис. 11.10.

Благодаря этому методу проверки вашего файла стало возможным быстро разворачивать и сворачивать видимые уровни детализации с помощью команд инкремента (`zr`) или декремента (`zm`).

```

+ s/foldfileC_2.c  s/foldfile.txt
1 1. This is Headline ONE with NO indentation and NO fold level.
2   1.1 This is sub-headline ONE under headline ONE
3     This is a paragraph under the headline. Its fold
4     level is 2.
5   1.2 This is sub-headline TWO under headline ONE.
6 2. This is Headline TWO. No indentation, so no folds!
7   2.1 This is sub-headline ONE under headline TWO.
8     Like the indented paragraph above, this has fold level 2.
9 +---- 3 lines: - Here is a bullet at fold level 3-----
12      And, another set of bullets:
13 +---- 2 lines: - Bullet one-----
15   2.2 This is heading TWO under Headline TWO.
16 3. This is Headline THREE.

```

Рис. 11.10. Результат команды `:set foldlevel=2`
(Linux gvim, цветовая схема: zellner)

Несколько слов о других методах свертывания

Мы не можем охватить абсолютно все методы свертывания, но, чтобы подогреть ваш аппетит, бегло взглянем на метод `syntax`.

Возьмем тот же файл на языке C, что и раньше, однако на этот раз мы позволим Vim решать, что свертывать, основываясь на синтаксисе языка C. Правила,

регулирующие свертывание на языке C, сложны, но данного фрагмента кода достаточно, чтобы продемонстрировать автоматические возможности Vim.

Сначала убедитесь, что вы избавились от всех свертываний: введите `zE` (исключить все свертывания). На экране должен отображаться весь код без визуальных маркеров в столбце свертывания.

Удостоверьтесь, что свертывание включено:

```
:set foldenable
```

При ручном свертывании можно было этого не делать, так как по умолчанию параметр `foldenable` включен, а значение `foldmethod` установлено на `manual`. Теперь введите следующие две команды Vim `ex`:

```
:syntax on
:set foldmethod=syntax
```

Свертывание отобразится, как показано на рис. 11.11.

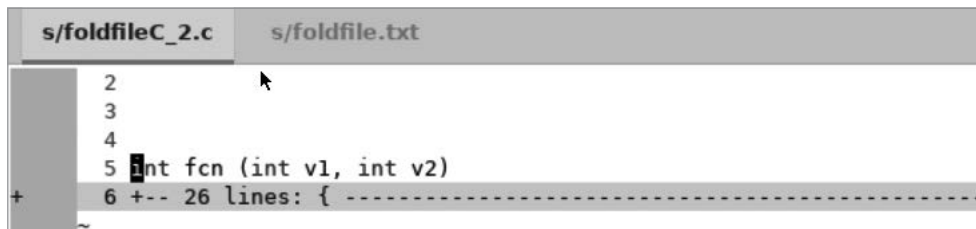


Рис. 11.11. После команды `:set foldmethod=syntax`
(Linux `gvim`, цветовая схема: `zellner`)

Vim свернул все заключенные в скобки фрагменты кода, потому что они являются логическими смысловыми блоками в C. Если ввести `zo` в строке 6 данного примера, Vim развернет ветку и покажет внутреннее свертывание.

Каждый метод свертывания применяет свои правила для скрытия части текста. Мы рекомендуем вам засучить (завернуть?) рукава и углубиться в познание этих методов с помощью документации Vim.

Режим сравнения Vim (доступный через команду `vimdiff`) сочетает в себе свертывания, управление окнами и подсветку синтаксиса, о чем мы поговорим позже. Как видно на рис. 11.12, режим показывает различия между файлами, обычно между двумя версиями одного документа.

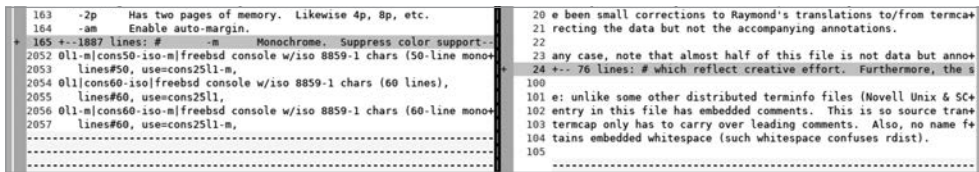


Рис. 11.12. Режим Vim diff и его использование свертываний
(Linux gvim, цветовая схема: zellner)

Автоматические и умные отступы

Vim имеет четыре мощных и продвинутых метода для автоматического выравнивания текста. В общем случае Vim ведет себя почти так же, как vi с включенным параметром `autoindent`, и даже использует такое же название для описания своего поведения (о том, как vi делает автоматические отступы, см. подраздел «Управление отступом» в главе 7).

Вы можете выбрать метод расстановки отступов, просто указав его в команде `:set`, например, как:

```
:set cindent
```

Существуют следующие методы Vim, перечисленные от простых к сложным.

- **autoindent** — автоматический отступ, практически в точности повторяет `autoindent` в vi. Отличие лишь в том, куда помещается курсор после удаления отступа.
- **smartindent** лишь немного эффективнее, чем `autoindent`, но он распознает некоторые базовые синтаксические примитивы языка C для определения уровней отступов.
- **cindent** включает более глубокое понимание синтаксиса языка C и предлагает сложные настройки, выходящие за рамки простых уровней отступов. Например, `cindent` можно настроить так, чтобы он соответствовал вашим (или вашего начальника) любимым стилям программирования, включая (но не ограничиваясь) отступы фигурных скобок (`{}`), их местоположение, наличие отступов одной или обеих скобок и даже то, как отступ соответствует включенному тексту.
- **Indentexpr** позволяет вам задавать свое собственное выражение, которое Vim будет использовать для определения отступа каждой новой строки. Этот метод дает вам полный контроль над правилами. Подробнее об этом можно прочитать в главе 12 и в документации Vim. Если другие три метода не удовлетворяют вашим потребностям в автоматическом отступе, то `indentexpr`, несомненно, справится с этой задачей.

Расширения autoindent Vim для autoindent vi

Метод `autoindent` для Vim работает похожим образом в `vi`, и можно сделать его точно таким же, установив параметр `compatible`. Одно из приятных расширений `autoindent` в Vim — возможность определять «тип» файла и добавлять необходимые символы комментариев при переносе строк комментариев на новую строку. Эта функция совместима либо с параметром `wrapmargin` (текст переносится в пределах столбцов `wrapmargin` справа), либо с параметром `textwidth` (текст переносится, когда длина строки превышает значение `textwidth`). На рис. 11.13 видно разницу между одинаковыми входными данными, обработанными `autoindent` Vim и `vi`.

```
1 #! /bin/sh
2
3 if [ $xyz -eq 0 ]
4 then
5     # this block of comments I typed
6     # with option textwidth set to 40,
7     # autoindent on, and
8     # (automatically), syntax=sh.
9     # Notice how each line has the '#'
10    # with a separating space, all
11    # courtesy of vim's autoindent...
12    # now I will type the same text but
13    # instead with the option
14    # "compatible" (with vi) set...
15
16    # this block of comments I typed
17    with option textwidth set to 40,
18    autoindent on, and (automatically),
19    syntax=sh. Notice how each line has
20    the '#' with a separating space, all
21    courtesy of vim's autoindent... now
22    I will type the same text but
23    instead with the option "compatible"
24    (with vi) set...
```

Рис. 11.13. Разница между `autoindent` Vim и `vi`
(Linux `gvim`, цветовая схема: `zellner`)

Обратите внимание, что во втором фрагменте текста (строка 16 и далее) нет символа комментария в начале. Кроме того, при установленном параметре `compatible` (для имитации поведения `vi`) параметр `textwidth` игнорируется и текст переносится только потому, что у `wrapmargin` задано значение.

Параметр `smartindent`

`smartindent` слегка улучшает `autoindent`. Он удобен, но если вы пишете код на языке, похожем на C, с довольно сложным синтаксисом, вам больше подойдет `cindent`.

`smartindent` автоматически вставляет отступы, когда:

- новая строка следует за строкой с левой фигурной скобкой (`{`);
- новая строка начинается с ключевого слова из параметра `cinwords`;
- новая строка создается перед строкой, начинающейся с правой фигурной скобки (`}`), *если* курсор расположен на строке со скобкой и пользователь нажимает `O` (открыть строку сверху);
- новая строка — это закрывающая (правая) фигурная скобка (`}`).



Обычно требуется включить `autoindent` при использовании `smartindent`:

```
:set autoindent
```

Параметр `cindent`

Постоянные пользователи Vim, которые программируют на C-подобных языках, интересуются параметрами `cindent` или `indentexpr`. Хотя `indentexpr` более мощный, гибкий и настраиваемый метод, `cindent` удобнее для большинства задач программирования. У него множество настроек, которые покрывают почти все потребности программистов (и корпоративных стандартов). Поработайте с ним сначала со стандартными настройками, а затем измените их по своему вкусу, если ваши стандарты отличаются.



Если параметр `indentexpr` задан, он отменяет действия `cindent`.

Три параметра определяют поведение `cindent`:

- `cinkeys` — определяет клавиши на клавиатуре, которые сигнализируют Vim повторно рассчитать отступ;
- `cinoptions` — определяет стиль отступа;

- `cinwords` — определяет ключевые слова, которые сигнализируют Vim, когда нужно добавить дополнительный отступ в последующих строках.

`cindent` следует правилам `cinkeys` для определения отступов. Давайте разберемся с `cinkeys` и другими настройками.

Параметр `cinkeys`

`cinkeys` представляет собой разделенный запятыми список значений:

`0{,0},0),:,0#,!^F,o,0,e`

Ниже представлены значения с контекстами и краткими описаниями.

- `0{` — ноль означает, что следующий символ (`{`) должен быть *первым в строке* контекста. То есть, если вы введете `{` в начале строки, Vim автоматически подберет отступ для этой строки.

Но это не значит, что ноль убирает отступ для этого символа. Ноль просто проверяет положение символа в строке.

Отступ для `{` по умолчанию равен нулю: новый отступ не будет добавляться поверх текущего. Следующий пример показывает типичные результаты:

```
main ()
{
    if ( argv[0] == (char *)NULL )
    { ...
```

- `0},0)` — эти две настройки также проверяют положение символов `}` и `)` в *начале строки* контекста. Если вы введете их в начале строки, Vim пересчитает отступ.

По умолчанию отступ для `}` соответствует отступу открывающей фигурной скобки. Для `)` он равен одному `shiftwidth`.

- `:` — это метка `C`, или контекст оператора `case`. Если `:` (двоеточие) введено в конце метки или оператора `case`, Vim пересчитает отступ.

По умолчанию отступом для `:` является столбец 1 — первый столбец в строке. Это отличается от нулевого отступа, который сохраняет отступ на уровне предыдущей строки. Если отступ равен единице, новая строка начинается в *самом* крайнем слева столбце.

- `0#` — и снова это *начало строки* контекста. Если первый символ строки — `#` (решетка), Vim пересчитает отступ.

По умолчанию как и в предыдущем определении, то есть отступ сдвигает всю строку в первый столбец. Это соответствует правилу начинать определения макросов (`#define ...`) с первого столбца.

- `!^F` — специальный символ `!` определяет любой последующий символ как триггер для пересчета отступа текущей строки. В данном случае раздражителем служит `^F`, или `Ctrl+F`, поэтому, если вы нажимаете `Ctrl+F` в строке, Vim пересчитает отступ для нее.
- `o` — этот контекст служит для создания новой строки ниже текущей с помощью клавиши `Enter` в режиме *ввода* или команды `o` (открыть новую строку).
- `O` — этот контекст служит для создания новой строки *над* текущей с помощью команды `O` (открыть новую строку выше).
- `e` — это контекст *else*. Если строка начинается со слова `else`, Vim автоматически пересчитает отступ. Vim не распознает данный контекст, пока последняя *e* из *else* не будет введена.

Правила синтаксиса `cinkeys`

Каждое определение `cinkeys` состоит из необязательного префикса (`!`, `*` или `0`) и клавиши, для которой пересчитывается отступ. У префиксов следующие значения.

- `!` — обозначает клавишу (по умолчанию `Ctrl+F`), которая заставляет Vim повторно рассчитать отступ текущей строки. Вы можете указать несколько таких клавиш (используя синтаксис `+=`) без замены ранее существовавшей команды. Любая добавленная к определению `!` клавиша будет также выполнять свои функции по умолчанию.
- `*` — заставляет Vim пересчитать отступ текущей строки, прежде чем ввести клавишу.
- `0` — устанавливает контекст *начала строки*. Клавиша, указанная после `0`, запускает переоценку отступа только в том случае, если она первая в строке.



Помните, в `vi` и Vim между «первым символом в строке» и «первым столбцом в строке» есть разница. Команда `^` переходит к первому символу строки, но не к первому столбцу (выравнивание по левому краю). То же самое происходит и при вводе `I`. Таким же образом префикс `0` относится к символу, который вводится как первый в строке, вне зависимости от того, выровнен ли он по левому краю или нет.

Для `cinkeys` есть специальные имена клавиш и способ замены любых зарезервированных символов, например префиксов. Ниже представлены параметры специальных клавиш.

- `<>` — используйте эту форму для буквального определения клавиш. Для специальных непечатаемых клавиш используйте буквенные версии. Например,

вы можете определить буквальный символ `:` с помощью `<: >`. Или определить стрелку вверх как `<Up>`.

- `^` — используйте символ каретки (^) для обозначения управляющего символа. Например, `^F` означает клавиши `Ctrl+F`.
- `o`, `O`, `e`, `:` — мы видели эти специальные клавиши в значении по умолчанию для `cinkeys`.
- `= word`, `=~ word` — это параметры для определения слова с особым поведением отступа. Если строка `word` совпадает и находится в начале новой строки, Vim автоматически вычислит новый отступ.

Форма `=~word` аналогична `=word`, только она не учитывает регистр.



Термин `word` не совсем точно отражает его смысл. Он означает начало слова, потому что триггер срабатывает при обнаружении совпадения строки. Однако это не значит, что конец строки должен быть также концом слова. В документации Vim приводится пример слова `end`, которое совпадает как с `end`, так и с `endif`.

Параметр `cinwords`

`cinwords` задает ключевые слова, которые при наборе вызывают дополнительный отступ в текущей строке. По умолчанию это такие слова: `if`, `else`, `while`, `do`, `for`, `switch`.

Они соответствуют стандартным ключевым словам в языке C.



Регистр ключевых слов имеет значение. При их проверке Vim даже не учитывает параметр `ignorecase`. Если вам нужны разные регистры ключевых слов, перечислите все варианты в `cinwords`.

Параметр `cinoptions`

`cinoptions` управляет тем, как Vim форматирует строки текста в соответствии с синтаксисом языка C. Он содержит ряд параметров для различных стилей форматирования кода, таких как:

- глубина отступа для фрагмента кода внутри фигурных скобок;
- наличие или отсутствие новой строки перед фигурной скобкой после условного оператора;

- выравнивание фрагментов кода относительно фигурных скобок, в которые они заключены.

`cinoptions` имеет такие параметры со значением по умолчанию: `s, e0, n0, f0, {0, }0, ^0, :s, =s, l0, b0, gs, hs, ps, ts, is, +s, c3, C0, /0, (2s, us, U0, w0, W0, m0, j0, )20, *30`.

Параметр `cinoptions` позволяет настроить отступы в Vim по-разному. С его помощью можно задать небольшие различия в контекстных блоках, количество строк для определения контекста и уровни отступов для разных ситуаций. Также вы можете переопределить число столбцов для отступов или использовать множитель (может быть любым числом) `shiftwidth`. Например, `cinoptions=f5` сделает для открывающей фигурной скобки (`{`) отступ в пять столбцов, если она не находится внутри *других* скобок. А если вы добавите в определение `w` (например, `cinoptions=f5w`), то открывающая фигурная скобка сдвинется на пять ширины сдвига.

Для отрицательного отступа (влево) нужно поставить знак минус (`-`) перед любым числовым значением.



Значение данной опции нужно изменять с особой осторожностью. Помните, что синтаксис `=` полностью переопределяет его. Поскольку параметр `cinoptions` содержит множество всевозможных настроек, используйте следующие команды осмотрительно: `+=` для добавления параметра, `-=` для удаления текущего параметра и `-=`, за которым следует `+=`, для изменения существующего параметра.

Далее приведен краткий список параметров, которые вы, скорее всего, захотите изменить.

- `>n` (по умолчанию `s`) — любая строка, для которой указан отступ, должна иметь отступ на `n` позиций. По умолчанию это значение `s`, которое означает, что отступ для строки равен одной ширине сдвига.
- `fn, {n` — `f` определяет, насколько глубоко делать отступ открывающей невложенной фигурной скобки (`{`). Значение по умолчанию равно нулю, то есть фигурные скобки выравниваются по их логическому аналогу. Например, фигурная скобка после строки с оператором `while` располагается под буквой `w` слова `while`.

Фигурная скобка (`{`) ведет себя так же, как `f`, но применяется к *вложенным* открывающим фигурным скобкам. И в этом случае отступ по умолчанию тоже равен нулю.

На рис. 11.14 и 11.15 показан один и тот же текст в Vim, где значение параметра `shiftwidth` равно 4 (четыре столбца), однако в первом случае `cinoptions=s,f0,{0`, а во втором — `cinoptions=s,fs,{s`.

```

18
19 while (condition)
20 {
21     if (someothercondition)
22     {
23         printf("looks like I've got both conditions!\n");
24     }
25 }
26

```

Рис. 11.14. Результат `:set cinoptions=s,f0,{0`
(терминал WSL Ubuntu Linux, цветовая схема: zellner)

```

26
27 while (condition)
28 {
29     if (someothercondition)
30     {
31         printf("looks like I've got both conditions!\n");
32     }
33 }
34

```

Рис. 11.15. Результат `:set cinoptions=s,fs,{s`
(терминал WSL Ubuntu Linux, цветовая схема: zellner)

- `}n` — эта настройка позволяет определить смещение закрывающей фигурной скобки относительно открывающей. Значение по умолчанию равно нулю (выравнено по соответствующей фигурной скобке).
- `^n` — добавляет `n` к текущему отступу внутри набора фигурных скобок (`{...}`), если открывающая фигурная скобка находится в первом столбце.
- `:n`, `=n`, `bn` — эти три настройки управляют отступом в операторах `case`. С помощью `:` Vim делает отступ от метки `case` на `n` символов от позиции соответствующего оператора `switch`. Значение по умолчанию равно одной ширине сдвига.

Параметр `=` определяет отступ строк кода от соответствующей метки `case`. По умолчанию отступ равен одной ширине сдвига.

Настройка `b` определяет место размещения операторов `break`. Значение по умолчанию (ноль) выравнивает `break` с другими операторами внутри соответствующего блока `case`. Любое ненулевое значение выравнивает `break` с соответствующей ему меткой `case`.

- `)n`, `*n` — эти две опции передают Vim количество строк, которые необходимо просмотреть, чтобы найти незакрытые круглые скобки (по умолчанию 20) и незакрытые комментарии (по умолчанию 30) соответственно.



По идее, эти последние два параметра ограничивают доступные для Vim ресурсы машины. Однако с современными мощными компьютерами эти значения можно увеличить. Попробуйте для начала удвоить их, установив 40 и 60 соответственно.

Параметр `indentexpr`

Если `indentexpr` определен, он замещает `cindent`, позволяя точно настроить правила отступа под потребности редактирования вашего языка.

`indentexpr` задает выражение, которое оценивается при каждом создании новой строки в файле. Результат этого выражения превращается в целое число, которое Vim использует в качестве отступа для новой строки.

Кроме того, `indentexpr` может определять полезные ключевые слова аналогично параметру `cindent`, определяющему строки, для которых выполняется пересчет отступа.

Беда в том, что написать пользовательские правила отступов с нуля — нетривиальная задача для любого языка. Однако хорошая новость в том, что, вероятно всего, работа уже выполнена. В каталоге `$VIMRUNTIME/indent` можно найти более 120 файлов отступов, включающих такие распространенные языки программирования, как *ada*, *awk*, *docbook* (файл отступов называется *docbk*), *eiffel*, *fortran*, *html*, *java*, *lisp*, *pascal*, *perl*, *php*, *python*, *ruby*, *scheme*, *sh*, *sql* и *zsh*. Есть даже файл отступов, определенный для *xinetd*!

Можно указать Vim автоматическое распознавание типа вашего файла и загружать файл определения отступа, поместив команду `filetype indent on` в файл `.vimrc`. Если правила отступа не удовлетворяют вашим потребностям, вы можете выключить определения с помощью команды `:filetype indent off`.

Мы рекомендуем опытным пользователям изучить файлы определения отступов, которые есть в Vim. А если вы разработали новые файлы определений или улучшили существующие, отправьте их на рассмотрение в vim.org. Возможно, они войдут в будущий пакет Vim.

Заключительное слово об отступах

Прежде чем завершить наше обсуждение, стоит обратить внимание на следующие моменты, касающиеся работы с автоматическими отступами.

- *Когда автоматические отступы не применяются.* Если вы изменяете отступ строки вручную, Vim больше не будет автоматически определять отступ для этой строки.

- *Копирование и вставка.* Vim самостоятельно применяет правила автоматических отступов ко всем вставляемым строкам, что может привести к непредсказуемым результатам. Как правило, в итоге вы рискуете получить текст с сильным перекосом в правую часть экрана и без соответствующего отступа в левую сторону.

Чтобы избежать этой проблемы, перед вставкой текста установите параметр `Vim paste`. Он полностью перенастроит все автоматические функции Vim и позволит вставить текст без автоматических отступов. Команда `:set nopaste` сбрасывает эту настройку.

Завершение по ключевым словам и словарю

Vim предлагает полный набор возможностей *завершения*, начиная от специальных ключевых слов языка программирования и заканчивая именами файлов, словами из словаря и целыми строками. Кроме того, Vim обобщает семантику завершения на основе словаря, чтобы включить завершение на основе синонимов для введенного слова из тезауруса!

В этом разделе мы рассмотрим различные методы завершения, их синтаксис и описание того, как они работают, на примерах. Методы завершения включают:

- целую строку;
- ключевые слова текущего файла;
- ключевые слова параметра `dictionary`;
- ключевые слова параметра `thesaurus`;
- ключевые слова текущего и *включаемого* файла;
- теги (как в `ctags`);
- имена файлов;
- макросы;
- командную строку Vim;
- методы, определяемые пользователем;
- `omni`;
- предложения по написанию;
- ключевые слова параметра `complete`.

Кроме последнего метода, остальные начинаются с **Ctrl+X**. Вторая клавиша определяет конкретный тип завершения, который пытается выполнить Vim. Например, команда автоматического завершения имен файлов — **Ctrl+X Ctrl+F**. К сожалению, не все команды такие мнемонические. Vim использует непереназначенные клавиши, что позволяет вам сократить большинство этих команд до одного нажатия с помощью подходящего переназначения команд. Например, вы можете переназначить **Ctrl+X Ctrl+N** на просто **Ctrl+N**.

У всех методов завершения практически одинаковое поведение: они циклически просматривают список возможных завершений при повторном вводе второй клавиши. Таким образом, если вы выбрали автоматическое завершение имени файла с помощью **Ctrl+X Ctrl+F** и не получили необходимое слово с первой попытки, можно продолжить нажимать **Ctrl+F**, чтобы увидеть другие варианты. Кроме того, если нажать **Ctrl+N** (то есть «следующий»), вы переместитесь вперед по возможным вариантам, если **Ctrl+P** (то есть «предыдущий») — назад.

Давайте рассмотрим некоторые методы автоматического завершения на примерах и подумаем, как они могут быть полезны.

Команды завершения вставки

Эти методы (используемые в режиме ввода) имеют широкий функционал, начиная с простого поиска слов в текущем файле и заканчивая поиском функций, переменных, макросов и других имен по всему коду. Последний метод сочетает особенности других, обеспечивая приятный компромисс между эффективностью и сложностью.



Чтобы упростить себе жизнь, вы можете выбрать свой любимый метод завершения и назначить ему удобную клавишу. Например, **Tab**:

```
:imap Tab <C-P>
```

Правда, в этом случае придется пожертвовать возможностью легко вставлять табуляции, но зато это позволит применять ту же клавишу, которая используется для завершения в командной строке в операционной системе DOS и оболочках *xterm*, *konsole* и т. д. (помните, вы всегда можете вставить табуляцию, заключив ее в кавычки с помощью **Ctrl+V**).

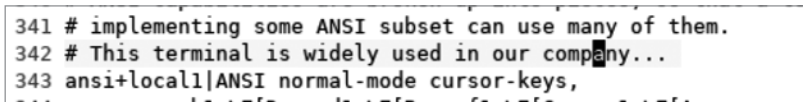
Переназначение клавиши **Tab** также соответствует обычной клавише завершения в командном режиме *ex* Vim.

В следующих подразделах описаны многочисленные способы, которыми Vim позволяет вам выполнять завершение.

Завершение целых строк

Вы можете завершить целые строки с помощью сочетания клавиш **Ctrl+X Ctrl+L**. Данный метод выполняет обратный поиск в текущем файле строки, соответствующей введенным вами символам. Для понимания принципов работы завершения рассмотрим пример.

Предположим, у вас есть файл, содержащий определения терминала или консоли, где описаны функции терминала и способы их управления. Допустим, ваш экран похож на рис. 11.16.



```

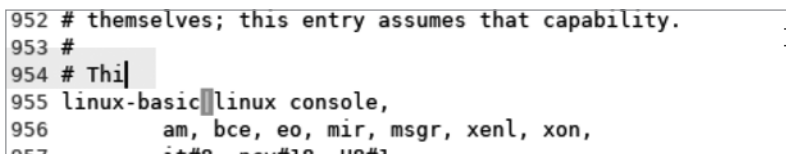
341 # implementing some ANSI subset can use many of them.
342 # This terminal is widely used in our company...
343 ansi+local|ANSI normal-mode cursor-keys,

```

Рис. 11.16. Пример построчного завершения (Linux gvim, цветовая схема: zellner)

Обратите внимание на подсвеченную строку, содержащую `this terminal is widely used in our company...`. Эта строка требуется во многих местах, где вы отмечаете терминалы как `widely used` (широко используемые) в своей компании. Просто введите достаточное количество символов строки, чтобы она стала уникальной или почти уникальной, а затем — **Ctrl+X Ctrl+L**. Таким образом отобразится частичный ввод строки (рис. 11.17):

Thi



```

952 # themselves; this entry assumes that capability.
953 #
954 # Thi|
955 linux-basic|linux console,
956         am, bce, eo, mir, msgr, xenl, xon,
957         it#0, rev#10, u0#1

```

Рис. 11.17. Частично напечатанная строка, ожидающая завершения (Linux gvim, цветовая схема: zellner)

С помощью комбинации клавиш **Ctrl+X Ctrl+L** в Vim можно вызвать список всех возможных завершений для текущей строки, основанных на ранее введенных в файле строках. На рис. 11.18 показан пример такого списка завершений.

На шкале полутонов (в печатной книге) сложно разглядеть текст, но на экране в Vim доступно цветное всплывающее окно, содержащее множественные вхождения строк, соответствующих началу нашей неполной строки. Также здесь отображается информация о том, где найдено совпадение, хотя на рисунке этого

не видно. Данный метод использует параметр `complete` для определения области видимости, в которой происходит поиск совпадений. Подробнее об области видимости мы поговорим в конце раздела.

```
963 # This version of terminfo.src is distributed with ncurses and is maintained
964 # This file describes the capabilities of various character-cell terminals,
965 # This file uses only the US-ASCII character set (no ISO8859 characters).
966 # This file assumes a US-ASCII character set. If you need to fix this, start
967 # this file.
968 # this file is becoming a historical document (this is part of the reason for
969 # this file deliberately has no copyright. It belongs to no one and everyone.
970 # This section describes terminal classes and brands that are still
971 # This is almost the same as "dumb", but with no prespecified width.
972 # This terminal is widely used in our company...
973 # This works with the System V, Linux, and BSDI consoles. It's a safe bet this
974 # This is better than kclone+color, it doesn't assume white-on-black as the
975 # This section lists entries in a least-capable to most-capable order.
976 # This completely describes the sequences specified in the DOS 2.1 ANSI.SYS
977 # This should only be used when the terminal emulator cannot redefine the keys
```

Рис. 11.18. Вид после нажатия `Ctrl+X Ctrl+L` (Linux gvim, цветовая схема: zellner)

Во время использования всплывающего¹ списка вы можете перемещаться вперед (`Ctrl+N`) или назад (`Ctrl+P`) по списку, а выбранные в нем элементы будут подсвечиваться. Также можно использовать клавиши со стрелками для перемещения вверх и вниз. Чтобы выбрать нужное совпадение, нажмите `Enter`. Если в списке нет подходящих вариантов, введите `Ctrl+E`, чтобы остановить метод сопоставления без замены какого-либо текста. Ваш курсор вернется в свою исходную позицию.

На рис. 11.19 показан результат выбора нужного варианта из списка.

```
954 # This entry is good for the 1.2.13 or later version of the Linux console.
955 # This terminal is widely used in our company...|
956 linux-basic|linux console,
```

Рис. 11.19. Результат ввода `Ctrl+X Ctrl+L` и выбора соответствующей строки (Linux gvim, цветовая схема: zellner)

Завершение по ключевым словам в файле

Комбинация `Ctrl+X Ctrl+N` позволяет выполнить прямой поиск по текущему файлу ключевых слов, соответствующих слову перед курсором. После нажатия этих клавиш вы сможете использовать `Ctrl+N` и `Ctrl+P` для перемещения по списку вперед и назад соответственно. Чтобы выбрать совпадение, нажмите `Enter`. Вы также можете использовать клавиши со стрелками для перемещения вверх и вниз.

¹ Всплывающий список находится в gvim; Vim ведет себя немного по-другому.



Следует учесть, что у термина «ключевое слово» широкое определение. Это могут быть ключевые слова, знакомые программистам, а также любые другие слова в файле. Слова определяются как последовательный набор символов в параметре `iskeyword`. Настройки по умолчанию `iskeyword` достаточно логичны, но вы можете переопределить параметр, чтобы включить или исключить определенные знаки пунктуации. Символы в `iskeyword` можно указать либо напрямую (например, `a-z`), либо с помощью соответствующих кодов ASCII (например, использование 97-122 для представления `a-z`).

К примеру, по умолчанию символ подчеркивания может быть частью слова, но точка и дефис воспринимаются как разделитель. Это прекрасно работает для C-подобных языков программирования, но для других сред это не всегда наилучший выбор.

Завершение по словарю

С помощью `Ctrl+X Ctrl+K` можно искать ключевые слова, соответствующие слову перед курсором, в файле, который задан параметром `dictionary`.

По умолчанию этот параметр не определен, однако существуют стандартные пути расположения файлов словаря. Кроме того, допускается создавать собственные подобные файлы. Вот наиболее распространенные:

- `/usr/share/dict/words` (Cygwin on MS-Windows, Ubuntu GNU/Linux);
- `/usr/share/dict/web2` (FreeBSD);
- `$HOME/.mydict` (личный список словарных слов).

Завершение по тезаурусу

Используйте `Ctrl+X Ctrl+T` для поиска ключевых слов, соответствующих ключевому слову перед курсором в файлах, определенных параметром `thesaurus`.

Когда Vim находит совпадение и если строка в файле тезауруса содержит более одного слова, Vim включает все эти слова в список вариантов завершения.

По-видимому, данный метод предназначен для предоставления синонимов, однако он позволяет *вам* определять ваш собственный стандарт. Давайте рассмотрим пример файла со следующими строками¹:

```
fun,enjoyable,desirable  
funny,hilarious,lol,rotfl,lmao  
retrieve,getchar,getcwd,getdirent,getline,getgrent,getgrgid,...
```

¹ Обратите внимание, что слова в каждой строке синонимов разделены запятыми. Чтобы запятая считалась частью слова, заключите ее в обратные кавычки.

Первые две строки содержат синонимы, типичные для английского языка (например, *fun* и *funny*). Третья строка может пригодиться программистам на языке C, которые регулярно используют имена функций, начинающихся с *get*. Для таких функций мы можем использовать синоним *retrieve*.

В реальной жизни мы бы разделили тезаурусы для английского языка и языка C, поскольку Vim способен работать с несколькими тезаурусами одновременно.

В режиме ввода наберите слово *fun*, а затем нажмите **Ctrl+X Ctrl+T**. На рис. 11.20 показано, что у вас отобразится в *gvim*.

Обратите внимание на следующее.

- Vim находит *любое* слово в записи тезауруса, а не только первое слово в каждой строке файла тезауруса.
- Vim включает варианты слов из всех строк в тезаурусе, которые соответствуют ключевому слову перед курсором. Таким образом, в данном случае Vim находит совпадения как для *fun*, так и для *funny*.

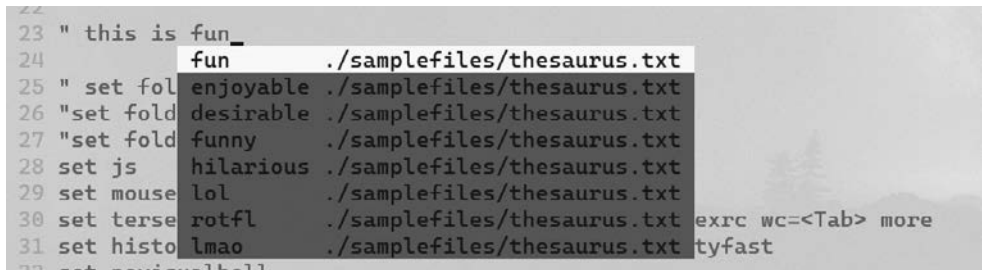


Рис. 11.20. Завершение тезауруса для слова *fun*
(WSL Ubuntu Linux, цветовая схема: zellner)



Другое интересное и, возможно, неожиданное поведение параметра *thesaurus* заключается в том, что сравнение может производиться с любыми словами в строке файла тезауруса, а не только с первым. Например, если в строке:

```
funny hilarious lol rotfl lmao
```

вы введете *hilar* и завершите его, Vim включит в список все слова *hilarious*, *lol*, *rotfl* и *lmao*. Смешно!

Заметили ли вы дополнительную информацию в списке вариантов для завершения? Вы можете узнать, где Vim нашел совпадение, добавив значение *preview* в параметр *completeopt*.

Теперь давайте рассмотрим другой пример с использованием того же файла. Введем в нем неоконченное слово `retrie`. Оно сопоставимо с *retrieve* — синонимом, который мы выбрали для *getting* и вписали все имена функции *get* в качестве синонимов. Теперь **Ctrl+X Ctrl+T** выводит нам всплывающее меню (в *gvim*) всех наших функций в качестве вариантов для завершения (рис. 11.21).

```

22
23 " Now create a retrieve_
24 retrieve      ./samplefiles/thesaurus.txt
25 getchar      ./samplefiles/thesaurus.txt
26 "set foldcol  ./samplefiles/thesaurus.txt
27 "set foldmeth  ./samplefiles/thesaurus.txt
28 "set foldlevel  ./samplefiles/thesaurus.txt
29 set js       ./samplefiles/thesaurus.txt
30 set mouse=a  ./samplefiles/thesaurus.txt
31 set terse sw=1 ./samplefiles/thesaurus.txt
32 set history=10 ./samplefiles/thesaurus.txt
33 set novisualbe ./samplefiles/thesaurus.txt
34 set nrformats= ./samplefiles/thesaurus.txt
35 set dictionary ./samplefiles/thesaurus.txt
36 set statusline= ./samplefiles/thesaurus.txt
37 set statusline= ./samplefiles/thesaurus.txt
38 set statusline= ./samplefiles/thesaurus.txt
39 set statusline= ./samplefiles/thesaurus.txt
40 set statusline= ./samplefiles/thesaurus.txt
41 set statusline= ./samplefiles/thesaurus.txt
42 set statusline= ./samplefiles/thesaurus.txt
43 set statusline= ./samplefiles/thesaurus.txt
44 set statusline= ./samplefiles/thesaurus.txt
45 set statusline= ./samplefiles/thesaurus.txt
46 set statusline= ./samplefiles/thesaurus.txt
47 set statusline= ./samplefiles/thesaurus.txt
48 set statusline= ./samplefiles/thesaurus.txt
49 set statusline= ./samplefiles/thesaurus.txt
50 set statusline= ./samplefiles/thesaurus.txt
51 set statusline= ./samplefiles/thesaurus.txt
52 set statusline= ./samplefiles/thesaurus.txt
53 set statusline= ./samplefiles/thesaurus.txt
54 set statusline= ./samplefiles/thesaurus.txt
55 set statusline= ./samplefiles/thesaurus.txt
56 set statusline= ./samplefiles/thesaurus.txt
57 set statusline= ./samplefiles/thesaurus.txt
58 set statusline= ./samplefiles/thesaurus.txt
59 set statusline= ./samplefiles/thesaurus.txt
60 set statusline= ./samplefiles/thesaurus.txt
61 set statusline= ./samplefiles/thesaurus.txt
62 set statusline= ./samplefiles/thesaurus.txt
63 set statusline= ./samplefiles/thesaurus.txt
64 set statusline= ./samplefiles/thesaurus.txt
65 set statusline= ./samplefiles/thesaurus.txt
66 set statusline= ./samplefiles/thesaurus.txt
67 set statusline= ./samplefiles/thesaurus.txt
68 set statusline= ./samplefiles/thesaurus.txt
69 set statusline= ./samplefiles/thesaurus.txt
70 set statusline= ./samplefiles/thesaurus.txt
71 set statusline= ./samplefiles/thesaurus.txt
72 set statusline= ./samplefiles/thesaurus.txt
73 set statusline= ./samplefiles/thesaurus.txt
74 set statusline= ./samplefiles/thesaurus.txt
75 set statusline= ./samplefiles/thesaurus.txt
76 set statusline= ./samplefiles/thesaurus.txt
77 set statusline= ./samplefiles/thesaurus.txt
78 set statusline= ./samplefiles/thesaurus.txt
79 set statusline= ./samplefiles/thesaurus.txt
80 set statusline= ./samplefiles/thesaurus.txt
81 set statusline= ./samplefiles/thesaurus.txt
82 set statusline= ./samplefiles/thesaurus.txt
83 set statusline= ./samplefiles/thesaurus.txt
84 set statusline= ./samplefiles/thesaurus.txt
85 set statusline= ./samplefiles/thesaurus.txt
86 set statusline= ./samplefiles/thesaurus.txt
87 set statusline= ./samplefiles/thesaurus.txt
88 set statusline= ./samplefiles/thesaurus.txt
89 set statusline= ./samplefiles/thesaurus.txt
90 set statusline= ./samplefiles/thesaurus.txt
91 set statusline= ./samplefiles/thesaurus.txt
92 set statusline= ./samplefiles/thesaurus.txt
93 set statusline= ./samplefiles/thesaurus.txt
94 set statusline= ./samplefiles/thesaurus.txt
95 set statusline= ./samplefiles/thesaurus.txt
96 set statusline= ./samplefiles/thesaurus.txt
97 set statusline= ./samplefiles/thesaurus.txt
98 set statusline= ./samplefiles/thesaurus.txt
99 set statusline= ./samplefiles/thesaurus.txt
100 set statusline= ./samplefiles/thesaurus.txt

```

Рис. 11.21. Завершение тезауруса для слова `retrie` (WSL Ubuntu Linux, цветовая схема: *zellner*)

Как и в случае с другими методами завершения, нажмите **Enter**, чтобы подтвердить выбор.



Не стоит путать завершение тезауруса с проверкой орфографии — другой прекрасной функцией Vim. Информацию о проверке орфографии в Vim можно найти в главе 13.

Завершение по ключевым словам в текущем файле и во включаемых файлах

Эта функция будет полезна для программистов на языке C и C++, где часто используются файлы `#include`. Комбинация **Ctrl+X Ctrl+I** выполняет поиск ключевых слов, сопоставимых с ключевым словом перед курсором, в текущем файле и включаемых файлах. Данный метод отличается от метода «искать в текущем файле» (**Ctrl+X Ctrl+P**) тем, что Vim проверяет текущий файл на наличие ссылок на файл `include` и выполняет поиск и в этих файлах.

Vim использует значение в `include` для выявления строк, которые ссылаются на файлы `include`. По умолчанию используется шаблон, указывающий Vim найти строки, сопоставимые со стандартной конструкцией C:

```
# include <somefile.h>
```


В этом случае Vim находит соответствия в файле `somefile.h` в стандартных каталогах файлов `include` в системе. Vim также использует параметр `path` в качестве списка каталогов для поиска включаемых файлов.

Завершение по тегу

Комбинация `Ctrl+X Ctrl+]` позволяет искать ключевые слова, соответствующие *тегам*, в текущем файле и включаемых файлах. Более подробно о тегах можно узнать в подразделе «Использование тегов» в главе 7.

Завершение по имени файла

Комбинация `Ctrl+X Ctrl+F` позволяет искать имена файлов, соответствующих ключевому слову перед курсором. Обратите внимание, что Vim завершает ключевое слово *именем файла*, а не найденным в файле словом.



В Vim 8.2 поиск файлов с возможными совпадениями имен файлов производится только в текущем каталоге. Это отличается от многих других функций Vim, которые используют параметр `path` для поиска файлов. Документация Vim указывает на то, что такое поведение временное и что `path` «пока» не используется. Однако это тянется уже более десяти лет...

Завершение по именам макросов и определений

Комбинация `Ctrl+X Ctrl+D` выполняет поиск по текущему файлу и включаемым файлам имен макросов и определений, указанных директивой `#define`.

Метод завершения с помощью команд Vim

Данный метод вызывается с помощью `Ctrl+X Ctrl+V` и предназначен для использования в командной строке Vim. Его задача — угадать наилучшее завершение для вводимого слова. Этот контекст поможет пользователям, разрабатывающим сценарии Vim.

Завершение с помощью пользовательских функций

Этот метод, вызываемый комбинацией `Ctrl+X Ctrl+U`, позволяет определить метод завершения с помощью вашей собственной функции. Vim использует для выполнения завершения функцию, на которую указывает параметр `completefunc`. Сценарии и написание функций Vim рассматриваются в главе 12.

Завершение по omni-функции

Рассматриваемый метод можно вызвать с помощью **Ctrl+X Ctrl+O**. Он использует определенные пользователем функции для завершения и от предыдущего отличается тем, что в нем функция зависит от типа файла и, следовательно, определяется и загружается по мере загрузки файла. Файлы завершения omni уже доступны для языков C, CSS, HTML, JavaScript, PHP, Python, Ruby, SQL и XML.

Завершение для исправления орфографии

Этот метод вызывается с помощью **Ctrl+X Ctrl+S**. Слово перед курсором используется как основное, и для него Vim предлагает варианты завершения. Если слово неправильно написано, Vim посоветует «более правильное» написание.

Завершение с помощью параметра complete

Это наиболее обобщенный параметр, вызываемый с помощью **Ctrl+N** и позволяющий вам сочетать все предыдущие методы завершения в одном. Для многих пользователей этот способ может быть наиболее удобным, поскольку он не требует глубокого понимания различных нюансов.

Чтобы настроить параметры этого метода завершения, вы должны через запятую задать список доступных источников в параметре **complete**. Любой доступный источник (обычно) обозначается одним символом. Варианты включают в себя:

- **.** (*точка*) — выполнить поиск в текущем буфере;
- **w** — выполнить поиск по буферам в других окнах (которые отображают текущую сессию Vim);
- **b** — выполнить поиск по другим загруженным буферам в списке буферов (которые могут не быть видимы ни в одном окне Vim);
- **u** — выполнить поиск по незагруженным буферам в списке буферов;
- **U** — выполнить поиск по буферам *не* из списка буферов;
- **k** — выполнить поиск по файлам словаря (перечисленным в параметре **dictionary**);
- **kspell** — использовать текущую схему проверки орфографии (это единственный параметр, не являющийся одиночным символом);
- **s** — выполнить поиск по файлам тезауруса (перечисленным в параметре **thesaurus**);

- **i** — выполнить поиск по текущим и включаемым файлам;
- **d** — выполнить поиск по текущим и включаемым файлам определенных макросов;
- **t,]** — выполнить поиск завершения тега.

Заключительные комментарии по поводу автозавершения Vim

Мы изучили разные способы автоматического завершения, но это не все. Время, потраченное на их изучение, окупается стократ, так как они значительно упрощают редактирование текста, особенно если вам нужно завершить определенный контекст или понятие.

Еще один момент: сочетание двух и более клавиш (особенно с **Ctrl**) может привести к ошибкам. Если вы собираетесь активно пользоваться автоматическим завершением, подумайте о том, чтобы переназначить его на одну клавишу или более простое сочетание. Так вы сможете быстрее и легче использовать любимые команды автоматического завершения.

Следующий пример показывает, как полезна эта настройка. Мы уже упоминали о переназначении клавиши **Tab** на общее сопоставление ключевых слов. Во время редактирования этой книги, используя теги DocBook XML (для седьмого издания), один из авторов набирал (при помощи консервативной команды **grep**) слово «выделение» более 1200 раз! С помощью завершения ключевых слов он мог ввести «выдел» и быть уверенным, что это соответствует нужному ему тегу «выделение». Так он сэкономил по три нажатия на каждое слово (если не ошибался в первых трех буквах), то есть как минимум 3600 нажатий!

А вот еще один способ оценки эффективности этого метода: один из авторов, Арнольд, печатает примерно четыре символа в секунду, так что экономия при вводе *только одного ключевого слова* составляет 3600, деленное на 4, — или *15 минут*. А еще он завершал от 20 до 30 ключевых слов тем же способом. Экономия налицо!

Стеки тегов

Стеки тегов обсуждались ранее в главе 7 в подразделе «Стеки тегов».

Вы можете не только переходить по тегам, по которым ищете, но и выбрать среди множества сопоставимых. Можно также выбирать теги и разделять окна

с помощью одной команды. Команды режима ex Vim для работы с тегами приведены в табл. 11.1.

Таблица 11.1. Команды тегов Vim

Команда	Функция
<code>ta[g][!]</code> [<i>строка_тега</i>]	Редактирование файла, содержащего <i>строка_тега</i> , как определено в файле тегов. Символ <code>!</code> заставляет Vim переключиться на новый файл, если текущий буфер был изменен, но не сохранен. Файл может как считываться, так и нет, в зависимости от настроек параметра <code>autowrite</code>
<code>[число]ta[g][!]</code>	Переход к новой записи в стеке тегов, соответствующей <i>числу</i>
<code>[число]po[p][!]</code>	Извлекает позицию курсора из стека, восстановив прежнее местоположение курсора. Если задано <i>число</i> , то перейти к соответствующей старой записи
<code>tags</code>	Отображает содержимое стека тегов
<code>ts[elect][!]</code> [<i>строка_тега</i>]	Выводит список тегов, соответствующих <i>строка_тега</i> , используя информацию в файле (-ах) тега. Если не задана <i>строка_тега</i> , используется последнее имя тега из стека тегов
<code>sts[elect][!]</code> [<i>строка_тега</i>]	Как <code>:tselect</code> , только разделяет окно для выбранных тегов
<code>[число]tn[ext][!]</code>	Переход к следующему сопоставимому тегу (по умолчанию — на первый), соответствующему <i>числу</i>
<code>[число]tp[revious][!]</code>	Перейти к предыдущему сопоставимому тегу
<code>[число]tN[ext][!]</code>	(по умолчанию — на первый), соответствующему <i>числу</i>
<code>[число]tr[ewind][!]</code>	Перейти к первому сопоставимому тегу. Если указано <i>число</i> , перейти к <i>число</i> -сопоставимому тегу
<code>tl[ast][!]</code>	Перейти к последнему сопоставимому тегу

Обычно Vim показывает, к какому сопоставимому тегу из общего их количества он перешел. Например:

```
tag 1 of >3
```

Знак «больше» (`>`) означает, что есть еще не проверенные теги. Вы можете использовать `:tnext` или `:tlast`, чтобы перейти к другим тегам. Если это сообще-

ние не отображается из-за какого-то другого сообщения, используйте `:ttn`, чтобы увидеть его.

Вот пример вывода команды `:tags`, где текущее местоположение обозначено знаком «большее» (`>`):

```
# TO tag      FROM line in file
1 1 main      1  harddisk2:text/vim/test
> 2 2 FuncA    58  -current-
3 1 FuncC     357 harddisk2:text/vim/src/amiga.c
```

Команда `:tselect` позволяет выбирать из нескольких подходящих тегов. Приоритет (поле `pri`) показывает, насколько хорошо тег соответствует запросу (глобальный или статический, с учетом или без учета регистра и т. д.); за более подробной информацией обратитесь к документации Vim:

```
nr pri kind tag      file ~
1 F  f  mch_delay     os_amiga.c
      mch_delay(msec, ignoreinput)
> 2 F  f  mch_delay     os_msdos.c
      mch_delay(msec, ignoreinput)
3 F  f  mch_delay     os_unix.c
      mch_delay(msec, ignoreinput)
Enter nr of choice (<CR> to abort):
```

Командам `:tag` и `:tselect` можно присвоить аргумент, начинающийся с `/`. В этом случае команда использует его как регулярное выражение, а Vim найдет все теги, сопоставимые с заданным регулярным выражением.

Например, команда `:tag /normal` найдет макрос `NORMAL`, функцию `normal_cmd` и т. д. Используйте команду `:tselect /normal` и введите номер нужного тега.

Команды командного режима Vim описаны в табл. 11.2. Вы также можете использовать мышь, если ее поддержка включена в вашей версии Vim.

Таблица 11.2. Команды работы с тегами в командном режиме Vim

Команда	Функция
<code>^]</code>	Ищет местоположение идентификатора под курсором в файле <i>тегов</i> и переходит в это место. Текущая позиция автоматически заносится в стек тегов
<code>g <LeftMouse></code> <code>Ctrl-<LeftMouse></code>	Возврат к предыдущему местоположению в стеке тегов, то есть извлечение одного элемента. Предыдущее число указывает, сколько элементов извлечь из стека
<code>^T</code>	

Параметры Vim, влияющие на поиск по тегам, описаны в табл. 11.3.

Табл. 11.3. Параметры Vim для управления тегами

Команда	Функция
taglength, t1	Задаёт количество значимых символов в теге, по которым будет производиться поиск. Значение по умолчанию — ноль — указывает, что все символы значимы
tags	Значение представляет собой список имен файлов, в которых нужно искать теги. В особом случае, если имя файла начинается с ./, точка заменяется на часть каталога пути текущего файла, что позволяет использовать файлы <i>тегов</i> в другой директории. Значение по умолчанию равно ./tags, tags
tagrelative	Когда значение равно true (по умолчанию) и используется файл <i>тегов</i> из другого каталога, имена файлов в файле <i>тегов</i> считаются относящимися к каталогу, в котором находится файл <i>тегов</i>

Vim может работать с файлами *etags* в стиле Emacs, но они нужны только для обратной совместимости. Данный формат не описан в документации Vim, а файлы *etags* использовать не рекомендуется.

Кроме того, Vim также ищет целое слово, содержащее курсор, а не только его часть от местоположения курсора.

Подсветка синтаксиса

Одно из важных преимуществ Vim перед vi — это подсветка синтаксиса. Она основана на использовании цвета, но также работает на монохромных экранах, хотя и не так эффективно. В этом разделе мы обсудим три темы: включение, настройку и создание собственного синтаксиса. Подсветка синтаксиса в Vim имеет много функций, изучение которых выходит за рамки книги, поэтому мы сконцентрируемся на основах.



Поскольку эффект подсветки синтаксиса Vim наиболее заметен в цвете, а книга черно-белая, мы настоятельно рекомендуем вам опробовать ее самостоятельно и увидеть, как цвет помогает распознавать контекст. Мы никогда не встречали пользователя, который отказался бы от нее после первого использования¹.

¹ Ну, за исключением одного из наших рецензентов!

Первоначальный запуск

Чтобы включить подсветку синтаксиса, достаточно выполнить команду:

```
:syntax enable
```

Теперь при редактировании файла с определенным синтаксисом, например, языка программирования вы увидите разноцветный текст в зависимости от контекста и синтаксиса. Если цвет не появился, попробуйте следующую команду:

```
:syntax on
```

КОГДА СИНТАКСИСА НЕДОСТАТОЧНО

Чтобы включить подсветку синтаксиса, обычно достаточно перечисленных выше команд. Но иногда это не работает по различным причинам.

Вероятно, Vim не знает тип вашего файла и поэтому не понимает, какой синтаксис подойдет.

Например, если вы создаете новый файл с неизвестным расширением или вообще без какого-либо расширения, Vim не сможет определить тип файла и выбрать подходящий синтаксис, так как файл новый и, следовательно, пустой. Мы часто сталкиваемся с этим при написании сценариев оболочки без расширения `.sh`. В этом случае подсветка появляется только после заполнения файла кодом.

Также возможно (хотя и маловероятно), что в Vim нет описания для вашего типа файла. Такое случается крайне редко и обычно решается явным указанием типа файла. К сожалению, создание синтаксиса с нуля является сложной задачей, но мы дадим вам несколько полезных советов чуть позже.

Вы можете принудительно задать конкретный стиль подсветки синтаксиса из командной строки `ex`. При запуске нового сценария оболочки мы всегда определяем синтаксис с помощью:

```
:set syntax=sh
```

В разделе «Динамическая конфигурация типа файла с помощью сценария» главы 12 мы покажем вам удобный способ избежать этого шага.

Когда вы включаете подсветку синтаксиса, Vim ищет соответствующий файл определения синтаксиса в каталоге `$VIMRUNTIME/syntax`. Это делается на основе типа вашего файла.

В каталоге файлов синтаксиса Vim содержится *почти 500 файлов синтаксиса* для различных языков (C, Java, HTML), форматов (календарь) и конфигурационных файлов (`fstab`, `xinetd`, `crontab`). Если Vim не может распознать ваш тип файла, попробуйте поискать подходящий в каталоге `$VIMRUNTIME/syntax`.

Настройка

Как только вы начали использовать подсветку синтаксиса, вы можете обнаружить, что некоторые цвета вам не подходят. Возможно, их сложно увидеть или они просто вам не нравятся. В Vim есть несколько способов изменения и настройки цветов.

К двум наиболее распространенным и существенным признакам того, что подсветка синтаксиса вышла из-под контроля, относятся:

- слабый контраст, когда цвета слишком похожи и плохо различимы между собой;
- слишком большое разнообразие цветов, что придает тексту контрастный вид.

Не спешите предпринимать решительные действия (например, написать свое собственное описание синтаксиса), чтобы подсветка синтаксиса работала именно для вас. Попробуйте сначала изменить и настроить цвета с помощью следующих двух команд и одного параметра соответственно: `colorscheme`, `highlight` и `background` — они помогут вам привести цвета к приемлемому балансу.

Кроме того, существуют и другие команды и параметры, с помощью которых можно настроить подсветку синтаксиса. После краткого знакомства с *группами синтаксиса* мы рассмотрим эти команды и параметры в следующих разделах, сделав акцент на трех, только что упомянутых.

Группы синтаксиса

Vim классифицирует различные типы текста по группам. Этим группам присваиваются определения цвета и подсветки. Кроме того, Vim позволяет создавать группы. Определения можно задавать на разных уровнях. Если вы присваиваете определение группе, содержащей подгруппы, каждая подгруппа унаследует определения родительской группы, если иное не определено.

Некоторые высокоуровневые группы для подсветки синтаксиса включают:

- *комментарии*, специфичные для языка программирования, например:

```
// Это комментарий для C++ и JavaScript
```
- *константы* — любая постоянная величина (значение), например `TRUE`;
- *идентификатор* — имена переменных и функций;
- *тип* — определения, такие как `int` и `struct` на языке C;
- *специальные символы*, такие как разделители.

Взяв *специальную* группу из предыдущего списка, рассмотрим пример подгрупп:

- специальный символ;
- тег;
- разделитель;
- специальный комментарий;
- отладка.

Имея базовое представление о подсветке синтаксиса, группах и подгруппах, мы можем изменять подсветку синтаксиса под наши нужды.

Команда `colorscheme`

Эта команда меняет цвета для различных синтаксических конструкций, таких как комментарии, ключевые слова или строки, переопределяя эти группы синтаксиса. Vim поставляется со следующими вариантами цветовых схем¹:

```
blue      delek  evening murphy    ron  torte
darkblue  desert koehler pablo    shine zellner
default  elflord morning peachpuff slate
```

Эти файлы находятся в каталоге `$VIMRUNTIME/colors`. Вы можете активировать любой из них с помощью команды:

```
:colorscheme имя_схемы
```

Чтобы просмотреть различные схемы в Vim или `gvim`, введите неполную команду `:color`, нажмите клавишу `Tab` для включения автозавершения команды, клавишу пробела, а затем продолжайте нажимать `Tab`, чтобы увидеть все варианты.

В `gvim` все еще проще: зайдите в меню `Edit`, наведите указатель мыши на подменю `Colorscheme` и «оторвите» меню. Теперь вы можете просматривать каждую схему, нажимая на соответствующую кнопку.

Большинство доступных цветовых схем созданы сообществом пользователей Vim. Возможно, вас заинтересует репозиторий GitHub (<https://github.com/flazz/vim-colorschemes/tree/master/colors>), в котором собрано около тысячи цветовых схем.

¹ Мы заметили, что у некоторых экземпляров класса Vim могут быть немного разные наборы цветовых схем по умолчанию.

Установка параметра `background`

При назначении цвета Vim сначала пытается определить цвет фона вашего экрана. В Vim есть лишь две категории фона: темный и светлый. Он устанавливает цвета по-разному для каждого из фонов. Хотя Vim делает все возможное, чтобы настроить цвета правильно, сложно объективно оценить результат его работы: кому-то установленные цвета покажутся удобными для просмотра, а кому-то — нет.

Поэтому, если цвет вам не нравится, попробуйте напрямую настроить параметр `background`. Введите:

```
:set background?
```

чтобы знать, что вы его изменяете. Затем запустите такую команду:

```
:set background=dark
```

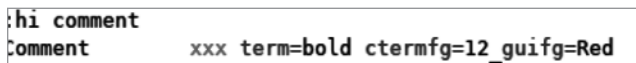
Используйте опцию `background` вместе с командой `colorscheme` для точной настройки цветов вашего экрана. Вместе они, как правило, обеспечивают комфортную цветовую палитру.

Команда `highlight`

Команда Vim `highlight` позволяет вам управлять различными группами и их подсветкой во время сеанса редактирования. Настройки для групп можно проверять либо в виде списка, либо запросив информацию о подсветке конкретной группы. Например, команда:

```
:highlight comment
```

возвращает результат, изображенный на рис. 11.22. В первом поле перечислены имена подсветок, в данном примере это `Comment` (Комментарий). Во втором поле всегда отображается строка `xxx` в том виде, в каком она выглядит согласно определениям подсветок в терминале или GUI.



```
:hi comment
comment      xxx term=bold ctermfg=12 guifg=Red
```

Рис. 11.22. Подсветка комментария (WSL Ubuntu Linux, цветовая схема: `zellner`)

Выходные данные показывают вид комментариев в файле. Символы `xxx` на распечатанной странице будут выглядеть темно-серыми, хотя на экране они красные¹. Вывод `term=bold` означает, что в черно-белом терминале комментарии будут

¹ Рисунки 11.22–11.27 в цвете можно посмотреть по ссылке <https://oreil.ly/LhSuQ>.

выделены жирным шрифтом. Выражение `ctermfg=12` означает, что в цветном терминале, таком как `xterm`, цвет текста для комментария будет соответствовать синему цвету DOS. И наконец, `guifg=Red` означает, что GUI будет отображать комментарии красным цветом.



Цветовой охват DOS более скудный по сравнению с современными наборами GUI. По факту он отображает лишь восемь цветов: black, red, green, yellow, blue, magenta, cyan и white. Каждый из них можно установить в качестве подсветки текста или фона для него, а также определить самое светлое место на экране как «яркое». Vim использует аналогичные сопоставления для определения цветов текста в окнах без графического интерфейса, например `xterms`.

Графические окна предоставляют практически неограниченный набор цветов. Vim позволяет определять некоторые основные цвета, такие как blue, но также их можно задать в формате `#rrggbb`, где `#` — обязательный символ решетки, а `rr`, `gg` и `bb` — шестнадцатеричные цифры, указывающие насыщенность основных цветов (красного, зеленого и синего соответственно). Например, красный цвет имеет значение `#ff0000`.

Используйте команду `highlight` для изменения настроек для групп, чьи цвета вам не нравятся. Например, набрав команду:

```
:highlight identifier
```

мы обнаружим, что идентификаторы для нашего GUI в этом файле темно-голубые (см. вывод на рис. 11.23).

```
Constant      xxx term=underline ctermfg=4 guifg=Magenta
Special       xxx term=bold ctermfg=5 guifg=SlateBlue
Identifier    xxx term=underline ctermfg=3 guifg=DarkCyan
Statement     xxx term=bold ctermfg=6 guifg=Brown
PreProc       xxx term=underline ctermfg=5 guifg=Purple
Type          xxx term=underline ctermfg=2 guifg=SeaGreen
Underlined    xxx term=underline ctermfg=5 guifg=SlateBlue
Ignore        ctermfg=15 guifg=bg
Error         xxx term=reverse ctermfg=15 ctermbg=12 guifg=White guibg=Red
Fold          xxx term=standout ctermfg=8 ctermbg=14 guifg=Blue guibg=Yellow
String        xxx links to Constant
Character     xxx links to Constant
Number        xxx links to Constant
Boolean       xxx links to Constant
Float         xxx links to Number
Function      xxx links to Identifier
```

Рис. 11.23. Подсветка для идентификаторов

Чтобы переопределить цвет идентификаторов, используйте команду `highlight`. Например, следующая настройка сделает их красными (да, результат выглядит не очень красиво):

```
:highlight identifiers guifg=red
```

Этот способ довольно негибкий: он применяется ко всем типам файлов и не подстраивается под различные фоны или цветовые схемы.

Чтобы увидеть все определения подсветки и их значения, снова введите команду `highlight`:

```
:highlight
```

Вывод этой команды показан на рис. 11.24.

Identifier	xxx	term=underline	ctermfg=9	guifg=red		
Statement	xxx	term=bold	ctermfg=4	guifg=Brown		
PreProc	xxx	term=underline	ctermfg=13	guifg=Purple		
Type	xxx	term=underline	ctermfg=9	guifg=Blue		
Underlined	xxx	term=underline	cterm=underline	ctermfg=5	gui=underline	guifg=SlateBlue

Рис. 11.24. Часть результатов команды `:highlight` (WSL Ubuntu Linux, цветовая схема: `zellner`)

Обратите внимание, что некоторые строки содержат полные определения (перечень `term`, `ctermfg` и т. д.), в то время как другие получают свои атрибуты от родительских групп (например, `String` ссылается обратно на `Constant`).

Замещение файлов синтаксиса

Чтобы изменить группу синтаксиса для конкретных определений синтаксиса, а не для всего класса группы, как мы узнали ранее, можно использовать каталог `after`. Здесь вы можете создать любое количество файлов синтаксиса, которые Vim будет обрабатывать *после* файла обычного синтаксиса.

Чтобы добавить его, просто включите команды подсветки (или любые команды обработки — это общий механизм) в специальном файле в каталоге под названием `after`, который содержится в параметре `runtimepath`. Теперь, когда Vim начнет устанавливать правила подсветки для вашего типа файла, он также выполнит ваши обычные команды из файла `after`.

Например, давайте изменим подсветку для файлов XML с синтаксисом `xml`. Это означает, что Vim загрузил определения синтаксиса из файла с именем `xml.vim`. Как и в предыдущем примере, мы хотим, чтобы идентификаторы всегда были красными. Поэтому мы создадим в каталоге `~/.vim/after/syntax` наш собственный файл с именем `xml.vim` и поместим в него следующую строку:

```
highlight identifier ctermfg=red guifg=red
```

Перед запуском этой настройки необходимо убедиться, что `~/.vim/after/syntax` есть в пути `runtimepath`:

```
:set runtimepath+=~/.vim/after/syntax
```

В нашем файле `.vimrc`

Для сохранения изменения данная строка должна быть добавлена в наш файл `.vimrc`.

Теперь каждый раз, когда Vim загружает определения синтаксиса для `xml`, он помещает определения для `identifier` нашими собственными настройками.

Создание своего собственного файла синтаксиса

Благодаря пройденному материалу мы знаем достаточно, чтобы попробовать написать свои собственные файлы синтаксиса, хотя бы простые. Однако еще остается множество деталей, которые необходимо учесть, прежде чем разработать полноценный файл синтаксиса.

Мы будем делать это шаг за шагом. А чтобы не усложнять задачу, рассмотрим нечто достаточно простое, но интересное для демонстрации возможностей.

Возьмем сгенерированный отрывок на латыни из файла `loremipsum.latin`:

```
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin eget  
tellus. Suspendisse ac magna at elit pulvinar aliquam. Pellentesque  
iaculis augue sit amet massa. Aliquam erat volutpat. Donec et dui at  
massa aliquet molestie. Ut vel augue id tellus hendrerit porta. Quisque  
condimentum tempor arcu. Aenean pretium suscipit felis. Curabitur semper  
eleifend lectus. Praesent vitae sapien. Ut ornare tempus mauris. Quisque  
ornare sapien congue tortor.
```

```
In dui. Nam adipiscing ligula at lorem. Vestibulum gravida ipsum iaculis  
justo. Integer a ipsum ac est cursus gravida. Etiam eu turpis. Nam laoreet  
ligula mollis diam. In aliquam semper nisi. Nunc tristique tellus eu  
erat. Ut purus. Nulla venenatis pede ac erat.
```

...

Для создания нового файла синтаксиса необходимо создать файл с таким же именем синтаксиса, в данном случае — `latin`. Это будет файл `latin.vim` в нашем личном исполняемом каталоге `$HOME/.vim/syntax`. Затем нужно определить синтаксис, добавив несколько ключевых слов с помощью команды `syntax keyword`. Выбрав в качестве ключевых слов `lorem`, `dolor`, `nulla` и `lectus`, иницилируйте файл синтаксиса с помощью строки:

```
syntax keyword identifier lorem dolor nulla lectus
```

Однако при редактировании `loremipsum.latin` подсветка синтаксиса может не появиться автоматически. Для активации используйте команду:

```
:set syntax=latin
```

Текст теперь должен выглядеть примерно так, как показано на рис. 11.25.

```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do
eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim
ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut
aliquip ex ea commodo consequat. Duis aute irure dolor in
reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla
pariatur. Excepteur sint occaecat cupidatat non proident, sunt in
culpa qui officia deserunt mollit anim id est laborum.
~

```

Рис. 11.25. Латинский файл с определенными ключевыми словами

На экране монитора текст черный, а ключевые слова — красные. В печатной версии книги они практически неразличимы: ключевые слова кажутся черными, хотя на самом деле — темно-серые.

Вы могли заметить, что первое вхождение *Lorem* не подсвечено. Это потому, что ключевые слова синтаксиса чувствительны к регистру. Добавьте следующую строку в верхнюю часть файла синтаксиса:

```
:syntax case ignore
```

и вы увидите, что *Lorem* тоже будет подсвечено как ключевое слово.

Чтобы все работало автоматически, нам нужно создать каталог `ftdetect` в `$HOME/.vim` и поместить в него файл `latin.vim`, содержащий единственную строку:

```
au BufRead,BufNewFile *.latin set filetype=latin
```

Эта строка сообщает Vim, что все файлы с именами, оканчивающимися `.latin`, являются файлами типа `latin` и, следовательно, Vim должен применять файл синтаксиса из `$HOME/.vim/syntax/latin.vim` при их отображении.

Теперь при редактировании `loremipsum.latin` вы увидите нечто похожее на рис. 11.26.

```

1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin eget
2 tellus. Suspendisse ac magna at elit pulvinar aliquam. Pellentesque
3 iaculis augue sit amet massa. Aliquam erat volutpat. Donec et dui at
4 massa aliquet molestie. Ut vel augue id tellus hendrerit porta. Quisque
5 condimentum tempor arcu. Aenean pretium suscipit felis. Curabitur semper
6 eleifend lectus. Praesent vitae sapien. Ut ornare tempus mauris. Quisque
7 ornare sapien congue tortor.
8
9 In dui. Nam adipiscing ligula at lorem. Vestibulum gravida ipsum iaculis
10 justo. Integer a ipsum ac est cursus gravida. Etiam eu turpis. Nam laoreet
11 ligula mollis diam. In aliquam semper nisi. Nunc tristique tellus eu
12 erat. Ut purus. Nulla venenatis pede ac erat.
13

```

Рис. 11.26. Файл Latin с идентифицированными ключевыми словами, игнорирующими регистр (WSL Ubuntu Linux, цветовая схема: zellner)

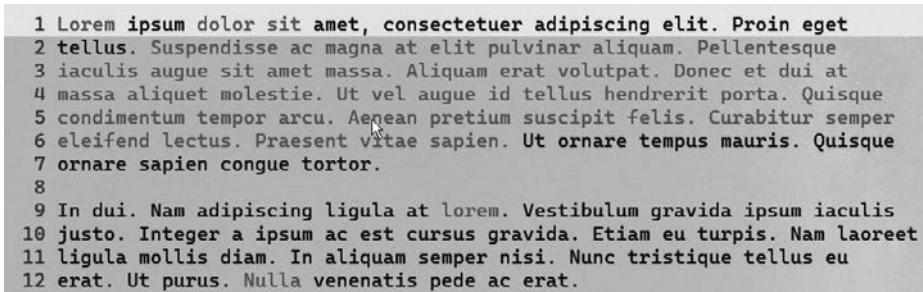
Для начала обратите внимание, что синтаксис был сразу активирован, как только Vim распознал ваш новый тип файла синтаксиса — *latin*. Ключевые слова теперь также сопоставляются без учета регистра.

Чтобы расширить возможности, можно определить *match*, использующий регулярное выражение для описания того, какой текст будет подсвечен, и присвоить его группе *Comment*. Например, мы можем обозначить все слова, начинающиеся с *s* и заканчивающиеся на *t*, как синтаксис *Comment* (помните, это просто пример!). Наше регулярное выражение — `\<s[^\t ]*t\>`. Мы также зададим область и подсветим ее как *Number*. Области переопределяются с помощью регулярных выражений *start* и *end*.

Наша область начинается с *Suspendisse* и заканчивается на *sapien\.* Чтобы добавить еще больше разнообразия, мы *включим* ключевое слово *lectus* в нашу область. В результате файл синтаксиса *latin.vim* будет выглядеть следующим образом:

```
syntax case ignore
syntax keyword identifier lorem dolor nulla lectus
syntax keyword identifier lectus contained
syntax match comment /\<s[^\t ]*t\>/
syntax region number start=/Suspendisse/ end=/sapien\./ contains=identifier
```

Теперь при редактировании *loremipsum.latin* мы увидим нечто похожее на рис. 11.27.



```
1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin eget
2 tellus. Suspendisse ac magna at elit pulvinar aliquam. Pellentesque
3 iaculis augue sit amet massa. Aliquam erat volutpat. Donec et dui at
4 massa aliquet molestie. Ut vel augue id tellus hendrerit porta. Quisque
5 condimentum tempor arcu. Aenean pretium suscipit felis. Curabitur semper
6 eleifend lectus. Praesent vitae sapien. Ut ornare tempus mauris. Quisque
7 ornare sapien congue tortor.
8
9 In dui. Nam adipiscing ligula at lorem. Vestibulum gravida ipsum iaculis
10 justo. Integer a ipsum ac est cursus gravida. Etiam eu turpis. Nam laoreet
11 ligula mollis diam. In aliquam semper nisi. Nunc tristique tellus eu
12 erat. Ut purus. Nulla venenatis pede ac erat.
```

Рис. 11.27. Новая подсветка синтаксиса *latin*
(WSL Ubuntu Linux, цветовая схема: *zellner*)

Если вы сами выполнили данный пример и увидели результат в цвете, то наверняка обратили внимание на некоторые моменты.

- Появились новые сопоставления подсветки: в первой строке *dolor sit* подсвечено красным, потому что оно соответствует регулярному выражению для *match*.
- Появились новые области подсветки: весь раздел от *Suspendisse* до *sapien.* подсвечен фиолетовым цветом (как говорится, без комментариев).

- Ключевые слова подсвечиваются, как и раньше.
- Внутри области подсветки ключевое слово *lectus* все еще подсвечивается красным, поэтому мы определили группу `identifier` как `contained`, а нашу область — как `contains identifier`.

Данный пример — всего лишь демонстрация силы функции подсветки синтаксиса, и мы надеемся, что он вдохновит вас на создание своих собственных определений синтаксиса.

Компиляция и проверка ошибок в Vim

Vim не является интегрированной средой разработки (IDE), но тем не менее немного упрощает жизнь программистов, позволяя во время сеанса редактирования компилировать и искать и исправлять ошибки.

Кроме того, Vim предлагает несколько удобных функций для отслеживания и перемещения по позициям в файле. Рассмотрим простой пример: цикл «правка-компиляция-правка» с использованием встроенных функций Vim и некоторых связанных с ним команд и параметров, а также функций, упрощающих работу. Все это зависит от одного и того же окна Vim — `Quickfix List`.

Vim позволяет вам компилировать файлы с помощью `make` при каждом изменении файла. Редактор использует поведение по умолчанию для управления результатами вашей сборки, чтобы вы могли с легкостью чередовать редактирование и компиляцию. Ошибки компиляции отображаются в специальном окне `Vim Quickfix List`, где можно просмотреть их, перейти к ним и исправить.

Рассмотрим небольшую программу на языке C, генерирующую числа Фибоначчи¹. Правильная и компилируемая форма кода выглядит следующим образом:

```
# include <stdio.h>
# include <stdlib.h>

int main(int argc, char *argv[])
{
    /*
     * arg 1: starting value
     * arg 2: second value
     * arg 3: number of entries to print
     */
}
```

¹ Файл доступен в репозитории книги GitHub (<https://www.github.com/learning-vi/vi-files>); см. приложение В.


```

if (argc - 1 != 3)
{
    printf ("Three command line args: (you used %d)\n", argc-1);
    printf ("usage: value 1, value 2, number of entries\n");
    return (1);
}

/* count = how many to print */
int count = atoi(argv[3]);

/* index = which to print */
long int index;

/* first and second passed in on command line */
long int first, second;

/* these get calculated */
long int current, nMinusOne, nMinusTwo;

first = atoi(argv[1]);
second = atoi(argv[2]);
printf("%i fibonacci numbers with starting values: %li, %li\n", count, first,
      second);
printf("=====\n");

/* print the first 2 from the starter values */
printf("%i %04li\n", 1, first);
printf("%i %04li ratio (golden?) %.3f\n", 2, second, (double) second/first);

nMinusTwo = first;
nMinusOne = second;

for (index=1; index<=count; index++)
{
    current = nMinusTwo + nMinusOne;
    printf("%li %04li ratio (golden?) %.3f\n",
          index,
          current,
          (double) current/nMinusOne);
    nMinusTwo = nMinusOne;
    nMinusOne = current;
}
}

```

Скомпилируйте эту программу из Vim (если имя файла — `fibonacci.c`) с помощью команды:

```
:make fibonacci
```

По умолчанию Vim передает команду `make` во внешнюю оболочку и вводит результаты в специальное окно **Quickfix List**. После компиляции предыдущего кода экран с окном **Quickfix List** выглядит примерно как на рис. 11.28.

```
53      (double) current/nMinusOne);  
54      nMinusTwo = nMinusOne;  
55      nMinusOne = current;  
56  }  
57 }
```

```
fibonacci.c Wed Jun 23 16:04:49 2021  
1 || cc fibonacci.c -o fibonacci
```

```
[Quickfix List][-] Wed Jun 23 16:04:49 2021
```

Рис. 11.28. Окно Quickfix List после чистой компиляции (WSL Ubuntu Linux, цветовая схема: zellner)



Если вы не видите окно QuickFix (оно не открылось автоматически), откройте его с помощью команды `ex` в Vim `:copen`.

Далее мы изменим некоторые строки в нашей программе, чтобы получилось некритическое количество ошибок.

Исправьте:

```
long int current, nMinusOne, nMinusTwo;
```

на следующее ошибочное описание:

```
longish int current, nMinusOne, nMinusTwo;
```

а в строках:

```
nMinusTwo = first;
nMinusOne = second;
```

переназначьте переменные на `xfirst` и `xsecond`:

```
nMinusTwo = xfirst;
nMinusOne = xsecond;
```

Замените:

```
printf("%d %04li ratio (golden?) %.3f\n", 2, second, (double) second/first);
```

на это:

```
printf("%d %04li ratio (golden?) %.3f\n", 2 second (double) second/first);
```

Теперь перекомпилируйте программу. На рис. 11.29 показано текущее содержимое окна `Quickfix List`.

```
47 for (index=1; index<=count; index++)
48 {
49     current = nMinusTwo + nMinusOne;
fibonacci.c Wed Jun 23 16:11:21 2021
1 || cc fibonacci.c -o fibonacci
2 || fibonacci.c: In function 'main':
3 fibonacci.c|32 col 3| error: 'longish' undeclared (first use in this function); did you mean 'long int'?
4 ||     longish int current, nMinusOne, nMinusTwo;
5 ||     ^~~~~~
6 ||     long int
7 fibonacci.c|32 col 3| note: each undeclared identifier is reported only once for each function it appears in
8 fibonacci.c|32 col 11| error: expected ';,' before 'int'
9 ||     longish int current, nMinusOne, nMinusTwo;
10 ||     ^~~~
[Quickfix List] [-] Wed Jun 23 16:11:21 2021
```

Рис. 11.29. Окно `Quickfix List` после рекомпиляции с ошибками (WSL Ubuntu Linux, цветовая схема: `zellner`)

Строка 1 окна `Quickfix List` показывает выполненную команду компиляции. Если бы не было ошибок, это была бы единственная строка в окне. Но поскольку они есть, то строка 3 начинается со списка ошибок и их контекста.

Vim перечисляет все ошибки в окне `Quickfix List` и позволяет вам получить доступ к коду, в котором они указаны несколькими способами. Редактор подсвечивает первую ошибку в окне `Quickfix List`. Затем он меняет позицию в исходном файле (прокручивая при необходимости) и помещает курсор в начало строки исходного кода, соответствующей ошибке.

Чтобы перейти к следующей ошибке, либо введите команду `:cnext`, либо поместите курсор на строку с ошибкой в окне `Quickfix List` и нажмите `Enter`. Vim переместит курсор в начало «неисправной» строки исходного кода, прокрутив при необходимости файл.

После внесения изменений и исправления ошибок можно снова начать цикл «компиляция — правка». Если у вас стандартная среда разработки (что обычно всегда

верно для устройств на Unix/Linux), то Vim по умолчанию поддерживает цикл «правка — компиляция — правка» без каких-либо настроек.

Если Vim не находит подходящую программу компиляции, укажите расположение нужных утилит через его параметры, после чего все должно заработать. Более подробное описание сред программирования и компиляторов выходит за рамки книги, однако, если вы хотите поэкспериментировать со своим окружением, обратите внимание на следующие опции.

- `:cnext`, `:cprevious` — перемещают курсор к *следующей* и *предыдущей* ошибке соответственно, указанным в окне Quickfix List.
- `:colder`, `:cnewer` — загружают следующий *более старый* или следующий *более новый* список ошибок в окно Quickfix List (Vim запоминает последние десять списков ошибок). Каждая команда принимает необязательное целое число *n* для загрузки *n*-го старого или нового списка ошибок.
- `errorformat` — определяет формат, соответствующий Vim для поиска ошибок, возвращаемых при компиляции. За более подробной информацией об этом обратитесь к встроенной документации Vim, например:
`:help errorformat`
- `makeprg` — параметр, содержащий имя команды `make` или программы `compile` среды разработки.

Дополнительные варианты использования окна Quickfix List

Vim предоставляет возможность создавать свой список позиций в файлах, используя `grep`-подобный синтаксис. Результаты отображаются в окне Quickfix List в формате, очень похожем на строки, возвращенные после описанного ранее процесса компиляции. Введите команду `:vimgrep` со следующим синтаксисом:

```
:vimgrep[!] /шаблон/[g][j] файл(ы)
```

Это, по сути, аналог стандартной утилиты `grep`, которая выполняет поиск в *файлах* строк, сопоставимых с *шаблоном*, и помещает результаты в окно QuickFix (см. документацию Vim для получения подробностей о флагах и их значениях).

Эта функция полезна для таких задач, как рефакторинг. В качестве примера мы сформировали статью в AsciiDoc. В процессе написания мы решили заменить обозначение для любого вхождения `++vim++` с `++vim++` на `__vim__`¹. То есть после выполнения команды:

```
:vimgrep /++vim++/ *.asciidoc
```

¹ Позднее оно еще немного изменилось.

все строки вида:

```
++vim++
```

должны были стать:

```
__vim__
```

а результат — отобразиться в окне Quickfix List (рис. 11.30).

```

1 appd.asciidoc|51 col 26| If the command ++vi++ or ++vim++ doesn't start your editor it is either not
2 appd.asciidoc|208 col 43| When done, you'll have an executable name ++vim++. To install
3 ch01.asciidoc|317 col 1| ++vim++ is the Unix command that invokes the Vim editor for an existing
4 ch01.asciidoc|318 col 50| file or for a brand new file. The syntax for the ++vim++ command is:
5 ch02.asciidoc|1562 col 36| |Start Vim, open file if specified|++vim++ __+file++__
6 ch04.asciidoc|98 col 32| There are other options to the ++vim++ command that can be helpful. You
7 ch04.asciidoc|142 col 10| Give the ++vim++ command with the option ++$-c /$$_++pattern++__
8 ch08.asciidoc|726 col 1| ++vim++::
9 ch08.asciidoc|952 col 1| ++VIM++::
10 ch08.asciidoc|965 col 54| If more than one version of Vim exists on a machine, ++VIM++ will
11 ch08.asciidoc|968 col 10| sets the ++VIM++ environment variable to __$$/$$usr$$/$$share$$/$$vim__,

```

Рис. 11.30. Окно Quickfix List после команды :vimgrep (WSL Ubuntu Linux, цветовая схема: zellner)



Не забудьте открыть окно QuickFix с помощью команды Vim ex:

```
:copen
```

Иначе вы не увидите результаты.

На рис. 11.30 видно, как Vim отображает результаты :vimgrep. Слева находятся номера строк буфера QuickFix; их можно скрыть командой Vim ex:

```
:set nonumber
```

Вывод :vimgrep содержит три поля, которые разделены вертикальной чертой (|). Первое поле — это имя файла, где vimgrep нашел совпадение с шаблоном. Второе поле — номер строки и столбца, где находится шаблон. А третье поле представляет собой фактический текст с шаблоном.

Вы можете перейти к любому совпадению, переместив курсор на интересующую вас строку или дважды щелкнув на ней кнопкой мыши. Vim откроет этот файл в другом (разделенном) окне и установит курсор на первый символ шаблона.

На рис. 11.30 подсвеченная строка (которую немного сложно читать из-за неудачной цветовой схемы) является указателем на строку 98, столбец 32 в файле ch04.asciidoc. А на рис. 11.31 видны результаты после двойного щелчка кнопкой мыши по строке (Vim расположил курсор в правильное место в файле ch04.asciidoc в соответствующем окне).

```

98 There are other options to the ++vim++ command that can be helpful. You
99 can open a file directly to a specific line number or pattern. You can
100 also open a file in read-only mode. Another option recovers all changes
101 to a file that you were editing when the system crashed.
102
103 The options described in the following section apply both to ++vi++
104 and to Vim.
105
106 [[vi8-ch-4-sect-2.1]]
107 ==== Advancing to a Specific Place
108
109 When you begin editing an existing file, you can call the file in and
110 then move to the first occurrence of a __pattern__ or to a specific line
ch04.asciidoc Mon Aug 30 14:53:30 2021

```

Рис. 11.31. Vim помещает курсор в файле, строке и столбце из окна QuickFix vimgrep (WSL Ubuntu Linux, цветовая схема: zellner)

Итак, было просто перемещаться по всем вхождениям и быстро изменять их на новые значения.



Может показаться, что данный пример демонстрирует задачу, которую проще решить с помощью команды:

```
:%s/++vim++/_vim_/g
```

Однако помните, что команда vimgrep более общая и способна работать с несколькими файлами. Мы показали работу vimgrep, а не единственный способ выполнения этой задачи. Как правило, в Vim существует множество путей что-либо сделать.

Пара заключительных слов о роли Vim для написания программ

В этой главе мы изучили множество мощных функций Vim. Чтобы овладеть ими, нам пришлось потратить немало времени и усилий, но результат того стоил. Если вы уже работали с vi, то вы преодолели первый этап обучения. Чтобы освоить все преимущества Vim, необходимо пройти второй этап и познакомиться с дополнительными функциями.

Если вы программист, то мы надеемся, что эта глава показала вам, насколько много Vim может предложить вам. Мы рекомендуем попробовать некоторые из этих функций и даже настроить Vim под свои нужды. А может быть, вы создадите расширения для Vim и поделитесь ими с другими пользователями. А теперь — к программированию!

Сценарии Vim

Иногда вам нужно больше, чем просто настройка вашего редактора. Vim позволяет сохранять все ваши любимые настройки в файле `.vimrc`, но, возможно, вам требуется более гибкая или «актуальная» конфигурация. С помощью сценариев Vim это можно сделать.

Начиная с проверки содержимого буфера и заканчивая обработкой непредвиденных внешних факторов, язык сценариев Vim позволяет выполнять сложные задачи и принимать решения на основе *ваших* нужд.

Если у вас есть конфигурационный файл Vim (`.vimrc`, `.gvimrc` или оба), то вы уже используете сценарии в Vim, просто не осознаете этого. Все команды и параметры Vim являются частью сценарного языка. Кроме того, Vim предоставляет все стандартные операторы управления потоком (`if...then...else`, `while` и т. д.), переменные и функции, как и любой другой язык.

В этой главе мы разберем пример и шаг за шагом создадим сценарий. Мы изучим простые конструкции, используем некоторые встроенные функции Vim и узнаем правила для написания работоспособных и надежных сценариев Vim.

Какой ваш любимый цвет (цветовая схема)?

Начнем с самого простого. Мы выберем цветовую схему, которая *нам* нравится. Это легко и требует всего одной команды Vim.

Vim имеет 17 настраиваемых цветовых схем¹. Вы можете установить любую из них, прописав команду `colorscheme` в вашем файле `.vimrc` или `.gvimrc`. Одна из наших любимых цветовых схем — «пустынная»:

```
:colorscheme desert
```

¹ Мы слышали, что называют немного другое количество и виды стандартных цветовых схем, но это довольно близко к истине.

Если вы добавите эту команду в свой файл конфигурации, то каждый раз при запуске Vim вы будете видеть свои любимые цвета.

Итак, наш первый сценарий банален. А что, если ваши цветовые предпочтения более сложны? Или вам нравится несколько схем? А если ваши вкусы зависят от времени суток? С помощью сценариев Vim мы можем это реализовать.



Выбор альтернативной цветовой схемы, зависящей от времени суток, может показаться банальным, но, возможно, не настолько, как вы думаете. Даже Google меняет цвета и тон вашей домашней страницы iGoogle в течение дня.

Условное выполнение

Один из авторов книги любит делить сутки на четыре части, каждой из которых посвящена своя цветовая схема:

- `darkblue` — с полуночи до 6 утра;
- `morning` — с 6 утра до полудня;
- `shine` — с полудня до 6 вечера;
- `evening` — с 6 вечера до полуночи.

Для этой цели мы создадим вложенный блок кода `if...then...else....`. Существует несколько различных синтаксисов, которые можно использовать для этого блока. Один из них более традиционный, с четко прописанным синтаксисом:

```
if услов выраж
    строка кода vim
    еще одна строка кода vim
    ...
elseif некоторое вторичное услов выраж
    код для этого условия
else
    выполняемый код, если ни одно из условий не выполняется
endif
```

Блоки `elseif` и `else` являются необязательными, и можно включить несколько блоков `elseif`. Vim также допускает более краткую C-подобную конструкцию:

```
услов ? выраж1 : выраж2
```

Vim проверяет условие *услов*. Если оно истинно, выполняется *выраж1*, иначе — *выраж2*.

Использование функции `strftime()`

Теперь нам нужно выяснить, какое «сейчас» время суток. Для этого мы можем воспользоваться встроенной *функцией* Vim `strftime()`. Она принимает два аргумента: первый — это формат строки времени, а второй — время в секундах с 1 января 1970 года (стандартное представление времени на языке C; по умолчанию показывает текущее время). Формат строки времени зависит от системы и может быть разным, поэтому при выборе необходимо проявить должную осторожность. К счастью, большинство широко распространенных форматов поддерживаются всеми системами. Для нашего примера достаточно знать, который час, поэтому мы можем использовать формат времени `%H`, создавая `strftime(%H)`.

Теперь мы знаем, как писать условный код и получать информацию о времени суток с помощью функции Vim. Поместите этот код в ваш файл `.vimrc` для выбора цветовой схемы в зависимости от времени:

```
" progressively check higher values... falls out on first "true"
if strftime("%H") < 6
    colorscheme darkblue
    echo "setting colorscheme to darkblue"
elseif strftime("%H") < 12
    colorscheme morning
    echo "setting colorscheme to morning"
elseif strftime("%H") < 18
    colorscheme shine
    echo "setting colorscheme to shine"
else
    colorscheme evening
    echo "setting colorscheme to evening"
endif
```

Обратите внимание, что мы ввели еще одну команду сценария Vim — `echo`. Она позволяет нам отобразить текущую схему для себя и проверить, что код действительно был выполнен и привел к желаемому результату. Сообщение должно появиться в окне состояния команды Vim или в виде всплывающего окна, в зависимости от того, где в последовательности запуска встречается команда `echo`.



Когда мы запускаем команду `colorscheme` с именем схемы (например, `desert`), мы не используем кавычки, но при команде `echo` с тем же именем ("`desert`") мы заключаем его в кавычки. Это важное различие!

Команда `colorscheme` — это прямая команда Vim, и ее параметр является буквальным. Если мы добавим кавычки, они будут интерпретироваться как часть имени цветовой схемы, что приведет к ошибке, потому что ни одна из схем не включает в свое имя кавычки.

С другой стороны, команда `echo` воспринимает слова без кавычек как выражения (вычисления, возвращающие значения) или функции. Поэтому нам нужно заключить в кавычки имя выбранной нами цветовой схемы.

Переменные

Если вы программист, вы, вероятно, заметили проблему в нашем сценарии: мы выполняем проверку времени суток с помощью функции `strftime()` в каждом условном блоке. Это не очень эффективно: значение может измениться во время выполнения.

Лучше вызвать функцию один раз и сохранить результат в *переменной* сценария Vim. Тогда мы сможем использовать эту переменную в нашем условном выражении без лишних затрат ресурсов.

С помощью команды `:let` можно присвоить значение переменной:

```
:let var = "значение"
```

Для наших целей мы можем назвать нашу переменную как угодно (с учетом контекста), потому что мы используем ее лишь один раз (хотя это потом изменится). Пока что мы позволяем Vim интерпретировать ее как глобальную по умолчанию. Позднее мы увидим, что можно использовать специальные префиксы для определения области видимости переменной.

Назовем нашу переменную `currentHour`¹. Поскольку мы присваиваем результат из функции `strftime()` лишь раз, у нас теперь более эффективный сценарий:

```
" progressively check higher values... falls out on first "true"
let currentHour = strftime("%H")
echo "currentHour is " currentHour
if currentHour < 6
  colorscheme darkblue
  echo "setting colorscheme to darkblue"
elseif currentHour < 12
  colorscheme morning
  echo "setting colorscheme to morning"
elseif currentHour < 18
  colorscheme shine
  echo "setting colorscheme to shine"
else
  colorscheme evening
  echo "setting colorscheme to evening"
endif
```

¹ Технический рецензент заметил, что имя переменной `currentHour` не совсем подходит, потому что ее значение на самом деле не час. Это имя отражает ее характер, но в качестве альтернативы можно было бы назвать переменную по-другому, например `colorIndex`. Мы согласны с этим замечанием, но все же оставляем сценарий в его исходном виде.

Мы можем еще немного улучшить код и избавиться от лишних строк, введя переменную `colorScheme`. Она хранит значение цветовой схемы, которую мы определяем по времени суток. Мы добавим заглавную *S*, чтобы отличать переменную от имени команды `colorscheme`, но мы могли бы использовать точно такие же буквы, и это не имело бы значения: Vim может определить по контексту, команда это или переменная:

```
" progressively check higher values... falls out on first "true"
let currentHour = strftime("%H")
echo "currentHour is " . currentHour
if currentHour < 6
    let colorScheme = "darkblue"
elseif currentHour < 12
    let colorScheme = "morning"
elseif currentHour < 18
    let colorScheme = "shine"
else
    let colorScheme = "evening"
endif
echo "setting color scheme to " . colorScheme
colorscheme colorScheme
```

Обратите внимание на использование точки (.) с командой `echo`. Этот оператор объединяет выражения в одну строку, которую затем выводит `echo`. В данном случае мы создаем строку из символов "setting color scheme to " и значение, присвоенное переменной `colorScheme`.



В этом сценарии команды выполняются не так, как мы предполагали. Если вы пытались запустить этот пример, то уже знаете это. Мы исправим ошибку в следующем разделе.

Команда `execute`

До сих пор мы улучшали способ выбора цветовой схемы, но наше последнее изменение привело к неожиданному повороту. Изначально мы решили выполнить дискретную команду `colorscheme`, основанную на времени суток. Наше последнее исправление выглядит верным, но после определения переменной (`colorScheme`) для хранения значения цветовой схемы мы обнаружили, что команда:

```
colorscheme colorScheme
```

выдает ошибку, показанную на рис. 12.1.

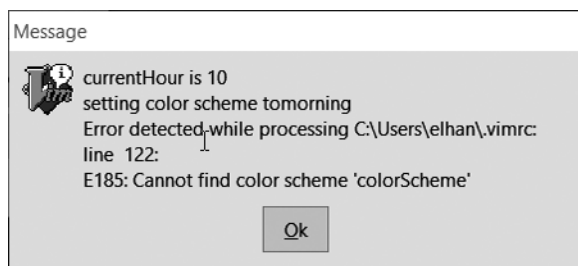


Рис. 12.1. Ошибка colorscheme colorScheme

Для выполнения команды Vim, которая использует переменную вместо буквенной строки, например `darkblue`, можно воспользоваться командой `execute`. Она принимает команду и подставляет значения переменных и выражений в нее. Мы можем использовать эту функцию наряду с объединением, показанным в предыдущем разделе, чтобы передать значение нашей переменной команде `colorscheme`:

```
execute "colorscheme " . colorScheme
```

Используемый здесь синтаксис (особенно кавычки) может сбить с толку. `colorscheme` является обыкновенной строкой, а не переменной или выражением, которые ожидает команда `execute`. Нам нужно, чтобы `execute` приняла имя как есть, а не вычисляла `colorscheme`. Поэтому мы превращаем имя команды в буквенную строку, заключая ее в кавычки. Перед закрывающей кавычкой важно поставить пробел, так как между командой и аргументом он должен быть.

Переменную `colorScheme` мы поместим *за пределы* кавычек, чтобы команда `execute` могла ее вычислить. Подумайте о поведении `execute` следующим образом.

- Простые слова оцениваются как переменные или выражения, и команда `execute` заменяет их значения.
- Кавычки, в которые заключены строки, воспринимаются буквально; команда `execute` не пытается определить их, чтобы вернуть значения.

Благодаря `execute` наша проблема решена, а Vim загружает нужную цветовую схему.

Команда `colorscheme` устанавливает свою собственную переменную `colors_name`. Вы можете использовать команду `echo` для просмотра значений переменных из вашего сценария или для проверки переменной `colors_name`, чтобы убедиться, что ваш сценарий выбрал правильную цветовую схему в зависимости от времени суток:

```
:echo colors_name
```

Определение функций

Мы создали хорошо работающий сценарий. Теперь давайте напишем код, который можно выполнять в любое время сеанса редактирования, а не только при запуске Vim. Мы вскоре приведем пример, но для начала нам нужно создать *функцию*, содержащую код нашего сценария.

В Vim можно определять собственные функции с помощью операторов `function...` `endfunction`. Вот примерный вид пользовательской функции:

```
function МояФункция(arg1, arg2...)
    строка кода
    еще одна строка кода
endfunction
```

Наш сценарий можно с легкостью превратить в функцию. Обратите внимание, что нет необходимости передавать какой-либо аргумент, поэтому круглые скобки в определении функции пустые:

```
function SetTimeOfDayColors()
    " progressively check higher values... falls out on first "true"
    let currentHour = strftime("%H")
    echo "currentHour is " . currentHour
    if currentHour < 6
        let colorScheme = "darkblue"
    elseif currentHour < 12
        let colorScheme = "morning"
    elseif currentHour < 18
        let colorScheme = "shine"
    else
        let colorScheme = "evening"
    endif
    echo "setting color scheme to " . colorScheme
    execute "colorscheme " . colorScheme
endfunction
```



Имена пользовательских функций Vim должны начинаться с заглавной буквы.

Теперь у нас есть функция, определенная в нашем файле `.gvimrc`. Однако код не будет выполняться, если не вызвать ее. Для этого воспользуйтесь оператором `call`. Например:

```
call SetTimeOfDayColors()
```

Теперь мы можем установить нашу цветовую схему в любое время и в любом месте внутри сеанса Vim. Достаточно просто поместить предыдущую строку `call` в наш файл `.gvimrc`. Результаты будут те же, что и в ранее приведенном примере, в котором мы запускали код, не используя функцию. Но в следующем разделе мы познакомимся с приемом Vim, с помощью которого наша функция будет вызываться многократно, динамически меняя цветовую схему в течение дня! И конечно, мы столкнемся с другими проблемами, которые предстоит решить.

Хитрый трюк

В предыдущем разделе мы создали функцию `Vim SetTimeOfDayColors()`, которая выбирает цветовую схему один раз. А что, если мы хотим многократно менять цветовую схему в зависимости от времени суток? Очевидно, что с помощью единовременного вызова в `.gvimrc` этого не достичь. Но есть один хитрый прием Vim, использующий параметр `statusline`.

Большинство пользователей Vim воспринимают строку состояния Vim как должное. По умолчанию у параметра `statusline` нет значения, но вы можете задать его так, чтобы он отображал практически любую информацию, доступную Vim в строке состояния. А поскольку строка состояния способна отображать динамическую информацию, такую как текущая строка и столбец, Vim пересчитывает и повторно выводит на экран строку состояния каждый раз, когда изменяется статус редактирования (а это практически любое действие в Vim). Таким образом, мы можем использовать это для вызова функции нашей цветовой схемы и динамического изменения цветовой схемы. Но, как мы вскоре убедимся, этот подход несовершенен.

Параметр `statusline` принимает выражение, вычисляет его и показывает в строке состояния. Это включает функции. Мы используем этот прием для вызова `SetTimeOfDayColors()` каждый раз, когда обновляется строка состояния, что происходит часто. Поскольку эта функция замещает стандартную строку состояния, а мы не хотим потерять ценную информацию, полученную по умолчанию, то включим это изобилие информации в следующее исходное определение нашей строки состояния:

```
set statusline=%<{t%h%m%r\ \ %a\ %{strftime(\"%c\")}%=0x%B\
  \\ line:%l,\ \ col:%c%V\ %P
```



Определение для `statusline` разделено на две строки. Если первый символ второй строки начинается с обратного слеша (`\`), Vim будет считать эту строку продолжением предыдущей и проигнорирует все пробелы вплоть до обратного слеша. Таким образом, если вы используете наше определение, убедитесь, что оно точно скопировано и вставлено. Если у вас не получается заставить его работать, вернитесь к запуску сценария с неопределенной `statusline`.

В документации Vim вы можете найти значения различных символов, которым предшествуют символы процента. Они используются для формирования строки состояния, например такой:

```
ch12.asciidoc  Thu 26 Aug 2021 12:39:26 PM EDT      0x3C line:1, col:1 Top
```

В текущей главе мы не будем подробно рассматривать все возможности настройки строки состояния, а сконцентрируемся на использовании параметра `statusline` для оценки функции.

Давайте добавим функцию `SetTimeOfDayColors()` в `statusline`. Чтобы дополнить то, что мы уже определили ранее, мы используем `+=` вместо простого знака равенства:

```
set statusline +=\ %{SetTimeOfDayColors()}
```

Теперь наша функция является частью строки состояния. И хотя она не привносит никакой интересной информации, она теперь поверяет время суток и при необходимости обновляет нашу цветовую схему в течение дня. Здесь мы сталкиваемся с двумя проблемами.

- У нас теперь есть функция сценария Vim, которая проверяет, который час, при каждом обновлении строки состояния. Это может снизить эффективность нашего сеанса, так как мы делаем много лишних вызовов `strftime()`.
- Когда во время сеанса `statusline` вычисляет подходящее время и меняет цветовую схему, он работает так, как нам нужно. Однако он также устанавливает цветовую схему независимо от того, нужно ее менять или нет при проверке времени.

В следующем разделе мы изучим более оптимальные способы достижения нашей цели, используя глобальные переменные.

Настройка сценариев Vim с помощью глобальных переменных

Vim предоставляет скалярные переменные (числа и строки) и массивы. Более того, вы можете указать область видимости переменной.

Области видимости переменных

Переменные Vim довольно просты, но перед тем, как обсуждать глобальные переменные, нужно научиться управлять *областью видимости* переменных. Область

видимости переменной в Vim определяется с помощью *префикса* в ее имени. Существуют следующие префиксы:

- **a:** — аргумент функции;
- **b:** — переменная, распознаваемая в рамках одного буфера Vim;
- **g:** — переменная, распознаваемая глобально, то есть на нее можно ссылаться *в любом месте* кода;
- **l:** — переменная, распознаваемая внутри функции (локальная переменная);
- **s:** — переменная, распознаваемая внутри исходного сценария Vim;
- **t:** — переменная, распознаваемая в рамках одной вкладки Vim;
- **v:** — переменная, управляемая Vim (также является глобальной);
- **w:** — переменная, распознаваемая в рамках одного окна Vim.



Если не задать область видимости переменной с помощью префикса, то по умолчанию она будет глобальной (**g:**) при определении вне функции и локальной (**l:**) при определении внутри функции.

Глобальные переменные

В ходе нашего последнего изменения сценария мы *почти* получили желаемое поведение. Наша функция вызывается каждый раз при обновлении строки состояния Vim, однако это происходит довольно часто и создает несколько проблем.

Во-первых, стоит беспокоиться о создаваемой нагрузке на процессор компьютера из-за такого частого вызова функции. Вероятно, для современных компьютеров подобная интенсивность навряд ли будет критичной, тем не менее это довольно плохая практика — постоянно переопределять цветовую схему. Если бы это было единственной проблемой, мы могли бы посчитать наш сценарий завершенным и не заморачиваться с его улучшением. Однако это не так.

Если вы выполняли примеры вместе с нами, вы уже знаете проблему. Постоянное исправление цветовой схемы во время работы создает заметное и раздражающее мерцание экрана, потому что каждое определение цветовой схемы (даже если она не меняется) заставляет Vim перечитать сценарий определения цветовой схемы, заново проинтерпретировать текст и повторно применить все правила цветовой подсветки синтаксиса. Даже очень мощные компьютеры вряд ли справятся с таким постоянным обновлением без мерцания. Нам нужно это исправить.

Мы можем один раз определить нашу цветовую схему и затем внутри условного блока проверять каждый раз, изменилась ли она и надо ли ее переопределить и отобразить. Мы делаем это, используя глобальные переменные, которые за-

даются с помощью команды `colorscheme colors_name`. Изменим нашу функцию с учетом этого:

```
function SetTimeOfDayColors()
  " progressively check higher values... falls out on first "true"
  let currentHour = strftime("%H")
  if currentHour < 6
    let colorScheme = "darkblue"
  elseif currentHour < 12
    let colorScheme = "morning"
  elseif currentHour < 18
    let colorScheme = "shine"
  else
    let colorScheme = "evening"
  endif
  " if our calculated value is different, call the colorscheme command.
  if g:colors_name != colorScheme
    echo "setting color scheme to " . colorScheme
    execute "colorscheme " . colorScheme
  endif
endfunction
```

Теперь у нас есть динамическая и эффективная функция. В следующем разделе мы внесем одно последнее улучшение.

Массивы

Было бы здорово, если бы каким-то образом мы могли извлечь значение нашей цветовой схемы без расширения блока `if...then...else`. С помощью массивов Vim мы можем улучшить сценарий и сделать его более читабельным.

Массив создается путем присваивания переменной списка значений, разделенных запятыми и заключенных в квадратные скобки. Мы объявляем для нашей функции массив с именем `Favcolorschemes` и определяем его вне функции как глобальный, чтобы получить к нему доступ из любой части кода:

```
let g:Favcolorschemes = ["darkblue", "morning", "shine", "evening"]
```

Эту строку нужно добавить в ваш файл `.gvimrc`. Теперь мы можем ссылаться на любое значение внутри массива `g:Favcolorschemes` по индексу, начиная с нулевого элемента. Например, `g:Favcolorschemes[2]` вернет значение `shine`.

С помощью математических функций Vim, где результаты целочисленного деления являются целыми числами (остатки отбрасываются), мы теперь в состоянии быстро и легко получить предпочитаемую цветовую схему в зависимости от времени суток. Взглянем на итоговую версию нашей функции:

```
function SetTimeOfDayColors()
  " currentHour will be 0, 1, 2, or 3
```

```
let g:CurrentHour = strftime("%H") / 6
if g:colors_name != g:Favcolorschemes[g:CurrentHour]
    execute "colorscheme " . g:Favcolorschemes[g:CurrentHour]
    echo "execute " "colorscheme " . g:Favcolorschemes[g:CurrentHour]
    redraw
endif
endfunction
```

Оператор `echo ...` выводит информацию и «объявляет» изменения, которые только что произошли в результате выполнения сценария. Оператор `redraw` указывает Vim немедленно перерисовать экран.

Поздравляем! Вы успешно написали полноценный сценарий Vim, учитывающий множество факторов, необходимых для создания любого полезного сценария, который может вам понадобиться.

Динамическая конфигурация типа файла с помощью сценария

Давайте рассмотрим еще один пример полезного сценария. Как правило, при редактировании нового файла единственным источником, откуда Vim может получить информацию для определения типа файла и установки `filetype`, является расширение имени файла. Например, расширение `.c` означает, что файл представляет собой код на языке C. Vim с легкостью определяет его и загружает соответствующее поведение, упрощая редактирование программы на языке C.

Однако не всем файлам нужно расширение имени. К примеру, хотя сценариям оболочки и принято назначать расширение `.sh`, этого можно не делать, особенно с учетом того, что мы (авторы этих строк) создали тысячи сценариев еще до того, как это расширение стало популярным. На самом деле Vim достаточно продвинутая программа и может распознать сценарий оболочки без использования расширения файла. Однако для определения типа файла Vim необходим контекст, и он может быть определен только при втором редактировании. Сценарии Vim могут это исправить!

Автокоманды

В нашем первом примере сценария мы опирались на привычку Vim постоянно обновлять строку состояния и «спрятали» нашу функцию в строке состояния, чтобы установить цветовую схему в зависимости от времени суток. В нашем сценарии для динамического определения типа файла мы полагаемся на более формальное соглашение Vim — *автокоманды*.

Автокоманды могут включать любые действительные команды Vim. Vim использует *события* для выполнения команд. Ниже перечислены некоторые события, которые запускают соответствующую команду при возникновении события:

- BufNewFile — когда Vim начинает редактирование нового файла;
- BufReadPre — перед тем как Vim переместится в новый буфер;
- BufRead, BufReadPost — во время редактирования нового буфера, но *после* чтения файла;
- BufWrite, BufWritePre — перед записью буфера в файл;
- FileType — после установки filetype;
- VimResized — после изменения размера окна Vim;
- WinEnter, WinLeave — при входе и выходе из окна Vim соответственно;
- CursorMoved, CursorMovedI — каждый раз, когда курсор перемещается в командном режиме vi и режиме ввода соответственно.

В целом существует около 80 событий Vim. Для любого из этих событий вы можете определить автоматическую команду autocmd, которая выполняется при возникновении события. Формат autocmd следующий:

```
autocmd [группа] событие шаблон [nested] команда
```

К элементам этого формата относятся:

- *группа* — необязательная группа команд (описана далее);
- *событие* — событие, которое запустит *команду*;
- *шаблон* — шаблон, соответствующий имени файла, для которого должна выполняться *команда*;
- *nested* — если этот флаг присутствует, то автокоманда может быть вложенной в другие;
- *команда* — команда, функция или пользовательский сценарий Vim для выполнения при возникновении события.

Наша цель — определить тип файла для любого открываемого нами файла, поэтому мы используем * для *шаблона*.

Следующий шаг — выбор события для запуска нашего сценария. Поскольку мы хотим определить тип нашего файла как можно скорее, запрашиваются два подходящих варианта: CursorMoved и CursorMovedI.

`CursorMoved` вызывает событие при перемещении курсора, что кажется затратным, потому что простое перемещение курсора навряд ли даст больше информации о типе файла. `CursorMovedI`, напротив, срабатывает при вводе текста и поэтому выглядит более предпочтительным.

Нам нужно написать функцию, которая будет выполняться каждый раз при этом событии. Назовем ее `CheckFileType()`. Теперь у нас есть все необходимое для создания нашей команды `autocmd`. Она выглядит следующим образом:

```
autocmd CursorMovedI * call CheckFileType()
```

Параметры проверки

В нашей функции `CheckFileType` необходимо проверить значение параметра `filetype`. Сценарии Vim используют специальные переменные для извлечения значений из параметров, добавляя к имени параметра (в нашем случае `filetype`) префикс в виде символа амперсанда (&). Следовательно, в нашей функции мы используем переменную `&filetype`.

Начнем с простой версии функции `CheckFileType()`.

```
function CheckFileType()
  if &filetype == ""
    filetype detect
  endif
endfunction
```

Команда Vim `filetype detect` является сценарием Vim, установленным в каталоге `$VIMRUNTIME`. Она просматривает множество критериев и пытается назначить вашему файлу определенный тип. Обычно это происходит один раз, поэтому если файл новый и `filetype` не может определить его тип, то в сеансе редактирования нельзя будет назначить форматирование синтаксиса.

Есть одна дилемма: наша функция вызывается каждый раз, когда курсор перемещается в режиме ввода, непрерывно пытаясь определить тип файла. Чтобы решить эту проблему, нужно сначала проверить наличие типа у файла. Если он есть, значит, наша функция достигла цели во время своего предыдущего выполнения, и поэтому ей больше не надо это делать. Не стоит беспокоиться об отклонениях от нормы, таких как ошибочное распознавание или когда в файле сначала используется один язык программирования, а потом — другой.

Отредактируем новый файл сценария оболочки и посмотрим на результаты:

```
$ vim ScriptWithoutSuffix
```

Введите следующее:

```
#!/bin/sh
```

```
inputFile="DailyReceipts"
```

Теперь Vim подсвечивает синтаксис, как показано на рис. 12.2.

Из рисунка видно, что Vim использует серый цвет для строки, но на черно-белом фоне незаметно, что `#!/bin/sh` синего цвета, `inputFile=` — черного, а `"DailyReceipts"` — фиолетового. Быстрая проверка параметра `filetype` с помощью команды `:set filetype` выдаст сообщение, показанное на рис. 12.3.

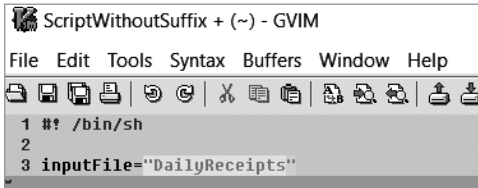


Рис. 12.2. Выявлен тип нового файла
(MS Windows gvim, цветовая схема: morning)



Рис. 12.3. Выявлен тип файла conf
(MS Windows gvim, цветовая схема: morning)

По какой-то причине Vim присвоил нашему файлу тип файла `conf`, что не совпадает с нашими желаниями.

Если вы попробуете воспроизвести данный пример, то увидите, что Vim присваивает тип файла сразу после ввода первого символа `#` в первом же событии `CursorMovedI`. Конфигурационные файлы для утилит и демонов (daemons) Unix, как правило, используют символ `#` для комментирования, поэтому эвристика Vim предполагает, что `#` в начале строки — это начало комментария в конфигурационном файле. Нам нужно научить Vim быть более разборчивым.

Изменим нашу функцию, чтобы обеспечить больше контекста. Сделаем так, чтобы Vim не пытался определить тип файла при первой же возможности, а ждал, пока пользователь не введет хотя бы 20 символов.

Переменные буфера

Нам нужно ввести переменную в нашу функцию, которая даст Vim команду временно *не пытаться* выявить тип файла, пока автокоманда `CursorMovedI` не вызовет функцию более 20 раз. Наше представление о новом файле, а также о количестве символов, которые мы хотим ввести в этот файл, специфично для буфера.

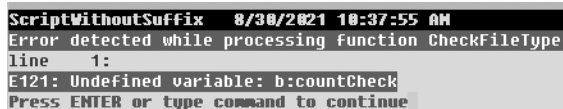
Другими словами, перемещение курсора в других буферах сеанса редактирования не должно учитываться при проверке. Поэтому мы вводим буферную переменную `b:countCheck`.

Далее мы пересматриваем функцию, чтобы проверить по крайней мере 20 перемещений курсора в режиме ввода (что означает примерно 20 введенных символов), и делаем проверку того, был ли уже присвоен тип файла:

```
function CheckFileType()
  let b:countCheck += 1

  " Don't start detecting until approximately 20 chars.
  if &filetype == "" && b:countCheck > 20
    filetype detect
  endif
endfunction
```

Но теперь мы получаем сообщение об ошибке, показанное на рис. 12.4.



```
ScriptWithoutSuffix 8/30/2021 10:37:55 AM
Error detected while processing function CheckFileType
line 1:
E121: Undefined variable: b:countCheck
Press ENTER or type command to continue
```

Рис. 12.4. `b:countCheck` приводит к ошибке `undefined`

Довольно известная ошибка. Как и ранее, мы были достаточно безрассудны, чтобы проверить значение переменной до ее определения. И на этот раз это целиком наша вина, поскольку данный сценарий отвечает за определение `b:countCheck`. Мы разберемся с этой тонкостью в следующем разделе.

Функция `exists()`

Важно знать, как управлять всеми вашими переменными и функциями: Vim требует, чтобы вы определили каждую из них, чтобы она *существовала* перед обращением к ней какого-либо выражения.

Мы можем с легкостью исправить ошибку нашего сценария, проверив существование переменной `b:countCheck` и присвоив ей значение с помощью команды `:let`, показанной ранее:

```
function CheckFileType()
  if exists("b:countCheck") == 0
    let b:countCheck = 0
  endif

  let b:countCheck += 1
```

```
" Don't start detecting until approx. 20 chars.
if &filetype == "" && b:countCheck > 20
    filetype detect
endif
endfunction
```

Теперь протестируйте код. На рис. 12.5 показан момент перед достижением предела в 20 символов, а на рис. 12.6 — что получилось после ввода 21-го символа.

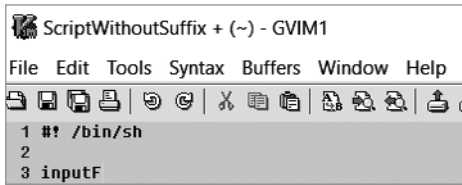


Рис. 12.5. Еще не определен ни один тип файла (MS Windows gvim, цветовая схема: morning)

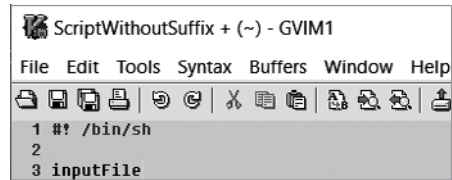


Рис. 12.6. Тип файла определен (MS Windows gvim, цветовая схема: morning)

У текста `/bin/sh` внезапно появилась подсветка синтаксиса. Быстрая проверка с помощью `set filetype` подтверждает, что Vim верно определил тип файла (рис. 12.7).

Для всех практических целей у нас есть полное и удовлетворительное решение, однако нелишним будет добавить еще одну проверку, чтобы Vim не пытался выявить тип файла после ввода примерно 200 символов:

```
function CheckFileType()
    if exists("b:countCheck") == 0
        let b:countCheck = 0
    endif

    let b:countCheck += 1

    " Don't start detecting until approx. 20 chars.
    if &filetype == "" && b:countCheck > 20 && b:countCheck < 200
        filetype detect
    endif
endfunction
```

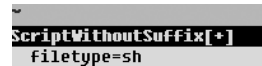


Рис. 12.7. Правильное определение

Хотя функция `CheckFileType()` вызывается при каждом движении курсора в Vim, мы несем незначительные издержки, поскольку функция прекращает работу при обнаружении типа файла или при превышении порога 200 символов. Это, вероятно, достаточно для обеспечения необходимой функциональности с минимальными издержками на обработку. Однако мы продолжим изучать другие алгоритмы, которые смогут предоставить нам более полное и удовлетворительное решение,

которое не только потребует минимальных затрат, но и фактически «отключается», когда в нем больше нет надобности.



Вы, возможно, заметили, что мы достаточно смутно описали точное значение нашего порогового числа 20 символов. Это сделано намеренно. Поскольку мы учитываем перемещения курсора в режиме ввода, было бы разумным предположить, что каждое движение курсора соответствует новому символу и добавляет достаточно контекста тексту для того, чтобы функция `CheckFileType()` могла определить тип файла.

Тем не менее засчитываются все перемещения курсора в режиме ввода, поэтому любой возврат для исправления опечаток также учитывается в счетчике порога. Чтобы убедиться в этом, выполните наш пример: введите символ `#`, вернитесь назад и повторите это десять раз. На 11-й раз символ `#` раскрасится, а тип файла будет теперь (неверно) определен как `conf`.

Автокоманды и группы

До сих пор наш сценарий игнорировал любые побочные эффекты от вызова нашей функции при каждом перемещении курсора. Мы минимизировали издержки за счет проверок на допустимость, что позволило избежать лишнего вызова тяжелой команды `filetype detect`. Но что, если даже минимальный код слишком ресурсоемкий для нашей функции? Нам нужен способ прекратить вызов кода, когда он нам не требуется. Для этого мы воспользуемся концепцией автокоманд Vim «*группы*» и их способностью удалять команды по их групповой принадлежности.

Мы изменим наш пример, сначала связав нашу функцию, вызываемую событием `CursorMovedI`, с группой. Для этого Vim предоставляет команду `augroup`. Ее синтаксис следующий:

```
augroup имягруппы
```

Все последующие определения `augroup` становятся связанными с *имягруппы* до следующего утверждения:

```
augroup End
```

Существует также группа по умолчанию для команд, не вошедших в блок `augroup`.

Свяжем нашу предыдущую команду `autocmd` с нашей собственной группой:

```
augroup newFileDetection
  autocmd CursorMovedI * call CheckFileType()
augroup End
```


Наша функция `CursorMovedI`, запускаемая при перемещении курсора в режиме ввода, является частью группы автокоманды `newFileDetection`. Полезность этого мы рассмотрим в следующем подразделе.

Удаление автокоманд

Чтобы наша функция работала максимально эффективно, требуется, чтобы она выполнялась только тогда, когда это необходимо. Нужно отменить ссылку на нее, как только истечет ее срок полезного использования (то есть как только мы определим тип файла или решим, что не можем этого сделать). Vim позволяет удалять автокоманду, просто ссылаясь на событие, шаблон, которому должны соответствовать имена файлов, или его группу:

```
autocmd! [группа] [событие] [шаблон]
```

Символ (!) следует за ключевым словом `autocmd`, чтобы указать, что команды, связанные с группой, событием или шаблоном, должны быть удалены.

Поскольку наша функция связана с пользовательской группой `newFileDetection`, мы теперь можем управлять ею и удалить ее, ссылаясь на группу в синтаксисе удаления автокоманды. Мы делаем это с помощью:

```
autocmd! newFileDetection
```

Это удаляет все автокоманды, связанные с группой `newFileDetection`, в том числе и нашу функцию.

Мы можем проверить как создание, так и удаление нашей автокоманды при запуске Vim (когда создаем новый файл) с помощью команды:

```
:autocmd newFileDetection
```

Пример реакции Vim показан на рис. 12.8.

```
:autocmd newFileDetection
--- Autocommands ---
newFileDetection  CursorMovedI
*                call CheckFileType()
Press ENTER or type command to continue
```

Рис. 12.8. Ответ на команду `autocmd newFileDetection`
(MS Windows gvim, цветовая схема: morning)

После определения и присвоения типа файла *или* достижения порога в 200 символов нам больше не понадобится наша автокоманда. Поэтому мы добавляем

последний штрих в наш код. Сочетая определение нашей `augroup`, команды `autocmd` и разработанной функции, строки в файле `.vimrc` будут выглядеть так:

```
augroup newFileDetection
  autocmd CursorMovedI * call CheckFileType()
augroup End

function CheckFileType()
  if exists("b:countCheck") == 0
    let b:countCheck = 0
  endif

  let b:countCheck += 1

  " Don't start detecting until approx. 20 chars.
  if &filetype == "" && b:countCheck > 20 && b:countCheck < 200
    filetype detect
    " If we've exceeded the count threshold (200), OR a filetype has been detected
    " delete the autocmd!
  elseif b:countCheck >= 200 || &filetype != ""
    autocmd! newFileDetection
  endif
endfunction
```

После включения цветовой подсветки синтаксиса мы можем убедиться, что наша функция удаляет саму себя, если введем ту же самую команду, какую мы использовали при вводе буфера:

```
:autocmd newFileDetection
```

Ответ Vim показан на рис. 12.9.



```
:autocmd newFileDetection
Autocommands
Press ENTER or type command to continue
```

Рис. 12.9. После того как был выполнен критерий удаления для нашей группы autocommand (MS Windows gvim, цветовая схема: morning)

Обратите внимание, что в группе `newFileDetection` не определены автокоманды. Вы можете удалить автогруппу следующим образом:

```
augroup! имягруппы
```

но это действие *не* удалит связанные автокоманды, и Vim, ссылаясь на эти автокоманды, каждый раз будет выдавать ошибку. Поэтому убедитесь, что вы удалили автокоманды внутри группы, прежде чем удалять группу.



Не путайте удаление автокоманд с удалением автогрупп.

Поздравляем! Вы написали свой второй сценарий Vim. Он позволил вам расширить ваши знания Vim и ознакомиться с различными функциями, доступными при создании сценариев.

Некоторые соображения по поводу сценариев Vim

Мы рассмотрели лишь малую часть возможностей сценариев Vim, но надеемся, что вы прочувствовали их силу и гибкость. Практически все ваши действия в Vim могут быть автоматизированы через сценарии.

В следующем разделе мы рассмотрим прекрасный пример встроенной документации Vim, более подробно обсудим понятия, которых мы касались ранее, и разберем несколько новых функций.

Полезный пример сценария Vim

Встроенная документация Vim содержит удобный сценарий, который, как мы думаем, может быть полезен для вас. Он создан специально для обновления текущих даты и времени в теге <meta> HTML-файла, но его легко можно адаптировать для многих других типов файлов, в которых желательно указывать время последнего изменения файла в его содержимом.

Вот практически оригинальный пример (мы лишь немного его изменили):

```
autocmd BufWritePre,FileWritePre *.html  mark s|call LastMod()|'s
fun LastMod()
  " if there are more than 20 lines, set our max to 20, otherwise, scan
  " entire file.
  if line("$") > 20
    let lastModifiedline = 20
  else
    let lastModifiedline = line("$")
  endif
  exe "1," . lastModifiedline . "g/Last modified: ./s//Last modified: " .
  \ strftime("%Y %b %d")
endfun
```

Ниже приведена краткая схема команды `autocmd`.

- `BufWritePre, FileWritePre` — это события, для которых запускается команда. В данном случае Vim выполняет автокоманду *перед* записью файла или буфера на запоминающее устройство.
- `*.html` — выполняет автокоманду для любого файла, имя которого заканчивается на `.html`.

- `mark s` — для улучшения читабельности мы изменили оригинальную команду `ks` на эквивалентную, но более очевидную — `mark s`. Она просто создает в файле отмеченную позицию с именем `s`, чтобы мы могли вернуться к этому месту позднее.
- `|` — символы вертикальной черты разделяют несколько команд Vim, которые выполняются в определении автокоманды. Это простые разделители, не имеющие никакого отношения к символам вертикальной черты оболочки Unix.
- `call LastMod()` — вызывает нашу пользовательскую функцию `LastMod()`.
- `|` — то же, что и предыдущее.
- `'s` — возвращает к строке, которую мы отметили именем `s`.

Стоит проверить этот сценарий, отредактировав файл `.html` и добавив строку:

```
Last modified:□
```

а затем запустив команду `w`.



Этот пример наглядный, но он не соответствует заданной цели заменить HTML-тег `<meta>`. Было бы лучше, если бы мы действительно использовали тег `<meta>`, чтобы замена искала часть `content=...` внутри `<meta>`. Но все же данный пример — хорошее начало на пути к решению этой задачи и полезный образец для других типов файлов.

Подробнее о переменных

Давайте подробнее рассмотрим, что такое переменные Vim и как с ними работать. В Vim существует пять типов переменных.

- *Number* (число) — 32-битное число со знаком. Это число может быть записано как в десятичной, так и в шестнадцатеричной (например, `0xffff`) или восьмеричной (например, `0117`) форме. В зависимости от возможностей компилятора Vim может поддерживать 64-битные числа. Команда `ex :version` помогает это выяснить (рис. 12.10).
- *String* (строка) — строка символов.

```
break          +netbeans_intg      +sy
indent         +num64              +ta
cmds           +packages           +ta
```

Рис. 12.10. Результат команды `:version`, показывающий поддержку 64-битного числа (WSL Ubuntu Linux, цветовая схема: `zellner`)

- *Funcref* (ссылка) — ссылка на функцию.
- *List* (список) — это аналог массива в Vim. Он представляет собой упорядоченный набор значений и может содержать любое их сочетание в качестве элементов.
- *Dictionary* (словарь) — это подобие *хеши* или *ассоциативного массива* в Vim. Он представляет собой неупорядоченную совокупность пар значений, первое из которых является *ключом* для извлечения связанного *значения*.

Выражения

Vim вычисляет выражения достаточно прямолинейным способом. Выражение может быть простым, как число или буквенная строка, или сложным, как составной оператор, состоящий из выражений.

Важно отметить, что математические функции Vim работают только с целыми числами. Если вам нужна плавающая точка и точность, вам следует использовать расширения, такие как системные вызовы внешних математических подпрограмм.

Расширения

Vim предлагает несколько расширений и интерфейсов для других языков сценариев. Примечательно, что к ним относятся Perl, Python и Ruby — три самых популярных языка программирования. Подробную информацию о том, как их использовать, вы найдете во встроенной документации Vim.

Еще несколько слов о autocmd

В предыдущем разделе «Динамическая конфигурация типа файла с помощью сценария» мы использовали команду Vim `autocmd`, чтобы связать события, при которых вызываются наши пользовательские функции. Это очень эффективно, но не стоит забывать о более простых применениях `autocmd`. Например, вы можете использовать `autocmd` для настройки специальных параметров Vim для различных типов файлов.

Хорошим примером может быть изменение параметра `shiftwidth` для разных типов файлов. Файлы с многочисленными отступами и вложенными уровнями могут выглядеть от более скромных отступов. Вы можете задать ваш `shiftwidth` равным 2 для HTML, чтобы код не «уходил» в правую часть экрана, но использовать `shiftwidth`, равный 4, для программ на языке C. Чтобы сделать это разграничение, добавьте следующие строки в ваш файл `.vimrc` или `.gvimrc`:

```
autocmd BufRead,BufNewFile *.html set shiftwidth=2
autocmd BufRead,BufNewFile *.c,*.h set shiftwidth=4
```

Внутренние функции

В дополнение ко всем командам Vim у вас есть доступ примерно к 200 внутренним функциям. Описание всех этих функций выходит за рамки наших целей, но полезно будет узнать, какие категории или типы функций доступны. Следующие категории взяты из встроенного файла справки Vim `usr_41.txt`.

- *Управление строками.* В этих функциях содержатся все стандартные строковые функции, которых ожидают программисты: от преобразования строк до операций с подстроками и многого другого.
- *Функции формирования списков.* Это целый массив функций для работы со списками. Они точно отображают функции массива языка Perl.
- *Функции для работы со словарями (ассоциативными массивами).* Эти функции позволяют извлекать, управлять, верифицировать и совершать другие действия со словарями. И опять же они очень похожи на хеш-функции Perl.
- *Функции для работы с переменными.* Эти функции являются «получателями» и «установщиками» для перемещения переменных по окнам и буферам Vim. Есть также функция `type()` для определения типов переменных.
- *Функции для работы с курсором и позиционированием.* Эти функции позволяют перемещаться по файлам и буферам и создавать метки, чтобы эти позиции можно было запомнить и вернуться к ним. Есть также функции, предоставляющие информацию о позиции (например, строка и столбец курсора).
- *Функции для работы с текстом в текущем буфере.* Эти функции управляют текстом внутри буферов: например, изменение или извлечение строки и т. д. Есть также функции поиска.
- *Системные функции и функции управления файлами.* Они включают функции для перемещения по операционной системе, в которой запущен Vim, такие как поиск файлов в путях, определение текущего рабочего каталога, а также создание и удаление файлов. Эта группа включает функцию `system()`, которая передает команды операционной системе для внешнего выполнения.
- *Функции даты и времени.* Они выполняют широкий спектр манипуляций с форматами даты и времени.
- *Функции буфера, окна и списка аргументов.* Эти функции предоставляют механизмы для сбора информации о буферах и аргументах для каждого из них. Например, при запуске Vim список файлов содержит список его аргументов, а функция `argc()` возвращает подсчет этого списка. Функции списка аргументов специфичны для аргументов, передаваемых из командной строки Vim. Функции буферов предоставляют информацию, специфичную для буферов и окон. Всего 25 функций. Для получения более подробной информации по-

ищите «Буферы, окна и список аргументов» в файле справки Vim `usr_41.txt`. Или воспользуйтесь командой `ex`:

```
:help function-list
```

- *Функции командной строки.* Эти функции получают местоположение командной строки, командную строку и тип командной строки и устанавливают местоположение курсора в командной строке.
- *Функции QuickFix и местоположения.* Эти функции извлекают и изменяют списки QuickFix.
- *Функции завершения режима ввода.* Эти функции используются для завершения команд и ввода.
- *Функции свертывания.* Эти функции предоставляют информацию для свертываний и разворачивают свернутый текст.
- *Функции синтаксиса и подсветки.* Эти функции извлекают информацию о группах подсветки синтаксиса и идентификаторах синтаксиса.
- *Функции проверки орфографии.* Эти функции находят слова с ошибками и предлагают варианты их правильного написания.
- *Функции истории.* Получают, добавляют и удаляют записи в истории.
- *Интерактивные функции.* Эти функции предоставляют пользователю интерфейс для таких действий, как выбор файла.
- *Функции GUI.* Включают в себя три простые функции, возвращающие текущий шрифт, координаты *x* и *y* окна *GUI*.
- *Функции сервера Vim.* Эти функции поддерживают связь с (возможно) удаленным сервером Vim.
- *Функции размера и позиции окна.* Эти функции получают информацию об окне и позволяют сохранять и восстанавливать «виды» окна.
- *Прочие функции.* Это разнообразные другие функции, которые не вписываются ни в одну из предыдущих категорий. Они включают такие функции, как `exists()`, которая проверяет существование элемента в Vim, и `has()`, проверяющая, поддерживает ли Vim определенную функцию.

Ресурсы

Мы надеемся, что смогли заинтересовать вас начать работать со сценариями Vim и предоставили для этого достаточно информации. На эту тему можно было бы написать целую книгу, однако существуют другие ресурсы, которые могут быть полезными.

Хорошей отправной точкой будет изучение блока про сценарии на официальном сайте Vim (<https://www.vim.org/scripts/index.php>). Здесь вы найдете более двух тысяч сценариев, доступных для загрузки. Весь пакет документов доступен для поиска, и вы можете оценить сценарии и даже внести свой вклад, создав собственные.

Также хотим напомнить о бесценной встроенной справке Vim. Мы рекомендуем ознакомиться со следующими разделами справки:

```
help autocmd
help scripts
help variables
help functions
help usr_41.txt
```

И не забывайте о бесконечном множестве сценариев Vim в исполняемом каталоге Vim. Все файлы с расширением `.vim` являются сценариями, что делает их прекрасной платформой для изучения кода на примерах.

Пробуйте! Это лучший способ научиться.

Прочие полезные возможности Vim

В главах 8–12 были рассмотрены мощные функции и техники Vim, которые, по нашему мнению, помогут вам эффективно использовать редактор. В этой главе вы узнаете о проверке орфографии в реальном времени (с предлагаемыми исправлениями), о редактировании двоичных файлов и управлении состоянием вашего сеанса Vim. Здесь вы найдете некоторые функции, которые не вошли в предыдущие разделы, а также идеи и философию редактирования в Vim и некоторые интересные факты Vim (хотя и предыдущие главы тоже были интересными!).

Напишите это по буквам (э-т-о)

Проверка орфографии Vim отличается своей скоростью и гибкостью. Вместо использования внешнего модуля `vimspell` Vim предлагает встроенные функции проверки. Чтобы узнать больше об этом, прочтите файл справки `spell.txt` или введите команду `ex`:

```
:help spell
```

По умолчанию проверка орфографии в Vim отключена. Включите данную функцию с помощью команды `ex`:

```
:setlocal spell
```

и задайте области проверки орфографии:

```
:setlocal spelllang=en_us
```

Vim помечает слова как «плохие» не из-за того, что они неправильно написаны, а из-за того, что они, например не начинающиеся с заглавной буквы в начале предложения, являются «редкими» (неизвестными программе) или «местными» (региональными) словами. Отметим, что считается ли слово «плохим» или «хорошим», зависит от контекста и предпочтений пользователя, а не от правильности

написания. На рис. 13.1 показан пример того, как Vim выделяет «плохие» и «заглавные» помеченные слова, а на рис. 13.2 — пример выделенных слов.

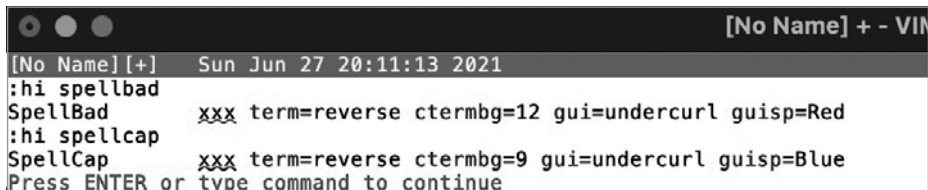


Рис. 13.1. Выделение синтаксиса проверки орфографии Vim (MacVim, цветовая схема: zellner)

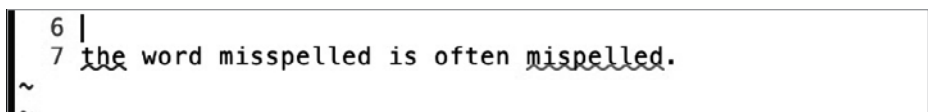


Рис. 13.2. Выделенные «плохие» слова

После завершения проверки орфографии можно перейти к следующему или предыдущему «плохому» слову с помощью команд командного режима `vi` — `]`s и `[`s соответственно. Если курсор расположен на «плохом» слове, команда `vi z` предоставит список возможных замен слова с их номерами. Введите соответствующий номер и нажмите клавишу `Enter` для замены. А если вы используете графический интерфейс или мышь, выберите слово, которое нужно заменить, щелкнув на нем. Чтобы отменить операцию, нажмите `Esc` или `Enter`, не выбирая слово.

Vim управляет орфографическим списком, загружая файлы закодированных слов в память и применяя два основных алгоритма для выявления орфографических ошибок: быстрый и медленный. Вы можете выбрать один из них. *Быстрая* проверка орфографии предполагает, что неправильно написанные слова будут точно соответствовать правильно написанному слову и что ошибка, скорее всего, заключается в перестановке символов или пропущенном символе. Эти ошибки считаются «небольшим отклонением» от правильного написания, и, следовательно, алгоритм эффективен и быстр.

Медленный алгоритм, напротив, предполагает, что слово может существенно отличаться от правильного написания. Например, вы могли напечатать слово, опираясь на его произношение, не зная, как оно на самом деле правильно пишется.

Если вы пишете грамотно, то проверка ваших ошибок, скорее всего, подойдет для быстрого алгоритма, который Vim рекомендует использовать для достижения

максимальной эффективности. Чтобы выбрать предпочитаемый метод проверки орфографии, установите значения параметра Vim `spellsuggest` на `fast` или `double`¹.

В табл. 13.1 мы приводим список самых распространенных команд командного режима `vi`.

Таблица 13.1. Распространенные команды проверки орфографии командного режима Vim

Команда	Действие
<code>]s</code>	Перемещает курсор к следующему вхождению неправильно написанного слова
<code>[s</code>	Перемещает курсор к предыдущему вхождению неправильно написанного слова
<code>zg</code>	Добавляет слово под курсором в список «хороших» слов
<code>zG</code>	Добавляет слово под курсором в список «хороших» слов в <code>internal-wordlist</code> (смотрите справку Vim). Добавленные в <code>internal-wordlist</code> слова считаются переходными и удаляются при выходе из Vim
<code>zw</code>	Добавляет слово под курсором в список «плохих» слов. Если это слово находится в списке «хороших» слов, оно будет оттуда удалено
<code>zW</code>	Добавляет слово под курсором в список «плохих» слов <code>internal-wordlist</code> . Как и в случае с <code>zG</code> , это добавление будет удалено при выходе из Vim
<code>[число] z=</code>	Отображает список вариантов для замены «плохих» слов в виде пронумерованного списка, а вы выбираете замену, вводя соответствующее <i>число</i> . Если ввести число перед <code>z=</code> , Vim автоматически заменяет «плохое» слово на соответствующее <i>числу</i>

Небольшое примечание:

- параметр `wrapscan` относится к командам `]s` и `[s`. То есть если нет больше неправильно написанных слов между текущим местоположением и концом буфера (или началом, в зависимости от направления команды), то курсор не перемещается;
- Vim разделяет слова на «хорошие» и «плохие», а не на «правильно написанные» и «неправильно написанные», так как некоторые термины могут быть правильными в контексте, но не являться реальными словами;
- команды `zg` и `zG` добавляют слова в список «хороших» или «плохих» слов в файл, который определяется параметром Vim `spellfile`. Это позволяет отделить данные слова от более глобальных файлов орфографии Vim;
- команда `z=` отображает список вариантов замены «плохого» слова, и вы можете выбрать вариант, введя соответствующий номер или щелкнув кнопкой мыши

¹ Есть еще третий метод, `best`, который, согласно Vim, лучше всего подходит для текста на английском языке, но самым быстрым все равно является `fast`.

на слове в графическом интерфейсе. Документация Vim подсказывает, что наиболее распространенным числовым префиксом для этой команды является 1, предполагая, что первый вариант, скорее всего, заменит «плохое» слово.

В табл. 13.2 приведен список команд `ex` с описанием их действия.

Таблица 13.2. Распространенные команды Vim `ex` для проверки орфографии

Команда	Действие
<code>:<i>[n]</i>spellgood слово</code>	Добавляет <i>слово</i> в список «хороших» слов. Если перед командой указано число <i>n</i> , она добавляет <i>слово</i> в <i>n</i> -й файл из списка файлов, определенного параметром <code>spellfile</code> . Если соответствующего <i>n</i> -го файла не существует (например, число равно трем, но параметром Vim <code>spellfile</code> определены только два файла), Vim выдаст ошибку и слово не будет добавлено
<code>:spellgood! слово</code>	Добавляет <i>слово</i> в список «хороших» слов в <code>internal-wordlist</code> , который очищается после каждого сеанса
<code>:spellwrong слово</code>	Добавляет <i>слово</i> в список «плохих» слов
<code>:spellwrong! слово</code>	Добавляет <i>слово</i> в список «плохих» слов в <code>internal-wordlist</code> , который очищается после каждого сеанса

В документации Vim есть подробные инструкции не только для простой проверки неправильно написанных слов, но и для других задач. Например, можно определить свои собственные файлы слов, используя стартовые наборы, такие как слова из OpenOffice (см. раздел 3 «Формирование файла орфографии» в файле справки Vim `spell.txt`). Вы можете использовать готовые файлы, создавать собственные или использовать некоторое сочетание обоих вариантов. Обратитесь к документации Vim (`:help spell`), чтобы получить более подробные описания настройки пользовательских конфигураций орфографии и других доступных команд и параметров.

Тезаурус

Кроме проверки орфографии, Vim также предоставляет функцию завершения слов с помощью тезауруса. Не путайте это с проверкой орфографии. Дополнительную информацию о параметрах тезауруса можно найти в подразделе «Завершение по тезаурусу» в главе 11. Эта функция может быть интересной, соблазнительной, занимательной, привлекательной, красивой, мощной, любопытной, восхитительной, захватывающей, необычной, очаровательной, впечатляющей, интригующей, прекрасной, притягательной, провокационной, читабельной, освежающей, стимулирующей и поразительной. ☺

Редактирование двоичных файлов

Официально Vim, как и vi, является *текстовым* редактором, однако в случае необходимости он может использоваться для редактирования двоичных файлов.

Зачем вообще редактировать их? Разве они не являются таковыми по определенной причине и не создаются каким-либо приложением в четко определенном и специфичном формате?

Зачастую такие файлы создаются компьютеризированными или аналоговыми процессами и не предназначены для редактирования вручную. Например, цифровые камеры часто хранят изображения в формате JPEG — сжатом двоичном формате для цифровых фотографий. Хотя они и двоичные, но содержат четко определенные сегменты или блоки, в которых хранится стандартная информация (если соответствуют спецификации). Цифровые изображения в формате JPEG хранят метаданные изображения (время съемки, разрешение, настройки камеры, дата и т. д.) в зарезервированных блоках отдельно от сжатых данных изображения. Использование функции редактирования двоичных файлов в Vim может быть полезно для корректировки метаданных изображений, например для исправления поля `year` в созданном блоке, чтобы изменить дату создания изображения.



Хотя функция редактирования двоичных файлов Vim может быть полезна, важно учитывать потенциальные проблемы, связанные с редактированием таких файлов. Например, некоторые двоичные файлы содержат цифровые подписи или контрольные суммы для обеспечения целостности файла. Правка файлов может привести к нарушению целостности и сделать их непригодными для использования. Поэтому мы не рекомендуем редактировать двоичные файлы без особой необходимости.

На рис. 13.3 показан сеанс редактирования JPEG-файла. Обратите внимание на местоположение курсора на поле даты. Вы можете напрямую редактировать информацию об этом изображении, изменяя показанные здесь поля.

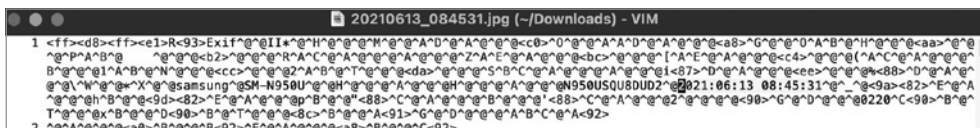


Рис. 13.3. Редактирование двоичного JPEG-файла

Для опытных пользователей, знакомых с особенностями двоичного формата, Vim может быть невероятно удобен для непосредственного внесения изменений,

которые в противном случае могли бы потребовать утомительного, повторяющегося обращения к другим инструментам.

ДВОИЧНОЕ РЕДАКТИРОВАНИЕ В ПОМОЩЬ

Один из нас столкнулся с ситуацией, когда функция двоичного редактирования в Vim спасла положение. Ему было поручено перенести старое приложение с несовременного компьютера на новый. Приложение частично состояло из множества классов Python (скомпилированных файлов .pyc). Однако при попытке перенести классы он обнаружил, что не существует исходного кода Python, и, следовательно, не было возможности перенести классы, скомпилировав их на новом компьютере.

На самом деле классы были исполняемыми на новом компьютере, но у них были встроенные старые, вышедшие из употребления имена и адреса. Ваш автор использовал функцию двоичного редактирования в Vim для изменения имен и адресов и обнаружил, что у всех старых имен хостов была та же длина строки, что и длина имен на новом компьютере. После простой массовой замены и сохранения классы Python стали великолепно работать в новой системе. Да, это было удачным совпадением, что у имен на старых и новых компьютерах была одинаковая длина. Тем не менее без Vim эта задача могла стать гораздо более сложной.

Существует два основных способа редактирования двоичных файлов в Vim. Вы можете установить параметр `binary` из командной строки Vim:

```
:set binary
```

или запустить Vim с помощью параметра `-b`.

Чтобы упростить редактирование двоичных файлов и предостеречь Vim от повреждения целостности файлов, Vim устанавливает следующие параметры.

- Значение параметров `textwidth` и `wrapmargin` устанавливается равным нулю, чтобы удерживать Vim от вставки ложных последовательностей новой строки в файл.
- Параметры `modeline` и `expandtab` сброшены (`nomodeline` и `noexpandtab`), чтобы удерживать Vim от расширения табуляций до пробелов `shiftwidth` и не позволить ему интерпретировать команды в `modeline`. Это может предотвратить неожиданные и нежелательные побочные эффекты.



Во время использования двоичного режима будьте осторожны при перемещении от одного окна к другому или от буфера к буферу. Vim использует события входа и выхода для установки и изменения параметров переключения буферов и окон, и вы можете запутать его, удалив некоторые из только что перечисленных защит. При редактировании двоичных файлов мы рекомендуем использовать однооконные и однобуферные сеансы.

Диграфы: символы, отличные от ASCII

Вы утверждаете, что «*Мессию*» сочинил George Frideric *Händel*, а не George Frideric *Handel*? Считаете, что *résumé* немного лучше передает мысль, чем *resume*? Используйте диграфы Vim для ввода специальных символов.

Даже текстовым файлам на английском языке иногда требуются специальные символы, особенно при упоминании глобализированного мира. Текстовым файлам на других языках требуется еще больше специальных символов.

Традиционно термин «*диграф*» описывает двухбуквенное сочетание, которое представляет собой один фонетический звук, например, *ph* в слове *digraph* или *phonetic*. Vim заимствует понятие двухбуквенного сочетания для описания своего механизма ввода специальных символов, особенно для ударений или таких знаков, как умляут в *ä*. Эти специальные знаки правильно называть *диакритическими знаками*, и Vim для их создания использует диграфы.

Vim позволяет вводить диакритические знаки различными способами, два из которых достаточно просты и интуитивны: префиксный символ (**Ctrl+K**) и клавиша **Backspace** между двумя символами клавиатуры. (Другие методы больше подходят для ввода символов по их необработанным числовым значениям в различных системах исчисления. Несмотря на свою эффективность, эти методы не поддаются простой мнемотехнике для диграфов.)

Первый метод ввода диакритических знаков представляет собой последовательность из трех символов: **Ctrl+K**, базовой буквы и знака пунктуации, указывающего на добавляемое ударение или знак. Например, чтобы набрать *с* с седилом (*ç*), введите **Ctrl+K C ,**. А для *а* с гравом (*à*) — **Ctrl+K a !**.

Греческие буквы набираются путем ввода соответствующей латинской буквы, сопровождаемой звездочкой (к примеру, введите **Ctrl+K P *** для нижнего регистра *π*). Русские буквы набираются путем ввода соответствующей латинской буквы, сопровождаемой знаком равенства (=) или, в некоторых местах, знаком процента (%). Для ввода перевернутого знака вопроса (*¿*) используйте **Ctrl+K ? Shift+I**, а для немецкой долгой *ß* (*ß*) — **Ctrl+K S S**.

Чтобы использовать второй метод Vim, установите параметр диграфа:

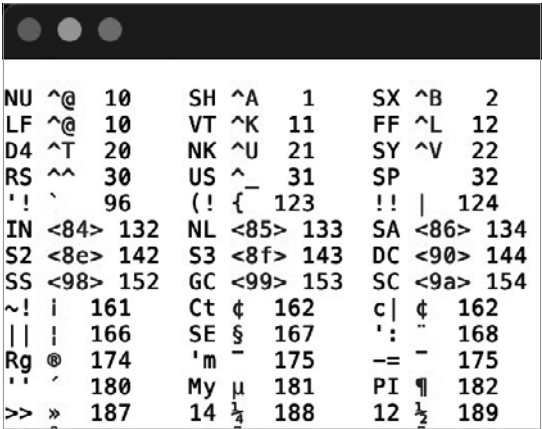
```
:set digraph
```

Теперь вы можете создавать специальные символы, вводя первый символ двухсимвольного сочетания, затем **Backspace**, а после этого — знак пунктуации, который задает диакритический знак. Таким образом, чтобы получить *ç*, введите **C Backspace ,** и **A Backspace !** для *à*.

Установка параметра `digraph` не мешает пользоваться методом `Ctrl+K`. Однако, если вы печатаете не очень уверенно, рекомендуется использовать *только* метод `Ctrl+K`, чтобы избежать случайного ввода диграфов.

Используйте команду `:digraph`, чтобы просмотреть все установки по умолчанию; более подробное описание можно получить с помощью команды `:help digraph-table`. На рис. 13.4 показана часть списка из команды `:digraph`.

На дисплее каждый диграф представлен тремя столбцами. Несмотря на то что изображение на дисплее может выглядеть запутанным из-за большого количества сочетаний в каждой строке, первый столбец отображает двухсимвольное сочетание, второй столбец — диграф, а третий — десятичное значение Юникода для диграфа.



NU	^@	10	SH	^A	1	SX	^B	2
LF	^@	10	VT	^K	11	FF	^L	12
D4	^T	20	NK	^U	21	SY	^V	22
RS	^^	30	US	^_	31	SP		32
'!	`	96	(!	{	123	!!		124
IN	<84>	132	NL	<85>	133	SA	<86>	134
S2	<8e>	142	S3	<8f>	143	DC	<90>	144
SS	<98>	152	GC	<99>	153	SC	<9a>	154
~!	i	161	Ct	¢	162	c	¢	162
	i	166	SE	§	167	':	"	168
Rg	@	174	'm	-	175	-=	-	175
''	'	180	My	μ	181	PI	¶	182
>>	»	187	14	¼	188	12	½	189

Рис. 13.4. Диграфы Vim (MacVim, цветовая схема: zellner)

Для вашего удобства приводим в табл. 13.3 знаки пунктуации, используемые в качестве последнего символа в последовательности для ввода часто используемых ударений и знаков.

Таблица 13.3. Как вводить ударения и прочие знаки

Знак	Пример	Символ, вводимый как часть диграфа
Акут	fiancé	Апостроф (')
Знак краткости	publică	Левая круглая скобка (()
Гачек	Dubček	Знак «меньше» (<)

Знак	Пример	Символ, вводимый как часть диграфа
Седиль	français	Запятая (,)
Циркумфлекс, или карет	português	Знак «больше» (>)
Гравис	voilà	Восклицательный знак (!)
Макрон	âtġmā	Дефис (-)
Черта	Søren	Слеш (/)
Тильда	señor	Вопросительный знак (?)
Умляют или диерезис	Noël	Двоеточие (:)

Редактирование файлов в других местах

Благодаря плавной интеграции протоколов сети Vim позволяет вам редактировать файлы на удаленных компьютерах, как если бы они были локальными! Если вы просто укажете URL имени файла, Vim откроет его в окне и запишет внесенные вами в него изменения в удаленную систему. (Это зависит от ваших прав доступа.) К примеру, следующая команда редактирует сценарий оболочки, принадлежащий пользователю `elhannah` в системе `flavorit1z`. Удаленный компьютер предлагает протокол безопасности SSH на 122-м порте (это нестандартный порт, предоставляющий дополнительную безопасность за счет скрытности):

```
$ vim scp://elhannah@flavorit1z:122//home/elhannah/bin/scripts/manageVideos.sh
```

Поскольку мы редактируем файл в домашнем каталоге `elhannah` на удаленном компьютере, мы укоротили URL, используя простое имя файла. Оно интерпретируется как имя пути относительно домашнего каталога пользователя на удаленном компьютере:

```
$ vim scp://elhannah@flavorit1z:122/bin/scripts/manageVideos.sh
```

Проанализируем полный URL, чтобы вы узнали, как создавать URL для вашего конкретного окружения.

- `scp` — первая часть до двоеточия представляет собой транспортный протокол. В данном примере используется `scp` — протокол копирования файлов, построенный на протоколе безопасной оболочки (SSH). Последующее `:` обязательно.
- `//` — вводит информацию о хосте, которая для большинства транспортных протоколов принимает форму `[пользователь@]имя_хоста[:порт]`.

- `elhannah@` — это не обязательно. Для протоколов безопасности, таких как `scp`, он указывает, от имени какого пользователя входить на удаленный компьютер. Если элемент опускается, по умолчанию используется ваше имя на локальном компьютере. Когда вам предлагается ввести пароль, вы должны ввести пароль пользователя удаленного компьютера.
- `flavoritlz` — это символическое имя удаленного компьютера, которое также можно указать в виде числового IP-адреса, например `192.168.1.106`.
- `:122` — это необязательный блок. Он указывает порт, на котором предоставляется протокол. Двоеточие отделяет номер порта от предшествующего ему имени хоста. Все стандартные протоколы используют общеизвестные порты, поэтому данный элемент URL можно опустить, если используется стандартный порт. В данном примере `122` *не* является стандартным портом для протокола `scp`, и, поскольку администратор системы `flavoritlz` выбрал предоставить услугу на порте `122`, эта спецификация обязательна.
- `//home/elhannah/bin/scripts/manageVideos.sh` — это файл, который мы хотим редактировать на удаленном компьютере. Мы начинаем с двух слешей, потому что задаем абсолютный путь. Для относительного пути или простого имени файла требуется только один слеш для отделения его от предшествующего ему имени хоста. Относительный путь относится к домашнему каталогу пользователя, от имени которого вы вошли в систему (в данном примере это относится к `/home/elhannah`).

Ниже представлен неполный список поддерживаемых протоколов.

- `ftp`: `u` `sftp`: — регулярный FTP и безопасный FTP.
- `scp`: — безопасное удаленное копирование через SSH.
- `http`: — передача файлов с помощью стандартного протокола браузера.
- `dav`: — относительно новый, но популярный открытый стандарт для передачи по сети.
- `rcp`: — удаленное копирование. Обратите внимание, что этот протокол небезопасен: *никогда не используйте его*.

Того, что мы описали на данный момент, достаточно, чтобы разрешить удаленное редактирование, но процесс может быть не таким прозрачным, как локальное редактирование файла: из-за временной необходимости переместить данные из удаленного хоста вам может быть предложено ввести пароли для выполнения работы. Это может быть утомительным, если вы привыкли периодически сохранять ваш файл на диск во время редактирования, поскольку каждая «запись» прерывается, чтобы запросить пароль для завершения транзакции.

Все транспортные протоколы из предыдущего списка позволяют настроить службу таким образом, чтобы разрешить вам доступ без пароля, но детали разнятся. Обратитесь к документации для получения подробностей о специфических протоколах и конфигурациях¹.

Навигация по каталогам и их изменение

Если вы часто пользуетесь Vim, то, возможно, заметили, что можете просматривать каталоги и перемещаться по ним с помощью клавиш, аналогичных тем, которые применяются в файлах.

Рассмотрим каталог, содержащий две папки репозитория: `/home/elhannah/.git/vim` (это два разных каталога git). Отредактируйте `/home/elhannah/.git/vim` таким образом:

```
$ vim /home/elhannah/.git/vim
```

На рис. 13.5 показано примерно то, что вы могли бы увидеть.



Рис. 13.5. «Редактирование» Vim каталога vim
(WSL Ubuntu Linux, цветовая схема: zellner)

Vim отображает три типа информации: вводные комментарии (заключенные в знаки равенства), каталоги (отображаемые с помощью завершающего слеша) и файлы. Каждый каталог или файл расположен на своей собственной строке.

Существует множество способов использования этой функции, но, приложив незначительные усилия, вы сможете быстро и интуитивно пользоваться стандартными

¹ Мы проверили и успешно настроили удаленный доступ через протокол scp: для редактирования удаленных файлов без необходимости ввода пароля. Установка scp: (как и других протоколов) выходит за рамки нашей книги, но мы считаем данный протокол эффективным способом удобного и прозрачного удаленного редактирования.

командами перемещения Vim (например, `w` для перехода к следующему слову, `j` или стрелка вниз для перемещения вниз на одну строку) и щелчками кнопкой мыши на элементах. Ниже представлены некоторые особенности режима каталога.

- Когда курсор располагается на имени каталога, нажатие клавиши `Enter` перемещает в этот каталог.
- Если курсор находится на имени файла, нажатие клавиши `Enter` открывает данный файл для редактирования.



Если вы хотите сохранить окно каталога для работы в этом же каталоге позднее, можете отредактировать файл под курсором, введя `o`. Vim разделит окно на две части и откроет файл во вновь созданном окне. Также можно использовать данный способ для перемещения в другой каталог: если курсор находится на имени каталога, Vim разделит окно и «отредактирует» каталог, в который вы переместились в новом окне.

- Вы можете удалять и переименовывать файлы и каталоги с помощью сочетания клавиш `Shift+R`. Возможно, немного нелогично, но Vim создает приглашение командной строки, с помощью которого вы выполняете переименование. Это должно выглядеть примерно как на рис. 13.6.

Чтобы завершить переименование, отредактируйте второй аргумент командной строки.

Удаление файла происходит аналогичным образом: поместите курсор на имя файла, который вы хотите удалить, и нажмите `Shift+D`. Vim отобразит диалоговое окно и попросит подтвердить удаление. Как и в случае с функцией переименования, Vim запрашивает подтверждение в области командной строки экрана.

```

appa.asciidoc
learning-the-vi-and-vim-editors-8e-[R] Tue Jun 29 10:41:25 2021 0x78 line:10, col:1 Top type:[netrw]
Moving /home/elhannah/.git/vim/oreilly/learning-the-vi-and-vim-editors-8e/xyzy.txt to : /home/elhannah/.git/vim/oreilly
/learning-the-vi-and-vim-editors-8e/xyzy.txt

```

Рис. 13.6. Приглашение переименовать в «редактируемом» каталоге (WSL Ubuntu Linux, цветовая схема: zellner)

- Одним из преимуществ редактирования каталогов является быстрый доступ к файлам с помощью функции поиска Vim. Например, чтобы быстро перейти к файлу `ch12.asciidoc` в каталоге `/home/elhannah/.git/vim/oreilly/learning-the-vi-and-vim-editors-8e` и отредактировать его, вы можете осуществить поиск части или всего имени файла. В данном случае попробуйте просто найти число 12:

`/12`

и, когда курсор окажется на имени файла, нажмите `Enter` или `O`.



Когда вы прочтете онлайн-справку по редактированию каталогов, вы увидите, что Vim описывает этот процесс как часть комплекса редактирования файлов с сетевыми протоколами. Однако мы выделили эту тему в отдельный раздел, чтобы она не потерялась в большом объеме деталей по редактированию сетевых протоколов.

Резервные копии в Vim

Vim предоставляет возможность создания резервных копий файлов, чтобы предотвратить потерю данных в случае непреднамеренной ошибки во время редактирования.

Для управления резервными копиями существует несколько параметров: `backup`, `writebackup`, `backupskip`, `backupcopy`, `backupdir` и `backupext`. Последние четыре управляют тем, где и как создаются резервные копии.

Если оба параметра `backup` и `writebackup` выключены (то есть `nobackup` и `nowritebackup`), то Vim не создает резервные копии файлов. Если `backup` включен, Vim удаляет любую старую резервную копию и создает новую для текущего файла. Если `backup` выключен, а `writebackup` включен, Vim создает файл резервной копии на протяжении сеанса редактирования и удаляет его после завершения сессии.

Параметр `backupdir` позволяет указать каталог или несколько каталогов (через запятую), где будут храниться файлы резервных копий. Например, если вы хотите, чтобы резервная копия всегда создавалась во временном каталоге вашей системы, установите `backupdir` в `C:\TEMP` для Windows или в `/tmp` для GNU/Linux.



Если вы хотите, чтобы резервная копия вашего файла всегда создавалась в текущем каталоге, вы можете указать «.» (точку) в качестве каталога резервной копии. Вы также можете попробовать сначала создать резервную копию в скрытом подкаталоге, если он существует, а затем в текущем каталоге, если скрытого подкаталога не существует. Чтобы сделать это, установите значение `backupdir` равным чему-то вроде `./mybackups.` (точка в конце обозначает текущий каталог файла). Этот гибкий параметр поддерживает множество стратегий для определения местоположений резервных копий.

Если вы хотите создавать резервные копии во время редактирования, но не для всех файлов, используйте параметр `backupskip`, чтобы определить разделенный запятыми список шаблонов. Vim не станет создавать резервные копии для файлов, соответствующих шаблонам из списка. Например, можно запретить создавать резервные копии для файлов, редактируемых в каталогах `/tmp` или `/var/tmp`, установив `backupskip` равным `/tmp/*,/var/tmp/*`.

По умолчанию Vim создает резервную копию с тем же именем файла, что и у исходного, и с суффиксом `~` (тильда). Это достаточно безопасный суффикс, потому что имена файлов, заканчивающиеся этим суффиксом, редки. С помощью параметра `backupext` вы можете изменить суффикс на другой. Например, если вы хотите, чтобы у ваших резервных копий было расширение имени файлов `.bu`, установите `backupext` равным строке `.bu`.

И наконец, параметр `backupcopy` определяет, *как* создается резервная копия. Мы рекомендуем установить этот параметр на `auto`, чтобы позволить Vim выбрать наилучший метод резервного копирования.

Преобразование текста в HTML

Возникала ли у вас когда-либо необходимость представить ваш код или текст группе? Пробовали ли вы когда-нибудь проанализировать код, используя чужую конфигурацию Vim? Подумайте о преобразовании вашего текста или кода в HTML и просмотре его из браузера.

Vim предоставляет три метода для создания HTML-версии вашего текста. Все они создают новый буфер с тем же именем, что и у исходного файла, но с расширением `.html`. Vim разбивает текущее окно на две части и отображает HTML-версию файла в новом окне.

- Метод «Преобразовать в HTML» в `gvim` является самым удобным, и он встроен в программу (описан в главе 9). Откройте меню **Syntax** в `gvim` и выберите **Convert to HTML**.
- *Сценарий* `2html.vim` — это базовый сценарий, вызываемый параметром меню **Convert to HTML**, описанным в предыдущем пункте. Вызовите его с помощью команды:

```
:runtime!syntax/2html.vim
```

Он не принимает диапазон, а преобразует весь буфер.

- *Команда* `tohtml` более гибка, чем сценарий `2html.vim`, потому что вы можете указать конкретный диапазон строк, который нужно преобразовать. К примеру, чтобы преобразовать строки буфера с 25-й по 44-ю, введите:

```
:25,44Tohtml
```



Поскольку дистрибутив Vim все еще включает `tohtml.vim` в каталоги дополнительных модулей (и автозагрузок), мы не можем успешно использовать эту функцию. Вместо нее лучше применять другие методы преобразования, которые действительно работают.

Одним из преимуществ использования `gvim` для HTML-преобразования является возможность точного определения цветов и создания соответствующих HTML-

директив. Эти методы также работают и в не-GUI-контексте, но результаты могут быть менее точными и полезными.



Только вам решать, как управлять вновь созданным файлом. Vim не сохраняет его для вас, он просто создает буфер. Мы рекомендуем разработать стратегию управления сохранением и синхронизацией HTML-версий ваших текстовых файлов. Например, вы могли бы создать некоторые автокоманды для запуска создания и сохранения ваших HTML-файлов.

Сохраненный HTML-файл можно просмотреть в любом веб-браузере. Некоторые люди могут не знать, как открыть файлы в локальной системе в своих браузерах. Однако это достаточно просто: практически все браузеры предлагают параметр меню **Открыть файл** в меню **Файл** и отображают диалоговое окно выбора файла, чтобы вы смогли перейти к папке, содержащей HTML-документ. Если вы собираетесь использовать эту функцию на регулярной основе, мы рекомендуем создать набор закладок для всех ваших файлов.

В чем же разница?

Иногда изменения между версиями файлов могут быть очень незначительными и едва уловимыми. Но есть инструмент, который может показать эти различия и сэкономить кучу времени. Этот инструмент называется **diff**. Он интегрирован в Vim из Unix и вызывается с помощью команды **vimdiff**.

Есть два способа вызова этой функции: в качестве отдельной команды или как параметр Vim:

```
$ vimdiff старый_файл новый_файл
$ vim -d старый_файл новый_файл
```

Как правило, первый файл в сравнении является старой версией, а второй — более новой, но это только по соглашению. На самом деле всегда можно поменять порядок.

На рис. 13.7 показан пример вывода **vimdiff**. Из-за ограниченности пространства мы уменьшили ширину и выключили параметр **wrap**, чтобы проиллюстрировать различия.

Хотя этот рисунок не может полностью передать визуальную составляющую в печатной книге, он дает представление о некоторых важных особенностях поведения.

- В строке 4 в левой части строки отображается слово **new**, которого нет в правой части. Подсвеченное (красным) слово указывает на различие между двумя строками. Аналогичным образом в строке 32 в правой части строки подсвечено слово **reflect**, которого нет слева.

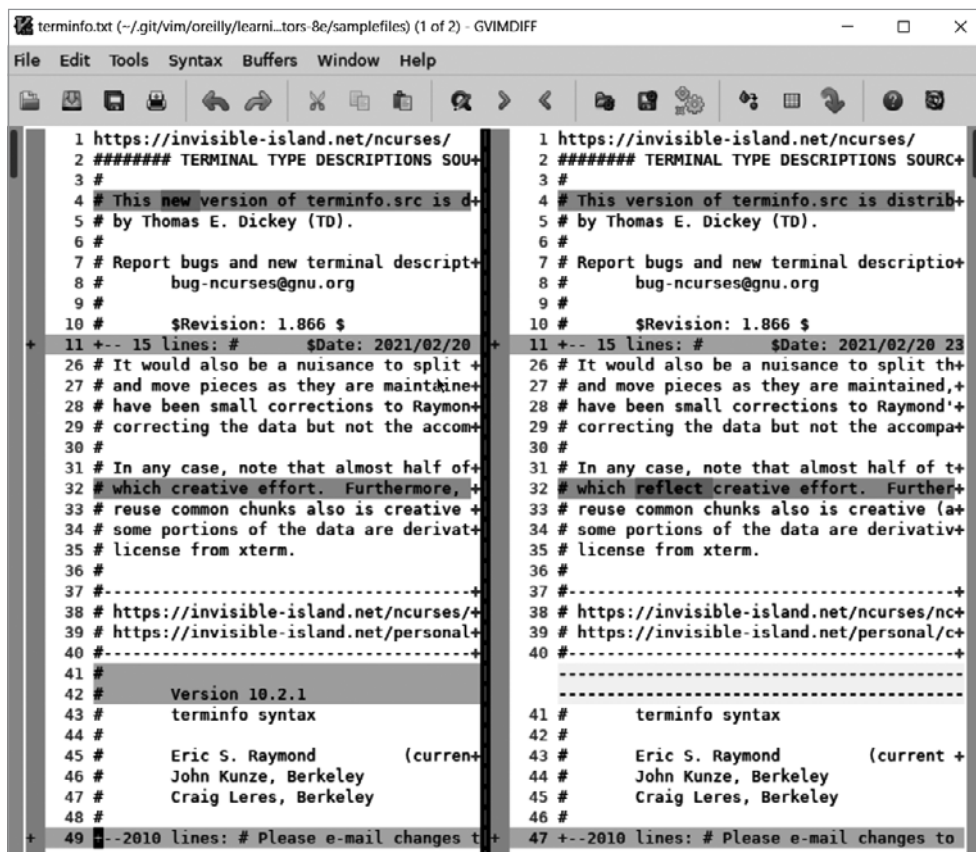


Рис. 13.7. Результаты vimdiff (WSL Ubuntu gvimdiff, цветовая схема: zellner)

- В обеих частях строки 11 Vim создал 15-строковое свертывание. Эти 15 строк в обеих частях файла идентичны, поэтому Vim свернул их, чтобы максимально увеличить полезную информацию на экране.
- Строки 41–42 слева подсвечены, а справа соответствующие позиции указаны в виде последовательности дефисов (-), что означает отсутствие строк. С этого момента нумерация строк различается, потому что в правой части на две строки меньше, но соответствующие строки в двух файлах все еще выстраиваются горизонтально.
- Строка 49 в левой части, соответствующая строке 47 справа, демонстрирует еще одно свертывание, но уже в 2010 строк, что означает, что оставшиеся 2010 строк в обоих файлах идентичны.

Функция vimdiff поставляется со всеми установками Vim в Unix-системах, потому что команда diff является стандартной для Unix. Отличные от Unix установки

Vim должны поставляться со своими версиями `diff`. Vim допускает замену команд `diff`, если они создают стандартные выходные данные `diff`.

Переменная `diffexpr` определяет выражение, заменяющее собой стандартное поведение `vimdiff`, и, как правило, реализуется в виде сценария, который работает со следующими переменными:

- `v:fname_in` — первый входной файл для сравнения;
- `v:fname_new` — второй входной файл для сравнения;
- `v:fname_out` — файл, фиксирующий выходные данные `diff`.

viminfo: итак, где же я остановился?

Большинство текстовых редакторов начинают сеанс редактирования файлов с первой строки первого столбца. То есть при каждом запуске редактора файл загружается и редактирование начинается с начала. Если вы часто редактируете один и тот же файл, продвигаясь по нему все дальше, то было бы удобнее начинать новую сессию с того места, где вы закончили в последний раз. Vim предоставляет такую возможность.

Существует два различных метода для сохранения информации сеанса для будущего использования: параметр `viminfo` и команда `:mksession`. В этом разделе мы рассмотрим оба приема.

Параметр viminfo

Vim использует опцию `viminfo`, чтобы определить, какую информацию о сеансе редактирования сохранять, как и где. Данная настройка представляет собой строку с разделенными запятыми подпараметрами, которые сообщают Vim, сколько информации и где сохранить. Вот некоторые из них.

- `<n` — указывает Vim сохранить строки для каждого регистра, максимум до *n* строк.



Если вы не присвоите никакого значения этому параметру, все строки будут сохранены. Хотя первоначально это может показаться логичным, задумайтесь, насколько часто вы редактируете очень большие файлы и вносите в них масштабные изменения. Например, если вы обычно редактируете файл из 10 000 строк и удаляете все строки, а затем сохраняете его, то все 10 000 строк сохранятся в файле `.viminfo` для этой записи. Если это продлевается многократно для нескольких файлов, то размер `.viminfo` сильно увеличится, что может вызвать длительные задержки при запуске Vim.

Рекомендуем указать некий разумный лимит. Мы используем 50.

- `/n` — определяет количество сохраняемых элементов истории шаблонов поиска. Если не указано, Vim использует значение из параметра `history`.
- `:n` — определяет максимальное число команд из истории командной строки, которое необходимо сохранить. Если не указано, Vim использует значение из параметра `history`.
- `'n` — определяет максимальное количество файлов, для которых Vim хранит информацию. Если вы определили параметр `viminfo`, этот подпараметр обязателен.

В файле `viminfo` сохраняется следующая информация:

- история командной строки;
- история строки поиска;
- история строки ввода;
- регистры;
- маркеры файлов (например, созданный `mx` маркер сохраняется, и вы можете переместить курсор к маркеру `x` при повторном редактировании файла);
- последние шаблоны поиска и замены;
- список буферов;
- глобальные переменные.

Этот параметр действительно удобен для поддержания непрерывности между сеансами. Например, если вы редактируете большой файл и меняете шаблон поиска, то шаблон запоминается так же, как и местоположение курсора в файле. Чтобы продолжить поиск в новом сеансе, вам нужно ввести `n` для перехода к следующему вхождению шаблона поиска.

Команда `mksession`

Команда `:mksession` в Vim сохраняет всю информацию о текущем сеансе. Параметр `sessionoptions` содержит разделенную запятыми строку, указывающую, какие компоненты сеанса необходимо сохранить. Этот способ сохранения информации сеанса более полный и специфичный, чем `viminfo`. Команда `mksession` сохраняет информацию обо всех файлах, буферах, окнах и т. д., которые были открыты в текущей сессии, так что сеанс можно восстановить полностью, включая все отредактированные файлы и настройки параметров, даже размеры окон.

Чтобы сохранить сеанс этим способом, введите:

```
:mksession [имя_файла]
```

где *имя_файла* указывает файл, в который нужно сохранить информацию сеанса. При последующем выполнении команды `source` с помощью файла сценария, созданного Vim, можно восстановить сеанс. Имя файла по умолчанию — `Session.vim`. Например, для сохранения сеанса с именем `mysession.vim` нужно выполнить команду:

```
:mksession mysession.vim
```

а для восстановления сеанса:

```
:source mysession.vim
```

Сравните это с параметром `viminfo`, который сохраняет и восстанавливает только информацию о редактировании для каждого файла.

Ниже представлено, что можно сохранить из сеанса, и приведены параметры в `sessionoptions` для сохранения:

- `blank` — пустые окна;
- `buffers` — скрытые и незагруженные буферы;
- `curdir` — текущий каталог;
- `folds` — вручную созданные свертывания, открытые/закрытые свертывания и параметры локальных свертываний;



Не имеет смысла сохранять свертывания, созданные не вручную. Автоматически созданные свертывания будут воссоздаваться автоматически!

- `globals` — глобальные переменные, которые начинаются с буквы верхнего регистра и содержат как минимум одну букву нижнего регистра;
- `help` — окно справки;
- `localoptions` — параметры, определенные локально для окна;
- `options` — параметры, установленные с помощью команды `:set`;
- `resize` — размер окна Vim;
- `sesdir` — каталог, в котором расположен файл сеанса;
- `slash` — обратные слешы в именах файлов, замещенные обычными слешами;
- `tabpages` — все страницы вкладок;



Если вы не укажете это в строке `sessionoptions`, только текущие вкладки сеанса будут сохранены как отдельный объект. Это обеспечивает вам гибкость при определении сеансов либо на уровне вкладки, либо глобально для всех вкладок.

- `unix` — формат конца строки Unix;
- `winpos` — местоположение окна Vim на экране;
- `winsize` — размер окон буферов на экране.

Например, если вы хотите сохранить сеанс вместе со всей информацией обо всех буферах, вкладках, глобальных переменных, параметрах, размерах и местоположении окон, вы должны определить параметр `sessionoptions` с помощью:

```
:set sessionoptions=buffers,folds,globals,options,resize,winpos
```

Какова длина моей строки?

Vim позволяет использовать строки практически неограниченной длины. Вы можете развернуть их на несколько строк экрана, чтобы увидеть их все без горизонтальной прокрутки, либо отобразить только начало каждой строки на одной строке и прокрутить вправо, чтобы увидеть скрытые части.

Если вы предпочитаете одну строку текста на строку экрана, выключите параметр `wrap`:

```
:set nowrap
```

С помощью `nowrap` Vim отображает столько символов, сколько позволяет ширина экрана. Экран можно представить как окно, через которое просматривается широкая строка. Например, строка из 100 символов содержит на 20 символов больше, чем может вместить экран шириной 80 столбцов. В зависимости от того, какой символ отображается в первом столбце экрана, Vim определяет, какие символы в 100-символьной строке не отображаются. То есть если первый столбец экрана — это пятый символ строки, то символы 1–4 находятся слева от видимой области и, следовательно, не отображаются. Символы 5–84 видны на экране, а символы с 85-го по 100-й расположились в правой части экрана и также невидимы.

Vim управляет тем, как отображается строка при перемещении курсора влево и вправо по длинной строке. Vim сдвигает строку влево и вправо минимум на `sidescroll` символов. Вы можете установить это значение следующим образом:

```
:set sidescroll=n
```

где `n` — это число столбцов для прокрутки. Мы рекомендуем установить `sidescroll` равным 1, потому что современные компьютеры обеспечивают достаточную вы-

числительную мощность для плавного смещения экрана на один столбец за раз. Если ваш экран замедлился и время отклика слишком долгое, увеличьте значение до более высокого, чтобы минимизировать количество обновлений на экране.

Значение `sidescroll` определяет *минимальное* количество сдвигов. Как, вероятно, вы и ожидали, Vim сдвигает строку достаточно далеко, чтобы завершить любую команду перемещения. Например, ввод `w` перемещает курсор к следующему слову в строке. Тем не менее в обработке перемещений Vim есть свои нюансы. Если следующее слово частично видимо (справа), Vim переходит к первому символу этого слова, но не сдвигает строку. Команда `w` сдвигает строку достаточно далеко влево, чтобы поместить курсор на первый символ следующего слова, но лишь настолько, чтобы отобразить первый символ.

Вы можете управлять этим поведением с помощью параметра `sidescrolloff`. Он определяет минимальное количество столбцов, которые должны располагаться справа и слева от курсора. Например, если вы установили значение `sidescrolloff` равным `10`, Vim сохранит как минимум десять символов контекста по мере приближения курсора к любому краю экрана. Теперь, когда вы перемещаетесь влево и вправо по строке, ваш курсор никогда не будет находиться ближе десяти столбцов от любого края экрана, поскольку Vim выводит на экран достаточное количество текста для сохранения этого контекста. Вероятно, это лучший способ настройки Vim в режиме `nowrap`.

Vim дает удобные визуальные подсказки с помощью параметра `listchars`, который определяет, как отображать символ при включенном `list`. Vim также имеет в этом параметре две настройки, которые определяют, следует ли для длинных строк использовать дополнительные символы, чтобы показать наличие символов за пределами правого или левого краев экрана. Например:

```
:set listchars=extends:>
:set listchars+=precedes:<
```

обязывают Vim отобразить `<` в первом столбце, если длинная строка содержит больше символов слева от видимого экрана, и `>` в последнем столбце, чтобы указать, что есть еще символы справа от видимого экрана. На рис. 13.8 показан пример.



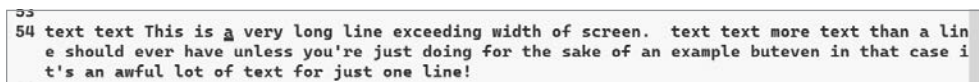
```
54 < a very long line exceeding width of screen. text text more text than a line should ever h>
```

Рис. 13.8. Длинная строка в режиме `nowrap` (WSL Ubuntu Linux, цветовая схема: `morning`)

Напротив, если вы предпочитаете, чтобы на экране отображалась вся строка целиком, укажите Vim переносить строки с помощью параметра `wrap`:

```
:set wrap
```

Теперь строка отображается, как показано на рис. 13.9.



```
54 text text This is a very long line exceeding width of screen. text text more text than a lin
e should ever have unless you're just doing for the sake of an example but even in that case i
t's an awful lot of text for just one line!
```

Рис. 13.9. Длинная строка в режиме wrap (WSL Ubuntu Linux, цветовая схема: morning)

Очень длинные строки, которые не полностью помещаются на экране, представляются в виде одиночного символа @ на первой позиции, пока курсор и файл не будут расположены так, чтобы строка могла влезть полностью. Строка из рис. 13.9 будет выглядеть как на рис. 13.10, если она находится в нижней части экрана.



```
53 $
@
@
@
```

Рис. 13.10. Индикатор длинной строки (WSL Ubuntu Linux, цветовая схема: morning)

Кроме того, в Vim есть возможность сделать символы пробелов видимыми. Это может быть полезно для быстрой визуальной проверки, когда пробелы являются важной частью текста. Для визуализации символов пробелов следует добавить точку в параметр `listchars`:

```
:set list
:set listchars+=space:.
```

Чтобы удалить ее и вернуть обычное отображение пробелов, выполните следующие команды:

```
:set list
:set listchars-=space:.
```

Сокращенные версии команд и параметров Vim

В Vim существует множество команд и параметров, поэтому мы рекомендуем начать с изучения их названий. Практически у всех команд и параметров (по крайней мере у тех, которые имеют более одного символа) есть соответствующая короткая форма. Это может сэкономить время, но *убедитесь*, что вы понимаете, что

сокращаете! Мы сталкивались с некоторыми неожиданными результатами при использовании сокращений, которые принимали за что-то другое.

Когда вы наберетесь опыта и определитесь со своими любимыми командами и параметрами Vim, использование сокращенных форм для них повысит вашу продуктивность. Обычно Vim использует Unix-подобные сокращения для параметров и допускает самую короткую уникальную исходную подстроку для сокращений команд.

Некоторые сокращения для общих команд включают:

```
n    next
prev previous
q    quit
se   set
w    write
```

Некоторые сокращения для общих параметров включают:

```
ai autoindent
bg background
ff fileformat
ft filetype
ic ignorecase
li list
nu number
sc showcmd (he showcase — нет такого параметра)
sm showmatch
sw shiftwidth
wm wrapmargin
```

Короткие формы команд и параметров экономят время, если вы хорошо их знаете. Однако при написании сценариев и настройке сеансов с помощью команд в файлах `.vimrc` или `.gvimrc` вы, скорее всего, сэкономите время, если будете использовать полные имена команд и параметров. В этом случае ваш конфигурационный файл и сценарий будет проще читать и отлаживать.



Обратите внимание, что это не тот подход, что используется с набором сценариев файлов Vim (`syntax`, `autoindent`, `colorscheme` и т. д.) в дистрибутиве Vim, хотя мы ничего не имеем против него. Мы просто рекомендуем для удобства управления вашими собственными сценариями придерживаться полных имен.

Несколько простых хитростей (не обязательно специфичных для Vim)

Ниже представлены несколько техник, которые могут быть полезны не только для Vim.

- *Быстрая перестановка.* Если вы ошиблись при вводе и ввели два символа в неправильном порядке, просто поставьте курсор на первый неверный символ и наберите `xr` (удалить символ, поместить символ). (Об этом мы упоминали ранее в пункте «Перестановка двух букв» подраздела «Перемещение текста» в главе 2.)
- *Другая быстрая перестановка.* Если вам нужно поменять две строки местами, поставьте курсор на верхнюю строку и введите `ddp` (удалить строку, поместить строку после текущей строки).
- *Быстрая справка.* Не забывайте о встроенной справке Vim. Быстрое нажатие функциональной клавиши **F1** разбивает экран на части и отображает введение в онлайн-справку. (Это относится к `gvim`. Если вы работаете в эмуляторе терминала, эта программа может взять на себя приоритет **F1**.)
- *Какую команду я только что использовал?* В своей простейшей форме Vim позволяет вам получить доступ к недавно выполненным командам с помощью клавиши стрелки в командной строке. Перемещаясь вверх и вниз с помощью клавиш стрелок, Vim отображает недавние команды, доступные для редактирования. Вне зависимости от того, будете ли вы редактировать команду в истории Vim, при нажатии **Enter** она выполнится.

Вы можете даже запустить встроенную команду редактирования истории, введя **Ctrl+F** в командную строку. Откроется небольшое «командное» окно (по умолчанию высота равна 7), в котором можно перемещаться с помощью обычных команд перемещения Vim. Здесь также доступен поиск, как в обычном буфере, и есть возможность вносить изменения.

В окне редактирования команд можно легко найти недавнюю команду, изменить ее при необходимости и выполнить, нажав **Enter**. Имеется даже возможность записать буфер в выбранный файл, чтобы сохранить историю команды для дальнейшего использования.



Более подробный пример использования окна истории командной строки в качестве инструмента вы найдете в пункте «Знакомство с окнами истории» подраздела «Удвойте удовольствие» в главе 14.

- *Немного юмора.* Попробуйте ввести команду:

```
:help "the weary"
```

и прочтите ответ Vim.

Другие ресурсы

Полезные онлайн-ресурсы включают HTML-преобразования встроенной справки Vim для Vim 7 (<http://vimdoc.sourceforge.net/html/doc/version7.html>) и Vim 8 (<https://vimhelp.org>)¹.

Кроме того, https://vimhelp.org/vim_faq.txt.html представляет собой список часто задаваемых вопросов по Vim. Вопросы и ответы разделены, но находятся на одной странице. Мы рекомендуем прокрутить до раздела с ответами и начать изучать с этого места.

На официальной странице Vim раньше размещались основные советы по Vim, но из-за проблем со спамерами администраторы перенесли советы в wiki (<http://vim.wikia.com/wiki/Category:Integration>), где спамом легче управлять.

¹ Спасибо Карло Тойбнеру, который занимается текущей HTML-документацией Vim.

Несколько продвинутых приемов работы с Vim

В этой главе мы поделимся опытом, который мы приобрели за много (слишком много?) лет изучения и использования Vim. Точная настройка некоторых установок по умолчанию и переназначение стандартных команд помогут превратить многочасовое ежедневное использование Vim в гораздо более приятное занятие. Мы надеемся, что эти идеи и методы вдохновят вас на новые свершения.

Несколько удобных переназначений

В командном режиме Vim столько действий и команд, что навряд ли какие-то клавиши доступны для свободного использования без изменения их поведения по умолчанию. К счастью, в большинстве случаев Vim выбирает правильное переназначение, и, хотя вы можете изначально быть согласны или не согласны с его выбором, вы практически во всех случаях быстро запомните используемые вами команды.

Мы выбрали некоторые удобные варианты, заменив переназначения, которые были либо бессмысленными, излишними и обслуживались более чем одной клавишей, либо были эффективнее при переназначении их на что-то более полезное.

Упрощенный выход из Vim

В разделе «Сохранение файлов и завершение работы» главы 5 мы научили вас завершать сеанс `vi` и Vim. Однако не у всех получается сделать это с первого раза. И в самом деле, вопрос «Как выйти из редактора Vim?» является одним из самых популярных на Stack Overflow (<https://stackoverflow.blog/2017/05/23/stackoverflow-helping-one-million-developers-exit-vim>), где он был задан более чем миллионом пользователей!

Мы с легкостью можем сократить три или четыре нажатия клавиш, необходимых для выхода из Vim, до всего лишь одного, используя следующие простые переназначения клавиш:

```
:nmap q :q<cr>
:nmap Q :q!<cr>
```

`:nmap` — эквивалент стандартной команды `ex :map`. Мы не будем разбирать все аналоги различных команд; вместо этого обратитесь к `:help :map-modes`.

Эти два переназначения позволят вам выходить из Vim либо аккуратно (**Q**), либо принудительно (**Shift+Q**) с помощью одного-единственного нажатия клавиши!

Изменение размера вашего окна

Изменять размер окна в любое время можно с помощью GUI. Однако, если вы предпочитаете пользоваться клавиатурой, а не мышью, попробуйте клавиши **_** (нижнее подчеркивание) и **+**. Хотя их использование избыточно¹ по сравнению с другими более легкодоступными клавишами, они мнемонически сопоставимы с «увеличить» и «уменьшить» соответственно².

Таким образом, мы переназначаем **_** для уменьшения размера персонализированного окна, а **+** — для увеличения. Добавьте следующие строки в ваш файл `.vimrc` либо введите их в качестве команды `ex`, чтобы изменения сохранились:

```
map _ :resize -1<CR>
map + :resize +1<CR>
```

Теперь вы можете легко уменьшить или увеличить окно. Это пригодится в нашем дальнейшем обсуждении окна истории команд Vim.

Удвойте удовольствие

У нас есть два любимых переназначения, которые, на наш взгляд, соответствуют философии Vim. То есть, когда управляющий символ в командном режиме `vi` удваивается, это обычно означает быстрый доступ к стандартному или

¹ Клавиша со знаком минус, по сути, избыточна для `k`, с той лишь небольшой разницей, что она помещает курсор на первый непустой символ, в то время как символ `+` совершенно избыточен для `Enter` и действительно может быть переназначен без какой-либо потери функциональности.

² Да, технически клавиши при нажатой `Shift` представляют нижнее подчеркивание и плюс, но важнее мнемоническая схема, в которой одна клавиша с символом минус расположена рядом с другой, у которой есть символ плюс. (Просто поясняем.)

интуитивному поведению. Например, `dw` удаляет слово, а удвоенное `d` (`dd`) удаляет текущую строку. Аналогично `yu` извлекает текущую строку.

Мы применяем эту философию для двух переназначений, делая функции Vim быстродоступными с помощью более интуитивных нажатий клавиш. Эти настройки активируют эффективные истории команды и поиска Vim, которые при активации появляются в новом горизонтально разделенном окне.

Знакомство с окнами истории

В Vim есть относительно малоизвестная функция, которая, по нашему мнению, является одной из мощнейших: окно командной строки. Vim хранит истории ваших команд `ex` и ваших шаблонов поиска. К ним можно получить доступ из окна командной строки Vim, которое появляется внизу вашего экрана в виде нового короткого окна. Обе истории хранятся отдельно и могут управляться отдельно.

Вы можете открыть окно командной строки или шаблона поиска двумя способами (для каждого). Стандартное поведение Vim использует `Ctrl+F` или `q:` (команда `vi` командного режима). Воспользуйтесь справкой Vim, чтобы получить дополнительную информацию об окне командной строки:

```
:help c_Ctrl-F
```

Чтобы закрыть любое окно Vim, используйте регулярную команду `:q`.

Мы вскоре обсудим некоторые классные вещи, которые можно сделать в этом окне. Но для начала сделаем процесс открытия окна более простым и интуитивным. (`Ctrl+F` и `q:` достаточно просты, но так ли они интуитивны?)

Итак, рассмотрим, как можно быстро открыть окно командной строки.

Два двоеточия лучше, чем одно

Придерживаясь принципа «удвоение *чего-либо* делает *что-то* усиленным и, надемся, интуитивным», мы решили, что хорошо было бы удвоить двоеточие, которое само по себе запускает команду `ex`. Мы предположили, что двойное двоеточие `::` будет «усиленной командой `ex`» при открытии окна командной строки.

Помните, что мы определяем переназначение и предполагаем, что оно вызывается в командном режиме. Чтобы избежать переназначения команды `ex`, мы используем команду `:noremap` и переназначаем `::` на последовательность с `q`. Однако для правильного переназначения `::` мы воспользуемся командой `:nnoremap` (за подробностями обратитесь к `:help :map-modes`):

```
:nnoremap :: q:
```

Теперь проверим, что команда работает, введя ее в диалоговом режиме или добавив ее в ваш файл `.vimrc`. Затем в командном режиме быстро напечатайте два двоеточия. Вы должны увидеть окно командной строки с курсором на последней строке. Если вы заходите из командного режима `vi`, это *всегда* будет пустая строка.

Почему? Существует два разных типа поведения. Если вы в процессе ввода команды `ex` и набираете `Ctrl+F`, Vim открывает окно истории командной строки в командном режиме `vi` с курсором, расположенным в конце последней строки, отображая неполную вводимую вами команду.

При вводе окна командной строки из командного режима `vi` нет выполняемой команды `ex`, поэтому курсор находится на пустой строке.

И два слеша лучше, чем один

Мы также считаем, что использование двойного слеша (`//`) для быстрой активации окна истории поиска командной строки является интуитивно понятным.

Кроме стандартных способов активации окна истории поиска — `q/` и `q?`, мы рекомендуем использовать `//` и `??` для быстрого доступа к истории поиска и обратного поиска соответственно. Аналогично использованию `::` для активации окна командной строки этот подход имеет логический смысл для просмотра истории шаблонов поиска Vim.

Обратите внимание, что, как и в предыдущем примере, мы предполагаем, что эти команды будут вызваны в командном режиме `vi`. Для того чтобы избежать возможных ошибок, рекомендуется использовать команду `:nnoremap`:

```
:nnoremap // q/  
:nnoremap ?? q?
```

КАКАЯ ВЫСОТА У ВАШЕГО ОКНА КОМАНДЫ?

Кроме того, высота окна командной строки Vim по умолчанию составляет семь строк. Однако мы считаем, что десять строк более удобны. Вы можете установить это значение в своем файле `.vimrc`, прописав:

```
set cmdwinheight = 10
```

Обратите внимание, что, если вы уже переназначили `_` и `+` для расширения и сжатия вашего окна, изменение размера окна командной строки будет также удобным для вас.

Теперь опробуем это. В командном режиме быстро наберите два слеша. Вы должны увидеть окно истории шаблонов с курсором на последней строке — это последний использованный вами шаблон поиска.

Имейте в виду, что Vim вставляет один символ в крайний левый столбец окна командной строки, указывающий на режим окна с помощью : для истории командной строки и / или ? для истории шаблона поиска.



Вы находитесь в окне команд (или шаблонов поиска), которое имеет свои особенности и ограничения. Некоторые команды ех недоступны, например :e, :grep, :help и :sort. Кроме того, невозможно перемещаться в другое окно, оставляя текущее открытым, с помощью комбинации клавиш **Ctrl+W** **Ctrl+W**. Хотя эти ограничения не умаляют эффективности окна командной строки, они подчеркивают, что оно предназначено для определенных целей.

Следует учитывать несколько моментов, связанных с буфером командной строки в Vim.

- Вы можете перемещаться по этому буферу, как по любому другому буферу Vim. Не стесняйтесь экспериментировать с вашими любимыми командами Vim:

```
:w имя_файла
```

Сохранить буфер в файл.

```
:r имя_файла
```

Передать файл в буфер командной строки.

- Вы можете записывать и сохранять содержимое буфера командной строки почти так же, как любой другой буфер.
- И наоборот, вы можете передавать файлы в буфер командной строки.

Последние два момента важны, поскольку они обеспечивают гибкость при сохранении истории командной строки, что может оказаться полезным в дальнейшем. Вы можете затем загрузить в сеанс файл, содержащий команды, и выборочно найти и выполнить ваши команды.

Переходим к более интересному

Теперь, когда мы освоили окно командной строки, давайте займемся другими интересными вещами.

Поиск сложно запоминаемой команды

Начнем мы с обсуждения различных способов поиска *той самой* команды, которую вам необходимо выполнить.

Прямой поиск команды

Вы знаете, что вы использовали уже много раз для сохранения команду Vim, но вы не помните, какая именно это была команда и когда. Вы только *помните*, что она была связана с преобразованием описаний *TEST* в *PROD*. Чтобы найти эту команду, вводите `::` и готовьтесь к поиску. Существует несколько подходов, но все они используют Vim для поиска других команд Vim.

Например, самый простой и прямой подход — это поиск в буфере истории команд командной строки. Вы помните, что использовали что-то с *TEST* и затем *PROD* в одной команде. Просто воспользуйтесь следующей записью:

```
?.*TEST..*PROD
```

и Vim поместит курсор на первую найденную строку, соответствующую этому регулярному выражению. Теперь вы можете снова выполнить эту команду, нажав **Enter**. Vim закроет окно командной строки и воспроизведет команду автоматически.

Если первое вхождение *не* то, что вы ищете, команда `vi n` переместит вас к следующему совпадению. Используйте `n` столько раз, сколько это необходимо, пока вы не найдете нужную команду.



Обратный поиск по командам в окне командной строки Vim учитывает наши настройки `wrapsap`¹. Если установлен `powtapsap` и в буфере нет вхождений заданного шаблона выше текущей строки (или нижней, в зависимости от направления поиска), Vim отобразит сообщение «поиск достиг верха...» (или низа), не найдя при этом ничего.

Фильтруем буфер

Возможно, по какой-то причине вам не нравится искать нужную команду в буфере командной строки способом из предыдущего раздела.

В этом случае вы можете отфильтровать или изменить содержимое буфера окна командной строки так же, как фильтруете текст в обычных окнах Vim. Предположим, вы хотите найти команды, содержащие слова *TEST* и *PROD*.

Вместо простого поиска в буфере можно очистить буфер, удалив все строки, *не* соответствующие заданным критериям, используя инвертированный глобальный поиск (`:vg`):

```
:vg/.*TEST..*PROD/d
```

Теперь в буфере остались *только* строки, содержащие нужные слова. Выберите нужную команду и выполните ее.

¹ Это также применимо для поведения в окне командной строки для шаблонов поиска.

Удаленные строки остаются удаленными (недоступными) до конца текущего сеанса редактирования. Однако при следующем открытии файла в Vim все удаленные строки будут восстановлены.

Манипулирование результатами фильтров

Vim позволяет не только искать и повторно выполнять команды из буфера истории, но и редактировать их перед повторным выполнением. Например, если вы хотите выполнить команды, *похожие* на те, что были совершены ранее, но с небольшими изменениями, вы можете использовать команды из буфера истории как основу.

Предположим, у вас есть набор команд Vim, которые вы уже выполняли, но теперь необходимо заменить одно слово на другое в каждой команде. Возвращаясь к предыдущему примеру, скажем, вместо преобразования *TEST* в *PROD* нам нужно преобразовать *PROD* в *QA*.

Как и раньше, начнем с поиска и, возможно, фильтрации вариантов команд в истории буфера, а затем заменим *PROD* на *QA* и *TEST* на *PROD* (предполагая, что это такое простое преобразование). После того как вы отредактировали команду в строке истории, просто нажмите **Enter**, чтобы выполнить ее. Все готово!

Анализ известной речи

Один из нас заинтересовался конкретной политической речью, которую много обсуждали и оспаривали. И у него возникла идея проанализировать расшифровку стенограммы речи и с помощью Vim отредактировать ее и отфильтровать содержание по частоте встречаемости слов.



Здесь использованы ссылки на очень известную и политически заряженную речь. Мы не намеревались делать выводы или пропагандировать идеологию. Это просто пример того, как один из нас быстро использовал Vim в качестве инструмента для анализа информации, которая обычно не рассматривается как вид редактирования, для которого можно было бы ожидать использование Vim.

Вы можете найти речь в репозитории книги на GitHub (<https://www.github.com/learning-vi/vi-files>) (см. раздел «Получение доступа к файлам» приложения В) в файле `book_examples/famous-speech.txt`.

Чтобы упростить процесс редактирования, автор использовал `awk`, постепенно разрабатывая команду и повторяя ее до достижения желаемого результата. Помните, что вы можете заменить диапазон строк в буфере выводом команды, выполненной в этом диапазоне. В данном случае в командном режиме `vi` автор ввел:

```
:%!awk 'END { print NR }'
```


Это заменило буфер количеством строк в нем. Это *не* то, что он хотел, но это было хорошим началом.

Следующие десять минут он провел, улучшая эту команду в окне истории командной строки Vim. Вот фрагмент кода, который он написал (номера строк добавлены для удобства, а длинные строки перенесены для лучшей читаемости):

```
1 1,$!awk '{ while (i = 1; i <= NF; i++) word[$i]++ } END { print word }'
2 1,$!/usr/bin/awk '{ while (i = 1; i <= NF; i++) word[$i]++ }
   END { print word }'
3 1,$!/usr/bin/awk '{ for (i = 1; i <= NF; i++) word[$i]++ }
   END { print word }'
4 1,$!/usr/bin/awk '{ for (i = 1; i <= NF; i++) word[$i]++ }
   END { for (words in word) print word[words], words }'
5 1,$!/usr/bin/awk '{ for (i = 1; i <= NF; i++) word[$i]++ }
   END { for (words in word) print word[words], words }'
6 1,$!/usr/bin/awk '{ for (i = 1; i <= NF; i++) word[$i]++ }
   END { for (words in word) print word[words], words }' | sort
7 1,$!/usr/bin/awk '{ for (i = 1; i <= NF; i++) word[$i]++ }
   END { for (words in word) print word[words], words }' | sort
8 wq
9 1,$!/usr/bin/awk '{ for (i = 1; i <= NF; i++) word[$i]++ }
   END { for (words in word) print word[words], words }' | sort -n
10 g/fight
11 1,$!/usr/bin/awk 'BEGIN { FS= "[, . ]+" } { for (i = 1; i <= NF; i++) word[$i]++ }
   END { for (words in word) print word[words], words }' | sort -n
12 g//
13 g/law
```



Вы могли заметить в предыдущем списке различные вызовы awk. Это побочный эффект от частых перемещений автора с одного компьютера и ОС на другой. Мы оставили видимыми варианты и рассчитываем, что вы используете один из них, который будет работать в вашем окружении.



Очень важно понимать, что команды, перечисленные в предыдущем списке, являются результатом повтора одной и той же команды. После завершения результатом будут эти команды в окне истории командной строки.

Например, после изменения строки 1 (перенесена):

```
1,$!awk '{ while (i = 1; i <= NF; i++) word[$i]++ }
   END { print word }'
```

на (перенесена)

```
1,$!/usr/bin/awk '{ while (i = 1; i <= NF; i++) word[$i]++ }
   END { print word }'
```

в следующий раз при посещении окна истории команд командной строки эти две строки будут последними непустыми строками в этом окне.

Строка 2 исправила строку 1, чтобы указать правильное местоположение `awk`. При выполнении (поскольку диапазон был указан как `1,$`) Vim заменил весь буфер на:

```
awk: cmd. line:1: { while (i = 1; i <= NF; i++) word[$i]++ } END { print word }
awk: cmd. line:1:          ^ syntax error
awk: cmd. line:1: { while (i = 1; i <= NF; i++) word[$i]++ } END { print word }
awk: cmd. line:1:          ^ syntax error
```

Что ж, это было плохо. К счастью, ввод и приводит к сбросу буфера и позволяет вернуться к исходной расшифровке речи¹.

Не забудьте ввести `::`, чтобы отредактировать вашу последнюю команду и привести ее в соответствие с примером выше. Строка 3 исправила одну синтаксическую ошибку, а строка 4 — еще одну. Это немного смущает.

Наконец, следующая строка (строка 5 перенесена, чтобы вместились на страницу):

```
1,$!/usr/bin/awk '{ for (i = 1; i<= NF; i++) word[$i]++ }
                  END { for (words in word) print word[words], words }'
```

выдает настоящие результаты! Первые несколько строк в буфере теперь выглядят примерно так:

```
3 weeks.
4 State
1 you've
1 written
25 you're
1 telephone
1 Congress
1 ever,
5 biggest
38 are
```

Теперь мы видим вывод строки для каждого слова в речи. Число перед каждым словом — количество его повторений. Это здорово, но немного неорганизовано. Поэтому давайте отсортируем результаты (строка 6 перенесена):

```
1,$!/usr/bin/awk '{ for (i = 1; i<= NF; i++) word[$i]++ }
                  END { for (words in word) print word[words], words }' | sort
```

Неплохо, но попробуем реализовать числовую сортировку... Добавьте параметр `-n` (числовая сортировка) в `sort` (мы теперь находимся на строке 9, перенесена):

```
1,$!/usr/bin/awk '{ for (i = 1; i<= NF; i++) word[$i]++ }
                  END { for (words in word) print word[words], words }' | sort -n
```

¹ Использование `u` упрощает повторение различных манипуляций в буферах. Как только вы разовьете мышечную память для этого, оно станет очень естественным.

Вот этот результат уже гораздо лучше. Последние несколько строк буфера теперь выглядят примерно так:

```
115 that
125 they
134 you
146 in
167 I
203 a
227 and
265 of
326 to
394 the
```

Неудивительно, что распространенное слово *the* встречается чаще всего.

Давайте выполним одну последнюю итерацию. Если вы заметили, в предпоследней итерации, которую мы только что рассматривали, к некоторым словам были все еще прикреплены знаки пунктуации. Это ведет к неправильному подсчету таких слов, как *car*, *car*, и *car.*, хотя это одно и то же слово. Итак, для последнего изменения давайте определим разделитель полей как регулярное выражение в правиле `awk BEGIN` (строка 11 снова перенесена):

```
1,$!/usr/bin/awk 'BEGIN { FS= "[, . ]+" } { for (i = 1; i<= NF; i++) word[$i]++ }
END { for (words in word) print word[words], words }' | sort -n
```

Обратите внимание на различия в подсчетах, указывающие на то, что наши результаты, вероятно, немного ближе к истине:

```
144 that
153 in
155 you
168 I
203 a
210
227 and
266 of
328 to
394 the
```



Не забудьте выполнять отмену (u) после каждого повтора, чтобы передать исходный текст следующей команде.

Наш пример основывался на использовании `awk` в качестве фильтра *du jour*, но в Unix и GNU/Linux есть множество мощных команд, которые при применении к буферу Vim дают идентичные по эффективности результаты. Мы оба *предпочитаем* `awk`, но также часто используем `sed`, `grep`, `wc`, `head`, `tail`, `sort` и многие другие,

не отдавая предпочтения какой-либо из них. Стоит отметить, что каналы прекрасно работают и увеличивают возможности управления буферами Vim.

Хотя данный пример может показаться надуманным, ваш автор действительно выполнил эти действия во время обсуждения упомянутой речи, а результаты были использованы для разрешения «спора». Время, затраченное на итерацию от изначальной команды `awk` до окончательной, усовершенствованной команды, заняло всего несколько минут. Результаты стали пробой пера в жаркой дискуссии о том, *что* было в речи. Мы не высказываем своего мнения по этому поводу, но подчеркиваем, что это *было* продуктивное и полезное упражнение в социальной среде. Да, возможно, Vim не является типичным участником общественных собраний, но, может быть, может им стать. ☺

Еще несколько случаев использования

Как уже упоминалось, хоть предыдущий пример и кажется надуманным, он один из многих, где мы манипулировали файлами для извлечения полезной информации, не прибегая к внешним инструментам. Другие примеры включают:

- **экспортированные файлы.** Один из нас отслеживает свои упражнения с помощью Garmin™ (<https://www.garmin.com/en-US>). Файлы экспорта Garmin являются текстовыми файлами (рис. 14.1). Подумайте, как вы могли бы извлечь информацию аналогично нашему примеру;

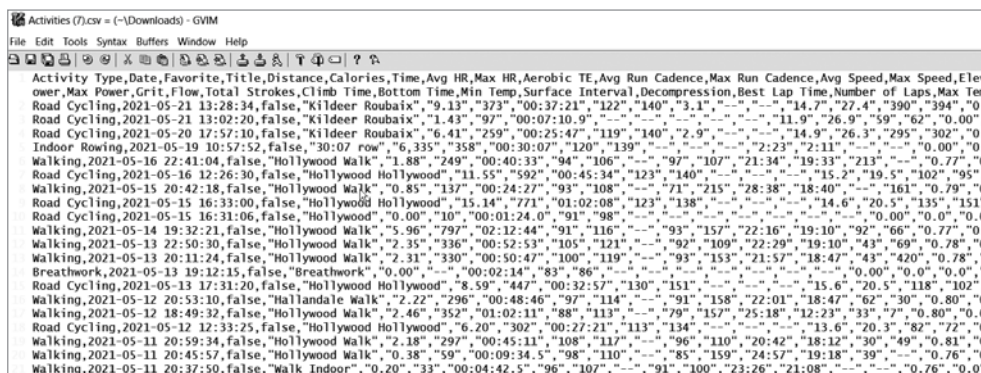


Рис. 14.1. Пример файла Garmin CSV

- **файлы системного журнала.** Мы использовали ту же самую технику для извлечения, манипулирования и форматирования файлов системного журнала Unix (включая `/var/log/messages`). Хотя существует множество инструментов для мониторинга и анализа этих файлов (такие как Splunk, <https://www.splunk.com>), иногда достаточно просто открыть Vim и быстро применить настроенные команды из окна истории командной строки;

- *файлы журналов поставщиков.* Их фильтрация аналогична фильтрации файлов системного журнала.



Лишний раз стоит напомнить, что установка большого количества сохранений истории команд Vim может быть полезной. Для этого нужно определить параметр `viminfo` в вашем конфигурационном файле `.vimrc`:

```
" example
set viminfo='50,:1000
```

Если вы редактируете команду в буфере истории командной строки, а контекст редактирования может предложить завершение, нажатие клавиши **Tab** отображает меню завершения, которое можно просматривать с помощью **Tab** и клавиш со стрелками. Нужное совпадение выбирается клавишей **Enter**, но будьте осторожны, так как это незамедлительно запускает выполнение измененной команды. На рис. 14.2 показано завершение для неполных сопоставлений параметров, а на рис. 14.3 представлен пример завершения имени файла. (Более подробно о завершении командной строки можно прочитать в разделе «Встроенная справка» главы 8.)

```
sysctl.conf[R0] darkblue 37:48 2021 0x23 line:77, col:1 Bot type:[sysctl]
colorscheme default
hi StatusLine delek
hi StatusLine desert ite
190 colorscheme delek
help :highlight
hi StatusLineTerm guibg=cyan
hi StatusLine guibg=cyan
[Command Line] Thu Jun 3 10:37:53 2021 0x0 line:190, col:14 91% type:[vim]
-- Command-line completion (sysctl) --
```

Рис. 14.2. Примерный список завершений для команды, требующей аргумента (в данном случае цветовая схема)

```
# /etc/sysctl.d/10-console-messages.conf nks under certain conditions
# /etc/sysctl.d/10-ipv6-privacy.conf cted)
# /etc/sysctl.d/10-kernel-hardening.conf ion/sysctl/fs.txt
#f /etc/sysctl.d/10-link-restrictions.conf
#f /etc/sysctl.d/10-lxd-inotify.conf
/etc/sysctl.d/10-magic-sysrq.conf
/etc/sysctl.d/10-network-security.conf
/etc/sysctl.d/10-pttrace.conf
ysctl /etc/sysctl.d/10-zero-page.conf 0x23 line:77, col:1 Bot type:[sysctl]
c /etc/sysctl.d/99-cloudimg-ipv6.conf
h /etc/sysctl.d/99-sysctl.conf
h /etc/sysctl.d/README
190 e /etc/sysctl.d/10-pttrace.conf
help :highlight
hi StatusLineTerm guibg=cyan
hi StatusLine guibg=cyan
[Command Line] Thu Jun 3 10:41:39 2021 0x0 line:190, col:17 91% type:[vim]
```

Рис. 14.3. Примерный список завершений для команды, требующей имени файла

Чтобы не потерять важные команды, которые вы создали, и не превысить лимит сохраненных команд, вам может быть удобно записывать эти команды в текстовые файлы. Потом вы сможете «загрузить» эти файлы в вашу историю командной строки.

Увеличиваем скорость

Давайте посмотрим, как сделать вашу историю шаблона поиска более полезной. Мы используем окно истории шаблона поиска для редактирования, итерации и улучшения процесса поиска. Однажды настроенным поиском легко воспользоваться.

Вспомним наше переназначение клавиш для открытия окна шаблона поиска:

```
:nnoremap // q/  
:nnoremap ?? q?
```

Окно шаблона поиска Vim — это тот же буфер, что и окно истории команды, однако одновременно может быть активен только один из них. Разница между ними лишь в том, *что* загружается в окно и что происходит при нажатии клавиши **Enter** на любой строке буфера (выполнить команду *ex* в противовес поиску шаблона). Функциональность и возможности, описанные ранее, также доступны в окне шаблона поиска. Вы можете искать любой шаблон, который использовали ранее. Это как поиск поисков, что можно считать метауровнем. Если ваша история поиска шаблонов достаточно велика, в ней, скорее всего, будут эффективные поисковые запросы, которые вы использовали ранее.

Вы можете использовать стандартные функции редактирования Vim внутри этого окна: перемещение, изменение и выполнение поисковых запросов из прошлых поисков.

Как и ранее, мы можем создавать полезный фильтр постепенно, повторяя команду и используя при этом окно истории командной строки тем же методом, чтобы отточить поиск, постепенно создавая новые шаблоны.

Vim использует регулярные выражения для выполнения сложного поиска. Рассмотрим это на примере файла, содержащего имена исполняемых файлов, которые названы по определенному образцу в соответствии с их ролью. То есть этот файл может быть назван таким образом, что разделенные точками поля будут описывать его атрибуты (например, *production.accounting.receivables.east.rollup*). Первое поле должно быть *production*, *test* или *devel*, а все имя состоять из пяти полей.

Мы не будем вдаваться в подробности, которые были описаны ранее (в разделе «Переходим к более интересному» данной главы), достаточно просто начать с поиска строки, содержащей *production*:

```
/production/
```

Эта команда найдет все строки, в которых встречается слово *production*. Команда *//* переместит нас в окно истории шаблона поиска, где мы редактируем этот шаблон, добавляя «.» в качестве разделителя, чтобы сузить поиск (обратите внимание, что мы убрали слеш, поскольку они *не* появляются в окне истории и шаблона поиска):

```
production\.
```

И наконец, жизнеспособный результат мог бы выглядеть примерно так:

```
\(production\|test\|devel\)\\(\.[[:alnum:]]_*)\{3}\.[[:alnum:]]_\\{1,}
```

Ранее мы использовали подобный шаблон для создания команды `match` в нашем файле `.vimrc`, чтобы автоматически подсвечивать имена исполняемых файлов во время редактирования файлов, дополняя обычную подсветку синтаксиса.



Мы оставляем вам расшифровку последнего регулярного выражения. Наша цель — не объяснить регулярное выражение и его работу, а показать способы создания эффективного регулярного выражения с помощью окна истории поиска шаблона Vim.

Если вы хотите сохранить свои любимые регулярные выражения, вы можете отследить их и сохранить в буфере окна шаблона поиска для дальнейшего использования.

Улучшаем строку состояния

По какой-то причине строка состояния Vim содержит не так уж много полезной информации (рис. 14.4).



Рис. 14.4. Стандартная строка состояния Vim

Однако вы можете добавить дополнительную информацию о текущем файле, используя следующую строку в файле `.vimrc` (если вам нужна дополнительная информация, обратитесь к документации `:help statusline`):

```
set statusline=%<%t%h%m%r\ \ %a\ %{strftime(\"%c\")}%=0x%B\ \ line:%l,\ \ col:%c%V\ %P\ %v
```

Пример строки состояния, использующей эти настройки, представлен на рис. 14.5.



Рис. 14.5. Строка состояния Элберта

Файл `.vimrc` Элберта, из которого взят пример, доступен в репозитории книги на GitHub (<https://www.github.com/learning-vi/vi-files>) (подробнее — в разделе «Получение доступа к файлам» приложения В).

Ниже представлено краткое объяснение встроенных флагов, использованных в примере. Все флаги начинаются с символа %:

- %a — статус списка параметров. То есть, если вы редактируете четвертый файл из восьми, строка состояния будет отображать (4 of 8);
- %B — шестнадцатеричное представление символа под курсором;
- %c — текущий номер столбца;
- %h — флаг «справки» буфера (в данном случае не показан, поскольку мы не редактируем файл справки);
- %l — номер текущей строки;
- %m — измененный флаг ([+], если буфер был изменен; в противном случае отсутствует);
- %P — текущее местоположение в буфере в виде процента;
- %r — флаг «только для чтения» ([RO], если буфер только для чтения; в противном случае отсутствует);
- %{strftime...} — результаты выполнения команды внутри фигурных скобок (в данном случае strftime). Передача %c запрашивает стандартные дату и время;
- %t — текущее имя файла (последний компонент имени файла, эквивалентен выводу *базовое_имя(1)*);
- %v — тип редактируемого файла. Это больше, чем просто проверка имени файла на наличие расширения. Vim выявляет тип файла на основании его содержимого. Хотя мы не можем это доказать, вероятно, Vim использует механизм, аналогичный или идентичный механизму команды `file` (если интересно, см. страницу руководства *файл(1)*);
- %V — текущий номер виртуального столбца;
- %= — центрировать информацию вокруг этого указателя (все, что «до», выровнено по левому полю, все, что «после» — по правому);
- %< — отсечь здесь, если строка состояния слишком длинная.

Резюме

Мы надеемся, что представленный в этой главе материал разжигает ваш аппетит для дальнейшего исследования возможностей Vim. Будьте уверены, всегда есть что-то еще, что можно узнать о Vim. Принимая это во внимание, вы сможете сделать свою работу удобнее и продуктивнее.

ЧАСТЬ III

Vim в более широкой среде

В части III мы сделаем шаг назад, чтобы взглянуть на картину в целом и рассмотреть роль Vim в более широком мире разработки ПО и работы с компьютерами, а затем подведем итоги. Эта часть включает в себя следующие главы.

- Глава 15 «Vim как IDE: требуется сборка».
- Глава 16 «vi повсюду».
- Глава 17 «Эпилог».

Vim как IDE: требуется сборка

Хотя `vi` является универсальным текстовым редактором, он был с самого начала популярен среди программистов благодаря своим многочисленным функциям, которые способствовали процессу программирования, особенно на языке C. (Вспомните параметр `showmatch`, функции автоматического отступа и в особенности возможности `ctags`, а также средства для маневрирования в документации `troff`.)

Неудивительно, что Vim продолжает эту традицию, но в отличие от `vi` он сам программирует и поддерживает *плагины*, что позволяет загружать новый код и добавлять функции напрямую в редактор.

Как и в случае с другими популярными языками сценариев, эта *расширяемость* привела к появлению огромного количества новых функций и средств для использования с Vim, намного больше, чем любой человек мог бы создать, работая в одиночку.

И неудивительно, что большой процент этих плагинов нацелен на упрощение программирования и разработки ПО с помощью Vim.

В данной главе мы рассмотрим (кратко!) менеджеры плагинов и некоторые из наиболее интересных и популярных плагинов для использования в разработке программного обеспечения.

Однако имейте в виду, что вселенная плагинов Vim невероятно большая. Для тщательного рассмотрения всех возможных плагинов потребовалась бы отдельная книга, гораздо более объемная, чем эта! Поэтому в текущей главе наше повествование включает гораздо меньше технических деталей, чем в других главах книги. Пожалуйста, помните об этом.

Менеджеры плагинов

Кроме плагинов, которые выполняют определенные действия, есть еще плагины, которые управляют другими плагинами. Их задача заключается в загрузке и инициализации плагинов, а также в упрощении их установки и использования без

необходимости загружать их вручную или добавлять много специфичного для плагина кода в ваш файл `.vimrc`.

В Vim имеется свой собственный менеджер плагинов, доступ к которому можно получить с помощью команды `:packadd` (добавить пакет). Вы можете использовать эту команду с плагинами, входящими в стандартную комплектацию Vim, или с любыми другими, которые соответствуют критериям, указанным в `:help packadd` (мы не будем их здесь указывать). Позднее мы рассмотрим один из этих стандартных плагинов. Рекомендуем вам ознакомиться и с другими, которые поставляются с Vim.

Один из самых популярных менеджеров плагинов называется Vundle (<https://github.com/VundleVim/Vundle.vim>) (сокращение от Vim bundle (пакет Vim)). На сайте есть инструкции для «быстрого старта», которые мы здесь обобщим, имея в виду GNU/Linux или другую систему в стиле POSIX.

1. Убедитесь, что в вашей системе установлены Git и curl.
2. На всякий случай сохраните копию вашего файла `.vimrc` и каталога `.vim` где-то в безопасном месте.
3. Клонировать Vundle непосредственно на его место:

```
git clone https://github.com/VundleVim/Vundle.vim.git ~/.vim/bundle/Vundle.vim
```

4. Сконфигурируйте ваш плагин. Вот как должен выглядеть ваш файл `.vimrc` (или скопируйте/вставьте с домашней страницы Vundle); мы опустили некоторые комментарии, чтобы сделать код короче:

```
set nocompatible          " be iMproved, required
filetype off              " required

" set the runtime path to include Vundle and initialize
set rtp+=~/.vim/bundle/Vundle.vim
call vundle#begin()
" alternatively, pass a path where Vundle should install plugins
"call vundle#begin('~/.vim/paths')

" let Vundle manage Vundle, required
Plugin 'VundleVim/Vundle.vim'

" The following are examples of different formats supported.
" Keep Plugin commands between vundle#begin/end.
" plugin on GitHub repo
Plugin 'tpope/vim-fugitive'
...

" All of your Plugins must be added before the following line
call vundle#end()          " required
filetype plugin indent on  " required
" To ignore plugin indent changes, instead use:
"filetype plugin on
"
```

```
...
"
" see :h vundle for more details or wiki for FAQ
" Put your non-Plugin stuff after this line
```

5. Выберите ваши плагины и поместите их между вызовами `vundle#begin()` и `vundle#end()`. (Это сложный этап. ☺)
6. Установите перечисленные в вашем файле `.vimrc` плагины. Вы можете сделать это либо с помощью команды Vim `:PluginInstall`, либо из командной строки:

```
vim +PluginInstall +qall
```

Это запустит Vim, установит плагины, а затем завершит работу. Запускайте `:PluginInstall` при каждом добавлении новых плагинов в файл `.vimrc`; затем Vundle выполнит загрузку и установку плагинов за вас.

Поиск подходящего плагина

В Vim доступны тысячи плагинов, многие из которых (а возможно, и большинство) размещены на GitHub, но не все. Поиск подходящего плагина для выполнения нужной задачи может оказаться сложнее, чем кажется (рис. 15.1).

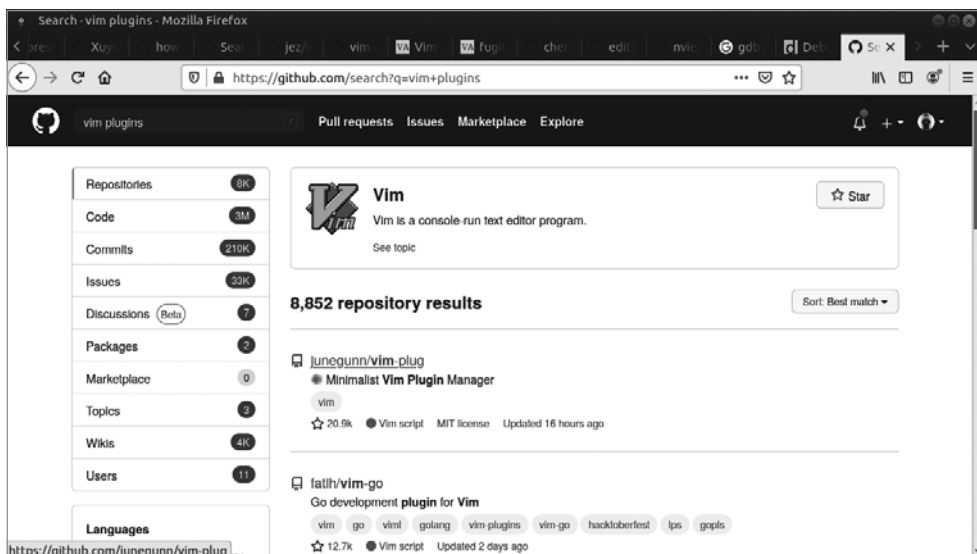


Рис. 15.1. Поиск GitHub плагинов Vim; 8852 результата и наименования

К счастью, люди в Vim Awesome (<https://vimawesome.com>) проделали невероятную работу по поиску в Интернете плагинов и собрали информацию о них в одном месте (рис. 15.2).

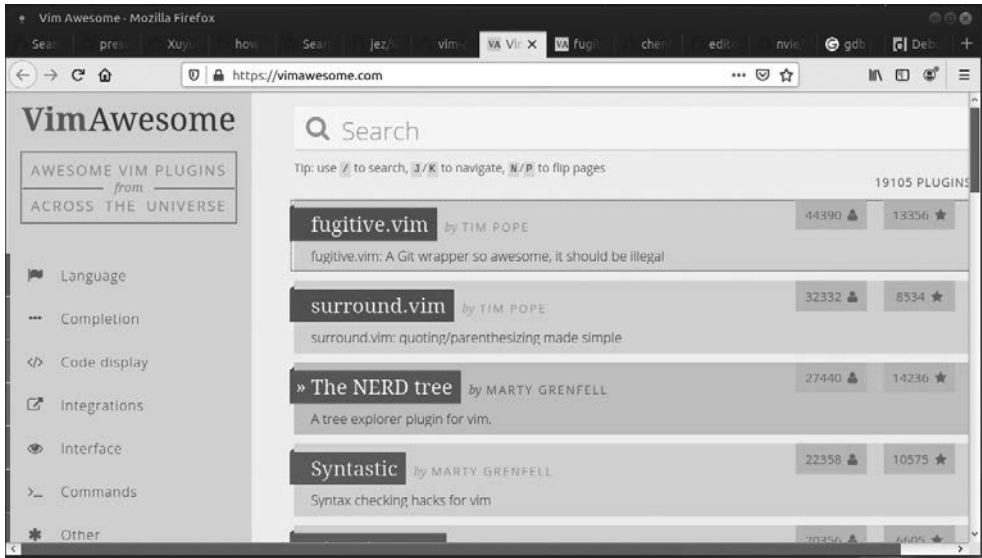


Рис. 15.2. Vim Awesome

Вы можете не только искать информацию о плагинах в Vim Awesome (<https://vimawesome.com>), но также установить свою личную версию их базы данных, следуя инструкциям и используя исходный код на странице <https://github.com/vim-awesome/vim-awesome>.

Зачем нам нужна IDE?

Интегрированная среда разработки (IDE) представляет собой единую среду, которая обеспечивает все необходимое для разработки ПО. Существует много таких сред, как коммерческих, так и с открытым исходным кодом. Если вы разработчик программного обеспечения, скорее всего, вы знакомы хотя бы с одной из них.

IDE обычно предоставляют как минимум следующие функции:

- редактирование текста (естественно);
- просмотр и перемещение по дереву файлов проекта по разработке ПО;
- перемещение по исходным объектам (например, переход от вызова функции к определению функции);
- интеграцию с одной или несколькими системами контроля исходного кода. Для достижения наших целей нам нужна интеграция с Git;
- завершение текста во время печати. Например, пока вы печатаете имя функции, IDE показывает вам ожидаемые параметры;

- выделение семантических ошибок. Если в вашей программе есть семантическая ошибка (например, необъявленная переменная), IDE выделит ошибку, чаще всего подчеркивая ее цветной волнистой линией;
- интегрированную отладку. IDE показывает исходный код по мере того, как отладчик перемещается внутри него.

Учитывая, что много программистов проводят большое количество времени, работая в Vim, вполне естественно, что они хотят, чтобы Vim предоставлял функции, аналогичные функциям IDE, поскольку это увеличивает их продуктивность. Мы вскоре увидим, как плагины Vim обеспечивают эти функции и как можно преобразовать Vim в вашу личную IDE.

Самостоятельная работа

Первые системы Unix отличались тем, что они давали пользователю свободу, а не навязывали определенную политику. То есть система позволяла делать что-либо различными способами. Vim унаследовал это от Unix: у вас есть средства для создания практически всего, что вы хотите. Конечно же, это означает, что вам придется потратить время и усилия на изучение того, как создать то, что вам нужно! Но зато вы получите среду, которая идеально подходит под ваши потребности. К счастью, как мы видели ранее, скорее всего, уже существует соответствующий плагин Vim. Вам надо просто найти его! В следующих подразделах мы рассмотрим несколько самых популярных плагинов для разработки ПО. Затем мы дадим обзор некоторых комплексных решений, которые направлены на превращение Vim в IDE. При просмотре данного раздела не забывайте, что это лишь краткий обзор и что в этой теме есть гораздо больше интересного!

EditorConfig: последовательная настройка редактирования текста

Во время нашего исследования мы наткнулись на проект EditorConfig (<https://editorconfig.org>). Цель проекта — создать спецификацию того, как различные текстовые редакторы и IDE должны форматировать разнообразные типы файлов. Например, для одного типа файла вам может потребоваться, чтобы все ваши редакторы делали отступ четырьмя пробелами, а для другого — символами `Tab`. Файл `.editorconfig` позволит вам реализовать это. Ваш редактор, неважно какой, считает данный файл и затем согласно его спецификации отформатирует вашу работу.

Многие IDE поддерживают файлы `.editorconfig`. Vim требуется плагин, который можно найти по адресу <https://github.com/editorconfig/editorconfig-vim>. Там же находятся и инструкции по установке. Для получения дополнительной информации и описания перейдите по ссылке https://www.vim.org/scripts/script.php?script_id=3934.

NERDTree: обход дерева файлов внутри Vim

Плагин NERDTree (<https://github.com/preservim/nerdtree>) — важный компонент для использования Vim в качестве IDE. Установив его, вы можете легко открывать и закрывать окно с помощью команды `:NERDTreeToggle`. В документации рекомендуется переназначить ее на последовательность клавиш `Ctrl+N`:

```
map <C-n> :NERDTreeToggle<CR>
```

Эта команда открывает новое окно в левой части экрана, отображая стандартное дерево файлов. Введите `?` в окне NERDTree, чтобы увидеть список команд, которые позволяют расширять или сжимать каталоги и открывать файлы в текущем или новом окне. Поведение команд зависит от того, является ли текущая строка в окне NERDTree файлом или каталогом. Ниже перечислены некоторые команды.

- `?` — отображение справки NERDTree.
- `i` — открыть файл в новом окне с помощью команды `:split`.
- `o` — расширить/сжать каталог. Для файла — открыть файл в предыдущем окне.
- `s` — открыть файл в новом окне с помощью команды `:vsplit`.
- `t` — открыть файл или каталог в новой вкладке. Вкладки описаны в главе 10.
- `T` — бесшумно открыть файл или каталог в новой вкладке.

Существует еще множество возможностей. Полная документация доступна в файле `doc/NERDTree.txt`.

(Что? Никаких иллюстраций? Терпение. Смотрите следующий подраздел.)

nerdtree-git-plugin: NERDTree с индикаторами состояния Git

NERDTree сам по себе чрезвычайно полезный инструмент. Однако в наши дни *подавляющее большинство* в качестве системы контроля исходного кода используют, как правило, Git. Многие IDE могут отобразить статус управления исходным кодом файла в своих проводниках файлов (измененных, не под управлением исходного кода, подкаталоги содержат неотслеживаемые файлы и т. д.). Плагин `nerdtree-git-plugin`, доступный по адресу <https://github.com/Xuyuanp/nerdtree-git-plugin>, расширяет возможности NERDTree.

На рис. 15.3 показано два окна редактирования и окно NERDTree с плагином `nerdtree-git-plugin` в левой части. В этом окне мы видим, что каталог `atomtable` не отслеживается (не зарегистрирован в Git), а каталог `support` содержит измененный файл, как и каталог `helpers`. Другие значки (не показанные здесь) указывают на измененный и (или) неотслеживаемый статус данного файла. Все это делает статус Git-файлов вашего проекта легко различимым.

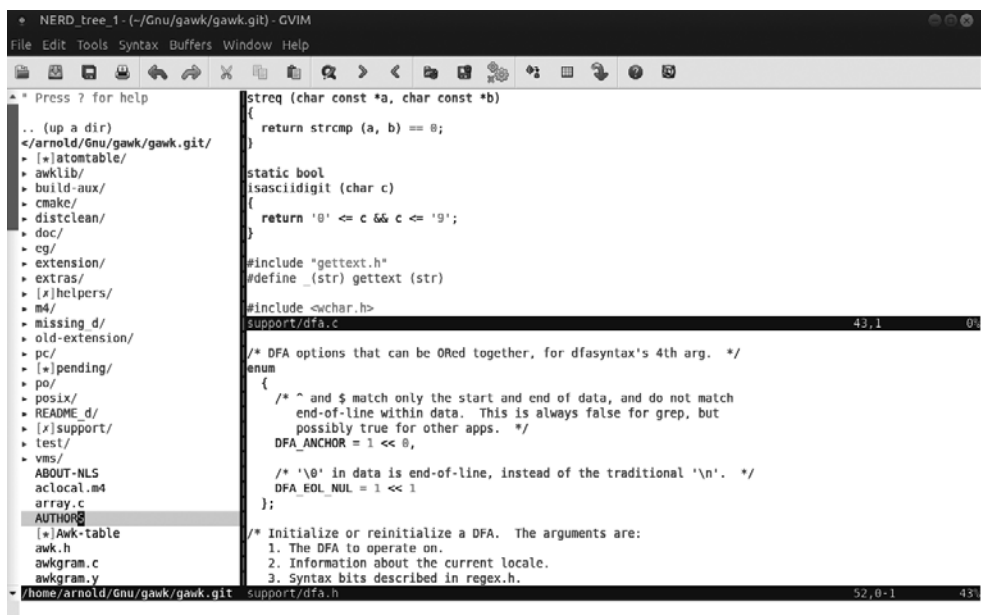


Рис. 15.3. NERDTree с указателями Git

Fugitive: запуск Git из Vim

Пользователи Vim, управляющие своими файлами с помощью Git, часто переключаются между окном Vim и терминалом, чтобы запускать команды Git. Однако плагин Fugitive (<https://github.com/tpope/vim-fugitive>) позволяет работать с Git, не покидая Vim.

Для этого вам нужно просто вместо запуска команды `git` в эмуляторе терминала использовать `:Git` (или даже просто `:G`) и продолжить как обычно (`:Git add`, `:Git status`, `:Git commit` и т. д.). Fugitive помещает любые выходные данные из Git в новый временный буфер, если это необходимо. При фиксации файла вы можете редактировать сообщение о фиксации (`commit message`) в текущем экземпляре Vim.

Цитируя веб-страницу Fugitive, можно отметить, что он делает для вас гораздо больше, чем просто запуск команды `git`.

- Поведение по умолчанию заключается в непосредственном повторении выходных данных команды. Бесшумные команды, такие как `:Git add`, избегают сообщения «Нажмите **Enter** или введите команду для продолжения».
- `:Git commit`, `:Git rebase -i` и другие команды, вызывающие редактор, выполняют свое редактирование в текущем экземпляре Vim.

- `Git diff`, `:Git log` и другие команды, выводящие большой объем информации, загружают свои выходные данные во временный буфер. Для применения этого поведения к любой команде наберите `:Git --paginate` или `:Git -p`.
- `:Git blame` использует временный буфер с переназначениями для дополнительной сортировки. Нажмите ввод на строке, чтобы отобразить фиксацию, в которой была изменена строка, или `g?` для просмотра других доступных переназначений. Если опустить аргумент имени файла, то текущий отредактированный файл будет использован для отображения с вертикальной прокруткой.
- `:Git mergetool` и `:Git difftool` загружают свои наборы изменений в список быстрого исправления.
- Команда `:Git` без аргументов открывает итоговое окно с «запорченными» файлами и непомятыми и неизвлеченными фиксациями (unpushed and unpulled commits). Нажмите `g?` для получения списка переназначений для различных операций, включая сравнение, организацию, фиксацию, перебазирование и скрытие. (Это преемник `:Gstatus`.)
- Эта команда (наряду с другими) всегда использует репозиторий текущего буфера, поэтому вам не стоит беспокоиться о текущем рабочем каталоге.

На рис. 15.4 показаны выходные данные `:Git blame` для этой главы.

```

+ 22.fugitiveblame - (/tmp/v8cYwBz) - GVIM1
File Edit Tools Syntax Buffers Window Help

+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) [[ide-1]]
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) == Vim as IDE: Some Assembly Required
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300)
+ afc75e3d (Arnold D. Robbins 2020-10-22 20:51:03 +0300) Although ++vi++ is a general purpose text editor,
+ afc75e3d (Arnold D. Robbins 2020-10-22 20:51:03 +0300) from Day One it was also a programmer's text editor. It has multiple
+ (Not Committed Yet 2020-11-01 17:39:49 +0200) features for making programming easier, particularly programming in C.
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) (Consider the ++showmatch++ option, the automatic indentation features,
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) and in particular the ++ctags++ facilities, as well as the facilities
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) for maneuvering within ++troff++ documentation.)
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300)
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) Unsurprisingly, Vim continues in this tradition, but unlike ++vi++,
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) it itself is programmable, and in particular it supports __plugins__,
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) the ability to load new code and add features directly into the editor.
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300)
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) As with many of the popular scripting languages, this __extensibility__
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) has led to an explosion of new features and facilities for use with
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) Vim; many more than any one person could have ever created working alone.
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300)
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) Also not suprisingly, a large percentage of these plugins are aimed
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) at making programming and software development with Vim much easier.
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300)
+ 6a0ab0ff (Arnold D. Robbins 2020-11-01 16:04:24 +0200) In this chapter we look (briefly!) at plugin managers and at some of the
+ 6a0ab0ff (Arnold D. Robbins 2020-11-01 16:04:24 +0200) more interesting and popular plugins for use in software development.
+ 6a0ab0ff (Arnold D. Robbins 2020-11-01 16:04:24 +0200) Be aware, however, that the universe of vim plugins is very large.
+ 6a0ab0ff (Arnold D. Robbins 2020-11-01 16:04:24 +0200) Thorough coverage of all the possible plugins would require a separate
+ 6a0ab0ff (Arnold D. Robbins 2020-11-01 16:04:24 +0200) book, much bigger than this one is!
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) [[ide-1.2]]
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) == Plugin Managers
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300)
+ 6a29f452 (Arnold D. Robbins 2020-10-20 20:01:29 +0300) Besides plugins that actually do something, there are also plugins

+ /tmp/v8cYwBz/22.fugitiveblame 1,1 Top ide.asciidoc 1,1 Top

```

Рис. 15.4. Запуск `:Git blame` в окне

При перемещении курсора на четвертую строку (идентификатор фиксации afc75e3d) и нажатии клавиши **Enter** отображается вид, показанный на рис. 15.5.

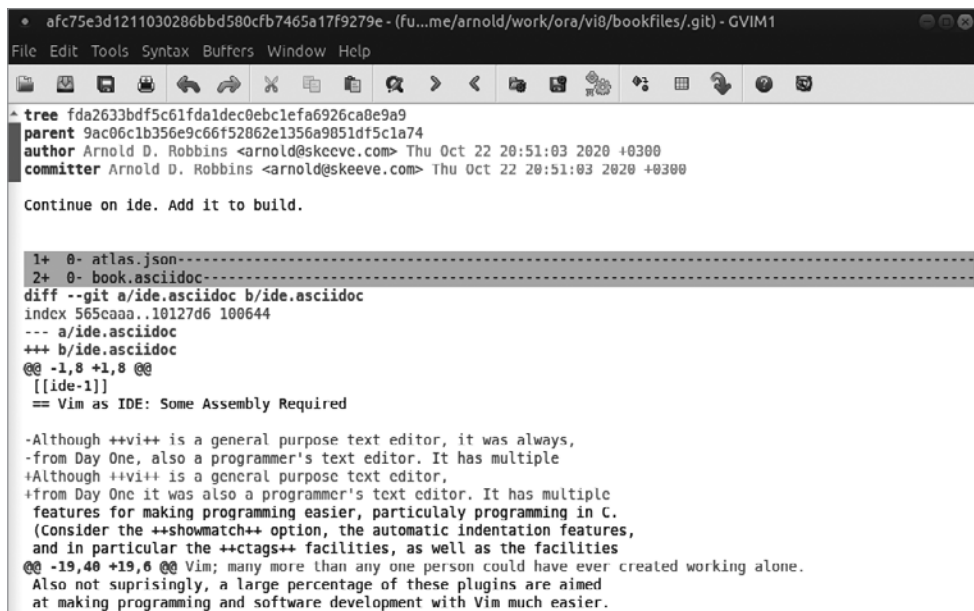


Рис. 15.5. Просмотр одной фиксации

Существует несколько «скринкастов», демонстрирующих возможности Fugitive. С ними стоит ознакомиться:

- «Дополнение к командной строке git» (<http://vimcasts.org/e/31>);
- «Работа с индексом git» (<http://vimcasts.org/e/32>);
- «Решение конфликтов слияния с помощью vimdiff» (<http://vimcasts.org/e/33>);
- «Просмотр базы данных объекта git» (<http://vimcasts.org/e/34>);
- «Изучение истории репозитория git» (<http://vimcasts.org/e/35>).

Мы рекомендуем посмотреть их и потратить некоторое время на освоение этого плагина. Проведя лишь несколько минут с ним, мы были в восторге!

Завершение

Одно из самых мощных средств, предоставляемых IDE, — *завершение*. В зависимости от используемой IDE, языка программирования и различных настроек IDE поможет вам завершить то, что вы вводите. Например, IDE может заполнить имя

длинной функции за вас или, когда вы вводите открытую скобку вызова функции, отобразить ожидаемые типы параметров, позволяя заполнить подходящие значения. В данном разделе мы подробно рассмотрим плагины завершения в Vim и предоставим рекомендации для некоторых других платформ.

YouCompleteMe: динамическое завершение и семантическая проверка

Плагин YouCompleteMe (<https://github.com/ycm-core/YouCompleteMe>) очень эффективный. Он обеспечивает завершение по мере ввода и семантическую проверку для множества языков программирования. На момент написания данной книги он поддерживает языки C, C++, C#, Go, JavaScript, Python, Rust и TypeScript. Для использования плагина на вашем языке программирования может потребоваться установка дополнительного ПО.

Вы можете установить YouCompleteMe напрямую из источника, следуя инструкции на сайте GitHub. Однако более простым способом является установка плагина через пакетный менеджер вашей системы.

В системе Ubuntu GNU/Linux необходимо выполнить следующие шаги:

```
sudo apt install vim-addon-manager
sudo apt install vim-youcompleteme
vim-addon-manager install youcompleteme
```

Первая команда устанавливает `vim-addon-manager`, который представляет собой еще один менеджер плагинов для Vim. При использовании его с Vundle не должно возникать конфликтов.

Вторая команда устанавливает YouCompleteMe. Третья устанавливает его в Vim, но только для текущего пользователя (он запускается без `sudo`).

После завершения установки Vim начинает предлагать вам варианты завершения во всплывающем окне. Нажимайте **Tab**, чтобы переключаться между вариантами. По мере ввода YouCompleteMe будет сокращать количество вариантов завершения (рис. 15.6).

Это все уже очень круто, но YouCompleteMe предлагает еще больше возможностей, включая семантический анализ вашего кода. Для C и C++ он использует `clangd` — часть набора компиляторов LLVM, чтобы непрерывно перекомпилировать вашу программу. Для различных языков могут использоваться разные механизмы, которые могут потребовать установки.

Чтобы активизировать семантический анализ для C и C++, нужно сообщить YouCompleteMe, как вы компилируете вашу программу. Это можно сделать всевозможными способами, в зависимости от того, каким образом построен ваш проект (Make, CMake, Gradle и т. д.).

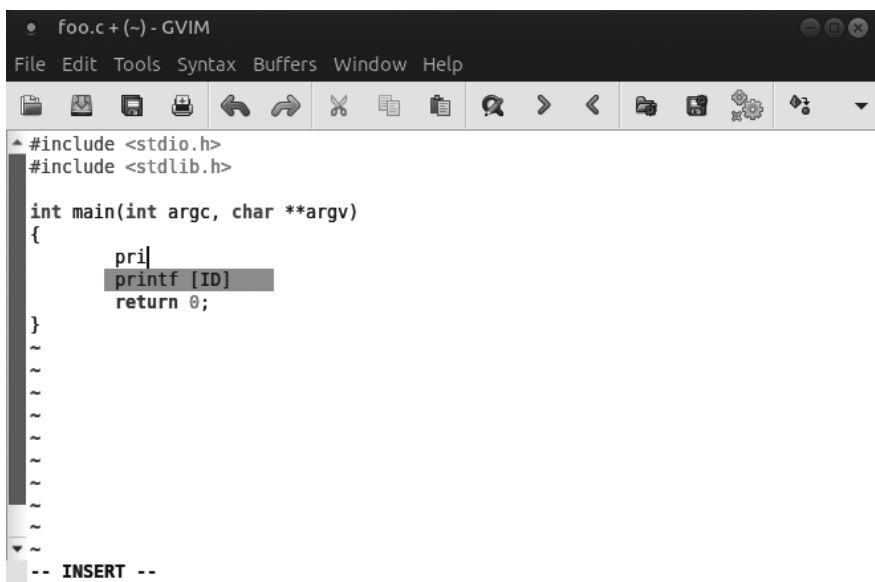


Рис. 15.6. Всплывающее окно YouCompleteMe

Для проектов, основанных на Makefile¹, это достаточно легко. Вам нужно установить простую программу Python под названием `compiledb` следующим образом (это для Ubuntu; у других систем GNU/Linux должен быть аналогичный механизм):

```
sudo apt install python3-pip
sudo pip3 install compiledb
compiledb make
```

Вам нужно запустить `compiledb make` лишь один раз (если вы не меняете ваши параметры компиляции). Это создаст файл `compile_commands.json` в корневом каталоге вашего верхнего проекта, который может использовать YouCompleteMe. После завершения этой операции Vim подсветит строки с ошибками и предупреждениями компиляции (рис. 15.7).

На рис. 15.7 первый индикатор проблемы подсвечен красным, указывая на ошибку компиляции. Перемещение курсора на эту строку приведет к тому, что Vim отобразит ошибку в строке состояния. В данном случае эта проблема связана с необъявленной переменной.

Второй индикатор подсвечен желтым, указывая на предупреждение. Здесь проблема заключается в несоответствии типов между аргументом `size_t`, который не имеет знака, и параметром `int`, который ожидает `printf()`.

¹ Настоящие программисты используют только Make.

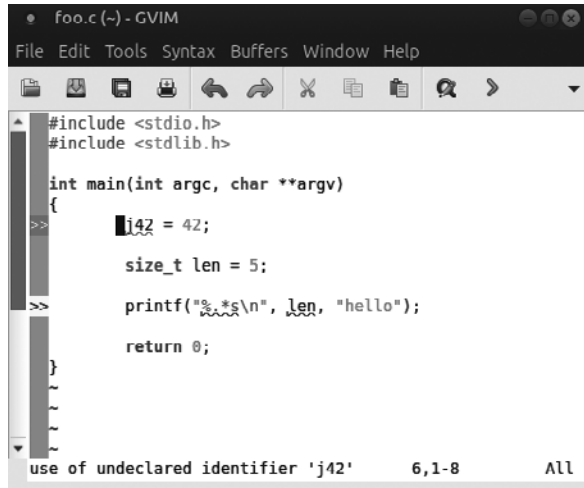


Рис. 15.7. Индикаторы ошибок YouCompleteMe

Обратите внимание, что обе проблемные области подчеркнуты волнистыми линиями (ошибки — красным, предупреждения — голубым). Как только вы все исправите, индикаторы проблем исчезнут. Это *невероятно* удобно: жаль, что мы не знали об этом плагине много лет назад!



Настройка YouCompleteMe может быть сложной задачей. Если вы работаете на языке С вместо С++, добавление следующего кода в `~/.ycm_extra_conf.py` может быть полезным. Или может быть достаточно поместить `'-std=c99'` в ваш файл `compile_commands.json`. Если честно, нас эта часть YouCompleteMe разочаровала. Тем не менее наличие семантических предупреждений стоит того!

```

import os
import ycm_core

flags = [
    '-fexceptions',
    '-ferror-limit=10000',
    '-DDEBUG',
    '-std=c99',
    '-xc',
    '-isystem/usr/include/',
]

SOURCE_EXTENSIONS = [ '.cpp', '.cxx', '.cc', '.c', ]

def FlagsForFile( filename, **kwargs ):
    return {
        'flags': flags,
        'do_cache': False # True
    }

```

Интересно, что YouCompleteMe не ограничен исходным кодом программы. Он может использоваться во время редактирования практически любых файлов, например, *ChangeLog* или даже текста формата AsciiDoc этой книги!

Другие механизмы завершения и проверки

В Vim доступен ряд других механизмов завершения. Вот некоторые из них.

- Asynchronous Lint Engine (ALE) (<https://github.com/dense-analysis/ale>). Он сфокусирован на динамическом контроле (семантической проверке) программ с поддержкой многих языков и некоторой поддержкой завершения.
- Syntastic (<https://github.com/vim-syntastic/syntastic>). Эффективный механизм проверки синтаксиса с поддержкой многих языков, часто используемый с другими плагинами.
- Conquer of Completion (<https://github.com/neoclide/coc.nvim>). Это общий плагин с поддержкой многих языков и форматов файлов.
- Jedi-vim (<https://github.com/davidhalter/jedi-vim>) обеспечивает автоматическое завершение в Python. Это то, что YouCompleteMe использует в Python.
- Kite (<https://www.kite.com>). Это плагин, обеспечивающий автозаполнение на основе искусственного интеллекта для Vim и других редакторов и IDE. Он поддерживает Python, C, C++, C#, Go, Java, Bash и многие другие языки. Kite является коммерческим ПО как с бесплатной версией, так и с платной.

Есть подозрение, что использование этих механизмов совместно с YouCompleteMe может быть несовместимо. Для получения наилучшего результата вам стоит поэкспериментировать с ними и выбрать подходящую настройку для своих нужд.

Termdebug: прямое использование GDB внутри Vim

Начиная с версии Vim 8.1, можно запустить терминал внутри окна Vim и выполнять программы, которые взаимодействуют с пользователем внутри окна редактора. Vim поставляется с плагином Termdebug, который позволяет запускать GDB (отладчик GNU) из Vim.

Чтобы использовать этот плагин, начните редактировать файл. Затем загрузите Termdebug с помощью встроенного пакетного менеджера Vim (`:packadd`) и запустите его следующим образом:

```
:packadd termdebug  
:Termdebug
```

Это разделит экран на три окна: в верхнем запустится GDB, в среднем появятся выходные данные отлаживаемой команды, а в нижнем отобразится исходный файл.

Если вам нужно переместить исходный файл вправо, как на рис. 15.8, то в разделе «Перемещение окон» главы 10 описано, как изменить компоновку окон Vim.

На рис. 15.8 вы видите взаимодействия GDB в верхнем левом углу. В левом нижнем находятся выходные данные предыдущей команды `run`, в которой автор допустил ошибку (ой!).

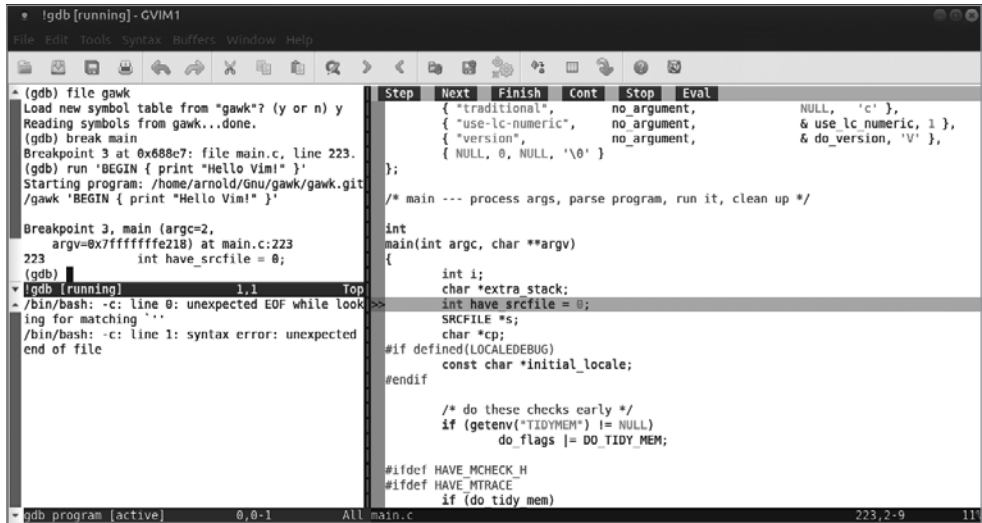


Рис. 15.8. Работающий с файлом плагин Termdebug

В правом окне отображается исходный код с подсветкой и отметками строк, в которых была достигнута контрольная точка. Кнопки наверху позволяют вам продолжить отладку.

Это прекрасная интеграция GDB с Vim, даже лучше, чем предоставляемая Clewn (см. подраздел «Драйвер Clewn GDB» в главе 16).

Если вы планируете часто использовать GDB, стоит поместить команду `:packadd termdebug` в ваш файл `.vimrc`.

Универсальные IDE

Если вы по ходу нашего повествования добавляли к себе рассмотренные нами плагины, значит, ваш Vim уже обладает большей частью функциональности IDE. Неудивительно, что многие люди прошли этот путь до нас и предложили свои собственные способы подстройки Vim под IDE.

Возможно, вам захочется ознакомиться со способами, которые обнаружили мы. По крайней мере, они помогут вам найти полезные плагины, о которых мы еще не писали.

Однако знайте: мы не опробовали их все, а те, что мы испытали, использовались недолго. Поэтому рассматривайте этот список как отправную точку для ваших собственных изысканий.

- *Vim как IDE* — это скорее своеобразный справочник по плагинам Vim для разработки ПО, а не то, что можно просто подключить и использовать. Это ценно благодаря множеству ссылок на источники дополнительной информации. См. <https://github.com/jez/vim-as-an-ide>.
- *vimspector* — это «многоязычный графический отладчик для Vim». Акцент здесь сделан на отладке кода, а не на полноценном IDE. См. <https://github.com/puremourning/vimspector>.
- *C/C++ IDE* — это набор плагинов для создания IDE для C и C++. Если верить сайту <https://github.com/kingofctrl/vim.cpp>, вы получаете:
 - автоматическую загрузку (так в оригинале) последней версии libclang и сборку библиотеки `ucl_core`, необходимой YCM;
 - установку в один шаг;
 - поддержку всех GNU/Linux;
 - загрузку по запросу для быстрого запуска;
 - семантическое автозаполнение;
 - проверку синтаксиса;
 - подсветку синтаксиса для C++11/14;
 - сохранение архивных записей;
 - мгновенный предварительный просмотр файлов markdown.

Следующие плагины переназначены для Python.

- *Режим Python*

Согласно сайту <https://github.com/python-mode/python-mode>, этот плагин предоставляет все, что нужно для разработки python-приложения в Vim:

- поддержку Python и 3.6+;
- подсветку синтаксиса;
- поддержку Virtualenv;
- запуск кода Python (`<leader>r`);
- добавление/удаление контрольных точек (`<leader>b`);

- улучшенные отступы Python;
- перемещения и операторы Python (`[]`, `3[[], []]`, `vaC`, `viM`, `daC`, `ciM`);
- улучшенные свертывания Python;
- одновременный запуск нескольких программ проверки кода (`:PymodeLint`);
- автоматическое исправление ошибок PEP8 (`:PymodeLintAuto`);
- поиск в документации Python (`<leader>K`);
- перестройку (рефакторинг) кода;
- автозавершение кода с помощью IntelliSense;
- переход к определению (`<C-c>g`).

● *Vim и Python — идеальное сочетание*

На странице Real Python <https://realpython.com/vim-and-python-a-match-made-in-heaven> описывается пошаговый процесс настройки Vim в качестве Python IDE.

● *Обновление Vim 2017*

Страница <https://haridas.in/vim-upgrade-2017.html> предоставляет инструкции по настройке Vim и рекомендации по использованию различных плагинов для повседневного программирования.

● *Vim как Python IDE*

Цитируя <https://rapphil.github.io/vim-python-ide>:

«Целью этого проекта является использование Vim в качестве мощной и полноценной среды разработки Python. Для этого мы составили список потрясающих плагинов, доступных в сообществе, и предоставили процедуру автоматической установки для этого набора».

Этот проект интересен тем, что в нем предусмотрено все. Он проверяет, устанавливает и создает определенную версию Vim, чтобы убедиться, что все будет работать. Если вы хотите использовать Vim для разработки Python и согласны с выбором, которые он за вас делает, это может оказаться самым простым способом.

● *Плагины Vim chenfm*

Это набор конфигурационных файлов Vim, который создает IDE из 24 различных плагинов. На странице <https://github.com/chenfm/VimPlugins> можно найти краткие инструкции по установке на английском языке и руководство на китайском. Хотя данный набор использует множество плагинов, он является отличным источником информации о дополнительных плагинах, которые, возможно, вы захотите изучить.

Кодировать — это здорово, но если я писатель?

Существует не один десяток плагинов, которые помогают людям писать в Vim, а не только разрабатывать ПО.

Томас Фернандес представил прекрасный список «Топ-10 плагинов Vim для писателей» в своем блоге (<https://tomfern.com/posts/vim-for-writers>). Проще будет процитировать список плагинов из статьи с описаниями и ссылками.

- *vim-pencil* — мой любимый плагин. Vim-pencil предлагает множество приятных функций, таких как инструменты навигации, более “умная” отмена на основе пунктуации и правильный, более гибкий перенос (<https://github.com/reedes/vim-pencil>).
- *vim-ditto* — Ditto подсвечивает повторяющиеся слова в абзаце, что очень помогает избежать тавтологии (<https://github.com/dbmrq/vim-ditto>).
- *vim-goyo* — аналог Writeroom для Vim, goyo убирает все отвлекающие элементы, такие как строка режима и номера строк (<https://github.com/junegunn/goyo.vim>).
- *vim-colors-pencil* — элегантная низкоконтрастная цветовая схема, подходящая для письма (<https://github.com/reedes/vim-colors-pencil>).
- *vim-litecorrect* — Litecorrect автоматически исправляет типичные ошибки ввода, такие как *teh* вместо *the* (<https://github.com/reedes/vim-litecorrect>).
- *vim-lexical* — объединенные тезаурус и проверка орфографии. Vim-lexical позволяет мне перемещаться от одной орфографической ошибки к другой с помощью `]s`, `[s` и быстро находить синонимы с помощью `<leader> t` (<https://github.com/reedes/vim-lexical>).
- *vim-textobj-sentence* — плагин для более комфортной навигации по предложениям. Я могу перемещаться между предложениями с помощью `(` и `)`, я могу вырезать предложение с помощью `dis`. Зависит от *vim-textobj-user* (<https://github.com/kana/vim-textobj-user>) (см. <https://github.com/reedes/vim-textobj-sentence>).
- *vim-textobj-quote* — этот плагин умело создает “кавычки”, так что мне не нужно этого делать самому (<https://github.com/reedes/vim-textobj-quote>).
- *ALE* — Asynchronous Lint Engine представляет собой инструмент анализа многоязычия, который не ограничен кодом. Он поддерживает множество средств проверки стиля, такие как *proselint* (<http://proselint.com>) и *LanguageTool* (<https://languagetool.org>) (см. <https://github.com/dense-analysis/ale>).

В статье Томаса есть еще кое-что интересное; рекомендуем просмотреть ее целиком.

Заключение

Ребята из Vim Awesome отлично сказали: Vim и правда невероятен! В этой главе мы коснулись лишь верхушки айсберга, который представляет собой мир плагинов Vim. Надеемся, что вы соберете свою собственную IDE и полностью раскроете силу и возможности Vim, которые значительно превосходят возможности простых редакторов.

Просто не забывайте иногда выходить на свежий воздух во время изучения всех доступных вам параметров. Удачи!

vi повсюду

Введение

Мы рассказали о множестве функций, которые делают vi и Vim такими мощными редакторами. Но vi не просто редактор. Это целая философия: способ *мыслить* о словах по-новому. Он позволяет вам отображать текст как *объекты*. Эти объекты, когда вы их освоите, формируют подход к редактированию, сильно отличающийся от «наведи и щелкни» и «что видишь, то и получаешь» (WYSIWYG). Текст как объекты представляет собой интересную абстракцию, настолько популярную, что она перетекает в другие инструменты, часть из которых может вас удивить. В данной главе вы познакомитесь с некоторыми распространенными и не очень примерами vi-мышления.

Различные способы улучшения и оптимизации работы с командной строкой

Обычно навыки пользователей vi выходят далеко за пределы редактирования текста. На протяжении многих лет инструменты командной строки (эмуляторы терминала, окна DOS и т. д.) обеспечивали базовую функциональность редактирования и истории командной строки. Однако благодаря вкладу в открытый исходный код окружение командной строки значительно улучшилось. vi является одной из наиболее популярных реализаций управления историей командной строки во многих оболочках командной строки.

В Unix командная строка называется *оболочкой*. Существует множество оболочек, среди которых наиболее популярными являются: sh (исходная оболочка Bourne), Bash (оболочка GNU Bourne-again), csh (оболочка C)¹, ksh (оболочка Korn) и zsh (оболочка Z).

¹ Мы не будем обсуждать оболочку csh, за исключением упоминания о том, что исходный csh и vi были написаны одним человеком — Биллом Джоем, когда он был студентом Калифорнийского университета в Беркли. Отметим также, что практически все оболочки реализуют язык оболочки Bourne, в то время как csh имеет свои отличительные особенности.

Большинство (но не все) современных оболочек позволяют редактировать командную строку в режиме `vi`, что мы вскоре и увидим.

Совместное использование нескольких оболочек



Прежде чем начать использовать то, что мы собираемся представить, обязательно изучите наши инструкции. Это стоит сделать, если вы не хотите потерять критически важные файлы истории!

В последующих примерах мы кратко опишем параметры для активации редактирования истории команд, а также объясним, как перемещаться по истории команд с помощью клавиш `vi`. Поскольку вам придется запускать не одну оболочку для проверки различных параметров, вы создадите экземпляры оболочки, каждый со своим собственным понятием «окружения», то есть переменными и поведением, специфичными для конкретной оболочки. Однако некоторые оболочки имеют значения по умолчанию для файлов истории, и, когда вы запускаете их, они не замещают существующее определение файла истории.

Например, если вы регулярно используете `zsh` и вызываете другую оболочку (`ksh`), это не изменит значение переменной файла истории (`HISTFILE`) и команды `ksh` будут записаны в файл истории `zsh`. При выходе из `ksh` существующая `zsh` остается «ошеломленной» и «сбитой с толку», а файл для загрузки будет поврежден! Итак, вот что вы должны сделать, чтобы не допустить подобного.

1. В вашем домашнем каталоге создайте и отредактируйте загрузочный файл для каждой оболочки: `ksh` (`.kshrc`), `Bash` (`.bashrc`) и `zsh` (`.zshrc`). Убедитесь, что вы не перезаписываете какие-либо из уже имеющихся у вас файлов.
2. Удостоверьтесь, что вы не потеряете никаких ценных данных истории, добавив или проверив существование этих строк:

```
# make Backspace key do what it should do
stty sane

# set command-line editing to vi mode.
set -o vi

# keep history files in a hidden folder please.
myhistorydir=${HOME}/.history
# make the directory, fail silently if it's already there
mkdir -p ${myhistorydir}

# save lots of commands. computer memory is cheap and reliable.
HISTSIZE=5000
HISTFILESIZE=5000
# save command history in this file. Note that we incorporate the shell's name
```

```
# into the file name. this prevents collisions and corrupt history
# files inadvertently assigned by different shells (it happens!)
HISTFILE=${myhistorydir}/.$(basename $0).history
```

Таким образом, каждая оболочка будет сохранять историю в отдельном файле, основанном на имени оболочки.

Библиотека readline

Многие инструменты GNU и GNU/Linux используют библиотеку **readline** для интерактивного ввода. Библиотека **readline** позволяет программам на языке C (или C++) читать пользовательские входные данные, обеспечивая редактирование строки в строке ввода.

Оболочка Bash

Интерактивное редактирование командной строки оболочки с использованием Emacs или набора команд **vi** было впервые представлено в оболочке Korn в 1980-х годах. Оболочка GNU Bourne-again (Bash) реализовала те же возможности, но на основе автономной библиотеки многократного использования **readline**.

Когда **readline** активирована, в вашем терминале появляется окно с одной командой, которую вы можете редактировать с помощью команд предпочитаемого текстового редактора, будь то Emacs или **vi**. Чтобы включить режим редактирования строк, используйте либо **-o emacs** для Emacs, либо **-o vi** для **vi**. Мы, естественно, предпочитаем последний. Как правило, чтобы нужный вам параметр всегда был установлен, одна из этих команд помещается в файл **.bashrc** домашнего каталога.

Кроме того, **readline** хранит историю выполненных вами команд, позволяя перемещаться по списку команд вверх и вниз, чтобы повторно вызвать и затем отредактировать предыдущие команды. Например, **k** перемещает вас вверх по списку истории, а **j** — вниз. А с помощью **h** и **l** вы можете перемещаться горизонтально внутри текущей строки.

Кроме регулярных команд **vi**, библиотека **readline** предоставляет некоторые дополнительные команды в командном режиме, которые выполняют расширения, полезные в командной строке. Они описаны в табл. 16.1.

Таблица 16.1. Дополнительные команды **vi** для использования в оболочке

Команда	Действие
#	Вставить # в начало строки, превращая строку в комментарий
=	Перечислить файлы с заданным префиксом

Команда	Действие
*	Вставить расширения всех файлов с заданным префиксом
Tab	Взять предыдущий префикс и расширить его, насколько это возможно, оставляя его при этом уникальным, например, с несколькими файлами chapterXX и префиксом ch
Tab	расширит ch до chapter

Консольное редактирование в Bash

Возьмем пример из реальной жизни, когда один из нас самостоятельно обучился Unix, изучая команды в различных каталогах Unix (/bin, /usr/bin, /usr/local/bin и т. д.). Используя возможность писать сложные команды в приглашении командной строки, он на лету написал сценарий, чтобы с легкостью просматривать различные команды с помощью команды man, как показано здесь. Символ \$ является основным или *первичным* приглашением, а > — *вторичным* приглашением, запускаемым, когда Bash знает, что команда еще не завершена:

```
$ cd /usr/bin
$ for man in a*
> do
>   printf "\n\n$man, look at man page? "
>   read yesno
>   if [ ${yesno:-yes} = "yes" ]
>   then
>     man $man
>   fi
>   printf "\n\nhit enter to continue "
>   read dummy
> done
```

Оболочка выполняет цикл «вопрос/ответ» для каждого файла в /usr/bin, начиная с буквы a. Чтобы увидеть команды, начинающиеся с других букв, он использовал режим редактирования Bash vi, введя Esc K для редактирования только что запущенной командной строки для выполнения с новой буквой. Редактировать однострочные команды в Bash просто, но многострочные команды более запутанны. При повторном вызове предыдущей команды она отображается в виде одной очень длинной строки, которая переносится на экране:

```
$ for man in b*; do           printf "\n\n$man, look at man page? ";
read yesno;                  if [ ${yesno:-yes} = "yes" ];           then
man $man;                    fi;           printf "\n\nhit enter to continue ";
read dummy; done
```

Обратите внимание, что разделитель оболочки ; разделяет все строки и Bash сохраняет все пробелы. Чтобы перейти к каждой новой строке, начните с f; , чтобы перейти к первой точке с запятой. Используйте ; для перехода к следующему

вхождению и `,` для перехода к предыдущему. Хотя этот способ редактирования немного громоздкий, каждую строку легко найти.

В нашем примере автор хотел изменить *a* на *b* или на другой символ. Для этого он использовал свою любимую команду `vi` для перехода к *a* * и внесения изменений. Редактирование в командной строке в сочетании с большим объемом сохраненной истории позволяет ему вернуться к этому «сценарию» в любое время, найдя его в истории Bash и отредактировав вновь.

Многострочные команды в Bash

В Bash есть параметр, который улучшает процесс редактирования многострочных команд: `shopt -s lithist`. В результате Bash сохраняет многострочные команды в файле истории в виде нескольких строк вместо сжатия их в одну строку с разделителями в виде точек с запятой. После активации параметра повторно вызванная команда выглядит следующим образом:

```
$ for man in a*
do
  printf "\n\n$man, look at man page? "
  read yesno
  if [ ${yesno:-yes} = "yes" ]
  then
    man $man
  fi
  printf "\n\nhit enter to continue "
  read dummy
done
```

Для навигации по тексту вновь вызванной команды все еще нужно использовать горизонтальные команды перемещения. Команды `j` и `k` позволяют перемещаться вверх и вниз по списку истории, а не внутри повторно вызванной многострочной команды. Однако вы можете добавить дополнительные привязки клавиш, чтобы перемещаться между строками физического экрана. Для этого используются команды `readline: next-screen-line` и `previous-screen-line`¹. Чтобы добавить эти комбинации клавиш, отредактируйте ваш файл `.inputrc`; подробнее об этом можно узнать в подразделе «Файл `.inputrc`» далее и на странице руководства `readline(3)`.

Использование Vim для редактирования команд Bash

Если встроенный редактор не устраивает вас, вы можете вызвать свой любимый редактор для изменения команды, просто набрав `v`. Таким образом, содержимое вашей командной строки будет открыто в любом редакторе, который указан в ва-

¹ Спасибо за этот совет Чету Реми, осуществляющему поддержку Bash.

шей переменной окружения `EDITOR`. Мы, конечно же, ожидаем, что это будет `vi` или `Vim`, но вы можете выбрать любой редактор на свой вкус.



Есть одна загвоздка. `Bash` немедленно выполнит все, что находится в буфере редактора, когда вы завершите работу с ним. Допустим, вы ввели что-то похожее на:

```
$ rm -fr /
```

и затем нажали `Esc` и `V`, чтобы войти в свой редактор. Если вы затем решите, что не хотите ничего менять, и попытаетесь выйти из редактора (`:q` или `:q!`), то исходный текст будет незамедлительно выполнен, что может привести к катастрофическим последствиям!

Чтобы избежать этого, используйте безопасный способ выхода из `Vim` с помощью команды `:cq`. Это заставит `Vim` завершить работу с ненулевым кодом возврата, который сообщит `Bash`, что возникла ошибка и команду выполнять не нужно.

После такого инцидента Элберт считает, что это достаточно веская причина, чтобы рассмотреть использование оболочки `Z` (смотрите далее).

Другие программы

`Bash` — это не единственная программа, которая использует `readline`. Если она доступна во время компоновки программы, то `GNU Awk (gawk)` использует ее для своего встроенного отладчика `AWK`, а также ее можно применять в таких программах, как `GDB` (отладчик `GNU`) и `ftp`, для интерактивной передачи файлов по сети в большинстве систем `GDB/Linux`.

Интеграция `readline` с `GDB` особенно полезна, так как отладка часто включает ввод повторяющихся команд, а возможность простого поиска и редактирования предыдущих команд значительно облегчает отладку. Вспомним, к примеру, прохождение цепочки указателей «следующий» в связанном списке.

Файл `.inputrc`

Подождите! Это еще не все!

Практически в каждой ТВ-рекламе

Вы можете настроить поведение `readline`, поместив команды в ее файл инициализации. Этот файл определяется переменной окружения `INPUTRC`. Если `INPUTRC` не задана, тогда `readline` ищет файл с именем `.inputrc` в вашем домашнем каталоге. Если данный файл недоступен, `readline` обращается к `/etc/inputrc`.

На странице руководства *readline(3)* описывается формат и возможное содержимое файла. Библиотека постепенно растет и развивается, поэтому мы не будем здесь приводить полное описание. Вместо этого приведем пример личного файла `.inputrc` автора:

```
set editing-mode vi
set horizontal-scroll-mode On
control-h: backward-delete-char
set comment-begin #
set expand-tilde On
"\C-r": redraw-current-line
```

Ниже приведено краткое объяснение каждой из этих строк.

- `set editing-mode vi` включает режим редактирования `vi` вместо режима Emacs по умолчанию. Таким образом, даже в GDB, `ftp` и любой другой программе, использующей *readline*, доступен набор команд `vi`.
- `set horizontal-scroll-mode On` отображает текст сплошной строкой на экране и помечает правое поле символом `>`, если за ним скрывается текст. Перемещение вправо за `>` прокручивает строку. Когда фрагмент строки исчезает за левой частью экрана, там же появляется символ `<`.
- `control-h: backward-delete-char` позволяет `^H` (который обычно эквивалент клавиши `Backspace`) удалять символы.
- `set comment-begin #` при запуске команды `#` вставляет символ `#` для создания комментария. Для оболочки это закомментирует текущую строку, но вставит это в историю для дальнейшего повторного вызова и редактирования. В любом случае это значение по умолчанию для `Bash`, но файл нашего автора не менялся с 2002-го!
- `set expand-tilde On` выполняет расширение тильды при расширении слова, что полезно, если *readline* используется с оболочкой, которая не выполняет расширение тильды, например `rc shell`.
- `"\C-r": redraw-current-line` позволяет перерисовывать текущую строку при нажатии `Ctrl+R`. Это удобно, если выходные данные системы смешиваются с входными¹.

Страница руководства *readline* содержит множество других параметров, включая те, которые окрашивают выходные данные в эмуляторах терминала, поддерживающих цвет.

¹ Для пользователей Emacs это, как правило, комбинация клавиш вашего обратного поиска по истории. Обратный поиск `vi` — это просто нажатие `/` (после запуска режима поиска по истории нажатием `Esc`).

Другие оболочки Unix

Оболочка Korn (*ksh*), разработанная Дэвидом Корном в «Лабораториях Белла», является первой оболочкой с возможностью редактирования командной строки. Для активации режима *vi* в этой оболочке используйте команду `set -o vi`, которую позже взяла на вооружение и Bash. Редактор *ksh* не основан на библиотеке *readline*¹.

Оболочка Z (*zsh*) также предоставляет режим командной строки *vi*, но он немного отличается от *ksh* и Bash. Вам может понадобиться добавить дополнительные привязки клавиш, если вы привыкли к одной из этих оболочек. Однако *zsh* имеет свои преимущества, в том числе возможность легкого редактирования многострочных команд.

И наконец, *tcsh* (Tenex *csh*) также позволяет работать в режиме *vi*, который активируется с помощью `bindkey -v`. Поскольку никто из нас не является пользователем *tcsh*, нам особо нечего сказать об этой оболочке.

Оболочка Z (zsh)

Как уже упоминалось, оболочка Z (<https://www.zsh.org>) обладает своим собственным режимом командной строки с мощным многострочным редактором истории. Рассмотрим пример, приведенный ранее. В *zsh* различия, которые обеспечивают большую визуальную контекстуальную ясность посредством специализированных подсказок, выглядят следующим образом:

```
{elhannah,/usr/bin} for man in a*
for> do
for>   printf "\n\n$man, look at man page? "
for>   if [ ${yesno:-yes} = "yes" ]
for if> then
for then>     man $man
for then> fi
for>   printf "\n\nhit enter to continue "
for>   read dummy
for> done
```

А теперь мы переходим к настоящему различию в редактировании командной строки! Здесь мы показываем, как *zsh* представляет многострочную команду в режиме редактирования после нажатия **Esc K**:

```
{elhannah,/usr/bin} for man in a*
do
    printf "\n\n$man, look at man page? "
    if [ ${yesno:-yes} = "yes" ]
```

¹ Оболочка Korn до сих пор доступна для использования (<https://github.com/ksh93/ksh>) и регулярно обновляется.

```

then
    man $man
fi
    printf "\n\nhit enter to continue "
    read dummy
done

```

Теперь у вас есть миниатюрный сеанс `vi` для гораздо более точного редактирования.



Несмотря на то что этот миниатюрный сеанс позволяет вам вставлять («открывать») новые строки кода с помощью `o` и `O`, имейте в виду, что вставленную строку следует завершать с помощью `Esc`. Если вы нажмете `Enter`, `zsh` немедленно выполнит весь фрагмент текста.

Сохраняйте как можно больше истории

К настоящему моменту вы, вероятно, уже оценили важность истории и редактирования командной строки. В разделе «Совместное использование нескольких оболочек», где мы показали, как «сохранить вашу работу», мы определили переменные *истории* (связанные с количеством команд для сохранения) следующим образом:

```

HISTSIZE=5000
HISTFILESIZE=5000
HISTFILE=${myhistorydir}/.$(basename $0).history

```

Представьте ваш диалог, вашу историю команд, выполненных в вашей любимой оболочке, в виде большого файла. Теперь ваша история командной строки — это настоящий документ, дополненный преимуществами редактирования командной строки для быстрого и эффективного поиска давно выполненных команд. Чем выше значение `HISTSIZE`, тем больше память вашей оболочки.

Современные компьютеры обладают большим объемом памяти и дискового пространства. Это избавляет от необходимости тщательно отслеживать размер истории. Мы выбрали 5000 в качестве золотой середины и считаем, что это число обеспечивает доступную для поиска историю команд, измеряемую годами.

Следующий пример демонстрирует, как эффективно можно использовать историю и редактирование командной строки, чтобы быстро восстановить «все, что мы делали раньше». К слову, один из нас периодически работает с графикой и видео, но часто проводит много времени без использования каких-либо видео/графических приложений или команд. Однако, если у него есть фрагмент команды или название основного приложения или утилиты, он легко может найти старые примеры и получить доступные для редактирования команды для мгновенной

продуктивности. Например, он активно использует команду `ffmpeg`, которая имеет *множество* параметров и их сочетаний. Простой поиск (`Esc /ffmpeg Enter`) в сочетании с итерацией с помощью `n` или `N` позволяет легко извлечь все сохраненные команды `ffmpeg` и сделать их доступными для редактирования.

Редактирование командной строки: некоторые заключительные мысли

Когда вы начнете использовать редактирование командной строки в режиме `vi`, имейте в виду, что ваша история команд подобна доступному для редактирования файлу. Предыдущие команды находятся прямо у вас под рукой (на клавишах `Home` и не только!). Используйте это, чтобы улучшить свои навыки при работе с приложениями, извлекая предыдущие команды и повторяя их синтаксис. Настройте *полностью* свои сохраненные команды! Позвольте вашему компьютеру делать свою работу. Мы предоставили вам начальный набор *инструкций*. Теперь вам стоит изучить руководство и поискать настройки истории (их много).



Помните, что все, что вводится в оболочке, интерпретируется как сценарий. Зная это, используйте некоторые возможности оболочки. Например, воспользуйтесь тем фактом, что все, что идет после символа `#` (и включая его), является комментарием и не выполняется. Добавление стратегических комментариев к командам, которые может потребоваться быстро запомнить, дает вам еще один способ для поиска старых команд. Хотя это плохая практика с точки зрения безопасности, один из авторов часто прибегал к добавлению `# system name passwd` к команде `echo` при изменении пароля для компьютера. В итоге он мог легко найти свой самый последний пароль в истории.

Windows PowerShell

PowerShell — это объектно-ориентированная командная строка консольного вида, созданная компанией Microsoft для быстрой и эффективной автоматизации. Впечатление и ощущение от использования программы напоминает оболочки Unix, а ее объектно-ориентированный характер расширяет философию Unix «все является текстом» до «все является объектом». Синтаксический анализ командной строки знаком любому пользователю консолей MS-DOS `command.com/cmd.exe` с некоторыми встроенными приятностями IntelliSense. Однако для нас этого недостаточно! К счастью, вы можете перемещаться по консоли PowerShell с помощью команд `vi`. В приглашении PowerShell просто введите команду:

```
Set-PSReadlineOption -EditMode vi
```

Чтобы сделать данную настройку постоянной, добавьте эту команду в файл `.profile`, который находится в одном из шести конфигурационных файлов PowerShell, в зависимости от ваших предпочтений.

Инструменты разработчика

Разработчики используют *множество* инструментов разработки и вынуждены изучать новые, чтобы не отставать от современных технологий. Наличие функциональных возможностей `vi` в инструментах разработки делает их более удобными для непосредственного использования разработчиками, знакомыми с `vi` и Vim.

В этом разделе мы рассмотрим инструменты отладки с функциональностью `vi` и плагины Vim для двух версий Microsoft Visual Studio® IDE.

Драйвер Clewn GDB

Clewn реализует полную поддержку `gdb` в редакторе `vim`: контрольные точки, переменные наблюдения, завершение команд `gdb`, окна сборки и т. д.

[...]

Clewn — это программа, управляющая `vim` через интерфейс сокета `netBeans`, она выполняется одновременно с `vim` и взаимодействует с `vim`. Clewn можно использовать только с `gvim`, графической реализацией `vim`, так как `vim` на терминале не поддерживает `netBeans`.

*Домашняя страница Clew Project
(<http://clewn.sourceforge.net>)*

Clewn — интересная программа, которая позволяет использовать Vim для просмотра исходного кода во время отладки в GDB. Clewn управляет Vim через интерфейс NetBeans¹.

Чтобы использовать Clewn, запустите его в окне терминала. Clewn пригласит вас с помощью приглашения (`gdb`) и заставит `gvim` открыть другое окно для отображения исходника. Этот процесс продемонстрирован на рис. 16.1.

¹ NetBeans (<https://netbeans.org>) является открытым источником IDE для Java, JavaScript, HTML5, PHP, C/C++ и других языков программирования. Интерфейс Vim NetBeans позволяет использовать его в качестве редактора внутри NetBeans.

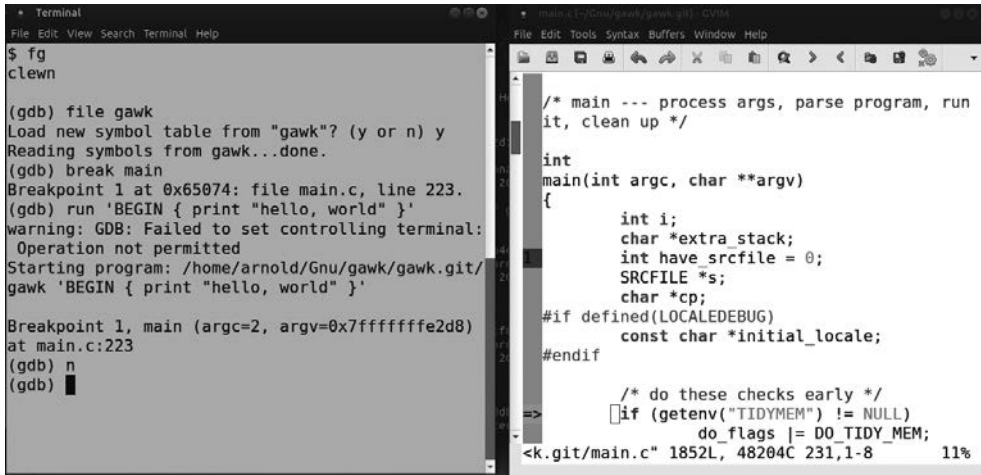


Рис. 16.1. Clewn в действии

К сожалению, у Clewn нет поддержки. Тем не менее один из нас регулярно использует эту программу, и она продолжает прекрасно работать в текущих системах GNU/Linux.

Домашняя страница проекта Clewn — <http://clewn.sourceforge.net>. Исходный код для Clewn включен в репозиторий GitHub этой книги (<https://www.github.com/learning-vi/vi-files>). Дополнительная информация изложена в разделе «Получение доступа к файлам» приложения В.

CGDB: обязательные операции GDB

CGDB — это облегченный консольный интерфейс для отладчика GNU. Он разделяет экран на две части: нижнюю — сеанс GDB и верхнюю — исходный код программы. Интерфейс выполнен в стиле vim, поэтому пользователи vim должны чувствовать себя комфортно при использовании CGDB.

*Страница CGDB GitHub
(<https://github.com/cgdb/cgdb>)*

Отладка является здесь ключевой темой. Это неудивительно, учитывая, что vi и Vim являются в первую очередь редакторами для программистов. Когда инструменты для разработки ПО предоставляют пользователю интерфейс, аналогичный Vim, это делает их более удобными.

CGDB может быть использован в окне эмулятора терминала. CGDB разделяет экран, отображая приглашение на ввод команды (`gdb`) в нижней части, а ваш исходный код — в верхней. При этом синтаксис исходного кода подсвечивается разными цветами (рис. 16.2).

Для перехода из командного окна в окно исходного кода можно использовать клавишу `i`, а для возврата в командное окно — `Esc`. По окну исходного кода допускается перемещаться с помощью команд поиска Vim, включая регулярные выражения.

CGDB поставляется с подробным руководством Texinfo, объясняющим его использование. Это прекрасная альтернатива Clewn, намного удобнее, чем встроенный в GDB текстовый пользовательский интерфейс (`gdb -tui`).

Домашняя страница проекта CGDB — <https://github.com/cgdb/cgdb>. Рекомендуем посетить ее для получения дополнительной информации!

```

Terminal
File Edit View Search Terminal Help
1521 static void
1522 parse_args(int argc, char **argv)
1523 {
1524     /*
1525      * The + on the front tells GNU getopt not to rea
1526      */
1527     > const char *optlist = "+F:f:v:W;bcCd::D::e:E:ghi:
1528     int old_optind;
1529     int c;
1530     char *scan;
1531     char *src;
1532
/home/arnold/Gnu/gawk/gawk.git/main.c
Type "apropos word" to search for commands related to "word"...
Reading symbols from gawk...done.
(gdb) break parse_args
Breakpoint 1 at 0x67cd8: file main.c, line 1527.
(gdb) run 'BEGIN { print "hello, world" }'
Starting program: /home/arnold/Gnu/gawk/gawk.git/gawk 'BEGIN {
print "hello, world" }'

Breakpoint 1, parse_args (argc=2, argv=0xffffffff2f8) at main.
c:1527
(gdb) █
  
```

Рис. 16.2. CGDB в действии

Vim внутри Visual Studio

Microsoft Visual Studio, возможно, самая распространенная в мире IDE. Ее можно модернизировать с помощью плагинов, также называемых *расширениями*.

VsVim (<https://github.com/VsVim/VsVim>) — это расширение с открытым исходным кодом, которое предоставляет эмулятор Vim для Visual Studio. На момент напи-

сания книги он поддерживал версии Visual Studio 2017 и 2019. Проект активно развивается и поддерживается.

Если вы работаете в Visual Studio и хотели бы редактировать в стиле Vim, рассмотрите это расширение.

Vim для Visual Studio Code



Плагины Vim для Visual Studio и Microsoft Visual Studio Code различаются. Хотя их часто объединяют в одно приложение, они не являются разными плагинами.

Visual Studio Code: быстрое знакомство

Visual Studio Code (обычно называемый *VS Code*) — это облегченная версия IDE от Microsoft — Visual Studio. VS Code, как и его коммерческий старший брат, представляет собой мощную экосистему интеграции разработки и управления проектами. Хотя он предлагает похожие функциональные возможности, отличается достаточно сильно, включая параметры плагинов Vim. Однако это не является большой проблемой, поскольку добавление плагинов Vim достаточно просто в обоих приложениях.

Первый шаг, который необходимо сделать, — это загрузить и установить (<https://code.visualstudio.com>) VS Code. Процесс очень прост.

Расширения VS Code

Дополнения к VS Code называются *расширениями* или *плагинами*. Самый быстрый способ добраться до функций VS Code — воспользоваться универсальной командой «сделай это». Просто нажмите **Ctrl Shift+P** для Microsoft Windows и GNU/Linux или **Command Shift+P** для MacOS. (Любопытно, что F1 делает это на всех трех операционных системах.) Начните вводить **install extensions** (установить расширения), и VS Code отобразит выпадающий список выбора (рис. 16.3).

VS Code отображает вертикальное левостороннее окно, в котором перечислены установленные расширения и доступные для установки в (обширной) экосистеме плагинов. Если ввести в окне поиска **vim**, вы увидите нечто подобное рис. 16.4.

Подсвеченный элемент **vscodevim** является выбранным плагином. Нажмите кнопку **Install**. Это выведет на экран диалоговое окно, показанное на рис. 16.5.

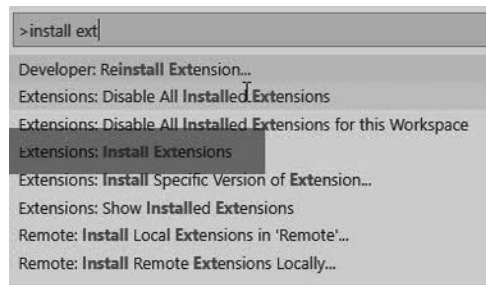


Рис. 16.3. Окно «сделай это» VS Code

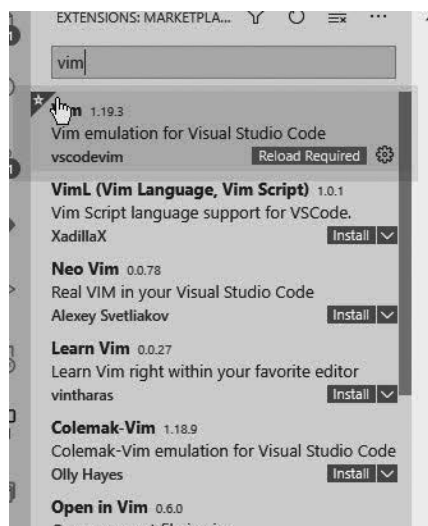


Рис. 16.4. Поиск расширения в VS Code

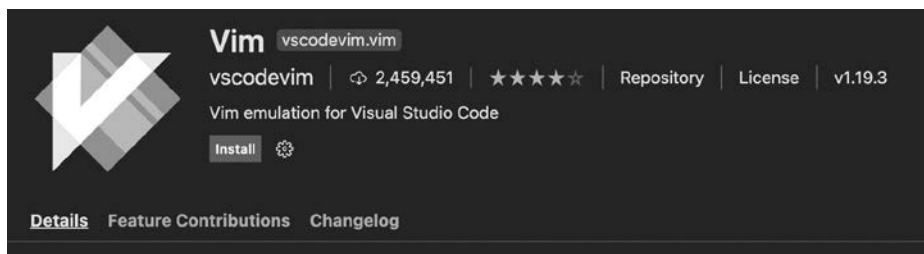


Рис. 16.5. Установка диалогового окна vscodevim

Чтобы отключить или деинсталлировать расширение, найдите его тем же способом, как и ранее. Это выведет на экран диалоговое окно, показанное на рис. 16.6. Теперь нажмите **Disable** или **Uninstall**.

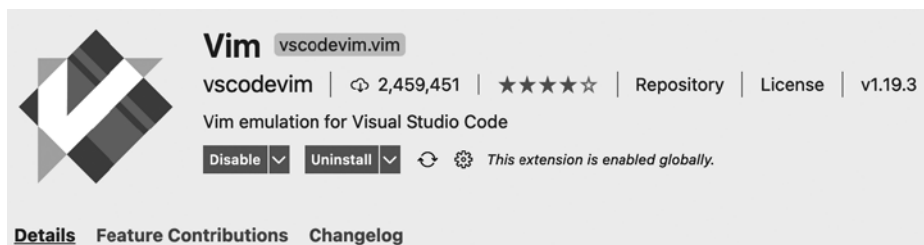


Рис. 16.6. Отключение или деинсталляция диалогового окна vscodevim

Настройки `vscodevim`

Чтобы увидеть доступные настройки плагина `vscodevim`, воспользуйтесь универсальной командой VS Code `Ctrl Shift+P` и выполните поиск по запросу `settings`. Вероятно, настроек будет много, поэтому выберите вариант `Preferences: Open Settings (UI)` (рис. 16.7).



Рис. 16.7. Поиск настроек в VS Code

Теперь выполните поиск в диалоговом окне *параметров*, и вы увидите около 100 параметров настроек, связанных с `vim`, как показано на рис. 16.8.

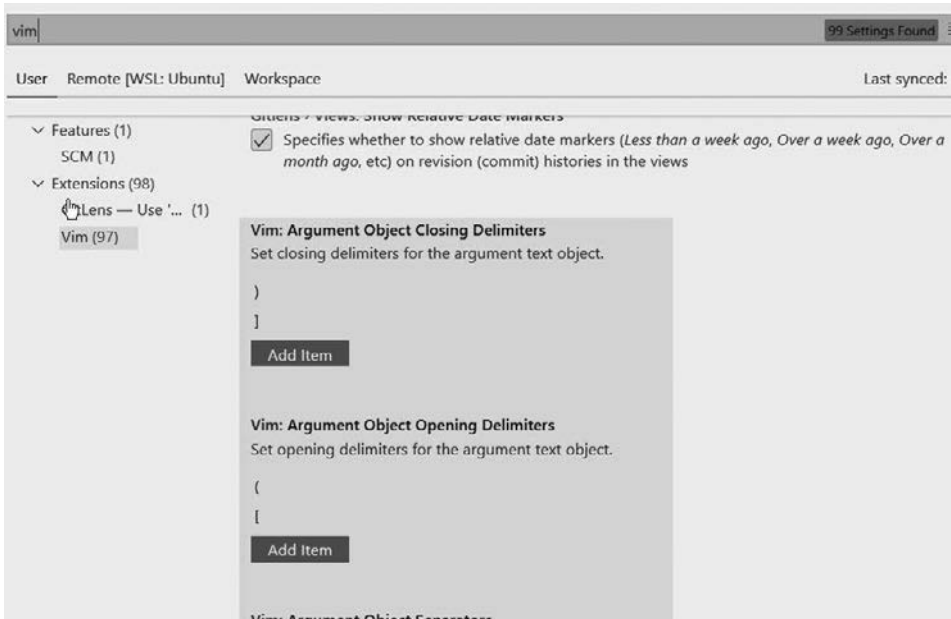


Рис. 16.8. Параметры настройки VS Code для Vim

Мы оставляем настройки на ваше усмотрение. Обратите внимание, что многие из них просто предоставляют ссылку для открытия файла установочных параметров VS Code в формате JSON. К сожалению, некоторые распространенные настройки Vim требуют, чтобы вы умели редактировать этот файл.

Например, в подразделе «Упрощенный выход из Vim» в главе 14 мы описываем, как упростить выход из Vim, переназначив `q` и `Q` в командном режиме `vi` на `:q^M` и `:q!^M` соответственно (выйти из Vim и *на самом деле* выйти из Vim!)¹. Однако вы не можете сделать эти переназначения в настройках расширений VS Code. Ниже представлен фрагмент кода JSON для создания переназначения:

```
"vim.normalModeKeyBindingsNonRecursive": [
  {
    "before": ["q"],
    "commands": [":q"]
  },
  {
    "before": ["Q"],
    "commands": [":q!"]
  },
  {
    "before": ["ctrl+v"],
    "commands": ["Ctrl+Shift+G"]
  }
],
```



Поскольку действия происходят внутри VS Code, IntelliSense будет достаточно полезным для завершения `vim.XX`, где `XX` является действующей настройкой `vscodevim`. Обратите внимание, что в предыдущем примере мы использовали версию `NonRecursive` для `normalModeKeyBindings`, чтобы предотвратить повторную интерпретацию `q` как снова нуждающуюся в переназначении. Это распространенный и хорошо известный метод в переназначении Vim, определяемый с помощью контекстуального задания префиксов командам переназначения посредством погетар. (Более полное обсуждение переназначений команд `vi` вы найдете в подразделе «Команда `map`» в главе 7.)

Vim предназначен не только для VS Code

Мы выбрали VS Code для обсуждения плагинов Vim для IDE в качестве примера, так как VS Code — невероятно популярная и развивающаяся программа.

Тем не менее VS Code — всего лишь одна из многих доступных IDE, практически каждая из которых предлагает свою собственную эмуляцию Vim для редактирования или обладает быстродоступными плагинами, подобными Vim. Мы использовали и проверили плагины Vim или эмуляцию в таких IDE, как NetBeans, Eclipse, PyCharm и JetBrains. А они далеко не единственные. Велика вероятность, что для вашей IDE есть подходящий плагин Vim.

¹ Помните, что с помощью `^M` Vim отображает возврат каретки, который вы вводите с помощью `Ctrl+V Ctrl+M`. См. подраздел «Команда `map`» в главе 7.

Утилиты Unix

Абстракции `vi` скрыты во многих утилитах Unix/Linux, о которых вы можете и не знать. Далее мы приведем несколько примеров утилит и того, как команды `vi` повышают вашу эффективность, когда вы не редактируете.

Больше или меньше?

`more` является первоначальной *пейджерной* программой на основе экрана: программой, созданной для представления данных на одном экране за раз. Она была разработана как часть системы BSD Unix в тот же период времени, что и исходный `vi`. Помимо представления содержимого файла, `more` считывает стандартные входные данные, если не заданы какие-либо файлы, что упрощает использование в конце конвейера.

Через некоторое время после того, как `more` стал стандартом, появился `less` (каламбур на `more`) — улучшенный пейджер. Сегодня оба инструмента доступны в современных системах.

Мы думаем (по иронии судьбы), что `more` на самом деле `less` (меньше), а `less` — это `more` (больше). `more` является действующим инструментом Unix с базовой нумерацией страниц и взаимодействием, в то время как `less` — это практически потоковый редактор. Несмотря на отсутствие реальных функций редактирования, `less` обеспечивает надежную навигацию, аналогичную Vim.

Один из нас на своих персональных компьютерах (GNU/Linux) всегда переименовывал `more` на `more_or_less` и связывает `/usr/bin/less c /usr/bin/more` (таким образом преобразовывая все, что `more`, в `less`).

Если вы не имеете прав администратора или совместно используете сеть, в которой другие могут возражать против изменений, вы можете установить тот же функционал с помощью псевдонима. Для этого необходимо добавить псевдоним в ваш личный файл `.bashrc`:

```
alias 'more=less'
```



Один из наших технических рецензентов предупредил, что переименование `more` и привязка `less` к `more` может привести к неожиданному поведению и путанице, вероятно влияя на выполнение сценариев и установщиков, которые ожидают изначальное поведение `more`. Мы согласны с предупреждением и настоятельно рекомендуем проявлять осторожность при внесении этих изменений.

Для справки: мы вносили только что описанные изменения при каждой установке GNU/Linux с момента появления команды `less` и ни разу не заметили никаких неблагоприятных эффектов в системе. Тем не менее ваш опыт может отличаться.

Еще одна причина использования `less` вместо `more` (по крайней мере в большинстве случаев) — это возможность установки переменной окружения `PAGER` в вашем файле `.bashrc`:

```
export PAGER=less
```

Однако «меньше — лучше». Ниже представлено несколько функций `less`, похожих на Vim:

- `b`, `d` — перейти вверх или вниз на одну страницу соответственно (недоступно в `more`);
- `gg`, `G` — перейти в начало или в конец входного файла или потока соответственно (недоступно в `more`);
- `/шаблон`, `?шаблон`, `n`, `N` — выполнять поиск *шаблона* вперед или назад и найти следующее сопоставление в том же или в противоположном направлении;
- `v` — открыть текущий файл в редакторе, имя которого указано в переменной окружения `EDITOR`. Это работает только для файла, а не для стандартного ввода.

Конфигурация дисплея `less`

Отчасти неизвестной функцией `less` является возможность настраивать способ отображения текста. В большинстве случаев это неинтересно, но теперь, когда мы заменили `less` на `more`, реальная разница и приятное улучшение изменят то, как вы будете видеть страницы руководства, если выполнить следующие команды. Попробуйте просмотреть несколько страниц руководства до и после, чтобы оценить визуальный эффект (например, `man man`):

```
# variables and dynamic settings to improve less
export LESS="-ces -r -i -a -PM"
# Green:
export LESS_TERMCAP_mb=$(tput bold; tput setaf 2)
# Cyan:
export LESS_TERMCAP_md=$(tput bold; tput setaf 6)
export LESS_TERMCAP_me=$(tput sgr0)
# Yellow on blue:
export LESS_TERMCAP_so=$(tput bold; tput setaf 3; tput setab 4)
export LESS_TERMCAP_se=$(tput rmso; tput sgr0)
# White:
export LESS_TERMCAP_us=$(tput smul; tput bold; tput setaf 7)
export LESS_TERMCAP_ue=$(tput rmul; tput sgr0)
export LESS_TERMCAP_mr=$(tput rev)
export LESS_TERMCAP_mh=$(tput dim)
export LESS_TERMCAP_ZN=$(tput ssubm)
export LESS_TERMCAP_ZV=$(tput rsubm)
export LESS_TERMCAP_ZO=$(tput ssupm)
export LESS_TERMCAP_ZW=$(tput rsupm)
```

Этот файл доступен в репозитории книги на GitHub (<https://www.github.com/learning-vi/vi-files>). См. раздел «Получение доступа к файлам» приложения В.

Существует гораздо больше функций. Но мы оставляем их вам для самостоятельного изучения. Подсказка: в приглашении `less` введите `h` для получения справки.

screen

`screen` представляет собой мультиплексор сеанса работы с терминалом, обеспечивающий несколько параллельных рабочих сеансов внутри одного окна терминала. Современные эмуляторы терминалов обычно реализуют несколько сеансов во вкладках, что делает `screen` еще более полезным. Ответ на вопрос о том, почему `screen` стоит использовать, мы получим после демонстрации полезного примера конфигурационного файла. Так что, пожалуйста, потерпите.

`screen` обеспечивает точное поведение терминала для своих сеансов и, таким образом, требует, чтобы вы использовали специальный символ префикса для активации его команд и функций. По умолчанию в роли префикса выступает сочетание `Ctrl+A`. Например, чтобы отобразить список самых доступных команд `screen`, необходимо ввести `Ctrl+A ?`.



Последующее изложение предполагает использование систем GNU/Linux или Unix и наличие приложения `screen`. Проверьте, есть ли в вашей системе `screen`, с помощью команды `type` из приглашения командной строки:

```
$ type screen
```

Ответ должен быть похож на `/usr/bin/screen`. Если вы не получите подобный ответ, установите `screen` с помощью вашего пакетного менеджера. Начните с <https://www.gnu.org/software/screen>, если хотите (или вам необходимо) создать `screen` из исходника.

Первоначальный запуск screen

`screen`, как и многие другие приложения Unix, считывает необязательный конфигурационный файл пользователя для установки параметров и определения сеансов. Конфигурационный файл `screen` называется `.screenrc` и находится в вашем домашнем каталоге. Для удобства вы можете либо скопировать файл `.screenrc` из репозитория книги на GitHub (<https://www.github.com/learning-vi/vi-files>), либо создать файл самостоятельно и добавить в него следующие строки:

```
startup_message off
defscrollback 20000
```

```
# Help screen, key bindings:
```

```

bindkey -k k9 exec sed -n '/^# Help/s/^/^M/p;/^# F[1-9]/p' $HOME/.screenrc
# F2 : list windows:
bindkey -k k2 windowlist
# F3 : detach screen (retains active sessions -- can reconnect later):
bindkey -k k3 detach
# F10: previous window (e.g., window 4 -> window 3):
bindkey -k k; prev
# F11: next window (e.g., window 2 -> window 3):
bindkey -k F1 next
# F12: kill all windows and quit screen (you will be prompted):
bindkey -k F2 quit
screen -t "edits for chapter 1"
screen -t "manage screen captures"
screen -t "manage todos"
screen -t "email and messages"
screen -t "git status and commits"
screen -t "system status"
screen -t "remote login to NAS" ssh 10.0.0.999
screen -t "solitaire"

select 1

```

Теперь запустите `screen`:

```
$ screen
```

Вы увидите сеанс обычного вида, то есть приглашение на ввод команды в окне. Последняя строка в предыдущем фрагменте кода `select 1` указывает `screen` начать ваше взаимодействие в первом окне или сеансе (определенном как «правки для главы 1»). Прелесть `screen` в том, что вы можете находиться в одном из восьми сеансов, определенных в конфигурационном файле, каждый из которых полностью независим от других.

В `screen` существует много способов выбора сеансов и навигации по ним. Мы продемонстрируем подобную навигацию, как в `vi`.



`screen` использует для большинства действий односимвольные команды. Каждой из этих команд должен предшествовать начальный управляющий символ `screen`, которым по умолчанию является `Ctrl+A`.

Меню `screen`

Для вывода на экран списка доступных сеансов в `screen` используется команда в виде двойных кавычек (`"`). Для просмотра списка сеансов можно нажать `Ctrl+A`". В результате на экране появится нечто похожее на рис. 16.9.


```
~/git/learning-the-vi-and-vim-editors-8e — screen • zsh — ttys001
Num Name
0 edits for chapter 1
1 manage screen captures
2 manage todos
3 email and messages
4 git status and commits
5 system status
7 solitaire
```

Рис. 16.9. Доступные сеансы на экране

Хотя данная функция кажется простой, для выбора нужной строки сеанса необходимо использовать стандартные команды перемещения курсора **vi**, такие как **k** и **j** для перемещения вверх и вниз соответственно. Этим демонстрируется философия перемещения, которую выбрали разработчики **screen** и которая является сходной с философией перемещения в **vi**.

Навигация по выходным данным сеанса

В сеансах **screen** гораздо больше взаимодействия с **vi**. **screen** буферизует текст во время ваших сеансов, и вы можете перемещаться по этому сохраненному тексту с помощью команд **vi**, нажав **Ctrl+A Esc**. По умолчанию **screen** хранит лишь около 100 строк текста. В конфигурации выше мы увеличили это значение до 20 000, что является более разумным для современных систем. Удостоверьтесь, что вы сделали это. Одна сотня буферизованных строк текста — это совсем не много.

Каждый буфер хранится отдельно для каждого сеанса. Таким образом, помимо истории командной строки, по которой вы можете выполнять поиск, редактировать и повторно выполнять, у вас есть подобный **vi** доступ к **vi**-подобным командам как для ввода команд, *так и* для вывода их результатов!

Основы поиска по буферу **screen** начинаются с **vi**-основ. После нажатия **Ctrl+A Esc** вы можете перемещаться по буферу с помощью команд перемещения (строка наверх — **k**, строка вниз — **j**, прокрутка вверх — **^B**, прокрутка вниз — **^F** и т. д.). Прямой поиск, как и следовало ожидать, запускается с помощью **/**, а обратный поиск — **?**. Обратите внимание, что использование символа **/** для прямого поиска при первом поиске из нижней части буфера не найдет ничего, поскольку вы уже находитесь внизу. Для получения более полных инструкций по **vi**-подобным командам для **screen** перейдите на страницу руководства **screen** и выполните поиск **vi-like** (рис. 16.10).

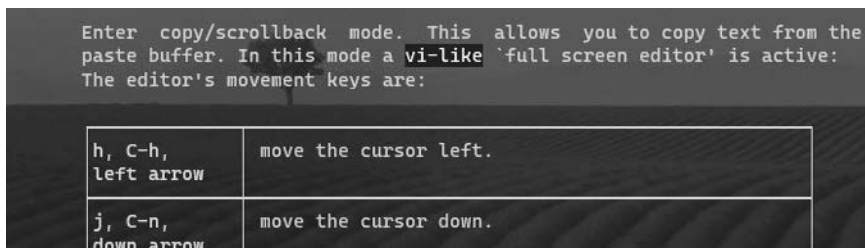


Рис. 16.10. Инструкции vi для screen

Пользуясь навигацией по screen и vi-подобному буферу, вы получаете доступ к информации, про которую, возможно, думали, что потеряли. Мы не раз с подобным сталкивались.

Преимущества привязки клавиш screen

В показанном ранее примере кода .screenrc были строки, не относящиеся к определению сеансов работы с терминалом, а являющиеся рекомендуемыми привязками клавиш. Например, строки:

```
# F2 : list windows
bindkey -k k2 windowlist
```

привязывают функциональную клавишу 2 (k2 соответствует F2) к команде windowlist, которая обычно вызывается внутри сеанса screen с помощью **Ctrl+A**". Это более интуитивное сочетание клавиш для действия, которое вы, вероятно, будете использовать постоянно: просмотр меню доступных запущенных сеансов screen.

Другие переназначения в вашем примере конфигурации включают:

- **F3** — полностью отключить от сеанса screen;
- **F10** — перейти в сеанс screen численно на один ниже текущего (например, если вы в сеансе четыре, перейдите в третий);
- **F11** — это дополнение к **F10**: перейти к следующему доступному сеансу;
- **F12** — прервать *все* сеансы и выйти из screen. Вероятно, вы редко будете использовать это, но приятно всегда иметь под рукой подобную функциональную клавишу.

Итак, теперь вы знаете, как точно переназначить клавиши для общих или нужных команд screen, но как отследить или запомнить то, что вы переназначили? Для этого смотрите привязку к **F9**:

```
# Help screen, key bindings:
bindkey -k k9 exec sed -n '/^# Help/s/^/^M/p;/^# F[1-9]/p' $HOME/.screenrc
```

Мы привязали **F9** для выполнения редактора потоков `sed`. Он извлекает закомментированную часть всех переназначений клавиш конфигурационного файла и отображает этот вывод в вашем приглашении для ввода. Прямо как в оболочке, все за пределами символа `#`, включая сам символ, является комментарием. Нажатие **F9** приводит к чему-то похожему на:

```
$                                     F9 нажата
# Help screen, key bindings
# F2 : list windows
# F3 : detach screen (retains active sessions -- can reconnect later)
# F10: previous window (e.g., window 4 -> window 3)
# F11: next window (e.g., window 2 -> window 3)
# F12: kill all windows and quit screen (you will be prompted)
```

Мы выбрали **F9**, потому что некоторые эмуляторы терминала закрепляют **F1** за своей собственной справкой.

Великолепие screen

Недавно мы задавались вопросом: «С учетом наличия эмуляторов терминала с несколькими сеансами какие особенности делают `screen` интересным для использования?».

Ответ: `screen` позволяет выходить из сеансов и возобновлять их позже. Ранее мы определили **F3** для отключения от сеанса `screen`. Попробуйте. Вы вернетесь к приглашению командной строки и оболочке, которые использовались до входа в `screen`. Если вы нажмете **F2**, то ничего не произойдет, так как вы вне сеанса `screen`.

Чтобы узнать, к каким сеансам `screen` можно подключиться, воспользуйтесь командой `screen` «список»:

```
$ screen -list
There is a screen on:
      8491.pts-0.office-win10 (04/06/21 18:58:39)      (Detached)
1 Socket in /run/screen/S-elhannah.
```

Вернуться к работе в определенном наборе сеансов `screen` можно с помощью параметра присоединения, введя идентификационный номер:

```
$ screen -Ar 8491
```

Теперь вы можете продолжить работу в своих сеансах `screen`. Не беспокойтесь, если закроете окно, в котором запущен `screen`. Это не приведет к завершению работы ваших сеансов. Вы запросто переподключитесь, как было описано выше! Мы использовали эту функцию, чтобы поддерживать несколько сеансов внутри `screen` (обычно от пяти до восьми), отключаться и вновь присоединяться под разными удаленными логинами, и сохранять эти сеансы активными на протяжении более трех месяцев подряд!

И наконец, браузеры!

С момента своего появления интернет-браузеры пытаются идти в ногу с технологиями. Первые браузеры были непродуманными (по современным стандартам) и могли отображать только информацию со ссылками и примитивную графику. Мешанина из стандартов создавала неудобный и несогласованный пользовательский опыт.

К счастью, стандарты сформировались, графика и воспроизведение стали эффективными, совместимыми и переносимыми (по большей части). Современные браузеры согласованы и удобны для обычных пользователей. Недавно сторонние разработчики создали расширения, добавляющие абстракции Vim в работу браузера.

Среди расширений для Chrome мы особенно выделяем два: Wasavi — реализация Vim для редактирования текстовых полей в браузере (например, заполнение текстовой области в форме обратной связи с клиентами) и Vimium — удобный инструмент навигации по страницам, закладкам, URL-адресам и поиску с использованием Vim-подобных абстракций. Оба расширения могут значительно улучшить эффективность вашей работы в браузере.



Важно отметить, что данные расширения разработаны для Chrome и доступны для любых браузеров, основанных на Chrome. Кроме того, Microsoft Edge обеспечивает совместимость с расширениями Chrome, что приятно удивит пользователей этого браузера.

Wasavi

Wasavi (<https://chrome.google.com/webstore/detail/wasavi/dgogifpkoiilgiofhhodbodcfgomelhe>) — это плагин для Chrome с открытым исходным кодом, который доступен на GitHub (<https://github.com/akahuku/wasavi>).

Давайте взглянем на рис. 16.11. На нем представлен текстовый виджет со строками, которые нужно переместить.

Строки нужно переместить в другое место в текстовом виджете. Сделаем это с помощью Wasavi, используя `vi`-подобные команды.

На рис. 16.12 представлен тот же текстовый виджет, но с действующим Wasavi.

1. Наличие пронумерованных строк говорит нам, что мы находимся в `vi`.
2. Чередование фона — еще один признак.
3. Мы можем использовать команды `vi` для перемещения этих строк.
4. Строка состояния `vi Wasavi`.

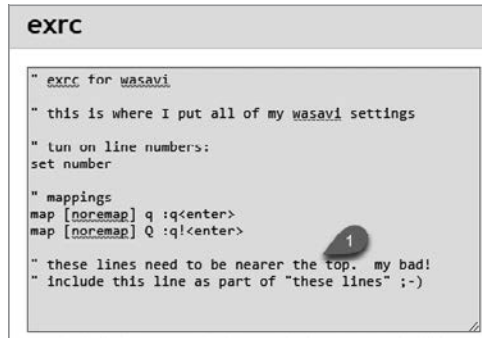


Рис. 16.11. Браузер текстового виджета для редактирования

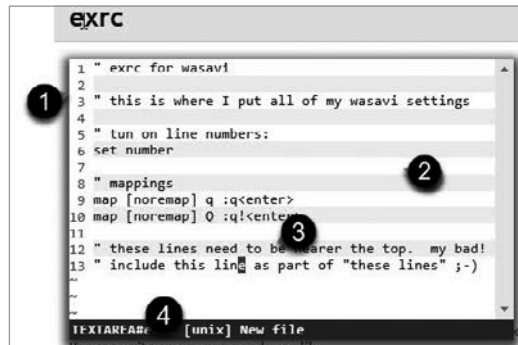


Рис. 16.12. Браузер текстового виджета для редактирования с помощью Wasavi

И наконец, на рис. 16.13 показан тот же самый текстовый виджет, но уже после трансформации его содержимого.

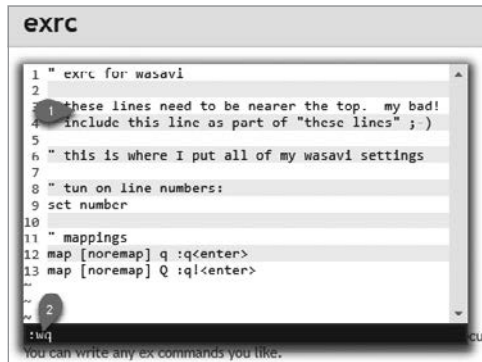


Рис. 16.13. Текстовый виджет после редактирования

1. Простые шаги `vi 11G, 3dd, gg, p` достигают цели (`11G` — переходит на строку 11, `3dd` — удаляет три строки, которые мы хотим переместить, `gg` — переходит в начало файла; `p` — помещает три удаленные нами строки в новое место).
2. Сохранение «файла».

Vim + Chromium = Vimium

Vimium — это расширение для Chrome, которое позволяет работать с браузером, используя команды, аналогичные Vim. Вы можете прочитать о нем на странице <https://chrome.google.com/webstore/detail/vimium/dbepggeogbaibhgnhhdjopepiihcmeb?hl=en>. Расширение также доступно в той или иной форме и для других браузеров, таких как Firefox и Safari.

Vimium позволяет перемещаться по Интернету `vi`-подобным способом. Поскольку браузер — это *не* текстовый редактор, нет точного соответствия между командами Vim и действиями Vimium. Однако, если вы хорошо владеете Vim, процесс обучения будет достаточно простым, а функции Vim — иметь смысл (в основном). Vimium преобразует опыт браузера, где нужно использовать мышь, в привычное «делай все с помощью клавиатуры». Мы считаем, что Vimium является необходимым инструментом для работы с браузерами.

Не спускайте с Vimium глаз

Мы рекомендуем закрепить расширение Vimium в списке расширений Chrome, чтобы всегда видеть, *активен* Vimium или *неактивен*. Это также позволит быстро отключить Vimium из-за каких-то проблем с ним, что периодически случается. Крайне редко возникает необходимость включить Vimium, поскольку для большинства сайтов, где Vimium отключен, есть свои причины. К примеру, в Google Mail, где Google уже предоставил полный набор переназначений клавиш для навигации по почте.

Vimium имеет множество функций. Мы выделим некоторые из них, чтобы помочь вам начать использовать данное расширение.

Поиск ссылок и переход по ссылке без щелчка кнопкой мыши

Vimium позволяет использовать команду Vim `f` для выделения всех видимых ссылок в браузере с помощью *значков*. Обычно эти значки принимают вид уникальных одно- или двухсимвольных идентификаторов. Эта мощная функция позволяет быстро переходить по ссылкам без необходимости двигать мышь и (точно) нажимать на ссылку. Для страниц с небольшим количеством ссылок обычно требуется два нажатия клавиш: первое — `f`, а второе — одно- или двухсимвольный

идентификатор ссылки. Для меньшего числа ссылок Vimium старается сократить идентификатор до одного символа.

Для интернет-страниц со множеством ссылок, в дополнение к ускорению и упрощению доступа к ссылке, Vimium предоставляет унифицированное воспроизведение идентификаторов и упрощает процесс просмотра доступных на странице ссылок. На рис. 16.14 приведен пример страницы социальной сети с пронумерованными пояснениями разделов страницы. В зависимости от разрешения вашего носителя значки/идентификаторы могут быть расплывчатыми, но при просмотре они становятся четкими.

- SC — профиль автора в социальной сети.
- AD — список друзей автора.
- FD — список групп автора.
- DE — ярлык «Текстовый редактор Vim» на форуме социальной сети.
- XX, где XX — идентификатор контакта для отправки сообщения.



Рис. 16.14. Страница социальной сети со значками Vimium

Vimium также позволяет использовать команду Vim F, аналогичную команде f, но с возможностью открытия ссылки, связанной с идентификатором, на новой вкладке.

Поиск по тексту

Точно так же, как символ / инициирует поиск в Vim, Vimium инициирует поиск аналогичным способом. Важно убедиться, что поиск запущен, проверив наличие окна ввода в нижней части окна браузера (рис. 16.15).



Рис. 16.15. Поиск текста с помощью Vimium

Большинство опытных пользователей браузеров уже знают, что **Ctrl+F** или **CMD+F** активирует внутреннюю функцию браузера по поиску на странице. Однако в духе работы с Vim использование символа / является более естественным и удобным способом «продолжать работать с исходными клавишами». По мере ввода текста в строке поиска браузер перемещает вас к первому вхождению и подсвечивает соответствующий шаблон. Нажатие **Enter** завершает строку поиска.

Как и в Vim, нажатие клавиш n и N в Vimium позволяет перейти к следующему и предыдущему сопоставлению шаблона поиска соответственно.



По умолчанию Vimium выполняет поиск в открытом тексте, то есть что вы видите, то и находите. Он поддерживает регулярные выражения, но мы предлагаем вам изучить этот вопрос самостоятельно.

Навигация по браузеру

Vimium использует перемещения Vim для навигации по истории и вкладкам браузера. Ниже представлен краткий обзор команд, которые вам уже должны быть знакомы:

- j, k — прокручивание браузера вверх и вниз по строкам за раз соответственно;
- H, L — переход на страницу назад и вперед соответственно. Обратите внимание, команда чувствительна к регистру!

Мы также используем две знакомые команды Vim для перехода к верхней или нижней части страницы:

- gg — переход к верхней части веб-страницы;
- GG — переход к нижней части веб-страницы.

Другие команды помогут вам перемещаться между страницами и вкладками:

- `]]`, `[[` — переход к следующей или предыдущей странице, если веб-сайт имеет несколько связанных страниц. Как правило, у таких страниц есть кнопки со стрелками влево и вправо или кнопки `NEXT` и `PREV`.

Хорошим примером здесь будет список обзоров товаров на сайте магазина, который содержит не одну страницу отзывов. Также эти команды могут быть полезны для перемещения между страницами, связанными с кликбейтными (интригующими) слайдами, без необходимости нажимать на сообщения «нажми на меня»;

- `^` (*стрелка вверх или каретка*) — переход к последней просмотренной вкладке. Например, если у вас открыто много вкладок, символ `^` быстро переместит вас к одной из двух последних использованных вкладок.

Полезные переназначения клавиш

Мы заметили, что команды `K` (перейти к вкладке справа) и `J` (перейти к вкладке слева) противоречат интуитивному порядку. Так, поскольку `k` используется для прокрутки вниз, более логичным было бы назначить `K` для перехода к ближайшей левой вкладке, а не к ближайшей правой. К счастью, это можно сделать в параметрах Vimium.

Аналогичным образом в разделе «Маркеры» главы 4 мы описали, как команда `vi ''` (два апострофа) возвращает в начало строки предыдущей отметки или контекста. Нам показалось полезным переназначить `''` для команды Vimium `visitPreviousTab` (то же самое, что и описанная выше `^`). Это позволит вам перемещаться между вкладками, вводя последовательно одиночную кавычку.

Для этого откройте параметры Vimium и введите пользовательские переназначения клавиш, как показано на рис. 16.16. Это включает все три переназначения клавиш, а также новое переназначение для `q`.

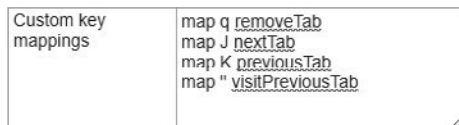


Рис. 16.16. Полезные переназначения клавиш Vimium

Когда вы потерялись и запутались

На любой странице, где активирован Vimium, вы можете быстро получить справку, введя `?`. Vimium накладывает на страницу краткое описание команд и их действий. Очень здорово и удобно, что действия Vimium работают на собственной странице

справки Vimium. Это пригодится для поиска специфических функций. Нажатие Esc возвращает вас к исходной веб-странице. На рис. 16.17 представлен неполный снимок экрана оверлея справки Vimium.

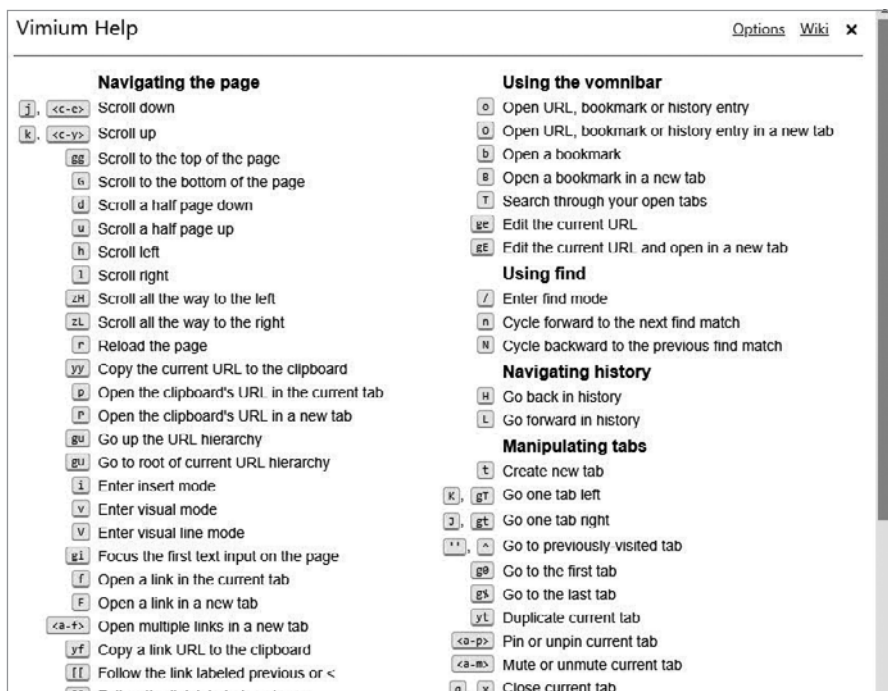


Рис. 16.17. Справка Vimium

Пример использования

Возьмем как пример обычное действие: онлайн-покупку ноутбука на крупном торговом веб-сайте.

Важным требованием является наличие у ноутбука емкой батареи. После отбора кандидатов для покупки мы легко можем перейти к просмотру всех отзывов, заранее зная, что они упорядочены на нескольких страницах с возможностью переключения между ними с помощью кнопок «следующий» и «предыдущий». Начиная с первой страницы, мы выполняем поиск любого упоминания о *батарее* с помощью `/battery Enter`. Затем мы можем перемещаться вверх и вниз по странице отзывов ко всем экземплярам и упоминаниям в комментариях батареи.

Далее, поскольку Vimium знает общие ссылки, которые ведут на следующие и предыдущие упорядоченные страницы, мы используем `[ ]` для перемещения между страницами отзывов. Vimium также запоминает шаблон поиска, что позволяет ис-

пользовать `n` и `N` для поиска дополнительных комментариев в отзывах, связанных с *батареями*. Это обычное упражнение, которое помогает быстро изучать товары и оценивать их эффективность, надежность и другие важные характеристики.

vi для MS Word и Outlook

Многие разработчики ПО, работающие в окружении Unix или GNU/Linux, иногда вынуждены использовать Microsoft Word для документации и Microsoft Outlook для электронной почты.

Мы, будучи сами разработчиками ПО, не понаслышке знаем, что смена инструмента после длительного кодирования в Vim может быть непростой. Использование команд `vi` для редактирования в Word позволяет нам создавать более качественную документацию быстрее. Как бы то ни было, мы стремимся минимизировать время, затрачиваемое на работу с Word.

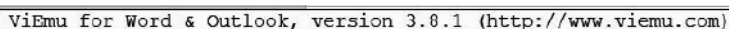
Мы также обнаружили, что Outlook не всегда удобен в использовании. Проще всего применить цепочку диалога, в которой можно ссылаться на комментарии в ней. Мы предпочли бы найти соответствующий текст и процитировать его, скопировав и вставив в наш комментарий. *Если бы только* мы могли применить `/` для поиска текста, `u` для копирования, а затем `"` (вернуться к последнему местоположению) и `p`, это позволило бы нам экономить время и сосредоточиться на том, что мы хотим сказать, вместо того чтобы отвлекаться на мышь и прокрутку.

К счастью, существует коммерческий плагин для Word и Outlook, который решает эти проблемы: ViEmu (<http://www.viemu.com>).

Арнольд больше не работает в Word или Outlook, но Элберт продолжает. Вначале он был скептически настроен, но в конце концов дал плагину шанс и купил его. Поскольку парадигма редактирования в Outlook по сути аналогична MS Word, а плагин создан для обоих, мы будем обсуждать Word, но имейте в виду, что все применимо и к Outlook.

ViEmu ведет себя очень похоже на `vi`, а плагин легко активировать и деактивировать в соответствии с вашими потребностями. Поскольку мы уже много раз описывали способы использования `vi`, мы не будем подробно описывать то же самое для ViEmu. Достаточно знать, что он имеет много функций, характерных для `vi`, так что пользователи Vim могут легко освоить его.

Бесплатная пробная версия ViEmu доступна на его официальном сайте <http://www.viemu.com>. После установки желтая строка состояния в нижней части приложения подскажет вам, активен ли плагин (рис. 16.18).



ViEmu for Word & Outlook, version 3.8.1 (<http://www.viemu.com>)

Рис. 16.18. Строка состояния ViEmu в Word или Outlook

Чтобы выключить ViEmu, нажмите **Ctrl Alt Shift+V**. На рис. 16.19 показана строка состояния, когда ViEmu неактивен.

ViEmu disabled - use ViEmu settings or Ctrl-Shift-Alt-V to toggle it back on

Рис. 16.19. Строка состояния ViEmu, когда ViEmu неактивен

Рисунок 16.20 отображает доступные параметры и настройки для плагина.

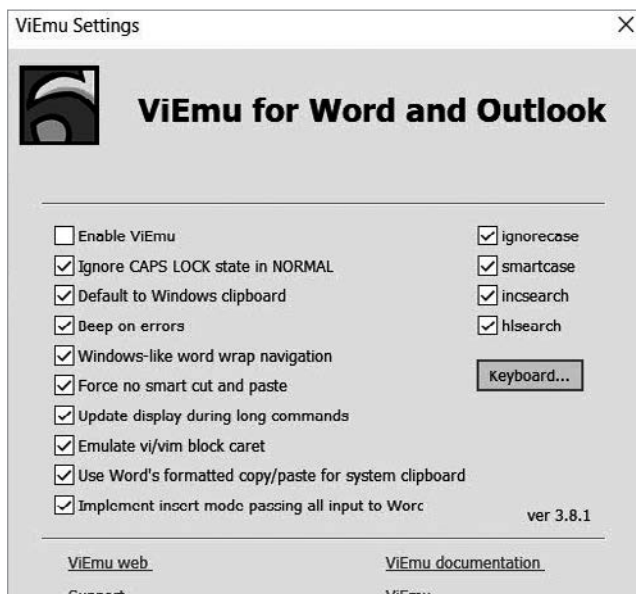


Рис. 16.20. Настройки ViEmu

ViEmu обеспечивает баланс между функциональностью **vi** и приверженностью Word. Стоит отметить некоторые моменты в поведении плагина, так как они связаны с тем, как команды **vi** обрабатываются в Word.

- Шрифты сохраняются в контексте документа.
- Упорядоченные списки автоматически перенумеровываются и перетасовываются с помощью команд **vi** (например, можно удалить элемент списка и поместить его в другое место в списке).
- Поскольку Word воспринимает абзацы как единый блок, команда **dd** (удалить строку) удалит весь абзац, а не только строку.

Ниже приведен пример магии упорядоченного списка с использованием первых трех навыков из книги Стивена Р. Кови «7 навыков высокоэффективных людей».

Во время ввода списка (рис. 16.21) мы обнаружили, что порядок навыков был неверный. Обратите внимание, что курсор находится на навыке 2.

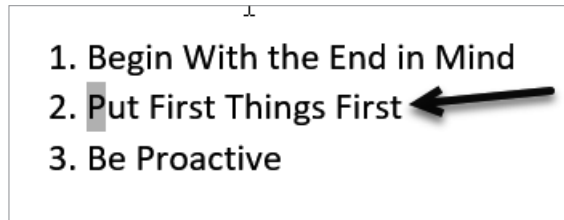


Рис. 16.21. Первые три из «7 навыков» в неправильном порядке

Мы ввели команду `vi ddp`, чтобы поменять местами строки, и — вуаля! Теперь они расположены в правильном порядке (рис. 16.22).

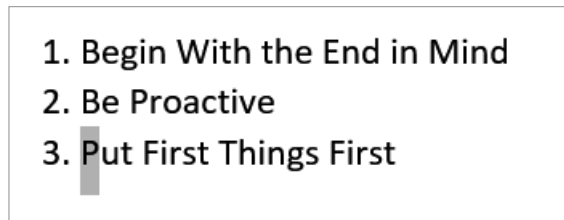


Рис. 16.22. Первые три навыка теперь в правильном порядке

И вновь все эти поведения стали удобными и знакомыми. Чтобы подогреть интерес и гарантировать надежность ViEmu, ниже мы приводим неполный список использованных и проверенных функций.

- Изменение/удаление в объектах:
 - в круглых скобках: `ci(, di(;`
 - в квадратных скобках: `ci[, di[;`
 - в фигурных скобках: `ci{, di{.`
- Удаление на объектах (удаление, *включая* символы в скобках):
 - в круглых скобках: `da(;`
 - в квадратных скобках: `da[;`
 - в фигурных скобках: `da{.`
- Полная и выборочная поддержка регулярных выражений. Вы можете настроить плагин для поддержки своих любимых функций регулярных выражений.
- Всегда доступная полная документация (http://www.viemu.com/wo/viemu_doc.html)!

Остальные *особенности* использования ViEmu мы оставили вам для самостоятельного изучения. Стоит с ними ознакомиться, если вы часто работаете в Word и предпочитаете поведение Vim.

На веб-сайте плагина описаны доступные параметры и цены.

Достойны упоминания: инструменты с некоторыми функциями vi

Сложно определить, что именно *является* частью vi, а что — *нет*. Взять хотя бы регулярные выражения, которые существовали задолго до появления vi и Vim. Тем не менее некоторые инструменты содержат удобные функции, которые отражают суть vi: эффективное быстрое редактирование, поиск и замена и т. д. Мы упоминаем о них здесь, чтобы подчеркнуть, что инструменты с такими функциями существуют и заслуживают внимания.

Google Mail

В Google Mail имеется собственный набор переназначенных быстрых клавиш клавиатуры, некоторые из которых знакомы пользователям vi. Для навигации по списку электронных писем достаточно использовать клавиши j и k. Клавиша s «выбирает» текущую строку для любых действий, x удаляет выбранное письмо, а mU помечает его как непрочитанное. Нажатие Enter открывает текущее электронное письмо.

Если письмо уже открыто, то x удаляет его, а u перемещает вас вверх по списку электронных писем.

Чтобы ознакомиться с полным списком быстрых клавиш, введите ? на панели списка (не в режиме ввода).

Microsoft PowerToys

В Microsoft есть дополнение PowerToys (<https://docs.microsoft.com/en-us/windows/powertoys>) с множеством крутых инструментов для улучшения работы Windows. Большинство из них выходят за рамки книги. Однако стоит упомянуть одну утилиту — PowerRename, которая позволяет переименовывать несколько файлов за раз, что обычно долгий и утомительный процесс. PowerRename выделяется своей способностью использовать регулярные выражения для идентификации файлов и для определения способа применения переименования.

В PowerToys также есть мощный инструмент переназначения Keyboard Manager, позволяющий задать команды клавишам так, как вам хочется. Хотя навигация по Рабочему столу Windows навряд ли будет похожей на работу с vi, по крайней мере у вас есть возможность легко настроить действия клавиш, подобно vi. Предыдущие менеджеры клавиатуры были либо платными, либо сложны в управлении, либо и то и другое.

Резюме

От хорошо известных примеров, таких как редактирование истории командной строки, до экосистем программного обеспечения, таких как IDE, и интернет-браузеров — по всему этому ясно, что vi является популярной парадигмой редактирования. Забавно, что Джон, автор плагина ViEmu для Microsoft Word, получал запросы от инженеров Microsoft на его плагин! Мы обнаружили, что практически все самые рейтинговые текстовые редакторы имеют возможность эмуляции Vim или плагины для этого. Даже в Emacs есть Viper — плагин для эмуляции vi! (См. раздел «Наслаждение чистым вкусом» приложения В.) Поэтому, если вы когда-нибудь окажетесь в новой среде, поищите настройки vi/Vim или плагины. Вероятно, они существуют.

ГЛАВА 17

Эпилог

Если вы дочитали до этой главы, мы благодарим вас за терпение и проявленный интерес.

Вы прошли долгий путь: начали с основ, включая определение текстовых редакторов и редактирования текста, затем изучили командный режим `vi` и базовый редактор `ex`, а также регулярные выражения и мощный язык команд, лежащий в основе `vi` и Vim.

После этого вы глубоко погрузились в некоторые из многочисленных функций Vim, которые делают его на порядок мощнее, чем исходный `vi`. Vim отлично подходит для редактирования обычного текста (мы написали на нем книгу), но он по-настоящему блистателен как редактор для программистов.

И наконец, вы увидели, как расширяемость Vim позволяет встраивать его в полноценную IDE, а также то, как командно-управляемую модель редактирования `vi` можно найти в других инструментах.

Если вы по-прежнему управляете текстовой информацией с клавиатуры, рекомендуем вам потратить время на изучение Vim. Как и при вводе текста вслепую всеми пальцами рук, использование Vim при написании кода делает редактирование текста и программ более эффективным, чем любой управляемый мышью GUI-редактор.

Наслаждайтесь!

Приложения

Здесь представлены справочные материалы, которые должны быть интересны пользователю `vi` и `Vim`.

- Приложение А «Редакторы `vi`, `ex` и `Vim`».
- Приложение Б «Установка параметров».
- Приложение В «Более светлая сторона `vi`».
- Приложение Г «`vi` и `Vim`: исходный код и разработка».

ПРИЛОЖЕНИЕ А

Редакторы vi, ex и Vim

В данном приложении собраны основные функции **vi** в формате краткой справки. Здесь приведены команды, вводимые в строку приглашения с двоеточием (их также называют командами **ex**, потому что они были введены в первоначальной версии этого редактора), а также наиболее популярные функции Vim.

В этом приложении представлены следующие темы.

- Синтаксис командной строки.
- Обзор операций **vi**.
- Команды **vi**.
- Конфигурации **vi**.
- Основы **ex**.
- Алфавитный указатель команд **ex**.

Синтаксис командной строки

Три наиболее распространенных способа запуска сеанса Vim:

```
vim [параметры] файл  
vim [параметры] -с номер файл  
vim [параметры] -с /шаблон файл
```

Вы можете открыть *файл* для редактирования, выбрав *номер* строки или первую строку, соответствующую *шаблону*. Если *файл* не указан, редактор открывается с пустым буфером.

Параметры командной строки

Поскольку vi и ex — это одна программа, у них общие параметры. Тем не менее некоторые опции имеют смысл только для одной из версий программы. Квадратные скобки указывают на необязательные параметры. Параметры командной строки, специфичные для Vim, помечены отдельно.

- **+*[число]*** — начать редактирование с номера строки *число* или с последней строки файла, если *число* опущено.
- **+/*шаблон*** — начать редактирование с первой строки, сопоставимой с *шаблоном*¹.
- **+?*шаблон*** — начать редактирование с последней строки, сопоставимой с *шаблоном*².
- **-b** — редактирование файла в двоичном режиме. {Vim}
- **-с *команда*** — выполнить данную команду ex при запуске. Для vi допустим только один параметр -с, а Vim принимает до десяти. Более старая версия этого параметра **+*команда*** поддерживается до сих пор.
- **--cmd *команда*** — как -с, но выполняет *команду* до считывания какого-либо конфигурационного файла. {Vim}
- **-C** — Solaris 10 vi: то же, что и -x, но предполагает, что файл уже зашифрован. Не подходит для Solaris 11 vi.

Vim: запуск редактора в сопоставимом с vi режиме.

- **-d** — запуск в режиме сравнения (diff). Работает как vimdiff. {Vim}
- **-D** — режим отладки для использования со сценариями. {Vim}
- **-e** — запуск как в режиме ex (скорее редактирование строк, чем полноэкранный режим).
- **-h** — вывести на экран справку, а затем выйти. {Vim}
- **-i *файл*** — использовать указанный файл вместо файла по умолчанию (~/.viminfo) для сохранения или восстановления состояния Vim. {Vim}

¹ Если вы используете Vim и настроили .viminfo для сохранения последнего местоположения курсора в файле, поиск начнется в указанном направлении с восстановленного местоположения курсора. Также, в зависимости от параметра wgapscan (ws) в конфигурационном файле .vimrc, поиск может остановиться (:set nowgapscan) или продолжиться (:set wgapscan) после достижения верхней или нижней строки в файле.

² То же.

- `-l` — войти в режим Lisp для запущенных команд Lisp (поддерживается не во всех версиях).
- `-L` — перечислить файлы, которые были сохранены из-за прерванного сеанса редактора или полного отказа системы (поддерживается не во всех версиях). Для Vim этот параметр такой же, как и `-r`.
- `-m` — запустить редактор с выключенным параметром `write`, чтобы пользователь не смог записывать в файлы. Vim все еще разрешает вносить изменения в буфер, но не позволит записывать файл даже с помощью `:w!`. {Vim}
- `-M` — не позволяет изменять текст в файлах. Аналогично `-m` плюс блокирует любые изменения в буфере. {Vim}
- `-n` — не использовать файл подкачки; записывать изменения только в память. {Vim}
- `--noplugin` — не загружать никакие плагины. {Vim}
- `-N` — запустить Vim в несопоставимом с `vi` режиме. {Vim}
- `-o[число]` — запустить Vim с определенным *числом* открытых окон. По умолчанию открывается одно окно для каждого файла. {Vim}
- `-O[число]` — запустить Vim с определенным *числом* открытых окон, расположенных горизонтально (разделены вертикально) на экране. {Vim}
- `-r [файл]` — режим восстановления; восстанавливает и возобновляет редактирование файла после прерванного сеанса или полного отказа системы. Если файл не указан, перечисляет файлы, доступные к восстановлению.
- `-R` — редактировать файлы в режиме «только для чтения». Vim предупреждает вас, если вы пытаетесь внести изменения. При сохранении файла с изменениями Vim предложит вам отменить контекст только для чтения.
- `-s` — бесшумный режим; без отображения приглашения. Полезно при выполнении сценария. Это поведение также можно установить через старый параметр `-`. Для Vim применим, только когда используется с `-e`.
- `-s файл_сценария` — считывать и выполнять команды, заданные в указанном файле сценария, как если бы они были введены с клавиатуры. {Vim}
- `-S командный_файл` — считывать и выполнять команды, заданные в командном файле после загрузки любых файлов для редактирования, указанных в командной строке. Сокращение от `vim -c 'source командный_файл'`. {Vim}
- `-t мег` — редактировать файл, содержащий *мег*, и поместить курсор на его определение.
- `-T min` — установить параметр `term` (тип терминала). Это значение переопределяет переменную окружения `$TERM`. {Vim}

- `-u файл` — считать информацию о конфигурации из указанного конфигурационного файла вместо `.vimrc`. Если аргумент файла равен `NONE`, Vim не считывает никаких конфигурационных файлов, не загружает плагинов и выполняется в совместимом режиме. Если аргумент равен `NORC`, он также не считывает никаких конфигурационных файлов, но загружает плагины. {Vim}
- `-v` — запуск в полноэкранном режиме (по умолчанию для `vi`).
- `--version` — вывести на экран информацию о версии, а затем выйти. {Vim}
- `-V[число]` — подробный режим; выводит сообщения о том, какие параметры устанавливаются и какие файлы считываются или записываются. Вы можете установить уровень детализации, чтобы увеличить или уменьшить количество получаемых сообщений. Значение по умолчанию равно 10, что означает высокую детализацию. {Vim}
- `-w строки` — установить размер окна таким образом, чтобы *строки* отображались за раз; полезно при редактировании через междугороднее интернет-соединение. Старые версии `vi` не разрешают пробел между параметром и его аргументом. Vim не поддерживает этот параметр.
- `-W файл_сценария` — записать все введенные команды из текущего сеанса в указанный файл сценария. Созданный таким образом файл можно использовать с параметром `-s`. {Vim}
- `-x` — запросить ключ, который будет использоваться для зашифровки или дешифровки файла с помощью `crypt` (поддерживается не во всех версиях)¹.
- `-y` — немодальный `vi`; выполнять Vim только в режиме ввода, без командного режима. Это то же самое, что вызов Vim в качестве `evim`. {Vim}
- `-Z` — запустить Vim в ограниченном режиме. Запретить команды оболочки или приостановку редактора. {Vim}

Хотя большинство людей знают команды `ex` только по их использованию внутри `vi`, этот редактор также существует как отдельная программа и может быть вызван из оболочки (к примеру, для редактирования файлов как части сценария). Внутри `ex` вы можете ввести `vi` или команду `visual`, чтобы запустить `vi`. Аналогичным образом внутри `vi` можно ввести `Q`, чтобы выйти из редактора `vi` и войти в `ex`.

Существует несколько способов выйти из `ex`:

- `:x` — сохранить изменения и выйти;
- `:q!` — выйти без сохранения изменений;
- `:vi` — войти в редактор `vi`.

¹ Этот параметр устарел; шифрование команды `crypt` неэффективно, и не следует ее использовать.

Обзор операций vi

В этом разделе представлены:

- режимы vi;
- синтаксис команд vi;
- команды строки состояния.

Командный режим

Как только файл открыт, вы переходите в командный режим, в котором можете:

- перейти в режим ввода;
- запустить команды редактирования;
- переместить курсор на другую позицию в файле;
- вызвать команды ex;
- вызвать оболочку;
- сохранить текущую версию файла;
- выйти из редактора.

Режим ввода

В режиме ввода вы можете набирать новый текст в файле. Обычно режим ввода активируется нажатием **i**. Клавиша **Esc** служит для выхода из данного режима и возврата в командный. Полный список команд, с помощью которых можно войти в режим ввода, будет представлен позднее в подразделе «Команды вставки».

Синтаксис команд vi

В vi команды редактирования имеют следующий общий вид:

[n] оператор [m] перемещение

К основным операторам редактирования относятся:

- **c** — начать изменение;
- **d** — начать удаление;
- **y** — начать копирование.

Если текущая строка является объектом операции, *перемещение* является оператором: *cc*, *dd*, *yy*. В противном случае операторы редактирования воздействуют на объекты, заданные с помощью команд перемещения курсора или сопоставления с шаблоном. Например, *cf* . заменяет текст вплоть до следующей точки. *n* и *m* — это количество повторений операции или число объектов, над которыми выполняется операция. Если заданы *n*, и *m*, то результат равен $n \times m$.

Объектом операции может быть любой из следующих текстовых фрагментов:

- *слово* — включает символы до символа пробела (пробел или табуляция) или знака пунктуации. Объект с заглавной буквы является разновидностью формы, которая распознает только символы пробелов;
- *предложение* — включает текст вплоть до символов *.*, *!*, *?*, после которых следуют два пробела. Vim ищет лишь один последующий пробел;
- *абзац* — включает текст вплоть до следующей пустой строки или макроса абзаца *nroff/troff*, определенного параметром *para=*;
- *раздел* — включает текст вплоть до следующего заголовка раздела *nroff/troff*, определенного параметром *sect=*;
- *перемещение* — включает текст вплоть до символа или другого текстового объекта, указанного спецификатором перемещения, включая поиски по шаблону.

Примеры

- *2cw* — заменить следующие два слова.
- *d}* — удалить вплоть до следующего абзаца.
- *d^* — обратное удаление до начала строки.
- *5yy* — скопировать следующие пять строк.
- *y]]* — скопировать вплоть до следующего раздела.
- *cG* — заменить до конца буфера редактирования.

Больше команд и примеров вы найдете в пункте «Изменение и удаление текста» далее.

Визуальный режим (только Vim)

Vim предоставляет дополнительное средство — *визуальный режим*. Он позволяет вам выделять фрагменты текста, которые могут затем использоваться в командах редактирования, таких как удаление или сохранение (восстановление исходного текста из буфера). Графические версии Vim позволяют использовать мышь для

выделения текста аналогичным образом. Для дополнительной информации вернитесь к подразделу «Перемещение в визуальном режиме» в главе 8.

- **v** — выделить текст в визуальном режиме по одному символу за раз.
- **V** — выделить текст в визуальном режиме по одной строке за раз.
- **Ctrl+V** — выделить текст в визуальном режиме фрагментами.

Команды строки состояния

Большинство команд не отображаются на экране, когда вы их вводите. Однако для редактирования этих команд используется строка состояния в нижней части экрана.

- **/** — выполнить прямой поиск шаблона.
- **?** — выполнить обратный поиск шаблона.
- **:** — вызвать команду **ex**.
- **!** — вызвать команду **Unix**, которая принимает в качестве входных данных объект из буфера и заменяет его на выходные данные команды. После **!** вводится команда перемещения, чтобы описать, что нужно передать команде **Unix**. Сама команда отображается в строке состояния.

Команды, которые задаются в строке состояния, должны быть завершены нажатием клавиши **Enter**¹. Кроме того, сообщения об ошибках и выходные данные команды **Ctrl+G** также отображаются в строке состояния.

Команды vi

В командном режиме **vi** предоставляет обширный набор одноклавишных команд, в то время как **Vim** предоставляет дополнительные многоклавишные команды.

Команды перемещения

Какие-то версии **vi** не распознают расширенные клавиши клавиатуры (например, стрелки, страница вверх, страница вниз, начало, вставить и удалить), в то время как другие могут это делать. Тем не менее все версии распознают клавиши из этого раздела. Многие пользователи **vi** предпочитают использовать эти клавиши, так как

¹ Если вы используете исходный **vi** или установили параметр **Vim compatible**, нажатие **Esc** выполнит команду, что может быть неожиданным. **Vim** (в режиме **nosocompatible**) просто отменит команду и не предпримет никаких действий.

это помогает им держать пальцы на основном ряду клавиатуры. Число перед командой повторяет ее указанное количество раз. Команды перемещения также можно использовать после оператора, работающего с текстом, который перемещается.

Символ

- h, j, k, l — влево, вниз, вверх, вправо (←, ↓, ↑, →).
- Пробел — вправо.
- Backspace — влево.
- Ctrl+H — влево.
- Ctrl+N — вниз.
- Ctrl+P — вверх.

Текст

- w, b — вперед, назад на слово (буквы, числа и символы подчеркивания составляют слова).
- W, B — вперед, назад на Слово (только символы пробелов разделяют элементы).
- e — конец слова.
- E — конец Слова.
- ge — конец предыдущего слова. {Vim}
- gE — конец предыдущего Слова. {Vim}
-), (— начало следующего, текущего предложения.
- }, { — начало следующего, текущего абзаца.
-]], [[— начало следующего, текущего раздела.
-][, [[] — конец следующего, текущего раздела. {Vim}

Строки

Длинные строки в файле могут отображаться на экране в виде нескольких строк, переносясь с одной на другую. Хотя большинство команд работают со строками, как это было определено в файле, некоторые команды обрабатывают строки в том виде, в котором они появляются на экране. Параметр Vim `wrap` позволяет вам управлять отображением длинных строк.

- 0, \$ — первая или последняя позиция текущей строки соответственно.
- ^, _ — первый непустой символ текущей строки.

- `+`, `-` — первый непустой символ следующей или предыдущей строки соответственно.
- `Enter` — первый непустой символ следующей строки.
- `n|` — столбец *n* на текущей строке.
- `g0`, `g$` — первая или последняя позиция строки экрана соответственно. {Vim}
- `g^` — первый непустой символ строки экрана. {Vim}
- `gm` — середина строки экрана. {Vim}
- `gk`, `gj` — перейти вверх или вниз на одну строку экрана соответственно¹. {Vim}
- `H` — верхняя строка экрана (исходная позиция — Home).
- `M` — средняя строка экрана.
- `L` — последняя строка экрана.
- *число*`H` — *число* строк после верхней строки.
- *число*`L` — *число* строк перед последней строкой.

Экраны

- `Ctrl+F`, `Ctrl+B` — прокрутить вперед или назад один экран соответственно.
- `Ctrl+D`, `Ctrl+U` — прокрутить вниз или вверх половину экрана соответственно.
- `Ctrl+E`, `Ctrl+Y` — отобразить еще одну строку внизу иливерху экрана соответственно. Если возможно, Vim удерживает позицию курсора на одной и той же строке: например, если курсор расположен на строке 50, то при использовании этой команды экран прокрутится вверх или вниз, но курсор при этом останется на строке 50.
- `z Enter` — отобразить строку с курсором в верхней части экрана.
- `z.` — отобразить строку с курсором в средней части экрана.
- `z-` — отобразить строку с курсором в нижней части экрана.
- `Ctrl+L` — перерисовать экран (без прокрутки).
- `Ctrl+R` — `vi`: перерисовать экран (без прокрутки); Vim: повторить последнее отмененное изменение.

Перемещение по видимой части экрана

- `H` — перейти в начало — к первому символу в верхней строке на экране.
- `M` — перейти к первому символу средней строки на экране.

¹ Если убрать `g`, команда не потеряет своих свойств.

- L — перейти к первому символу последней строки на экране.
- n H — перейти к первому символу строки на n строк ниже верхней строки.
- n L — перейти к первому символу строки на n строк выше последней строки.

Поиск

- /шаблон — прямой поиск шаблона. Завершается нажатием Enter.
- /шаблон/+ n — перейти к строке n после шаблона. Осуществляется прямой поиск шаблона.
- /шаблон/- n — перейти к строке n перед шаблоном. Осуществляется прямой поиск шаблона.
- ?шаблон — обратный поиск шаблона. Завершается нажатием Enter.
- ?шаблон?+ n — перейти к строке n после шаблона. Осуществляется обратный поиск шаблона.
- ?шаблон?- n — перейти к строке n перед шаблоном. Осуществляется обратный поиск шаблона.
- noh — приостановить подсвечивание поиска до следующего поиска. {Vim}
- n — повторить предыдущий поиск.
- N — повторить предыдущий поиск в обратном направлении.
- / — повторить предыдущий поиск вперед. Завершается нажатием Enter.
- ? — повторить предыдущий поиск назад. Завершается нажатием Enter.
- * — найти слово под курсором. Находит только точное совпадение. {Vim}
- # — найти слово под курсором в обратном направлении. Находит только точные совпадения. {Vim}
- g* — найти слово под курсором. Находит как точное совпадение, так и если оно часть другого слова. {Vim}
- g# — найти слово под курсором в обратном направлении. Находит как точное совпадение, так и если оно часть другого слова. {Vim}
- % — найти совпадение для текущей круглой, фигурной или квадратной скобки.
- f x — переместить курсор вперед к x в текущей строке.
- F x — переместить курсор назад к x в текущей строке.
- t x — переместить курсор вперед к символу перед x в текущей строке.
- T x — переместить курсор назад к символу после x в текущей строке.
- , — инвертировать направление поиска последней f, F, t или T.
- ; — повторить последнюю f, F, t или T.

Нумерация строк

- **Ctrl+G** — отобразить номер текущей строки.
- **gg** — перейти к первой строке в файле. {Vim}
- *число* **G** — перейти к строке с указанным номером.
- **G** — перейти к последней строке в файле.
- **:** *число* — перейти к строке с указанным номером.

Маркеры

- **m x** — поместить маркер *x* в текущую позицию.
- **` x** — (обратная кавычка) переместить курсор на маркер *x*.
- **' x** — (апостроф) перейти в начало строки, содержащей *x*.
- **``** — (обратные кавычки) вернуться к позиции перед самым последним переходом.
- **''** — (два апострофа) как и предыдущее, только в начало строки.
- **'''** — (апостроф и кавычка) перейти в позицию при последнем редактировании файла. {Vim}
- **`[, `]** — (обратная кавычка и квадратная скобка) перейти в начало или в конец предыдущей текстовой операции. {Vim}
- **'[, ']** — (апостроф и квадратная скобка) как и предыдущее, только в начало строки, где произошла операция. {Vim}
- **`.** — (обратная кавычка и точка) перейти к последнему изменению в файле. {Vim}
- **'. .** — (апостроф и точка) как и предыдущее, только в начало строки. {Vim}
- **'0** — (апостроф и ноль) перейти к вашему местоположению при последнем выходе из Vim. {Vim}
- **:marks** — перечислить активные маркеры. {Vim}

Команды вставки

- **a** — добавить после курсора.
- **A** — добавить в конец строки.
- **c** — начать операцию замены.
- **C** — заменить до конца строки.

- **gi** — вставить туда, где вы последний раз редактировали файл. {Vim}
- **gI** — вставить в начало строки. {Vim}
- **i** — вставить перед курсором.
- **I** — вставить в начало строки.
- **o** — открыть строку под курсором.
- **O** — открыть строку над курсором.
- **R** — начать перезапись текста.
- **s** — заменить символ.
- **S** — заменить всю строку.
- **Esc** — выйти из режима ввода.

Следующие команды работают в режиме ввода.

- **Backspace** — удалить предыдущий символ.
- **Delete** — удалить текущий символ.
- **Tab** — вставить табуляцию.
- **Ctrl+A** — повторить последнюю вставку. {Vim}
- **Ctrl+D** — сдвинуть строку влево к предыдущей ширине сдвига.
- **^ Ctrl+D** — сдвинуть курсор в начало строки, но только для одной строки.
- **0 Ctrl+D** — сдвинуть курсор в начало строки и переустановить уровень автоматического отступа на ноль.
- **Ctrl+E** — вставить символ, находящийся прямо под курсором. {Vim}
- **Ctrl+H** — удалить предыдущий символ (аналог **Backspace**).
- **Ctrl+I** — вставить табуляцию.
- **Ctrl+K** — начать вставку диакритического знака. {Vim}
- **Ctrl+N** — вставить следующее завершение шаблона слева от курсора. {Vim}
- **Ctrl+P** — вставить предыдущее завершение шаблона слева от курсора. {Vim}
- **Ctrl+T** — сдвинуть строку вправо к следующей ширине сдвига. {Vim}
- **Ctrl+U** — удалить текущую строку.
- **Ctrl+V** — вставить следующий символ дословно.
- **Ctrl+W** — удалить предыдущее слово.
- **Ctrl+Y** — вставить символ, находящийся прямо над курсором. {Vim}
- **Ctrl+[** — (аналог **Esc**) выйти из режима ввода.

Некоторые управляющие символы, перечисленные в предыдущей таблице, можно задать с помощью `stty`. Настройки вашего эмулятора терминала могут отличаться.

Команды редактирования

Напоминаем, что `c`, `d` и `u` являются базовыми операторами редактирования.

Изменение и удаление текста

Далее представлен список большинства самых распространенных операций.

- `cw` — заменить слово.
- `cc` — заменить строку.
- `c$` — заменить текст с текущей позиции до конца строки.
- `C` — аналог `c$`.
- `dd` — удалить текущую строку.
- *число* `dd` — удалить указанное число строк.
- `d$` — удалить текст с текущей позиции до конца строки.
- `D` — аналог `d$`.
- `dw` — удалить слово.
- `d}` — удалить вплоть до следующего абзаца.
- `d^` — удалить назад до начала строки.
- `d/шаблон` — удалить вплоть до первого вхождения *шаблона*.
- `dn` — удалить вплоть до следующего вхождения *шаблона*.
- `df x` — удалить в текущей строке все символы до именованного символа `x` включительно.
- `dt x` — аналог `df x`, только не включая `x`.
- `dL` — удалить вплоть до последней строки на экране.
- `dG` — удалить до конца файла.
- `gqap` — переформатировать текущий абзац в `textwidth`. {Vim}
- `g~w` — переключить регистр слова. {Vim}
- `guw` — изменить регистр слова на нижний. {Vim}
- `gUw` — изменить регистр слова на верхний. {Vim}
- `p` — вставить после курсора последний удаленный или скопированный текст.

- P — вставить перед курсором последний удаленный или скопированный текст.
- gp — аналог p, но оставляет курсор в конце вставленного текста. {Vim}
- gP — аналог P, но оставляет курсор в конце вставленного текста. {Vim}
-]p — аналог p, но соответствует текущему отступу. {Vim}
- [p — аналог P, но соответствует текущему отступу. {Vim}
- r x — заменить символ на x.
- R *текст* — заменить новым *текстом* (перезаписать), начиная с местоположения курсора. Нажатие Esc завершает режим замещения.
- s — заменить один символ.
- 4s — заменить четыре символа.
- S — заменить всю строку.
- u — отменить последнее изменение.
- Ctrl+R — повторить последнее изменение. {Vim}
- U — восстановить текущую строку.
- x — удалить символ в текущей позиции курсора.
- X — удалить один предыдущий символ.
- 5X — удалить пять предыдущих символов.
- . — повторить последнее изменение.
- ~ — инвертировать регистр и переместить курсор право. {vi и Vim с параметром notildeop}
- ~w — инвертировать регистр слова. {Vim с параметром tildeop}
- ~~ — инвертировать регистр строки. {Vim с параметром tildeop}
- Ctrl+A — увеличить число под курсором. {Vim}
- Ctrl+X — уменьшить число под курсором. {Vim}

Копирование и перемещение

Имена регистров — это буквы от a до z. Имена в верхнем регистре добавляют текст в соответствующий регистр.

- Y — копировать текущую строку.
- yy — копировать текущую строку.
- " x yy — копировать текущую строку в регистр x.
- ye — копировать текст в конец слова.

- `yw` — как `ye`, но включает символ пробела после слова.
- `y$` — копировать остальную часть строки.
- `" x dd` — удалить текущую строку в регистр `x`.
- `" x d перемещение` — удалить в регистр `x`.
- `" x p` — поместить содержимое регистра `x`.
- `y]]` — скопировать вплоть до заголовка следующего раздела.
- `J` — объединить текущую и следующую строки.
- `gJ` — эквивалент `J`, но без вставки пробела. {Vim}
- `:j` — то же, что и `J`.
- `:j!` — то же, что и `gJ`.

Сохранение и выход

Запись файла — это замена содержимого файла текущим текстом в буфере редактирования.

- `ZZ` — закрыть редактор и записать файл, если были внесены изменения.
- `:x` — аналог `ZZ`.
- `:wq` — записать файл и закрыть редактор.
- `:w` — записать файл (но остаться в редакторе).
- `:w файл` — сохранить копию *файла*.
- `:n, m w файл` — записать строки с *n* по *m* в новый *файл*.
- `:n, m w >> файл` — записать строки с *n* по *m* в существующий *файл*.
- `:w!` — записать файл (игнорируя защиту от записи).
- `:w! файл` — перезаписать *файл* с текущим текстом.
- `:w%.new` — записать текущий буфер под именем *файл.new*.
- `:q` — выйти из редактора (не работает, если были внесены изменения).
- `:q!` — принудительно выйти из редактора (без сохранения изменений).
- `Q` — выйти из `vi` и вызвать `ex`.
- `:vi` — вернуться в `vi` после команды `Q`.
- `%` — заменяет текущее имя файла в командах редактирования.
- `#` — заменяет альтернативное имя файла в командах редактирования.

Доступ к нескольким файлам

- `:e файл` — редактировать еще один *файл*; текущий файл становится альтернативным, названным `#`.
- `:e!` — перезагрузить текущий файл, отбросив все изменения, сделанные с момента последнего сохранения.
- `:e +файл` — открыть *файл* для редактирования и перейти на последнюю строку.
- `:e +число файл` — открыть *файл* для редактирования и перейти на строку с указанным номером.
- `:e #` — открыть альтернативный файл.
- `:ta тег` — редактировать файл и перейти к строке, содержащей указанный *тег*.
- `:n` — перейти к следующему файлу в списке файлов.
- `:n!` — принудительный переход к редактированию следующего файла без сохранения изменений в текущем.
- `:n файлы` — указать новый список *файлов*.
- `:rewind` — редактировать первый файл в списке файлов.
- `Ctrl+G` — отобразить информацию о текущем номере строки или столбца.
- `:args` — отобразить список редактируемых файлов.
- `:prev` — перейти к предыдущему файлу в списке файлов. {Vim}
- `:last` — перейти к последнему файлу в списке файлов. {Vim}

Команды окон (Vim)

В следующей таблице перечислены общие команды для управления окнами в Vim. Для краткости управляющие символы обозначены `^`. В разделе «Алфавитный указатель команд ex» далее можно найти дополнительные команды, такие как `split`, `vsplit` и `resize`.



Все односимвольные клавиши — в нижнем регистре. Клавиши верхнего регистра обозначаются с помощью префикса Shift.

- `:new` — открыть новое окно.
- `:new файл` — открыть указанный файл в новом окне.
- `:sp[lit] [файл]` — разделить текущее окно по горизонтали и открыть указанный файл для редактирования в новом окне.

- `:sv[split]` [*файл*] — то же, что и `:sp`, но делает новое окно доступным только для чтения.
- `:sn[ext]` [*файл*] — открыть следующий файл в списке файлов в новом окне.
- `:vsp[lit]` [*файл*] — то же, что и `:sp`, но разделяет окно вертикально.
- `:clo[se]` — закрыть текущее окно.
- `:hid[e]` — скрыть текущее окно, если оно не является единственным видимым окном.
- `:on[ly]` — сделать текущее окно единственным видимым окном.
- `:res[ize]` *число* — изменить размер окна на *число* строк.
- `:wa[ll]` — записать все измененные буферы в их файлы.
- `:qa[ll]` — закрыть все буферы и выйти.
- `Ctrl+W S` — то же, что и `:sp`.
- `Ctrl+W N` — то же, что и `:new`.
- `Ctrl+W ^` — открыть новое окно с альтернативным (отредактированным ранее) файлом.
- `Ctrl+W C` — то же, что и `:clo`.
- `Ctrl+W O` — то же, что и `:only`.
- `Ctrl+W J` — переместить курсор в следующее окно.
- `Ctrl+W K` — переместить курсор в предыдущее окно.
- `Ctrl+W P` — переместить курсор в предыдущее окно.
- `Ctrl+W H` / `Ctrl+W L` — переместить курсор в окно в левой или в правой части экрана соответственно.
- `Ctrl+W T` / `Ctrl+W B` — переместить курсор в окно в верхней или в нижней части экрана соответственно.
- `Ctrl+W Shift+K` / `Ctrl+W Shift+B` — переместить текущее окно в верхнюю или в нижнюю часть экрана соответственно.
- `Ctrl+W Shift+H` / `Ctrl+W Shift+L` — переместить текущее окно в крайнюю левую или правую часть экрана соответственно.
- `Ctrl+W R` / `Ctrl+W, Shift+R` — поворачивать окна по часовой или против часовой стрелки соответственно.
- `Ctrl+W +` / `Ctrl+W -` — увеличить или уменьшить размер текущего окна соответственно.
- `Ctrl+W =` — выровнять все окна по высоте.

Взаимодействие с системой

- `:r файл` — прочитать содержимое *файла* после курсора.
- `:r !команда` — прочитать выходные данные *команды* после текущей строки.
- `:число r ! команда` — как и предыдущее, только помещает после строки *число* (использовать ноль для верхней части файла).
- `:! команда` — выполнить *команду*, затем вернуться.
- `!перемещение команда` — отправить текст, предусмотренный *перемещением*, в *команду*; заменить выходными данными.
- `:n,t ! команда` — отправить строки *n–t* в *команду*; заменить выходными данными.
- `число !! команда` — отправить *число* строк в *команду*; заменить выходными данными.
- `:!!` — повторить последнюю системную команду.
- `:sh` — создать оболочку; вернуться в редактор с *EOF*.
- **Ctrl+Z** — приостановить редактор, возобновить с помощью `fg`. Это сворачивает `gvim` в значок.
- `:so файл` — считать и выполнить команды *ex* из *файла*.

Макросы

- `:ab ввод вывод` — использовать *ввод* в качестве сокращения для *вывода* в режиме ввода.
- `:unab ввод` — удалить сокращение для *ввода*.
- `:ab` — список сокращений.
- `:map строка_символов последовательность` — определить *строку_символов* как *последовательность* команд. Используйте `#1`, `#2` и т. д. для функциональных клавиш.
- `:unmap строка` — удалить переназначение для *строки* символов.
- `:map` — список строк символов, которые переназначены.
- `:map! строка_символов последовательность` — определить *строку_символов* как *последовательность* команд. Работает в режиме ввода текста. (Когда в режиме ввода введен символ, который соответствует заданному набору символов, он автоматически заменяется заданной последовательностью.)
- `:unmap! строка` — удалить переназначение на режим ввода для *строки* символов (вам может понадобиться заключить символы в кавычки с помощью **Ctrl+V**).

- `:map!` — список набора символов, переназначенных для режима ввода.
- `q x` — записать введенные символы в регистр, заданный буквой `x`. Если буква прописная, добавьте в регистр. {Vim}
- `q` — остановить запись. {Vim}
- `@x` — выполнить регистр, заданный буквой `x`.
- `@@` — выполнить последнюю команду регистра.

В `vi` следующие символы не используются в командном режиме и могут быть переназначены пользователем для своих нужд.

- *Буквы:* `g`, `K`, `q`, `V` и `v`.
- *Управляющие клавиши:* `Ctrl+A`, `Ctrl+K`, `Ctrl+O`, `Ctrl+W`, `Ctrl+X`, `Ctrl+_` и `Ctrl+\`.
- *Символы:* `_`, `*`, `\`, `=` и `#`.



Символ `=` используется `vi`, если установлен режим `Lisp`. В некоторых версиях `vi` данные символы могут быть задействованы, поэтому рекомендуем проверить их перед переназначением.

Vim не использует `Ctrl+K`, `Ctrl+_` или `Ctrl+\`. Наберите `:help noremap` для получения дополнительной информации о функциональных возможностях переназначений в Vim.

Прочие команды

- `<` — сдвинуть текст, описанный следующей командой перемещения, влево на одну ширину сдвига. {Vim}
- `>` — сдвинуть текст, описанный следующей командой перемещения, вправо на одну ширину сдвига. {Vim}
- `<<` — сдвинуть строку влево на одну ширину сдвига (восемь пробелов по умолчанию).
- `>>` — сдвинуть строку вправо на одну ширину сдвига (восемь пробелов по умолчанию).
- `>}` — сдвинуть вправо до конца абзаца.
- `<%` — сдвигать влево до соответствующей закрывающей или открывающей круглой, фигурной или квадратной скобки (курсор должен находиться на соответствующем символе).

- `[n] ==` — отступить на *n* строк в стиле языка C или с помощью программы, указанной в параметре `equalprg`. {Vim}
- `g` — запустить несколько многосимвольных команд в Vim.
- `K` — найти слово под курсором на страницах руководства (или через программу, определенную в `keywordprg`). {Vim}
- `Ctrl+O` — вернуться к предыдущему переходу. {Vim}
- `Ctrl+Q` — эквивалент `Ctrl+V`. {Vim} (В некоторых терминалах возобновляет поток данных.)
- `Ctrl+T` — вернуться к предыдущей позиции в стеке тегов. {Solaris vi, Vim.}
- `Ctrl+]` — найти тег в тексте под курсором.
- `Ctrl+\
— войти в режим строкового редактирования ex.`
- `Ctrl+^` — вернуться к ранее отредактированному файлу.

Конфигурация vi

В этом разделе описываются:

- команда `:set`;
- пример файла `.exrc`.

Команда :set

Команда `:set` позволяет задавать параметры, изменяющие характеристики вашей среды редактирования. Параметры можно записать в файлы `~/.exrc` или `~/.vimrc` или установить во время сеанса редактирования.

Двоеточие вводить не обязательно, если команда помещена в файл `.exrc`:

- `:set x` — активировать булев параметр *x*; отображает значения других параметров;
- `:set no x` — отключить параметр *x*. Не ставьте пробел между `no` и *x*;
- `:set x=значение` — присвоить *значение* параметру *x*;
- `:set` — отобразить измененные параметры;
- `:set all` — отобразить все параметры;
- `:set x ?` — отобразить значение параметра *x*.

Для получения дополнительной информации о параметрах `:set` для версий vi Heirloom и Solaris и для Vim обратитесь к приложению Б «Установка параметров».

Пример файла .exrc

В файле сценария `ex` комментарии начинаются с символа двойных кавычек. Ниже представлен пример пользовательского файла `.exrc`:

```
set nowrapscan           " Searches don't wrap at end of file
set wrapmargin=7         " Wrap text at 7 columns from right margin
set sections=SeAhBhChDh nomescg " Set troff macros, disallow message
map q :w^M:n^M           " Alias to move to next file
map v dwElp              " Move a word
ab ORA O'Reilly Media, Inc. " Input shortcut
```



Псевдоним `q` не нужен для Vim, в котором есть команда `:wn`. Псевдоним `v` скроет команду Vim `v`, которая вводит посимвольно операцию визуального режима.

Основы ex

Строчный редактор `ex`, команды которого работают с текущей строкой или с диапазоном строк в файле, служит основой для экранного редактора `vi`. Чаще всего `ex` используется из `vi`, где командам `ex` предшествует двоеточие, а выполняются они при нажатии **Enter**.

Также `ex` можно вызвать из командной строки тем же способом, что и `vi` (как и выполнить сценарий `ex`), или с помощью команды `vi Q` для входа в `ex`.

Синтаксис команд ex

Чтобы ввести в команду `ex` из `vi`, наберите:

```
: [адрес] команда [параметры]
```

Символ `:` обозначает команду `ex`. Она отображается в строке состояния при вводе. Чтобы выполнить ее, нажмите **Enter**. Адрес — это номер строки или диапазон строк, которые являются объектом команды. Параметры и адреса описаны далее. Подробную информацию о командах `ex` можно найти в разделе «Алфавитный указатель команд `ex`» далее.

Вы можете выйти из `ex` несколькими способами:

- `:x` — сохранить изменения и выйти;
- `:q!` — выйти без сохранения изменений;
- `:vi` — переключиться на редактор `vi` в текущем файле.

Адреса

Если адреса не заданы, то текущая строка рассматривается как объект команды. Если адрес — диапазон строк, формат следующий:

x, y

где x и y — это первая и последняя адресные строки (x должен предшествовать y в буфере). И x , и y могут быть как номером строки, так и символом. Если вместо $,$ использовать $;$, то текущая строка будет установлена в x перед интерпретацией y . Нотация $1, \$$ адресует все строки в файле, как это делает $\%$.

Символы адресов

- $1, \$$ — все строки в файле.
- x, y — строки с x до y .
- $x; y$ — строки с x до y с текущей строкой, сбрасываемой на x перед вычислением y .
- $\emptyset$ — верхняя часть файла.
- $.$ — текущая строка.
- *число* — абсолютный номер строки согласно *числу*.
- $\$$ — последняя строка.
- $\%$ — все строки; то же, что и $1, \$$.
- $x-n$ — n строк перед x .
- $x+n$ — n строк после x .
- $-[n]$ — одна или n предыдущих строк.
- $+ [n]$ — одна или n строк впереди.
- $'x$ — (апостроф) строка, отмеченная x .
- $''$ — (два апострофа) предыдущая отметка.
- */шаблон/* — вперед к строке, сопоставимой с *шаблоном*.
- *?шаблон?* — назад к строке, сопоставимой с *шаблоном*.

См. главу 6 для получения дополнительной информации об использовании шаблонов.

Параметры

! — обозначает разновидность команды, замещающей нормальное поведение. Символ **!** должен следовать сразу же за командой.

количество — количество повторений команды. *Количество* не может предшествовать команде, потому что число, стоящее перед командой *ex*, воспринимается как адрес строки. Например, *d3* удаляет три строки, начиная с текущей, а *3d* удаляет строку 3.

файл — имя файла, на который воздействует команда. Символ % обозначает текущий файл, а # — предыдущий.

Алфавитный указатель команд *ex*

Команды *ex* можно вводить, указав любое уникальное сокращение. Далее полное имя команды будет отображаться в качестве заголовка, а самое короткое возможное сокращение — в строке синтаксиса снизу. Предполагается, что примеры вводятся из *vi*, поэтому они включают приглашение *:*.

abbreviate

ab [*строка текст*]

Определить *строку*, которая будет переведена в *текст* при вводе. Если *строка* и *текст* не указаны, выведется список текущих сокращений.

Примеры

Примечание: ^M появляется, когда вы вводите ^V с последующим нажатием Enter.

```
:ab ora O'Reilly Media, Inc.  
:ab id Name:^MRank:^MPhone:
```

append

[*адрес*] *a*[!]
текст
.

Добавляет новый *текст* к указанному *адресу* или к текущему, если иной не указан. Символ ! деактивирует используемую во время ввода настройку *autoindent*, если она была активирована. Текст нужно вводить после команды. Чтобы завершить ввод, поставьте точку на новой строке.

Пример

```
:a                                Начать добавление в текущую строку
Append this line
and this line too.
.                                Завершить ввод текста для добавления
```

args

```
ar
args файл ...
```

Вывести элементы списка аргументов (файлы, названные в командной строке), при этом текущий аргумент (редактируемый файл) выводится в квадратных скобках ([]).

Второй синтаксис — для Vim, что позволяет вам сбросить список файлов для редактирования.

bdelete

```
[число] bd[!] [число]
```

Извлечь буфер *число* и удалить его из списка буферов. Символ ! служит для принудительного удаления несохраненного буфера. Буфер можно также задать по имени файла. Удаляет текущий буфер, если никакой не задан. {Vim}

buffer

```
[число] b[!] [число]
```

Начать редактирование буфера *число* в списке буферов. Чтобы принудительно переключиться из несохраненного буфера, добавьте символ !. Буфер можно также задать по имени файла. Редактирование текущего буфера продолжается, если никакой не задан. {Vim}

buffers

```
buffers[!]
```

Вывести элементы списка буферов. Некоторые буферы (например, удаленные) могут не отображаться в списке. Для их отображения добавьте символ ! в конце

команды. Кроме того, команда может быть выполнена с использованием сокращения `ls. {Vim}`

cd

`cd каталог`
`chdir каталог`

Заменить текущий каталог редактора на указанный в параметре *каталог*.

center

`[адрес] се [ширина]`

Центрировать строку внутри заданной *ширины*. Если *ширина* не задана, используется `textwidth. {Vim}`

change

`[адрес] с[!]`
текст
.

Заменить заданные строки на *текст*. Если вы хотите переключать настройку `autoindent` во время ввода нового *текста*, добавьте символ `!` в конце команды. Чтобы завершить ввод, поставьте точку на отдельной строке.

close

`clo[!]`

Закрыть текущее не последнее окно. Если буфер, отображаемый в окне, не используется в другом окне, он выгружается из памяти. Однако, если буфер содержит несохраненные изменения, он не будет закрыт, но вы можете скрыть его, добавив `!` в конце команды. `{Vim}`

copy

`[адрес] со назначение`

Скопировать строки из заданного *адреса* в указанный адрес *назначения*. Команда `t` (сокращение от `to`) является синонимом `copy`.

Пример

:1,10 co 50 *Скопировать первые 10 строк сразу после строки 50*

cquit

cq[!]

Выйти из Vim с указанием кода ошибки. Это полезно при работе с функцией Bash «вызвать внешний редактор для командной строки», когда вы не хотите, чтобы Bash выполнил текст в буфере редактирования. {Vim}

delete

[адрес] d [регистр] [количество]

Удалить строки, указанные как *адрес*. Если задан *регистр*, текст сохраняется и добавляется в именованный регистр. Имена регистров — это строчные буквы от a до z. Имена в верхнем регистре добавляют текст в соответствующий регистр. Если задано *количество*, команда удаляет соответствующее количество строк.

Примеры

:/Part I/,/Part II/-1d	<i>Удалить строку над Part II</i>
:/main/+d	<i>Удалить строку под main</i>
..,\$d x	<i>Удалить с этой строки до последней в регистр x</i>

edit

e[!] [+число] [имя_файла]

Начать редактировать *имя_файла*. Если *имя_файла* не задано, начать заново с копии текущего файла. Добавьте ! для редактирования нового файла, даже если текущий не был сохранен с момента последнего изменения. С заданным аргументом *+число* начать редактировать с указанной строки. В качестве альтернативы *число* может быть шаблоном в виде */шаблон*.

Примеры

:e file	<i>Редактировать файл в текущем буфере редактирования</i>
:e +/^Index #	<i>Редактировать альтернативный файл в первой строке, сопоставимой с заданным шаблоном</i>
:e!	<i>Начать заново в текущем файле</i>

file

`f [имя_файла]`

Переименовать текущий буфер на *имя_файла*. При следующем сохранении буфер будет записан в файл *имя_файла*. При смене имени устанавливается флаг «не отредактирован», чтобы показать, что вы редактируете несуществующий файл. Если новое имя файла совпадает с именем уже существующего файла на диске, вам нужно будет использовать `:w!`, чтобы перезаписать существующий файл. При определении имени файла можно использовать символ `%`, чтобы указать текущее имя. Символ `#` стоит использовать для указания альтернативного имени файла. Если *имя_файла* не задано, выводится текущее имя файла и состояние буфера.

Пример

```
:f %.new
```

fold

адрес `fo`

Свернуть строки, заданные адресом. Сворачивает несколько строк на экране в одну, которую позднее можно будет развернуть. Это не влияет на текст файла. {Vim}

foldclose

[*адрес*] `foldc[!]`

Закрыть свернутые строки по указанному адресу или по текущему, если иной не задан. Добавьте символ `!` в конце команды, чтобы закрыть более одного уровня свертываний. {Vim}

foldopen

[*адрес*] `foldo[!]`

Открыть свернутые строки по указанному адресу или по текущему, если иной не задан. Добавьте символ `!` в конце команды, чтобы открыть более одного уровня свертываний. {Vim}

global

[адрес] g[!]/шаблон/[команды]

Выполнить *команды* для всех строк, содержащих *шаблон*, или, если задан *адрес*, для всех строк внутри этого диапазона. Если *команды* не указаны, вывести все такие строки. Добавьте *!*, чтобы выполнить *команды* для всех строк, *не* содержащих *шаблон*. Смотрите также *v* далее в этом списке.

Примеры

:g/Unix/p	Вывести все строки, содержащие "Unix"
:g/Name:/s/tom/Tom/	Заменить "tom" на "Tom" во всех строках, содержащих "Name:"

hide

hid

Закрыть текущее окно, если оно не является последним, не удаляя при этом буфер из памяти. Данную команду безопасно использовать с несохраненным буфером. {Vim}

insert

[адрес] i[!]
текст

.

Вставить *текст* в строку перед заданным *адресом* или по текущему, если иной не указан. Добавьте символ *!*, чтобы переключить настройку *autoindent*, которая используется во время ввода *текста*. Чтобы завершить ввод, поставьте точку на отдельной строке.

join

[адрес] j[!] [количество]

Объединить строки в заданном диапазоне в одну строку, отрегулировав пробелы так, чтобы после точки (.) стояло два пробела, перед) не было пробелов и в остальных случаях был один пробел. Используйте *!*, чтобы не выполнять коррекцию пробелов.

Пример

`:1,5j!` *Объединить первые пять строк, сохраняя символ пробела*

jumps

`ju`

Вывести список переходов, используемых с командами `Ctrl+I` и `Ctrl+O` (то есть просмотреть историю своих перемещений по файлу в редакторе). Список переходов — это запись большинства команд перемещения, которые переносят несколько строк. Позиция курсора перед каждым переходом записывается. {Vim}

k

`[адрес] k` *символ*

Аналог `mark`. Описание `mark` представлено далее в списке.

last

`la[!]`

Редактировать последний файл из списка аргументов командной строки. {Vim}

left

`[адрес] le [n]`

Выводить по левому краю заданные *адресом* строки или текущую строку, если *адрес* не задан. Сделать отступ для строк равным *n* пробелам. {Vim}

list

`[адрес] l` [*количество*]

Вывести заданные строки таким образом, чтобы табуляция отображалась как `^I`, а конец строк — как `$. 1` — это временная версия `:set list`.

map

map[!] [*набор_символов команды*]

Использовать *набор_символов* как макрос для часто повторяющихся *команд* (действий). Используйте *!*, чтобы создать макрос для режима ввода. При отсутствии аргументов перечисляет определенные на данный момент макросы.

Примеры

:map K dwwP	<i>Переставить два слова</i>
:map q :w^M:n^M	<i>Записать текущий файл; перейти к следующему файлу</i>
:map! + ^[bi(^[ea)	<i>Заключить предыдущее слово в круглые скобки</i>



В Vim есть команды K и q, которые будут скрыты в примере псевдонимов.

mark

[*адрес*] та *символ*

Отметить указанную строку *символом* — одиночной строчной буквой. То же, что и k. Вернуться позднее к строке с помощью 'x (апостроф плюс x, где x — *символ*). Vim также использует для маркеров символы в верхнем регистре и числовые символы. Буквы нижнего регистра работают так же, как и в vi. Прописные буквы связаны с именами файлов и могут быть использованы между несколькими файлами. Однако числовые маркеры поддерживаются в специальном файле .vinfo, и их нельзя установить с помощью этой команды.

marks

marks [*символы*]

Вывести список маркеров, определенных *символом*, или все текущие маркеры, если *символ* не задан. {Vim}

Пример

:marks abc *Вывести маркеры a, b и c*

mkexrc

mk[!] *файл*

Создать файл `.exrc`, содержащий команды `set` для измененных параметров `ex` и переназначений клавиш. Сохраняет текущие настройки параметра, позволяя вам восстановить их позднее. *Файл* по умолчанию — это `.exrc` в текущем каталоге, если не указано другое. {Vim}

move

[адрес] *т* *назначение*

Переместить указанные в *адресе* строки по заданному *назначению*.

Пример

`:/Note/m /END/` *Переместить фрагмент текста после строки, содержащей "END"*

new

[*n*] new

Создать новое окно высотой *n* строк с пустым буфером. {Vim}

next

n[!] [[+число] *список_файлов*]

Отредактировать следующий файл в списке аргументов командной строки. Используйте `args` для перечисления этих файлов. Если предоставлен *список_файлов*, заменяет текущий список аргументов на *список_файлов*, а редактирование начнется в первом файле из нового списка. Если задать аргумент *+число*, редактирование начнется в строке *число*. В качестве альтернативы *число* может быть шаблоном в виде */шаблон*.

Пример

`:n char*` *Начать редактирование всех файлов "описания главы"*

nohlsearch

noh

Временно остановить выделение всех совпадений поиска, если используется параметр `hlsearch`. Выделение возобновляется в следующем поиске. {Vim}

number

[адрес] nu [количество]

или

[адрес] # [количество]

Вывести каждую строку, заданную *адресом*, предварив ее номером строки ее буфера. Допускается использовать `#` в качестве альтернативного сокращения для значения *количество*, которое задает количество показываемых строк, начиная с *адреса*.

only

on[!]

Сделать текущее окно единственным на экране. Окна, открытые в измененных буферах, не удаляются с экрана (скрываются), пока вы не используете символ `!`. {Vim}

open

[адрес] o [/шаблон/]

Войти в режим открытия (`vi`) в строки, заданные *адресом*, или в строки, сопоставимые с *шаблоном*. Выйти из режима открытия с помощью `Q`. Режим открытия позволяет вам использовать регулярные команды `vi`, но только по одной строке за раз. Он может быть полезен при очень удаленных интернет-подключениях `ssh`.

packadd

pa[!] имя_пакета...

Найти каталог плагина, сопоставимого с *именем_пакета*, и загрузить плагин. Для получения дополнительной информации см. справку Vim. {Vim}

preserve

pre

Сохранить текущий буфер редактора, как если бы система вот-вот должна была выйти из строя.

previous

prev[!]

Отредактировать предыдущий файл из списка аргументов командной строки. {Vim}

print

[адрес] р [количество]

Вывести на экран строки, заданные *адресом*. *Количество* указывает количество выводимых строк, начиная с *адреса*. Р — это еще одно сокращение.

Пример

:100;+5p

Отобразить строку 100 и следующие пять строк

put

[адрес] пу [символ]

Поместить ранее удаленные или скопированные из именованного регистра строки, заданные *символом*, в строку, заданную *адресом*. Если *символ* не задан, восстанавливает последний удаленный или скопированный текст.

qall

qa[!]

Закрыть все окна и завершить текущий сеанс редактирования. При использовании с символом ! аннулирует все несохраненные изменения. {Vim}

quit

q[!]

Завершить текущий сеанс редактирования. При использовании с символом ! аннулирует все несохраненные изменения. Если сеанс редактирования включает дополнительные файлы в списке аргументов, к которым не обращались, выход осуществляется с помощью q! или двух q. Vim закрывает окно редактирования, только если на экране есть еще открытые окна.

read

[адрес] r имя_файла

Скопировать текст *имени_файла* после строки, заданной *адресом*. Если *имя_файла* не указано, используется текущее имя файла.

Пример

:0r \$HOME/data *Считать названный файл в верхнюю часть текущего файла*

read

[адрес] r !команда

Считать выходные данные *команды* в текст после строки, заданной *адресом*.

Пример

:\$r !spell % *Поместить результаты проверки орфографии в конец файла*

recover

rec [файл]

Восстановить *файл* из области сохранения системы.

redo

red

Восстановить последнее отмененное изменение. То же, что и **Ctrl+R**. {Vim}

resize

res $[[\pm]n]$

Установить размер текущего окна равным n строк в высоту. Если указан + или –, увеличить или уменьшить высоту текущего окна на n строк соответственно. {Vim}

rewind

rew $[!]$

Вернуться к первому файлу из списка аргументов и начать его редактирование. Добавьте !, чтобы перемотать список, даже если текущий файл не был сохранен с момента последнего изменения.

right

[адрес] ri [ширина]

Выровнять по правому краю заданные *адресом* строки или текущую строку, если адрес не задан, по *ширине* столбца. Если ширина не задана, используется значение параметра `textwidth`. {Vim}

sbnext

[число] sbn [число]

Разделить текущее окно и начать редактировать буфер *число* из списка буферов. Если число не указано, редактируется следующий буфер в списке буферов.

sbuffer

[число] sb [число]

Разделить текущее окно и начать редактировать буфер *число* из списка буферов в новом окне. Редактируемый буфер может быть также задан именем файла. Если буфер не задан, откроется текущий буфер в новом окне. {Vim}

set

se *параметр1* *параметр2*...

Установить значение для опции с каждым *параметром* или, если не задан параметр, вывести на экран все опции, настройки по умолчанию которых были изменены. Для булевых опций каждый параметр можно сформулировать как *параметр* или *нопараметр*; другие опции могут быть назначены с помощью синтаксиса *параметр=значение*. Укажите *all*, чтобы перечислить текущие настройки. Форма *set параметр?* отображает значение *параметра*. Таблицы, где перечислены параметры *set*, можно найти в приложении Б.

Примеры

```
:set nows wm=10  
:set all
```

shell

sh

Создать новую оболочку. Возобновить редактирование после выхода из нее.

snext

```
[n] sn [[+число] список_файлов]
```

Разделить текущее окно и начать редактирование следующего файла из списка аргументов командной строки. Если указано значение *n*, редактируется файл в списке под этим номером. Если указан *список_файлов*, текущий список аргументов заменяется на *список_файлов*, после чего начинается редактирование первого файла. При наличии аргумента *+число* работа начинается со строки *число*. В качестве альтернативы *число* может быть шаблоном вида */шаблон*. {Vim}

source

so *файл*

Считать и выполнить команды *ex* из *файла*.

Пример

```
:so $HOME/.exrc
```

split

[*n*] sp [*+число*] [*имя_файла*]

Разделить текущее окно и загрузить *имя_файла* в новое окно или загрузить текущий буфер в оба окна, если *имя_файла* не указано. Сделать новое окно высотой *n* строк, или, если *n* не задано, окно разделяется на равные части. При наличии аргумента *+число* редактирование начинается со строки *число*. *Число* также может быть шаблоном вида */шаблон*. {Vim}

sprevious

[*n*] spr [*+число*]]

Разделить текущее окно и начать редактирование предыдущего файла из списка аргументов командной строки в новом окне. Если *число* указано, редактируется *n*-й предыдущий файл. При наличии аргумента *+число* редактирование начинается со строки *число*. *Число* также может быть шаблоном вида */шаблон*. {Vim}

stop

st

Приостановить сеанс редактирования. То же, что и **Ctrl+Z**. Для возобновления сеанса используйте команду оболочки **fg**.

substitute

[*адрес*] s [*/шаблон/замена/*] [*параметры*] [*количество*]

Заменить первый экземпляр *шаблона* в каждой указанной строке на *замена*. Если *шаблон* и *замена* не указаны, повторяется последняя замена. *Количество* определяет количество строк для замены, начиная с *адреса*.

Параметры

- **c** — запрашивать подтверждение перед каждым изменением.
- **g** — заменить все экземпляры *шаблона* в каждой строке (глобальный).
- **p** — вывести на экран последнюю строку, в которой была произведена замена.

Примеры

<code>:1,10s/yes/no/g</code>	<i>Заменить в первых десяти строках</i>
<code>:%s/[Hh]ello/Hi/gc</code>	<i>Подтверждение глобальных замен</i>
<code>:s/Fortran/\U&/ 3</code>	<i>Перевести в верхний регистр "Fortran" в следующих трех строках</i>
<code>:g/^[0-9][0-9]*s//Line &:/</code>	<i>Каждой строке, начинающейся с одной или более цифр, добавить "Line" и двоеточие</i>

suspend

su

Приостановить сеанс редактирования. То же, что и **Ctrl+Z**. Для возобновления сеанса используйте команду оболочки **fg**.

sview

`[n] sv [+число] [имя_файла]`

То же, что и **split**, но для нового буфера устанавливается параметр **readonly**. {Vim}

t

`[адрес] t назначение`

Скопировать строки *адреса* по указанному *назначению*. **t** эквивалентно **сору** и сокращение для **то**.

Пример

`:%t$` *Скопировать файл и добавить его в конец*

tag

`[адрес] ta мет`

Найти в файле *тегов* файл и строку, сопоставимые с *тегом*, и начать там редактирование.

Пример

Запустите `ctags`, затем переключитесь на файл, содержащий `main`:

```
:!ctags *.c  
:tag main
```

tags

tags

Вывести на экран список тегов в стеке тегов. {Vim}

unabbreviate

una *слово*

Убрать *слово* из списка сокращений.

undo

u

Отменить изменения, внесенные последней командой редактирования. В `vi` команда отмены отменяет саму себя, повторяя то, что вы отменили. Vim поддерживает несколько уровней отмены. Используйте `redo`, чтобы повторить отмененные изменения в Vim.

unhide

[*n*] unh

Разделить экран, чтобы в каждом окне был показан активный буфер из списка буферов. Заданное значение *n* ограничивает количество окон. {Vim}

unmap

unm[!] *строка*

Удалить *строку* из списка макросов клавиатуры. Используйте `!`, чтобы удалить макрос для режима ввода.

V

[адрес] v/шаблон/[команда]

Выполнить *команду* для всех строк, *не* содержащих *шаблон*. Если *команда* не задана, выводятся все такие строки. v эквивалентна g!. Команда global была описана ранее.

Пример

:v/#include/d

Удалить все строки, кроме строк с "#include"

version

ve

Вывести на экран информацию о версии редактора.

view

vie [+число] [имя_файла]

То же, что и edit, но для файла устанавливается значение readonly. Когда команда выполняется в режиме ex, возвращает в нормальный или визуальный режим. {Vim}

visual

[адрес] vi [тип] [число]

Войти в визуальный режим (vi) в строке, заданной *адресом*. Вернуться в режим ex можно, введя Q. Тип может быть одним из: -, ^ или . (смотрите команду z далее в этом разделе). Число указывает изначальный размер окна.

visual

vi [+число] файл

Начать редактировать *файл* в визуальном режиме (vi) по выбору в строке *число*. В качестве альтернативы *число* может быть шаблоном вида /шаблон. {Vim}

vsplit

[*n*] vs [+*число*] [*имя_файла*]

То же, что и команда **split**, но разделяет экран по вертикали. Аргумент *число* можно использовать для указания ширины нового окна. {Vim}

wall

wa[!]

Записать все измененные буферы с именами файлов. Символ ! позволяет принудительно записать буферы, помеченные как **readonly**. {Vim}

wnext

[*n*] wn[!] [[+*число*] *имя_файла*]

Записать текущий буфер и открыть следующий файл в списке аргументов или *n*-й следующий файл, если задано. Если указано *имя_файла*, редактировать его следующим. При наличии аргумента +*число* редактирование начинается в строке *число*. Число также может быть шаблоном вида /*шаблон*. Добавьте ! для принудительной записи буферов, помеченных как **readonly**. {Vim}

wq

wq[!]

Записать файл и выйти из него одним действием. Файл всегда записывается. Флаг ! заставляет редактор перезаписать любое текущее содержимое файла.

wqall

wqa[!]

Записать все измененные буферы и выйти из редактора. Добавьте ! для принудительной записи буферов, помеченных как **readonly**. **xall** еще один псевдоним для этой команды. {Vim}

write

[адрес] w[!] [[>>] файл]

Записать строки, заданные *адресом*, в *файл* или записать все содержимое буфера, если *адрес* не указан. Если и *файл* не задан, содержимое буфера сохраняется в текущий файл. Если используется >> *файл*, то в конец указанного *файла* добавятся строки. Символ ! заставит редактор перезаписать любое текущее содержимое *файла*.

Примеры

:1,10w name_list	Скопировать первые десять строк в файл name_list
:50w >> name_list	Теперь добавить строку 50

write

[адрес] w !команда

Записать все строки, заданные *адресом*, в *команду*.

Пример

:1,66w !pr -h myfile lpr	Вывести на экран первую страницу файла
----------------------------	--

X

x

Запросить ключ шифрования. Эта команда может быть предпочтительней, чем :set key, так как ввод ключа не отображается в консоли. Чтобы удалить ключ шифрования, просто сбросьте параметр key на пустое значение. {Vim}

xit

x

Записать файл, применив все изменения, и затем выйти.

yank

[адрес] у [символ] [количество]

Поместить строки, заданные *адресом*, в именованный регистр *символ*. Имена регистров — это строчные буквы от а до z. Имена в верхнем регистре добавляют текст в соответствующий регистр. Если *символ* не задан, помещает строки в общий регистр. *Количество* указывает количество строк для копирования, начиная с *адреса*.

Пример

:101,200 уа а Скопировать строки 101-200 в регистр а

z

[адрес] z [тип] [количество]

Вывести текстовое окно, в котором заданная *адресом* строка — вверху. *Количество* определяет количество строк, которые необходимо отобразить.

Ввод текста

- + — поместить указанную строку в верхней части окна (по умолчанию).
- - — поместить указанную строку в нижней части окна.
- . — поместить указанную строку в центр окна.
- ^ — вывести на экран предыдущее окно.
- = — поместить указанную строку в центр окна и оставить текущую строку на этой же строке.

&

[адрес] & [параметры] [количество]

Повторить предыдущую команду замены (*s*). *Количество* — это количество строк для замены, начиная с *адреса*. *Параметры* те же, что и для команды замены.

Примеры

:s/Overdue/Paid/	Заменить один раз текущую строку
:g/Status/&	Повторить замену для всех строк "Status"
:g/Status/&g	Повторить замену для всех строк "Status" глобально

@

[адрес] @ [символ]

Выполнить содержимое регистра, заданное *символом*. Если указан *адрес*, переместить сначала курсор на него. Если *символ* — это @, повторяется последняя команда @.

=

[адрес] =

Вывести на экран строку, номер которой задан *адресом*. По умолчанию это номер последней строки.

!

[адрес] !команда

Выполнить *команду* в оболочке. Если указан *адрес*, использовать заданные им строки в качестве стандартного ввода для *команды* и заменить эти строки выходными данными или ошибкой выходных данных. Это называется *фильтрацией* текста через команду.

Примеры

```
:!ls  
:11,20!sort -f
```

*Вывести список файлов в текущем каталоге
Отсортировать строки 11–20 текущего файла*

<>

[адрес] < [количество]

или

[адрес] > [количество]

Сдвинуть строки, заданные *адресом*, либо влево (<), либо вправо (>). При смещении строк добавляются или удаляются только пробелы и табуляции. *Количество* определяет количество сдвигаемых строк, начиная с *адреса*. Параметр *shiftwidth* управляет числом столбцов, которые сдвигаются. Повторение < или > увеличивает величину сдвигов. Например, :>>> сместит в три раза дальше, чем :>.

~

[адрес] ~ [количество]

Заменить последнее использованное регулярное выражение (даже если оно получено из поиска, а не из команды *s*) на шаблон замены из последней команды *s* (замены). За подробностями обратитесь к главе 6.

Адрес

адрес

Вывести на экран строки, заданные *адресом*.

Enter

Вывести на экран следующую строку в файле. (Только для *ex*, а не для приглашения : в *vi*.)

Установка параметров

В этом приложении представлены некоторые параметры команды `set` для Heirloom `vi`, Solaris `/usr/xpg7/bin/vi` и Vim 8.2.

Параметры Heirloom и Solaris `vi`

Таблица Б.1 содержит краткое описание важных параметров команды `set`. В первом столбце в алфавитном порядке перечислены параметры, а в скобках указано сокращение параметра (если имеется). Второй столбец показывает значение по умолчанию, которое `vi` использует, если вы не введете явную команду `set` (либо вручную, либо в файле `.exrc`). В последнем столбце описывается, что делает параметр при активации.

Таблица Б.1. Параметры `set` Heirloom и Solaris `vi`

Параметр	По умолчанию	Описание
<code>autoindent (ai)</code>	<code>noai</code>	В режиме ввода делает такой же отступ для строки, как в строке сверху или снизу. Используется с параметром <code>shiftwidth</code>
<code>autoprint (ap)</code>	<code>ap</code>	Меняет отображение после каждой команды редактора. При глобальной замене отображается последняя замена
<code>autowrite (aw)</code>	<code>noaw</code>	Автоматически записывать (сохранять) файл, если он был изменен перед открытием другого файла с помощью <code>:n</code> или при вызове команды <code>Unix :!</code>
<code>beautify (bf)</code>	<code>nobf</code>	Игнорирует все управляющие символы во время ввода (кроме табуляции, новой строки или перевода страницы)
<code>directory (dir)</code>	<code>/var/tmp</code>	Называет каталог, в котором <code>ex/vi</code> хранит файлы буферов. Каталог должен быть доступен для перезаписи

Таблица Б.1 (продолжение)

Параметр	По умолчанию	Описание
edcompatible	noedcompatible	Запоминает флаги, использовавшиеся в последней команде замены (глобальная, подтверждающая), и использует их в следующей команде замены. Несмотря на имя, никакая из версий ed на самом деле этого не делает
errorbells (eb)	noerrorbells	Подает звуковой сигнал при возникновении ошибки
exrc (ex)	noexrc	Разрешает выполнение файлов .exrc, которые находятся за пределами домашнего каталога пользователя
flash (fp)	fp	Мигание экрана вместо звукового сигнала
hardtabs (ht)	8	Позволяет определить границы для вкладок аппаратного обеспечения терминала
ignorecase (ic)	noic	Не учитывает регистр при поиске
lisp	nolisp	Вставляет отступы в соответствующем формате Lisp. (), { }, [] и] изменяются, чтобы иметь значение для Lisp
list	nolist	Выводит табуляцию как ^I, а концы строк отмечает с помощью \$
magic	magic	Задаёт для символов подстановки . (точка), * (звездочка) и [] (квадратные скобки) специальное значение в шаблонах
mesg	mesg	Позволяет отображать системные сообщения в терминале во время редактирования в vi
novice	nonovice	Требует использования длинных имен команды ex, таких как сору или read. Только для Solaris vi
number (nu)	nonu	Отображает номера строк в левой части экрана во время сеанса редактирования
open	open	Позволяет войти в режим open или визуальный режим из ex. Хотя его нет в Solaris vi, данный параметр традиционно был в vi и может быть в вашей Unix-версии vi
optimize (opt)	noopt	Отменяет возвраты каретки в конце строк при выводе нескольких строк. Это ускоряет работу в медленных терминалах при выводе на экран строк с межстрочным символом пробела (пробелы или табуляции)
paragraphs (para)	IPLPpQP LIpplpipbp	Позволяет определить разделители абзацев для перемещения с помощью { или }. Пары символов в значении этого параметра являются именами макроса troff, которые определяют начало абзаца
prompt	prompt	Отображает приглашение ex (:), когда задана команда vi Q

Параметр	По умолчанию	Описание
readonly (ro)	noro	Любые записи (сохранения) файла будут выдавать ошибку, если не использовать ! после записи (работает с w, ZZ или autowrite)
redraw (re)		Перерисовывает экран при каждой правке (другими словами, режим ввода перекрывает существующие символы, а удаленные строки немедленно закрываются). Значение по умолчанию зависит от скорости передачи данных и типа терминала. noredraw используется в медленных терминалах: удаленные строки отображаются как @, а вставленный текст перезаписывает существующий, пока вы не нажмете Esc. Этот параметр, по сути, устарел; позвольте vi выбирать, как его установить
remap	remap	Разрешает последовательности вложенных переназначений
report	5	Отображает сообщение в строке состояния при каждой правке, влияющей по крайней мере на заданное количество строк. Например, bdd выводит сообщение «6 строк удалены»
scroll	[1/2 окна]	Задаёт количество строк для прокрутки с помощью команд ^D и ^U
sections (sect)	SHNNH NU	Позволяет определить разделители разделов для команд перемещения [[и]]. Пары символов в значении этого параметра являются именами макроса troff, с которых начинаются разделы
shell (sh)	/bin/sh	Имя пути оболочки, используемое для выходов из оболочки (:!) и команды оболочки (:sh). Значение по умолчанию является производным от окружения оболочки, что различается в разных системах, но часто приводит к /bin/sh
shiftwidth (sw)	8	Позволяет определить число пробелов в обратной (^D) табуляции при использовании параметра autoindent и для команд << и >>
showmatch (sm)	nosm	В vi при вводе) или } ненадолго перемещает курсор на сопоставимую (или {. Если соответствующей скобки нет, подается звуковой сигнал об ошибке. Очень полезно для программирования
showmode	noshowmode	В режиме ввода отображает в строке приглашения сообщение, обозначающее тип ввода, который вы выполняете, например OPEN MODE или APPEND MODE
slowopen (slow)		Задерживает отображение во время ввода. По умолчанию зависит от скорости передачи данных и типа терминала

Таблица Б.1 (продолжение)

Параметр	По умолчанию	Описание
sourceany	nosourceany	Разрешает чтение файлов .exrc, не принадлежащих текущему пользователю. Только для Heilroom vi
tabstop (ts)	8	Определяет количество пробелов, которые вставляются при табуляции во время сеанса редактирования (принтеры все еще используют системную табуляцию, равную 8)
taglength (tl)	0	Определяет количество символов, которые значимы для тегов. Значение по умолчанию (ноль) означает, что все символы значимы
tags	tags /usr/lib/ tags	Определяет имя пути файлов, содержащих теги. См. команду Unix ctags. По умолчанию vi ищет файл <i>тегов</i> в текущем каталоге и в /usr/lib/tags
tagstack	tagstack	Включает наложение местоположений тегов в стеке. Только для Solaris vi
term		Позволяет установить тип терминала
terse	noterse	Отображает более короткие сообщения об ошибках. Парадоксально, но для этого параметра нет сокращения
timeout (to)	timeout	Тайм-аут переназначений клавиатуры равен одной секунде*
ttytype		То же, что и term
warn	warn	Отображает предупреждение «Нет записи с последнего изменения» (No write since last change)
window (w)		Отображает определенное количество строк файла на экране. Значение по умолчанию зависит от скорости передачи данных и типа терминала
wrapmargin (wm)	0	Определяет правое поле. Если значение больше нуля, автоматически вставляется возврат каретки, чтобы разбить строку
wrapsan (ws)	ws	Начать сначала поиск по файлу при достижении конца файла (кольцевой поиск)
writeany (wa)	nowa	Разрешает сохранять в любой файл

* Когда у вас есть переназначения нескольких клавиш (например, :map zzz 3dw), вероятно, удобнее будет использовать `notimeout`. В противном случае вам придется вводить zzz в течение одной секунды. А когда у вас есть переназначение режима ввода на клавишу управления курсором (например, :map! ^[OB ^[ja), рекомендуем использовать `timeout`. Иначе vi не отреагирует на нажатие Esc, пока вы не наберете следующую клавишу.

Параметры Vim 8.2

В предыдущем разделе мы перечислили 46 параметров команды `set` для Heirloom и Solaris. В Vim 8.2 более 400 (!) таких параметров. В табл. Б.2 перечислены те из них, которые мы считаем наиболее полезными.

В этой таблице все описано очень кратко. Гораздо больше информации можно найти в файле онлайн-справки `Vim options.txt`.

Таблица Б.2. Параметры `set` Vim

Параметр	По умолчанию	Описание
<code>autoread (ar)</code>	<code>noautoread</code>	Выявляет, был ли файл внутри Vim изменен извне с помощью иного редактора, и автоматически обновляет буфер Vim, добавив измененную версию файла
<code>background (bg)</code>	<code>dark</code> или <code>light</code>	Vim попытается использовать цвета заднего и переднего планов, подходящие для конкретного терминала. Значение по умолчанию зависит от текущего терминала или системы управления окнами
<code>backspace (bs)</code>	<code>0</code>	Регулирует поведение клавиши <code>Backspace</code> в режиме ввода. Значения: <code>0</code> соответствует стандартному поведению <code>vi</code> , <code>1</code> — позволяет стирать символы перевода строки и отступы, <code>2</code> — позволяет стирать символы за пределами текущей строки, начала ввода и отступов
<code>backup (bk)</code>	<code>nobackup</code>	Создает резервную копию перезаписываемого файла и затем оставляет ее после успешной записи файла. Если вы хотите, чтобы старый файл удалялся автоматически после записи нового, используйте параметр <code>writebackup</code> . См. также <code>writebackup</code>
<code>backupdir (bdir)</code>	<code>., ~/tmp/, ~/</code>	Перечень каталогов для файла резервной копии, разделенных запятыми. Резервная копия файла создается в первом каталоге в списке, где это возможно. Если список пуст, то резервную копию создать невозможно. Имя <code>.</code> (точка) означает тот же каталог, что и у редактируемого файла
<code>backupext (bex)</code>	<code>~</code>	Строка, присоединенная к имени файла, чтобы создать имя резервной копии файла

Таблица Б.2 (продолжение)

Параметр	По умолчанию	Описание
binary (bin)	nobinary	Позволяет изменить несколько других параметров, чтобы упростить редактирование двоичных файлов. Предыдущие значения этих параметров сохраняются и восстанавливаются при выключении bin. Каждый буфер имеет свой набор сохраненных значений параметров. Этот параметр должен быть установлен до начала редактирования двоичного файла. Вы также можете посмотреть параметр командной строки -b
breakat (brk)	" ^I!@*~+;:,./?"	Прерывает строку на любом символе в строке breakat, если параметр set linebreak включен. См. также параметры breakindent, linebreak и showbreak, чтобы настроить эту функцию
breakindent (bri)	nobreakindent	Делает отступ для строк, перенесенных параметром breakat
cdpath (cd)	То же, что и значение в переменной окружения CDPATH	Список каталогов, в которых Vim будет осуществлять поиск при использовании команды ex cd или cdin аналогично тому, как это делает переменная \$CDPATH в оболочке. Если вы используете эту переменную в своей оболочке, данное поведение будет знакомо
cindent (cin)	nocindent	Включает автоматические умные отступы в коде на языке C
cinkeys (cink)	0{,0},: ,0#,!^F, o,O,e	Определяет список клавиш, которые могут привести к изменению отступа текущей строки в режиме ввода. Работает только при включенном cindent
cinoptions (cino)		Влияет на то, как cindent изменяет отступы строк в коде на языке C. Подробнее описано в онлайн-справке
cinwords (cinw)	if, else, while, do, for, switch	Когда вы начинаете строку с одного из этих ключевых слов, добавляется дополнительный отступ, если установлены smartindent или cindent. Для cindent это происходит только в подходящем месте (внутри {...})
cmdwinheight (cwh)	Число (по умолчанию 7)	Определяет количество строк в окне командной строки
colorcolumn (cc)	Пустая строка	Выделяет строки, перечисленные в разделенном запятыми списке. Это полезно для визуального выравнивания текста по вертикали

Параметр	По умолчанию	Описание
<code>columns (co)</code>	80 и ширина терминала	Задаёт количество символов, которые отображаются на каждой строке текстового файла. Если используется GUI-версия Vim, можно изменить значение, чтобы адаптировать его согласно своим предпочтениям и устройству. См. также <code>lines</code>
<code>comments (com)</code>	<code>sl:/*,mb:*,ex:*/,://, b:#,:%,:XCOMM,n:>,fb:-</code>	Разделённый запятыми список строк, которые могут начать строку комментария. Подробнее описано в онлайн-справке
<code>compatible (cp)</code>	<code>cp; noncp</code> , если обнаружен файл <code>.vimrc</code> или файл времени выполнения Vim <code>defaults.vim</code>	Делает поведение Vim более похожим на <code>vi</code> . Он включен по умолчанию, чтобы избежать сюрпризов. Наличие <code>.vimrc</code> выключает совместимость с <code>vi</code> ; обычно это желательный побочный эффект*
<code>completeopt (cot)</code>	<code>menu, preview</code>	Разделённый запятыми список параметров для завершения режима ввода
<code>cpoptions (cpo)</code>	<code>aABcEfS</code>	Последовательность односимвольных флагов, каждый из которых указывает на способ, с помощью которого Vim будет или не будет точно воспроизводить <code>vi</code> . Если флаг не задан, используются стандартные настройки Vim. Подробнее описано в онлайн-справке
<code>cursorcolumn (cuc)</code>	<code>nocursorcolumn</code>	Подсвечивает столбец, в котором находится курсор, с помощью <code>cursorcolumn</code> . Это полезно при выравнивании текста по вертикали. Может замедлить экранное отображение
<code>cursorline (cul)</code>	<code>nocursorline</code>	Подсвечивает строку, в которой находится курсор, с помощью <code>cursorline</code> . Помогает найти текущую строку в сеансе редактирования. Для достижения перекрестного эффекта рекомендуем использовать совместно с <code>cursorcolumn</code> . Может замедлить экранное отображение
<code>cursorlineopt (culopt)</code>	<code>string, ""</code>	Определяет поведение <code>cursorline</code> (<code>cursorline</code> должен быть для этого установлен). Для наибольшей эффективности рекомендуем установить его значение — <code>number</code> . Это выделит только номера строк. Хотя бывает полезно выделить всю строку, это действие может запутать, если использовать его совместно с подсветкой синтаксиса, так как выделение изменяет цвет и фон строк

Таблица Б.2 (продолжение)

Параметр	По умолчанию	Описание
define (def)	^#\s*define	Шаблон для поиска определений функций или макросов. Значение по умолчанию приведено для программ на языке C. Для C++ используйте ^\s*define\ [a-z]*\s*const\s*[a-z]*\). При использовании команды :set вам нужно удвоить обратные слешы
dictionary (dict)	Пустая строка	Разделенный запятыми список имен файлов для автодополнения (завершения)
digraph (dg)	nodigraph	Полезен для ввода диграфов с помощью <i>символ1</i> , <i>Backspace</i> , <i>символ2</i> . См. описание в разделе «Диграфы: символы, отличные от ASCII» главы 13
directory (dir)	., ~/tmp, /tmp	Список имен каталогов для файлов подкачки, разделенных запятыми. Файл подкачки будет создан в первом каталоге, где это возможно. Если список пуст, файл подкачки не будет использован и данные не будут сохранены! Имя . (точка) означает, что файл подкачки будет в том же каталоге, что и редактируемый файл. Использование . в начале списка рекомендуется, чтобы при повторном редактировании того же файла вы получили предупреждение
equalprg (ep)		Указывает на внешнюю программу для выравнивания текста (с помощью =). Если параметр пуст, используются внутренние функции форматирования
errorfile (ef)	errors.err	Указывает имя файла для регистрации ошибок в режиме быстрой правки. Когда используется аргумент командной строки -q, errorfile устанавливается равным этому аргументу
errorformat (efm)	(Too long to print)	scanf-подобное описание формата для строк в файле регистрации ошибок
expandtab (et)	noexpandtab	При вставке табуляции расширяет ее до подходящего числа пробелов
fileformat (ff)	unix	Отвечает за способ окончания строк при чтении/записи текущего буфера. По умолчанию опция содержит список из dos (CR/LF), unix (LF) и mac (CR), что означает, что Vim будет автоматически распознавать все три формата

Параметр	По умолчанию	Описание
<code>fileformats (ffs)</code>	<code>dos,unix</code>	Устанавливает список форматов окончания строк, которые Vim будет автоматически распознавать при открытии файлов. Несколько имен активируют автоматическое выявление конца строки при чтении файла
<code>fixendofline (fix eol)</code>	<code>boolean, on</code>	Контролирует поведение при сохранении файла с отсутствующими символами конца строки. Если вам это не нужно, например при редактировании двоичного файла, убедитесь, что параметр выключен
<code>formatoptions (fo)</code>	Vim по умолчанию: <code>tcq</code> ; vi по умолчанию: <code>vt</code>	Последовательность букв, определяющая, как выполняется автоматическое форматирование. Подробнее описано в онлайн-справке
<code>gdefault (gd)</code>	<code>nogdefault</code>	Вызывает команду замены для изменения всех вхождений
<code>guifont (gfn)</code>		Разделенный запятыми список шрифтов, которые можно использовать при запуске GUI версии Vim
<code>hidden (hid)</code>	<code>nohidden</code>	Скрывает текущий буфер, когда он выгружается из окна, вместо его удаления
<code>history (hi)</code>	Vim по умолчанию: <code>20</code> ; vi по умолчанию: <code>0</code>	Определяет, сколько команд <code>ex</code> , строк поиска и выражений будет храниться в истории команд. Рекомендуем установить большое число. Пример использования истории командной строки вы найдете в разделе «Увеличиваем скорость» главы 14
<code>hlsearch (hls)</code>	<code>nohlsearch</code>	Подсвечивает все вхождения последнего шаблона поиска
<code>icon</code>	<code>noicon</code>	Vim пытается изменить имя значка, связанного с окном, в котором он выполняется. Отменяется с помощью параметра <code>iconstring</code>
<code>iconstring</code>		Значение строки, используемое в качестве имени значка для окна
<code>ignorecase (ic)</code>	<code>noignorecase</code>	Позволяет не учитывать регистр при поиске. См. также <code>smartcase</code>
<code>include (inc)</code>	<code>^#\s*include</code>	Определяет шаблон для поиска команд <code>include</code> . Значение по умолчанию предназначено для кода на языке C
<code>incsearch (is)</code>	<code>noincsearch</code>	Включает пошаговый поиск

Таблица Б.2 (продолжение)

Параметр	По умолчанию	Описание
isfname (isf)	@,48-57,/,.,-,_,+,,,,\$,:;~	Список символов, которые можно включить в имена файлов и путей. В системах, отличных от Unix, значения по умолчанию другие. Знак @ обозначает любой алфавитный символ. Он также используется в других параметрах isXXX, описанных далее
isident (isi)	@,48-57,_,192-255	Список символов, которые могут быть включены в идентификаторы. В системах, отличных от Unix, значения по умолчанию другие
iskeyword (isk)	@,48-57,_,192-255	Список символов, которые могут быть включены в ключевые слова. В системах, отличных от Unix, значения по умолчанию другие. Ключевые слова используются при поиске и распознавании со многими командами, такими как w, [i и т. д.
isprint (isp)	@,161-255	Список символов, которые можно отобразить
laststatus (ls)	2	Отвечает за то, как и когда отображать строку состояния. Значения: 0 — никогда, 1 — только при наличии как минимум двух окон и 2 — всегда
linebreak (lbr)	nolinebreak	Позволяет разбить длинную строку на символах, определенных в breakat. Строка не разбивается на две части при достижении края окна, а продолжается на следующей строке
lines	24 и высота терминала	Определяет количество строк, которые будут отображаться на экране. Может быть полезным, если вы используете GUI и предпочитаете определять число строк при запуске Vim. См. также columns
listchars (lcs)	eol:\$	Позволяет настроить, какие символы будут использоваться для отображения непечатаемых символов, таких как пробелы, табуляции, концы строк и т. д. Работает в сочетании с параметром list, который включает или выключает режим отображения непечатаемых символов. (Еще более детальным является определение начальных и конечных пробелов с помощью lead:.andtrail:.definitions::set listchars+=lead:.,trail:.)
makeef (mef)	/tmp/vim##.err	Имя файла регистрации ошибок для команды :make. В системах, отличных от Unix, значения по умолчанию другие. Символы ## заменяются на число, чтобы сделать имя уникальным

Параметр	По умолчанию	Описание
<code>makeprg (mp)</code>	<code>make</code>	Позволяет задать команду, которая будет выполняться при вызове команды <code>:make</code> . Символы <code>%</code> и <code>#</code> в значении расширяются
<code>matchpairs (mps)</code>	<code>(:),{:},[:]</code>	Определения пар совпадающих символов, разделенных запятыми и двоеточиями. Это должны быть два разных символа. Полезным будет добавить <code><:></code> для HTML-сопоставления. <code>:set matchpairs+="<:>"</code>
<code>modifiable (ma)</code>	<code>modifiable</code>	Если выключен, не позволяет вносить изменения в буфер
<code>mouse</code>	<code>a</code> для GUI, MS-DOS и Win32	Позволяет включить мышь в версиях Vim без графического интерфейса. Работает в MS-DOS, Win32, QNX <code>pterm</code> и <code>xterm</code> . Подробнее описано в онлайн-справке
<code>mousehide (mh)</code>	<code>nomousehide</code>	Скрывает указатель мыши во время печати. Восстанавливает указатель при движении мыши
<code>numberwidth (nuw)</code>	<code>vi</code> по умолчанию: 8; <code>Vim</code> по умолчанию: 4	Позволяет задать ширину столбца, отображающего номера строк (установить с помощью <code>number</code> или <code>relativenumber</code>). Vim всегда использует последнюю позицию для разделяющего <code>space</code> . Мы рекомендуем установить хотя бы значение 6
<code>paste</code>	<code>nopaste</code>	Позволяет изменить огромное количество параметров, чтобы вставка в окно Vim с помощью мыши не искажала вставляемый текст. Выключение восстанавливает прежние значения этих параметров. Подробнее описано в онлайн-справке
<code>relativenumber (rnu)</code>	<code>norelativenumber</code>	Пронумеровывает строки в левой части окна относительно текущей строки. Например, текущая строка отображает верный номер строки, а все строки над и под ней показывают смещение от текущей строки. Это может быть полезным для блочных команд, избавляя от необходимости подсчета строк
<code>ruler (ru)</code>	<code>noruler</code>	Отображает номер строки и столбца позиции курсора
<code>scrollbind (scb)</code>	<code>noscrollbind</code>	Синхронизирует прокрутку нескольких окон, у которых включена эта опция. Полезно для сравнений <code>diff</code>

Таблица Б.2 (продолжение)

Параметр	По умолчанию	Описание
<code>scrolloff</code> (so)	0 (5 в <code>defaults.vim</code>)	Позволяет установить минимальное количество строк, которые должны быть видимыми выше и ниже текущей позиции курсора при прокрутке текста. Значение по умолчанию 0 означает, что Vim будет прокручивать текст, когда курсор достигнет верхней или нижней границы окна. Делает прокрутку более плавной. Нам нравится устанавливать значение 3 для <code>scrolloff</code>
<code>scrollopt</code> (sbo)	<code>ver,jump</code>	Определяет поведение <code>scrollbind</code> . Значение <code>ver</code> связывает вертикальную прокрутку между окнами <code>scrollbind</code> . Более подробную информацию см. в справке Vim
<code>secure</code>	<code>nosecure</code>	Отключает определенные виды команд в файле загрузки. Автоматически включается, если у вас нет собственных файлов <code>.vimrc</code> и <code>.exrc</code>
<code>shellpipe</code> (sp)		Определяет строку оболочки, используемую для перенаправления выходных данных команды в файл (например, из <code>:make</code>). Значение по умолчанию зависит от оболочки
<code>shellredir</code> (srr)		Определяет строку оболочки, используемую для перенаправления выходных данных фильтра во временный файл. Значение по умолчанию зависит от оболочки
<code>showbreak</code> (sbr)	Пустая строка	Определяет строку, которая будет показана в начале каждой строки, которая продолжает предыдущую перенесенную строку (полезна для визуального отличия перенесенных строк от новых абзацев)
<code>showcmd</code> (sc)	Vim <code>showcmd</code> , Unix <code>noshowcmd</code> , определенные также в <code>defaults.vim</code>	Позволяет показывать команды командного режима <code>vi</code> по мере их ввода. Vim отображает команду в правой части строки командного режима <code>ex</code> . Например, команда <code>vi 5cw</code> для изменения пяти слов постепенно отображается по мере ее ввода. Полезно для отслеживания команды в процессе ее построения
<code>showmode</code> (smd)	Vim по умолчанию: <code>smd</code> ; <code>vi</code> по умолчанию: <code>nosmd</code>	Помещает сообщение в строку состояния для режимов ввода и замены, а также для визуального режима
<code>sidescroll</code> (ss)	0	Определяет количество столбцов для горизонтальной прокрутки. Значение по умолчанию помещает курсор в середину экрана

Параметр	По умолчанию	Описание
smartcase (scs)	nosmartcase	Отменяет параметр ignorecase, если шаблон поиска содержит символы в верхнем регистре
spell	nospell	Включает проверку орфографии
spelllang (spl)	en	Определяет список языков, которые будут использоваться для проверки орфографии
suffixes	*.bak,~, .o, .h, .info, .swp	Если несколько файлов сопоставимы с шаблоном во время завершения имени файла, значение этой переменной устанавливает среди них приоритет, чтобы выбрать одну версию, которую будет использовать Vim
taglength (tl)	0	Определяет количество символов, значимых для тегов. Значение по умолчанию (ноль) означает, что все символы значимы
tagrelative (tr)	Vim по умолчанию: tr; vi по умолчанию: notr	Определяет, как будут интерпретироваться имена файлов в файле теги. Если опция включена, то имена файлов выбираются относительно каталога, где находится файл тегов
tags (tag)	./tags, tags	Определяет список файлов тегов, разделенных пробелами или запятыми. Начальное ./ замещается на полный путь к текущему файлу
tildeop (top)	notildeop	Интерпретирует команду ~ как оператор
undolevels (ul)	1000	Позволяет задать максимальное количество изменений, которые можно отменить. Значение 0 устанавливает совместимость с vi, где команда u отменяет саму себя на первом уровне отмены. В системах, отличных от Unix, значения по умолчанию могут быть другие
viminfo (vi)		Считывает файл viminfo при запуске и записывает его при выходе. Значение сложное: оно управляет различными видами информации, которые Vim хранит в файле. Подробнее описано в онлайн-справке
writebackup (wb)	writebackup	Позволяет определить, будет ли создаваться резервная копия файла перед его перезаписью. Резервная копия удаляется после успешной записи файла, если параметр backup включен

* Начиная с версии 8.2, Vim выключает `compatible`, если в системе существует файл времени выполнения `Vim defaults.vim` или общесистемный файл `defaults.vim`. Такое поведение по умолчанию намного удобнее и позволяет избежать путаницы у новичков, которые только начинают использовать Vim и не понимают, почему программа ведет себя по-разному.

ПРИЛОЖЕНИЕ В

Более светлая сторона vi

Конечно, vi дружелюбен к пользователям. Просто он избирателен в выборе друзей.

Аноним

Данное приложение касается многих тем, связанных с vi. Оно включает в себя:

- получение доступа к файлам, упомянутым в книге;
- онлайн-руководство по vi из части I;
- логотип vi Powered для вашего веб-сайта (и другие логотипы);
- интересные штуки, связанные с vi;
- педаль Vim;
- некоторые необычные и удивительные вещи, которые люди делали с помощью vi на протяжении многих лет;
- «Домашняя Страница Любителей Vi» (<https://thomer.com/vi/vi.html>);
- другой клон vi;
- краткое упоминание различий между vi и Emacs;
- некоторые интересные цитаты, связанные с vi.

Получение доступа к файлам

Многие мелочи, которые мы здесь представим, когда-то были в свободном доступе в Интернете. Увы, сейчас это не так. Однако мы создали репозиторий на GitHub с различными файлами. Просто скопируйте <https://www.github.com/learning-vi/vi-files>, чтобы создать собственную копию репозитория:

```
git clone https://www.github.com/learning-vi/vi-files
```

Примеры файлов

Некоторые файлы, использованные в качестве примеров в части 1, находятся в каталоге `book_examples`.

Источник clewn

Программу `clewn`, упомянутую в подразделе «Драйвер Clewn GDB» в главе 16, можно найти в каталоге `clewn-1.15`. Чтобы создать и установить ее, воспользуйтесь следующим набором команд:

```
cd clewn-1.15
./configure
make
sudo make install
```

Онлайн-руководство vi

Прежде всего, это онлайн-руководство Уолтера Зинца из журнала *UnixWorld*, которое неоднократно упоминалось в части I.

Это руководство давно исчезло со своего изначального сайта, но мы смогли найти копию по адресу <https://www.ele.uri.edu/faculty/vetter/Other-stuff/vi/009-index.html>. Чтобы вы не зависели от работы сайта после публикации книги, мы поместили копию в репозиторий GitHub (<https://www.github.com/learning-vi/vi-files>)¹.

В нашем репозитории GitHub руководство находится в каталоге `unix-world-tutorial`. Если вы используете Firefox (к примеру) в качестве своего браузера, достаточно выполнить следующее:

```
$ cd unix-world-tutorial
$ firefox ./009-index.html &
```

Используйте выбранный вами браузер

Руководство охватывает следующие темы.

- Основы редактора.
- Адреса линейного режима.
- Команда `g` (глобальная).
- Команда замены.

¹ Нижние колонтитулы этой веб-страницы указывают на источник, где мы их нашли. При просмотре нашей копии вам не обязательно посещать сайт.

- Окружение редактирования (команда `set`, теги, `EXINIT` и `.exrc`).
- Адреса и столбцы.
- Команды замещения `r` и `R`.
- Автоматические отступы.
- Макросы.

Руководство включает контрольные вопросы в конце некоторых разделов, которые вы можете использовать, чтобы проверить свои знания. Вы также можете сразу перейти к вопросам, чтобы выяснить, насколько хорошо вы усвоили материал этой книги!

vi Powered!

Далее рассмотрим логотип *vi Powered* (рис. В.1). Это маленький GIF-файл, который можно добавить на свою личную веб-страницу, чтобы показать, что она создана с помощью `vi`.

Логотип находится в каталоге `vi-powered` репозитория GitHub (<https://www.github.com/learning-vi/vi-files>).

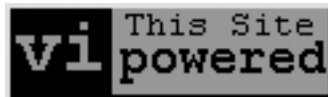


Рис. В.1. vi Powered!

Исходная веб-страница логотипа, созданная Антонио Валле на испанском языке (<http://www.abast.es/~avelle/vi.html>), уже давно не существует. Сейчас доступна англоязычная домашняя страница (<https://darryl.com/vi.shtml>), и на ней есть инструкции по добавлению логотипа (<https://darryl.com/addlogo.html>), которые состоят из простых шагов.

1. Загрузите логотип. Возьмите его из нашего репозитория GitHub (<https://www.github.com/learning-vi/vi-files>) или введите <https://darryl.com/vipower.gif> в вашем (графическом) веб-браузере, а затем сохраните логотип в файл или используйте утилиту веб-поиска командной строки, такую как `wget`.
2. Добавьте следующий код на вашу веб-страницу в подходящее место:

```
<A HREF="https://darryl.com/vi.shtml">
<IMG SRC="vipower.gif">
</A>
```

Это поместит логотип на вашу страницу и превратит его в гиперссылку, открывающую домашнюю страницу *vi Powered*. Вы можете добавить атрибут `ALT="This`

Web Page is vi Powered" в тег , чтобы пользователи неграфических браузеров могли увидеть текст.

3. Добавьте следующий код в тег <HEAD> вашей веб-страницы:

```
<META name="editor" content="/usr/bin/vi">
```

Настоящие Веб-мастера предпочитают использовать vi вместо модных инструментов HTML, как и Настоящие Программисты предпочитают troff вместо текстовых процессоров WYSIWYG. Используйте логотип *vi Powered*, чтобы с гордостью это продемонстрировать. ☺

Вы найдете несколько вариантов логотипа Vim по адресу <https://www.vim.org/logos.php>. Несколько логотипов *Vim Powered* для веб-сайтов находятся здесь: <https://www.vim.org/buttons.php>.

vi для любителей Java

Несмотря на заголовок, этот раздел описывает не язык программирования Java, а кофе Java, который люди пьют.

Представим, что наш гипотетический Настоящий Программист, использующий vi для написания кода на языке C++, своей документации troff и веб-страниц, время от времени хочет выпить чашечку кофе. Теперь он может пить свой кофе из кружки, на которой напечатана команда vi!

Итак, вот третий элемент: кружки со ссылками vi, футболки, толстовки, фартуки для барбекю, детские слюнявчики и даже коврики для мыши — все это доступно по адресу <https://www.cafepress.com/geekcheat/366808>.

Педаль Vim

Если вас утомляет ручная (буквально) смена режимов в vi и Vim, возможно, вы захотите делать это по-другому. Воспользуйтесь подключаемой через USB педалью, при нажатии которой срабатывает команда I, а при отпускании — Esc.

Описание проекта с деталями, ссылками, инструкциями и фотографиями вы найдете по адресу <https://github.com/alevchuk/vim-clutch>.

Еще одна педаль Vim представлена на <https://l-o-o-s-e-d.net/vim-clutch>. Попутно автор описывает несколько других проектов, посвященных педали Vim.

Поразите своих друзей!

Набор полезных элементов, относящихся к `vi`, был когда-то доступен в архивах FTP на `alf.uib.no`. Исходные архивы располагались по адресу `ftp://afl.uib.no/pub/vi`. Набор был воспроизведен в `ftp://ftp.uu.net/pub/text-processing/vi`. Оба сайта больше недоступны.

К счастью, Клемент Коль предоставил свою копию архива, чтобы мы добавили ее в наш репозиторий GitHub (<https://www.github.com/learning-vi/vi-files>), и мы благодарны ему за это¹.

К сожалению, последнее обновление этих файлов было в мае 1995 года. Тем не менее базовый функционал `vi` не изменился, и большая часть информации и макросов в архиве все еще полезны. В исходном архиве было четыре подкаталога:

- `docs` — документация по `vi`, а также некоторые публикации `comp.editors`;
- `macros` — макросы `vi`;
- `comp.editors` — разнообразные материалы, размещенные в `comp.editors`;
- `programs` — исходный код для клонов `vi` для различных платформ (и других программ).

Мы не включили каталог `programs`, поскольку он на сегодняшний день по большей части устарел.

Наибольший интерес представляют каталоги `docs` и `macros`. В первом вы найдете огромное количество статей и ссылок, включая руководство для новичков, объяснения ошибок, быстрые ссылки и множество заметок типа «как сделать это» (например, как сделать заглавной лишь первую букву предложения в `vi`). Там есть даже песня о `vi`!

Каталог `macros` содержит более 50 файлов, которые выполняют различные функции. Мы упомянем лишь три из них. Файлы с исходными архивированными именами, оканчивающимися на `.tar.Z`, были распакованы в отдельных каталогах в репозитории GitHub (<https://www.github.com/learning-vi/vi-files>).

- `evi-tar` — «эмулятор» Emacs. Идея заключается в том, чтобы превратить `vi` в немодальный редактор (тот, что все время находится в режиме ввода с командами, выполняемыми с помощью управляющих клавиш). На самом деле это делается с помощью сценария оболочки, который замещает переменную окружения `EXINIT`.

¹ Также благодаря Бакулу Ша, который указал нам на доступную онлайн-копию по адресу <https://web.archive.org/web/19970209203017/http://archive.uwp.edu/pub/vi/>.

- *hanoi* — самый известный необычный макрос vi, решающий проблему программирования Ханойских башен. Эта программа просто отображает перемещения, а не извлекает диски. Ради забавы мы перепечатали ее в «Ханойские башни, версия vi».
- *turing-tar* — это программа, использующая vi для реализации настоящей машины Тьюринга! Довольно интересно понаблюдать, как она выполняет программы.

Кроме этих макросов, существует множество других. Ознакомьтесь с ними!

ХАНОЙСКИЕ БАШНИ, ВЕРСИЯ VI

```
" From: gregm@otc.otca.oz.au (Greg McFarlane)
" Newsgroups: comp.sources.d,alt.sources,comp.editors
" Subject: VI SOLVES HANOI
" Date: 19 Feb 91 01:32:14 GMT
"
" Submitted-by: gregm@otc.otca.oz.au
" Archive-name: hanoi.vi.macros/part01
"
" Everyone seems to be writing stupid Tower of Hanoi programs.
" Well, here is the stupidest of them all: the hanoi solving
" vi macros.
"
" Save this article, unshar it, and run uudecode on
" hanoi.vi.macros.uu. This will give you the macro file
" hanoi.vi.macros.
" Then run vi (with no file: just type "vi") and type:
"      :so hanoi.vi.macros
"      g
" and watch it go.
"
" The default height of the tower is 7 but can be easily changed
" by editing the macro file.
"
" The disks aren't actually shown in this version, only numbers
" representing each disk, but I believe it is possible to write
" some macros to show the disks moving about as well. Any takers?
"
" (For maze solving macros, see alt.sources or comp.editors)
"
" Greg
"
" ----- REAL FILE STARTS HERE -----
set remap
set noterse
```

```

set wrapscan
" to set the height of the tower, change the digit in the following
" two lines to the height you want (select from 1 to 9)
map t 7
map! t 7
map L 1G/t^MX/^0^M$P1GJ$An$BGC0e$X0E0F$X/T^M@f^M@h^M$A1GJ@f01$Xn$PU
map g IL
map I KMYNOQNOSkRTV
map J /^0[^t]*$^M
map X x
map P p
map U L
map A "fyl
map B "hyl
map C "fp
map e "fy2l
map E "hp
map F "hy2l
map K 1Go^[
map M dG
map N yy
map O p
map q tllD
map Y o0123456789Z^[0q
map Q 0iT^[
map R $rn
map S $r$
map T ko0^M0^M^M^[
map V Go/^[

```

Домашняя страница любителей Vi

«Домашняя страница любителей Vi» (<https://thomer.com/vi/vi.html>) содержит следующие элементы:

- таблицу неизвестных клонов vi со ссылками на исходный код или двоичные дистрибутивы;
- ссылки на другие сайты vi;
- обширную коллекцию ссылок на документацию, учебные пособия, справочники и руководства vi на различных уровнях;
- макросы vi для написания HTML-документов и решения задачи Ханойских башен, а также FTP-сайты для других наборов макросов;

- разнообразные ссылки vi: стихи, «настоящая история» vi, обсуждения различий vi и Emacs и кофейные кружки vi (см. раздел «vi для любителей Java»).

Имейте в виду, что этот сайт, вероятно, очень давно не обновлялся. Какие-то ссылки работают, а какие-то — нет.

Другой клон vi

На рис. В.2–В.9 изображена история vigor, *другого* клона vi.

Исходный код для vigor доступен по адресу <http://vigor.sourceforge.net>.



Рис. В.2. История vigor, часть 1

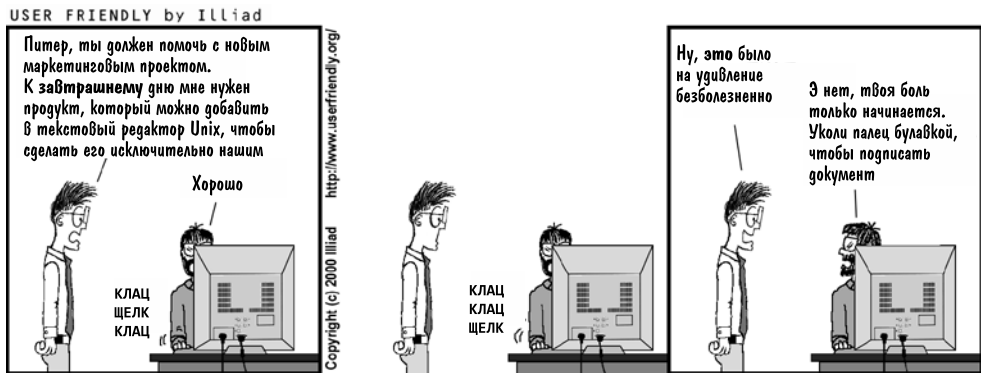


Рис. В.3. История vigor, часть 2

USER FRIENDLY by Illiad



Что ты имеешь в виду?

Предложил ему сделать помощника Скрепыша для vi. Он, конечно же, думает, что это великолепная идея

ЩЕЛК
ЩЕЛК
КЛАЦ



Рис. В.4. История vigor, часть 3

USER FRIENDLY by Illiad



Нет, это точно не отличная новость! Похоже, ты ничего не понимаешь. Твои планы породили хаос в мире Unix



Рис. В.5. История vigor, часть 4

USER FRIENDLY by Illiad



Наверное, я пойду и напишу письмо в Microsoft Word...



Рис. В.6. История vigor, часть 5

USER FRIENDLY by Illiad



Рис. В.7. История vigor, часть 6

USER FRIENDLY by Illiad



Рис. В.8. История vigor, часть 7

USER FRIENDLY by Illiad



Рис. В.9. История vigor, часть 8

Наслаждение чистым вкусом

```
vi это [[13~^[[15~^[[15~^[[19~^[[18~^
[^[[29~^[[34~^[[26~^[[32~^ лучший редактор
по сравнению с этим emacs. Я знаю, меня ^[[14~
сожгут за это, но правду надо сказать.
^[[D^[[D^[[D^[[D ^[[D^[[D^[[D^[[B^
выход ^X^C выход :x :wq черт :w:w :w :x ^C^C^Z^D
```

Йеснер Лауриссен из alt.religion.emacs

Мы не можем обсуждать `vi` как часть культуры Unix, не ознакомившись с одним из самых долгих продолжающихся споров в сообществе Unix: `vi` против Emacs¹.

Дискуссии о том, что же лучше, внезапно проникли в `comp.editors` (и другие новостные группы) на многие годы (это прекрасно проиллюстрировано на рис. В.10).



Рис. В.10. Это не религиозная война. В самом деле!

Ниже представлены некоторые аргументы в пользу `vi`.

- `vi` доступен на всех Unix-системах. Если вы устанавливаете системы или меняете одну на другую, вам все равно может понадобиться использовать `vi`.
- В большинстве случаев вы можете держать пальцы на нижнем ряду клавиатуры. Это большой плюс для операторов, владеющих методом слепого ввода.
- Команды представляют собой один (или иногда два) регулярный символ, что делает их ввод более удобным, чем множество управляющих и метасимволов, необходимых в Emacs.

¹ Вообще, это на самом деле целая война между сторонниками различных философий, но давайте постараемся быть уважительными. Другая подобная схватка: BSD против System V, — была улажена с помощью POSIX. System V одержала победу, но BSD получила существенное признание. ☺

- vi, как правило, менее ресурсоемкий, чем Emacs. Время запуска существенно быстрее, иногда даже в десять раз.
- С появлением дополнительных функций в Vim (и других клонов vi), таких как пошаговый поиск, несколько окон и буферов, GUI, подсветка синтаксиса и умные отступы, а также возможности программирования через языки расширения, технологический разрыв между двумя редакторами существенно уменьшился, если полностью не исчез.

Для полноты картины следует упомянуть еще один элемент: в GNU Emacs всегда были пакеты vi-эмуляции, но они часто были низкого качества. Тем не менее *viper-mode* отлично эмулирует vi и может быть полезен при изучении Emacs (для тех, кто в этом заинтересован).

В конце концов, всегда помните, что вы являетесь окончательным судьей при выборе ПО. Используйте те инструменты, которые дают вам максимальную эффективность. Для многих задач vi и Vim — великолепные инструменты.

Цитаты о vi

И наконец, с разрешения Брэма Моленара, автора Vim, представляем вам несколько цитат о vi.

«ТЕОРЕМА: vi совершенен.

ДОКАЗАТЕЛЬСТВО: в римской системе цифр VI — это 6. Есть три натуральных числа меньше 6, которые делят 6, — это 1, 2 и 3. Учитывая, что $1 + 2 + 3 = 6$, получаем, что 6 — совершенное число. Следовательно, vi совершенен».

Артур Татейши

Реакция Нейтана Т. Олджера:

«Итак, к чему вышесказанное приводит Vim? В римской системе цифр VIM может быть: $(1000 - (5 + 1)) = 994$, что равно $2 \times 496 + 2$; 496 делится на 1, 2, 4, 8, 16, 31, 62, 124 и 248 и $1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248 = 496$. 496 является совершенным числом. Следовательно, Vim вдвойне совершеннее vi, к тому же обладает дополнительными полезными функциями. ☺

Таким образом, Vim *больше* чем совершенен».

Эта цитата подводит итог для истинных любителей vi.

«Для меня vi — это дзен. Использовать vi означает практиковать дзен. Каждая команда — это коан. Глубокая для пользователя, непостижимая для непосвященного. Вы открываете истину каждый раз, когда его используете».

Сатиш Редди

vi и Vim: исходный код и разработка

На всякий случай, если в вашей системе еще не установлены `vi` и `Vim`, в этом приложении описано, где найти исходный код для обоих редакторов и исполняемые установочные файлы для наиболее популярных операционных систем.

Ничего схожего с оригиналом

На протяжении долгих лет исходный код `vi` был недоступен без лицензии на исходный код Unix. Хотя образовательные учреждения могли получить лицензии по достаточно низкой цене, коммерческие лицензии всегда были дорогими. Этот факт подтолкнул к созданию `Vim` и других клонов `vi`.

В январе 2002 года исходный код для `V7` и `32V UNIX` стал доступен по лицензии с открытым исходным кодом¹. Это открыло доступ практически ко всему коду, разработанному для `BSD Unix`, включая `ex` и `vi`.

Изначальный код не компилируется исключительно в современных системах, таких как `GNU/Linux`, а перенос его затруднителен². К счастью, работа уже сделана. Если вам хочется использовать оригинальный, настоящий `vi`, вы можете загрузить исходный код и собрать его самостоятельно. Дополнительную информацию можно найти по адресу <https://github.com/n-t-roff/heirloom-ex-vi>.

Мы смогли без проблем собрать `Heirloom vi` в системе `Ubuntu GNU/Linux`, просто следуя инструкциям в файле `README`.

¹ Больше информации об этом вы найдете на сайте `Unix Heritage Society` (<https://www.tuhs.org>).

² Мы знаем. Мы пробовали.

Где взять Vim

Большинство современных операционных Unix-систем используют Vim в качестве стандартной версии vi¹. То есть, когда вы запускаете vi, вы получаете Vim.

Однако многие такие системы могут быть немного устаревшими и не иметь самую последнюю версию Vim. Например, на момент публикации восьмого издания (конец 2021-го) текущая версия — Vim 8.2, в то время как в большинстве систем установлена Vim 8.0.

В этом разделе мы расскажем, как установить актуальную версию Vim (или любую версию, которая вам нравится) в GNU/Linux (в данном случае Ubuntu). Для других дистрибутивов GNU/Linux процесс в основном аналогичен.

Если команда vi или vim не запускает ваш редактор, это может означать, что он либо не установлен, либо ваш путь не включает исполняемый каталог Vim. Убедитесь, что переменная окружения PATH содержит следующие каталоги (если это не помогает, возможно, Vim не установлен, в таком случае прочтите инструкции по установке Vim):

/usr/bin	<i>Это в любом случае должно быть в вашем \$PATH</i>
/bin	<i>Как и это</i>
/opt/local/bin	
/usr/local/bin	

Проверьте вашу версию Vim с помощью команды ex version. Vim отобразит нечто подобное:

```
VIM - Vi IMproved 8.2 (2019 Dec 12, compiled May  8 2021 05:44:12)
macOS version
Included patches: 1-2029
Compiled by root@apple.com
Normal version without GUI.  Features included (+) or not (-):
+acl               -farsi               +mouse_sgr         +tag_binary
-arabic            +file_in_path        -mouse_sysmouse    -tag_old_static
+autocmd           +find_in_path        -mouse_urxvt       -tag_any_white
+autochdir         +float               +mouse_xterm       -tcl
-autoservername    +folding             +multi_byte        -termguicolors
-balloon_eval      -footer              +multi_lang        +terminal
-balloon_eval_term +fork()              -mzscheme          +terminfo
-browse            -gettext             +netbeans_intg     +termresponse
+builtin_terms     -hangul_input        +num64             +textobjects
+byte_offset       +iconv               +packages          +textprop
```

¹ Исключения составляют унаследованные системы на базе Unix, такие как HP/UX и AIX, в которых стандартный vi — это оригинальный vi.

```

+channel          +insert_expand    +path_extra       +timers
+cindent          -ipv6             -perl             +title
-clientserver     +job              +persistent_undo  -toolbar
+clipboard        +jumplist         +popupwin         +user_commands
+cmdline_compl    -keymap           +postscript       -vartabs
+cmdline_hist     +lambda           +printer          +vertsplitt
+cmdline_info     -langmap          -profile          +virtualedit
+comments         +libcall          +python/dyn       +visual
-conceal          +linebreak        -python3          +visualextra
+cryptv           +lispindent       +quickfix         +viminfo
+cscope           +listcmds         +reltime          +vreplace
+cursorbind       +localmap         -rightleft       +wildignore
+cursorshape      -lua              +ruby/dyn         +wildmenu
+dialog_con       +menu             +scrollbind       +windows
+diff             +mksession        +signs            +writebackup
+digraphs         +modify_fname     +smartindent      -X11
-dnd              +mouse            -sound            -xfontset
-ebcdic           -mousethshape     +spell            -xim
-emacs_tags       -mouse_dec        +startuptime      -xpm
+eval             -mouse_gpm        +statusline       -xsmp
+ex_extra         -mouse_jsbterm    -sun_workshop     -xterm_clipboard
+extra_search     -mouse_netterm    +syntax           -xterm_save

    system vimrc file: "$VIM/vimrc"
    user vimrc file: "$HOME/.vimrc"
  2nd user vimrc file: "~/.vim/vimrc"
    user exrc file: "$HOME/.exrc"
    defaults file: "$VIMRUNTIME/defaults.vim"
  fall-back for $VIM: "/usr/share/vim"
Compilation: gcc -c -I. -Iproto -DHAVE_CONFIG_H -DMACOS_X_UNIX -g -O2
-U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=1
Linking: gcc -L/usr/local/lib -o vim -lm -lncurses -liconv -framework Cocoa

```

Если вы не нашли то, что вам нужно, читайте дальше.



Забавно, что на Mac mini одного из наших авторов с установленной версией OS X 10.4.10 не только команда `vi` вызывает Vim, но и документация («страница руководства») ссылается на Vim!

Если ни один из предыдущих методов не помог, у вас, вероятно, не установлен Vim. Vim доступен в различных формах для разных платформ, и его (обычно) легко найти и установить. Следующие подразделы помогут вам сделать это для вашей платформы:

- Unix и его разновидности, включая GNU/Linux и Cygwin;
- Windows XP;
- Macintosh macOS.



Описанный здесь процесс установки требует среды разработки, способной компилировать исходный код. Хотя большинство разновидностей Unix предоставляют компиляторы и соответствующие инструменты, некоторые (в особенности последние релизы дистрибутивов Ubuntu GNU/Linux) требуют загрузки и установки дополнительных пакетов перед тем, как вы сможете скомпилировать код.

Существуют также готовые пакеты Vim, предлагающие простую установку для GNU/Linux (пакеты Red Hat RPMs, Debian), Solaris (сопутствующий программный продукт) и HP-UX. На домашней странице Vim есть ссылки для всех этих систем. В случае нестандартных систем, несомненно, поможет поиск в Интернете.

Чтобы убедиться, что вы готовы к компиляции Vim, выполните быструю проверку gcc:

```
$ type gcc
gcc is /usr/bin/gcc
```

Установка Vim для Unix и GNU/Linux

Многие современные окружения Unix уже поставляются с предустановленной версией Vim. Большинство дистрибутивов GNU/Linux просто связывают стандартное местоположение `vi /usr/bin/vi` с исполняемым файлом Vim. Так что большинству пользователей даже не понадобится устанавливать его. Как уже было упомянуто ранее, Solaris 11 `vi` — это фактически Vim!

В системах Ubuntu GNU/Linux минимальная версия Vim устанавливается как `vi`. Чтобы установить полную версию, включая GNU, введите:

```
sudo apt install vim-gtk3
```

В других системах вам понадобится сделать нечто подобное с помощью пакетного менеджера вашей системы.

Поскольку существует множество разновидностей Unix и различных типов (например, Solaris, HP-UX, *BSD, все дистрибутивы GNU/Linux), то, если вы не можете установить Vim с помощью пакетного менеджера, можно загрузить исходный код Vim, скомпилировать его и установить.

Vim распространяется в виде сжатого tar-файла (используя файлы `gzip` или `bzip2` — `.gz` и `.bz2` соответственно). В каждой крупной версии Vim есть несколько заплаток для исправления проблем и недочетов, обнаруженных после релиза.

Можно загрузить tar и файлы-заплатки, а затем применить их по отдельности, чтобы выполнить сборку из последнего исходного кода. Однако процесс этот утомителен, и всегда существуют сотни файлов-заплаток для любой заданной версии.

Вместо этого гораздо удобнее просто *клонировать* исходный код из репозитория Vim Git (<https://github.com/vim/vim>) на GitHub. Это действие приведет примерно к такому выводу:

```
$ git clone git://github.com/vim/vim
Cloning into 'vim'...
remote: Enumerating objects: 34, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (27/27), done.
Receiving objects: 100% (113446/113446), 90.87 MiB | 1.07 MiB/s, done.
Resolving deltas: 100% (95729/95729), done.
Updating files: 100% (3347/3347), done.
```

Чтобы выполнить сборку, перейдите в каталог `src` и запустите `configure`. Поищите в Интернете параметры `configure` перед его запуском. Вывод объемный:

```
$ cd vim/src
$ ./configure
configure: creating cache auto/config.cache
checking whether make sets $(MAKE)... yes
checking for gcc... gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
...
```

Следующим шагом будет запуск `make`. Здесь вывод тоже объемный:

```
$ make
/bin/sh install-sh -c -d objects
touch objects/.dirstamp
CC="gcc -Iproto -DHAVE_CONFIG_H " srcdir=. sh ./osdef.sh
gcc -c -I. -Iproto -DHAVE_CONFIG_H -g -O2 -U_FORTIFY_SOURCE
-D_FORTIFY_SOURCE=1 -o objects/arabic.o arabic.c
gcc -c -I. -Iproto -DHAVE_CONFIG_H -g -O2 -U_FORTIFY_SOURCE
-D_FORTIFY_SOURCE=1 -o objects/arglist.o arglist.c
gcc -c -I. -Iproto -DHAVE_CONFIG_H -g -O2 -U_FORTIFY_SOURCE
-D_FORTIFY_SOURCE=1 -o objects/autocmd.o autocmd.c
...
```

Когда все будет готово, у вас появится исполняемое имя `vim`. Чтобы установить его, получите `root`-права и запустите `make install`. Вот и все!

Установка Vim для окружений Windows

MS Windows gvim

Существует три основных варианта для Microsoft Windows. Первый — использовать самоустанавливающийся исполняемый файл `gvim82.exe`, который можно загрузить с домашней страницы Vim. Просто запустите этот файл, и он сделает

все остальное. Мы устанавливали Vim с помощью исполняемого файла на других компьютерах с Windows (начиная с Windows XP и выше), и он всегда работал безупречно.



В определенный момент процесса установки появляется окно DOS с предупреждением о том, что что-то не поддается проверке. Мы никогда не сталкивались с тем, чтобы это становилось проблемой.

Сygwin для Windows

Второй вариант для пользователей Windows — установка Cygwin (<http://www.cygwin.com>) — набора общих инструментов GNU, переносимого на платформу Windows. Это полноценная реализация практически всего основного ПО, используемого на платформах Unix. Vim является частью стандартной установки Cygwin и запускается в консоли оболочки Cygwin.

ИСПОЛЬЗОВАНИЕ VIM С CYGWIN

Текстовая консоль Vim прекрасно работает в Cygwin, но для запуска `gvim` в Cygwin необходим сервер XWindow System. Если сервер не запущен, то `gvim` Cygwin автоматически переключается на текстовый режим.

Чтобы `gvim` Cygwin заработал (допустим, вы хотите запустить его на локальном экране), запустите сервер X Cygwin из командной строки в оболочке Cygwin:

```
$ X -multiwindow &
```

Параметр `-multiwindow` позволяет Windows управлять приложениями Cygwin через сервер X. Если сервер X Cygwin не установлен, поищите дополнительную информацию на домашней странице Cygwin. Значок X на Панели задач Windows свидетельствует о том, что сервер X запущен.

Если у вас одновременно установлены Cygwin Vim и `vim.org` Vim, есть риск вызвать путаницу в расположении конфигурационных файлов Vim, что может привести к запуску идентичных версий Vim с разными параметрами. К примеру, в Cygwin и Windows много разных понятий того, что является домашним каталогом.

Подсистема Windows для Linux и Vim

Windows Subsystem for Linux (WSL) — это виртуальная среда, которая полностью совместима с ядром Linux. Она разработана компанией Microsoft для установки и запуска дистрибутивов GNU/Linux. Список поддерживаемых дистрибутивов постоянно растет, а большинство популярных дистрибутивов GNU/Linux доступны в WSL.

Подробнее о WSL смотрите в разделе «Запуск `gvim` в Microsoft Windows WSL» главы 9.



WSL — относительно новое дополнение к Microsoft Windows. Хотя мы описали WSL довольно кратко, мы считаем, что это лучший выбор, и рекомендуем GNU/Linux в WSL и Vim вместо ранее упомянутого Cygwin.

Установка Vim для окружения Macintosh

Существует два варианта для использования Vim в системе macOS. Вы можете использовать нативную версию или установить графическую версию с помощью Homebrew. Далее мы расскажем об обоих вариантах.

Нативный macOS Vim

Vim в macOS является предустановленным стандартным инструментом. Проще использовать настройки по умолчанию Apple, поскольку обновления операционной системы также следят за обновлениями Vim. Примечательно, что macOS Vim — это версия без графического интерфейса, но есть популярная сторонняя версия Vim GUI под названием MacVim, которую мы и рекомендуем.

MacVim активно поддерживается, и у него знакомый и удобный дизайн, соответствующий стилю и эргономике Macintosh.

На странице MacVim на GitHub (<https://github.com/macvim-dev/macvim>) находится информация README.md, показанная на рис. Г.1.

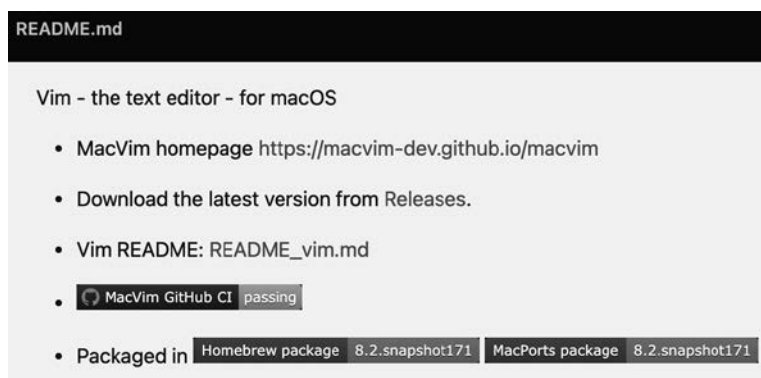


Рис. Г.1. Домашняя страница MacVim, README.md

Для загрузки последней версии MacVim нажмите на Releases в строке Download the latest version from Releases (Загрузить последнюю версию). В нижней части этой страницы посмотрите Assets (рис. Г.2). Мы рекомендуем загрузить MacVim.dmg и выполнить стандартную установку macOS.

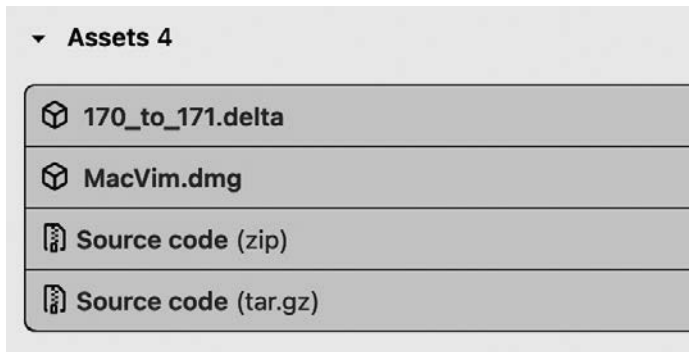


Рис. Г.2. Средства MacVim (загрузки)



Один из нас использует псевдоним `zsh` на своем MacBook Pro, добавляя следующую строку в свой профиль `.zshrc`:

```
alias vi="/Applications/MacVim.app/Contents/bin/mvim"
```

Установка Vim с помощью Homebrew

Пользователи Macintosh, которые хотят работать в операционной системе, более похожей на GNU, вероятно, знакомы с Homebrew (<https://brew.sh>) — менеджером приложений, предоставляющим пакеты GNU для компьютеров Macintosh.

Чтобы установить Vim с помощью Homebrew, выполните команду:

```
brew install vim
```

Для получения дополнительной информации о параметрах Homebrew Vim посетите Homebrew Formulae (<https://formulae.brew.sh/formula/vim>).

Другие операционные системы

В файле справки Vim `vi_diff.txt` перечислено больше операционных систем, на которых поддерживается Vim, например:

- IBM OS/390;
- OpenVMS;
- QNX.

Раньше поддерживались некоторые устаревшие системы, но, скорее всего, они уже не представляют никакого интереса.

Об авторах

Арнольд Роббинс — профессиональный программист и технический писатель родом из Атланты. Женат. Арнольд — счастливый отец четырех замечательных детей и любитель Талмуда (Вавилонского и Иерусалимского). С конца 1997 года он и его семья проживают в Израиле.

Арнольд работал с системами Unix с 1980 года, когда его познакомили с PDP-11 на шестой версии Unix. Он имеет опыт работы с различными коммерческими Unix-системами от Sun, IBM, HP и DEC. Он также использует GNU/Linux с 1996 года.

С 1987-го Арнольд активно использует `awk` и принимал участие в работе над `gawk`, версией `awk` проекта GNU. Будучи членом группы по голосованию POSIX 1003.2, он помог сформировать стандарт POSIX для `awk`. На протяжении долгого времени осуществляет поддержку `gawk` и его документации.

Также работал системным администратором и преподавал курсы непрерывного образования по Unix и построению сетей. Однако у него были и неудачные попытки работы с небольшими компаниями по разработке ПО, о которых он предпочитает не вспоминать.

Некоторое время Арнольд работал в ведущей израильской компании, где занимался написанием высококлассного ПО для управления и контроля. Затем он устроился инженером-программистом в Intel, а позднее в McAfee. В настоящее время занят в менее крупной компании, которая занимается мониторингом сетевой безопасности для промышленности и в сфере управления зданиями. Его личный веб-сайт — <http://www.skeeve.com>.

Издательство O'Reilly не дает Арнольду отдохнуть: он является автором или со-автором бестселлеров «UNIX. Справочник», *Effective awk Programming* (четвертое издание), *sed & awk* (второе издание) с Дейлом Догерти, *Classic Shell Scripting* с Нельсоном Х. Ф. Бибом и некоторых карманных справочников.

Элберт Ханна начал свою карьеру как профессиональный музыкант, но после судьбоносной аварии на велосипеде, в результате которой он повредил руку и потерял возможность играть на виолончели, решил сменить направление и выбрал информационные технологии. С этой наукой он познакомился во время реабили-

тации. Хотя музыка остается его любимым хобби, профессиональная деятельность Элберта сосредоточена в области компьютерных технологий.

Он получает особенное удовольствие, осознавая, что пишет эту книгу о Vim, используя Vim, так как в той или иной степени редактор `vi` внес свой вклад в выбор его профессионального пути.

Элберт познакомился с Unix во время работы в телекоммуникационной отрасли, где использовали дистанционный ввод заданий (RJE), связанный с мейнфреймом IBM. Он обнаружил, что многие процессы протекали проще, когда они перенаправлялись на компьютер компании AT&T, который работал на системе V Unix с использованием множества «специализированных» команд для преобразований и отчетов, а затем вновь возвращались обратно на мейнфрейм.

В работе с Unix Элберту требовалось глубокое понимание `ed`, что стало отправной точкой для его долгого пути изучения, обожания и евангелизации `vi` и в конечном итоге Vim. Седьмое и восьмое издания книги дают возможность Элберту выразить свою признательность и уважение к влиянию Vim на вселенную программирования.

Элберт специализировался на интеграции разрозненных систем. Многие пользователи используют его приложения, не осознавая, что внутри объединено несколько отдельных приложений. Если копнуть достаточно глубоко, можно найти его фотографию на обложке *CEO Magazine* (примерно в середине 90-х годов), где он был отмечен за свою работу по интеграции телекоммуникационных услуг и распределительных приложений.

Он разработал внешний веб-инструмент, предоставляющий быстрые, простые и эффективные способы поиска информации от стороннего продукта. Данный инструмент быстро зарекомендовал себя как популярный метод устранения неполадок для групп поддержки и сотрудников отдела разработки. Элберт также провел презентацию в Лас-Вегасе в 2018-м, продемонстрировав веб-инструмент.

Элберт внес свой вклад в написание более одной сотни технических публикаций (под псевдонимом) и был упомянут в двухчастной статье *The Great FOSS Debates: Kernel Truths* и *FOSS Debates, Part 2: Standard Deviations* вместе с Линусом Торвальдсом.

Иллюстрация на обложке

Животное на обложке восьмого издания — долгопят, ночное млекопитающее, родственник лемура. Его родовое название *Tarsius* произошло от очень длинной таранной кости — предплюсны (tarsus). Хотя когда-то эти животные были широко распространены, на сегодняшний день только десять видов и четыре подвида обитают исключительно на островах Филиппин, Малайзии, Брунея и Индонезии. Долгопяты живут в лесах, умеют перепрыгивать с ветки на ветку с невероятной проворностью и скоростью.

Это небольшое животное: длина его тела составляет всего 15 см, а хвост с кисточкой на конце — около 25 см. У долгопята круглая морда и огромные глаза, тело покрыто шелковистой коричневой или серой шерсткой. У этих животных самые большие глаза по отношению к размеру тела среди всех млекопитающих. Каждое глазное яблоко — около 16 мм в диаметре, такого же размера и его мозг. Долгопят может поворачивать шею на 180 градусов в обоих направлениях, как сова. Его лапы длинные и тонкие, как и его пальцы, которые оканчиваются круглыми мясистыми подушечками. Долгопяты активны только ночью, а днем прячутся в зарослях лиан или на верхушках высоких деревьев. Они хищники, предпочитают насекомых, рептилий, птиц и даже летучих мышей. Хотя эти животные очень любопытны, они, как правило, одиночки.

Большинство долгопятов не выживает в неволе из-за специфических требований к среде обитания и питанию, что делает программы по их разведению практически невозможными. На сегодняшний день популяция долгопятов повсеместно сокращается из-за развития сельского хозяйства, охоты и вырубki лесов, приведших к тому, что большинство представителей этого вида занесены в Красную книгу МСОП как уязвимый вид. Например, долгопяты с острова Сиау находятся под угрозой исчезновения.

Стоит отметить, что многие животные, изображенные на обложках издательства O'Reilly, находятся на грани вымирания.

Цветная иллюстрация выполнена Карен Монтгомери, которая взяла за основу черно-белую гравюру из книги Ричарда Лидеккера *Royal Natural History* («Королевская история естествознания»).

Уважаемый читатель!

Вы прочитали интересную книгу о редакторах vi и Vim. Для закрепления информации самое время применить полученные знания на практике. Это можно сделать вместе с компанией КРОК, специалисты которой приняли решение улучшить качество переводной ИТ-литературы в русскоязычном сообществе и выполнили научное редактирование переведенного текста книги. Если вы представитель бизнеса и потенциальный заказчик ИТ-решений — профессионалы из КРОК смогут помочь вам внедрить решения, о которых вы прочитали в книге. Если вы студент — приходите на практику в КРОК и закрепляйте знания опытом. Если вы опытный специалист — присылайте резюме и добро пожаловать в дружную команду профессионалов.

Тимур Напреев, научный редактор

КРОК

СОЗДАЕМ НАСТОЯЩЕЕ,
ИНТЕГРИРУЕМ БУДУЩЕЕ



croc.ru

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.

